

Advanced Textbooks in Control and Signal Processing

Hai Lin
Panos J. Antsaklis

Hybrid Dynamical Systems

Fundamentals and Methods



Springer

Advanced Textbooks in Control and Signal Processing

Series Editors

Michael J. Grimble, Electronic and Electrical Engineering Department, University of Strathclyde, Glasgow, UK

Lorenzo Marconi , Department of Electrical, Electronic, and Information Engineering, Università di Bologna, Bologna, Bologna, Italy

More information about this series at <http://www.springer.com/series/4045>

Hai Lin · Panos J. Antsaklis

Hybrid Dynamical Systems

Fundamentals and Methods



Springer

Hai Lin
Department of Electrical Engineering
University of Notre Dame
Notre Dame, IN, USA

Panos J. Antsaklis
Department of Electrical Engineering
University of Notre Dame
Notre Dame, IN, USA

ISSN 1439-2232 ISSN 2510-3814 (electronic)
Advanced Textbooks in Control and Signal Processing
ISBN 978-3-030-78729-5 ISBN 978-3-030-78731-8 (eBook)
<https://doi.org/10.1007/978-3-030-78731-8>

MATLAB is a registered trademark of The MathWorks, Inc. See <https://www.mathworks.com/trademarks> for a list of additional trademarks.

© Springer Nature Switzerland AG 2022

This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

To our Families

To

Jing and our daughter Yuxuan

and to my parents Houji Lin and Furong Li

– Hai Lin

To

Melinda and our daughter Leah

and to our grandchildren Alex and Natalia

– Panos J. Antsaklis

And to our Students

Series Editor's Foreword

The *Advanced Textbooks in Control and Signal Processing* series provides a vehicle for the presentation of both fundamental concepts and innovative topics in various areas of advanced control and signal processing. The main theme of a text can focus on either topic but there is also a need for texts that cover both areas. In fact, the theoretical basis and tools needed for these topics are often similar for many of the systems design problems now encountered. Some applications require equal weight to be given to the control system design and the signal processing aspects. There is, for example, an important role for image processing in autonomous vehicle control.

It is hoped that prospective authors will welcome the opportunity to publish a more rounded and structured presentation of some of the emerging control and signal processing technologies in this textbook series. However, there is also a place in the series for contemporary presentations of foundational material on the topics.

The authors are well-known and respected for their work in hybrid systems and the research group at Notre Dame helped to establish the basis of the subject well before it became a popular topic of research. The subject has reached a level of maturity where its potential in applications is now being explored and made use of in industry. This is certainly a topic that is valuable for courses since it provides a concrete example of the need for a total system view or “Systems Engineering”. The topic is multidisciplinary and the continuous feedback control aspects are treated together with the computer science event-driven discrete-variable dynamics.

There is often a feeling that engineers with good computer science skills tune control systems without a proper understanding of dynamical systems and that the reverse—that good control engineers may not fully appreciate the need to integrate the logic-based supervisory and switching system with the feedback design—is also true. The depth of understanding provided by hybrid systems studies should help both disciplines and encourage a team approach. It is therefore a worthy topic for inclusion in the series.

The book has an introductory Chap. 1 including an instructor's guide, which is an early indication that the authors have focused on producing a student-friendly text. The Notes and Further Reading sections and the Exercises at the end of chapters are further evidence that they have worked hard to produce a useful course textbook.

Chapter 2, on modeling hybrid systems, introduces many new ideas for traditional researchers in control. This is because of the discrete-event nature of the problems considered. Switched systems, now a very active research area, and in particular piecewise affine systems are introduced and the range of applications described. The numerous examples are very helpful, perhaps essential, in explaining topics that may be unfamiliar to those with more classical control backgrounds.

The following chapter, on formal verification methods, illustrates the novel and multidisciplinary ideas needed for this type of system, and of course, the verification of models and software is now a critical stage in the design of systems. Chapter 4 returns to the more familiar topics of stability and stabilization, and the Lyapunov methods. This is also presented in a very pedagogical style which is useful for switching systems that require ideas, such as dwell time (lower bound on two consecutive switching times), that are not so well-known.

Chapter 5 is on optimal control, an approach to the design of hybrid systems that seems to be one of the more natural. The basic concepts of optimal control are introduced before extending Pontryagin's minimum principle to hybrid control systems. Fortunately, an intuitive understanding of the solution for continuous-time systems helps in the understanding of the extended problem involving hybrid systems. In an effort to focus the solution on a more specific problem with a simpler solution, a class of switched systems is considered later in the chapter. The model predictive control solution for hybrid systems is then introduced. The useful mixed-logic dynamical (MLD) system models are defined and mixed-integer programming is described. The optimal control problem for a MLD system is then transferred into a mixed-integer programming problem for solution.

Chapter 6 addresses formal synthesis methods in hybrid systems. Discrete and differential games are introduced as a means of allowing for uncertainties. Several approaches are described for the formal synthesis of hybrid control systems. There are also extensive appendices, providing background information on the various mathematical problems encountered, which should be useful to students and engineers.

Hybrid systems theory was initially an important area for academic research studies but there was not a focus on applications. The interest in the subject from industry indicates this is no longer the case since it deals with some intractable problems that need a formal solution procedure. *Hybrid Dynamical Systems* is a welcome addition to the *Advanced Textbooks in Control and Signal Processing* series.

Mike J. Grimble
Department of Electronic and Electrical
Engineering
University of Strathclyde
Glasgow, Scotland, UK

Preface

Hybrid dynamical systems refer to a class of complex dynamical systems consisting of both event-driven discrete and time-driven continuous dynamics that interact with each other and jointly govern the evolution of the system's behavior over time. A typical example is a robot—the state of which is described by continuous variables, such as its position, orientation, and velocity, controlled by a digital processor that reasons in terms of logic symbols and rules. Hybrid systems arise naturally when digital computers or digital communication systems interact with continuous physical processes and can be commonly found in manufacturing systems, power grids, traffic management systems, and sensor–actuator networks to mention but a few.

In the past, design methods have considered the design and verification of the discrete cyber behavior separately from the continuous physical dynamics. However, the separation between discrete and continuous dynamics rarely holds in practice, since many times there are tight interactions between them, especially in high-performance systems, and ignoring their coupling may cause undesirable behaviors and even catastrophic failures. The tight coupling between discrete and continuous dynamics in hybrid dynamical systems makes the traditional methods used to study dynamical systems inadequate and calls for new methodologies. The theoretical challenges of hybrid systems, together with their practical importance, have attracted significant attention from researchers in control theory, mathematics, and computer science since the early 1990s. After almost three decades of intense development, hybrid dynamical system theory has grown into a vibrant and multidisciplinary field of study. Nowadays, hybrid dynamical system theory plays a fundamental role in emerging fields, such as Cyber-Physical Systems, Autonomy, Future Transportation Systems, and Smart Cities to mention but a few. Clearly, understanding the fundamentals of hybrid systems has become necessary for our next generation engineering students, especially for those majoring in control, automation, industrial engineering, manufacturing, computer engineering, and computer science.

It should be noted that the multidisciplinary nature of hybrid dynamical systems makes their study quite challenging for those who would like to obtain a comprehensive view of the entire field. In addition, many important results are scattered in the literature in technical papers or research monographs and are presented using quite different settings and terminologies. The goal of this book is to provide an accessible

and comprehensive introduction to hybrid dynamical systems with an emphasis on results that are central in providing a good understanding of their role and importance. We seek to balance the emphasis on methods rooted in different disciplines and present to the reader a complete picture of the whole field of hybrid dynamical systems.

The book is self-contained and can be used primarily as a textbook in graduate-level courses and as a reference for researchers in the field. Examples and simulations from this book together with some of the materials presented may also be used in advanced undergraduate courses as supplementary materials.

Notre Dame, USA

Hai Lin
Panos J. Antsaklis

Acknowledgements

Our view of hybrid dynamical system theory has been shaped primarily by our research in this area over several decades, by productive collaborations and discussions with our colleagues, and by challenges posed by our students. The materials in this book have been used by us in courses on hybrid, cyber-physical, and intelligent control systems offered at the University of Notre Dame in the past several years. We appreciate our students' feedback on the course material and their help in taking notes to put together preliminary versions of this manuscript. Special thanks to Bo Wu, Jin Dai, Shilu Dai (for his help on the MATLAB simulations of Examples in Chapter 4), Vince Kurtz, (for his help on the preparation of the section on Signal Temporal Logic), Shirantha Welikala (for his help in the proofreading of our complete draft), Junhui Liu, Xunlin Zhu, and Ali Karimoddini. We are thankful to NSF for providing funding to our research projects on hybrid dynamical systems in the past years. We would also like to thank our Springer Editor Oliver Jackson and his assistants, for their encouragement and patience. Writing this book has truly been a multi-year project—it has taken much longer than we initially thought. Finally, we thank our families for their understanding and support.

Contents

1	Introduction	1
1.1	What is a Hybrid Dynamical System?	1
1.2	Why Do We Need a New Theory?	2
1.3	Research Approaches	4
1.4	Book Structure and Contents	5
1.5	A Brief Instructor's Guide	6
	References	7
2	Modeling of Hybrid Systems	11
2.1	Finite Automata	12
2.1.1	Finite Automaton Model	14
2.1.2	Properties of Finite Automata	16
2.1.3	Regular Languages	19
2.2	Hybrid Automata	21
2.2.1	Hybrid Automata Models	22
2.2.2	Hybrid Automata Composition	27
2.2.3	Hybrid Execution	29
2.2.4	Determinism and Non-blocking Properties	34
2.3	Switched and Piecewise Affine Systems	38
2.3.1	Switched Systems	38
2.3.2	Piecewise Affine Systems	45
2.3.3	Existence and Uniqueness of Solutions	48
2.4	Summary	53
2.5	Notes and Further Reading	54
2.6	Exercises	56
	References	62
3	Formal Verification	65
3.1	Labeled Transition Systems	66
3.1.1	Transition Systems	66
3.1.2	Labeled Transition System	69

3.2	Linear Temporal Logic	71
3.2.1	Linear Temporal Logic	71
3.2.2	LTL Model Checking	74
3.3	Computation Tree Logic	79
3.3.1	Computation Tree Logic	80
3.3.2	CTL Model Checking	82
3.3.3	Comparison Between LTL and CTL	85
3.4	Bisimulation	87
3.4.1	Simulation Relation	87
3.4.2	Bisimulation Quotient	89
3.4.3	Computing Bisimulations	91
3.5	Timed Automata	96
3.5.1	Timed Automata	96
3.5.2	Timed Language	99
3.5.3	Timed Computation Tree Logic	101
3.5.4	Timed Automata Model Checking	104
3.5.5	Extensions of Timed Automata	114
3.5.6	Zone Automata and Symbolic Reachability Analysis	117
3.6	Linear Hybrid Automata	124
3.6.1	Linear Hybrid Automata	124
3.6.2	Runs of Linear Hybrid Automata	125
3.6.3	Symbolic Reachability Analysis of Linear Hybrid Automata	129
3.6.4	Symbolic Model Checking	135
3.7	Verification of More General Hybrid Systems	143
3.7.1	Linear Dynamics	143
3.7.2	Barrier Certificate	145
3.8	Summary	147
3.9	Notes and Further Reading	149
3.10	Exercises	150
	References	154
4	Stability and Stabilization	157
4.1	Lyapunov Stability Theory	158
4.1.1	Lyapunov Stability	158
4.1.2	Stability of Linear Time-Invariant Systems	165
4.1.3	Lyapunov Stability for Time-Varying Systems	170
4.1.4	Converse Lyapunov Theorem	173
4.2	Stability of Hybrid Automata	176
4.3	Arbitrary Switching	179
4.3.1	Common Lyapunov Functions	180
4.3.2	Common Quadratic Lyapunov Functions	181
4.3.3	Commutative Systems	182
4.3.4	Triangular Systems	185

4.3.5	A Lie Algebraic Condition	186
4.3.6	Switched Quadratic Lyapunov Functions	187
4.4	Constrained Switching	189
4.4.1	Stability with Dwell Time	190
4.4.2	Stability with Average Dwell Time	191
4.4.3	Discrete-Time Case	192
4.5	Multiple Lyapunov Functions	195
4.5.1	Multiple Lyapunov Functions Theorem	196
4.5.2	Piecewise Quadratic Lyapunov Functions	203
4.6	Design of Stabilizing Switching Sequences	207
4.6.1	Quadratic Stabilization	207
4.6.2	Piecewise Quadratic Stabilization	212
4.7	Summary	216
4.8	Notes and Further Reading	217
4.9	Exercises	218
	References	224
5	Optimal Control	227
5.1	Optimal Control Problem	228
5.2	The Minimum Principle	229
5.2.1	Calculus of Variations	229
5.2.2	Necessary Conditions for Optimal Control Solutions	234
5.2.3	Pontryagin's Minimum Principle	245
5.3	Hybrid Optimal Control	250
5.3.1	Hybrid Optimal Control Problem	250
5.3.2	Basic Hybrid Minimum Principle	258
5.3.3	Extensions of the Hybrid Minimum Principle	260
5.3.4	Further Extensions of Hybrid Minimum Principles	269
5.4	Optimal Control of Switched Systems	278
5.4.1	Optimal Control Problem for Switched Systems	278
5.4.2	Two-Stage Optimization	279
5.4.3	Embedding Optimization	289
5.5	Model Predictive Control	297
5.5.1	Mixed-Logic Dynamical Systems	298
5.5.2	Mixed-Integer Programming	303
5.6	Summary	305
5.7	Notes and Further Reading	307
5.8	Exercises	308
	References	312
6	Formal Synthesis	315
6.1	Discrete Games	316
6.1.1	Game Structure	316
6.1.2	Safety Games	318
6.1.3	Reachability Games	321
6.1.4	Büchi Games	322

6.2	Differential Games	324
6.2.1	Open-Loop Strategy	325
6.2.2	Closed-Loop Strategy for One-Player Game	328
6.2.3	State-Feedback Strategy for Two-Player Games	336
6.2.4	Two-Player Zero-Sum Games	339
6.2.5	Pursuit–Evasion Games	340
6.3	Unifying Discrete Games and Differential Games	342
6.3.1	Solving Discrete Games by Value Iteration	342
6.3.2	Differential Safety Games	345
6.4	Timed Games	349
6.4.1	Timed Game Automata	349
6.4.2	Abstraction of a Timed Game	352
6.4.3	Symbolic Approach	355
6.4.4	Timed Automata Optimal Control	359
6.5	Hybrid Games	361
6.5.1	Hybrid Game Automata	362
6.5.2	Solve the Reach-While-Avoid Operator as a Pursuit– Evasion Game	363
6.6	Control-Oriented Abstraction	365
6.6.1	Temporal Logic over Reals	366
6.6.2	Linear Control Systems	367
6.6.3	Multi-affine Control Systems	376
6.6.4	Approximate Bisimulation and Nonlinear Control Systems	380
6.7	Optimization-Based Approaches	383
6.7.1	Signal Temporal Logic	383
6.7.2	Trajectory Synthesis	386
6.7.3	Robust Semantics	388
6.8	Summary	392
6.9	Notes and Further Reading	393
6.10	Exercises	395
	References	398
Appendix A: Continuous and Sampled-Data Systems		401
Appendix B: Languages and Automata		423
Index		437

About the Authors

Dr. Hai Lin obtained his B.S. degree from the University of Science and Technology Beijing and his M.S. degree from the Chinese Academy of Sciences in 1997 and 2000, respectively. In 2005, he received his Ph.D. degree from the University of Notre Dame. Dr. Lin is currently a Full Professor at the Department of Electrical Engineering, University of Notre Dame. Before returning to his alma mater, he worked as an Assistant Professor at the National University of Singapore from 2006 to 2011. His teaching and research interest lies in the multidisciplinary study of the problems at the intersections of control, formal methods, and machine learning. His current research thrusts include cyber-physical systems, multi-robot cooperative tasking, human-machine collaboration, robotic manipulation and legged locomotion. Professor Lin has been served on several committees and editorial boards, including IEEE Transactions on Automatic Control. He served as the Chair for the IEEE CSS Technical Committee on Discrete Event Systems, the Program Chair for IEEE ICCA 2011 and IEEE CIS 2011 and the Chair for IEEE Systems, Man, and Cybernetics Singapore Chapter for 2009 and 2010. He is a senior member of IEEE and a recipient of the 2013 NSF CAREER award.

Panos J. Antsaklis is the H. Clifford and Evelyn A. Brosey Professor of Electrical Engineering and Concurrent Professor of Computer Science and Engineering and of Applied and Computational Mathematics and Statistics at the University of Notre Dame. He is a graduate of the National Technical University of Athens, Greece, and holds M.S. and Ph.D. degrees from Brown University. His research addresses problems of control and automation and examines ways to design control systems that will exhibit a high degree of autonomy. His current research focuses on Cyber-Physical Systems and the interdisciplinary research area of control, computing, and communication networks and on hybrid and discrete event dynamical systems.

In addition to publications in journals, conference proceedings, book chapters, and encyclopedias, he has co-authored two graduate textbooks on Linear Systems and three research monographs—one on model-based control of networked systems and two on supervisory control of discrete event systems—and has co-edited six books

on Intelligent Autonomous Control, Hybrid Systems, and Networked Embedded Control Systems. He is the recipient of several teaching awards at the University of Notre Dame.

He is IEEE, IFAC, and AAAS Fellow and a recipient of the 2006 Engineering Alumni Medal of Brown University and holds an Honorary Doctorate from the University of Lorraine in France. He is the recipient of the 2013 Faculty Award and the 2020 Research Achievement Award of the University of Notre Dame.

He has been a plenary and keynote speaker in a number of conferences, and he is the Founding President of the Mediterranean Control Association and co-Editor-in-Chief of the journal *Foundations and Trends in Systems and Control*. He served as the Chair of the Scientific Advisory Board of the Max Planck Institute in Magdeburg, Germany, for 6 years. He is a recipient of the IEEE Millennium Medal and he is a Distinguished Member and a Distinguished Lecturer of the IEEE Control Systems Society, where he also served as its President in 1997. He served as the Editor-in-Chief of the *IEEE Transactions on Automatic Control*, one of the leading journals in the field of Control Systems for 8 years, 2010–2017.

Chapter 1

Introduction



1.1 What is a Hybrid Dynamical System?

Hybrid dynamical systems contain heterogeneous dynamics that interact with each other and jointly determine the systems' behaviors over time. By heterogeneous dynamics, we mean two different kinds of dynamics: Time-driven continuous-variable dynamics, the evolutions of which are typically governed by physical laws and are described by differential or difference equations; and event-driven discrete-variable dynamics, the evolution of which depends on "if-then-else" type of rules, usually described by automata or Petri nets. These two kinds of dynamics interact with each other and generate complex dynamical behaviors, such as switching once the value of a continuous variable passes through a threshold and state jumping upon a certain discrete event occurring, to mention but a few.

Such hybrid dynamical systems are ubiquitous and can be found in manufacturing systems, chemical plants, traffic management, power grid, and communication systems. As a simple example of hybrid dynamical systems, consider a typical room temperature control system in winter. Assume that the set point of the thermostat is 70 °F. The furnace will turn on if the room temperature is below the set point, and turn off otherwise. The room temperature control system is a typical hybrid system as the furnace, along with the heat flow characteristics of the room, forms the continuous-variable dynamics, whereas the on–off thermostat can be modeled as a discrete event system with two states "ON" and "OFF". In addition, the transition between these two discrete states is triggered by the temperature in the room, while the evolution of the temperature depends on whether the furnace is on or off, i.e., the discrete state of the thermostat. Hence, the temperature control system contains interacting discrete and continuous dynamics, and can be modeled and studied as a hybrid system. This temperature control hybrid system is further discussed in the next chapter on the modeling of hybrid systems.

Hybrid dynamical systems arise naturally in many advanced control systems where multiple simple control laws are combined to achieve more challenging control performance or to handle broader operating scenarios. Such examples include

the switching control to avoid collisions [1, 2], cooperative control of multi-robot systems [3, 4], locomotion of legged robots [5, 6], and integrated task and motion planning for autonomous robots [7, 8].

As another example, in a manufacturing process, parts may be processed in a particular machine, but only the arrival of a part triggers the process; that is, the manufacturing process is composed of the event-driven dynamics of the parts moving among different machines and the time-driven dynamics of the processes within particular machines [9]. Hybrid systems also have a central role in networked embedded control systems that interact with the physical world, and as such play a key role in understanding the evolution of systems that contain an information and networking core and interact tightly with the physical world and human operators; such systems are also referred to as cyber-physical systems (CPS) [10, 11]. Hybrid systems also arise from the hierarchical organization of complex systems, and from the interaction of discrete planning algorithms and continuous control algorithms in autonomous, intelligent systems [12, 13]. Hybrid systems are found not only in man-made engineered systems but also in natural systems as well. For instance, gene regulatory networks inside our cells are of a hybrid nature, as the gene translation and transcription processes happen continuously, while the turn-on or turn-off of particular genes is discrete [14, 15]. Studies in hybrid systems could provide a unified modeling framework for CPS, and systematic methods for performance analysis, verification, and design. Hybrid system analysis and controller synthesis techniques may provide efficient approaches for the design and verification of complex engineering systems. Hybrid system analysis may also shed additional light on biological processes leading to better ways of influencing them.

1.2 Why Do We Need a New Theory?

The design and verification of the discrete components have been traditionally studied separately from the continuous physical dynamics—the former typically using automata or Petri net models and the latter using differential or difference equations. In practice, however, there is often a tight coupling between the cyber (discrete) and physical (continuous) components. To fully understand the system’s behavior and meet high-performance specifications, one needs to model and study all dynamics together with their interactions. Only then problems such as optimization of the whole manufacturing process may be addressed in a meaningful and comprehensive manner. Ignoring these couplings may cause errors or infeasibility in implementation, or even catastrophic failures. For example, it is well-known, although counter-intuitive, that switching among a collection of exponentially stable linear time-invariant systems may cause the systems to exhibit divergent behaviors. On the contrary, one may even obtain global stability by switching among a family of unstable systems—see Chap. 4 on Stability.

As a dramatic example, Ariane 5 launcher went into self-destruction mode 37 seconds after liftoff on June 4, 1996. Later investigations found that it was caused by

a software error. However, the code was exactly the same as the code for Ariane 4, which had worked perfectly before. What had changed were the physical dynamics of the launcher as Ariane 5 had been sized up significantly compared to its predecessor. Within the new physical environment, the trusted code quickly led into a catastrophe [16, 17]. Therefore, it becomes necessary to develop a unified framework for studying such kinds of heterogeneous dynamical systems instead of building them in an ad hoc manner. There is no doubt that the necessity and importance of developing a unified framework to deal with hybrid dynamical systems stem primarily from two important aspects, namely their wide applicability in practice and the new theoretical accomplishments that make the development of such a unified framework possible.

So far, we have seen a few examples of hybrid dynamical systems existing in various man-made and natural systems. As another example, hybrid models may be used to a significant advantage in the automotive engine control [18, 19], where there is a need for control algorithms with guaranteed properties, implemented via embedded controllers, that can substantially reduce emissions and gas consumption, while maintaining high performance. Note that an accurate model of a four-stroke gasoline engine has a natural hybrid representation, because from the engine control point of view, the power train and air/fuel dynamics are continuous-time processes, while the pistons have four modes of operation that correspond to the stroke they are in and so their behavior is represented as a discrete event process, described, say, via a finite state machine model. These processes interact tightly, as the timing of the transitions between two phases of the pistons is determined by the continuous motion of the power train, which, in turn, depends on the torque produced by each piston. Note that in the past, the practice has been to convert the discrete part of the engine behavior into a more familiar and easier-to-handle continuous model, where only the average values of the appropriate physical quantities are modeled. Using hybrid models one may represent time and event-based behaviors and their interactions more accurately so as to meet ever more challenging design requirements in the design of control systems for problems such as cutoff control [20] and idle speed control of the engine [21].

Besides their substantial practical importance, hybrid systems also represent a fascinating and highly challenging area of study that encompasses a variety of theoretical research questions. For example, the introduction of switching and state jumps in continuous systems significantly extends their dynamical behaviors and they can be modeled by hybrid system models. Hybrid system models are of interest in themselves, and have been successfully used to model a large variety of complex systems, such as gene regulatory networks [14, 22], communication networks [23, 24], and robotic systems [25, 26]. However, there is a price associated with the increased modeling power, namely the difficulty in analyzing properties of the evolution or solution of a hybrid system model, such as the existence and uniqueness of solutions, the continuity of trajectories with respect to initial conditions, and the state explosion issues in formal methods-oriented approaches. These difficulties have motivated significant and intense research activities in the analysis and synthesis of hybrid systems. On the other hand, the introduction of switching logic into controllers may help to achieve performance that exceeds the performance of any fixed classical linear or

nonlinear smooth controller; for example, some nonlinear systems that cannot be stabilized by any smooth feedback control law can be asymptotically stabilized by a hybrid controller [27]. Moreover, to meet challenging high-performance design requirements that reflect multiple objectives such as speed of response, accuracy, optimality, robustness, and disturbance attenuation, a multi-modal (hybrid) control architecture may be the best choice. When the requirements are represented by time- and event-based behaviors or when the system to be controlled has tight interactions of continuous and discrete dynamics, one needs to employ hybrid control methods [28, 29].

1.3 Research Approaches

The history of hybrid system research can be traced back at least to the study of engineering systems that contained relays and/or hysteresis [30] in the 1960s. Hybrid systems began to seriously attract the attention of control and computer science researchers in the early 1990s, mainly because of the widespread development and implementation of digital micro-controllers and embedded devices. The last quarter-century has seen considerable research activities in modeling, analysis, and synthesis of hybrid systems.

The methods for analysis and design of hybrid systems span a wide spectrum, ranging from methods rooted in the discrete (cyber-) domain on one end to traditional approaches for continuous (physical) systems on the other. Rooted at opposite ends, both computer scientists and control theorists have made significant contributions to the field of hybrid systems by extending traditional methods from their discrete or continuous domains to deal with hybrid systems; see, e.g., [29, 31–35].

Computer scientists tend to look at hybrid systems primarily as discrete (computer) programs interacting with the physical environment. They extend their computational models, such as finite state machines, automata, and Petri nets, from discrete systems to hybrid systems by embedding the continuous-variable dynamics into these discrete models. Typically, these approaches are able to deal with complex discrete dynamics, and emphasize analysis results (verification) and simulation methodologies. Such approaches typically ask whether certain properties, such as safety, liveness, and fairness that are formulated in temporal logic formulas, hold true or not for a given hybrid system model. This is called the verification of hybrid systems [11, 36–38], and one of the main verification methods is symbolic model checking, which is based on the computation of reachable sets for hybrid systems. Consequently, a good deal of research effort has focused on developing sophisticated techniques drawn from optimal control, game theory, and computational geometry to calculate or approximate the reachable sets for various classes of hybrid systems.

On the other hand, researchers from the area of dynamical systems and control theory have considered hybrid systems as collections of differential/difference equations with discontinuous or multi-valued vector fields. Representative modeling frameworks in this category include piecewise affine/linear systems [39, 40] and

switched systems [33, 41]. These approaches extend the models and methodologies of traditional continuous-variable systems, such as ordinary differential/difference equations, by including discrete variables so as to describe the jumping or switching phenomena. Typically, these approaches are able to deal with complex continuous-variable dynamics and focus mainly on stability, controllability, robustness, and synthesis issues.

Work on integrating methods from these two domains has started to emerge in the literature. The difficulty in integrating these two approaches comes from the fact that formal methods pursued in computer science traditionally lie in the realm of discrete mathematics, while control theory approaches lie mainly in the realm of continuous mathematics. This book seeks to balance the emphasis on methods from both, computer science and control theory, and present to the reader a relatively complete picture of the entire field of hybrid dynamical systems. To this end, the topics introduced and discussed in this book include both approaches mentioned above.

1.4 Book Structure and Contents

Our goal is to provide a detailed introduction to the topic with an emphasis on results, which in our opinion are most important in providing a comprehensive picture of the theory of hybrid dynamical systems. Additional approaches are briefly discussed in Notes and Further Reading sections at the end of each chapter and references are provided for further reading. Each chapter also contains a chapter Summary section and an Exercises section. The book contents are organized as follows.

Chapter 2 introduces several mathematical modeling frameworks for hybrid systems. We start with a brief introduction to finite automata, and discuss in detail the extension to hybrid automata. The dynamical properties, such as well-posedness, of solutions for hybrid automata are studied with examples. Then, we turn to more control-oriented models, such as switched systems and piecewise affine systems. The relationships among these models are discussed as well. Additional models, such as mixed-logic dynamic systems, timed automata, and linear hybrid automata, which are used in later chapters, are briefly introduced in Notes and Further Reading section at the end of the chapter.

Chapter 3 investigates the verification of hybrid systems with a particular focus on model checking approaches. An overview of model checking and temporal logic for finite discrete transition systems is given first with the emphasis on basic ideas and illustrative examples. The major part of this chapter discusses the extensions of the traditional model checking approaches to hybrid systems. Two approaches are discussed in detail. One is based on the existence of a finite abstraction and the other is symbolic model checking. In particular, we study timed automata and their abstractions in detail to illustrate the abstraction-based approach, while we use linear hybrid automata to introduce the basic principles behind the symbolic reachability analysis and symbolic model checking. Software tools and recent trends

in the literature are further discussed in Notes and Further Reading section at the end of the chapter.

Chapter 4 discusses results on the stability and stabilization of hybrid (switched) systems. We start with a brief review of the classical Lyapunov stability theory, and motivate the study of stability issues in switched and hybrid systems. First, we talk about the existence of a common Lyapunov function to guarantee the stability under arbitrary switching. Then, we introduce multiple Lyapunov functions as the main tool to study switched/hybrid system stability issues. The chapter concludes with a discussion on the design of switching laws to achieve closed-loop stability.

Chapter 5 presents optimal control compensation approaches for hybrid systems. We start the chapter with a brief review of classical optimal control techniques with a focus on the calculus of variations and minimum principle. Then, we present a generalization of the minimum principle to hybrid systems. The second half of the chapter is devoted to numerical approaches, and several methods, such as two-stage optimization, embedding approaches, and converting a moving horizon hybrid optimal control problem into mixed-integer programming, are discussed in detail with examples.

Chapter 6 introduces control synthesis methods for hybrid systems. We start with a brief review of game theory-based approaches, where both discrete games and differential games are discussed. Then, we focus on how to extend these techniques to the synthesis of hybrid systems. Similar to Chap. 3, both abstraction-based and symbolic approaches are discussed. We use timed games to formulate the formal synthesis problem for real-time systems and to illustrate the abstraction-based approaches. We use hybrid games to introduce the basic ideas behind the symbolic approaches. In addition, methods based on optimization and approximate bisimulation are also discussed.

The appendices provide background material in modeling, continuous linear systems, sampled-data systems, finite automata, and ω -automata theory.

There are several topics that have been studied in the literature which are not treated in this book in any detail, primarily because of length considerations. These topics are briefly discussed and references are provided in Notes and Further Reading sections of appropriate chapters. Such topics include reachability, controllability, and observability for switched and piecewise affine systems, switched system identification and fault diagnosis, stochastic hybrid systems, hybrid Petri nets, and hybrid supervisory control.

1.5 A Brief Instructor's Guide

The book is mainly written for graduate students with some background in control theory, and the best option is, of course, to study the book from cover to cover. Drafts of the book have been used by us multiple times to teach a semester-long (15 weeks) graduate class on hybrid systems at Notre Dame, to first year graduate students with backgrounds in robotics, mathematics, or computer science.

It is also possible to use selected chapters from the book to offer a quarter-based module either emphasizing the control-oriented approaches or the formal methods perspective of hybrid systems. Either way, it is important to discuss at length the role of hybrid automata in Chap. 2 as a general class of models for hybrid systems and also gain familiarity with the system behavior via simulations. In the study of models, it would also be beneficial to study the topics that have been included in Appendix A. Simulation tools may be found on the MathWorks Stateflow website.

If the instructor would like to emphasize control-oriented approaches, Chaps. 3 and 6 could be skipped. Important approaches originating in continuous systems are based on the switched, piecewise affine models in Chap. 2 and the mixed-logic models in Chap. 5. Important topics for such systems are the stability and stabilization considerations in Chap. 4 and the optimal control design approaches in Chap. 5.

On the other hand, if the emphasis is more on computer science approaches, then the Chaps. 4 and 5 on stability and optimal control could be omitted. Based on approaches for discrete systems are the hybrid automata in Chap. 2, verification in Chap. 3, and synthesis methods in Chap. 6.

Several examples are included in every chapter which help to clarify the main concepts and gain insights. Recommendations for further study embedded in the chapters and exercises at the end of each chapter could be used as homework assignments.

Depending on the background and interest of students, instructors may also select chapters/sections from the book for other classes, for example, selected chapters have been used for our classes on CPS (Chaps. 2, 3, and 6) and formal methods class (Chaps. 3 and 6) at Notre Dame by the authors.

Anyone who attempted to write a textbook knows that it is impossible to write a perfect book—the existence of a perfect book is itself questionable! We apologize for any mistakes or any omissions in the selection of topics or references. We will be glad to hear your suggestions or comments on how to further improve the book and we are thanking you in advance.

References

1. Silvia Mastellone, Dušan M Stipanović, Christopher R Graunke, Koji A Intlekofer, and Mark W Spong. Formation control and collision avoidance for multi-agent non-holonomic systems: Theory and experiments. *The International Journal of Robotics Research*, 27(1), 107–126, 2008.
2. Jeremy H Gillula, Gabriel M Hoffmann, Haomiao Huang, Michael P Vitus, and Claire J Tomlin. Applications of hybrid reachability analysis to robotic aerial vehicles. *The International Journal of Robotics Research*, 30(3), 335–354, 2011.
3. Lynne E Parker, Daniela Rus, and Gaurav S Sukhatme. Multiple mobile robot systems. In *Springer Handbook of Robotics*, pages 1335–1384. Springer, 2016.
4. Francesco Bullo. *Lectures on network systems*. Kindle Direct Publishing, 2019.
5. Philip Holmes, Robert J Full, Dan Koditschek, and John Guckenheimer. The dynamics of legged locomotion: Models, analyses, and challenges. *SIAM review*, 48(2):207–304, 2006.

6. Kaveh Akbari Hamed and Jessy W Grizzle. Reduced-order framework for exponential stabilization of periodic orbits on parameterized hybrid zero dynamics manifolds: Application to bipedal locomotion. *Nonlinear Analysis: Hybrid Systems*, 25:227–245, 2017.
7. Leslie Pack Kaelbling and Tomás Lozano-Pérez. Integrated task and motion planning in belief space. *The International Journal of Robotics Research*, 32(9–10), 1194–1227, 2013.
8. Rafael Rodrigues da Silva, Bo Wu, and Hai Lin. Formal design of robot integrated task and motion planning. In *2016 IEEE 55th Conference on Decision and Control (CDC)*, pages 6589–6594. IEEE, 2016.
9. Christos G Cassandras, David L Pepyne, and Yorai Wardi. Optimal control of a class of hybrid systems. *IEEE Transactions on Automatic Control*, 46(3), 398–415, 2001.
10. R. Baheti and H. Gill. *Cyber-physical systems. The Impact of Control Technology*, pages 161–166, 2011.
11. Rajeev Alur. *Principles of cyber-physical systems*. MIT Press, 2015.
12. Panos J Antsaklis. Intelligent control. *Encyclopedia of Electrical and Electronics Engineering*, 10:493–503, 1997.
13. Panos J Antsaklis. The quest for autonomy revisited. ND ISIS Tech. Report, 11(004), 2011.
14. Hidde De Jong. Modeling and simulation of genetic regulatory systems: a literature review. *Journal of computational biology*, 9(1):67–103, 2002.
15. Rajeev Alur, Calin Belta, Franjo Ivančić, Vijay Kumar, Max Mintz, George J Pappas, Harvey Rubin, and Jonathan Schug. Hybrid modeling and simulation of biomolecular networks. In *International workshop on hybrid systems: Computation and control*, pages 19–32. Springer, 2001.
16. Arjan J Van Der Schaft and Johannes Maria Schumacher. *An introduction to hybrid dynamical systems*, volume 251. Springer, London, 2000.
17. C. Baier and J. P. Katoen. *Principles of model checking*. MIT press Cambridge, 2008.
18. Andrea Balluchi, Luca Benvenuti, Maria Domenica Di Benedetto, Claudio Pinello, and Alberto Luigi Sangiovanni-Vincentelli. Automotive engine control and hybrid systems: Challenges and opportunities. *Proceedings of the IEEE*, 88(7), 888–912, 2000.
19. Xiaoqing Jin, Jyotirmoy V Deshmukh, James Kapinski, Koichi Ueda, and Ken Butts. Powertrain control verification benchmark. In *Proceedings of the 17th international conference on Hybrid systems: computation and control*, pages 253–262, 2014.
20. A Balluchi, M Di Benedetto, C Pinello, C Rossi, and A Sangiovanni-Vincentelli. Cut-off in engine control: a hybrid system approach. In *Proceedings of the 36th IEEE Conference on Decision and Control*, volume 5, pages 4720–4725. IEEE, 1997.
21. Elena De Santis, MARIA DOMENICA Di Benedetto, and Giordano Pola. Digital idle speed control of automotive engines: A safety problem for hybrid systems. *Nonlinear Analysis: Theory, Methods & Applications*, 65(9), 1705–1724, 2006.
22. Eric H Davidson. *The regulatory genome: gene regulatory networks in development and evolution*. Elsevier, 2010.
23. J. P. Hespanha. A model for stochastic hybrid systems with application to communication networks. *Nonlinear Analysis: Theory, Methods & Applications*, 62(8), 1353–1383, 2005.
24. Christos G Cassandras and John Lygeros. *Stochastic hybrid systems*. CRC Press, 2018.
25. Jerry Ding, Jeremy H Gillula, Haomiao Huang, Michael P Vitus, Wei Zhang, and Claire J Tomlin. Hybrid systems in robotics. *IEEE Robotics & Automation Magazine*, 18(3), 33–43, 2011.
26. Bruno Siciliano and Oussama Khatib. *Springer handbook of robotics*. Springer, 2016.
27. J. P. Hespanha, D. Liberzon, and A. S. Morse. Logic-based switching control of a nonholonomic system with parametric modeling uncertainty. *Systems & Control Letters*, 38(3), 167–177, 1999.
28. P. J. Antsaklis and A. Nerode. Hybrid control systems: An introductory discussion to the special issue. *IEEE Trans. Automat. Contr.*, 43(4):457–460, 1998.
29. Panos J Antsaklis. A brief introduction to the theory and applications of hybrid systems. In *Proc IEEE, Special Issue on Hybrid Systems: Theory and Applications*, 2000.

30. H. Witsenhausen. A class of hybrid-state continuous-time dynamic systems. *Automatic Control, IEEE Transactions on*, 11(2):161–167, Apr 1966.
31. Panos J Antsaklis and Anil Nerode. Hybrid control systems: An introductory discussion to the special issue. *IEEE Transactions on Automatic Control*, 43(4), 457–460, 1998.
32. Hai Lin and Panos J Antsaklis. Hybrid dynamical systems: An introduction to control and verification. *Foundations and Trends® in Systems and Control*, 1(1), 1–172, 2014.
33. Hai Lin and Panos J Antsaklis. Stability and stabilizability of switched linear systems: a survey of recent results. *IEEE Transactions on Automatic control*, 54(2):308–322, 2009.
34. Panos J Antsaklis and Xenofon D Koutsoukos. Hybrid systems: Review and recent progress. *Software-Enabled Control: Information Technology for Dynamical Systems*, pages 273–298, 2003.
35. Panos Antsaklis, Xenofon Koutsoukos, and Janan Zaytoon. On hybrid control of complex systems: A survey. *European Journal of Automation, APII-JESA*, 32:9–10, 1998.
36. R. Alur, C. Courcoubetis, N. Halbwachs, T. Henzinger, P. H. Ho, X. Nicollin, A. Olivero, J. Sifakis, and S. Yovine. The algorithmic analysis of hybrid systems. *Theoretical Computer Science*, 138:3–34, 1995.
37. T. Henzinger. The theory of hybrid automata. In M.K. Inan and R.P. Kurshan, editors, *Verification of Digital and Hybrid Systems*, volume 170 of NATO ASI Series, pages 265–292. Springer, Berlin Heidelberg, 2000.
38. Rajeev Alur. Formal verification of hybrid systems. In *Proceedings of the ninth ACM international conference on Embedded software*, pages 273–278, 2011.
39. Eduardo Sontag. Nonlinear regulation: The piecewise linear approach. *Automatic Control, IEEE Transactions on*, 26(2):346–358, 1981.
40. M. Johansson. *Piecewise Linear Control Systems: A Computational Approach*, volume 284 of *Lecture Notes in Control and Information Sciences*. Springer-Verlag, 2003.
41. D. Liberzon. *Switching in Systems and Control*. Birkhauser, Boston, 2003.

Chapter 2

Modeling of Hybrid Systems



Hybrid dynamical systems contain both time-driven continuous dynamics and event-driven discrete dynamics that interact with each other, and their interactions determine the behavior of the system over time. To fully understand the tight interactions between discrete and continuous dynamics, we need a unified modeling framework. Various models have been developed for hybrid systems; they can be broadly classified into two groups according to their origins. On the one hand, computer scientists look at hybrid systems primarily as discrete (computer) programs interacting with the continuous environment. They extend their computational models, such as finite state machines, automata, and Petri nets, from discrete systems to hybrid systems by embedding the continuous-variable dynamics into these discrete models. On the other hand, researchers from the areas of dynamical systems and control regard hybrid systems as a collection of differential/difference equations with discontinuous or multi-valued variables. They extend the models and methodologies for traditional continuous-variable systems, such as those described by ordinary differential/difference equations, by including discrete variables to describe the jumping or switching phenomena. Both approaches have their own strengths. For instance, the first group of approaches has traditionally been able to deal with complex discrete dynamics emphasizing analysis results (verification) and simulation methodologies, while the second group typically deals with complex continuous-variable dynamics and mainly concerns itself with stability, robustness, optimality, and synthesis issues.

In this chapter, we will introduce several modeling frameworks for hybrid systems. First, we start with a brief review of discrete event systems and briefly discuss automata theory. Then, hybrid automata are introduced with a focus on well-posedness and the characterization of their trajectories. Hybrid automata were proposed in the computer science literature to model hybrid systems arising from computer programs interacting with the physical world. In contrast, control engineers usually consider hybrid systems arising from physical dynamical systems controlled by digital devices. The introduction of digital devices causes difficulties because of the switching of dynamics and resetting of states. In response, control engineers

have extended traditional models, e.g., state-space models, by explicitly introducing switchings and state jumps to model hybrid systems. In particular, two widely used modeling frameworks for hybrid systems proposed in control are introduced, namely switched and piecewise affine system models. Piecewise affine systems can be seen as a special case of switched systems with linear dynamics at each mode and mutual exclusive partitions of the state space. It is also shown how both switched and piecewise affine system models can also be described by hybrid automata.

The reason for studying hybrid systems using several different models instead of a single general model, say a hybrid automaton, is that most often a variety of analysis and synthesis goals need to be addressed and some models offer greater insight, transparency, and efficiency when certain goals are to be achieved. Of course, using multiple models is not unique to hybrid systems; in fact, multiple models are present in almost every discipline.

In the following, Finite Automata are briefly discussed in Sect. 2.1, with additional materials found in Appendix B of the book. Hybrid Automata, which provide a general and most important modeling framework for hybrid systems, are introduced in Sect. 2.2. In Sect. 2.3, Switched Systems and Piecewise Affine Systems are introduced and discussed. A collection of useful results and properties of state-space models for linear, continuous-, and discrete-time, time-invariant systems may also be found in Appendix A. Note that modeling of continuous and discrete signals and systems, as well as their relations when discrete descriptions are extracted from continuous ones, is further discussed in Appendix A. Sampled-data system models, which are common in the study of digital control and have a close relationship to hybrid systems, are also briefly discussed. Notes and Further Reading may be found in Sect. 2.5, and Exercises are included at the end of this chapter.

2.1 Finite Automata

Finite automata are widely used models to describe discrete event systems. A *discrete event system (DES)* is a dynamical system which has discrete-valued states and the evolution among states is triggered by the occurrence of discrete events. For example, the operation of a vending machine can be seen as a DES. Usually, the machine remains in the “ready” mode or state, which can be seen as the initial state of the system. When a customer inserts enough coins, the vending machine will change into “waiting” mode and wait for the customer’s choice for the drink. According to the customer’s choice, the machine will respond with either “Coke” or “Pepsi”, respectively, and deliver the product. After this round of service, the machine will return to the “ready” state and wait for the next customer.

Thus, the system has four states—“ready”, “waiting”, “Output Coke”, and “Output Pepsi”, and four events—“coins received”, “Coke being chosen”, “Pepsi being chosen”, and “Drink being taken”. Let’s denote the set of states as a set

$$X = \{q_0, q_1, q_2, q_3\},$$

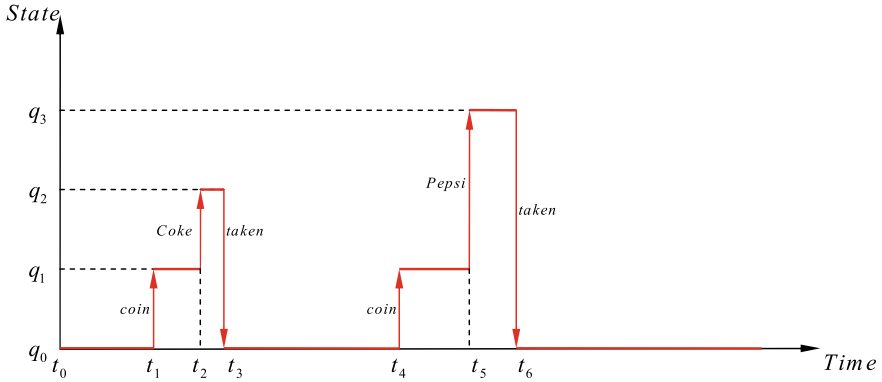


Fig. 2.1 A typical state trajectory of the vending machine example. The transition of states is triggered by discrete events

which is a discrete finite symbol set and corresponds to the four discrete states of the vending machine, respectively. For simplicity, we denote the event set as

$$\Sigma = \{\text{coin}, \text{Coke}, \text{Pepsi}, \text{taken}\}.$$

Then, the behavior of the system can be described by a collection of sequences of triples consisting of states, events, and time instants of the form

$$(q_0, \sigma_1, t_1)(q_1, \sigma_2, t_2) \dots,$$

where $q_0 \in X$ is the initial state, and for each $i \geq 1$, $q_i \in X$ stands for the i th state of the system, $\sigma_i \in \Sigma$ denotes the i th event, and $t_i \in \mathbb{R}_{\geq 0}$ is the time instant when the i th event σ_i occurs. A typical state trajectory of the vending machine example is plotted in Fig. 2.1.

Usually in DES, we ignore the timing information and focus on the ordering of state and event pairs. There are two reasons for doing this. First, when exactly an event happens may not be as significant as the consequences of the event. In our case, the consequences are reflected by state transitions in the DES. For example, we would like to make sure it is always the case that customers paid before getting the drink. Secondly, the relationships between state transitions and events are complex. In addition, it may be difficult to predict when exactly such an event is going to occur. Hence, a logical model is sufficient to study the qualitative properties of the DES.

2.1.1 Finite Automaton Model

Finite automata are popular logical models that have been used in practice successfully. Formally, finite automata can be defined as below.

Definition 2.1 A finite automaton A is a tuple $(Q, Q_0, \Sigma, \delta, F)$ where

- Q is a finite set of states;
- $Q_0 \subseteq Q$ is the set of initial states;
- Σ is a finite set of symbols representing inputs;
- $\delta : Q \times \Sigma \rightarrow 2^Q$ represents the state transition relations;
- $F \subseteq Q$ is a set of accepting states (also known as marked states).

Here, 2^Q stands for the collection of all subsets of Q , i.e., the power set of Q . The map $\delta(q, \sigma)$ specifies the set of states that the current state q can move into when it reads the symbol σ . Note that the set $\delta(q, a_i)$ may contain more than one states, which means that there may exist many possible transitions for each state and symbol; the automaton A is called *non-deterministic* in this case. In contrast, an automaton A is *deterministic* if $|Q_0| = 1$ and $|\delta(q, \sigma)| \leq 1$ for all $q \in Q$, and $\sigma \in \Sigma$. Here, $|\cdot|$ denotes the cardinality of a set. It is also worth pointing out that the set $\delta(q, \sigma)$ may be empty for all $\sigma \in \Sigma$, which means that the automaton A stays at the state q and has nowhere to go. When this happens, the automaton A is called *blocking*. Namely, $\exists q$ (there exists q), such that for $\forall \sigma \in \Sigma$ (for any σ), $\delta(q, \sigma) = \emptyset$. Otherwise, A is *non-blocking*. For illustration, let's revisit the vending machine example.

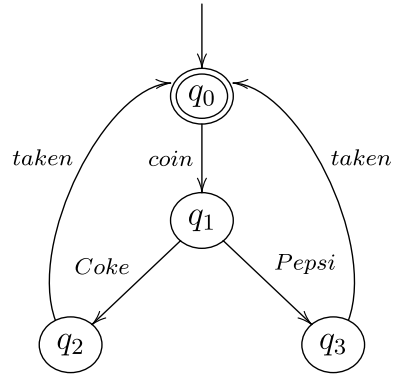
Example 2.1 Formally, the vending machine can be modeled as a finite automaton $A = (Q, Q_0, \Sigma, \delta, F)$, where

- $Q = \{q_0, q_1, q_2, q_3\}$;
- $Q_0 = \{q_0\}$;
- $\Sigma = \{coin, Coke, Pepsi, taken\}$;
- $\delta(q_0, coin) = \{q_1\}, \delta(q_1, Coke) = \{q_2\}, \text{ and } \delta(q_1, Pepsi) = \{q_3\}, \delta(q_2, taken) = \{q_0\}, \delta(q_3, taken) = \{q_0\}$;
- $F = \{q_0\}$.

Intuitively, the dynamics of the system can be modeled by the following digraph in Fig. 2.2. In this figure, the nodes stand for the discrete states in Q , respectively, while the double-circled nodes represent the marked states in F , and the initial state q_0 is identified by a small arrow without source nodes. The edges between nodes correspond to the state transition relations δ with the triggering events labeled. It can be easily verified that the automaton for the vending machine example is deterministic and non-blocking. $\square$

The dynamical behavior of a finite automaton is conveniently described by the strings of its state evolution. Formally, we have the following definitions.

Fig. 2.2 A finite automaton model for the vending machine example



Definition 2.2 A set of symbols, $\Sigma = \{a_1, a_2, \dots\}$, is called an *alphabet*. A word s is a sequence of symbols chosen from the alphabet Σ .

We define the run of a finite automaton A on a given input word as follows.

Definition 2.3 A run r of a finite automaton $A = (Q, Q_0, \Sigma, \delta, F)$ on a finite word $s = a_0a_1 \dots a_{n-1}$, with $a_i \in \Sigma$ for all $i = 0, \dots, n-1$, is a sequence $q_0, q_1, \dots, q_n$ of $n+1$ states in Q such that $q_0 \in Q_0$, and $q_{i+1} \in \delta(q_i, a_i)$ for $0 \leq i < n$. A run $r = q_0, q_1, \dots, q_n$ is *accepting* if $q_n \in F$, and the word s is accepted by A if A has an accepting run on s .

For example, a run of the vending machine example could be

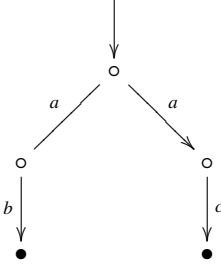
$$q_0 \xrightarrow{\text{coin}} q_1 \xrightarrow{\text{Coke}} q_2 \xrightarrow{\text{taken}} q_0.$$

It means that a customer inserts a coin into the vending machine, and the machine asks the customer to choose between “Coke” and “Pepsi”. After the customer chooses “Coke”, the machine responds and returns to the ready state q_0 .

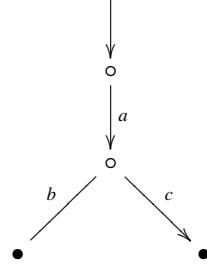
Note that a deterministic finite automaton (DFA) can have at most one run on a given input word, while a non-deterministic finite automaton can have many different runs on a given input word. A word s is accepted by a non-deterministic automaton A if A has at least one accepting run on s . The collection of all finite words $s \in \Sigma^*$ accepted by A is called the *language accepted by A* , denoted as $\mathcal{L}(A)$. Here, we use Σ^* to represent the set of all words of finite length with symbols from Σ . Two automata A_1 and A_2 are called *language equivalent* if they accept the same language, namely $\mathcal{L}(A_1) = \mathcal{L}(A_2)$.

Example 2.2 It is easy to verify that the following two finite automata accept the same language

$T_1 :$



$T_2 :$



In fact, $\mathcal{L}(T_1) = \mathcal{L}(T_2) = \{ab, ac\}$. However, it is obvious the two finite automata are different: T_1 is non-deterministic while T_2 is deterministic. $\square$

2.1.2 Properties of Finite Automata

An important property of finite automata is their *closure* under Boolean operations, i.e., the union or intersection of two finite automata is also a finite automaton, so is the negation of a finite automaton. To be more precise, we now formally state this property:

Proposition 2.1 *Let A_1, A_2 be two finite automata. Then there exists a finite automaton A such that $\mathcal{L}(A) = \mathcal{L}(A_1) \cup \mathcal{L}(A_2)$.*

Proof Let $A_1 = (Q^1, Q_0^1, \Sigma, \delta_1, F_1)$ and $A_2 = (Q^2, Q_0^2, \Sigma, \delta_2, F_2)$. Without loss of generality, assume that Q^1 and Q^2 are disjoint. Define $A = (Q, Q_0, \Sigma, \delta, F)$, where $Q = Q^1 \cup Q^2$, $Q_0 = Q_0^1 \cup Q_0^2$, $F = F_1 \cup F_2$, and

$$\delta(q, \sigma) = \begin{cases} \delta_1(q, \sigma) & \text{if } q \in Q^1, \\ \delta_2(q, \sigma) & \text{if } q \in Q^2. \end{cases}$$

Then, it is easy to verify that $\mathcal{L}(A) = \mathcal{L}(A_1) \cup \mathcal{L}(A_2)$, since a run is accepted by the constructed automaton if it can be accepted by either A_1 or A_2 and the reverse is also true. We left the details as an exercise for readers. $\square$

In the proof, we call A the union of A_1 and A_2 , denoted as $A_1 \cup A_2$.

Proposition 2.2 *Let A_1, A_2 be two finite automata. Then there exists a finite automaton A such that $\mathcal{L}(A) = \mathcal{L}(A_1) \cap \mathcal{L}(A_2)$.*

Proof Let $A_1 = (Q^1, Q_0^1, \Sigma, \delta_1, F_1)$ and $A_2 = (Q^2, Q_0^2, \Sigma, \delta_2, F_2)$. Define $A = (Q, Q_0, \Sigma, \delta, F)$, where $Q = Q^1 \times Q^2$, $Q_0 = Q_0^1 \times Q_0^2$, $F = F_1 \times F_2$, and

$\delta((q_1, q_2), \sigma) = \delta_1(q_1, \sigma) \times \delta_2(q_2, \sigma)$. Then, we can verify that $\mathcal{L}(A) = \mathcal{L}(A_1) \cap \mathcal{L}(A_2)$, since a run is accepted by the constructed automaton only if it can be accepted by both A_1 and A_2 , and vice versa. We left the details as an exercise for the readers. $\square$

In the proof, We call A the product of A_1 and A_2 , denoted as $A_1 \times A_2$. Note that both the union and the product constructions are computationally effective and polynomial in the size of the constituent automata.

Next, we consider the issue of *complementation* for an automaton. First, we focus on the case when the automaton is both *total* and deterministic.

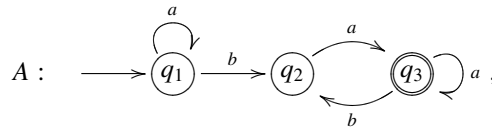
Definition 2.4 An automaton is called total if for all states $q \in Q$ and any event $\sigma \in \Sigma$, the transition $\delta(q, \sigma) \neq \emptyset$.

Note that for any given automaton A , it is always possible to obtain a language equivalent total automaton by introducing a trash state q_t and adding a transition $\delta(q, \sigma) = \{q_t\}$ if and only if $\delta(q, \sigma) = \emptyset$ in A . Hence, without loss of generality we assume an automaton to be total. For a total and deterministic automaton A , it is very easy to obtain its complement.

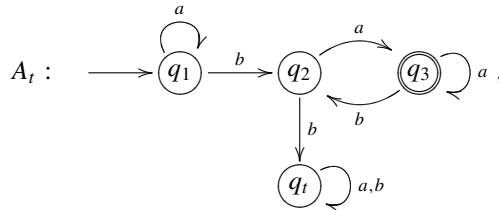
Proposition 2.3 Let $A = (Q, Q_0, \Sigma, \delta, F)$ be a total deterministic finite automaton, and let $\bar{A} = (Q, Q_0, \Sigma, \delta, Q \setminus F)$, then $\mathcal{L}(\bar{A}) = \Sigma^* \setminus \mathcal{L}(A)$. Here the operator $\setminus$ stands the difference between two sets, i.e., the set difference operator.

The proof of this proposition is left as an exercise. Instead, we will take a look at a simple example.

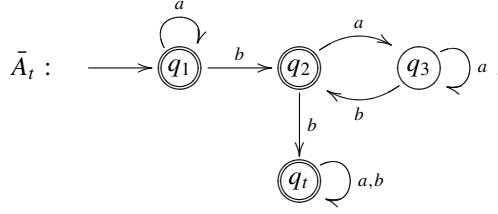
Example 2.3 Consider an automaton



which is deterministic but not total, since $\delta(q_2, b) = \emptyset$. First, we make it total by introducing a trash state q_t , namely



which is deterministic and total. Also, $\mathcal{L}(A_t) = \mathcal{L}(A)$. Applying the procedure in the above proposition, we obtain the complementation as



□

However, the above procedure does not work for non-deterministic finite automata. There may exist many runs on a given input word for a non-deterministic finite automaton, and it is not enough that only some of these runs reject, i.e., not accept, the input word. In fact, if a word $s \in \mathcal{L}(\bar{A})$, all runs should reject the word. Thus, we need to determine the language of the non-deterministic finite automaton before taking its complementation.

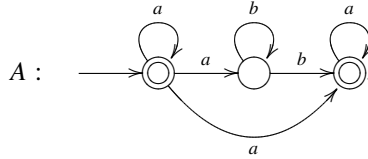
The following important result establishes the fact that there always exists a deterministic automaton that has the same language as the non-deterministic one.

Proposition 2.4 *Let A be a non-deterministic finite automaton. Then there exists a deterministic finite automaton A_d such that $\mathcal{L}(A_d) = \mathcal{L}(A)$.*

Proof Let $A = (Q, Q_0, \Sigma, \delta, F)$. Define $A_d = (2^Q, \{Q_0\}, \Sigma, \delta_d, F_d)$. The state set of A_d consists of all subsets of states in Q . The set $F_d = \{\mathcal{F} \in 2^Q \mid \mathcal{F} \cap F \neq \emptyset\}$ is the collection of sets of states that intersect F nontrivially. Finally, $\forall X \in 2^Q, \sigma \in \Sigma$ define $\delta_d(X, \sigma) = \bigcup_{x \in X} \delta(x, \sigma)$. To show that $\mathcal{L}(A_d) = \mathcal{L}(A)$, we need to show both $\mathcal{L}(A_d) \subseteq \mathcal{L}(A)$ and $\mathcal{L}(A) \subseteq \mathcal{L}(A_d)$, which is left as an exercise. □

In other words, A_d collapses all possible runs of A on a given input word into one run over a larger state set. This construction is called *subset construction*. It can be seen that the construction involves an exponential increase in the number of states (i.e., if A has n states, then A_d has 2^n states). But it gives us the justification that just focusing on deterministic automata does not cause any loss of generality.

Example 2.4 To illustrate the subset construction process, we consider a non-deterministic automaton



where $Q = \{1, 2, 3\}$, $\Sigma = \{a, b\}$, $F = \{1, 3\}$, $Q_0 = \{1\}$. A deterministic finite automaton A_d can be constructed as follows:

- $2^Q = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\};$
- $\{Q_0\} = \{\{1\}\};$
- $F_d = \{\{1\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}.$

For the transition function in A_d , according to Proposition 2.4, it can be inferred that

$$\delta_d(\{1\}, a) = \delta(1, a) = \{1, 2, 3\};$$

$$\delta_d(\{1\}, b) = \delta(1, b) = \emptyset;$$

$$\delta_d(\{1, 2, 3\}, a) = \delta(1, a) \cup \delta(2, a) \cup \delta(3, a) = \{1, 2, 3\};$$

$$\delta_d(\{1, 2, 3\}, b) = \delta(1, b) \cup \delta(2, b) \cup \delta(3, b) = \{2, 3\};$$

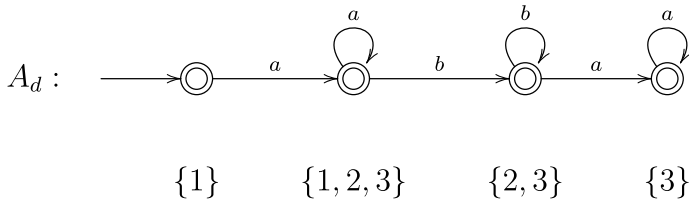
$$\delta_d(\{2, 3\}, a) = \delta(2, a) \cup \delta(3, a) = \{3\};$$

$$\delta_d(\{2, 3\}, b) = \delta(2, b) \cup \delta(3, b) = \{2, 3\};$$

$$\delta_d(\{3\}, a) = \delta(3, a) = \{3\};$$

$$\delta_d(\{3\}, b) = \delta(3, b) = \emptyset.$$

As a result, A_d is shown as



It can be seen that the two automata are language equivalent, i.e., $\mathcal{L}(A_d) = \mathcal{L}(A)$. $\square$

Hence, without loss of generality, we will focus on DFA unless we state otherwise. Combining the above two propositions, we can conclude that the complementation operation on finite automata is also closed.

2.1.3 Regular Languages

The languages that can be accepted by finite automata are regular languages. To formally define regular languages, first, we need to introduce several operations on finite words and languages.

Definition 2.5 For any two finite words $s_1 = a_1a_2 \dots a_m \in \Sigma^*$ and $s_2 = b_1b_2 \dots b_n \in \Sigma^*$, the *concatenation* of s_1 and s_2 is denoted as $s_1 \oplus s_2$, and

$$s_1 \oplus s_2 = a_1a_2 \dots a_mb_1b_2 \dots b_n \in \Sigma^*.$$

We use ε to denote a special event, called an empty event, with the property that $\varepsilon \oplus s = s \oplus \varepsilon = s$ for any $s \in \Sigma^*$. The *concatenation* of L_1 with L_2 , denoted as $L_1 \oplus L_2$, is the language

$$L_1 \oplus L_2 = \{s \in \Sigma^* | s = u \oplus v, \text{ with } u \in L_1, v \in L_2\}.$$

Definition 2.6 The *Kleene closure* of L , denoted as L^* , is the language

$$L^* = \bigcup_{i \in \mathbb{N}} L^i,$$

where $L^0 = \{\epsilon\}$, $L^{i+1} = L^i \oplus L$;

Definition 2.7 The *choice* between L_1 and L_2 , denoted as $L_1 + L_2$, is the language $L_1 + L_2 = L_1 \cup L_2$.

Formally, regular languages can be defined as follows.

Definition 2.8 The class of *regular languages*, denoted by $L_R \subseteq 2^{\Sigma^*}$, is defined by

- $\emptyset, \{\epsilon\} \in L_R$;
- $\forall \sigma \in \Sigma, \{\sigma\} \in L_R$;
- If $K_1, K_2 \in L_R$, then $K_1 + K_2, K_1 \oplus K_2, K_1^* \in L_R$.

This is a recursive definition that commonly used in the computer science literature. It basically says that the union, concatenation and Kleene closure of regular languages are still regular, while the simplest regular languages can be either empty or just containing a single alphabet. It is usually convenient to represent the language accepted by a finite automaton by *regular expressions*, which are defined by

- $\emptyset, \epsilon, \sigma$ are regular expressions for any $\sigma \in \Sigma$;
- if r_1 and r_2 are regular expressions, so are $r_1 + r_2, r_1 \cdot r_2$, and r_1^* .

The regular language denoted by a regular expression is defined by

- $\mathcal{L}(\emptyset) = \emptyset, \mathcal{L}(\epsilon) = \{\epsilon\}$, and $\mathcal{L}(\sigma) = \{\sigma\}$ for any $\sigma \in \Sigma$;
- $\mathcal{L}(r_1 + r_2) = \mathcal{L}(r_1) \cup \mathcal{L}(r_2)$;
- $\mathcal{L}(r_1 \cdot r_2) = \mathcal{L}(r_1) \oplus \mathcal{L}(r_2)$;
- $\mathcal{L}(r^*) = \mathcal{L}^*(r)$.

Example 2.5 The vending machine automaton accepts a language that can be expressed as

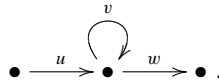
$$(coin \cdot (Coke + Pepsi) \cdot taken)^*,$$

where the operator “ $\cdot$ ” stands for concatenation of events (usually ignored), “ $+$ ” means choice between events, and “ * ” indicates any finite number of repetitions of an event or string of events. The above expression characterizes the language accepted by the vending machine automaton, which can be presented as a finite number of repeats of the pattern—the event *coin* followed by either the event *Coke* or the event *Pepsi*, and then the event *taken*. Note that nothing happening, denoted as ϵ , is also acceptable as it corresponds to zero repeating of such a pattern. Its generated language is the *prefix closure* of the accepted language, i.e., the set of all prefixes of the strings in $(coin \cdot (Coke + Pepsi) \cdot taken)^*$, which is denoted as $pr((coin \cdot (Coke + Pepsi) \cdot taken)^*)$. $\square$

Regular languages characterize the class of languages that can be accepted by a finite automaton in the following sense: Given a finite automaton, A , there exists a regular expression γ such that $\mathcal{L}(\gamma) = \mathcal{L}(A)$. On the other hand, given a regular language K (or a regular expression γ), we can build a finite automaton (not unique, but there is always a deterministic finite automaton) that accepts K (or the regular language represented by γ , i.e., $\mathcal{L}(\gamma)$). The formal proof of these claims can be found in Appendix B.

Since finite automata are closed under Boolean operations, regular languages are also closed under union, intersection, and complementation. It is worth pointing out that regular languages just form a strict subclass of finite languages. We conclude our brief introduction to finite automata with an example of a non-regular language that cannot be accepted by any finite automaton.

Example 2.6 Consider a language $K = \{a^i b^i \mid i \geq 0\}$ defined over $\Sigma = \{a, b\}$. It basically requests that there has to be the same number of b after a sequence of finite appearances of a . We show that K is non-regular, by contradiction: Assume that K is regular, then there exists a deterministic finite automaton G such that $\mathcal{L}(G) = K$. Suppose that the number of states in G is n , and we select $s = a^m b^m$ with $m > n$. It is obvious that $s \in K$. Consider a prefix t of s with $|t| = n$. Note that $t = a^n$ and there exists at least one state that has been visited twice when t is executed (since G has only n distinct states). Denote the first of such states as x , so we can rewrite s as $s = uvw$, where $\delta(q_0, u) = x$, $\delta(x, v) = x$, and $\delta(x, w) \in F$ with an extension of the transition relation $\delta(q, \cdot)$ to a string of events:



Assume that $|u| = i$, $|v| = k$; then we have $i \leq n < m$ and $k \geq 1$. So, $s = a^i a^k a^{m-i-k} b^m$. Also, since $\delta(x, v) = x$ and $\delta(x, w) \in F$, the following string $s' = a^i (a^k)^j a^{m-i-k} b^m$ should be accepted by G for any integer j . If we pick $j = 2$, we have $s' \in a^{m+k} b^m \in \mathcal{L}(G)$. However, it is obvious that $s' \notin K$, which is a contradiction. $\square$

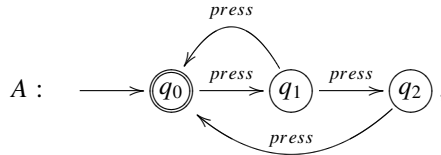
Our above introduction to automata is quite standard. Additional information about automata may be found in Appendix B. The interested reader may refer to any classical textbook on finite automata, such as [1, 2], for a complete treatment.

2.2 Hybrid Automata

Finite automata have been successfully used in the modeling and analysis of typical DESs, such as communication protocols and computer programs, where logic correctness (e.g., deadlock-free) is the main concern [3, 4]. However, finite automata cannot model, for example, cyber-physical systems, where both discrete event dynamics and continuous physical dynamics coexist and interact with each other

[5, 6]. As a motivation to extend the finite automaton model, let's take a look at a light switch example.

Example 2.7 The light switch functions in the following manner. Initially the light is off, and it will turn on if we press the on/off button. The light will be brighter if we press the switch again immediately following the first press (within 3 s); otherwise, the light will turn off after the second press. If we model the switch as a finite automaton, we may obtain the following model:



where the initial state q_0 stands for light-off, q_1 means light-on, and q_3 represents brighter. This model is non-deterministic as we cannot distinguish a quick double press from two press events. For instance, a double press may turn off the light instead of getting a brighter light. This is definitely problematic. $\square$

To resolve this issue, one solution is to add a clock variable x , i.e., $\dot{x} = 1$ and $x \geq 0$, into the model and extend the finite automaton to a hybrid automaton.

2.2.1 Hybrid Automata Models

Hybrid automata provide formal models for hybrid dynamical systems, and can be seen as an extension of finite automata by adding continuous dynamics into each of its discrete states (also called modes). Each mode is associated with constraints within which the continuous dynamics evolves. Edges between modes are annotated with guards that specify the conditions for the mode transition to be triggered; each edge is also associated with a reset map indicating how the continuous variables are being updated after the discrete transition. Following [7–9], hybrid automata are defined below (with some restrictions for simplicity).

Definition 2.9 A hybrid automaton H is a collection

$$H = \{Q, X, f, Init, Inv, E, G, R\},$$

where

- $Q = \{q_1, q_2, \dots\}$ is a finite set of discrete states;
- $X \subseteq \mathbb{R}^n$ represents the state space of the continuous state variables;
- $f : Q \times X \rightarrow \mathbb{R}^n$ assigns to each discrete state $q \in Q$ an analytic vector field $f(q, \cdot)$;

- $Init \subseteq Q \times X$ is the set of initial states;
- $Inv : Q \rightarrow 2^X$ assigns to each discrete state $q \in Q$ a set $Inv(q) \subseteq X$ called the *invariant set*;
- $E \subseteq Q \times Q$ is the set of discrete transitions;
- $G : E \rightarrow 2^X$ assigns to each discrete transition $(q, q') \in E$ a *guard set* $G(q, q') \subset X$;
- $R : E \times X \rightarrow 2^X$ is a *reset map*.

We refer to (q, x) , where $q \in Q$ and $x \in X$, as the state of H . As our first example of hybrid automata, we revisit the light switch example.

Example 2.8 Recall that the finite automata model fails to capture a critical feature of the light switch as it cannot distinguish a quick double press from two (separated) presses. The key information missing is the time elapse sine the first press. To gain such information, we need to introduce a continuous variable x and propose the following hybrid automaton model:

- the discrete states (a.k.a. locations or modes) $Q = \{q_0, q_1, q_2\}$ representing the status of the light as before;
- the continuous variable $X = \{x\}$ and $x \in \mathbb{R}$;
- the flow dynamics $f(q, x) = 1$, i.e., $\dot{x} = 1$, in all discrete states $q \in Q$;
- the initial state $Init = (q_0, 0)$;
- for each discrete mode $Inv(q) = \{x \geq 0\}$;
- there are four discrete transitions $E = \{(q_0, q_1), (q_1, q_2), (q_1, q_0), (q_2, q_0)\}$;
- the guard conditions for each discrete transition are defined as $G(q_0, q_1) = true$, $G(q_1, q_2) = \{x \leq 3\}$, $G(q_1, q_0) = \{x < 3\}$, and $G(q_2, q_0) = true$, where *true* means that all continuous-variable values satisfy the guard condition;
- the reset maps associated with the discrete transitions in E are give as follows: $R((q_0, q_1), x) = 0$, $R((q_1, q_2), x) = x$, $R((q_1, q_0), x) = x$, and $R((q_2, q_0), x) = x$.

A hybrid automaton can be visualized as a directed graph. To represent graphically a hybrid automaton, we first draw a directed graph (V, E) with a one-to-one mapping between the vertices V and the discrete states Q , while E is the same as in the definition of the hybrid automaton. Secondly, for each vertex of the graph, we specify the discrete mode $q \in Q$, the differential equation implied by the vector field $\dot{x} = f(q, x)$, and the invariant condition $Inv(q)$ in each vertex of the graph. On each edge $(q, q') \in E$, the guard condition $G(q, q')$ and the reset map $x' := R((q, q'), x)$ are specified. Finally, the initial state may be marked by an arrow (an edge without a source vertex) pointing at the vertex $q \in Q_0$ with the set $Init(q)$ specified on it. The graphical representation of the hybrid automaton model for the light switch example is shown in Fig. 2.3.

In the hybrid automaton model, the continuous variable x is reset to zero every time when we first press the switch while the light is off. It starts counting the time lapse from the first press till the next time the switch is pressed. If the time duration is less than 3 s, i.e., $x \leq 3$, then the system recognizes a quick double press and transits

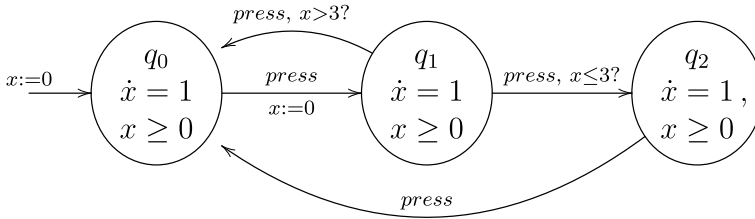


Fig. 2.3 A hybrid automaton model of the light switch

the mode to q_2 (brighter); otherwise, the second press is recognized as the press to turn off the light. Hence, x is called a clock variable, and the hybrid automaton in this example actually belongs to timed automata—a subclass of hybrid automata. $\square$

As another example, let's model the temperature control system, described in Chap. 1, as a hybrid automaton.

Example 2.9 Consider a typical room temperature control system. The system contains two discrete modes corresponding to the on–off operation of the thermostat. The temperature follows different continuous dynamics depending on whether the furnace is on or off. It is assumed that the furnace is initially off and the temperature is below 60° . As we start the temperature control process, the furnace is turned on and remains on heating the room as long as the temperature is below 70° . Once the room temperature reaches 70° , the furnace will be commanded to turn off. Due to latency and other practical reasons, the furnace actually turns off just before the temperature hits 71° . As the furnace is off, the room temperature starts dropping due to heat losses. Once the temperature drops below 70° , the furnace will be commanded to turn on; for practical reasons, it actually turns on just before the temperature drops to 69° . Formally, the temperature control system can be modeled as a hybrid automaton with elements in the model being identified as follows:

- $Q = \{\text{ON}, \text{OFF}\}$;
- $X = \mathbb{R}$ is the continuous-variable state denoting the room temperature;
- $f(\text{ON}, x) = -x + 100$ and $f(\text{OFF}, x) = -x$;
- $\text{Init} = \{\text{OFF}\} \times \{x \leq 60\}$;
- $\text{Inv}(\text{ON}) = \{x \in \mathbb{R} : x \leq 71\}$, and $\text{Inv}(\text{OFF}) = \{x \in \mathbb{R} : x \geq 69\}$;
- $E = \{(\text{ON}, \text{OFF}), (\text{OFF}, \text{ON})\}$;
- $G(\text{ON}, \text{OFF}) = \{x \in \mathbb{R} : x \geq 70\}$, and $G(\text{OFF}, \text{ON}) = \{x \in \mathbb{R} : x \leq 70\}$;
- $R((\text{ON}, \text{OFF}), x) = \{x\}$, and $R((\text{OFF}, \text{ON}), x) = \{x\}$.

The graphical representation of the hybrid automaton model for the temperature control example is shown in Fig. 2.4. $\square$

The temperature control example illustrates the switching of dynamics due to discrete transitions. Another important dynamical phenomenon is that upon a discrete transition, the continuous state may suddenly change its value (state jump) as well. To illustrate such a phenomenon, consider the following example from [10].

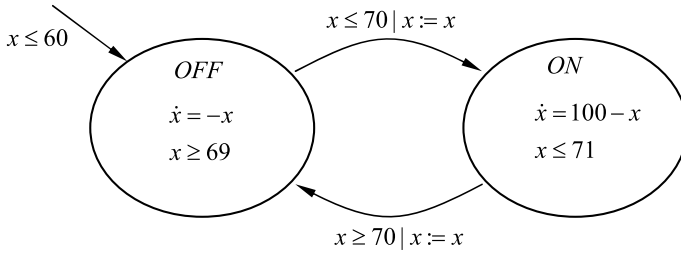
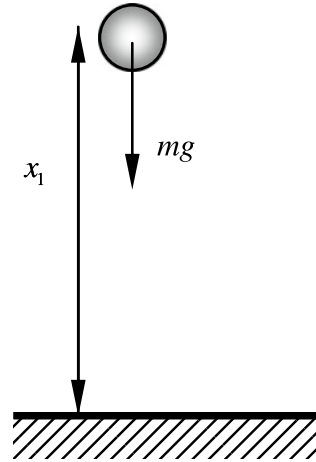


Fig. 2.4 Temperature control hybrid automaton model

Fig. 2.5 The bouncing ball example



Example 2.10 Consider a bouncing ball as shown in Fig. 2.5. The vertical position of the ball is denoted by x_1 and the velocity is denoted by x_2 . As long as the ball is above the ground ($x_1 > 0$), the continuous dynamics can be described by

$$\begin{cases} \dot{x}_1 = x_2, \\ \dot{x}_2 = -g, \end{cases}$$

where g is the gravitational constant. When the ball hits the ground ($x_1 = 0$), a discrete jump takes place. The speed x_2 is reset according to $x_2 := -cx_2$, where $c \in (0, 1)$ is a coefficient of restitution.

It is straightforward to define a hybrid automaton as follows to describe this process:

- $Q = \{q\}$;
- $X = \mathbb{R}^2$;
- $f(q, x) = \begin{bmatrix} x_2 \\ -g \end{bmatrix}$;
- $Init = \{q\} \times \{x \in \mathbb{R}^2 \mid x_1 > 0, x_2 = 0\}$;
- $Inv(q) = \{x \in \mathbb{R}^2 \mid x_1 \geq 0\}$;

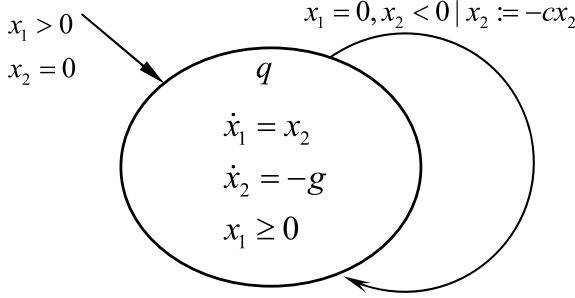


Fig. 2.6 The bouncing ball hybrid automaton model

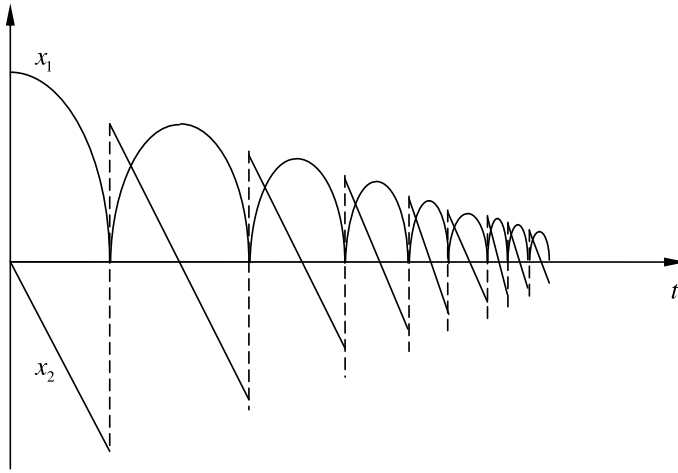


Fig. 2.7 Typical trajectories for the position x_1 and velocity x_2 of the bouncing ball example

- $E = \{(q, q)\}$;
- $G(q, q) = \{x \in \mathbb{R}^2 \mid x_1 = 0, x_2 < 0\}$;
- $R((q, q), x) = \{x_1 = x_1, x_2 = -cx_2\}$.

Then its graphical representation of the hybrid automaton model is plotted in Fig. 2.6. It only contains a single discrete mode, while discrete (self-)transitions occur when the ball hits the ground ($x_1 = 0$ and $x_2 < 0$, i.e., the guard set condition $G(q, q)$ holds). Accompanying the transition, the velocity x_2 is reset following the reset map, which corresponds to a state jump, and the velocity x_2 changes its direction suddenly after bouncing, i.e., $x_2 := -cx_2$, with $0 < c < 1$.

Typical trajectories for the position x_1 and velocity x_2 of the bouncing ball example are plotted in Fig. 2.7. Usually, we also need to consider the evolution of the discrete states of a hybrid automaton. In this example, however, there is only one discrete state, so the time evolution of the discrete state is q for the whole duration. $\square$

2.2.2 Hybrid Automata Composition

A useful extension of the hybrid automaton model presented here is to add a synchronizing event for each discrete transition. In particular, a hybrid automaton with synchronizing events is a tuple $(H, \Sigma) = \{Q, X, f, \text{Init}, \text{Inv}, E, \Sigma, G, R\}$, where H is a hybrid automaton while Σ is a finite set of discrete events. The discrete transition becomes $E \subseteq Q \times \Sigma \times Q$. An element $(q, \sigma, q') \in E$ stands for a discrete transition from mode q to q' in H , and it is triggered by an event σ in Σ . We can define the composition operation of two hybrid automata through *synchronizing events*.

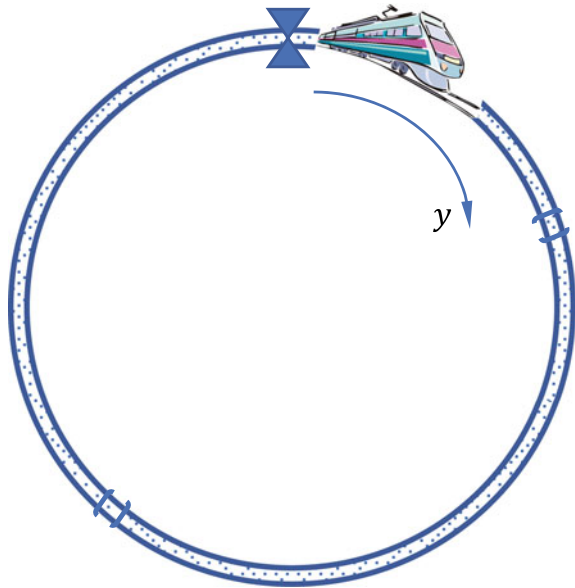
Definition 2.10 Given two hybrid automata $(H_i, \Sigma_i) = \{Q_i, X_i, f_i, \text{Init}_i, \text{Inv}_i, E_i, \Sigma_i, G_i, R_i\}$, for $i = 1, 2$, their composition is a hybrid automaton with

- $Q = Q_1 \times Q_2$, $X = X_1 \cup X_2$, $\Sigma = \Sigma_1 \cup \Sigma_2$, $f = f_1 \wedge f_2$ (consistent for $x_i \in X_1 \cap X_2$), $\text{Init} = \text{Init}_1 \wedge \text{Init}_2$, and $\text{Inv} = \text{Inv}_1 \wedge \text{Inv}_2$ (Here “ $\wedge$ ” means the conjunction of two predicates, i.e., both invariant or initial conditions hold.).
- E is defined as follows: $(\{q_1, q_2\}, \sigma, \{q'_1, q'_2\}) \in E$ if either
 - $\sigma \in \Sigma_1 \cap \Sigma_2$, $(q_1, \sigma, q'_1) \in E_1$, and $(q_2, \sigma, q'_2) \in E_2$;
 - $\sigma \in \Sigma_1$, $\sigma \notin \Sigma_2$, $(q_1, \sigma, q'_1) \in E_1$, and $q_2 = q'_2$;
 - Or $\sigma \notin \Sigma_1$, $\sigma \in \Sigma_2$, $(q_2, \sigma, q'_2) \in E_2$, and $q_1 = q'_1$.
- For any $(\{q_1, q_2\}, \sigma, \{q'_1, q'_2\}) \in E$, we have that
 - $G = G_1 \wedge G_2$, $R = R_1 \wedge R_2$ if $\sigma \in \Sigma_1 \cap \Sigma_2$;
 - $G = G_1$, $R = R_1$ if $\sigma \in \Sigma_1$, $\sigma \notin \Sigma_2$;
 - $G = G_2$, $R = R_2$ if $\sigma \notin \Sigma_1$, $\sigma \in \Sigma_2$.

Note that after composition, we obtain a hybrid automaton. It is also possible to extend the composition operation to more than two hybrid automata. The consistency requirement on f for $x_i \in X_1 \cap X_2$ is in the following sense: If $\text{Inv}_1(q_1) \cap \text{Inv}_2(q_2) \neq \emptyset$ then $\dot{x}_i = f_1^i(q_1, x) = f_2^i(q_2, x)$ for $x_i \in X_1 \cap X_2$. Intuitively, it means that the flow condition f_i for a common physical variable x_i should be the same at both H_1 and H_2 at any given time instant. This is very natural as the same physical variable cannot follow different physical laws at the same time. If non-consistency were to be found in the composition process for the same physical variable, it could be an indication that we probably should use two different variables to distinguish them as they may actually describe different physical quantities.

The idea behind introducing synchronizing events and composition of hybrid automata is to facilitate the modeling of larger scale and more complicated hybrid dynamical systems, by breaking them into smaller and simpler modules that are easier to model as hybrid automata. Then, we can introduce synchronizing events among different modules and compose them into a hybrid automaton model describing the original, complex system.

Fig. 2.8 Illustration of the train, the circular track, and the gate



Example 2.11 Let's use a classical train-gate-controller system to illustrate the basic idea of event synchronization and composition of hybrid automata. The system consists of three components, the train, the gate, and the gate controller. A road crosses the train track and it is guarded by a gate which must be lowered to stop the traffic when the train approaches and raised after the train has passed the road. The gate controller receives information from sensors located on the track and lowers or raises the gate.

The train moves clockwise on a circular track as illustrated in Fig. 2.8. The length of the track is $L = 25$. The location of the train is indicated by the state variable y where $0 \leq y < 25$. The velocity of the train is described by the differential equation $\dot{y} = f(y)$ where $f(y)$ is an appropriate function of y . The gate is located at $y = 0$ on the train track, while the sensors are at $y = 5$ and $y = 15$. We model the combination of train and controller as a hybrid automaton as shown in Fig. 2.9. The controller receives information from the sensors and detects when the train approaches or moves away from the crossing, and correspondingly triggers the transition between two control modes.

The height of the gate is represented by the state variable x . When the gate is lowered, the height of the gate decreases according to the equation $\dot{x} = (1 - x)/2$. When the gate is raised, the height increases according to $\dot{x} = (10 - x)/2$. The hybrid automaton in Fig. 2.10 is used to model the dynamic behavior of the gate. The automaton has two control modes, "Lower" and "Raise". The transitions of the automaton are labeled with the events "Up" and "Down", which are generated by the controller. The hybrid automaton of the overall system is obtained by their composition and is shown in Fig. 2.11. $\square$

Fig. 2.9 Hybrid automaton modeling the train and controller

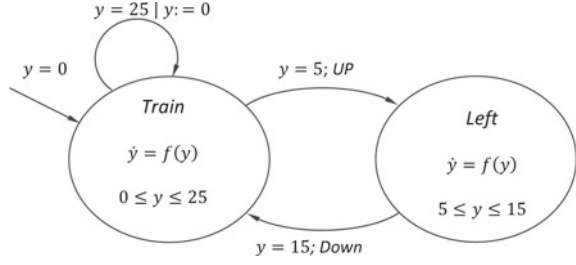


Fig. 2.10 Hybrid automaton modeling the gate

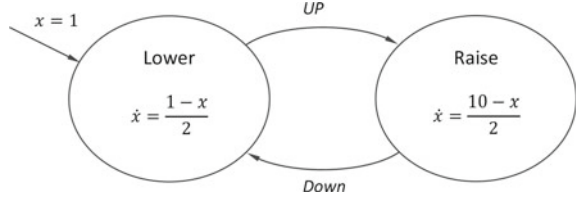
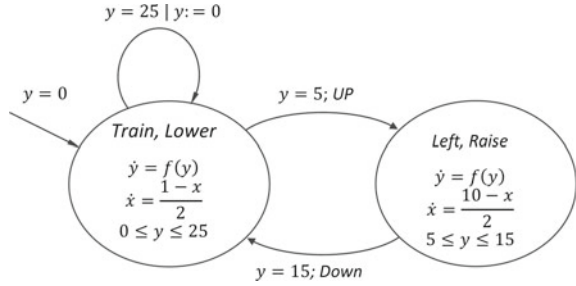


Fig. 2.11 Composed hybrid automaton model for the train-gate-controller example



2.2.3 Hybrid Execution

Given a hybrid automaton, a typical state trajectory can be described as follows: Starting from a point in an initial region, say $(q_0, x_0) \in \text{Init}$, first, the q_0 th mode dynamics are followed, i.e., $\dot{x} = f(q_0, x)$, with the initial condition $x(0) = x_0$ while $x(t) \in \text{Inv}(q_0)$. Assume that the continuous state $x(t)$ evolves into the guard set $G(q_0, q_1)$ and a discrete transition from q_0 to q_1 occurs at time t_1 , where $x(t_1) \in G(q_0, q_1)$. Accompanying the transition, the continuous state x is also being reset by following the reset map $R((q_0, q_1), x(t_1))$, which is contained in the invariant set of mode q_1 , $\text{Inv}(q_1)$. Then, the continuous state x evolves again following the dynamics of mode q_1 as illustrated in Fig. 2.12.

Next, we formally define hybrid time trajectories and hybrid executions. Our definitions and notation in this section mainly follow [9].

Definition 2.11 A hybrid time trajectory is a finite or infinite sequence of intervals $\mathcal{T} = \{I_i\}_{i=1}^N$, where

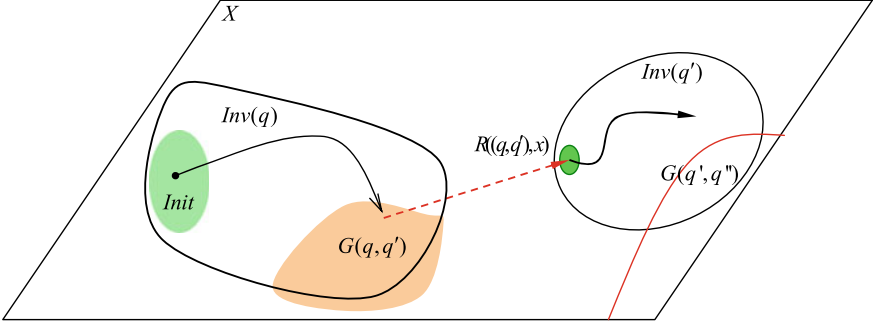


Fig. 2.12 A typical trajectory of a hybrid automaton

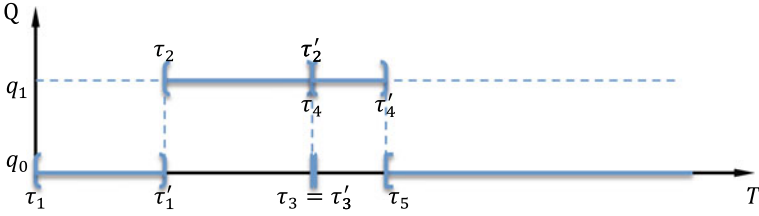


Fig. 2.13 Illustration of hybrid time trajectory

- $I_i = [\tau_i, \tau'_i] \subseteq \mathbb{R}$ for all $i \leq N$;
- If $N < \infty$, then either $I_N = [\tau_N, \tau'_N]$ or $I_N = [\tau_N, \tau'_N)$;
- $\tau_i \leq \tau'_i = \tau_{i+1}$ for all i .

A hybrid time trajectory is a sequence of intervals of the real line, the end points of which overlap, as illustrated in Fig. 2.13. The interpretation is that the end points of the intervals are the times at which discrete transitions take place. Hybrid time trajectories can extend to infinity if $\mathcal{T}$ is an infinite sequence or if it is a finite sequence ending with an interval of the form $[\tau_N, \infty)$. We use $t \in \mathcal{T}$ as a shorthand notation to denote that there exists i such that $t \in I_i$ with $I_i \in \mathcal{T}$. Each hybrid time trajectory $\mathcal{T}$ is linear-ordered by the relation $<$, which is defined as follows. For $t_1, t_2 \in \mathcal{T}$, there exist i and j such that $t_1 \in [\tau_i, \tau'_i]$ and $t_2 \in [\tau_j, \tau'_j]$. We say $t_1 < t_2$ if $t_1 < t_2$ or $i < j$.

The following operations defined in [9] on the hybrid time trajectory $\mathcal{T}$ are useful for the definition and classification of executions of a hybrid automaton H .

- **Prefix:** $\mathcal{T} = \{I_i\}_{i=1}^N$ is said to be a prefix of $\mathcal{T}' = \{I'_i\}_{i=1}^M$, denoted as $\mathcal{T} \sqsubseteq \mathcal{T}'$ if $\mathcal{T} = \mathcal{T}'$ or $\mathcal{T}$ is finite and $N \leq M$, $I_i = I'_i$ for $i = 1, \dots, N$.
- **Index set:** Define the index set of $\mathcal{T} = \{I_i\}_{i=1}^N$ as $<\mathcal{T}> = \{1, 2, \dots, N\}$ if N is finite, or $<\mathcal{T}> = \mathbb{N}$ if N is infinite.
- **Length:** Define the length of $\mathcal{T} = \{I_i\}_{i=1}^N$ as $|\mathcal{T}| = \sum_{i \in <\mathcal{T}>} (\tau'_i - \tau_i)$.

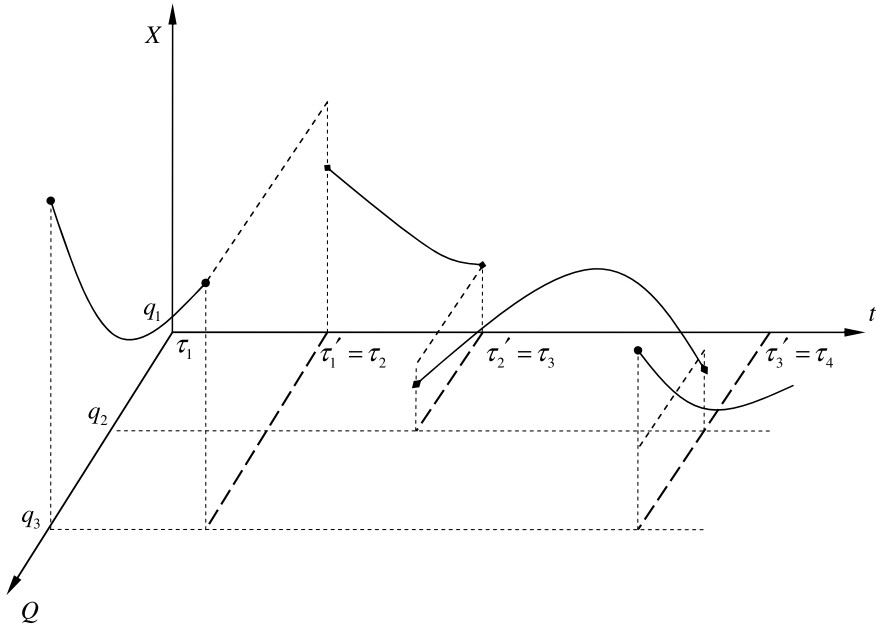


Fig. 2.14 A typical trajectory of a hybrid automaton

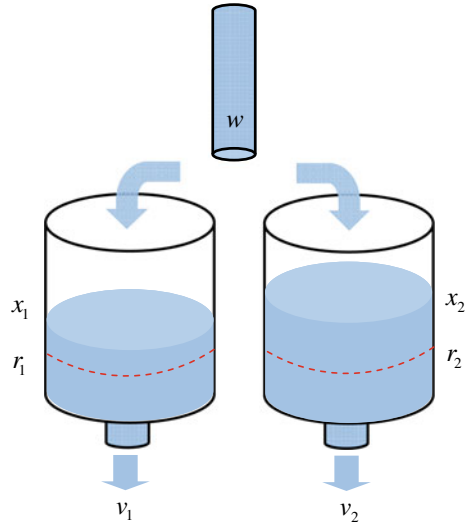
Next, we introduce a concept for hybrid automata that is analogous to a solution of a continuous dynamical system. With a slight abuse of notation, we use q and x to also denote the time evolution of the discrete and continuous states, respectively.

Definition 2.12 An execution of a hybrid automaton H is a collection $\mathcal{X} = (\mathcal{T}, q, x)$, where

- $\mathcal{T}$ is a hybrid time trajectory;
- $q : \mathcal{T} \rightarrow Q$ is a map;
- $x = \{x^i, i \in \mathcal{T}\}$ is a collection of differentiable maps $x^i : I_i \rightarrow X$ such that
 1. $(q(1), x(1)) \in \text{Init}$;
 2. for all $t \in [\tau_i, \tau'_i]$, $\dot{x}^i(t) = f(q(i), x^i(t))$ and $x^i(t) \in \text{Inv}(q(i))$;
 3. for all $i \in \mathcal{T} \setminus N$ (it means that at all switching indices i except the final switching index if N is finite), $e = (q(i), q(i+1)) \in E$, and $x^i(\tau'_i) \in G(e)$, $x^{i+1}(\tau_{i+1}) \in R(e, x^i(\tau'_i))$.

In Fig. 2.14, a typical execution of a hybrid automaton is illustrated, where the horizontal t -axis represents a hybrid time trajectory. Notice that the hybrid executions consist of concatenations of several pieces of the continuous flow x^i . There are discrete transitions and possible state jumps between any two successive pieces of continuous flow. Note that multiple discrete transitions may take place at the same τ_i . Continuous flows keep the discrete part q of the hybrid state constant; the continuous part evolves over time according to the differential equation $\dot{x} = f(q, x)$, as long

Fig. 2.15 Two-tank system example



as $x \in \text{Inv}(q)$. If during the continuous flow it happens that the continuous part hits a guard set, i.e., $x \in G(e)$ for some $e = (q, q') \in E$, then the edge e becomes enabled. The state may then instantaneously jump from (q, x) to any (q', x') with $x' \in R(e, x)$. Then the process repeats, and the continuous parts of the state evolve according to the differential equation $\dot{x} = f(q', x)$.

Example 2.12 Consider a two-tank system as shown in Fig. 2.15. Water is added to these two tanks through a hose (with constant flow rate w), and the goal is to keep the water levels of the tanks above a certain level, r_1 and r_2 . However, both tanks are leaking; also the hose is assigned to only one tank at any time instant, either Tank 1 or Tank 2.

It is assumed that the leaking rates for both tanks are constant, and are v_1 and v_2 , respectively. Let x_i denote the level of water in Tank i , for $i = 1, 2$. Also assume that the initial water levels are above r_1 and r_2 , respectively.

To make sure that the water level of Tank i is above r_i for $i = 1, 2$, we employ the following control law: switching the inflow to Tank 1 whenever $x_1 \leq r_1$ and to Tank 2 whenever $x_2 \leq r_2$.

It is straightforward to define a hybrid automaton to describe this process:

- $Q = \{q_1, q_2\}$;
- $X = \mathbb{R}^2$;
- $f(q_1, x) = \begin{bmatrix} w - v_1 \\ -v_2 \end{bmatrix}$ and $f(q_2, x) = \begin{bmatrix} -v_1 \\ w - v_2 \end{bmatrix}$;
- $\text{Init} = \{q_1, q_2\} \times \{x \in \mathbb{R}^2 \mid x_1 \geq r_1, x_2 \geq r_2\}$;
- $\text{Inv}(q_1) = \{x \in \mathbb{R}^2 \mid x_2 \geq r_2\}$, and $\text{Inv}(q_2) = \{x \in \mathbb{R}^2 \mid x_1 \geq r_1\}$;

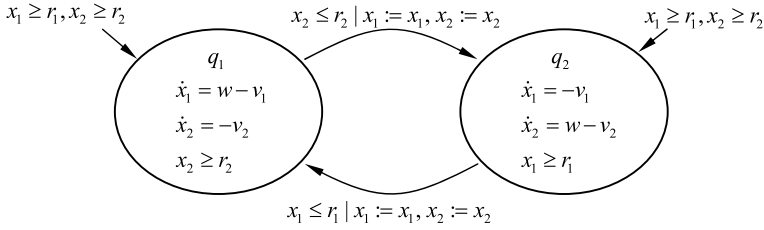
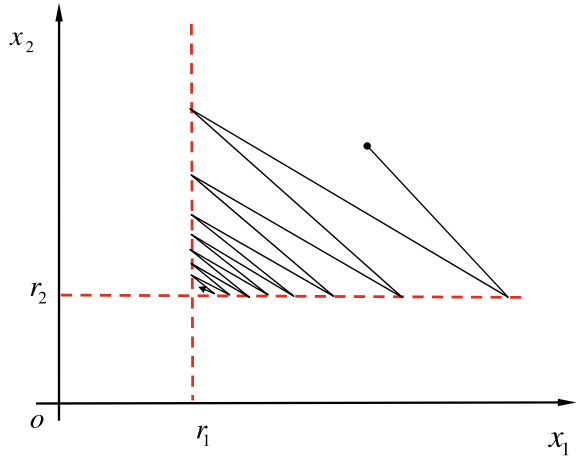


Fig. 2.16 The water tank hybrid automaton model

Fig. 2.17 Illustration of a typical trajectory of the water tank example



- $E = \{(q_1, q_2), (q_2, q_1)\}$;
- $G(q_1, q_2) = \{x \in \mathbb{R}^2 \mid x_2 \leq r_2\}$, and $G(q_2, q_1) = \{x \in \mathbb{R}^2 \mid x_1 \leq r_1\}$;
- $R((q_1, q_2), x) = \{x\}$, and $R((q_2, q_1), x) = \{x\}$.

The graphical representation of the hybrid automaton model is plotted in Fig. 2.16. It can be shown that the water tank hybrid automaton accepts a unique infinite execution for each initial state. As an illustration, a typical state trajectory is plotted in Fig. 2.17 for an initial condition $x_1(0) \geq r_1$ and $x_2(0) \geq r_2$, under the assumption that $\max\{v_1, v_2\} < w < v_1 + v_2$.

It can be seen that the switching between the two tanks becomes faster and faster, and the state trajectories converge to the point $(r_1, r_2)^T$. This phenomenon is independent of the initial conditions provided that $x_1(0) \geq r_1$ and $x_2(0) \geq r_2$. In other words, the region $Init = \{q_1, q_2\} \times \{x \in \mathbb{R}^2 \mid x_1 \geq r_1, x_2 \geq r_2\}$ is invariant, which means that all trajectories that start from this region will stay there.

However, the condition, $\max\{v_1, v_2\} < w < v_1 + v_2$, implies that the rate at which water is added to the system is less than the rate at which water is removed. Physical intuition suggests that in this case at least one of the water tanks will eventually become empty. Why does the analysis of the hybrid automaton fail to predict that? This is because there are an infinite number of switches within a finite time interval,

called the Zeno phenomenon in the literature; see, e.g., [10]. However, in reality every switch of the hose between tanks actually takes some time, so the Zeno phenomenon predicted by the hybrid automata model in Fig. 2.16 will not happen in the real world. $\square$

Motivated by the complexity of dynamical behaviors, in the following, we classify the executions of hybrid automata. We first introduce the prefix definition.

Definition 2.13 $\mathcal{X} = (\mathcal{T}, q, x)$ is said to be a prefix of $\hat{\mathcal{X}} = (\mathcal{T}', \hat{q}, \hat{x})$, denoted as $\mathcal{X} \sqsubseteq \hat{\mathcal{X}}$ if $\mathcal{T} \sqsubseteq \mathcal{T}'$, and for all $i \in \langle \mathcal{T} \rangle$, $\forall t \in I_i$, $(q(i), x^i(i)) = (\hat{q}(i), \hat{x}^i(i))$.

When $\mathcal{X} \sqsubseteq \hat{\mathcal{X}}$ and in addition $\mathcal{X} \neq \hat{\mathcal{X}}$, then $\mathcal{X}$ is called a strict prefix of $\hat{\mathcal{X}}$ and denoted as $\mathcal{X} \sqsubset \hat{\mathcal{X}}$.

Then, following [9], we introduce the following notation:

- an execution is *maximal*, if it is not a strict prefix for any other execution;
- an execution is *finite*, if $\mathcal{T}$ is finite with a closed final interval;
- an execution is *infinite*, if $\mathcal{T}$ is either an infinite sequence or $\tau'_N = \infty$;
- an execution is *Zeno*, if it is infinite but $|\mathcal{T}| < \infty$ (finite total time duration). Equivalently, an execution is Zeno if it takes an infinite number of discrete transitions in a finite amount of time.

It is easy to see that, under our definitions, the transition times τ_i of a Zeno execution converge to some finite accumulation point from the left. In other words, the definition of an execution precludes the situation where the transition times have a right accumulation point.

2.2.4 Determinism and Non-blocking Properties

Before the formal definitions of non-blocking and deterministic hybrid automaton are stated, the following notation is introduced:

- let $\mathcal{E}_H(q_0, x_0)$ represent all executions of a hybrid automaton H with the initial condition $(q_0, x_0) \in \text{Init}$;
- let $\mathcal{E}_H^M(q_0, x_0) \subseteq \mathcal{E}_H(q_0, x_0)$ denote all maximal executions of H with the initial condition $(q_0, x_0) \in \text{Init}$;
- let $\mathcal{E}_H^*(q_0, x_0) \subseteq \mathcal{E}_H(q_0, x_0)$ denote all finite executions of H with the initial condition $(q_0, x_0) \in \text{Init}$;
- let $\mathcal{E}_H^\infty(q_0, x_0) \subseteq \mathcal{E}_H(q_0, x_0)$ denote all infinite executions of H with the initial condition $(q_0, x_0) \in \text{Init}$.

We can now define the following properties for hybrid automata.

Definition 2.14 A hybrid automaton H is *non-blocking*, if $\mathcal{E}_H^\infty(q_0, x_0) \neq \emptyset$ for all initial conditions $(q_0, x_0) \in \text{Init}$.

Definition 2.15 A hybrid automaton H is *deterministic*, if $\mathcal{E}_H^M(q_0, x_0)$ contains at most one solution for all initial conditions $(q_0, x_0) \in \text{Init}$.

To check the non-blocking and deterministic properties of a hybrid automaton, we need to define the set of states reachable by a hybrid automaton H as

$$\text{Reach}_H = \left\{ (\hat{q}, \hat{x}) : \exists (q_0, x_0) \in \text{Init}, (\mathcal{T}, q, x) \in \mathcal{E}_H^*(q_0, x_0), \text{ s.t. } (q(N), x^N(\tau_N)) = (\hat{q}, \hat{x}) \right\}, \quad (2.1)$$

and the set of states from which continuous evolution is impossible as

$$\text{Out}_H = \{(q, x) \in Q \times X : \forall \epsilon > 0, \exists t \in [0, \epsilon), \Phi(t, q, x) \notin \text{Inv}(q)\}, \quad (2.2)$$

where $\Phi(t, q, x)$ is a solution trajectory following the dynamics of $\dot{x} = f(q, x)$.

In general, the exact computation of Reach_H and Out_H may be very complicated, but they can be calculated for some simple hybrid automata. To illustrate the calculation of Reach_H and Out_H , let's revisit the bouncing ball example.

Example 2.13 First, let's calculate Reach_H and Out_H for the hybrid automaton model for a bouncing ball. Note that $\text{Init} \subseteq \text{Reach}_H$, so we have

$$\{q_1\} \times \{x \in \mathbb{R}^2 \mid x_1 \geq 0, x_2 = 0\} \subseteq \text{Reach}_H.$$

On the other hand, we note that the position of the ball cannot be negative, so

$$\text{Reach}_H \subseteq \{q_1\} \times \{x \in \mathbb{R}^2 \mid x_1 \geq 0\}.$$

By checking the state trajectories, all states in the set $\{q_1\} \times \{x \in \mathbb{R}^2 \mid x_1 \geq 0\}$ are actually reachable (with proper initial conditions). So we conclude that

$$\text{Reach}_H = \{q_1\} \times \{x \in \mathbb{R}^2 \mid x_1 \geq 0\}.$$

Next, we calculate the set Out_H . First note that the continuous states outside of $\text{Inv}(q)$ cannot be reached under mode q , i.e., $\text{Out}_H \supseteq \bigcup_{q \in Q} (\{q\} \times \text{Inv}(q)^C)$, so

$$\{q_1\} \times \{x \in \mathbb{R}^2 \mid x_1 < 0\} \subseteq \text{Out}_H.$$

On the other hand, $\text{Out}_H \subseteq \text{Reach}_H^C$, so

$$\text{Out}_H \subseteq \{q_1\} \times \{x \in \mathbb{R}^2 \mid x_1 < 0\}.$$

Hence, we only need to check whether the states on the boundary, i.e., $\{q_1\} \times \{x \in \mathbb{R}^2 \mid x_1 = 0\}$, have continuous extensions under $\dot{x} = f(q_1, x)$ or not. For such a purpose, we further divide the boundary into two pieces, i.e., $B_1 = \{q_1\} \times \{x \in \mathbb{R}^2 \mid x_1 = 0, x_2 \geq 0\}$ and $B_2 = \{q_1\} \times \{x \in \mathbb{R}^2 \mid x_1 = 0, x_2 < 0\}$. It is easy to see that for states in B_1 , there exists a positive $\epsilon > 0$ such that for all $0 < t < \epsilon$, the continuous state flow $\Phi(t, q_1, x)$ following the dynamics $\dot{x}_1 = x_2, \dot{x}_2 = -g$, is contained in

$Inv(q_1) = \{x_1 \geq 0\}$. However, for states in B_2 , for all $\epsilon > 0$, there exists $t \in [0, \epsilon)$ such that $\Phi(t, q, x) \notin Inv(q_1)$. Hence, $B_1 \subseteq Out_H$ by definition. In conclusion, we have

$$Out_H = \{q_1\} \times (\{x \in \mathbb{R}^2 \mid x_1 < 0\} \cup \{x \in \mathbb{R}^2 \mid x_1 = 0, x_2 < 0\}).$$

□

The next example calculates the $Reach_H$ and Out_H for the temperature control system.

Example 2.14 For $Reach_H$, first, all initial states are reachable, so

$$Init = \{OFF\} \times \{x \leq 60\} \subseteq Reach_H.$$

Then from the state trajectory, we get

$$Reach_H = (\{OFF\} \times \{x \leq 60\}) \cup (\{ON\} \times \{x \leq 69\}) \cup (\{ON, OFF\} \times \{69 \leq x \leq 71\}).$$

For Out_H , note that $\bigcup_{q \in Q} (q \times Inv^c(q)) \subseteq Out_H$ so

$$(\{OFF\} \times \{x < 69\}) \cup (\{ON\} \times \{x > 71\}) \subseteq Out_H.$$

On the other hand, since $\{OFF\} \times \{x > 69\}$ and $\{ON\} \times \{x < 71\}$ are feasible,

$$Out_H \subseteq (\{OFF\} \times \{x \leq 69\}) \cup (\{ON\} \times \{x \geq 71\}).$$

After checking the boundary $\{OFF\} \times \{x = 69\}$ and $\{ON\} \times \{x = 71\}$, we get

$$Out_H = (\{OFF\} \times \{x \leq 69\}) \cup (\{ON\} \times \{x \geq 71\}).$$

□

Intuitively, a hybrid automaton is non-blocking if for all reachable states for which continuous evolution is impossible, there exists at least one possible discrete transition. This fact is stated in the following lemma.

Lemma 2.1 ([9]) *A hybrid automaton H is non-blocking, if for all states $(q, x) \in Reach_H \cap Out_H$, $\exists(q, q') \in E$ such that $x \in G(q, q')$.*

The lemma is very intuitive as it basically states that if a continuous evolution is impossible to continue further, then a discrete transition must be available. The conditions of the non-blocking result are tight, but not necessary unless the automaton is deterministic. If the conditions are violated, then there exists an execution that blocks. However, unless the automaton is deterministic, starting from the same initial state a non-blocking execution may also exist.

Example 2.15 Let's check the conditions in Lemma 2.1 to determine whether the hybrid automaton model for the bouncing ball is non-blocking. To do that, we first calculate the intersection of $Reach_H$ and Out_H , which is

$$Reach_H \cap Out_H = \{q_1\} \times \{x \in \mathbb{R}^2 \mid x_1 = 0, x_2 < 0\}.$$

Since $Reach_H \cap Out_H \subseteq \{q_1\} \times G(q_1, q_1)$, the conditions in Lemma 2.1 hold. Hence, the hybrid automaton model for the bouncing ball is non-blocking. $\square$

Next, we check the hybrid automaton model for the temperature control example.

Example 2.16 For the hybrid automaton model of the thermostat example, the intersection of $Reach_H$ and Out_H is given as

$$Reach_H \cap Out_H = (\{OFF\} \times \{x \leq 60\}) \cup (\{OFF\} \times \{x = 69\}) \cup (\{ON\} \times \{x = 71\}).$$

Note that $\{OFF\} \times (\{x \leq 60\} \cup \{x = 69\}) \subset \{OFF\} \times G(OFF, ON)$, where $G(OFF, ON) = \{x \leq 70\}$. Also, $\{ON\} \times \{x = 71\} \subseteq \{ON\} \times G(ON, OFF)$, where $G(ON, OFF) = \{x \geq 70\}$. Hence, the conditions in Lemma 2.1 hold, so we conclude that the hybrid automaton model of the thermostat example H is non-blocking. $\square$

Intuitively, a hybrid automaton may be non-deterministic if either there is a choice between continuous evolution and a discrete transition, or if a discrete transition can lead to multiple destinations (under the assumption, continuous evolution is unique). The following result provides a formal statement of this fact.

Lemma 2.2 ([9]) *A hybrid automaton H is deterministic, if and only if for state $(q, x) \in Reach_H$,*

1. *if $x \in G(q, q')$ for some $(q, q') \in E$, then $(q, x) \in Out_H$;*
2. *if $(q, q') \in E$, $(q, q'') \in E$, and $q' \neq q''$, then $x \notin G(q, q') \cap G(q, q'')$;*
3. *if $(q, q') \in E$ and $x \in G(q, q')$, then $R((q, q'), x)$ contains at most one element.*

Intuitively, the first condition indicates that once a discrete transition becomes valid then it must be fired; the second condition excludes non-deterministic choices in discrete transitions, while the third condition on the reset map requests that the states are uniquely updated after a discrete transition. The bouncing ball example is used to illustrate the lemma.

Example 2.17 To decide whether the hybrid automaton model for the bouncing ball is deterministic, we need to check the conditions of Lemma 2.2.

The first condition, “if $x \in G(q, q')$ for some $(q, q') \in E$, then $(q, x) \in Out_H$ ”, holds true since there is only one transition (q_1, q_1) , and $(\{q_1\} \times G(q_1, q_1)) \subseteq Out_H$ as $G(q_1, q_1) = \{x \in \mathbb{R}^2 \mid x_1 = 0, x_2 < 0\}$ while $Out_H = \{q_1\} \times (\{x \in \mathbb{R}^2 \mid x_1 < 0\} \cup \{x \in \mathbb{R}^2 \mid x_1 = 0, x_2 < 0\})$.

The second condition trivially holds since there is only one transition (q_1, q_1) in H .

The third condition also holds true since the possible state set under the reset map $R((q_1, q_1), x) = \begin{bmatrix} x_1 \\ -cx_2 \end{bmatrix}$ is a singleton.

In view of these, we can verify that the hybrid automaton model for the bouncing ball is deterministic. $\square$

It will now be shown that the hybrid automaton for the temperature control example is non-deterministic.

Example 2.18 Consider the state $\{OFF\} \times \{x = 69.5\} \in Reach_H$; we can verify that the first condition is not satisfied. So the hybrid automaton model in the thermostat example H is non-deterministic. The basic reason for being non-deterministic is due to the fact that the furnace can be turned off at any point if the temperature is above 70 but below 71. In other words, any transition that occurs in such a range generates a valid hybrid trajectory. So, there are infinitely many trajectories from any initial state. So, the hybrid automaton H is non-deterministic. $\square$

Combining the above two lemmas, a hybrid automaton H is well-posed (its solution exists and is unique for a given initial condition), if and only if the conditions in Lemmas 2.1 and 2.2 hold. Interested readers may refer to [9] for proofs. Put together, we can conclude that the hybrid automaton model for the bouncing ball is well-posed.

Hybrid automata are rich in expressiveness and are therefore quite suitable for modeling and simulating hybrid dynamical systems. However, due to the coexistence of heterogeneous discrete and continuous parts, they are not directly suitable for solving analysis and synthesis problems. In the following sections, we will introduce different kinds of hybrid models from the literature that are more amenable to computations.

2.3 Switched and Piecewise Affine Systems

In the previous section, hybrid automata models were introduced, which can be seen as extensions of finite automata to **include continuous variables and physical dynamics at each state**. This section discusses hybrid models derived by **extending classical physical dynamical models to include discrete variables and dynamics**. In particular, this section focuses on switched systems and also discusses a special class of switched systems, namely piecewise affine systems.

2.3.1 Switched Systems

Differential equations are widely used mathematical models for physical dynamical systems. As an example, we look at the dynamics of a wheeled robot.

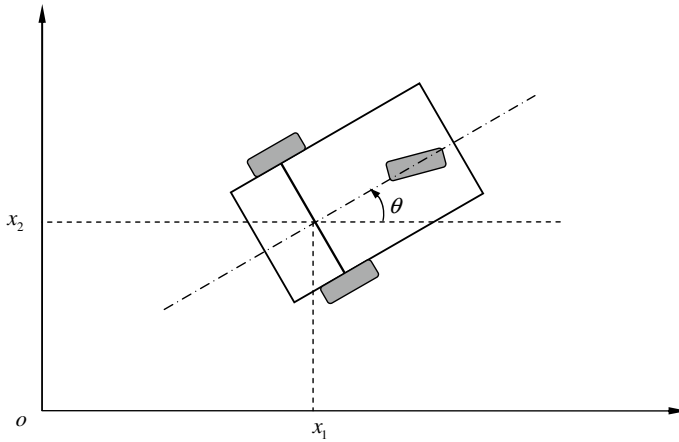


Fig. 2.18 Wheeled mobile robot example

Example 2.19 Consider a differential drive wheeled mobile robot as shown in Fig. 2.18. In this example, we use x_1 and x_2 to denote coordinates of the middle point of the rear axle, and θ stand for the angle of the vehicle with respect to the x_1 -axis. The kinematics of the robot can be modeled by the following differential equation:

$$\begin{aligned}\dot{x}_1(t) &= u_1 \cos \theta \\ \dot{x}_2(t) &= u_1 \sin \theta \\ \dot{\theta}(t) &= u_2,\end{aligned}\tag{2.3}$$

where u_1 and u_2 —the forward velocity and angular velocity, respectively—are the control input variables, while the positions x_1 , x_2 , and the direction of the vehicle θ

are the state variables. We usually put all state variables into a vector $x = \begin{bmatrix} x_1 \\ x_2 \\ \theta \end{bmatrix}$, and

call x the state vector, similarly for control inputs, $u = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$. The above differential equations can be written as a vector state equation,

$$\dot{x}(t) = f(x(t), u(t)),\tag{2.4}$$

where $f(\cdot, \cdot)$ is a map, from $\mathbb{R}^3 \times \mathbb{R}^2$ to $\mathbb{R}^3$ in our example. $\square$

A usual assumption on the vector field is the continuity of f , and this is indeed the case in our example. There is a long history and a huge literature of studying dynamical systems modeled by differential equations, using state equations with continuous vector fields, and many results can be found in the literature on the

existence and uniqueness of solutions, and on system properties such as stability and controllability. Later in this section, we review basic concepts of solutions of differential equations and we extend these concepts to hybrid systems. First of all, let's see an example to motivate why we need to extend our study to the case of differential state equations with discontinuous right-hand sides.

Example 2.20 Let's revisit the wheeled robot example. Our task is to park the vehicle at the origin. Namely, we need to design control signals u_1 and u_2 to drive the state variables x_1 , x_2 , and θ to zero. What makes the problem interesting is that it can be shown that there does not exist any time-invariant continuous state feedback control law to asymptotically stabilize this system, namely to drive all variables to zero asymptotically.

To see why, we first review an important result from the nonlinear control literature: A necessary condition for the existence of continuous state feedback control in the form of $u = k(x)$ to asymptotically stabilize a physical dynamical system $\dot{x} = f(x, u)$ at the origin¹ is that the image of every neighborhood of $(0, 0)$ in $\mathbb{R}^n \times \mathbb{R}^m$, under the map

$$(x, u) \rightarrow f(x, u),$$

contains some neighborhood of zero in $\mathbb{R}^n$. This is called Brockett's condition. Intuitively, it means that **starting near zero and applying small controls, we must be able to move in all directions**.

To show that the vehicle dynamics fail to satisfy Brockett's condition, consider the map

$$\begin{bmatrix} x_1 \\ x_2 \\ \theta \\ u_1 \\ u_2 \end{bmatrix} \rightarrow \begin{bmatrix} u_1 \cos \theta \\ u_1 \sin \theta \\ u_2 \end{bmatrix}. \quad (2.5)$$

Choose a neighborhood in the x, u space where $|\theta| < \frac{\pi}{2}$. The image of such a neighborhood under the above map does not contain a vector of the form $\begin{bmatrix} 0 \\ a \\ 0 \end{bmatrix}$, for $a \neq 0$.

To see this, note that since $\cos \theta \neq 0$ for $|\theta| < \frac{\pi}{2}$, to make $u_1 \cos \theta = 0$ we need to set $u_1 = 0$. This implies that $u_1 \sin \theta = 0$ as well. Thus, **for a small value of the angle θ we cannot move in all directions**. This is intuitively true as the wheels are not allowed to slip.

Hence, we have to resort to discontinuous control laws for our parking problem if we would like to design a state feedback controller—there are many advantages to using state feedback controllers. It turns out that **one may use switching between two continuous state feedback controllers to achieve the task**. $\square$

¹ It means that the closed-loop system $\dot{x} = F(x)$ with $F(x) = f(x, k(x))$ is asymptotically stable at the origin.

Note that switching among different controllers is often used in the control literature and practice. Examples include gain scheduling, bang-bang control, fuzzy logic, and supervisory control in adaptive control.

Switched System Model

A *switched system* is a dynamical system that consists of a finite number of continuous-variable subsystems and a logical rule that orchestrates switching among these subsystems. Mathematically, these subsystems are usually described by a collection of indexed differential or difference equations:

$$\begin{cases} \dot{x}(t) = f(x(t), q(t)), \\ q(t^+) = \delta(x(t), q(t)), \end{cases} \quad (2.6)$$

with initial condition $\{q(t_0), x(t_0)\} \in \text{Init}$, where $x(t) \in \mathbb{R}^n$ is the continuous state vector, and $q(t) \in Q = \{q_1, q_2, \dots, q_N\}$ is the discrete state (mode). Here the notation t^+ stands for the immediate time instant after switching, namely $t^+ = \lim_{\delta \rightarrow 0, \delta > 0} t + \delta$.

The logical rule that orchestrates switching among these subsystems generates switching signals, which are usually described as classes of piecewise constant maps, $\sigma : \mathbb{R}_{\geq 0} \rightarrow Q$. Note that a piecewise constant signal $\sigma(t)$ can only have a finite number of discontinuities in any finite time interval of $\mathbb{R}_{\geq 0}^+$.

In general, a *switching signal* $\sigma(t)$ is a piecewise constant function mapping $t \in \mathbb{R}_{\geq 0}$ to Q . The particular value of $\sigma(t)$ at an instant time t may depend on the time t , its own past history $\sigma(\tau)$ for $\tau \leq t$, the continuous state $x(t)$, and/or some external signals. Accordingly, we classify the switching signals into the following classes.

Autonomous Versus Controlled Switching

A switching is called *autonomous* if it does not depend upon external command switching signals. Otherwise, it is called controlled switching. An example to illustrate the difference between autonomous switching and controlled switching is that of the gearbox in automobiles. Autonomous switching occurs in cars with automatic transmission, where the gear ratio is changed automatically based on the vehicle's state (e.g., velocity and acceleration) but not by the driver's command. On the other hand, *controlled switching* occurs when driving a vehicle with manual transmission, where the switching between different gear ratios is triggered by the driver.

Time-Dependent Versus State-Dependent Switching

A switching signal is *time-dependent* if its value depends only on the current time. For example, a time periodic switching signal is time-dependent. On the other hand, a *state-dependent* switching signal depends on the continuous state values. For

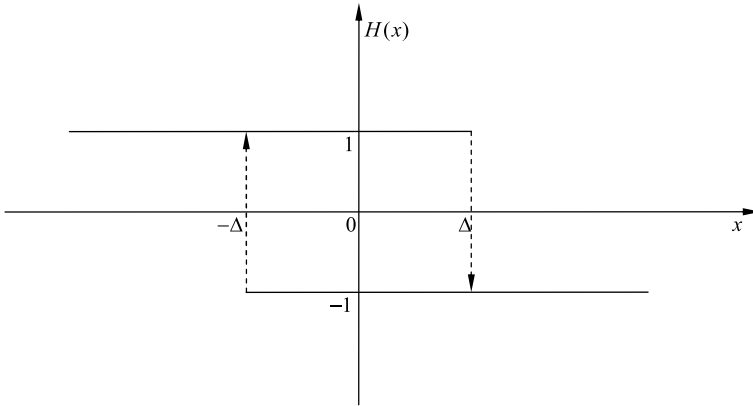


Fig. 2.19 A hysteresis function

example, when the continuous state $x(t)$ hits a switching surface $S_{q,q'}$, it will switch the discrete mode from q to q' . The next example illustrates a state-dependent switching.

Example 2.21 A dynamical system with hysteresis exhibits lag effects as its parameters and evolution depend not only on its current environment but also on its past history. Hysteresis occurs in many industrial, economic, and bio-molecular systems. A simple dynamical system with hysteresis can be represented by a differential equation, $\dot{x} = H(x)$, with a discontinuous $H(x)$ as shown in Fig. 2.19. If x is below $-\delta$, $H(x)$ takes the value of 1. If x increases its value and passes $-\delta$, $H(x)$ remains its positive one value until x is greater than δ . Once we further increase the value of x and pass the δ threshold, namely $x \geq \delta$, $H(x)$ jumps its value to -1 (as illustrated by the dashed downward arrow in Fig. 2.19) and remains there unless x drops below $-\delta$. Similar behavior can be observed if one decreases the value of x from above δ to below $-\delta$. In other words, $H(x)$ is a multi-valued function between $-\delta$ to δ , and its value depends on the history of x .

Dynamical systems with hysteresis can be modeled as switched systems with two discrete modes $Q = \{q_1, q_2\}$, and

$$f(x, q_1) = 1, f(x, q_2) = -1,$$

$$\delta(x, q_1) = \begin{cases} q_1, & x \leq \Delta, \\ q_2, & x \geq \Delta, \end{cases}$$

$$\delta(x, q_2) = \begin{cases} q_1, & x \leq -\Delta, \\ q_2, & x \geq -\Delta, \end{cases}$$

with the initial condition $Init = \{q_1, q_2\} \times \mathbb{R}$. □

Example 2.22 For the wheeled robot example, we now propose a multi-modal controller to stabilize the system. First, let us introduce the new variables:

$$\begin{aligned}
y_1 &= \theta, \\
y_2 &= x_1 \cos \theta + x_2 \sin \theta, \\
y_3 &= x_1 \sin \theta - x_2 \cos \theta,
\end{aligned}$$

and

$$\begin{aligned}
D_1 &= \{x \in R^3 : |x_3| > \frac{\|x\|}{2}\}, \\
D_2 &= \{x \in R^3 : x \notin D_1\}.
\end{aligned}$$

Next, define the following two candidate controllers:

$$u^1 = \begin{bmatrix} u_1^1 \\ u_2^1 \end{bmatrix} = \begin{bmatrix} -4y_2 - 6y_3/y_1 - y_3y_1 \\ -y_1 \end{bmatrix}, \quad (2.7)$$

$$u^2 = \begin{bmatrix} u_1^2 \\ u_2^2 \end{bmatrix} = \begin{bmatrix} -y_2 - \text{sgn}(y_2y_3)y_3 \\ -\text{sgn}(y_2y_3) \end{bmatrix}, \quad (2.8)$$

where $\text{sgn}(\cdot)$ is the signum function.

With these controllers, the closed-loop system can be represented as a switched system

$$\dot{x}(t) = f_\sigma(x(t)),$$

where $x = [x_1, x_2, \theta]^T$, $\sigma \in \{1, 2\}$, and

$$f_i(x) = \begin{bmatrix} u_1^i \cos(\theta) \\ u_1^i \sin(\theta) \\ u_2^i \end{bmatrix}, \quad i = 1, 2, \quad (2.9)$$

with switching law

$$\sigma(x) = \begin{cases} 1 & x \in D_1, \\ 2 & x \in D_2. \end{cases} \quad (2.10)$$

Note that it can be shown that the switched system is exponentially stable. $\square$

All definitions above can be extended to the discrete-time cases as well. As a very simple example to illustrate the discrete-time switched system, we consider an integrator with upper saturation as follows.

Example 2.23 Consider an integrator with upper saturation

$$x(k+1) = \begin{cases} x(k) + u(k), & \text{if } x(k) + u(k) \leq 1, \\ 1, & \text{if } x(k) + u(k) \geq 1. \end{cases} \quad (2.11)$$

It can be represented as a switched system

$$x(k+1) = f_\sigma(x(k), u(k)),$$

where $\sigma \in \{1, 2\}$, and

$$f_1(x, u) = x(k) + u(k), \quad f_2(x, u) = 1. \quad (2.12)$$

The switching law can be defined as below

$$\sigma(x) = \begin{cases} 1, & x \in D_1, \\ 2, & x \in D_2, \end{cases} \quad (2.13)$$

where $D_1 = \{x + u \leq 1\}$ and $D_2 = \{x + u \geq 1\}$. $\square$

Properties of switched systems have been studied for the past 50 years initially to consider engineering systems that contained relays and/or hysteresis. Recently, there has been increasing interest in the stability analysis and switching control design of switched systems; see, e.g., [11]. The primary motivation for studying such switched systems comes partly from the fact that switched systems and switched multi-controller systems have numerous applications in the control of mechanical systems, process control, automotive industry, power systems, aircraft and traffic control, and many other fields. In addition, there exists a class of nonlinear systems which can be stabilized by switching control, but cannot be stabilized by any continuous static state feedback control law [12].

Relationship to Hybrid Automata

We now explore the relationship between hybrid automata and switched systems. A switched system can be modeled as a hybrid automaton (see Definition 2.9). Specifically,

- $Q = \{q_1, q_2, \dots, q_N\}$ is the same;
- $X = \mathbb{R}^n$;
- $f = f(x(t), q(t))$ is the same;
- $Init \subseteq Q \times X$ is the same;
- Inv : for all $q \in Q$, $Inv(q) = \{x \in \mathbb{R}^n | q = \delta(x, q)\}$, i.e., all modes of dynamics are feasible on the whole state space;
- $E : (q, q') \in E$ when $q \neq q'$ and there exists $x \in X$ such that $q' = \delta(x, q)$;
- $G : \text{for } (q, q') \in E, G(q, q') = \{x \in \mathbb{R}^n | q' = \delta(x, q)\}$;
- R is the identity map, i.e., no state jumps.

For illustration, let's revisit the hysteresis example.

Example 2.24 A system with hysteresis can be modeled as a hybrid automaton. In particular, its representation as a hybrid automaton model is shown in Fig. 2.20. $\square$

As shown above, any switched system can be modeled as a hybrid automaton. On the other hand, any hybrid automaton without state jumps (i.e., the reset mapping R

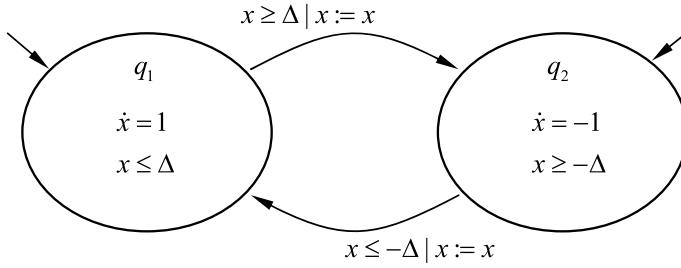


Fig. 2.20 A hybrid automaton model for a dynamical system with hysteresis

is identity for any discrete transition) can be modeled as a switched system with the same Q , X , f , $Init$, and

$$\delta(x, q) = \begin{cases} q, & x \in Inv(q), \\ q', & x \in G(q, q'), \end{cases}$$

as illustrated by the following example.

Example 2.25 The temperature control example can be seen as a switched system with

$$f(x, OFF) = -x, \quad f(x, ON) = -x + 100,$$

and the switching signal is determined by

$$\delta(x, OFF) = \begin{cases} OFF, & x \geq 69, \\ ON, & x \leq 70, \end{cases}$$

and

$$\delta(x, ON) = \begin{cases} OFF, & x \geq 70, \\ ON, & x \leq 71. \end{cases}$$

□

2.3.2 Piecewise Affine Systems

Piecewise affine systems are switched systems switching among a collection of linear differential or difference equations with **state space being partitioned by a finite number of linear hyperplanes**; see, e.g., [13, 14]. Mathematically, a piecewise affine system can be represented by

$$\dot{x}(t) = A_q x(t) + b_q, \quad \text{for } x \in \Omega_q, \quad (2.14)$$

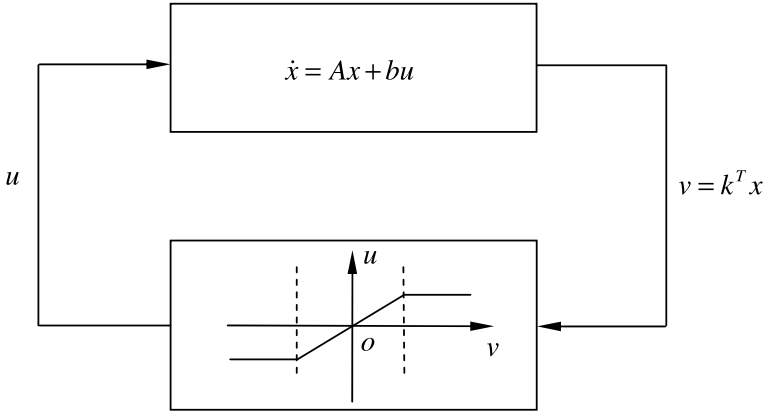


Fig. 2.21 A feedback control system with saturation

where $x(t) \in \mathbb{R}^n$ is the continuous-time state, $q \in Q$ is a finite index, and $\Omega_q \subseteq \mathbb{R}^n$. The sets Ω_q are assumed to provide a subdivision of the whole state space $\mathbb{R}^n$, that is $\bigcup_{q \in Q} \Omega_q = \mathbb{R}^n$ and Ω_q for $q \in Q$ are polyhedral and disjoint sets (only share common boundaries). Piecewise affine system models arise very often in practical applications. For example, piecewise affine systems can be used to model systems with discontinuous dynamics that arise because of saturation constraints, friction in mechanical systems, and so on.

Example 2.26 Consider a saturated linear system:

$$\dot{x} = Ax + b \cdot \text{sat}(v), \quad v = k^T x,$$

where the saturation function $\text{sat}(\cdot)$ is defined as

$$\text{sat}(v) = \begin{cases} -1, & v \leq -1, \\ v, & -1 < v \leq 1, \\ 1, & v > 1. \end{cases}$$

Graphically, the system can be represented by the block diagram shown in Fig. 2.21.

To describe this system as a piecewise affine system, we first divide the state space into three disjoint regions as follows:

- negative saturation region, $\Omega_1 = \{x \in \mathbb{R}^n | k^T x \leq -1\}$;
- linear operation region, $\Omega_2 = \{x \in \mathbb{R}^n | -1 \leq k^T x \leq 1\}$;
- positive saturation region, $\Omega_3 = \{x \in \mathbb{R}^n | k^T x \geq 1\}$.

It is clear that the three regions Ω_i are all polyhedral (i.e., can be described by a set of linear inequalities), for $i = 1, 2, 3$. Then, the saturated system can be modeled as the following piecewise affine system:

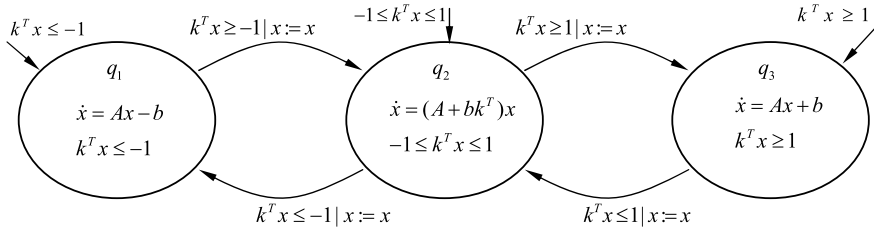


Fig. 2.22 A hybrid automaton model for the feedback control system with saturation

$$\dot{x} = \begin{cases} Ax - b, & x \in \Omega_1, \\ (A + bk^T)x, & x \in \Omega_2, \\ Ax + b, & x \in \Omega_3. \end{cases}$$

□

Piecewise affine systems can be seen as a special class of switched systems and therefore as a special class of hybrid automata. Any piecewise affine system can be written as a hybrid automaton where

- Q is the finite index set;
- $X = \mathbb{R}^n$;
- f : for all $q \in Q$, $f(q, x) = A_q x(t) + b_q$;
- $Inv(q) : \forall q \in Q, Inv(q) = \Omega_q$;
- $Init = \bigcup_{q \in Q} (\{q\} \times Inv(q))$;
- $E : (q, q') \in E$ when $q \neq q'$ and $\bar{\Omega}_q \cap \bar{\Omega}_{q'} \neq \emptyset$ (namely two partitions are adjacent);
- G : for any pair $(q, q') \in E$, $G(q, q') = \bar{\Omega}_{q'}$;
- R : for any pair $(q, q') \in E$, its reset map is the identity map, i.e., $R(\cdot, x) = \{x\}$.

For illustration, let's revisit the saturated control system example (Example 2.26).

Example 2.27 The saturated system example can be written in the form of a hybrid automaton as shown in Fig. 2.22. □

Furthermore, a piecewise affine system can be written as a switched system with

$$\dot{x}(t) = f(x, q) = A_q x(t) + b_q.$$

$$\delta(x, q) = \begin{cases} q, & x \in \Omega_q, \\ q', & x \in \Omega_{q'}. \end{cases}$$

Example 2.28 To model the saturated system in Example 2.26 as a switched system, let $\delta(x, q_2)$ be given by

$$\delta(x, q_2) = \begin{cases} q_1, & x \in \Omega_1, \\ q_2, & x \in \Omega_2, \\ q_3, & x \in \Omega_3. \end{cases}$$

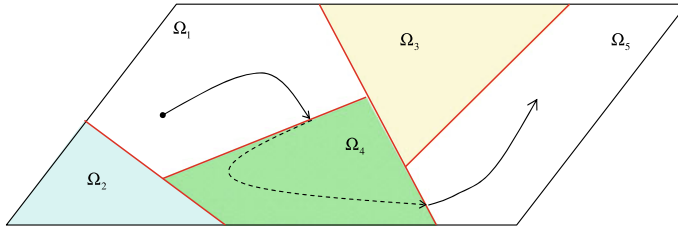


Fig. 2.23 Typical trajectory of a piecewise affine system

Note that $\delta(x, q_1)$ and $\delta(x, q_3)$ can be obtained in the same way. $\square$

A typical *trajectory* of a piecewise affine system is illustrated in Fig. 2.23. We observe that the trajectory consists of patched trajectory pieces concatenated together. A concatenating point stands for the occurrence of a switching, which is caused by the evolution of the continuous state $x(t)$ intersecting the boundary of Ω_q , e.g., the temperature being above some value.

2.3.3 Existence and Uniqueness of Solutions

To understand the behavior of a dynamical system, we study the trajectories its model generates starting from given initial conditions—this is called the initial value problem. Here, we study the state trajectories $x(t)$ that the switched or piecewise affine models can generate from any initial state $x(t_0)$.

Recall that in the previous section on hybrid automata, we introduced the definition of hybrid trajectories and discussed the existence of non-blocking and deterministic trajectories. These definitions and discussions are applicable to switched or piecewise affine systems, since we have shown that they both can be described by hybrid automata models.

Although necessary and sufficient conditions for the existence of a non-blocking and deterministic hybrid trajectory from a given initial condition for a hybrid automaton were given, these conditions are in general hard to check. We may gain more insight into the well-posedness of solutions if we focus on specific models; by *well-posedness*, we mean that a solution exists and it is unique. Therefore, in this subsection, we will focus on switched systems, and in particular piecewise affine systems, to investigate the well-posedness of their solutions.

Solutions of Ordinary Differential Equations

First, we present a brief review of the definition of solutions for differential equations as both switched and piecewise affine systems are generalizations of such equations.

Consider solutions of the following ordinary constant coefficient differential equation

$$\dot{x}(t) = f(x(t)), \quad (2.15)$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is a vector field. Let $x(t_0) = x_0$ be the initial condition. A solution of (2.15) on $[t_0, t_1]$, for some $t_1 > t_0$, is a continuously differentiable map $x : [t_0, t_1] \rightarrow \mathbb{R}^n$ that satisfies the Eq. (2.15). Without loss of generality, we assume that $t_0 = 0$.

It is known that if the vector field $f(x)$ is continuous and locally Lipschitz around the initial condition x_0 , then there exists a unique solution $x(t)$ from x_0 . The local Lipschitz condition means that for any x_1 and x_2 in some neighborhood of x_0 , we have

$$|f(x_1) - f(x_2)| \leq L|x_1 - x_2|,$$

for some constant $L > 0$. Here, $|\cdot|$ stands for the Euclidean norm on $\mathbb{R}^n$.

To illustrate this condition, we take a look at the following well-known example.

Example 2.29 Consider the following differential equation:

$$\dot{x}(t) = \sqrt{|x(t)|}, \quad (2.16)$$

with the initial condition $x(0) = 0$. Note that the vector field $f(x) = \sqrt{|x|}$ is continuous everywhere but not locally Lipschitz at the origin. It is easy to verify that starting from the initial condition $x_0 = 0$, the function $x(t) = 0$ for $t \geq 0$ satisfies the differential equation and the initial condition. However, the solution is not unique as the function $x(t) = t^2/4$ also satisfies the differential equation and the initial condition. Actually, it has infinitely many solutions starting from $x_0 = 0$, since the function

$$x_a(t) = \begin{cases} 0, & 0 \leq t \leq a, \\ \frac{(t-a)^2}{4}, & t \geq a, \end{cases} \quad (2.17)$$

satisfies the differential equation and the initial condition for arbitrary $a \geq 0$. $\square$

Switched systems or piecewise affine systems usually have a non-continuous right-hand side of the differential equation, i.e., $f(x)$ is not continuous since it usually needs to switch among different vector fields. Hence, they may not have solutions in the classical sense as the following example shows.

Example 2.30 Consider the following scalar system:

$$\dot{x}(t) = \begin{cases} 1, & x > 0, \\ \frac{1}{2}, & x = 0, \\ -1, & x < 0. \end{cases} \quad (2.18)$$

Starting from the initial condition $x_0 = 0$, no solution in a classical sense exists. To see this, we assume that there exists a continuously differentiable solution $x(t)$,

which satisfies the differential equation and initial condition x_0 . However, $\dot{x}(t) = 1$ for $t > 0$ while $\dot{x}(t) = \frac{1}{2}$ for $t = 0$, which is a contradiction with respect to the continuous differentiability of classical solutions. $\square$

Generalized Solutions

As shown in the previous example, solution concepts in the classical sense may not be adequate for switched or piecewise linear systems. The requirement to be continuously differentiable in the classical solutions is too strong as the trajectory may hit the switching surface and trigger a switching that makes the derivative of the solution become discontinuous. For differential equations with discontinuous right-hand side, such as in switched systems or piecewise affine systems, we need to generalize the solution concepts.

Definition 2.16 An absolutely continuous function $x : [t_0, t_1] \rightarrow \mathbb{R}^n$ is a solution of the differential equation (2.15) with an initial condition $x(t_0) = x_0$ in the Caratheodory sense if $x(t)$ satisfies the differential equation for almost all $t \in [t_0, t_1]$ and $x(t_0) = x_0$.

Alternatively, *Caratheodory solutions* are absolutely continuous curves that satisfy the integral version of the differential equation, namely

$$x(t) = x(t_0) + \int_{t_0}^t f(x(\tau))d\tau, \quad (2.19)$$

where $t > t_0$ and the integral is the Lebesgue integral. To illustrate the definition, we revisit the example.

Example 2.31 As we have shown above, the following scalar system does not have solutions in the classical sense starting from the origin.

$$\dot{x}(t) = \begin{cases} 1, & x > 0, \\ \frac{1}{2}, & x = 0, \\ -1, & x < 0. \end{cases} \quad (2.20)$$

However, it is easy to verify that $x : [0, \infty) \rightarrow \mathbb{R}$, where $x(t) = t$ is a solution in the Caratheodory sense as it satisfies the differential equation almost everywhere except at $t = 0$. $\square$

The Caratheodory solutions make sense when the crossing times are sufficiently few (mathematically speaking, the set of time t with discontinuous derivatives is of zero measure). However, it is not uncommon that this condition does not hold as the following example illustrates.

Example 2.32 Consider the following scalar system:

$$\dot{x}(t) = \begin{cases} -1, & x \geq 0, \\ 1, & x < 0. \end{cases} \quad (2.21)$$

Starting from the initial condition $x_0 = 0$, no solution exists either in a classical sense or in the Caratheodory sense. $\square$

If we take a further look at the above example, we find that the difficulty is caused by a sequence of arbitrarily fast switching between these two vector fields. Starting from $x_0 = 0$, the state x quickly becomes negative since its derivative is -1 at the origin. Hence a mode switching occurs and the vector field becomes positive, which pushes $x(t)$ back to the origin. Again, the vector field switches back to a negative one and the above process repeats. This is actually a Zeno phenomenon—an infinite number of switchings within a finite time interval—as we saw in the previous section on hybrid automata.

To fix this issue, we need to relax the restriction that the switching, which causes discontinuity in the solution's derivatives, only occurs on a set of time instants of measure zero. In our previous example, switching happened “densely” in time. Hence, another generalized solution concept—Filippov solution—needs to be introduced [15].

The main idea is to look at the vector field around a particular point, and allow the trajectory to follow any possible convex combination of the neighboring vector field vectors. Specifically, we form a set-valued map F to every point x in $\mathbb{R}^n$ as

$$F(x) = \bigcap_{\delta > 0} \bar{co}_{s \in B(x, \delta)} \{f(s)\}, \quad (2.22)$$

where $\bar{co}$ stands for the convex combination and $B(x, \delta)$ denotes the open ball with radius $\delta > 0$ centered at the point x . This is called Filippov map, and its corresponding *Filippov solution* is defined as follows.

Definition 2.17 An absolutely continuous function $x : [t_0, t_1] \rightarrow \mathbb{R}^n$ is a Filippov solution of the differential equation (2.15) from initial condition $x(t_0) = x_0$ if for almost all $t \in [t_0, t_1]$,

$$\dot{x}(t) \in F(x(t)),$$

and $x(t_0) = x_0$.

Let's revisit the previous example.

Example 2.33 Using the above Filippov map on the scalar system,

$$\dot{x}(t) = \begin{cases} -1, & x \geq 0, \\ 1, & x < 0. \end{cases} \quad (2.23)$$

We have $\dot{x}(t) = -1$ for $x > 0$, and $\dot{x}(t) = -1$ for $x < 0$. However, $\dot{x}(t) \in \bar{co}\{-1, 1\} = [-1, 1]$ for $x = 0$.

Then, the Filippov solution does exist from the initial condition $x_0 = 0$, and it is a constant zero. We can understand this Filippov solution through the following arguments.

In order for $x(t)$ to stay at the origin, we must have $\dot{x}(t) = 0$, i.e.,

$$\dot{x}(t) = \alpha_1(t)(-1) + \alpha_2(t)(+1) = 0,$$

which implies that $\alpha_1(t) = \alpha_2(t)$. In addition, we need to have $\alpha_1(t) + \alpha_2(t) = 1$. Hence, we have $\alpha_1(t) = \alpha_2(t) = \frac{1}{2}$. Hence, the unique dynamic on the sliding mode is

$$\dot{x}(t) = 0,$$

and $x(t) = 0$. □

Well-Posedness

After seeing the necessity for extending classical solutions to study solutions of switched and piecewise affine systems, we consider the well-posedness of these solutions in initial value problems.

Although all the definitions and notation for the solutions defined for hybrid automata are also applicable to switched systems or piecewise affine systems, these conditions for the existence and uniqueness of solutions for a general hybrid automaton are highly nontrivial. We would like to ask whether one could go further on the well-posedness of solutions for a switched system or a piecewise affine system. Unfortunately, the answer is negative in general unless for some special cases. Instead of trying to give a survey of theoretical results, we use examples to illustrate the difficulty of the problem.

For this, we check the “well-posedness” of a piecewise affine system. In particular, a piecewise affine system is said to be well-posed if for any initial state $x(t_0) = x_0$, there exists an $\varepsilon > 0$ such that there is a unique solution $x(t)$ satisfying $\dot{x}(t) = A_q x(t) + b_q$ with $x(t_0) = x_0$ and $x(t) \in \Omega_q$ for all $t \in [t_0, t_0 + \varepsilon)$. In other words, well-posedness refers to the local existence and uniqueness of solutions for the piecewise affine system starting from a given initial condition. If local uniqueness holds for all initial conditions and existence holds globally, then uniqueness must also hold globally since there is no point at which solutions can split. The following example from [16] shows that establishing well-posedness is nontrivial even for piecewise affine systems.

Example 2.34 Consider a planar piecewise affine system

$$\dot{x} = \begin{cases} \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} x + \begin{bmatrix} 0 \\ -1 \end{bmatrix}, & \text{if } y = \begin{bmatrix} 0 & 1 \end{bmatrix} x \leq 0; \\ \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} x + \begin{bmatrix} 0 \\ 1 \end{bmatrix}, & \text{if } y = \begin{bmatrix} 0 & 1 \end{bmatrix} x \geq 0, \end{cases}$$

where $x \in \mathbb{R}^2$ is the state.

Note that from any initial state $x(0) = (c, 0)^T$ with $|c| \leq 1$, there are two possible solutions. First, let's check

$$\begin{cases} x_1(t) = -1 + (c + 1) \cos t, \\ x_2(t) = -(c + 1) \sin t, \end{cases}$$

which satisfies the equation $\dot{x}_1 = x_2$, $\dot{x}_2 = -x_1 - 1$, and $\exists \varepsilon > 0$ such that $x_2(t) \leq 0$ for $0 \leq t < \varepsilon$. So it is a solution of the above piecewise affine system.

On the other hand, starting from the same initial condition, the following function

$$\begin{cases} x_1(t) = 1 + (c - 1) \cos t, \\ x_2(t) = -(c - 1) \sin t, \end{cases}$$

satisfies the equation $\dot{x}_1 = x_2$, $\dot{x}_2 = 1 - x_1$, and $\exists \varepsilon > 0$ such that $x_2(t) \geq 0$ for $0 \leq t < \varepsilon$. So it is also a solution of the above piecewise affine system. So the above piecewise affine system is not well-posed as a dynamical system. $\square$

Also, there are examples showing that the solutions of piecewise affine systems may not be continuous with respect to the initial conditions [16]. Namely, two trajectories starting from two arbitrarily close points could evolve far away from each other. Since piecewise affine systems are a special case of switched systems and hybrid automata, it implies that the well-posedness problem for general hybrid systems is nontrivial. In fact, it still remains an open problem.

2.4 Summary

In this chapter, we introduced several modeling frameworks for hybrid systems.

- Hybrid dynamical systems include both time-driven continuous and event-driven discrete dynamics, which, together with their interaction, give rise to interesting hybrid phenomena.
- We started with a brief review of finite automata as a modeling framework for DESs. It was shown that the finite automata are closed under Boolean operations, namely intersection, union, and complementation. It was also shown that finite automata accept regular languages exactly. Furthermore, for any non-deterministic finite automaton, there exists a language equivalent deterministic finite automaton.

- We then extended the finite automata models by including continuous dynamics described by differential equations to obtain a hybrid automata model. Hybrid automata models subsume finite automata models, and include state-space differential/difference equation models for continuous time-driven dynamics as special cases.
- We also introduced synchronizing events, based on which we can compose several hybrid automata into one hybrid automaton. This provides a manner to model a complicated hybrid system in a modular way.
- The existence and uniqueness of solutions in hybrid automata are highly nontrivial. Other interesting dynamical behaviors, such as discontinuity with respect to initial conditions and Zeno phenomena, could emerge and pose significant challenges for the study of hybrid systems.
- Switched systems are widely used in the control literature to model switching among several different continuous dynamical systems (called discrete modes). The switching from one mode to a new mode could depend on the continuous state and the current mode. Here, we introduced switched systems as a special case of hybrid automata models without state jumps and with a common continuous state space $\mathbb{R}^n$.
- Piecewise affine systems can be seen as a special case of switched systems, where all continuous dynamics are restricted to be linear and reside in a piecewise linear partition of the continuous state space $\mathbb{R}^n$. All switchings in a piecewise affine system are autonomous and are due to the state trajectory crossing (piecewise linear) boundaries that separate two different modes.
- Even for piecewise affine systems with just two modes separated by a linear plane, the well-posedness (existence and uniqueness) of solutions with respect to a given initial condition is challenging. This motivated us to introduce more general solution concepts, such as solutions in the Caratheodory or Filippov sense.

2.5 Notes and Further Reading

The results presented on finite automata and the formal language theory mainly follow [1], where interested readers may find the proofs of all stated results and more examples. Readers may refer to the book [4] for a comprehensive discussion of DESSs.

Hybrid automata models were first proposed for verification in the 1990s; see, e.g., [7, 17]; interested readers may refer to the survey [18] and its references where the approaches are from a computer science point of view. Our discussions of hybrid automata modeling, determinism, and non-blocking properties are based on [9]. Note that there are lecture notes available online on hybrid automata; see, e.g., [19].

It is possible to further generalize the hybrid automata models by introducing inputs and outputs; see, e.g., hybrid I/O automata [20]. In particular, continuous control inputs U can be introduced in the continuous dynamics for each mode q and discrete events Σ can be added to trigger discrete transitions $(q_i, q_j) \in E$. It

is also possible to include inputs to control the reset value of the reset map and consider set-valued mappings and differential or difference inclusions; see, e.g., [21, 22]. The extension of hybrid automata to handle probabilistic uncertainties gives the stochastic hybrid automata models [23–25], where the continuous-variable dynamics inside each invariant set of the discrete modes are described by stochastic differential equations and the discrete transitions also occur randomly.

There exist a number of software packages dealing with hybrid automata modeling and simulation, such as SHIFT [26], Ptolemy [27], and CHARON [28]. There also exist commercial modeling tools such as Stateflow (see www.mathworks.com) and Modelica (see www.modelica.org) that have been successfully used in industry.

An alternative modeling framework to automata with added continuous dynamics is a Petri net framework with added continuous dynamics. Significant work has been done in this area including the introduction of hybrid Petri nets, but these developments are not covered in this book; for details, see, e.g., [29–31] and the references therein.

Switched systems and piecewise linear systems are widely used to model dynamical systems that exhibit mode switching due to either external inputs or environmental changes, e.g., the slippage of a legged robot and the gear switch of a car. Piecewise affine systems, also called piecewise linear systems, have been widely used in the study of circuits, see, e.g., [32], since they can approximate nonlinear dynamics with arbitrary accuracy [33]. There also exist efficient computational techniques for the identification of piecewise affine models from input–output data [34, 35], such as clustering-based methods [36], mixed-integer programming [37], and Bayesian methods [38].

In this chapter, the description of switched and piecewise affine systems focused on systems that contained subsystems with continuous-time dynamics described by ordinary differential equations with constant coefficients. Such continuous-time descriptions were also used in the hybrid automata descriptions. It should be noted that the dynamics may also be discrete-time described, for example, by ordinary difference equations. In this case theoretical results, such as well-posedness of solutions, can be derived in a straightforward manner; see, e.g., [39–41]. Such models with discrete-time dynamics are used in later chapters of this book, in stability and optimal control.

For readers who are interested in learning more about continuous dynamical systems, there are many textbooks available for linear and nonlinear systems; see, e.g., [42, 43] where comprehensive descriptions of results and properties of dynamical systems described by ordinary differential or difference equations may be found.

Besides switched systems and piecewise affine systems, there are other modeling frameworks arising from the control and mathematics literature, attempting to capture the hybrid behavior of dynamical systems, such as autonomous or controlled switchings and state jumps. Examples include impulsive differential inclusions [21, 44, 45], complementarity systems [46], and mixed-logic dynamical systems [47]. We will delay our discussions on mixed-logic dynamical systems to Chap. 5 on optimal control because of their computational advantages. For the discrete-time case, it is shown that piecewise affine systems, mixed-logic dynamical systems, linear comple-

mentarity systems, and min-max-plus-scaling systems are actually equivalent [41], in the sense that one can be converted to the other under certain conditions. A software toolbox HYSDEL (see <http://people.ee.ethz.ch/~cohysys/hysdel/>) has been developed to automate the translation process between these modeling frameworks; see, e.g., [48].

Besides hybrid automata, switched systems, and piecewise affine system models, in the following chapters on verification and optimal control, we will see other modeling frameworks, such as transition systems (Sect. 3.1), mixed-logic dynamical systems (Sect. 5.5), timed automata (Sect. 3.5), rectangular automata (Sect. 3.5.5), and linear hybrid automata (Sect. 3.6). Their relationships with the models in this chapter, especially with hybrid automata, are discussed when the new models are introduced.

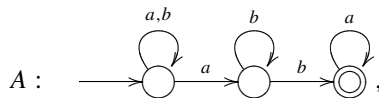
2.6 Exercises

1. **Simulations:** In this chapter, a number of running examples were used to illustrate different aspects of the theory. In particular,

- (a) Vending Machine (Examples 2.1, 2.5);
- (b) Light Switch (Examples 2.7, 2.8);
- (c) Temperature Control (Examples 2.9, 2.11, 2.16, 2.18, 2.25);
- (d) Bouncing Ball (Examples 2.10, 2.13, 2.15, 2.17);
- (e) Train-Gate Controller (Example 2.11);
- (f) Two-Tank System (Example 2.12);
- (g) Wheeled Robot (Examples 2.19, 2.20, 2.22);
- (h) System with Hysteresis (Examples 2.21, 2.24);
- (i) System with Saturation (Examples 2.23, 2.26, 2.27, 2.28);
- (j) Conditions for Solution (Examples 2.29–2.34).

It is strongly recommended to simulate each of these examples in this and later chapters so as to gain insight into their behaviors. The importance of simulations with the exploration of changes when certain system parameters change in switched and hybrid systems cannot be overstated. Commercial software, such as MATLAB and Stateflow, provides an excellent starting point for simulation work.

2. Given a finite automaton A as

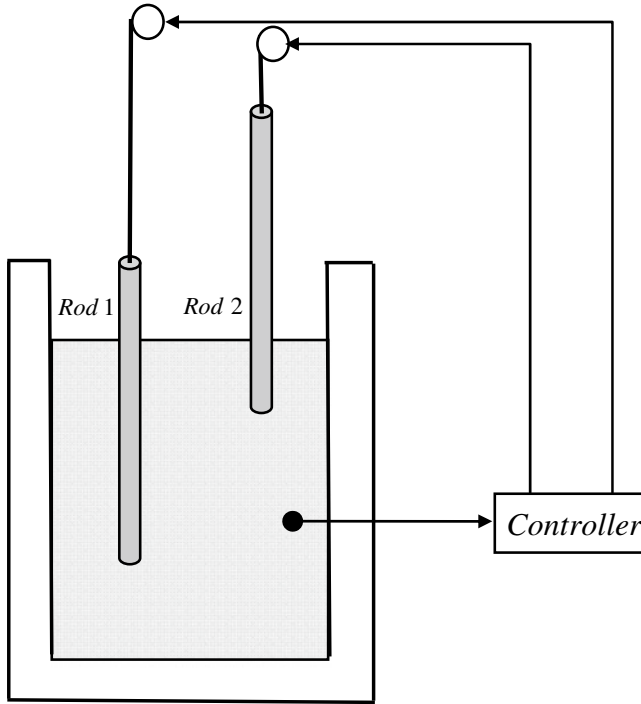


- (a) find a regular expression that is accepted by A ;
 - (b) show that A is non-blocking and non-deterministic;
 - (c) find a language equivalent deterministic finite automaton of A ;
 - (d) find a finite automaton A^c that accepts the complement of $\mathcal{L}(A)$, i.e., $\mathcal{L}(A^c) = \Sigma^* - \mathcal{L}(A)$.
3. Find a finite automaton A such that $\mathcal{L}(A) = \mathcal{L}(r)$ for a regular expression $r = (a + b)^*(ab)^*$.
 4. Show that the language $K = \{a^p, p \text{ is a prime number}\}$ on $\Sigma = \{a\}$ is non-regular.
 5. A water level in a tank is controlled by a monitor, which continuously senses the water level and turns a pump on and off. When the pump is off, the water level decreases at the rate of 2cm/s. When the pump is on, the water level increases by 1cm/s. It is required to keep the water level between 10 and 12 cm. The pump receives a signal from the monitor with some uncertain delay ranging from 0.1s to 0.5s. Assume that the tank is empty at the beginning.
 - (a) Model the system as a hybrid automaton.
 - (b) Show a typical trajectory of the hybrid automaton.
 - (c) Can it be modeled as a switched system and/or piecewise affine system? If yes, derive the model.
 - (d) Do computer simulations.
 6. Consider the following piecewise affine system:

$$\dot{x} = \begin{cases} A_1 x, & \text{if } x_1 \leq 0, \\ A_2 x, & \text{if } x_1 > 0, \end{cases}$$

$$\text{where } A_1 = \begin{bmatrix} -5 & -4 \\ -1 & -2 \end{bmatrix} \text{ and } A_2 = \begin{bmatrix} -2 & -4 \\ 20 & -2 \end{bmatrix}.$$

- (a) Model the system as a switched system.
 - (b) Model the system as a hybrid automaton.
 - (c) Simulate the behavior of the system using MATLAB and Stateflow.
7. A system to cool a nuclear reactor is composed of two independently moving rods. Initially, the coolant temperature x is 510° and both rods are outside the reactor core. The temperature inside the reactor increases according to the following (linearized) equation $\dot{x} = 0.1x - 50$.



When the temperature reaches 550° , the reactor must be cooled down using the rods. Three things can happen as follows:

- the first rod is put into the reactor core;
- the second rod is put into the reactor core;
- none of the rods are put into the reactor.

For mechanical reasons, a rod can be placed in the core if no rod has been there for at least 20 s. If no rod is available, the reactor should be shut down. The two rods can reduce the temperature of the coolant according to the following ODEs:

$$\text{Rod 1 : } \dot{x} = 0.1x - 56;$$

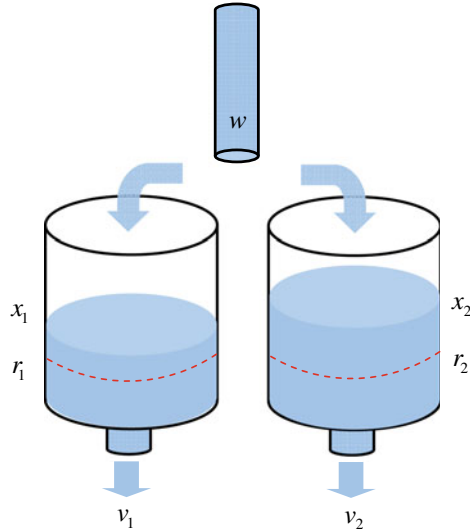
$$\text{Rod 2 : } \dot{x} = 0.1x - 60.$$

When the temperature is decreased to 510° , the rods are removed from the reactor core.

- (a) Model the system as a hybrid automaton.
- (b) Calculate the sets *Reach* and *Out*.
- (c) Determine whether the hybrid automaton model is non-blocking and deterministic.

(d) Do computer simulations.

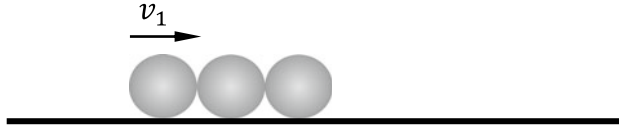
8. Let's revisit the two-tank example (also shown below). Both tanks are leaking and water is added through a hose (with constant flow rate w) to maintain the water levels above r_1 and r_2 , respectively. Note that the hose can serve only one tank at a time, either Tank 1 or Tank 2.



It is assumed that the leaking rates for both tanks are constant, and are v_1 and v_2 , respectively. Let x_i denote the level of water in Tank i , for $i = 1, 2$. Also assume that the initial water levels are above r_1 and r_2 , respectively.

To make sure that the water level of Tank i is above r_i for $i = 1, 2$, we employ the following control law: switching the inflow to Tank 1 whenever $x_1 \leq r_1$ and to Tank 2 whenever $x_2 \leq r_2$.

- It has been shown that the hybrid automaton model exhibits Zeno behavior and needs refinement. The task remaining for you is to calculate the length of execution time $|\mathcal{T}|$ (a.k.a. Zeno time or accumulating point).
 - There are delays between the sensor detecting the water level and the actuator turning on a valve. These delays are uncertain but bounded. Make appropriate assumptions if necessary and derive a new hybrid automaton model to capture the delay effects for the two-tank example. Does the new model still exhibit Zeno behavior?
 - Do computer simulations.
9. Consider three balls with unit mass. Suppose that they are touching at time $t = \tau_0$ as shown in the figure below.



The initial velocity of Ball 1 is $v_1(\tau_0) = 1(m/s)$, while Balls 2 and 3 are at rest, i.e., $v_2(\tau_0) = v_3(\tau_0) = 0$. Assume that the impact is a sequence of simple inelastic impacts occurring at $\tau'_0 = \tau'_1 = \tau'_2 = \dots$ (following the notion from hybrid time trajectory). The first inelastic collision occurs at τ'_0 between Balls 1 and 2, resulting in $v_1(\tau_1) = v_2(\tau_1) = 1/2$ and $v_3(\tau_1) = 0$. Then, Ball 2 hits Ball 3 instantaneously giving $v_1(\tau_2) = 1/2$ while $v_2(\tau_1) = v_3(\tau_1) = 1/4$. Now, Ball 1 is faster than Ball 2, so Ball 1 hits Ball 2 again and results in a new inelastic collision. This leads to an infinite sequence of collisions.

- (a) Model the inelastic collisions of the three-ball system described above as a hybrid automaton.
 - (b) Does the hybrid automaton model exhibit a Zeno execution? If so, what is the accumulation point?
10. Compute the accumulation point of the bouncing ball example.
 11. Consider a variant of the bouncing ball example, where we assume that the bounces are not instantaneous but require time $\epsilon > 0$.
 - (a) Derive a hybrid automaton model.
 - (b) Does the hybrid automaton model exhibit any Zeno execution?
 - (c) Simulate the automaton for different values of ϵ . What happens as $\epsilon \rightarrow 0$?
 12. Consider another variant of the bouncing ball example, where the ground is modeled as a stiff spring with spring constant $\xi > 0$ and no damping.
 - (a) Derive a hybrid automaton model.
 - (b) Does the hybrid automaton model exhibit any Zeno execution?
 - (c) Simulate the automaton for different values of ϵ . What happens as $\xi \rightarrow \infty$?
 13. Consider the following discontinuous differential equations:

$$\begin{cases} \dot{x}_1 = -\text{sgn}(x_1) + 2\text{sgn}(x_2), \\ \dot{x}_2 = -2\text{sgn}(x_1) - \text{sgn}(x_2), \end{cases}$$

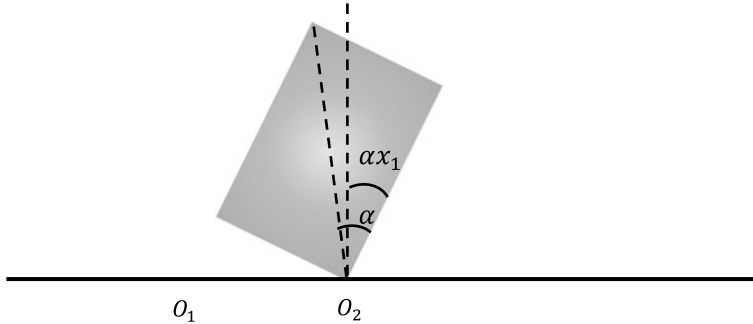
where

$$\text{sgn}(z) = \begin{cases} +1 & \text{if } z \geq 0, \\ -1 & \text{if } z < 0. \end{cases}$$

Assume $x(0) \neq 0$.

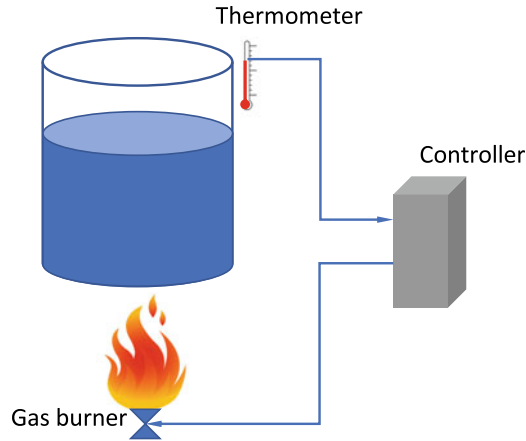
- (a) Obtain a hybrid automaton model for the above dynamical system.
- (b) Calculate the sets *Reach* and *Out*.

- (c) Determine whether the hybrid automaton model is non-blocking and deterministic.
 - (d) Does the hybrid automaton model exhibit any Zeno executions?
 - (e) Can it be modeled as a switched system and/or piecewise affine system? If yes, please derive the model.
 - (f) Do computer simulations.
14. The rocking motion of rigid bodies during earthquakes (as shown in the figure below) can be modeled as a hybrid automaton with two discrete modes—leaning to the left or leaning to the right.



We assume that the block does not slip, therefore, when leaning to the left it rotates about pivot O_1 and when leaning to the right it rotates about pivot O_2 . The continuous state of the system consists of the angle that the block makes with the vertical (measured here as the fraction of the angle made by the diagonal) and the angular velocity. We also assume that a fraction r , of the angular velocity, is lost every time the flat side hits the ground and the block switches from one pivot to the other.

- (a) Obtain a hybrid automaton model for the rocking block system.
 - (b) Calculate the sets *Reach* and *Out*.
 - (c) Determine whether the hybrid automaton model is non-blocking and deterministic.
 - (d) Does the hybrid automaton model exhibit any Zeno executions for every initial state?
 - (e) Do computer simulations.
15. Consider a digital temperature control system of a water heater as illustrated below.



The system is composed of

- a tank that contains water and that can be heated using a gas burner;
- a gas burner that can be turned on or turned off;
- a thermometer that monitors the temperature of the water inside the tank and periodically issues signals when the temperature of the water in the tank is above or below certain thresholds;
- a controller that observes the signals issued by the thermometer and issues orders to the gas burner in order to maintain the temperature of the water within a given range.

Answer the following questions:

- (a) build hybrid automata models for the water tank as well as the thermometer (with a sampling period of 0.1 s);
- (b) derive the product hybrid automaton of tank and thermometer;
- (c) determine whether the hybrid automaton model is non-blocking and deterministic;
- (d) can it be modeled as a switched system and/or piecewise affine system? If yes, please derive the model;
- (e) do computer simulations.

References

1. J. E. Hopcroft, R. Motwani, and J. D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Prentice Hall, third edition, 2006.
2. Michael Sipser. *Introduction to the Theory of Computation*, volume 2. Thomson Course Technology Boston, 2006.

3. P. Ramadge and W. M. Wonham. The control of discrete event systems. *Proceedings of the IEEE*, 77(1):81–98, 1989.
4. C. G. Cassandras and S. Lafortune. *Introduction to Discrete Event Systems (2nd Edition)*. Springer-Verlag, 2008.
5. E.A. Lee. Cyber physical systems: Design challenges. In *Object Oriented Real-Time Distributed Computing (ISORC)*, 2008 11th IEEE International Symposium on, pages 363–369, May 2008.
6. R. Baheti and H. Gill. Cyber-physical systems. *The Impact of Control Technology*, pages 161–166, 2011.
7. R. Alur, C. Courcoubetis, N. Halbwachs, T. Henzinger, P. H. Ho, X. Nicollin, A. Olivero, J. Sifakis, and S. Yovine. The algorithmic analysis of hybrid systems. *Theoretical Computer Science*, 138:3–34, 1995.
8. T. Henzinger. Hybrid automata with finite bisimulations. In Z. Füllöp and G. Gécseg, editors, *ICALP’95: Automata, Languages, and Programming*. Springer-Verlag, 1995.
9. J. Lygeros, K.H. Johansson, S.N. Simic, Jun Zhang, and S.S. Sastry. Dynamical properties of hybrid automata. *Automatic Control, IEEE Transactions on*, 48(1):2–17, 2003.
10. Karl Henrik Johansson, Magnus Egerstedt, John Lygeros, and Shankar Sastry. On the regularization of zeno hybrid automata. *Systems & Control Letters*, 38(3):141–150, 1999.
11. D. Liberzon. *Switching in Systems and Control*. Birkhauser, Boston, 2003.
12. R. W. Brockett. Asymptotic stability and feedback stabilization. In R. W. Brockett, R. S. Millman, and H. J. Sussmann, editors, *Differential Geometric Control Theory*, pages 181–191. Boston, MA: Birkhuser, 1983.
13. Eduardo Sontag. Nonlinear regulation: The piecewise linear approach. *Automatic Control, IEEE Transactions on*, 26(2):346–358, 1981.
14. M. Johansson. *Piecewise Linear Control Systems: A Computational Approach*, volume 284 of *Lecture Notes in Control and Information Sciences*. Springer-Verlag, 2003.
15. A. F. Filippov. *Differential Equations with Discontinuous Right Hand Sides*. Norwell, MA: Kluwer Academic, 1988.
16. J. Imura and A. van der Schaft. Characterization of well-posedness of piecewise-linear systems. *IEEE Trans. Automat. Contr.*, 45(9):1600–1619, 2000.
17. Rajeev Alur, Costas Courcoubetis, Thomas A. Henzinger, and Pei-Hsin Ho. Hybrid automata: An algorithmic approach to the specification and verification of hybrid systems. In Robert L. Grossman, Anil Nerode, Anders P. Ravn, and Hans Rischel, editors, *Hybrid Systems*, volume 736 of *Lecture Notes in Computer Science*, pages 209–229. Springer Berlin Heidelberg, 1993.
18. T. Henzinger. The theory of hybrid automata. In M.K. Inan and R.P. Kurshan, editors, *Verification of Digital and Hybrid Systems*, volume 170 of *NATO ASI Series*, pages 265–292. Springer Berlin Heidelberg, 2000.
19. J. Lygeros, S. Sastry, and C. Tomlin. *EE 291E Lecture notes: The Art of Hybrid Systems*. 2001.
20. N. Lynch, R. Segala, and F. Vaandrager. Hybrid I/O automata. *Information and Computation*, 185(1):103–157, 2003.
21. J. Aubin, J. Lygeros, M. Quincampoix, S. Sastry, and N. Seube. Impulse differential inclusions: A viability approach to hybrid systems. *IEEE Trans. Automat. Contr.*, 47(1):2–20, 2002.
22. R. Goebel, R. G. Sanfelice, and A. R. Teel. *Hybrid Dynamical Systems: modeling, stability, and robustness*. Princeton University Press, 2012.
23. Jianghai Hu, John Lygeros, and Shankar Sastry. Towards a theory of stochastic hybrid systems. In *Hybrid Systems: Computation and Control*, pages 160–173. Springer, 2000.
24. G. Pola, ML Bujorianu, J Lygeros, and MDD Benedetto. Stochastic hybrid models: An overview. In *Proc. IFAC Conf. Anal. Design Hybrid Syst*, pages 45–50, 2003.
25. Christos G Cassandras and John Lygeros. *Stochastic hybrid systems*. CRC Press, 2018.
26. Akash Deshpande, Aleks Göllü, and Pravin Varaiya. Shift: A formalism and a programming language for dynamic networks of hybrid automata. In Panos Antsaklis, Wolf Kohn, Anil Nerode, and Shankar Sastry, editors, *Hybrid Systems IV*, volume 1273 of *Lecture Notes in Computer Science*, pages 113–133. Springer Berlin Heidelberg, 1997.

27. Jie Liu, Xiaojun Liu, Tak-Kuen J.Koo, B. Sinopoli, S. Sastry, and E.A. Lee. A hierarchical hybrid system model and its simulation. In *Decision and Control, 1999. Proceedings of the 38th IEEE Conference on*, volume 4, pages 3508–3513 vol.4, 1999.
28. Rajeev Alur, Radu Grosu, Yerang Hur, Vijay Kumar, and Insup Lee. Modular specification of hybrid systems in charon. In *Hybrid Systems: Computation and Control*, pages 6–19. Springer, 2000.
29. Panos J Antsaklis and Anil Nerode. Hybrid control systems: An introductory discussion to the special issue. *IEEE Transactions on Automatic Control*, 43(4):457–460, 1998.
30. René David and Hassane Alla. *Discrete, continuous, and hybrid Petri nets*, volume 1. Springer, 2005.
31. Fabio Balduzzi, Alessandro Giua, and Giuseppe Menga. First-order hybrid petri nets: a model for optimization and control. *IEEE transactions on robotics and automation*, 16(4):382–399, 2000.
32. Domine MW Leenaerts and Wim MG Van Bokhoven. *Piecewise linear modeling and analysis*. Springer, 1998.
33. Ji-Nan Lin and Rolf Unbehauen. Canonical piecewise-linear approximations. *IEEE Transactions on circuits and systems. I: Fundamental theory and applications*, 39(8):697–699, 1992.
34. Simone Paoletti, Aleksandar Lj Juloski, Giancarlo Ferrari-Trecate, and René Vidal. Identification of hybrid systems a tutorial. *European journal of control*, 13(2-3):242–260, 2007.
35. Andrea Garulli, Simone Paoletti, and Antonio Vicino. A survey on switched and piecewise affine system identification. *IFAC Proceedings Volumes*, 45(16):344–355, 2012.
36. Giancarlo Ferrari-Trecate, Marco Muselli, Diego Liberati, and Manfred Morari. A clustering technique for the identification of piecewise affine systems. *Automatica*, 39(2):205–217, 2003.
37. Jacob Roll, Alberto Bemporad, and Lennart Ljung. Identification of piecewise affine systems via mixed-integer programming. *Automatica*, 40(1):37–50, 2004.
38. Aleksandar Lj Juloski, Siep Weiland, and WPMH Heemels. A bayesian approach to identification of hybrid systems. *Automatic Control, IEEE Transactions on*, 50(10):1520–1533, 2005.
39. Giancarlo Ferrari-Trecate, Francesco Alessandro Cuzzola, Domenico Mignone, and Manfred Morari. Analysis of discrete-time piecewise affine and hybrid systems. *Automatica*, 38(12):2139–2146, 2002.
40. Gang Feng. Stability analysis of piecewise discrete-time linear systems. *IEEE Transactions on Automatic Control*, 47(7):1108–1112, 2002.
41. Wilhemus PMH Heemels, Bart De Schutter, and Alberto Bemporad. Equivalence of hybrid dynamical models. *Automatica*, 37(7):1085–1091, 2001.
42. Panos J Antsaklis and Anthony N Michel. *Linear systems*. Springer, 2006.
43. Hassan K Khalil and Jessy W Grizzle. *Nonlinear systems*. Prentice hall Upper Saddle River, NJ, 2002.
44. Mouffak Benchohra, Johnny Henderson, and Sotiris Ntouyas. *Impulsive differential equations and inclusions*, volume 2. Hindawi Publishing Corporation New York, 2006.
45. Wassim M Haddad, VijaySekhar Chellaboina, and Sergey G Nersesov. *Impulsive and hybrid dynamical systems: stability, dissipativity, and control*. Princeton University Press, 2006.
46. Arjan J Van Der Schaft and Johannes Maria Schumacher. *An introduction to hybrid dynamical systems*, volume 251. Springer London, 2000.
47. A. Bemporad and M. Morari. Control of systems integrating logic, dynamics, and constraints. *Automatica*, 35(3):407–427, 1999.
48. Fabio Danilo Torrisi and Alberto Bemporad. Hysdel-a tool for generating computational hybrid models for analysis and synthesis problems. *Control Systems Technology, IEEE Transactions on*, 12(2):235–249, 2004.

Chapter 3

Formal Verification



The verification of real-time codes implemented in embedded systems is a very important problem, as many of these systems, such as autopilot systems and medical devices, are safety critical and need guarantees of their proper operations [1]. These embedded systems are interacting with the physical world, and thus continuous variables, such as time clocks, have to be taken into consideration. Hence, modeling such a system as a hybrid system becomes a natural choice. A typical question is whether certain properties, such as safety (e.g., bad things never happen), liveness (e.g., good things eventually happen), fairness (e.g., all nodes get a chance to transmit), and other properties, hold true or not for a given hybrid system model. Such problems are commonly known as formal verification problems. In this chapter, we will focus on the model checking approach in the formal verification literature since model checking can be fully automated and has been gaining a lot of attention from both academia and industry. Other approaches, such as deductive approaches, will be briefly mentioned at the end of this chapter.

This chapter starts with an introduction to temporal logic and model checking for finite transition systems since they form the basis for the formal verification of hybrid systems. Unfortunately, it is not straightforward to apply traditional formal verification methods, such as model checking [2], to hybrid systems since these methods were originally developed for discrete and finite state models of circuits and communication protocols and usually require an extensive search of all states that can be reached from the initial states. However, this is not possible as the states in hybrid systems are infinite and uncountable.

Motivated by this challenge, significant research activities have been devoted to the verification problems for hybrid systems. Two different approaches have been proposed in the literature. One is by getting an equivalent abstract transition system that has only a finite number of states. The other is symbolic model checking that represents the sets of states and operates on these sets of states in a symbolic manner rather than enumerating each state explicitly. We will introduce both approaches in

this chapter. As an alternative method to model checking, we will take a glance at the deductive methods in formal verification [3] at the end of this chapter as well.

This chapter is organized as follows: After introducing the definition of labeled transition systems in Sect. 3.1, we turn to the linear temporal logic (LTL) (Sect. 3.2) and computation tree logic (CTL) (Sect. 3.3) that are both interpreted over labeled transition systems. Both finite automata and hybrid automata can be seen as labeled transition systems. Model checking approaches for finite state transition systems with respect to LTL or CTL specifications are discussed in Sects. 3.2 and 3.3, respectively. We then extend the model checking approaches to finite state transition systems to the case of infinite state transition systems. The main idea behind the extension is to obtain an equivalent finite transition system, called an abstraction, for the infinite transition system under consideration. The equivalence is in the sense of bisimulation, which is formally defined in Sect. 3.4. Then, we consider the verification problem for hybrid systems. In particular, timed automata (Sect. 3.5) are introduced and their finite quotient transition systems are obtained. Unfortunately, it is known that even a slight generalization of timed automata could make the reachability problem undecidable [4], i.e., it becomes equivalent to some well-known undecidable problems, such as the halting problem [5]. We will therefore turn to a symbolic reachability analysis and symbolic model checking for linear hybrid automata in Sect. 3.6 based on affine formulas. Methods for the verification of more general hybrid systems, such as approximations of reachable sets or barrier certificates, are discussed in Sect. 3.7. More sophisticated techniques drawn from optimal control, game theory, and computational geometry have been developed in the literature to calculate or approximate the reachable sets for various classes of hybrid systems, and we will briefly review these efforts and available software tools at the end of this chapter.

3.1 Labeled Transition Systems

Model checking is a method to verify algorithmically whether a model, which is usually derived from a hardware or software design, satisfies certain properties. The properties that we are interested in are changing with respect to time. For example, whether the statement “the machine is busy” is true or not will vary in time and depend on the current state of the dynamical system. Sometimes, the statement is true, and sometimes the statement is false, but the statement is never true and false simultaneously. Also, one may be interested in checking whether a statement eventually becomes true or not, e.g., “proposal gets approved”. Formally, these properties can be formulated in temporal logic, and the model is expressed as a transition system.

3.1.1 Transition Systems

Transition systems are graph models that describe the evolution of the states under the action of transitions.

Definition 3.1 A transition system T is a four tuple $T = (S, S_0, U, \rightarrow)$ defined by

- a set of states S ;
- a set of initial states $S_0 \subseteq S$;
- a set of actions U ;
- a transition relation $\rightarrow \subseteq S \times U \times S$.

A transition system is called finite when the state set S and the action set U contain only a finite number of elements. Clearly, a finite automaton can be cast as a transition system with a finite number of states. A transition system may have an infinite number of states, and can be used to represent a large class of dynamical systems. For example, hybrid automata can be rewritten in the form of transition systems.

Example 3.1 A hybrid automaton, $H = \{Q, X, f, Init, Inv, E, G, R\}$, can be represented as a transition system $T_H = (S, S_0, U, \rightarrow)$, where

- $S = Q \times X$;
- $S_0 = Init$;
- $U = E \cup \mathbb{R}_{>0}$;
- $\{(q, x), u, (q', x')\} \in \rightarrow$ if one of the following conditions hold:
 - Discrete transition: When $u = (q, q')$, $x \in G(q, q')$ and $x' \in R(q, q', x)$.
 - Continuous transition: When $u \in \mathbb{R}_{>0}$, $q = q'$, and there exists a solution $(q, x(t)) \in Inv$ for $0 \leq t \leq u$ such that $\dot{x}(t) = f(q, x(t))$ for $0 \leq t \leq u$, and $x(0) = x, x(u) = x'$.

□

Hence, transition systems provide us with a very general framework for dynamical systems, more general than hybrid automata. The dynamical behavior of a transition system is conveniently described by the strings of its state evolution. Formally, we have the following definitions and notation.

Definition 3.2 A string $\alpha \in S^*$ ($\alpha \in S^\omega$) is a run of the transition system $T = (S, S_0, U, \rightarrow)$ if

- (1) $\alpha(1) \in S_0$;
- (2) there exists a string $\beta \in U^*$ ($\beta \in U^\omega$) such that $(\alpha(i), \beta(i), \alpha(i+1)) \in \rightarrow$, for $i = 1, \dots, |\alpha| - 1$ ($i \geq 1$ for $\beta \in U^\omega$).

Note that S^* stands for the collection of finite strings with all elements from S , while S^ω contains all strings of infinite length. Here $|\alpha|$ stands for the length of the run α , which potentially contains an infinite number of transitions, i.e., $|\alpha| \in \mathbb{N} \cup \{\omega\}$. Here, $\mathbb{N}$ denotes the set of natural numbers and ω stands for infinity. For $i < |\alpha|$, the i th state of α , written as $\alpha(i)$, is the state s_i reached after i transitions. A *complete execution* is a run which is maximal, that is, which cannot be extended. It is either infinite, or it ends at a state s_n out of which no transition is defined in which case we have a *deadlock*.

Consider a transition system $T = (S, S_0, U, \rightarrow)$. For a particular state $s \in S$ and an action $a \in U$, the set of successor states of s by the action a are given by

$$post_a(s) = \left\{ s' \in S \mid (s, a, s') \in \rightarrow \right\}. \quad (3.1)$$

The successor states of s in T for all possible actions are

$$post(s) = \bigcup_{a \in U} post_a(s). \quad (3.2)$$

Accordingly, the set of successor states for a set $P \subseteq S$ can be defined by

$$post(P) = \bigcup_{s \in P} post(s). \quad (3.3)$$

Similarly, we can define

$$pre_a(s) = \left\{ s' \in S \mid s \in post_a(s') \right\}, \quad (3.4)$$

$$pre(s) = \bigcup_{a \in U} pre_a(s), \quad (3.5)$$

and

$$pre(P) = \bigcup_{s \in P} pre(s). \quad (3.6)$$

Intuitively, $pre(P)$ describes the set of states that can be transited to a state in P within one step transition, while $post(P)$ contains the states that can be reached by a state in P within one step transition.

The set of states that are accessible from P in two transition steps can be characterized by $post(post(P))$, and denoted as $post^2(P)$. Inductively, one can denote the states that are accessible from P in $n \in \mathbb{N}$, transition steps as $post^n(P)$ that can be calculated recursively by

$$post^0(P) = P, \quad (3.7)$$

$$post^n(P) = post\left(post^{n-1}(P)\right). \quad (3.8)$$

Then, the states that can be accessible from P are the union of all $post^i(P)$ for $i \geq 0$, that is

$$post^*(P) = \bigcup_{i \in \mathbb{N}, i \geq 0} post^i(P). \quad (3.9)$$

In particular, $post^*(Q_0)$ is the set of reachable states for the transition system T and is denoted as $Reach(T)$. Similar definitions can be provided for $pre^n(P)$ and $pre^*(P)$.

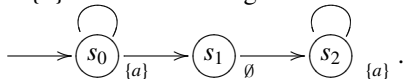
3.1.2 Labeled Transition System

Actions can be seen as inputs, and we can also introduce outputs for transition systems, which is called labels. To define labels, we associate the states of a transition system with properties that hold true for the corresponding states. The properties of interest are denoted as symbols p_i , say $p_1 = \text{“the machine is busy”}$, $p_2 = \text{“the machine is broken”}$, and so on. The collection of such symbols (assumed to be finite) forms a set, denoted as $\mathcal{P} = \{p_1, p_2, \dots\}$ and called an *atomic proposition* set. A labeled transition system is a transition system with all its states being labeled with true or false for all the atomic propositions in $\mathcal{P}$.

Definition 3.3 A *labeled transition system* is a tuple (T, l) , where $T = (S, S_0, U, \rightarrow)$ is a transition system and $l : S \rightarrow 2^{\mathcal{P}}$ (where $2^{\mathcal{P}}$ stands for the collection of all subsets of $\mathcal{P}$) is a label function that assigns each state s in T a subset of predicates $l(s) \subseteq \mathcal{P}$ satisfied by the state s .

Let's take a look at a simple example of labeled transition systems from [6].

Example 3.2 Consider $\mathcal{P} = \{a\}$ and the following labeled transition system (T, l) :



In this example, the state set of the transition system T is $\{s_0, s_1, s_2\}$, and the initial state set $S_0 = \{s_0\}$.

The action set U is not explicitly given here, and we simply use τ to represent all actions, i.e., $U = \{\tau\}$.

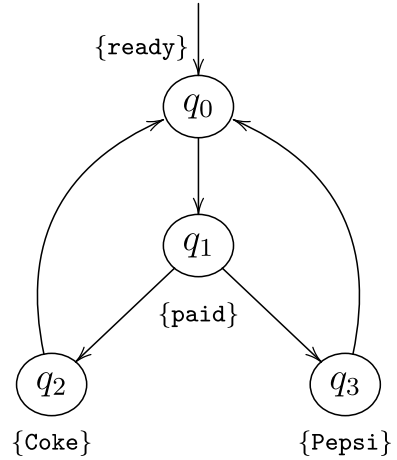
The transition relation $\rightarrow$ contains (s_0, τ, s_0) , (s_0, τ, s_1) , (s_1, τ, s_2) , and (s_2, τ, s_2) .

The labeling function $l : S \rightarrow 2^{\mathcal{P}}$ is defined as $l(s_0) = \{a\}$, $l(s_1) = \emptyset$, and $l(s_2) = \{a\}$. $\square$

Given a finite run α of the transition system T , we can define a *trace* generated from the labeled transition system (T, l) corresponding to the run α as a string $\gamma \in (2^{\mathcal{P}})^*$, where $\gamma(i) = l(\alpha(i))$. The collection of all finite traces that can be generated by the labeled transition system (T, l) is called the finite trace generated by (T, l) , denoted as $\mathcal{T}(T, l)$. Note that the above definitions can be extended to the case where α is of infinite length. Then, γ is an infinite trace as defined above, i.e., $\gamma \in (2^{\mathcal{P}})^\omega$, and the collection of all such infinite length traces is called the ω -trace (or simply trace) generated by (T, l) , denoted as $\mathcal{T}_\omega(T, l)$.

Example 3.3 For instance, the ω -trace generated by the labeled transition (T, l) in Example 3.2 contains an infinite number of repeated $\{a\}$, which is denoted as $\{a\}^\omega$, as it can stay in the state s_0 forever by following the self-loop on s_0 . Another infinite run in T could be described as follows. After a finite number (non-zero) of self-loops on s_0 , the system transitions to s_1 and s_2 , and then follows the self-loop on s_2 infinite times. The corresponding ω -traces can be represented as

Fig. 3.1 Transition model for the vending machine example



$$\{a\}^+ \cdot \emptyset \cdot \{a\}^\omega,$$

where $\cdot$ stands for the concatenation between strings. In total, we can write the ω -traces generated by (T, l) as

$$\mathcal{T}_\omega(T, l) = \{\{a\}^\omega, \{a\}^+ \cdot \emptyset \cdot \{a\}^\omega\}.$$

For simplicity, we usually drop the brackets $\{\}$ and concatenation $\cdot$, and write $\mathcal{T}_\omega(T, l)$ as $a^\omega + a^+\emptyset a^\omega$, which is also called an ω -regular expression.

On the other hand, the finite traces generated by (T, l) can be captured as

$$\mathcal{T}(T, l) = \{\{a\}^*, \{a\}^+\emptyset\{a\}^*\},$$

where $\{a\}^*$ represents a finite repeat of $\{a\}$. Succinctly, we say that $\mathcal{T}(T, l)$ is generated by a regular expression $a^* + a^+\emptyset a^*$. $\square$

Let's revisit the vending machine example from Chap. 2, but remodel it as a labeled transition system.

Example 3.4 Consider the vending machine example. We are interested in properties like “When the machine is ready, it can accept payment and then allow people to choose between Coke and Pepsi”. Hence, we choose the atomic proposition set $\mathcal{P}$ as $\{\text{ready}, \text{paid}, \text{Coke}, \text{Pepsi}\}$. Now the vending machine can be modeled as a labeled transition system as shown in Fig. 3.1.

For the vending machine labeled transition system model above, its ω -trace set $\mathcal{T}_\omega(T_1, l_1)$ is generated by the ω -regular expression $(\text{ready} \cdot \text{paid} \cdot (\text{Coke} + \text{Pepsi}))^\omega$. $\square$

It should be noted that the choice of the atomic proposition set $\mathcal{P}$ depends on what properties we are interested in. For instance, if we are just interested in the property

such as “Paid before getting drinks” for the above vending machine example, it would make sense to choose a new atomic proposition set $\mathcal{P}' = \{\text{paid}, \text{drink}\}$, and redefine the labeling function as $l'(q_0) = \emptyset$, $l'(q_1) = \{\text{paid}\}$, and $l'(q_2) = l'(q_3) = \{\text{drink}\}$.

In the next two sections, we will give a brief description of the model checking basics for finite transition systems (with respect to temporal logic properties). Our treatment mainly follows [6]. Interested readers may also refer to [2, 6] for a more comprehensive and detailed discussion on model checking.

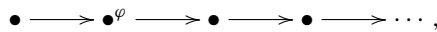
3.2 Linear Temporal Logic

We need to introduce a formal way to construct more complex expressions describing properties of a labeled transition system (T, l) , whose truth value can vary with respect to time. For such a purpose, temporal logic was proposed [7]. Temporal logic is a formalism for describing properties of sequences of states as well as tree structures of states. There are many variations of temporal logic, and we will first introduce LTL and then CTL. Their relationship will be demonstrated using examples.

3.2.1 Linear Temporal Logic

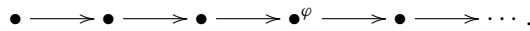
LTL is an extension of propositional logic geared to reasoning about infinite sequences of states. Formulas of LTL are built from a set of atomic propositions, like “it is raining”, and are closed under the application of Boolean connectives, such as conjunction, disjunction, and negation, and temporal operators. In particular, the following *temporal operators* are used for describing the properties along a specific run of a transition system¹:

- $\circ$ (“next state”): It requires that a property holds in the next state of the path. Let’s use φ to denote the property of interest, then $\circ\varphi$ can be illustrated as



which means that the property φ holds true at the next state along the path starting from the current state. The rest of states on the path remain unmarked since their labeling won’t change the truth value of the formula $\circ\varphi$.

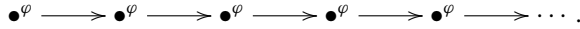
- $\Diamond$ (“eventually”): It is used to assert that a property will hold at some future state on the path. For example, the expression $\Diamond\varphi$ can be illustrated as



¹ We use run, path and trace interchangeably in this section.

This means that the property φ must become true at some point in the future along the path although we do not know when it will happen. For the same reason mentioned in the previous case, other states along the path remain unlabeled.

- $\Box$ (“always”): It specifies that a property holds at every state on the path. For example, $\Box\varphi$ can be illustrated as



Note that the property φ needs to hold true for all states along the path.

- $\sqcup$ (“until”): It is used to combine two properties. The combined property holds if there is a state on the path where the second property holds, and at every preceding state on the path, the first property holds. For example, the expression $\varphi_1 \sqcup \varphi_2$ can be illustrated as



Similar to the “eventually” operator $\Diamond$, the second property φ_2 needs to become true within a finite number of steps in the future (although we do not know when it will happen). In addition, the first property φ_1 needs to hold true before φ_2 becomes true. If the property φ_2 holds true at the initial (current) state, then it does not matter whether φ_1 is true or not at all. Namely, the following path satisfies $\varphi_1 \sqcup \varphi_2$ as well



We have the following relations among the above operators, where φ denotes a temporal logic specification: $\Diamond\varphi = \text{true} \sqcup \varphi$ and $\Box\varphi = \neg\Diamond\neg\varphi$. Therefore, one can just use $\circ$ and $\sqcup$ to express the rest of the temporal operators. These temporal operators can be nested with Boolean connectives to generate more complex temporal logic specifications. Formally, LTL formulas can be defined as follows.

Syntax

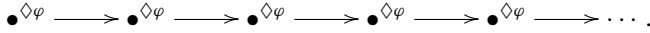
Definition 3.4 LTL formulas are recursively defined from predicates in $\mathcal{P}$ according to the following rules:

- (1) true , false , and p_i are LTL formulas for all $p_i \in \mathcal{P}$;
- (2) if φ_1 and φ_2 are LTL formulas, then $\varphi_1 \wedge \varphi_2$ and $\neg\varphi_1$ are LTL formulas;
- (3) if φ_1 and φ_2 are LTL formulas, then $\circ\varphi_1$ and $\varphi_1 \sqcup \varphi_2$ are LTL formulas.

Note that we can define other Boolean operators based on $\wedge$ and $\neg$, for example, $\varphi_1 \vee \varphi_2$ is equivalent to $\neg(\neg\varphi_1 \wedge \neg\varphi_2)$. As another example, the implication $\varphi_1 \Rightarrow \varphi_2$ can be represented equivalently as $\neg\varphi_1 \vee \varphi_2$. Similarly, other temporal operators, such as $\Diamond$ and $\Box$, can be represented in LTL as well. Let’s consider a couple of examples to illustrate the syntax of LTL formulas.

Example 3.5 A simple safety property, such as “no collision”, can be expressed as an LTL formula $\Box \neg \text{collision}$. Similarly, $\Diamond \text{finish}$ expresses a simple reachability property. $\square$

Example 3.6 The LTL formula $\Box \Diamond \varphi$ is true for traces (generated from a labeled transition system) that satisfy φ infinitely often, e.g., $\Box \Diamond \text{hungry}$, which is a liveness requirement. This can be understood as follows. For a trace to satisfy $\Box \Diamond \varphi$, all states along the trace should satisfy $\Diamond \varphi$. This is illustrate as follows:

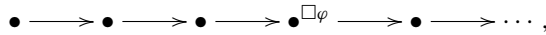


We also know that to satisfy $\Diamond \varphi$, the property φ must hold true in the finite future along the trace. Therefore, φ must hold true for an infinite number of times along the trace. Otherwise, if φ no longer hold true after N steps, then $\Diamond \varphi$ is violated for all states after N on the trace, which contradicts the fact that $\Diamond \varphi$ should hold true for all states on the trace.

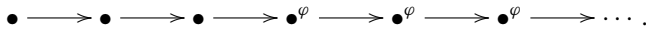
Therefore, a fairness requirement, say serve both customer 1 and customer 2 infinitely often, can be expressed as an LTL formula $(\Box \Diamond \text{serve1}) \wedge (\Box \Diamond \text{serve2})$. $\square$

Note that if we change the order of temporal operators, it may mean quite different things.

Example 3.7 For instance, the formula $\Diamond \Box \varphi$ means that φ will become true eventually and hold forever, e.g., $\Diamond \Box \text{battery dies}$. To see this, we illustrate a trace that satisfies $\Diamond \Box \varphi$



which requires that $\Box \varphi$ becomes true in a finite future. To satisfy $\Box \varphi$, it requires that φ holds true for all states on this trace afterwards, namely



This means that φ will become true eventually and hold forever. $\square$

Let's see a more complicated example.

Example 3.8 To specify the behavior of the traffic lights, we define a set of atomic propositions $\mathcal{P} = \{\text{red}, \text{green}, \text{yellow}\}$. The specification

$$\varphi = \Box(\text{green} \rightarrow \circ(\text{yellow} \sqcup (\text{yellow} \wedge \circ(\text{yellow} \sqcup \text{red}))))$$

describes the traffic light behavior. It basically requests that if the traffic light is green, it cannot immediately become red and has to be in yellow for a while. $\square$

Semantics

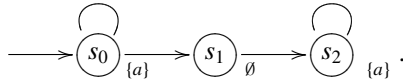
An LTL formula φ is interpreted over infinite sequences of sets of propositions, called a word $s = \mathcal{P}_1 \mathcal{P}_2 \mathcal{P}_3 \dots \in (2^{\mathcal{P}})^\omega$, where each $\mathcal{P}_i$ is a subset of $\mathcal{P}$. The satisfaction of a formula φ at position $t \in \mathbb{N}$ can be defined as follows.

Definition 3.5 A word $s \in (2^{\mathcal{P}})^\omega$ satisfies an LTL formula φ at t denoted by $s(t) \models \varphi$ if the following hold:

- (1) if $\varphi = p$, then $s(t) \models \varphi$ if and only if (iff) $p \in s(t)$;
- (2) if $\varphi = \neg \varphi_1$, then $s(t) \models \varphi$ iff $s(t) \not\models \varphi_1$;
- (3) if $\varphi = \varphi_1 \wedge \varphi_2$, then $s(t) \models \varphi$ iff $s(t) \models \varphi_1$ and $s(t) \models \varphi_2$;
- (4) if $\varphi = \circ \varphi_1$, then $s(t) \models \varphi$ iff $s(t+1) \models \varphi_1$;
- (5) if $\varphi = \varphi_1 \sqcup \varphi_2$, then $s(t) \models \varphi$ iff $\exists t' \geq t$ such that, for all $k \in [t, t')$, $s(k) \models \varphi_1$ and $s(t') \models \varphi_2$.

A word $s \in (2^{\mathcal{P}})^\omega$ satisfies φ if and only if $s(1) \models \varphi$. LTL formulas can be evaluated over traces generated from a labeled transition system. A labeled transition system (T, l) satisfies an LTL formula φ , denoted as $(T, l) \models \varphi$, if all ω -traces generated by (T, l) satisfy φ . If we denote all words satisfying the LTL formula φ as $\mathcal{W}(\varphi)$, then $(T, l) \models \varphi$ if and only if all ω -traces generated from (T, l) are contained in $\mathcal{W}(\varphi)$, i.e., $\mathcal{T}_\omega(T, l) \subseteq \mathcal{W}(\varphi)$.

Example 3.9 [6] Let's revisit the labeled transition system (T, l) in Example 3.2.



It can be seen that (T, l) does not satisfy the LTL formula $\circ a$, since the following ω -trace $\{a\}\emptyset\{a\}^\omega$ is contained in $\mathcal{T}_\omega(T, l)$ but violates $\circ a$. Hence $(T, l) \not\models \circ a$.

The same ω -trace also violates the LTL formula $\Box a$, so $(T, l) \not\models \Box a$ either. However, all ω -traces in $\mathcal{T}_\omega(T, l)$ satisfy the LTL formula $\Diamond \Box a$ as a eventually holds true forever in (T, l) , i.e., $\mathcal{T}_\omega(T, l) \subseteq \mathcal{W}(\Diamond \Box a)$. Hence, we have $(T, l) \models \Diamond \Box a$. $\square$

Note that $(T, l) \not\models \varphi$ does not necessarily imply that $(T, l) \models \neg \varphi$. For instance, in the above example, $(T, l) \not\models \circ a$ but neither does (T, l) satisfy its negation $\neg(\circ a)$. This can be seen as the ω -trace $\{a\}^\omega$ in $\mathcal{T}_\omega(T, l)$ violates $\neg(\circ a)$, which is equivalent to $\circ(\neg a)$.

3.2.2 LTL Model Checking

The LTL model checking problem is to determine whether a given labeled transition system (T, l) satisfies an LTL formula φ . In this subsection, we briefly introduce the automata-theoretic approach for LTL model checking [8]. The basic idea is to reduce the model checking problem to an *inclusion problem* between automata,

which basically asks whether the language accepted by one automaton is a subset of the language accepted by another automaton. In particular, *Büchi automata* are employed. A Büchi automaton is an extension of a finite automaton to accept an infinite input sequence.

Büchi Automaton

Definition 3.6 A Büchi automaton B is a tuple $(Q, Q^0, \Sigma, \delta, F)$, where $F \subseteq Q$ is a set of final states. A string $\alpha \in Q^\omega$ is a run of B if there exists $\beta \in \Sigma^\omega$ such that

- (1) $\alpha(1) \in Q^0$;
- (2) $\alpha(i+1) \in \delta(\alpha(i), \beta(i))$, for all $i \in \mathbb{N}$;
- (3) there exists infinitely many $j \in \mathbb{N}$ such that $\alpha(j) \in F$.

The language recognized or accepted by B is the collection of all such β , called ω -language and denoted by $\mathcal{L}_\omega(B)$.

Intuitively, a run of a Büchi automaton (in case of a deterministic automaton, there is exactly one possible run) consists of infinitely many states in the set of marked states. It is known that the class of ω -language recognized by Büchi automata is characterized as ω -regular language.

Definition 3.7 An ω -regular expression R , over the set Σ , has the following form:

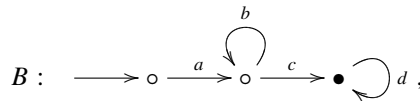
$$R = r_1 s_1^\omega + \dots + r_n s_n^\omega,$$

where $n \geq 1$ and r_i and s_i are regular expressions over Σ . Then, the ω -language corresponds to the expression R is

$$\mathcal{L}_\omega(R) = \mathcal{L}(r_1)\mathcal{L}(s_1)^\omega \cup \dots \cup \mathcal{L}(r_n)\mathcal{L}(s_n)^\omega,$$

where the term $(.)^\omega$ is defined as $K^\omega = \{t_1 t_2 t_3 \dots \mid t_i \in K, i \geq 1\}$, for $K \subseteq \Sigma^+ = \Sigma^* - \{\varepsilon\}$. In other words, K^ω consists of an infinite repetition of non-zero length strings.

Example 3.10 As a simple example, let's consider a Büchi automaton

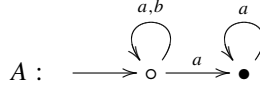


where the ω -language accepted by the above Büchi automaton B can be written into an ω -regular expression as ab^*cd^ω . In other words, $\mathcal{L}_\omega(B) = \mathcal{L}_\omega(ab^*cd^\omega)$, or we simply say that $\mathcal{L}_\omega(B) = ab^*cd^\omega$. $\square$

A language $L \subseteq \Sigma^\omega$ is called ω -regular if $L = \mathcal{L}_\omega(R)$ for some ω -regular expression R over Σ . It also, the class of ω -regular languages is closed under union, intersection, and complementation. However, different from finite automata, the class of ω -languages accepted by deterministic Büchi automata is a subclass of ω -regular language.

For example, the ω -regular language $L = (a + b)^*a^\omega$ over $\Sigma = \{a, b\}$ cannot be accepted by any deterministic Büchi automaton. To see why, let's show by contradiction. Assume that there exists a deterministic Büchi automaton B that accepts $L = (a + b)^*a^\omega$. First, consider a string $w_0 = a^\omega$. Since $w_0 = a^\omega \in L$, w_0 is accepted by B , and w_0 drives the states visiting F infinitely often. Hence, there exists a finite prefix of w_0 , denoted as u_0 , that drives the initial state into F . Now, consider another string $w_1 = u_0ba^\omega$. Again, $w_1 \in L$ and is accepted by B , so we can find a finite sub-string after u_0b that drives the state in B back into F again. Denote such a sub-string as u_1 . Similarly, we can find finite sub-strings u_i such that $u_0bu_1bu_2 \cdots bu_i$ drives the initial state of B into F . This process can continue for any large integer i . However, on the other hand, B only has finite states, so is F . Hence, there must exist j with $j \neq i$ and $j > i$ such that the strings $u_0bu_1bu_2b \cdots u_i$ and $u_0bu_1bu_2b \cdots u_ib \cdots u_j$ both drive the initial state to the same state in F , i.e., there exists a loop. Hence, B also accepts the ω -word $s = u_0bu_1bu_2b \cdots u_i(b \cdots u_j)^\omega$. However, $s \notin L$ as b appears in s infinitely often. This is a contradiction.

On the other hand, this language is recognized by a two-state non-deterministic Büchi automaton, e.g.,



Therefore, the expressive power of deterministic Büchi automata is strictly less than non-deterministic Büchi automata (NBA).

Automata Theoretic Approach for LTL Model Checking

As it was noted at the beginning of this subsection, the LTL model checking problem can be converted to a language inclusion problem between two Büchi automata. The basic steps of such a conversion can be described as follows:

1. convert the labeled transition system (T, l) into a Büchi automaton B_T such that $\mathcal{L}_\omega(B_T) = \mathcal{T}_\omega(T, l)$;
2. translate the negation of the specification, i.e., $\neg\varphi$, into an equivalent Büchi automaton, denoted as $B_{\neg\varphi}$ such that $\mathcal{L}_\omega(B_{\neg\varphi}) = \mathcal{W}(\neg\varphi)$;
3. build a Büchi automaton B such that $\mathcal{L}_\omega(B) = \mathcal{L}_\omega(B_{\neg\varphi}) \cap \mathcal{L}_\omega(B_T)$;
4. check the emptiness of $\mathcal{L}_\omega(B)$, as $(T, l) \models \varphi$ if and only if $\mathcal{L}_\omega(B) = \emptyset$.

The rationale behind this procedure is the following simple argument. Since $\mathcal{L}_\omega(B) = \mathcal{L}_\omega(B_{\neg\varphi}) \cap \mathcal{L}_\omega(B_T)$, it represents all the runs in B_T that satisfy $\neg\varphi$, i.e., do not satisfy φ . Therefore, $\mathcal{L}_\omega(B) = \emptyset$ implies that there is no trace generated from (T, l) that violates the property φ . In other words, φ holds true for all traces generated

from (T, l) . In the following, we will discuss details of the above-mentioned model checking procedure step by step.

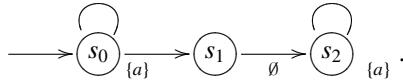
Converting a Label Transition System to a Büchi Automaton: Note that any labeled (finite) transition system can be converted into an equivalent Büchi automaton. The equivalence is in the sense that the traces generated by the labeled transition system coincide with languages accepted by the Büchi automaton. The conversion is straightforward and can be described as follows.

Given a labeled transition system (T, l) , with $T = (Q, Q_0, E, \rightarrow)$ and $l : Q \rightarrow 2^P$, it can be transformed into a Büchi automaton $B_T = (Q \cup \{t\}, \{t\}, \Sigma, \delta, Q \cup \{t\})$, where

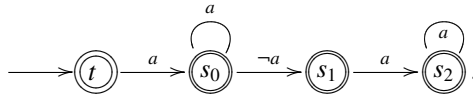
- $\Sigma = 2^P$;
- for any $q \in Q$, $q' \in \delta(q, \sigma)$ if and only if $\exists e \in E$ such that $(q, e, q') \in \rightarrow$ and $\sigma = l(q)$;
- in addition, $q \in \delta(t, \sigma)$ if and only if $q \in Q_0$ and $\sigma = l(q)$.

The following example illustrates the conversion process.

Example 3.11 Consider $P = \{a\}$ and the following labeled transition system (T, l) :



It can be converted to the Büchi automaton, B_T , as follows:

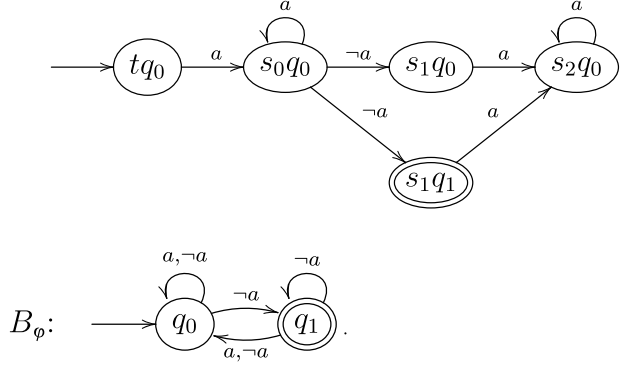


Note that the event set $\Sigma = 2^P = \{\{a\}, \emptyset\}$, and we denote $\{a\}$ as a and $\emptyset$ as $\neg a$ in the obtained Büchi automaton for simplicity. It can be verified that $\mathcal{L}_\omega(B_T) = \mathcal{T}_\omega(T, l)$. $\square$

Translate LTL Formula to Büchi Automata: Given an LTL formula φ , a Büchi automaton $B_\varphi = (Q, Q^0, \Sigma, \delta, F)$ is to be built, such that $\mathcal{L}_\omega(B_\varphi)$ is exactly the set of paths satisfying the formula φ , that is, $\alpha \models \varphi$ if and only if $\alpha \in \mathcal{L}_\omega(B_\varphi)$. The good news is that it is always possible. Namely, for any LTL formula, there exists such an equivalent Büchi automaton (but the reverse is not true). The basic idea of the translation is to use the collections of all sub-formulas of φ as the state of the Büchi automaton, and the state should contain exactly those sub-formulas that hold true for all runs starting from this state. The obtained Büchi automaton could be very large in the sense that the size of its states could be exponentially large compared with the length of φ . The procedures for this construction are presented in Appendix B. Instead, an example is given to illustrate the procedure.

Example 3.12 Consider the formula $\varphi = \Diamond \Box a$. Then $\neg \varphi = \neg \Diamond \Box a$ and $B_{\neg \varphi}$ is shown in Fig. 3.2 be as follows:

Fig. 3.2 The intersection of B_T and $B_{\neg\varphi}$



□

Intersection of Büchi Automata: The third step in LTL model checking is to obtain a Büchi automaton B such that $\mathcal{L}_{\omega}(B) = \mathcal{L}_{\omega}(B_{\neg\varphi}) \cap \mathcal{L}_{\omega}(B_T)$. To find the intersection of two Büchi automata $B_1 = (Q_1, Q_1^0, \Sigma, \delta_1, F_1)$ and $B_2 = (Q_2, Q_2^0, \Sigma, \delta_2, F_2)$, we need to construct a Büchi automaton that exactly accepts runs visiting both F_1 and F_2 infinitely often. A general case for the intersection of two Büchi automata is a bit complicated as a flag variable needs to be introduced to distinguish visits to F_1 from F_2 . Interested readers may refer to Appendix B for further details.

Our case is simpler as the Büchi automaton B_T , which is converted from the transition system, has all its states marked, namely the marked state set in B_T is Q_T itself. Hence, the intersection between $B_{\neg\varphi}$ and B_T can be obtained as

$$B_{\neg\varphi} \times B_T = (Q_{\neg\varphi} \times Q_T, Q_{\neg\varphi}^0 \times Q_T^0, \Sigma, \delta, F_{\neg\varphi} \times Q_T). \quad (3.10)$$

Moreover, $[q'_1, q'_2] \in \delta([q_1, q_2], \sigma)$ if and only if $q'_1 \in \delta_{\neg\varphi}(q_1, \sigma)$ and $q'_2 \in \delta_T(q_2, \sigma)$. Namely, the sets of state, initial state, and marked state are all given as a product, while the transition takes conjunctions. This is actually the same as we did for finite automata. We left as an exercise for readers to convince themselves why this works (when one Büchi automaton has all its states marked) and why we cannot do so for general Büchi automata (see Appendix B).

Example 3.13 Following Examples 3.11 and 3.12, we compute the intersection of B_T and $B_{\neg\varphi}$ as shown in Fig. 3.2. □

Checking the Emptiness of $\mathcal{L}_{\omega}(B)$: Checking the emptiness of a Büchi automaton B can be done as follows. Assume ρ is an accepting run of B , which means that ρ contains infinitely many states in F . Note that Q is finite, then there must exist strongly connected components² of B , and any strongly connected component that

² A strongly connected component C of a directed graph is a maximal subgraph such that every node in C is reachable from every other node in C along a directed path entirely contained within C .

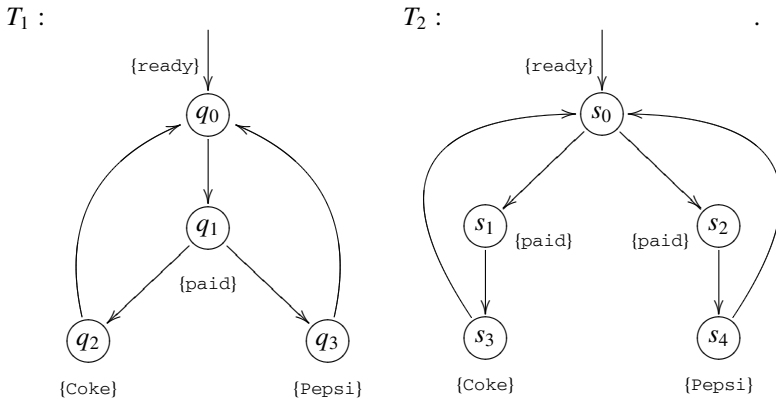
is reachable from an initial state and contains an accepting state may generate an accepting run of the automaton. Hence, checking the non-emptiness of $\mathcal{L}_\omega(B)$ is equivalent to finding a strongly connected component that is reachable from an initial state and contains at least one accepting state. Then, $\mathcal{L}_\omega(B)$ is empty if one cannot find such strongly connected components, i.e., a reachable accepting state with a cycle back to itself.

Example 3.14 The product Büchi automaton obtained in Example 3.13 accepts empty infinite runs (since the only marked state s_1q_1 does not belong to any strongly connected component), i.e., $\mathcal{L}_\omega(B_{\neg\varphi} \times B_T) = \mathcal{L}_\omega(B_{\neg\varphi}) \cap \mathcal{L}_\omega(B_T) = \emptyset$. Hence, we can conclude that $(T, l) \models \varphi$. $\square$

3.3 Computation Tree Logic

LTL is used to describe properties of sequences of states, and a transition system satisfies an LTL formula if all its paths (starting from initial states) satisfy the specification. However, in some applications, we may be interested in the existence of at least one successful/false path [9], for which we need to introduce path quantifiers (e.g., certain properties hold true for some but not all paths starting from a given state). To motivate the necessity of introducing path quantifiers in the temporal logic, let's revisit the vending machine example.

Example 3.15 Consider two labeled transition systems (T_1, l_1) and (T_2, l_2) over the atomic proposition set $\mathcal{P} = \{\text{ready}, \text{paid}, \text{Coke}, \text{Pepsi}\}$ as follows, which are used to model two vending machines:



For these labeled transition systems,

$$\mathcal{T}_\omega(T_1, l_1) = (\text{ready} \cdot \text{paid} \cdot (\text{Coke} + \text{Pepsi}))^\omega = \mathcal{T}_\omega(T_2, l_2).$$

It means that they are trace equivalent and therefore $(T_1, l_1) \equiv_{LTL} (T_2, l_2)$. In other words, for any LTL formula φ , if $(T_1, l_1) \models \varphi$ then $(T_2, l_2) \models \varphi$ and vice versa. Hence, they cannot be distinguished by LTL formulas (the same can be said for all linear time properties).

However, these two labeled transition systems behave quite differently— T_1 allows customers to choose either Pepsi or Coke after payment, while T_2 refuses to provide any options to customers and picks the next state non-deterministically by itself. $\square$

3.3.1 Computation Tree Logic

Motivated by the above example, we need to consider other kinds of temporal logic, other than LTL, that can help us to argue about properties such as the existence of choices in the future. For this, we introduce a class of branching time logic, called computation tree logic, which uses the following two *path quantifiers*:

- $\forall$: for all paths and
- $\exists$: for some paths

to describe the branching time structure starting at a particular state. These two quantifiers are used at a particular state to specify that all the paths or some of the paths starting from the state satisfy a certain property.

Syntax

Based on these quantifiers and temporal operators, we can define CTL formulas as shown below.

Definition 3.8 Let $\mathcal{P}$ be a finite set of atomic propositions. CTL formulas are state formulas recursively defined from atomic propositions in $\mathcal{P}$ according to the following rules:

- (1) true , false , and p_i are state formulas for all $p_i \in \mathcal{P}$;
- (2) if ϕ_1 and ϕ_2 are state formulas, then $\phi_1 \wedge \phi_2$ and $\neg\phi_1$ are state formulas;
- (3) if ϕ_1 and ϕ_2 are state formulas, then $\circ\phi_1$ and $\phi_1 \sqcup \phi_2$ are path formulas;
- (4) if φ is a path formula, then $\exists\varphi$ and $\forall\varphi$ are state formulas.

CTL uses atomic propositions as its building blocks to make statements about the states of a system. CTL then combines these propositions into formulas using logical operators and temporal operators. Similar to that in LTL, we can define

- $\exists\Diamond\phi = \exists[\text{true} \sqcup \phi]$;
- $\forall\Box\phi = \neg\exists\Diamond(\neg\phi)$;
- $\forall\Diamond\phi = \neg\exists\Box(\neg\phi)$;
- $\exists\Box\phi = \neg\forall\Diamond(\neg\phi)$.

Before we formally define the meaning (semantics) of CTL, we take a look at several examples of CTL formulas:

- $\exists\Diamond(\text{start} \wedge \neg\text{ready})$ means that it is possible to arrive at a state where `start` holds but not `ready`;
- $\forall\Box(\text{request} \implies \forall\Diamond\text{ack})$ stands for the property that once a request is made, it should be acknowledged eventually;
- $\forall\Box(\forall\Diamond\text{blink})$ means that everyone blinks infinitely often—all paths starting from the state in concern contain those states, from which all paths leading to a state that `blink` hold true;
- $\forall\Box(\exists\Diamond\text{recover})$ means that it is always possible to recover from an error—all paths starting from the state in concern contain those states, from which there exists a path leading to a state that `recover` holds true.

Semantics

CTL formulas are all state formulas, and are interpreted over states in a labeled transition system. They ask whether there exists a path (or for all paths), from the state, satisfying a certain property. Formally, we define the satisfaction of CTL formulas as follows.

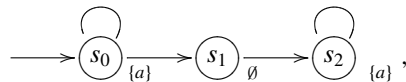
Definition 3.9 A state $s \in S$ in a labeled transition system (T, l) satisfies a CTL formula ϕ , denoted by $s \models \phi$ if the following hold:

- (1) if $\phi = p$, then $s \models \phi$ if and only if (iff) $p \in l(s)$ and $s \models \neg\phi$ iff $p \notin l(s)$;
- (2) if $\phi = \phi_1 \wedge \phi_2$, then $s \models \phi$ iff $s \models \phi_1$ and $s \models \phi_2$;
- (3) if $\phi = \exists \circ \phi_1$, then $s \models \phi$ iff there exists $s' \in \text{post}(s)$ and $s' \models \phi_1$;
- (4) if $\phi = \forall \circ \phi_1$, then $s \models \phi$ iff for all $s' \in \text{post}(s)$, it holds that $s' \models \phi_1$;
- (5) if $\phi = \exists\phi_1 \sqcup \phi_2$, then $s \models \phi$ iff there exists a string $\alpha \in S^\omega$ such that $\alpha(1) = s$, $\alpha(i) \rightarrow \alpha(i+1)$ for $i \in \mathbb{N}$, $\alpha(j) \models \phi_2$ for some $j \geq 1$, and $\alpha(i) \models \phi_1$ for all $1 \leq i < j$;
- (6) if $\phi = \forall\phi_1 \sqcup \phi_2$, then $s \models \phi$ iff for all strings $\alpha \in S^\omega$ such that $\alpha(1) = s$, $\alpha(i) \rightarrow \alpha(i+1)$ for $i \in \mathbb{N}$, $\alpha(j) \models \phi_2$ for some $j \geq 1$, and $\alpha(i) \models \phi_1$ for all $1 \leq i < j$.

For a CTL formula ϕ , we say that a labeled transition system (T, l) satisfies ϕ , denoted as $(T, l) \models \phi$, if and only if for all $s_0 \in S_0$, we have $s_0 \models \phi$.

Let's revisit the labeled transition system (T, l) in Example 3.2 to illustrate the semantics of CTL.

Example 3.16 Consider the labeled transition system (T, l) in Example 3.2



and a CTL formula $\exists(oa)$. It is obvious that the initial state s_0 in (T, l) satisfies the CTL formula $\exists(oa)$ as there exists $s' = s_0 \in \text{post}(s_0)$ with $a \in l(s_0)$. Therefore, $s_0 \models \exists(oa)$, and $(T, l) \models \exists(oa)$ as it holds for all initial states of (T, l) .

As another simple example, consider the CTL formula $\forall\Box a$. For a state to satisfy $\forall\Box a$, it is basically required all runs from the state a always to hold true. When this CTL formula is considered for all the initial states, it is equivalent to the LTL formula $\Box a$, and it is clear that $s_0 \not\models \forall\Box a$, and hence $(T, l) \not\models \forall\Box a$. $\square$

From Example 3.14, we already showed that (T, l) satisfies the LTL formula $\Diamond\Box a$. However, (T, l) does not satisfy the CTL formula $\forall\Diamond\forall\Box a$ although it looks very similar to the LTL formula $\Diamond\Box a$. To see why, note that the CTL formula $\forall\Diamond\forall\Box a$ requires all runs from the initial state s_0 to lead to a state that $\forall\Box a$ holds true. This is false as s_0 could stay at s_0 by self-looping, where $\forall\Box a$ is not satisfied. We will revisit this example in the next section after introducing an algorithm for CTL model checking.

Let's return to the motivating vending machine example.

Example 3.17 The two transition systems in Example 3.15 can be distinguished by a CTL formula $\Phi = \exists \circ (\exists \circ \text{Coke} \wedge \exists \circ \text{Pepsi})$, which basically asks whether it is possible to choose a product (Coke or Pepsi) after payment. In particular, we have $(T_1, l_1) \models \Phi$ but $(T_2, l_2) \not\models \Phi$. We will revisit this example and do the CTL model checking in the next subsection. $\square$

3.3.2 CTL Model Checking

Once such a computation tree logic formula has been specified, the next step is to make sure that the given labeled transition system satisfies the specification. Formally, the CTL model checking problem can be formulated as follows: Given a labeled transition system (T, l) and a CTL formula ϕ , determine if (T, l) satisfies ϕ .

The basic idea of CTL model checking is to mark each state s of T with the set of sub-formulas of ϕ which are true in the state s [2, 6]. Initially, this set is just $l(s)$. The marking process then goes through a series of stages. During the i th stage, sub-formulas with $i - 1$ nested CTL operators are processed. When a sub-formula is processed, it is added to the labeling of each state in which it is true. For convenience in notation, we denote the set of states satisfying a sub-formula ϕ_i as $Sat(\phi_i)$, which is a subset of S . Initially, we start with atomic propositions $p \in \mathcal{P}$ and have

$$Sat(p) = \{s \in S \mid p \in l(s)\}. \quad (3.11)$$

Then, we iteratively compute the set $Sat(\cdot)$ for more and more complicated sub-formulas. Once the algorithm terminates, we will have $(T, l) \models \phi$ if and only if $s_0 \in Sat(\phi)$ for all $s_0 \in S_0$.

Existential Normal Form

Before we proceed with CTL model checking, note that it can be shown that it is sufficient to use only three operators $\exists\circ$, $\exists\sqcup$, and $\exists\Box$; this is called *existential normal form* (ENF) of CTL. In other words, any CTL can be written into an ENF, since

- $\forall\circ\phi = \neg\exists\circ(\neg\phi)$;
- $\exists\Diamond\phi = \exists[\text{true} \sqcup \phi]$;
- $\forall\Box\phi = \neg\exists\Diamond(\neg\phi)$;
- $\forall\Diamond\phi = \neg\exists\Box(\neg\phi)$;
- $\forall[\phi_1 \sqcup \phi_2] = \neg\exists[\neg\phi_2 \sqcup (\neg\phi_1 \wedge \neg\phi_2)] \wedge \neg\exists\Box(\neg\phi_2)$.

Example 3.18 For example, the CTL formula $\forall\Diamond(\forall\Box a)$ can be written in ENF as

$$\begin{aligned}\forall\Diamond(\forall\Box a) &\equiv \forall\Diamond(\neg\exists(\text{true} \sqcup (\neg a))) \\ &\equiv \neg\exists\Box(\neg\neg\exists(\text{true} \sqcup (\neg a))) \\ &\equiv \neg\exists\Box(\exists(\text{true} \sqcup (\neg a))).\end{aligned}$$

□

Here the equivalence is in the following sense: Any labeled transition system that satisfies a CTL formula will also satisfy its equivalent CTL formula, and vice versa. Similar equivalence can be defined for LTL formulas or between LTL and CTL formulas.

CTL Model Checking Algorithms

Hence, without loss of generality, it is assumed that the CTL formula is given in ENF. Thus, for the intermediate stages of the algorithm, it is sufficient to be able to handle the following cases: ϕ is atomic or ϕ has forms $\neg\phi_1$, $\phi_1 \vee \phi_2$, $\exists\circ\phi_1$, $\exists[\phi_1 \sqcup \phi_2]$, and $\exists\Box\phi_1$. Then, we need to label the states based on the following rules:

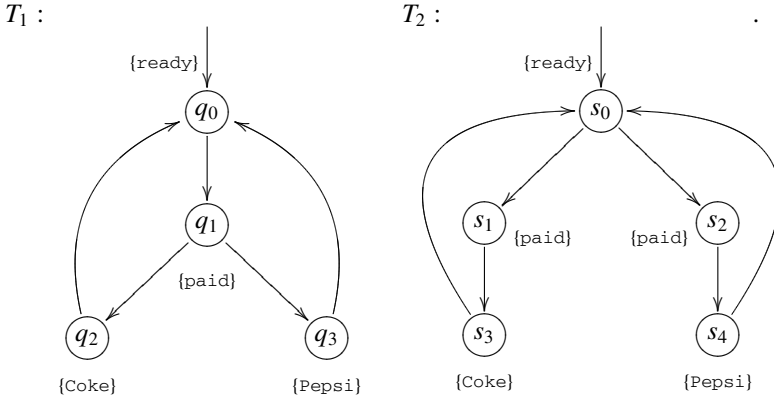
- for the case when ϕ is atomic, it has been handled by the labeling function $l(s)$;
- for formulas of the form $\neg\phi_1$, we label those states that are not labeled by ϕ_1 ;
- for $\phi_1 \vee \phi_2$, we label any state that is labeled either by ϕ_1 or by ϕ_2 ;
- for $\exists\circ\phi_1$, we label every state that has some successors that are labeled by ϕ_1 ;
- for a formula ϕ of the form $\exists[\phi_1 \sqcup \phi_2]$, we first find all states that are labeled by ϕ_2 , and then search backwards along a path till an initial state with all the states along the path are labeled with ϕ_1 . All such states should be labeled with ϕ ;
- the case that ϕ is of the form $\exists\Box\phi_1$ can be handled by decomposition of the graph into nontrivial strongly connected components. A strongly connected component C is a maximal subgraph such that every node in C is reachable from every other node in C along a directed path entirely contained within C . C is nontrivial if either it has more than one node or it contains one node with a self-loop. The following steps are taken:

- first, we obtain a sub-transition system T' with states in S' , which are labeled with ϕ_1 , by deleting the states where ϕ_1 is not labeled and all links to or from these states;
- then, we partition T' into strongly connected components;
- next, we find those states that belong to nontrivial components, and then work backwards to find all those states that can be reached by a path in which each state is labeled by ϕ_1 . All such states should be labeled with $\exists \Box \phi$.

In summary, in order to handle an arbitrary CTL formula ϕ , we successively apply the state labeling algorithm to the sub-formulas of ϕ , starting from the shortest, most deeply nested sub-formulas, and working onwards to include all sub-formulas of ϕ . By proceeding in this manner, we guarantee that whenever we process a sub-formula of ϕ , all of its sub-formulas have already been processed. It is known that the computation of CTL model checking is of polynomial complexity [2, 6].

Let's revisit the vending machine example to illustrate the CTL model checking procedure.

Example 3.19 Consider the transition system models for vending machine shown below. For convenience, we re-plot the transition systems as follows:



It has been claimed that the two transition systems can be distinguished by the CTL formula $\Phi = \exists \circ (\exists \circ \text{Coke} \wedge \exists \circ \text{Pepsi})$. To see this, we first check whether $(T_1, l_1) \models \Phi$ or not. Note that the CTL formula Φ is already in ENF. Following the CTL model checking procedure, we start from the most inside sub-formula to a more complicated formula iteratively:

- $Sat(\text{Coke}) = \{q_2\}, Sat(\text{Pepsi}) = \{q_3\};$
- $Sat(\exists \circ \text{Coke}) = \{q_1\}, Sat(\exists \circ \text{Pepsi}) = \{q_1\};$
- $Sat(\exists \circ \text{Coke} \wedge \exists \circ \text{Pepsi}) = \{q_1\};$
- $Sat(\exists \circ (\exists \circ \text{Coke} \wedge \exists \circ \text{Pepsi})) = \{q_0\}.$

Since $q_0 \in Sat(\Phi)$, we have $(T_1, l_1) \models \Phi$.

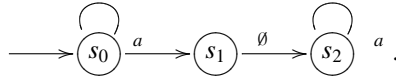
Let's turn to the second transition system (T_2, l_2) . Similarly, we obtain

- $Sat(Coke) = \{s_3\}, Sat(Pepsi) = \{s_4\};$
- $Sat(\exists \circ Coke) = \{s_1\}, Sat(\exists \circ Pepsi) = \{s_2\};$
- $Sat(\exists \circ Coke \wedge \exists \circ Pepsi) = \emptyset;$
- $Sat(\exists \circ (\exists \circ Coke \wedge \exists \circ Pepsi)) = \emptyset.$

Since $q_0 \notin Sat(\Phi)$, we have $(T_2, l_2) \not\models \Phi$. $\square$

As another example, let's revisit the labeled transition system (T, l) in Example 3.2.

Example 3.20 Consider the transition system model in Example 3.2 as follows.



We know that the labeled transition system satisfies the LTL formula $\varphi = \Diamond \Box a$ from Example 3.14. Now, we will check whether it satisfies the CTL formula $\Phi = \forall \Diamond \forall \Box a$.

First, we rewrite Φ in ENF as $\Phi = \neg(\exists \Box(\exists \Diamond(\neg a)))$. From sub-formula inside to the more complicated sub-formulas, we obtain

- $Sat(a) = \{s_0, s_2\};$
- $Sat(\neg a) = \{s_1\};$
- $Sat(\exists \Diamond \neg a) = \{s_0, s_1\};$
- $Sat(\exists \Box(\exists \Diamond \neg a)) = \{s_0\};$
- $Sat(\neg(\exists \Box(\exists \Diamond \neg a))) = \{s_1, s_2\}.$

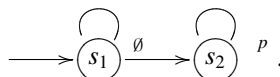
Hence, $s_0 \notin Sat(\Phi)$, so that the CTL formula $\Phi = \forall \Diamond \forall \Box a$ is not satisfied. $\square$

As shown by this example, CTL is different from LTL. In the sequel, we take a closer look at their differences.

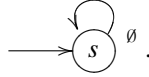
3.3.3 Comparison Between LTL and CTL

The difference between the semantics of LTL and CTL is that LTL formulas are interpreted over words, whereas CTL formulas are interpreted over the tree of trajectories generated from a given state of a transition system. CTL and LTL have different expressive power. For example, the CTL formula $\phi = \forall \Box \exists \Diamond p$ cannot be expressed by any LTL formula.

We show this claim by contradiction. Let's assume that there exists an LTL formula φ which is equivalent to the CTL formula $\phi = \forall \Box \exists \Diamond p$ in the sense that any labeled transition system which satisfies φ also satisfies ϕ and vice versa. Now, consider the transition system T as follows:



It can be verified that $(T, l) \models \phi$. Since it is assumed that ϕ and φ are equivalent, then, it can be concluded that $(T, l) \models \varphi$. As the LTL formula is a path formula, all of the paths starting from the initial state s should satisfy φ . In other words, we will have $\mathcal{T}_\omega(T, l) \subseteq \mathcal{W}(\varphi)$. Now consider another transition system T' :



It can be verified that $\mathcal{T}_\omega(T', l) \subseteq \mathcal{T}_\omega(T, l)$ and since $\mathcal{T}_\omega(T, l) \subseteq \mathcal{W}(\varphi)$, we will have $\mathcal{T}_\omega(T', l) \subseteq \mathcal{W}(\varphi)$ meaning that $(T', l) \models \varphi$. On the other hand since ϕ and φ are equivalent, this leads to $(T', l) \models \phi$ which is obviously not true.

Also, it can be shown that the LTL formula $\varphi = \Diamond \Box a$ cannot be expressed by any CTL formula. As we already know, the labeled transition system in Example 3.2 $(T, l) \models \Diamond \Box a$, but $(T, l) \not\models \forall \Diamond \forall \Box a$.

So, LTL and CTL are incomparable, but LTL and CTL do overlap, as it can be seen from the fact that the CTL formula $\forall \Box p$ is equivalent to the LTL formula $\Box p$. Both LTL and CTL are subsets of CTL*, which is a generalization of CTL by allowing Boolean combinations and nestings of temporal operators. This relationship is illustrated in Fig. 3.3. CTL* is also called full branching time logic because of its branching time structure, i.e., at each moment, there may exist alternate courses representing different possible futures. It was proposed as a unifying framework which subsumes both CTL and LTL, as well as a number of other logic systems. We will not give details for CTL* here. The interested readers may refer to [2, 6] for further details.

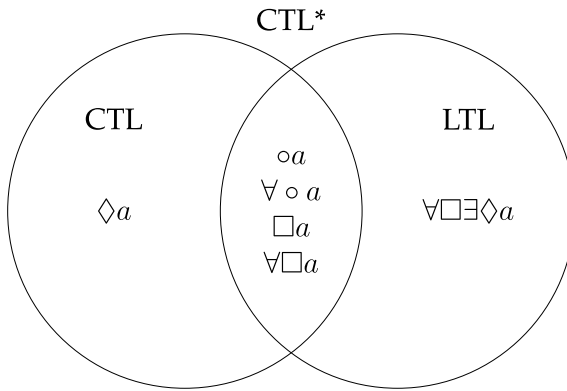


Fig. 3.3 Relationship between CTL, LTL, and CTL*

3.4 Bisimulation

Model checking tools face a combinatorial increase in the size of the state space, commonly known as the state explosion problem. Researchers have developed symbolic algorithms, partial order reduction methods, abstractions, and on-the-fly model checking in order to cope with this problem; see, e.g., [2, 6]. These tools were initially developed to reason about the logical correctness of discrete state systems, but have since been extended to deal with real-time and some other special cases of hybrid systems; see, e.g., [10, 11] and later sections in this chapter.

3.4.1 Simulation Relation

We now focus on abstraction-based approaches by obtaining equivalent quotient transition systems that satisfy the same temporal logics. Here the quotient is taken with respect to simulation or bisimulation equivalences [12] as defined below for labeled transition systems.

Definition 3.10 Let $T_i = (S_i, S_i^0, \rightarrow)$ with $i = 1, 2$ be two transition systems,³ and $l_i : S_i \rightarrow 2^P$ label their states, respectively. A relation $R \subseteq S_1 \times S_2$ is said to be a *simulation relationship* from a labeled transition system (T_1, l_1) to (T_2, l_2) if the following hold:

- (1) for any pair $(s_1, s_2) \in R$, their labels are the same, i.e., $l_1(s_1) = l_2(s_2)$;
- (2) for any initial state $s_1 \in S_1^0$, there exists $s_2 \in S_2^0$ such that $(s_1, s_2) \in R$;
- (3) for any pair $(s_1, s_2) \in R$, if $s'_1 \in \text{post}(s_1)$ in T_1 , then there exists $s'_2 \in S_2$ such that $s'_2 \in \text{post}(s_2)$ in T_2 and $(s'_1, s'_2) \in R$.

Intuitively, a labeled transition system simulates another system if, for every computation in the simulated system, there is a matching (w.r.t. labels) computation in the simulating system. If there exists a simulation relationship R from labeled transition

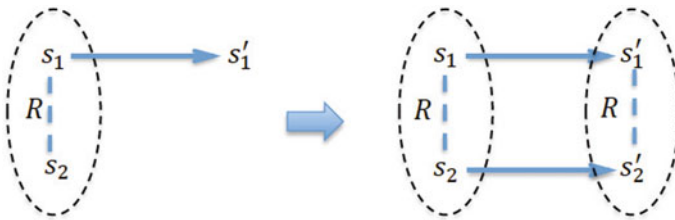


Fig. 3.4 If a labeled transition system (T_1, l_1) is simulated by another labeled transition system (T_2, l_2) , then for each transition in T_1 , there is a matching transition in T_2

³ Since we use the label-based simulation relation and we are only concerned about the existence of actions, we therefore omit the definition of action sets U_i and treat all actions equally.

system (T_1, l_1) to (T_2, l_2) , we also say that (T_1, l_1) is simulated by (T_2, l_2) , or (T_2, l_2) simulates (T_1, l_1) , denoted as $(T_1, l_1) \prec_R (T_2, l_2)$, since for any trace in (T_1, l_1) one can find a corresponding equivalent trace in (T_2, l_2) . This is illustrated in Fig. 3.4. Hence, if $(T_1, l_1) \prec_R (T_2, l_2)$, then we have $\mathcal{T}(T_1, l_1) \subseteq \mathcal{T}(T_2, l_2)$.

Definition 3.11 Relation R is said to be a *bisimulation relation* between (T_1, l_1) and (T_2, l_2) if R is a simulation relation from (T_1, l_1) to (T_2, l_2) and R^{-1} is a simulation relation from (T_2, l_2) to (T_1, l_1) , i.e., $(T_2, l_2) \prec_{R^{-1}} (T_1, l_1)$. If such a bisimulation relation R exists between (T_1, l_1) and (T_2, l_2) , then we say that (T_1, l_1) is *bisimilar* to (T_2, l_2) , denoted as $(T_1, l_1) \cong_{R \cup R^{-1}} (T_2, l_2)$.

Usually, we only care about the existence of such an R , so we usually omit R and simply write $(T_1, l_1) \prec (T_2, l_2)$ or $(T_1, l_1) \cong (T_2, l_2)$. Since $(T_1, l_1) \cong (T_2, l_2)$ implies both $(T_1, l_1) \prec (T_2, l_2)$ and $(T_2, l_2) \prec (T_1, l_1)$, two bisimilar labeled transition systems are trace equivalent, i.e., $(T_1, l_1) \cong (T_2, l_2)$ implies $\mathcal{T}(T_1, l_1) = \mathcal{T}(T_2, l_2)$. Furthermore, $(T_1, l_1) \cong (T_2, l_2)$ also implies $\mathcal{T}_\omega(T_1, l_1) = \mathcal{T}_\omega(T_2, l_2)$.

Since two bisimilar labeled transition systems generate exactly the same traces, they will satisfy the same collection of LTL formulas. Namely, they cannot be distinguished by LTL formulas, i.e., they are LTL equivalent, denoted as $(T_1, l_1) \equiv_{LTL} (T_2, l_2)$. However, two trace equivalent labeled transition systems may not necessarily be bisimilar. The following example illustrates this fact.

Example 3.21 Let's revisit the vending machine Example 3.15. It has been shown that T_1 and T_2 are trace equivalent. However, they are not bisimilar. To illustrate this point, we first consider the following binary relation $R \subseteq S_2 \times S_1$, where S_1 and S_2 stand for the the state set of T_1 and T_2 , respectively. In particular,

$$R = \{(s_0, q_0), (s_1, q_1), (s_2, q_1), (s_3, q_2), (s_4, q_3)\}.$$

It is easy to check that

1. for any pair $(s_i, q_j) \in R$, their labels are the same, i.e., $l_2(s_i) = l_1(q_j)$;
2. for the initial state s_0 , there exists q_0 such that $(s_0, q_0) \in R$;
3. for any pair $(s_i, q_j) \in R$, if $s'_i \in \text{post}(s_i)$ in T_2 then there exists $q'_j \in S_1$ such that $q'_j \in \text{post}(q_j)$ in T_1 and $(s'_i, q'_j) \in R$. For example, consider $(s_1, q_1) \in R$ and $s_3 \in \text{post}(s_1)$, we have $q_2 \in \text{post}(q_1)$ and $(s_3, q_3) \in R$.

Hence, R is a simulation relation from (T_2, l_2) to (T_1, l_1) , i.e., $(T_2, l_2) \prec_R (T_1, l_1)$.

However, its inverse $R^{-1} = \{(q_0, s_0), (q_1, s_1), (q_1, s_2), (q_2, s_3), (q_3, s_4)\}$ is not a simulation relation from (T_1, l_1) to (T_2, l_2) . To see this, consider the pair $(q_1, s_1) \in R^{-1}$ and $q_3 \in \text{post}(q_1)$, but there doesn't exist a state in the $\text{post}(s_1) = \{s_3\}$ that is related to q_3 in R^{-1} .

This example serves to illustrate the definitions of simulation and bisimulation relation and sheds light into why the above two transition systems are not bisimilar. To formally show that $(T_1, l_1) \not\cong (T_2, l_2)$ in the above example, we will wait till Example 3.25 after introducing the bisimulation algorithm. $\square$

From this example and Example 3.17, we can see that it is possible to distinguish between two trace equivalent but not bisimilar labeled transition systems using CTL formulas. Actually, the reverse is also true. If two labeled transition systems cannot be distinguished by CTL formulas, then they are bisimilar [2, 6]. Namely, two labeled transition systems are CTL equivalent if and only if they are bisimilar.

Therefore, a CTL model checking problem on a labeled transition system (T_1, l_1) can be carried on a bisimilar labeled transition system (T_2, l_2) . We hope that the bisimilar system (T_2, l_2) has smaller state space compared to (T_1, l_1) , and the computational complexity can be reduced. Furthermore, bisimulation preserves not only CTL properties but also all CTL* properties [2, 6].

3.4.2 Bisimulation Quotient

To reduce the computational complexity of model checking, we try to reduce the size of the state space of a transition system by clustering all bisimilar equivalent states. For such a purpose, we introduce the self-bisimulation relation for a labeled transition system first.

Definition 3.12 Consider a labeled transition system (T, l) and a binary relationship $R \subseteq S \times S$. The relation R is called a *self-bisimulation relation* for (T, l) if the following hold:

- (1) $\forall (s_1, s_2) \in R, l(s_1) = l(s_2)$;
- (2) $\forall s'_1 \in \text{post}(s_1), \exists s'_2 \in \text{post}(s_2)$ with $(s'_1, s'_2) \in R$;
- (3) $\forall s_2 \in \text{post}(s_2), \exists s_1 \in \text{post}(s_1)$ with $(s_1, s_2) \in R$.

States s_1 and s_2 are bisimilar, denoted as $s_1 \sim s_2$, if there exists such a binary relation R in (T, l) , defined in the above definition. The self-bisimulation relation defined above forms an equivalence relation as it is reflexive (as every state s is bisimilar to itself), symmetric (namely, if $s_1 \sim s_2$ then $s_2 \sim s_1$ by definition), and transitive (which means that if $s_1 \sim s_2$ and $s_2 \sim s_3$ then $s_1 \sim s_3$ by definition). Since R is an equivalence relation on S , it induces a partition of the state space into a collection of equivalent classes $S = \bigcup_{s \in S} [s]_R$, where $[s]_R$ is a collection of all bisimilar states to s , namely $[s]_R = \{s' \in S \mid (s, s') \in R\}$.

It can be shown that for any $s, s' \in S$, the following hold:

- $s \in [s]_R$;
- if $s' \in [s]_R$, then $[s]_R = [s']_R$;
- if $s' \notin [s]_R$, then $[s]_R \cap [s']_R = \emptyset$;
- $S = \bigcup_{s \in S} [s]_R$.

Hence, the bisimulation relation R does induce a partition of the state set S . For simplicity, we denote such a partition S/R , which stands for the quotient space, i.e., the set consisting of all equivalent classes. Based on the quotient space, we can define a quotient transition system as follows.

Definition 3.13 Given a labeled transition system (T, l) and a bisimulation relation $R \subseteq S \times S$, the *quotient transition system* can be defined as $T/R = (S/R, S^0/R, \rightarrow_R)$ where $S/R = \{[s]_R\}_{s \in S}$, the initial states

$$S^0/R = \{[s]_R \in S/R : S^0 \cap [s]_R \neq \emptyset\},$$

and for $[s_1]_R, [s_2]_R \in S/R$, $([s_1]_R, [s_2]_R) \in \rightarrow_R$ if and only if there exist $s_1 \in [s_1]_R$ and $s_2 \in [s_2]_R$ such that $(s_1, s_2) \in \rightarrow$. The new label map $l_R : S/R \rightarrow 2^{\mathcal{P}}$ is defined as $l_R([s]_R) = l(s)$.

Note that since when $s' \in [s]_R$ then $l(s') = l(s)$ by definition, the new label map l_R is well-defined. Also, it can be checked that there exists a relation on $S \times S/R$ that satisfies the bisimulation relation definition. Particularly, we can choose $\mathcal{R} \subseteq S \times S/R$ with $(s', [s]_R) \in \mathcal{R}$ if and only if $s' \in [s]_R$. Then, we can conclude that $(T, l) \cong_{\mathcal{R} \cup \mathcal{R}^{-1}} (T/R, l_R)$; see, e.g., [2, 6] for more detailed discussions. Hence, $(T, l) \cong (T/R, l_R)$. The hope is that the quotient transition system T/R has much fewer number of states compared to the original transition system T , so the model checking problem can be significantly simplified. This idea is used for timed automata model checking in the next section.

Example 3.22 Consider a labeled transition system (T, l) over an atomic proposition set $\mathcal{P} = \{a\}$ as shown in Fig. 3.5.

Consider the binary relation, $R = \{(q_2, q_3), (q_3, q_2)\} \cup Id$, where Id stands for all identical state pairs, such as (q_0, q_0) , (q_1, q_1) , and so on. It can be checked that R is reflexive, symmetric, and transitive, so R is an equivalence relation. Furthermore, all conditions in Definition 3.12 are satisfied, so R is a self-bisimulation relation for (T, l) . Based on R , we can partition the states in T into two equivalent classes as $[q_0]_R = \{q_0\}$, $[q_1]_R = \{q_1\}$ and $[q_2]_R = [q_3]_R = \{q_2, q_3\}$. Based on Definition 3.13, we obtain a quotient transition system $T/R = (S/R, S^0/R, \rightarrow_R)$ where

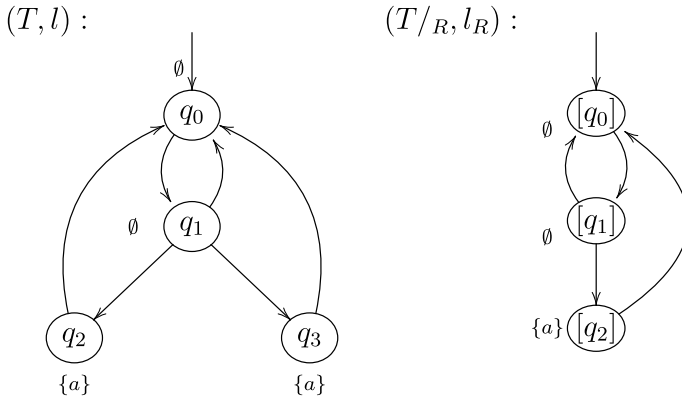


Fig. 3.5 A labeled transition system and its bisimulation quotient transition system

$S/R = \{[q_0]_R, [q_1]_R, [q_2]_R\}$, the initial states $S^0/R = \{[q_0]_R\}$, and the transition $\rightarrow_R$ relation contains $([q_0]_R, [q_1]_R)$, $([q_1]_R, [q_0]_R)$, $([q_1]_R, [q_2]_R)$, and $([q_2]_R, [q_0]_R)$. The new label map l_R is defined as $l_R([q_0]_R) = \emptyset$, $l_R([q_1]_R) = \emptyset$, and $l_R([q_2]_R) = \{a\}$. The quotient labeled transition system $(T/R, l/R)$ is shown in Fig. 3.5.

Based on the definition, it can be shown that the binary relation

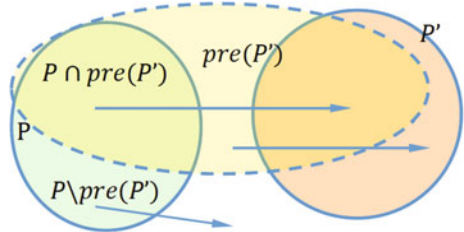
$$\mathcal{R} = \{(q_0, [q_0]_R), (q_1, [q_1]_R), (q_2, [q_2]_R), (q_3, [q_2]_R)\},$$

together with its reverse, forms a bisimulation relation between (T, l) and $(T/R, l/R)$. Hence, $(T, l) \cong (T/R, l/R)$. $\square$

3.4.3 Computing Bisimulations

In the previous subsection, we obtained a quotient transition system T/R , which is bisimilar to the original transition system and is called a *bisimulation quotient* of T . A crucial property of bisimulation is that for every equivalence class $P \in S/R$, the predecessor region $pre(P)$ is a union of equivalence classes. We show this claim by contradiction. Assume that there is an equivalence class $P' \in S/R$ whose predecessor region $pre(P')$ is not a union of equivalence classes. Namely, there exists $P \in S/R$ such that $P \cap pre(P') \neq P$ and $P \cap pre(P') \neq \emptyset$ as illustrated in Fig. 3.6. Then, the set difference $P \setminus pre(P') \neq \emptyset$ and there exist states $s_1 \in P \cap pre(P')$ and $s_2 \in P \setminus pre(P')$. Since s_1 and s_2 are both in the same equivalent class P , $s_1 \sim s_2$. However, there exists a state $s'_1 \in post(s_1)$ in P' , for which there is no bisimilar state in $post(s_2)$ as $post(s_2) \cap P' = \emptyset$, namely $\nexists s'_2 \in post(s_2)$ with $s'_1 \sim s'_2$. This violates the assumption that R is a bisimulation relation. Therefore, if $P_1, P_2 \in S/R$, then $pre(P_1) \cap P_2$ is either the empty set or all of P_2 . This property is used in the following algorithm to compute a bisimulation relation for a transition system.

Fig. 3.6 Refinement step in the bisimulation algorithm



Bisimulation Algorithm

Finding an equivalent relation is equivalent to finding the equivalent classes in S/R . Along this line, the following algorithm determines a partition of S so that the corresponding equivalent relation is a bisimulation.

Algorithm 3.1 (Bisimulation Algorithm)

Initialization: $\{(s_1, s_2) | l(s_1) = l(s_2)\} = S/R$

Refine:

while $\exists P, P' \in S/R$ such that $P \cap pre(P') \neq P$ and $P \cap pre(P') \neq \emptyset$ **do**

$P_1 = P \cap pre(P')$,

$P_2 = P \setminus pre(P')$

$S/R = (S/R \setminus \{P\}) \cup \{P_1, P_2\}$;

end while

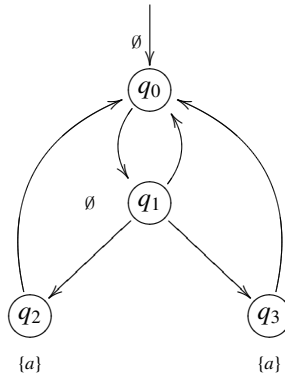
return S/R

If the algorithm terminates within a finite number of iterations of the loop, then there is a finite bisimulation quotient, and the algorithm returns a finite partition of the state space which is the coarsest bisimulation (i.e., the bisimulation with the fewest equivalence classes).

This is a pseudo algorithm. Implementation and termination for general transition systems are not obvious. For finite state systems, one can implement the algorithm and guarantee that it terminates, because we can enumerate the states for the finite state system. We use two simple examples below to illustrate the algorithm.

Example 3.23 Let's revisit the labeled transition systems (T, l) in Example 3.22.

$(T, l) :$



We would like to use the bisimulation algorithm to compute its quotient transition system. Following the algorithm, we initially select the atomic proposition consistent partition of the state space, i.e.,

$$S/R = \{\{q_0, q_1\}, \{q_2, q_3\}\}.$$

Next, we process the refinement step and pick $P = \{q_0, q_1\}$ and $P' = \{q_2, q_3\}$. Note that $pre(P') = pre(q_2) \cup pre(q_3) = \{q_1\}$, hence $P \cap pre(P') = \{q_1\} \neq P$ and $P \cap pre(P') = \{q_1\} \neq \emptyset$. So, we need to further partition the set P as $P_1 = P \cap pre(P') = \{q_1\}$ and $P_2 = P \setminus pre(P') = \{q_0\}$. The state partition is then updated to

$$S/R = (S/R \setminus \{P\}) \cup \{P_1, P_2\} = \{\{q_0\}, \{q_1\}, \{q_2, q_3\}\}.$$

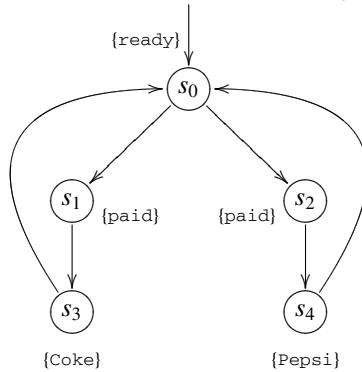
Next, we do another round of refinement, and pick $P = \{q_2, q_3\}$ and $P' = \{q_0\}$. Note that $pre(P') = pre(q_0) = \{q_2, q_3\}$, hence $P \cap pre(P') = \{q_2, q_3\} = P$. So, no further partition is needed and S/R remains the same, i.e.,

$$S/R = \{\{q_0\}, \{q_1\}, \{q_2, q_3\}\}.$$

Furthermore, we select $P = \{q_2, q_3\}$ and $P' = \{q_1\}$ for another round of refinement. Note that $pre(P') = pre(q_1) = \{q_0\}$, hence $P \cap pre(P') = \emptyset$. So, no further partition is needed either and S/R remains as the same, i.e., $S/R = \{\{q_0\}, \{q_1\}, \{q_2, q_3\}\}$.

We can continue the refinement process by selecting different combinations of P and P' from S/R until none violates the conditions in the while-loop. Since the number of states and its partition are finite, this algorithm always terminates after a finite number of iterations. In this example, the partition $S/R = \{\{q_0\}, \{q_1\}, \{q_2, q_3\}\}$ is actually the final result returned from the algorithm, based on which we can construct the quotient transition system as shown in Example 3.22. $\square$

Example 3.24 Consider the labeled transition system (T_2, l_2) in Example 3.15.



For this example, the state is $S = \{s_0, s_1, s_2, s_3, s_4\}$. As the initial partition, we select $S/R = \{\{s_0\}, \{s_1, s_2\}, \{s_3\}, \{s_4\}\}$ that is consistent with the labels. Then, we apply the bisimulation algorithm and consider $P = \{s_1, s_2\}$ and $P' = \{s_3\}$ from S/R . Calculations show that $pre(P') = \{s_1\}$. So $P_1 = P \cap pre(P') = \{s_1\}$ and $P_2 = P \setminus pre(P') = \{s_2\}$. Then, we update the partition of S as $S/R = \{\{s_0\}, \{s_1\}, \{s_2\}, \{s_3\}, \{s_4\}\}$,

which is trivial as it contains all the individual states. The algorithm stops and derives a quotient transition system of (T_2, l_2) as itself. $\square$

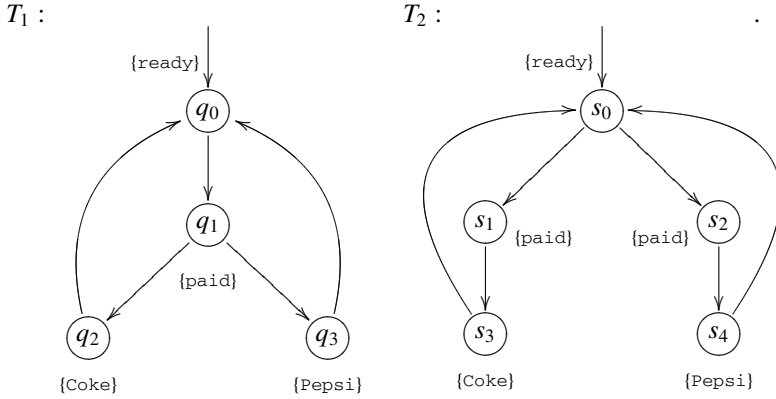
Bisimulation Between Labeled Transition Systems

The bisimulation algorithm can also be used to check the bisimilarity of two transition systems (T_1, l_1) , (T_2, l_2) with mutually exclusive states, i.e., $S_1 \cap S_2 = \emptyset$. For such a purpose, the following method can be used:

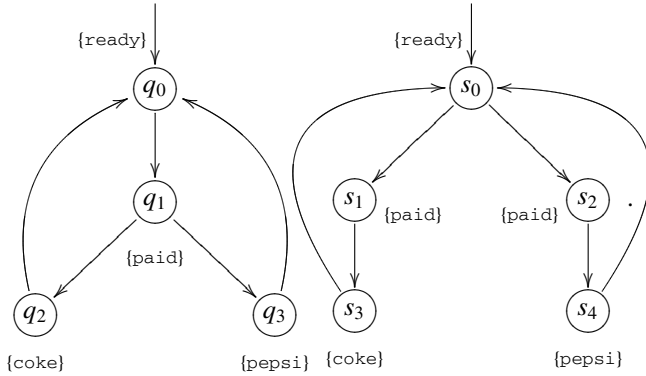
- compose (T_1, l_1) and (T_2, l_2) as $T_1 \oplus T_2 = (S_1 \cup S_2, \rightarrow_1 \cup \rightarrow_2, S_1^0 \cup S_2^0, l)$, where $l(s) = l_1(s)$ for $s \in S_1$ and $l(s) = l_2(s)$ for $s \in S_2$;
- use bisimulation algorithm to calculate the quotient of $(T_1 \oplus T_2, l)$, and check whether $\forall s_1^0 \in S_1^0, \exists s_2^0 \in S_2^0$, such that $s_1^0 \sim s_2^0$, and whether $\forall s_2^0 \in S_2^0, \exists s_1^0 \in S_1^0$, such that $s_2^0 \sim s_1^0$. If yes, then $(T_1, l_1) \cong (T_2, l_2)$. Otherwise, $(T_1, l_1) \not\cong (T_2, l_2)$.

We illustrate the idea using the vending machine example.

Example 3.25 Consider the labeled transition systems (T_1, l_1) and (T_2, l_2) in Example 3.15.



To check whether $(T_1, l_1) \cong (T_2, l_2)$, we first compose these two labeled transition systems together as $(T_1 \oplus T_2, l)$:



Then, we run the bisimulation algorithm on the composed system with the initial partition taken to be $S/R = \{\{q_0, s_0\}, \{q_1, s_1, s_2\}, \{q_2, s_3\}, \{q_3, s_4\}\}$.

Consider $P = \{q_1, s_1, s_2\}$ and $P' = \{q_2, s_3\}$. We have $pre(P') = \{q_1, s_1\}$, so $P \cap pre(P') = \{q_1, s_1\} \neq P$ and $P \cap pre(P') \neq \emptyset$, so we refine the partition as

$$S/R = \{\{q_0, s_0\}, \{q_1, s_1\}, \{s_2\}, \{q_2, s_3\}, \{q_3, s_4\}\}.$$

Next, consider $P = \{q_1, s_1\}$ and $P' = \{q_3, s_4\}$. We have $pre(P') = \{q_1, s_2\}$, so $P \cap pre(P') = \{q_1\} \neq P$ and $P \cap pre(P') \neq \emptyset$, so we refine the partition as

$$S/R = \{\{q_0, s_0\}, \{q_1\}, \{s_1\}, \{s_2\}, \{q_2, s_3\}, \{q_3, s_4\}\}.$$

Next, consider $P = \{q_0, s_0\}$ and $P' = \{q_1\}$. We have $pre(P') = \{q_0\}$, so $P \cap pre(P') = \{q_0\} \neq P$ and $P \cap pre(P') \neq \emptyset$, so we refine the partition as

$$S/R = \{\{q_0\}, \{s_0\}, \{q_1\}, \{s_1\}, \{s_2\}, \{q_2, s_3\}, \{q_3, s_4\}\}.$$

The problem now is that there does not exist an initial state in T_2 which is in the same equivalent class of q_0 , so we terminate the algorithm and conclude that (T_1, l_1) and (T_2, l_2) are not bisimilar. $\square$

In the next sections, we will use bisimulation quotient transition systems and model checking techniques to investigate the verification problems for timed and hybrid automata.

3.5 Timed Automata

Timed automata were introduced in [13] as formal models for real-time programs. Timed automata have been successfully used in modeling, analysis, and design of manufacturing systems [14], and scheduling and robotic systems [15]. In this subsection, we first formally define timed automata and show them to be a special case of hybrid automata. Then, model checking of real-time properties for timed automata is discussed by obtaining a finite bisimulation quotient transition system of timed automata.

3.5.1 Timed Automata

In timed automata, a finite number of real-valued clocks (or clock variables), denoted as $\mathbb{C} = \{x_1, \dots, x_n\}$, are introduced to record the elapse of time since a certain property became true or a particular state was visited. The clocks in timed automata are used more like stop-watches as they only can be reset to zero and be inspected independently of one another. Initially, all clocks start from zero and increase with a constant unit rate, i.e., $\dot{x}_i = 1$. We can compare the value of a particular clock variable x_i , called a valuation of clock x_i , to a constant which is usually assumed to be an integer or a rational number, but we do not compare two clock values directly. Conditions that depend on clock values are called *clock constraints*.

Definition 3.14 The set, $\Phi(\mathbb{C})$, of *clock constraints* of $\mathbb{C}$ is a set of finite logical expressions defined inductively as follows:

- (1) $(x_i \leq c)$, $(x_i < c)$, $(x_i \geq c)$, and $(x_i > c)$ are all clock constraints for $x_i \in \mathbb{C}$ and $c \geq 0$ being a rational number;
- (2) if δ_1 and δ_2 are clock constraints, so is their conjunction $\delta_1 \wedge \delta_2$.

Definition 3.15 The *atomic clock constraints*, denoted as $\mathcal{ACC}(\mathbb{C})$, over a set of clock variables $\mathbb{C}$ are those clock constraints in the form of $x_i \geq c$, $x_i > c$, $x_i \leq c$, or $x_i < c$, where $x_i \in \mathbb{C}$ and $c \geq 0$ is a rational number. In addition, $|\mathbb{C}|$ denotes the number of clocks. Then, the clock values can be seen as vectors in $\mathbb{R}^{|\mathbb{C}|}$ or in $\mathbb{R}_{\geq 0}^{|\mathbb{C}|}$ as clocks cannot be negative.

Example 3.26 Let $\mathbb{C} = \{x_1, x_2\}$, then the clock constraints $\Phi(\mathbb{C})$ may consist of the following logical expressions:

- $(x_1 \leq 3)$, $(x_2 \geq 1)$, $(x_1 > 3)$, $(x_2 > 1) \in \Phi(\mathbb{C})$; also
- $(x_2 = 1) \in \Phi(\mathbb{C})$, since $(x_2 = 1)$ is equivalent to $(x_2 \leq 1) \wedge (x_2 \geq 1)$;
- $(1 \leq x_2 \leq 3) \in \Phi(\mathbb{C})$, since $(1 \leq x_2 \leq 3)$ is equivalent to $(x_2 \geq 1) \wedge (x_2 \leq 3)$.

Notice that the clock constraints defined in $\Phi(\mathbb{C})$ are convex since only the conjunction between atomic clock constraints is allowed. Therefore, non-convex constraints, such as $x \neq 1$, are not defined in $\Phi(\mathbb{C})$. Furthermore, the expressions like

$(x_1 \leq x_2)$ or $x_1 \geq \sqrt{2}$ are not clock constraints as we cannot directly compare two clocks or compare a clock variable with an irrational number.

Clock constraints are used to specify conditions for a discrete transition to happen (guard sets) or the amount of time that one may stay in a particular discrete mode (invariant conditions). Next, we formally define timed automata and show that they are a special case of hybrid automata.

Definition 3.16 A *timed automaton* A is a tuple $(Q, Q^0, \mathcal{E}, \mathbb{C}, I, \rightarrow)$, where

- Q : finite set of discrete states;
- Q^0 : finite set of initial discrete states;
- $\mathcal{E}$: finite set of events;
- $\mathbb{C}$: finite set of clocks;
- $I : Q \rightarrow \Phi(\mathbb{C})$ is a map that labels each $q \in Q$ with a clock constraint;
- $\rightarrow \subseteq Q \times \mathcal{E} \times \Phi(\mathbb{C}) \times 2^{\mathbb{C}} \times Q$ is a transition relation. An element $(q, e, \varphi, \lambda, q')$ represents a transition from q to q' on event e such that the clock constraint $\varphi \in \Phi(\mathbb{C})$ and resetting the clocks $\lambda \subseteq \mathbb{C}$ to zero.

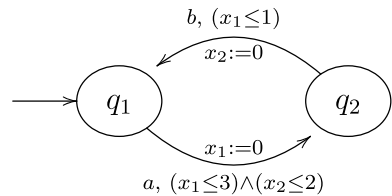
Consider a transition relation, $(q, e, \varphi, \lambda, q')$, defined above. A discrete transition q to q' is allowed to happen if the clock constraint φ holds true for the current clock values and the discrete event e is feasible. Upon transition, the clock variables in the set λ are reset to zero. So, the clock constraint φ is very similar to the guard set in the hybrid automaton definition, while the λ works similar to the resetting map. Actually, timed automata can be seen as a special class of hybrid automata as any timed automaton can be transformed into a hybrid automaton. Before we show the transformation, let's consider first a timed automaton example to illustrate the definition.

Example 3.27 Consider a timed automaton $A = (Q, Q^0, \mathcal{E}, \mathbb{C}, I, \rightarrow)$, where $Q = \{q_1, q_2\}$, $Q_0 = \{q_1\}$, $\mathcal{E} = \{a, b\}$, $\mathbb{C} = \{x_1, x_2\}$, $I(q_1) = I(q_2) = \mathbb{R}_{\geq 0}^2$, and the transition relation $\rightarrow$ is defined as

$$\rightarrow = \{(q_1, a, (x_1 \leq 3) \wedge (x_2 \leq 2), \{x_1\}, q_2), (q_2, b, (x_1 \leq 1), \{x_2\}, q_1)\}.$$

Graphically, the timed automaton A can be represented as shown in Fig. 3.7. The timed automaton starts from the initial mode q_1 and zero clock values, namely $x_1 = 0$ and $x_2 = 0$. The time will elapse and each clock increases its value at a constant unit rate. If the discrete event a happens within two time units (say at $t_1 \leq 2$), then

Fig. 3.7 Timed automata example



the transition $(q_1, a, (x_1 \leq 3) \wedge (x_2 \leq 2), \{x_1\}, q_2)$ will be triggered since the guard condition $(x_1 \leq 3) \wedge (x_2 \leq 2)$ holds true, and the discrete mode will be transited to q_2 while the clock x_1 will be reset to zero. Then, the evolution starts from mode q_2 with clock values $x_1 = 0$ and $x_2 = t_1$. Let's assume that event b occurs within one time unit at t_2 (i.e., with $t_2 - t_1 \leq 1$), then the transition $(q_2, b, (x_1 \leq 1), \{x_2\}, q_1)$ will be triggered, and the mode is switched back to q_1 and the clock x_2 is now reset to zero. $\square$

A timed automaton $A = (Q, Q^0, \mathcal{E}, \mathbb{C}, I, \rightarrow)$ can be viewed as a special case of a hybrid automaton $H_A = (Q, X, f, init, Inv, E, G, R)$ where

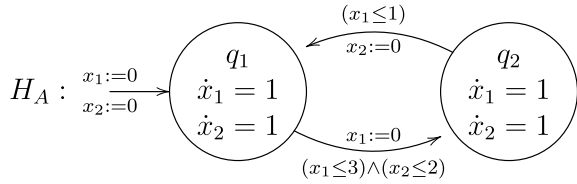
- Q is the same;
- $X = \mathbb{R}_{\geq 0}^{|\mathbb{C}|}$ defines the domain of clocks;
- $f(q, x) = \mathbf{1}$ for all $q \in Q$, where $\mathbf{1}$ stands for the vector in $\mathbb{R}^{|\mathbb{C}|}$ with all elements equal to one;
- $init = Q^0 \times \{\mathbf{0}\}$, where $\mathbf{0}$ stands for the null vector in $\mathbb{R}^{|\mathbb{C}|}$;
- $Inv(q) = \{x \in X \mid x \text{ satisfies the clock constraints in } I(q)\}$;
- $(q, q') \in E$ when there exist $e \in \mathcal{E}$, clock constraint $\varphi \in \Phi(\mathbb{C})$ and clocks $\lambda \subseteq \mathbb{C}$ such that $(q, e, \varphi, \lambda, q') \in \rightarrow$; and for this case, $G(q, q') = \{x \in X \mid x \text{ satisfies the clock constraints in } \varphi\}$ and $R(q, q', x_i) = 0$ if $x_i \in \lambda$ while x_i remains unchanged otherwise.

Example 3.28 Consider the timed automaton $A = (Q, Q^0, \mathcal{E}, \mathbb{C}, I, \rightarrow)$ in the previous example. It can be represented as a hybrid automaton $H_A = (Q, X, f, init, Inv, E, G, R)$ with

- $Q = \{q_1, q_2\}$;
- $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$, and $X = \mathbb{R}_{\geq 0}^2$;
- $Init = \left\{ (q_1, \begin{bmatrix} 0 \\ 0 \end{bmatrix}) \right\}$;
- $f(q, x) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ for all $q \in Q$;
- $Inv(q) = \mathbb{R}_{\geq 0}^2$ for all $q \in Q$;
- $E = \{(q_1, q_2), (q_2, q_1)\}$;
- $G(q_1, q_2) = \{x \in \mathbb{R}_{\geq 0}^2 : (x_1 \leq 3) \wedge (x_2 \leq 2)\}$, and $G(q_2, q_1) = \{x \in \mathbb{R}_{\geq 0}^2 : (x_1 \leq 1)\}$;
- $R(q_1, q_2, x) = \begin{bmatrix} 0 \\ x_2 \end{bmatrix}$, $R(q_2, q_1, x) = \begin{bmatrix} x_1 \\ 0 \end{bmatrix}$.

Graphically, it can be represented as a hybrid automaton in Fig. 3.8. So, a timed automaton can be seen as a special case of a hybrid automaton with constant one continuous flow rates, zero initial conditions, rational constant bounds, and elements always being reset to zero. $\square$

Fig. 3.8 Representation of a timed automaton as a hybrid automaton



3.5.2 Timed Language

In this subsection, we give a brief review of the formal language theory for timed automata. Readers can skip this subsection if the main interest is on the timed automata model checking.

Definition 3.17 A time sequence $\bar{\tau} = \tau_1 \tau_2 \dots$ is an infinite sequence of time values, $\tau_i \in \mathbb{R}_{\geq 0}$ satisfying

- Monotonicity: $\tau_i \leq \tau_{i+1}, \forall i \geq 1$.
- Progress: $\forall t \geq 0, \exists i \geq 1$ such that $\tau_i > t$.

Definition 3.18 A timed word over an alphabet Σ is a pair $(\bar{\sigma}, \bar{\tau})$ where $\bar{\sigma} = \sigma_1 \sigma_2 \dots$ is an infinite word over Σ and $\bar{\tau}$ is a time sequence. A timed language T over Σ is a set of timed words over Σ .

For a timed automaton, the alphabet is actually the event set $\mathcal{E}$ defined in the timed automaton.

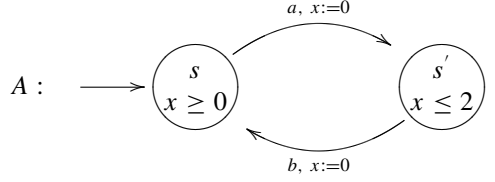
Example 3.29 For example, $T_1 = \{((ab)^\omega, \bar{\tau}) \mid 1 \leq (\tau_{2i} - \tau_{2i-1}) \leq 2, i \geq 1\}$ is a timed language over $\Sigma = \{a, b\}$, and the timed words in T_1 are like

$$\{(a, \tau_1)(b, \tau_2)(a, \tau_3)(b, \tau_4) \cdots (a, \tau_{2i-1})(b, \tau_{2i}) \cdots\},$$

with the requirement that once a happens the event b will happen within 1–2 time units. $\square$

A timed automaton $A = (Q, Q_0, \Sigma, \mathbb{C}, I, \rightarrow)$ reads a timed word $(\bar{\sigma}, \bar{\tau})$ if the following conditions hold: There is an initial state $q_0 \in Q_0$ such that a run of A starts from $(q_0, 0)$ and stays at q_0 till τ_1 . Then, there exists a transition, $(q_0, \sigma_1, \varphi_1, \lambda_1, q_1) \in \rightarrow$, in A , with $0 + \tau_1$ satisfying φ_1 . After the transition and a clock reset to zero in λ , the state of A becomes (q_1, x_1) , and it stays at q_1 for a duration of length $\tau_2 - \tau_1$, then the event σ_2 occurs and the transition $(q_1, \sigma_2, \varphi_2, \lambda_2, q_2) \in \rightarrow$ is taken with a clock reset, and so on.

Example 3.30 Consider the timed language T_1 from Example 3.29. The timed words in T_1 can be accepted by the following timed automaton:



A run corresponding to a timed word $(a, \tau_1)(b, \tau_2)(a, \tau_3)(b, \tau_4) \cdots (a, \tau_{2i-1})(b, \tau_{2i}) \cdots$ can be described as follows:

$$(s, 0) \xrightarrow{\tau_1} (s, \tau_1) \dashrightarrow^a (s', 0) \xrightarrow{\tau_2 - \tau_1} (s', \tau_2 - \tau_1) \dashrightarrow^b (s, 0) \rightarrow \cdots$$

Here we use the solid arrow to represent the elapse of time with duration at the superscript, while the dashed arrow is used to represent the transitions caused by an event. $\square$

As we are interested in nonterminating behaviors of timed automata, we hence adopt the Büchi acceptance condition in timed automata, and call them *Büchi timed automata (BTA)*.

Definition 3.19 A timed word is accepted by a BTA if its corresponding runs visit the marked states, $F \subseteq Q$, infinitely often. A timed language is timed ω -regular if it is accepted by a BTA.

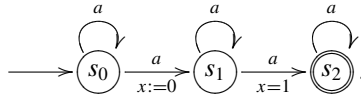
It can be shown that for a timed ω -regular language T , denoted as T , its untimed version, denoted as $Untime(T)$ with

$$Untime(T) = \{\bar{\sigma} \in \Sigma^\omega \mid \exists \bar{\tau} \text{ s.t. } (\bar{\sigma}, \bar{\tau}) \in T\},$$

is ω -regular. For example, the untimed version of T_1 is $\{(ab)^\omega\}$.

The untimed word corresponding to a timed word $(a_0, t_0), (a_1, t_1), \dots, (a_k, t_k)$ is the word $a_0 a_1 \cdots a_k$ obtained by deleting the occurrence times. The untimed language of a timed language L consists of all the untimed words corresponding to the timed words in L , denoted by $Untime(L)$. A timed language $L \subseteq T\Sigma^*$ is said to be timed regular if there exists a timed automaton A such that $\mathcal{L}(A) = L$. For a timed regular language L , $Untime(L)$ is a regular language.

Timed ω -regular language is closed under intersection and union, but it is not closed under complementation [13]. For example, the timed language $T = \{(a^\omega, \bar{\tau}) \mid \text{for some } 1 \leq i \leq j, \tau_j = \tau_i + 1\}$ is timed ω -regular since it is accepted by a BTA given below:



However, its complement $T^c = \{(a^\omega, \bar{\tau}) \mid \forall i, j : \tau_j \neq \tau_i + 1\}$ is not timed ω -regular. To see why, we first observe that any timed automaton accepting T^c needs to

remember all the time instances once the event a occurred within the past one time unit since it has to avoid a happening again exactly after one time unit. Note that it is possible that a occurs an infinite number of times within one time unit, which means that we need an infinite number of time clocks to track the occurrence of event a .

Therefore, the universality problem for timed automata (whether $\mathcal{L}(A) = T\Sigma^*$) is undecidable. Furthermore, the inclusion (whether $\mathcal{L}(A_1) \subseteq \mathcal{L}(A_2)$) and equivalence problems (whether $\mathcal{L}(A_1) = \mathcal{L}(A_2)$) for timed automata are undecidable as well.

3.5.3 Timed Computation Tree Logic

In temporal logic, we can specify some properties that will eventually hold true. However, we cannot explicitly specify when the property will be satisfied. Timed automata, on the other hand, provide us with the modeling power to explicitly record the time when certain states are visited and how long the system stays in that state. Hence, the temporal logic needs to be extended so as to include time variables. In particular, we briefly take a look at timed computation tree logic (TCTL), which is a real-time variant of CTL [16].

Syntax

Formulas in TCTL over an atomic proposition set $\mathcal{P}$ are formed according to the following syntax.

Definition 3.20 Let $\mathcal{P}$ be a finite set of atomic propositions, and $\mathbb{C}$ be a finite set of clock variables. TCTL formulas are state formulas recursively defined from atomic propositions in $\mathcal{P}$ and atomic clock constraints of $\mathbb{C}$ according to the following rules:

- (1) true , false , p and g are state formulas for all atomic propositions $p \in \mathcal{P}$ and atomic clock constraints $g \in \text{ACC}(\mathbb{C})$;
- (2) if Φ_1 and Φ_2 are state formulas, then $\Phi_1 \wedge \Phi_2$ and $\neg\Phi_1$ are state formulas;
- (3) if Φ_1 and Φ_2 are state formulas, then $\Phi_1 \sqcup^J \Phi_2$ is a path formula, where $J \subseteq \mathbb{R}_{\geq 0}$ is an interval whose bounds are rational numbers;
- (4) if φ is a path formula, then $\exists\varphi$ and $\forall\varphi$ are state formulas.

Compared with the definition of CTL, there is no “next” operator in TCTL. This is due to the fact that we are arguing about timed properties over the dense real time, so there is no meaningful next step from the current time instant. But, similar to CTL, we can use the “until” operator to define “eventually” and “always” as

- $\exists\Diamond^J\Phi = \exists\text{true} \sqcup^J \Phi$;
- $\forall\Diamond^J\Phi = \forall\text{true} \sqcup^J \Phi$;
- $\exists\Box^J\Phi = \neg\forall\Diamond^J\neg\Phi$;
- $\forall\Box^J\Phi = \neg\exists\Diamond^J\neg\Phi$.

In TCTL, one can explicitly specify properties hold within a time interval J . The interval J is usually written in an inequality form, e.g., $\Box \geq 2$ denotes $\Box^{[2, \infty)}$ and $\Box <^8$ denotes $\Box^{[0, 8)}$. Note that the special case $J = [0, \infty)$ can be reduced to a traditional CTL operator as the timing constraints $t \geq 0$ will trivially hold. That is $\Diamond^{[0, \infty)} \Phi = \Diamond \Phi$, $\Box^{[0, \infty)} \Phi = \Box \Phi$, and $\Phi_1 \Box^{[0, \infty)} \Phi_2 = \Phi_1 \Box \Phi_2$.

As an example, $\forall \Box(\text{on} \rightarrow \forall \Diamond^{\leq 2} \neg \text{on})$ is a valid TCTL that requests the switch be always turned off within an interval of length 2, immediately after it is turned on.

Semantics

The semantics of TCTL are interpreted over a timed automaton as a labeled transition system. First, some notation is helpful before translating a timed automaton into a transition system.

The state of a timed automaton $A = (Q, Q^0, \mathcal{E}, \mathbb{C}, I, \rightarrow)$ is a tuple (q, η) , where $q \in Q$ stands for the discrete location of the state and $\eta \in \mathbb{R}^{|\mathbb{C}|}$ stands for the clock value. Since time values cannot be negative, we restrict $\eta \in \mathbb{R}_{\geq 0}^{|\mathbb{C}|}$. Here, $|\mathbb{C}|$ stands for the number of the clock variables, and the value of a particular clock variable is represented as $\eta(x) \in \mathbb{R}_{\geq 0}$ for some $x \in \mathbb{C}$.

The time elapse is captured by the notation of $\eta + d$ for non-negative real $d \geq 0$, and $\eta + d$ is defined as new clock values with $(\eta + d)(x) = \eta(x) + d$ for all $x \in \mathbb{C}$.

We can also define the reset option of a clock variable due to discrete transitions. A subset of clock variable $\mathbb{X} \subseteq \mathbb{C}$ being reset is denoted as $\eta[\mathbb{X} := 0]$, where

$$\eta[\mathbb{X} := 0] = \begin{cases} \eta(x), & x \notin \mathbb{X}, \\ 0, & x \in \mathbb{X}. \end{cases}$$

In the case when $\mathbb{X}$ only contains one clock variable, namely $\mathbb{X} = \{x\}$, we can write $\eta[\{x\} := 0]$ simply as $\eta[x \mapsto 0]$.

The label of a timed automaton is a map from the discrete location Q to a subset of atomic propositions $\mathcal{P}$, i.e., $l : Q \rightarrow 2^{\mathcal{P}}$. A labeled timed automaton is then denoted as (A, l) .

Definition 3.21 A labeled timed automaton (A, l) with $A = (Q, Q^0, \mathcal{E}, \mathbb{C}, I, \rightarrow)$ can be seen as a labeled transition system (T_A^l, L) , where the transition system $T_A^l = (Q_A, Q_A^0, \mathcal{E} \cup \mathbb{R}_{\geq 0}, \rightarrow_A)$ is defined by

- $Q_A = \{(q, \eta) \in Q \times \mathbb{R}^{|\mathbb{C}|} \mid \eta(x) \text{ satisfies } I(q)\}$;
- $Q_A^0 = \{(q, \eta) \in Q^0 \times \mathbb{R}^{|\mathbb{C}|} \mid \eta(x) = 0, \forall x \in \mathbb{C}\}$;
- $(q, \eta) \xrightarrow{\delta}_A (q, \eta + \delta)$ for $\delta \in \mathbb{R}_{\geq 0}$ if for all $0 < \delta' \leq \delta$, $(\eta + \delta')(x)$ satisfies $I(q)$;
- $(q, \eta) \xrightarrow{e}_A (q', \eta[\lambda := 0])$ if $(q, e, \varphi, \lambda, q') \in \rightarrow$ and $\eta(x)$ satisfies φ .

Note that the transition system T_A^l has an infinite number of states, and two kinds of transitions, time-driven transitions and event-driven transitions. The label L for

T'_A is a map from Q_A to the subset of atomic propositions $\mathcal{P}$ or subsets of atomic clock constraints $\mathcal{ACC}(\mathbb{C})$, i.e., $L(q, \eta) \subseteq \mathcal{P} \cup \mathcal{ACC}(\mathbb{C})$. More precisely,

$$L((q, \eta)) = l(q) \cup \{g \in \mathcal{ACC}(\mathbb{C}) \mid \eta \models g\}, \quad (3.12)$$

which basically labels the atomic propositions that hold for the location q and the atomic clock constraints satisfied by the clock value of the state (q, η) . Here, by saying that the clock value η satisfies an atomic clock constraint $g \in \mathcal{ACC}(\mathbb{C})$, denoted as $\eta \models g$, for atomic clock constraint, we mean

- $\eta \models \text{true}$;
- $\eta \models x < c$ iff $\eta(x) < c$ for $x \in \mathbb{C}$;
- $\eta \models x \leq c$ iff $\eta(x) \leq c$ for $x \in \mathbb{C}$;
- $\eta \models x > c$ iff $\eta(x) > c$ for $x \in \mathbb{C}$;
- $\eta \models x \geq c$ iff $\eta(x) \geq c$ for $x \in \mathbb{C}$.

Here, $\eta(x) : \mathbb{C} \rightarrow \mathbb{R}_{\geq 0}$ is a clock valuation.

Then, we are ready to interpret a TCTL formula over a labeled timed automaton (A, l) . TCTL formulas are state formulas arguing about a state (q, η) of (A, l) , and the satisfaction of the state (q, η) for a TCTL formula Φ , denoted as $(q, \eta) \models \Phi$, is defined recursively as

- $(q, \eta) \models \text{true}$;
- $(q, \eta) \models p$ iff $p \in L(q, \eta)$ for $p \in \mathcal{P}$;
- $(q, \eta) \models g$ iff $\eta \models g$ for $g \in \mathcal{ACC}(\mathbb{C})$;
- $(q, \eta) \models \neg\Phi$ iff not $(q, \eta) \models \Phi$;
- $(q, \eta) \models \Phi_1 \wedge \Phi_2$ iff $(q, \eta) \models \Phi_1$ and $(q, \eta) \models \Phi_2$;
- $(q, \eta) \models \exists\varphi$ iff there exists a path from (q, η) satisfying φ ;
- $(q, \eta) \models \forall\varphi$ iff all the paths from (q, η) satisfy φ .

A path formula φ is interpreted over a path π ,

$$(q_0, \eta_0) \xrightarrow[A]{\delta_0} (q_0, \eta_0 + \delta_0) \xrightarrow[A]{e_0} (q_1, \eta_1) \xrightarrow[A]{\delta_1} (q_1, \eta_1 + \delta_1) \xrightarrow[A]{e_1} \dots$$

- The path π satisfies the path formula $\varphi = \Phi \sqcup^J \Psi$, denoted as $\pi \models \Phi \sqcup^J \Psi$ iff there exists i such that $(q_i, \eta_i + d) \models \Psi$ for some $0 \leq d \leq \delta_i$ with $\sum_{k=0}^{i-1} \delta_k + d \in J$, and $\forall j \leq i, (q_j, \eta_j + d') \models \Phi \vee \Psi$, for any $d' \in [0, \delta_j]$, with $\sum_{k=0}^{j-1} \delta_k + d' \leq \sum_{k=0}^{i-1} \delta_k + d$.
- The path π satisfies the path formula $\varphi = \diamond^J \Psi$, denoted as $\pi \models \diamond^J \Psi$ iff there exists $i \geq 0$ s.t. $(q_i, \eta_i + d) \models \Psi$ for some $0 \leq d \leq \delta_i$ with $\sum_{k=0}^{i-1} \delta_k + d \in J$.
- The path π satisfies the path formula $\varphi = \square^J \Psi$, denoted as $\pi \models \square^J \Psi$ iff for all $i \geq 0$ s.t. $(q_i, \eta_i + d) \models \Psi$ for any $0 \leq d \leq \delta_i$ with $\sum_{k=0}^{i-1} \delta_k + d \in J$.

Given a TCTL state formula Φ , we denote all states in (A, l) satisfying Φ as $Sat(\Phi)$ with

$$Sat(\Phi) = \{(q, \eta) \mid (q, \eta) \models \Phi\}.$$

A labeled timed automaton (A, l) satisfies a TCTL formula Φ , denoted as $(A, l) \models \Phi$, if and only all its initial states satisfy Φ , i.e., $(q_0, \mathbf{0}) \in Sat(\Phi)$ for all $q_0 \in Q_0$. From the definition, it is clear that $(A, l) \models \Phi$ if and only if $(T_A^l, L) \models \Phi$.

TCTL to CTL

Note that CTL can be considered as a subclass of TCTL with all intervals J being $[0, \infty)$, for which $t \in J$ trivially holds. On the other hand, it can be shown that any TCTL Φ can be converted to an equivalent TCTL with all intervals being $[0, \infty)$, denoted as $\bar{\Phi}$. Note that TCTL formulas with all intervals being $[0, \infty)$ are actually CTL formulas. Therefore, any TCTL formula Φ can be converted to an equivalent CTL formula.

The basic idea for such a conversion is to introduce an additional clock variable z in the timed automaton to track how long a certain sub-formula of Φ remains true. More precisely, given a labeled timed automaton (A, l) , with $A = (Q, Q^0, \mathcal{E}, \mathbb{C}, l, \rightarrow)$ and $l : Q \rightarrow 2^{\mathcal{P}}$ and a TCTL formula over $\mathbb{C}$ and $\mathcal{P}$, we introduce a new clock variable z into $\mathbb{C}$, and redefine $A \oplus z = (Q, Q^0, \mathcal{E}, \mathbb{C} \cup \{z\}, l, \rightarrow)$ as follows. The state in $A \oplus z$ will be $(q, \eta \oplus z)$, and it holds that

- $(q, \eta) \models \exists(\Phi \sqcup^J \Psi)$ iff the state $(q, \eta \oplus z)$ in $A \oplus z$ with z reset to zero satisfies $\exists((\Phi \vee \Psi) \sqcup ((z \in J) \wedge \Psi))$, which can be denoted as $(q, \eta \oplus z)[z \mapsto 0] \models \exists((\Phi \vee \Psi) \sqcup ((z \in J) \wedge \Psi))$;
- $(q, \eta) \models \forall(\Phi \sqcup^J \Psi)$ iff $(q, \eta \oplus z)[z \mapsto 0] \models \forall((\Phi \vee \Psi) \sqcup ((z \in J) \wedge \Psi))$.

For example, the TCTL formula $\exists \Diamond^{\leq 2} \Phi$ can be converted to a CTL formula $\exists \Diamond((z \leq 2) \wedge \Phi)$. They are equivalent in the sense that a labeled timed automaton $(A, l) \models \exists \Diamond^{\leq 2} \Phi$ if and only if $(A \oplus z, l) \models \exists \Diamond((z \leq 2) \wedge \Phi)$. Here, we can omit the requirement of $[z \mapsto 0]$ as the satisfaction $(A \oplus z, l) \models \exists \Diamond((z \leq 2) \wedge \Phi)$ is referring to all initial states in $(A \oplus z, l)$ and at initial states the clock z always starts from zero.

3.5.4 Timed Automata Model Checking

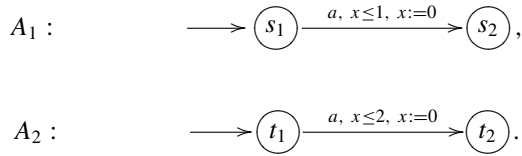
By introducing additional clock variables as shown above, the TCTL model checking problem can be converted to a CTL model checking problem. Hence, without loss of generality, we will focus on CTL model checking for timed automata. The CTL model checking problem for a labeled timed automaton is still difficult since there are an infinite number of states and transitions in it. To overcome this difficulty, we first abstract all transitions due to time elapse into one transition, which is called a *time-abstract transition system* of the timed automaton, and then derive a finite

quotient transition system, called a *region transition system* of the timed automaton, which is bisimilar to the time-abstract transition system.

Time-Abstract Transition System

When we explain the semantics of TCTL, we used a timed transition system interpretation of a timed automaton. Timed transition system interpretation is very natural, but it could be too stringent as the following example shows.

Example 3.31 [42] Consider the following two timed automata:



For timed bisimilarity, it needs that any specific time elapsing step in one timed automaton has to be matched by a time-elapsing step of exactly the same duration in the other. For instance, consider a run in A_1

$$(s_1, 0) \xrightarrow{0.5} (s_1, 0.5) \xrightarrow{a} (s_2, 0) \rightarrow \dots,$$

which can be matched by the following run in A_2 :

$$(t_1, 0) \xrightarrow{0.5} (t_1, 0.5) \xrightarrow{a} (t_2, 0) \rightarrow \dots.$$

By inspection, it not hard to observe that any run in A_1 can be matched by a run in A_2 with exactly the same time duration. However, the reverse does not hold. For example, the following run in A_2

$$(t_1, 0) \xrightarrow{1.5} (t_1, 1.5) \xrightarrow{a} (t_2, 0) \rightarrow \dots$$

does not have a matching run in A_1 . Therefore, A_1 and A_2 are not timed bisimilar. However, these two timed automata are almost identical except for the guard conditions. Hence they should behave “similarly”, but this fails to be captured by the natural timed bisimilarity. The problem is that the requirement of the exact duration for all time-elapsing steps between two matching runs is too stringent. If we relax this requirement by allowing a matching time-elapsing step to be of a different duration, we can get a more suitable equivalence relation for timed automata. $\square$

Motivated by the previous example, we introduce the *time-abstract transition system* of the corresponding timed automata.

Definition 3.22 Given a labeled timed automaton (A, I) , with $A = (Q, Q^0, \mathcal{E}, \mathbb{C}, I, \rightarrow)$ and $I : Q \rightarrow 2^{\mathcal{P}}$, its corresponding labeled time-abstract transition system is (T_A, L) with $T_A = (Q_A, Q_A^0, \mathcal{E} \cup \{\tau\}, \rightarrow)$ and $L : Q_A \rightarrow 2^{\mathcal{P}} \cup \mathcal{ACC}(\mathbb{C})$, where

- $Q_A = \{(q, \eta) \in Q \times \mathbb{R}^{|\mathbb{C}|} \mid \eta(x) \text{ satisfies } I(q)\}$;
- $Q_A^0 = \{(q, \eta) \in Q^0 \times \mathbb{R}^{|\mathbb{C}|} \mid \eta(x) = 0, \forall x \in \mathbb{C}\}$;
- the transition relation $\rightarrow \subseteq Q_A \times (\mathcal{E} \cup \{\tau\}) \times Q_A$ is defined as follows:
 - for $q \in Q$, $((q, \eta), \tau, (q, \eta')) \in \rightarrow$ if there exists $d \geq 0$ such that $\eta' = \eta + d$ and for all $0 < \delta \leq d$, $(\eta + \delta)(x)$ satisfies $I(q)$;
 - for an $e \in \mathcal{E}$, $((q, \eta), e, (q', \eta')) \in \rightarrow$ if $(q, e, \phi, \lambda, q') \in \rightarrow$ and $\eta(x)$ satisfies ϕ and $\eta' = \eta[\lambda := 0]$.

The rationale for introducing the labeled time-abstract transition system for a labeled timed automaton is due to the fact that a CTL formula is satisfied by (A, I) if and only if it is satisfied by (T_A, L) . However, the direct application of CTL model checking techniques to (T_A, L) is infeasible since the state space of T_A is infinite, i.e., T_A is an infinite transition system. Next, we are going to show that there exists a finite quotient transition system of (T_A, L) , called region transition system, that is bisimilar to (T_A, L) .

Region Equivalence

It can be shown that all constants can be assumed to be integers without loss of generality. For a timed automaton $A = (Q, Q^0, \mathcal{E}, \mathbb{C}, I, \rightarrow)$, we can construct another timed automaton $\alpha A = (Q, Q^0, \mathcal{E}, \mathbb{C}, I_\alpha, \rightarrow_\alpha)$ by simply replacing all constants c in A by αc , where α is an arbitrary positive rational number. For a clock valuation $\eta : \mathbb{C} \rightarrow \mathbb{R}_{\geq 0}^{|\mathbb{C}|}$, $\alpha\eta : \mathbb{C} \rightarrow \mathbb{R}_{\geq 0}^{|\mathbb{C}|}$ is defined as $\alpha\eta(x) = \alpha(\eta(x))$ for all $x \in \mathbb{C}$. Let T_A and $T_{\alpha A}$ be the time-abstract transition systems corresponding to timed automata A and αA , respectively. The labels for $T_{\alpha A}$ are defined as $L_\alpha(q, \eta) = L(q, \frac{1}{\alpha}\eta)$.

Lemma 3.1 (T_A, L) is bisimilar to $(T_{\alpha A}, L_\alpha)$.

Proof Consider the relation $(q, \eta) \sim (q, \alpha\eta)$. By definition, $L(q, \eta) = L_\alpha(q, \alpha\eta)$.

Note that, since $\alpha > 0$, $(x_i \leq c) \Leftrightarrow (\alpha x_i \leq \alpha c)$, and $(x_i \geq c) \Leftrightarrow (\alpha x_i \geq \alpha c)$. Therefore, $(q, \eta) \in Q_A^0 \Leftrightarrow (q, \alpha\eta) \in Q_{\alpha A}^0$.

If there exists a discrete transition, $(q, \eta) \xrightarrow{\tau} (q', \eta')$ in T_A , then the transition $(q, e, \phi, \lambda, q')$ exists in A with $\eta \models \phi$ and $\eta' = \eta[\lambda := 0]$. It implies that $(q, e, \phi_\alpha, \lambda, q')$ exists in αA . Hence, there exists the transition $(q, \alpha\eta) \xrightarrow{e} (q', \alpha\eta')$ in $T_{\alpha A}$ as $\alpha\eta \models \phi_\alpha$ and $\alpha\eta' = \alpha\eta[\lambda := 0]$.

For a τ transition in T_A , that is, for any $((q, \eta), \tau, (q, \eta')) \in \rightarrow$ in T_A , there exists $d \geq 0$ such that $\eta' = \eta + d$ and for all $0 < \delta \leq d$, $(\eta + \delta)(x)$ satisfies $I(q)$. Correspondingly, we have $(q, \alpha\eta) \xrightarrow{\tau} (q, \alpha\eta')$ in $T_{\alpha A}$ since we can pick $\alpha d \geq 0$, and $\alpha\eta' = \alpha\eta + \alpha d$ and for all $0 < \delta \leq \alpha d$, $(\alpha\eta + \alpha\delta)(x)$ satisfies $I_\alpha(q)$.

Hence, $\sim$ is a simulation relation from (T_A, L) to $(T_{\alpha A}, L_\alpha)$. Also, we can prove that the inverse of $\sim$ is a simulation relation from $(T_{\alpha A}, L_\alpha)$ to (T_A, L) . So, we conclude that (T_A, L) is bisimilar to $(T_{\alpha A}, L_\alpha)$. $\square$

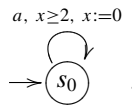
Since all constants c are assumed to be rational, there exists an integer α , say the common multiple of their denominators, such that all constants in αA are integers. Therefore, we may assume without loss of generality that all constants are integers. Let c_i denote the largest integer constant with which x_i is compared.

In the following, we are going to show that there exists a self-bisimulation relation R for (T_A, L) such that its quotient system has finite states.

First, we notice that the specific valuation of a clock, say x_1 , is not relevant if it is beyond the maximum integer c_1 that it is compared with, since all atomic clock constraints that the clock variable x_1 is involved in will return the same true or false for all valuations $\eta(x_1)$ that are strictly larger than c_1 . In addition, since atomic clock constraints only compare a clock variable x_i with an integer, so the truth value of an atomic clock constraint only depends on the integer value of the clock valuation, denoted as $\lfloor \eta(x_i) \rfloor$. For example, $\lfloor 1.23 \rfloor = 1$, $\lfloor 3.14 \rfloor = 3$, and $\lfloor 2 \rfloor = 2$.

These observations inspire us to partition the space of clock valuations based on their integer part. We illustrate this idea using a simple example.

Example 3.32 Consider a simple timed automaton:



We can partition the space of clock valuations $\{x \geq 0\}$ into the following regions:

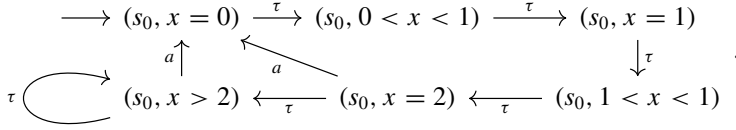
$$\{ \{0\}, \{0 < x < 1\}, \{1\}, \{1 < x < 2\}, \{2\}, \{x > 2\} \}.$$

Namely, we partition the space of clock valuations just based on the integer part of the clock value x . Since, the maximum integer compared with the clock x is 2 (in the guard condition $x \geq 2$), we stop partitioning after 2 since all clock valuations $x > 2$ will return the same results when compared with 2. The proposed partition based on integer part of the clock valuations clearly gives us a finite number of clock regions as shown below.



The next question is whether all the clock valuations from the same partition behave equivalently (in the sense of time-abstract bisimilarity). The regions only contain a single clock valuation that is trivial, so we only check for regions with more than one valuation. First, for the region $\{x > 2\}$, any two different clock valuations from this region both satisfy the guard condition $x \geq 2$, and therefore behave similarly. Secondly, for any two different clock valuations from the region $\{1 < x < 2\}$, they both can let time elapse (may be of different duration) and then trigger the discrete transition. Similar arguments can be applied to the region $\{0 < x < 1\}$. Therefore, it is not hard to see that the proposed partition just based on the integer part of the

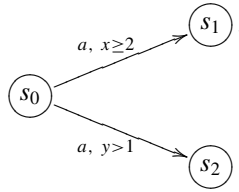
clock valuations actually works in the sense that it gives a finite number of equivalent clock regions, upon which we can build a finite quotient transition system that is time-abstract bisimilar to the original timed automaton.



□

Notice that the partitions based on integer parts of clock valuations gives us a finite number of equivalent classes, and seem to work for the simple example with one clock variable above. However, a simple extension to timed automata with two clock variables fails as illustrated in the following example.

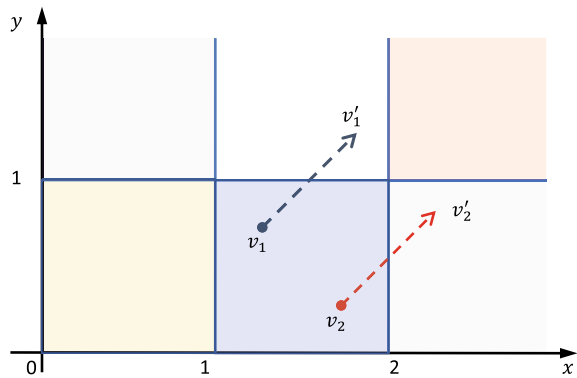
Example 3.33 Consider a simple timed automaton:



If we still use the integer part of clock valuations to partition the whole space of the clock valuations, we can obtain the partition as illustrated in Fig. 3.9.

Consider two clock valuations $v_1 = (1.2, 0.8)^T$ and $v_2 = (1.8, 0.2)^T$, which are both in the equivalent class based on our first proposed partition just based on integer parts. However, time elapse $\tau = 0.4$ will drive v_1 to $v'_1 = (1.6, 1.2)^T$ and $v'_2 = (2.2, 0.8)^T$. Then, the discrete transition $s_0 \xrightarrow{a} s_2$ could happen at state (s_0, v'_1) ,

Fig. 3.9 Timed automaton example



but the transition $s_0 \xrightarrow{a} s_1$ could happen at the state (s_0, v'_2) instead. In other words, the system actually behaves differently starting from two “equivalent” states. $\square$

The main reason why it fails is because the fractional parts of clock valuations actually matter. More precisely it is the ordering of the fractional parts among different clock valuations that matters since the ordering will determine which clock constraints associated with discrete transitions (a.k.a. guard conditions) are going to become true after certain time delays. In our example, we can see this clearly that the ordering of fractional parts of the clock x_1 and clock x_2 are different in v_1 and v_2 —the fractional part of $v_1(x_1)$ is strictly greater than the fractional part of $v_1(x_2)$ while the ordering is reversed in v_2 . To fix this bug, we need to refine our partition by further requesting that the ordering of fractional parts of clock valuations should be the same.

Putting all these observations together, we have the following proposal. Consider the relationship R ; we say $((q, \eta), (q', \eta')) \in R$, if the following hold:

1. the discrete locations must agree, i.e., $q = q'$;
2. if there exists $x_i \in \mathbb{C}$ such that $\eta(x_i) > c_i$, then $\eta'(x_i) > c_i$, and vice versa;
3. otherwise, for the clocks $x_i, x_j \in \mathbb{C}$ with $\eta(x_i), \eta'(x_i) \leq c_i$, and $\eta(x_j), \eta'(x_j) \leq c_j$, the following conditions hold:
 - (a) the integer parts coincide, namely $\lfloor \eta(x_i) \rfloor = \lfloor \eta'(x_i) \rfloor$;
 - (b) if the fractional part $\langle \eta(x_i) \rangle = 0$, then $\langle \eta'(x_i) \rangle = 0$, and vice versa;
 - (c) if $\langle \eta(x_i) \rangle \leq \langle \eta(x_j) \rangle$, then $\langle \eta'(x_i) \rangle \leq \langle \eta'(x_j) \rangle$, and vice versa.

Here we use the following notation. For any $\delta \geq 0$, $\langle \delta \rangle$ denotes the fractional part of δ , while $\lfloor \delta \rfloor$ the integer part, and $\delta = \lfloor \delta \rfloor + \langle \delta \rangle$. For example, $\lfloor 2.8 \rfloor = 2$ and $\langle \delta \rangle = 0.8$. Two clock values η and η' are called equivalent, denoted as $\eta \approx \eta'$, if there exists $q \in Q$ such that $((q, \eta), (q, \eta')) \in R$. The collection of all equivalent clock values is called a *clock region*, denoted by $[\eta] = \{\eta' \mid \eta \approx \eta'\}$. It is seen in the following example that clock regions are either open triangles, open line segments, open parallelograms, or points.

Example 3.34 Consider a timed automaton with a single location and two clocks $\mathbb{C} = \{x_1, x_2\}$. It is assumed that the maximum constants being compared with for clock x_1 is $c_1 = 2$, and $c_2 = 1$ for clock x_2 . Our task is to partition the first quadrant in $\mathbb{R}^2$ into clock regions.

First, the region $\{x_1 > 2, x_2 > 1\}$ satisfies Condition 2, and Condition 3 is not applicable. So, $\{x_1 > 2, x_2 > 1\}$ is a clock region.

Next, the region $\{x_1 > 2, 0 < x_2 < 1\}$ satisfies Condition 2 as all x_1 clock values are greater than c_1 in this region. To check Condition 3, we only need to check for clock x_2 . For any pair of clock values within the range $0 < x_2 < 1$, the integer parts are both 0, while the order requirement on the fraction part Condition 3(c) trivially holds since $x_i = x_j = x_2$ here. Hence, the region $\{x_1 > 2, 0 < x_2 < 1\}$ is a clock region. Following similar arguments, we can see that regions $\{0 < x_1 < 1, x_2 > 1\}$ and $\{1 < x_1 < 2, x_2 > 1\}$ are both clock regions, respectively.

adjacent triangle regions or the diagonal line segment, while Condition 3(b) excludes the line segments adjacent to it.

In summary, the clock regions are either single node points, open triangles, line segments, or open rectangles as depicted in Fig. 3.10 for this example. $\square$

Next, we are going to prove that it is a self-bisimulation relationship for the time-abstract transition system T_A of a timed automaton A .

Since the clock constants are all assumed to be integers, the clock valuations η and η' will satisfy the same clock constraints if they belong to the same clock region. Hence, for any $((q, \eta), (q', \eta')) \in R$ we have $L(q, \eta) = L(q', \eta')$. Also, clock values η and η' from the same clock region remain equivalent after resetting. That is, if $\eta \approx \eta'$ then we have $\eta[\lambda := 0] \approx \eta'[\lambda := 0]$ for any subset of clocks $\lambda \subseteq \mathbb{C}$. This is because the guard sets are formed by the line segments or nodes, while the resetting operation corresponds to a transition from the line segment to a line segment along the x_i -axis or the origin, which are all clock regions in both cases. Next, we prove that the binary relation R defined above is a bisimulation relation.

Theorem 3.1 *R is a self-bisimulation relation for the time-abstract transition system T_A of a timed automaton A .*

Proof First, for any $((q_1, \eta_1), (q_2, \eta_2)) \in R$ we have $L(q_1, \eta_1) = L(q_2, \eta_2)$.

Next, we show that any transition emanating from (q_1, η_1) can be mimicked by (q_2, η_2) . For such a purpose, we consider discrete and τ transitions separately.

For a discrete transition from (q_1, η_1) , say $(q_1, \eta_1) \xrightarrow{e} (q'_1, \eta'_1)$ in T_A , then the transition $(q_1, e, \phi, \lambda, q'_1)$ exists in A with $\eta_1 \models \phi$ and $\eta'_1 = \eta_1[\lambda := 0]$. Since, $((q_1, \eta_1), (q_2, \eta_2)) \in R$, $q_1 = q_2$ and $\eta_2 \models \phi$. Therefore, $(q_2, \eta_2) \xrightarrow{e} (q'_1, \eta_2[\lambda := 0])$ is defined in T_A . Also, since $\eta_1 \approx \eta_2$ then $\eta_1[\lambda := 0] \approx \eta_2[\lambda := 0]$. So, $((q'_1, \eta'_1), (q'_1, \eta_2[\lambda := 0])) \in R$.

For a τ transition in T_A , that is, for any $((q_1, \eta_1), \tau, (q_1, \eta'_1)) \in \rightarrow$ in T_A , there exists $d \geq 0$ such that $\eta'_1 = \eta_1 + d$ and for all $0 \leq \delta \leq d$, $(\eta_1 + \delta)(x)$ satisfies $I(q_1)$. Due to the continuity of time drift, we can pick d' for any d such that $\eta_1 + d \approx \eta_2 + d'$ and for all $0 \leq \delta' \leq d'$ $(\eta_2 + \delta')$ remains in $[\eta_2]$. Also, since $\eta_2 \approx \eta_1$, $(\eta_2 + \delta') \models I(q_1)$ for all $0 \leq \delta' \leq d'$. Hence, $((q_2, \eta_2), \tau, (q_2, \eta_2 + d'))$ is defined in T_A , and $((q_1, \eta'_1), (q_2, \eta_2 + d')) \in R$. $\square$

Region Transition System

Since R is a self-bisimulation relation for (T_A, L) , then we can derive a quotient transition system using R that is bisimilar to (T_A, L) . In particular, the quotient transition system can be obtained as

$$T_A/R = (Q_A/R, Q_A^0/R, \mathcal{E} \cup \{\tau\}, \rightarrow_R),$$

where

- $Q_{A/R} = \{(q, [\eta]) \mid (q, \eta) \in Q_A\}$;
- $Q_{A/R}^0 = \{(q_0, [\eta]) \mid (q_0, \eta) \in Q_A^0\}$;
- the transition relation $\rightarrow_R$ is defined as
 - τ transition: For $q \in Q$, $((q, [\eta]), \tau, (q, [\eta'])) \in \rightarrow_R$ if for all $\eta \in [\eta]$ there exists $d > 0$ such that $\eta' = \eta + d$ and for all $0 \leq \delta \leq d$, $(\eta + \delta)(x) \in [\eta] \cup [\eta'] \subseteq I(q)$.
 - Discrete transitions: For an $e \in \mathcal{E}$, $((q, [\eta]), e, (q', [\eta'])) \in \rightarrow_R$ if for any $\eta \in [\eta]$ there exists $\eta' \in [\eta']$ such that $((q, \eta), e, (q', \eta')) \in \rightarrow$ in T_A .

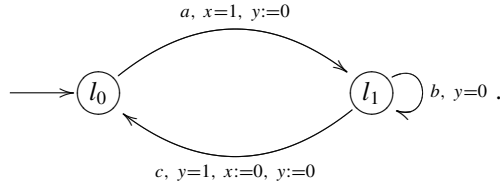
The label for the quotient transition system $T_{A/R}$ is a map $L_R : Q_{A/R} \rightarrow 2^{\mathcal{P}} \cup 2^{ACC}(\mathbb{C})$ and is defined as

$$L_R(q, [\eta]) = L(q, \eta).$$

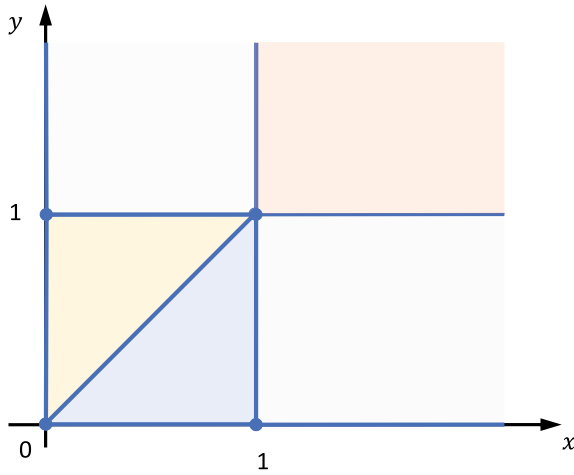
Note that for any $\eta_1, \eta_2 \in [\eta]$, $L(q, \eta_1) = L(q, \eta_2)$. So, the above definition is consistent.

The quotient transition system $T_{A/R}$ is using the equivalent clock regions as its states, so it is called a *region transition system* of the timed automaton. Since the clock region is finite, $T_{A/R}$ is a finite transition system. This is significant since we know how to do model checking on finite state machines and algorithms always terminate. Combining this fact with our knowledge that CTL* preserves in such cases bisimilar transition systems, we can do CTL* or CTL model checking for a timed automaton on its corresponding region transition system, as long as the CTL* or CTL formula is arguing over atomic propositions $\mathcal{P}$ defined for discrete locations.

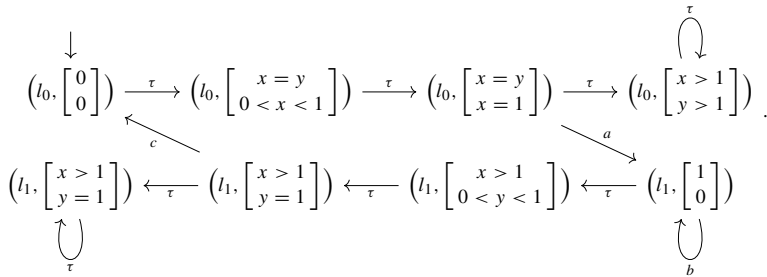
Example 3.35 Consider a timed automaton A below



The clock regions can be obtained as



Accordingly, we can obtain its region transition system as



□

Finally, let's establish a bound on the number of clock regions. Assume that there are n clock variables in a timed automaton A , which is denoted by $\{x_1, x_2, \dots, x_n\}$. Without loss of generality, we also assume that all constants that appeared in the timed automaton A are all integers, and the maximum integer constant compared with the clock x_i is c_i . Given a clock region, the clock variable x_i can take values either at the integer points, $\{0, 1, 2, \dots, c_i\}$, or insider one of the following intervals: $\{(0, 1), (1, 2), \dots, (c_i - 1, c_i), (c_i, \infty)\}$. This gives us total $2c_i + 2$ choices for clock variable x_i . Combining all clock variables, we have $\prod_{i=1}^n (2c_i + 2)$ possibilities, which is due to the integer part of clock variables. Now, let's consider the fractional part of clock variables. First, there are $n!$ number of possible orderings of the clock variables. Second, for every pair of adjacent clock variables x and y in one of such ordering, whether the fractional part of x is strictly less than the fractional part of y or coincides will give distinct clock regions. This will give additional 2^n choices. Putting all these together, the total number of clock regions is at most $2^n \cdot n! \cdot \prod_{i=1}^n (2c_i + 2)$.

3.5.5 Extensions of Timed Automata

In this subsection, we discuss extensions of the abstraction results for timed automata to more general hybrid system models.

Multi-rate Timed Automata

Multi-rate timed automata [10] are generalizations of timed automata, where each variable follows constant, rational slopes, which may be different in different locations.

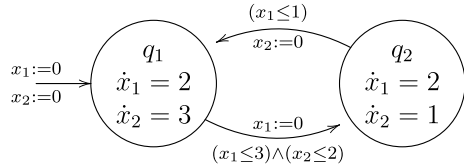
Definition 3.23 A multi-rate timed automaton is a hybrid automaton $H = (Q, X, Init, f, Inv, E, G, R)$, where

- Q is a set of discrete variables, $Q = \{q_1, \dots, q_m\}$;
- $X = \begin{bmatrix} x_1 \\ \dots \\ x_n \end{bmatrix}$, $\mathbf{X} = \mathbb{R}^n$;
- for each location $q \in Q$, the set $Init(q)$ is either empty or a singleton set;
- $f(q, x) = b_q$, where b_q is a constant vector with all rational components;
- $Init(q)$ is a rectangle for all q ;
- $E \subseteq Q \times Q$;
- the guard set $G(e)$ is a rectangle⁴ for all $e = (q, q') \in E$;
- for all $e \in E$, the reset map $R(e, x) = R_1(e, x) \times \dots \times R_n(e, x)$, where $R_i(e, x)$ either equals x_i (unchanged) or a constant (the i th component of x is reset to be a constant).

An example of a multi-rate automaton is shown in Fig. 3.11.

A multi-rate timed automaton is called *initialized* if the bounds on the derivative of a variable change after a discrete transition, then its value must be non-deterministically reset (“reinitialized”) within a fixed interval. Mathematically, an initialized multi-rate timed automaton requests that if for all transitions $e = (q, q') \in E$ and all $1 \leq i \leq n$, $R_i(e, x) = \{x_i\}$, then $f_i(q, \cdot) = f_i(q', \cdot)$. It can be shown that initialized multi-rate timed automata have finite bisimulation quotient systems [10]. The

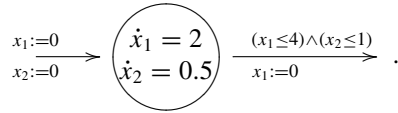
Fig. 3.11 Multi-rate timed automata example



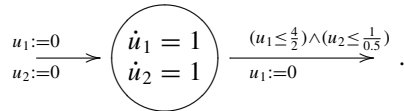
⁴ A set $R \subset \mathbb{R}^n$ is called a rectangle if $R = \prod_{i=1}^n R_i$ where R_i are bounded or unbounded intervals whose finite end points are rational. For example, the set $R = \prod_{i=1}^3 R_i$ with $R_1 = (1, \infty)$, $R_2 = \{3\}$, $R_3 = (-2, \frac{3}{4})$ is a rectangle.

main idea behind this is to re-scale the slope of each variable to 1 so as to convert the multi-rate timed automaton into a timed automaton. In addition, it is necessary to appropriately adjust all initial, invariant and guard sets, as well as reset maps. From the region equivalence of the resulting timed automaton, a bisimulation of the multi-rate timed automaton can be obtained.

Example 3.36 To illustrate the idea, we consider a part of a multi-rate timed automaton, namely



It can be converted into a timed automaton by normalizing b_q in to a vector of all 1's.



□

Hence, the LTL and CTL model checking problems for an initialized multi-rate timed automaton can be solved through the corresponding region transition system that has a finite number of states, provided that every position occurring in temporal formulas is either an automaton location or a rectangular set. Hence, the LTL and CTL model checking problem for an initialized multi-rate timed automaton can terminate in a finite number of steps (called decidable) [10].

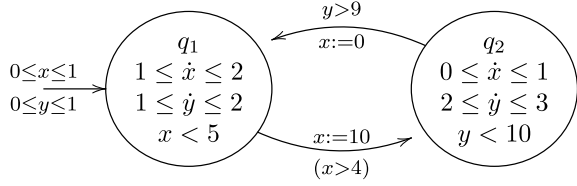
Rectangular Automata

A rectangular automaton is a special case of a hybrid automaton where all initial regions, invariant sets, vector fields, guard sets, and reset maps are rectangular. Formally,

Definition 3.24 A *rectangular automaton* is a hybrid automaton $H = (Q, X, Init, f, Inv, E, G, R)$, where

- Q is a set of discrete variables, $Q = \{q_1, \dots, q_m\}$;
- $X = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$, $\mathbf{X} = \mathbb{R}^n$;
- $f(q, x) = F(q)$ for all (q, x) , where $F(q) = F_1(q) \times \dots \times F_n(q)$ is a rectangle;
- $Init(q)$ and $Inv(q)$ are all rectangles for all q ;

Fig. 3.12 Rectangular automata example



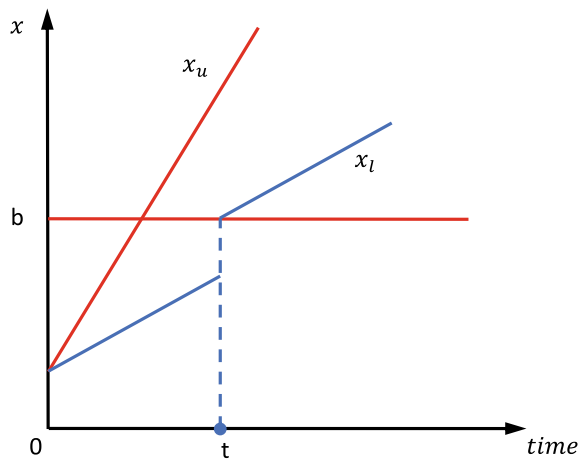
- $E \subseteq Q \times Q$;
- $G(e) = G_1(e) \times \cdots \times G_n(e)$ is a rectangle for all $e = (q, q') \in E$;
- for all $e \in E$, the reset map $R(e, x) = R_1(e, x) \times \cdots \times R_n(e, x)$, where $R_i(e, x)$ either equals x_i (unchanged) or a fixed interval (the i th component of x is reset to any value within this interval).

In other words, the derivative of each variable stays between two fixed bounds, which may be different in different discrete modes, i.e., $l_q(i) \leq \dot{x}_i \leq u_q(i)$, for all $1 \leq i \leq n$ and $q \in Q$. With each discrete transition between two modes $(q, q') \in E$, the value of variable x_i is either left unchanged $R_i(e, x) = \{x_i\}$ or reset non-deterministically to a new value within some fixed, constant interval when $R_i(e, x)$ is a fixed interval. An example of a rectangular automaton is shown in Fig. 3.12.

A rectangular automaton is called *initialized* if after a discrete transition, the bounds on the derivative of a variable change, then its value must be non-deterministically reset (“reinitialized”) within a fixed interval. Mathematically, an initialized rectangular automaton requests that if for all transitions $e = (q, q') \in E$ and all $1 \leq i \leq n$, $R_i(e, x) = \{x_i\}$ then $F_i(q) = F_i(q')$.

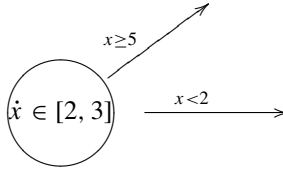
An initialized rectangular automaton can be converted into a language equivalent initialized multi-rate timed automaton. The main idea is to replace each variable x , which satisfies a differential inclusion of the form $\dot{x} \in [l, u]$ by two variables named x_l and x_u , which satisfy $\dot{x}_u = u$ and $\dot{x}_l = l$, respectively. The variables x_l and x_u keep track of the lower and upper bounds of x . The initial, invariant, and guard sets, as well as the reset maps, must be adjusted accordingly. For example, as shown in Fig. 3.13, the guard condition $x \geq b$ is translated to $x_u \geq b$, and the lower bound x_l

Fig. 3.13 Approximate the evolution of a rectangular automaton by its upper and lower bounds

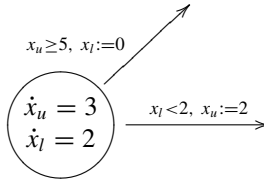


is reset to b after this transition. Similar translations can be adopted for the guard condition $x \leq a$ as the following example shows.

Example 3.37 To illustrate the idea, we consider below a part of a rectangular automaton:



It can be converted into a multi-rate timed automaton by introducing two variables x_u and x_l , representing the upper and lower bounds of x , respectively:



It then can be converted into a timed automaton. □

This conversion from a rectangular to a multi-rate timed automaton is language preserving. Hence, from the existence of finite bisimulation for an initialized multi-rate timed automaton, we can construct a finite language equivalence of the original initialized rectangular automaton.

Since we can obtain a language equivalent finite transition system for an initialized rectangular automaton, its LTL model checking problem can terminate in a finite number of steps (also known as decidable). However, the conversion from a rectangular to a multi-rate timed automaton may not preserve branching properties, and in general, initialized rectangular automata do not admit finite bisimulation quotients.

3.5.6 Zone Automata and Symbolic Reachability Analysis

We know that the number of states in the region transition system could be much larger than the number of states in the original timed automaton, which implies a potential explosion in the computation complexity. In fact, the number of regions is exponential in the number of clocks as well as the maximal constants appearing in the guards of a timed automaton [13]. For model checking, such fine partition may not be necessary and the computation is often prohibitively expensive to exhaustively

explore the state space. Therefore, the region transition systems are not widely used in practice. In this section, we will introduce an alternative practical technique for reachability analysis of timed automata that have been implemented in major timed automata-based model checking tools such as UPPAAL [17] and KRONOS [18].

Zone Automata

The main idea is that instead of considering clock regions whose number can easily explode, we try to obtain efficient abstract systems of timed automata with *clock zones*, which group clock regions into convex sets to get a coarser partition of the state space. A *clock zone* is a set of constraints each of which puts a lower or upper bound on a clock or on the difference of two clocks. For example, let x_1 and x_2 be two clocks, the following is a clock zone.

$$(0 \leq x_1 < 3) \wedge (2 < x_2 < 4) \wedge (x_1 - x_2 \leq 0).$$

Note that each clock region is also a clock zone by definition. For reachability analysis, the following three operations are defined for clock zones:

- for two clock zones z_1 and z_2 , $z_1 \wedge z_2$ denotes the intersection of the two clock zones. It can be used to capture the guard or invariant condition;
- for a clock zone z , $z^\nearrow$ denotes the set of interpretations $\eta + \delta$ for $\eta \in z$ and $\delta \in \mathbb{R}_{>0}^+$. This can simply be understood as letting time pass;
- for a subset λ of clocks and a clock zone z , $z[\lambda := 0]$ denotes the set of clock interpretations $\eta[\lambda := 0]$ for $\eta \in z$. This operation is equivalent to a clock reset.

A crucial and easy-to-check property of clock zones is that it is closed under the above three operations, which is left as an exercise. Define *zone* as a pair (q, z) for a state $q \in Q$ of the timed automata A and a clock zone z . Consider a transition t of A where $t = (q, e, \varphi, \lambda, q')$, and denote $\text{succ}(z, t)$ as the set of clock interpretations v' such that for some $v \in z$, the state (q', v') is reachable from (q, v) by executing the transition t and letting time elapse in the new location while satisfying the invariant condition. In other words, the set $(q', \text{succ}(z, t))$ is the successor of zone (q, z) under transition t . With the three operations on clock zones defined above, the computation of $\text{succ}(z, t)$ can be carried out as follows:

$$\text{succ}(z, t) = ((z \wedge \varphi)[\lambda := 0] \wedge I(q'))^\nearrow \wedge I(q'). \quad (3.13)$$

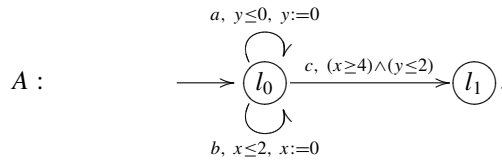
From the closeness of the operations, $\text{succ}(z, t)$ is also a clock zone which means that there is an edge between zones (q, z) and $(q', \text{succ}(z, t))$. Formally, a transition system called *zone automaton* whose states are zones that can be defined as follows.

Definition 3.25 Given a timed automaton $A = (Q, Q^0, \mathcal{E}, \mathbb{C}, I, \rightarrow)$, its associated zone automaton Z_A is a transition system $(Q_z, Q_z^0, \varepsilon, \rightarrow_z)$ where

- Q_z are zones of A ;
- $(q, ([X := 0]^\nearrow \wedge I(q))) \in Q_z^0, \forall q \in Q^0$;
- $((q, z), e, (q', succ(z, t))) \in \rightarrow_z$ for every $t = (q, e, \varphi, \lambda, q') \in \rightarrow$ and every clock zone z .

Note that theoretically, the number of zones is exponential in the number of clock regions since a zone is a set formed by the convex union of clock regions. However, in practice the number of reachable states in zone automata is much smaller, which leads to improved performance in model checking.

Example 3.38 [42] Consider the following timed automaton:



We can obtain the following zones, starting from the initial zone $Z_0 = (l_0, x = y = 0)$

$$\begin{aligned}
 Z_0 &= (l_0, x = y = 0) \rightarrow_z Z_1 = (l_0, x = y \geq 0) \\
 &\xrightarrow{a}_z Z_2 = (l_0, x \leq 2 \wedge y = 0) \\
 &\rightarrow_z Z_3 = (l_0, y \leq x \leq y + 2) \\
 &\xrightarrow{a}_z Z_4 = (l_0, x \leq 4 \wedge y = 0) \\
 &\rightarrow_z Z_5 = (l_0, y \leq x \leq y + 4) \\
 &\xrightarrow{c}_z Z_6 = (l_1, y \leq 2 \wedge 4 \leq x \leq y + 4).
 \end{aligned}$$

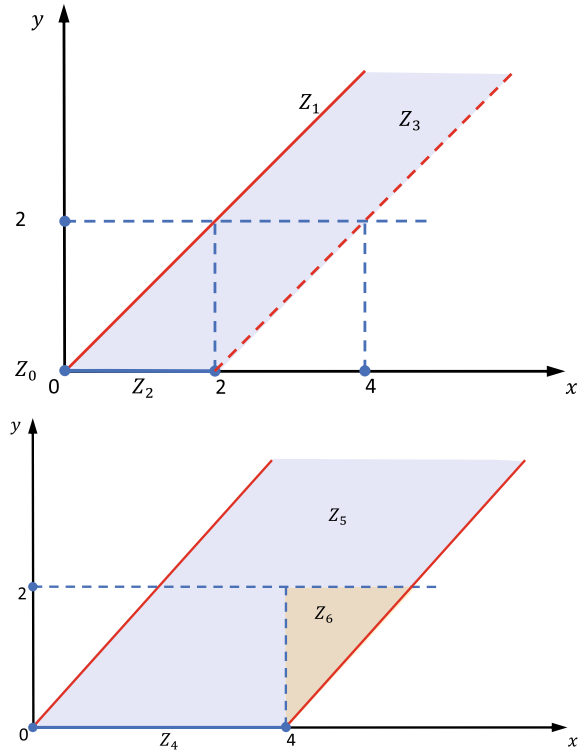
A graphical illustration of these zones is given in Fig. 3.14. □

Difference-Bound Matrices

To construct the zone automaton, one must be able to derive the successive clock zones after a transition according to (3.13). A very efficient way to deal with clock zone computation is to represent it using difference-bound matrices (DBM). For simplicity, we only consider the case where the inequalities are *non-strict*, i.e., $x \sim n, x - y \sim m$ where $\sim \in \{\leq, \geq\}$.

Suppose the timed automaton A has n clocks, $x_1, \dots, x_n$. Let's introduce a dummy clock x_0 that is always 0. Then there is a one-to-one correspondence between a clock zone and the DBM $R \in \mathbb{R}^{(n+1) \times (n+1)}$. For every pair i, j , the entry R_{ij} denotes the upper bound of $x_i - x_j$. Note that $R_{ii} = 0$ for all i by definition. For example, consider the clock zone below

Fig. 3.14 Graphical illustrations of the zone in Example 3.38



$$(3 \leq x_1 \leq 7) \wedge (0 \leq x_2) \wedge (3 \leq x_1 - x_2 \leq 5),$$

which can be expressed by matrix R of the following form:

$$R = \begin{bmatrix} 0 & -3 & 0 \\ 7 & 0 & 5 \\ \infty & -3 & 0 \end{bmatrix}. \quad (3.14)$$

Since the clock zone and DBMs are equivalent, in the rest of this section we do not explicitly distinguish them. Observe that this R does not capture many implied constraints. For instance, $R_{2,0} = \infty$ which implies $x_2 \leq \infty$ but since $x_1 \leq 7$ and $x_1 - x_2 \geq 3$, apparently this bound can be further tightened. The procedure to obtain the DBM R with the tightest possible constraints given a clock zone is called *canonicalization* and such R is said to be *canonical*. The general canonicalization step is as follows:

$$\text{if } R_{i,k} > R_{i,j} + R_{j,k} \text{ then } R_{i,k} = R_{i,j} + R_{j,k}.$$

The canonicalization is to apply the above rule repeatedly for all entries i, j with all $0 \leq k \leq n$. Formally, a matrix R is said to be canonical if and only if all

$0 \leq i, j, k \leq n, R_{i,k} \leq R_{i,j} + R_{j,k}$. Performing canonicalization to (3.14), we can get the following:

$$R' = \begin{bmatrix} 0 & -3 & 0 \\ 7 & 0 & 5 \\ 4 & -3 & 0 \end{bmatrix}. \quad (3.15)$$

It is easy to verify that R' is canonical. An alternative way to understand the canonicalization process is to consider a graph with $n + 1$ vertices corresponding to $x_0, x_1, \dots, x_n$, and each entry R_{ij} denotes the cost that is associated with the edge from vertex x_i to x_j . The process of canonicalization is nothing but computing the smallest cost for directed paths between every two vertices in this graph.

Another question that can be addressed by canonicalization is to decide whether the conjunction of all the clock constraints is satisfiable. It has been shown that when the DBM R represents unsatisfiable constraints, negative diagonal entries will appear in the canonicalization process. Note that a negative diagonal entry implies $x_i - x_i < 0$ which is not possible. For example, suppose we have the following clock zone

$$0 \leq x_1 \leq 5, 0 \leq x_2 \leq \infty, 6 \leq x_1 - x_2 \leq 10.$$

The corresponding R is

$$R = \begin{bmatrix} 0 & 0 & 0 \\ 5 & 0 & 10 \\ \infty & -6 & 0 \end{bmatrix}.$$

Canonicalization first compares R_{01} to $R_{02} + R_{21}$ and changes R_{01} to -6 . Then it compares R_{11} to $R_{10} + R_{01}$ and changes R_{11} to -1 and the matrix R' we get here is

$$R' = \begin{bmatrix} 0 & -6 & 0 \\ 5 & -1 & 10 \\ \infty & -6 & 0 \end{bmatrix}.$$

Note that there is a negative diagonal entry in R' which means the original constraints are unsatisfiable. The general canonicalization algorithm is summarized in Algorithm 3.2 below.

Algorithm 3.2 (Canonicalization Algorithm)

input DBM R with dimension $(n + 1) \times (n + 1)$

output Check if constraints represented by R are satisfiable, and if so, return canonical version of R

for $k = 0$ to n **do**

for $i = 0$ to n **do**

for $j = 0$ to n **do**

$R_{ij} = \min(R_{ij}, R_{ik} + R_{kj})$

end for

if $R_{ii} < 0$ **then**

```

    return "Unsatisfiable"
  end if
end for
end for

```

The following operations on DBMs are very useful in model checking reachability of a timed automata:

- **Intersection:** The intersection operation is used to capture the invariant and guard conditions. Consider two DBMs R and R' ; the (i, j) th entry of the resulting clock zone R'' is simply the minimum of R_{ij} and R'_{ij} . Then R'' will be canonicalized according to Algorithm 3.2.
- **Time elapse:** Given a DBM R , to characterize the reachable clock set starting from R , we simply set upper bounds on all clocks, that is, the elements in the first column other than x_0 , to ∞ .
- **Subset test:** Let R and R' be two canonical and non-empty DBMs, then the clock zone z represented by R is a subset of the clock zone z' represented by R' if and only if $R_{ij} \leq R'_{ij}$ for every $0 \leq i, j \leq n$. if $R \subseteq R'$ and $q = q'$, then we denote $(q, z) \subseteq (q', z')$.
- **Clock reset:** Given a DBM R and a clock x_j to reset, the clock reset operation is defined in Algorithm 3.3. Such implementation preserves the canonical form if the input is canonical.

Algorithm 3.3 (Reset Algorithm)

input DBM R in canonical form with dimension $(n + 1) \times (n + 1)$ and a clock x_j to be reset

output Canonical version of R with x_j reset to zero

for $i = 0$ to n **do**

$R_{ji} = R_{0i}$

$R_{ij} = R_{i0}$

end for

Now we are ready to illustrate the reachability analysis using clock zones by going through the following example.

Example 3.39 Consider a timed automaton A as shown in Fig. 3.15. We are interested in building the corresponding zone automaton Z_A for reachability analysis.

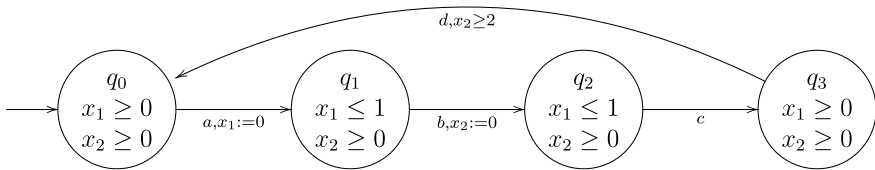


Fig. 3.15 A timed automaton

According to Definition 3.25, we first need to find its initial state. Since $Q^0 = \{q_0\}$, our initial state is of the form $q_z^0 = (q_0, z)$ where $z = [X := 0]^{\nearrow} \wedge I(q_0)$.

Let's get z step by step. First, let $R = [X := 0]$ or equivalently in DBM form

$$R = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Next, we need to find $R^{\nearrow}$ by following the time elapse operation of DBM,

$$R^{\nearrow} = \begin{bmatrix} 0 & 0 & 0 \\ \infty & 0 & 0 \\ \infty & 0 & 0 \end{bmatrix}.$$

Then we will calculate $R^{\nearrow} \wedge I(q_0)$. Write $I(q_0)$ in DBM form denoted as R' . Note that $I(q_0) = \{x \geq 0, y \geq 0\}$ and we denote any unspecified bound as ∞ .

$$R' = \begin{bmatrix} 0 & 0 & 0 \\ \infty & 0 & \infty \\ \infty & \infty & 0 \end{bmatrix}.$$

Now we get two clock zones in DBM form, $R^{\nearrow}$ and R' ; following the intersection operation of DBM, we have

$$z = R^{\nearrow} \wedge R' = \begin{bmatrix} 0 & 0 & 0 \\ \infty & 0 & 0 \\ \infty & 0 & 0 \end{bmatrix}.$$

Now we need to find the next state $q_z^1 = (q', succ(z, t))$. From the timed automaton A , $t = (q_0, a, \{X \in \mathbb{R}^2\}, \{x_1\}, q_1)$. Therefore, according to Definition 3.25, the next state is of the form (q_1, z') where $z' = succ(z, t)$. Calculation of z' is defined in (3.13) and is implemented in DBM operations. The resulting z' is as shown below after canonicalization:

$$z' = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ \infty & \infty & 0 \end{bmatrix}.$$

Note that since $q_0 \neq q_1$, we don't need to check if q_z^1 is contained in q_z^0 . The construction process goes on until either the specification is found to be satisfied or the zone automaton has been found (in this case, the model checker returns false). In this example, the found zone automaton is shown in Fig. 3.16.

Note that it only has 5 states while the region transition system of the same timed automaton has 28 clock regions. Also note that for q_z^4 , its successive state is found to be (q_1, z'') where

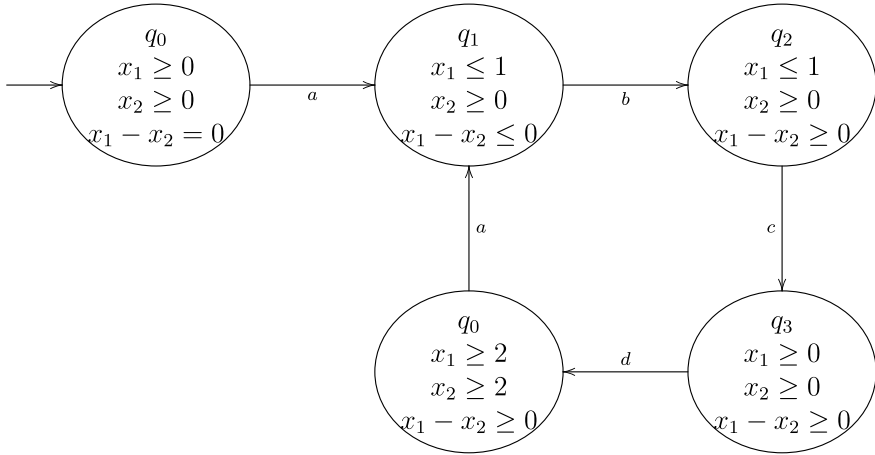


Fig. 3.16 Zone automaton

$$z'' = \begin{bmatrix} 0 & 0 & -2 \\ 1 & 0 & -1 \\ \infty & \infty & 0 \end{bmatrix}.$$

From the subset test, it is clear that $z'' \subseteq z'$, therefore $(q_1, z'') \subseteq q_z^1$ and q_z^4 transits to q_z^1 . $\square$

3.6 Linear Hybrid Automata

In timed automata, which are special cases of hybrid automata, the continuous flow rates are unit constants for all continuous variables (clocks). This may be restrictive in real applications, and it has motivated researchers to consider more general continuous dynamics at each mode. In particular, we consider linear hybrid automata, which can be seen as extensions of timed automata; they are also special cases of general hybrid automata [11].

3.6.1 Linear Hybrid Automata

Before we define linear hybrid automata, some preliminary notation is needed.

- Consider a finite set of continuous variables $X = \{x_1, x_2, \dots, x_n\}$, where each x_i is real-valued, and a valuation is a map from X to $\mathbb{R}^n$.

- We use $\dot{x}_i$ to represent the first derivative of the variable x_i with respect to time, and use $\dot{X}$ to represent the collection of all these first derivatives.
- The primed version of the variable x_i , namely x'_i , is used to present the new state valuations after a discrete transition. Accordingly, we use X' to represent the collection of all such primed versions of variables.

Definition 3.26 An *atomic affine formula* is an inequality between a rational constant and a linear combination of variables with rational coefficients, for example, $3x_i + 5x_j - 2x_k \leq 1/2$, where $x_i, x_j, x_k \in X$.

Definition 3.27 A *convex affine formula* is a finite conjunction of atomic affine formulas.

Definition 3.28 An *affine formula* is a finite disjunction of convex affine formulas.

A convex polyhedron is a subset of $\mathbb{R}^n$ that is the intersection of a finite number of atomic affine formulas. A polyhedron is a subset of $\mathbb{R}^n$ that is the union of a finite number of convex polyhedrons. Given a set of continuous variables $X = \{x_1, x_2, \dots, x_n\}$, the set of all convex polyhedrons on X is denoted by $CPoly(X)$, and the set of all polyhedrons on X is denoted by $Poly(X)$.

Now we are ready to define linear hybrid automata. Recall that the general hybrid automaton H is a collection, $H = \{Q, X, f, Init, Inv, E, G, R\}$.

Definition 3.29 A hybrid automaton H is a *linear hybrid automaton* if the following conditions are satisfied:

- for every $q \in Q$, the initial condition $Init$, and the invariant condition $Inv(q)$ are convex affine formulas. For each discrete transition $e \in E$, the guard condition $G(e)$ is a convex affine formula;
- for every $q \in Q$, $x \in X$, the dynamic flow $f(q, x)$ is a convex affine formula over the variables in $\dot{X}$ only, denoted by $F(q)$. For example, $F(q)$ can be in the form of $\dot{x} = c$ where c is a vector of rational constants or $\dot{x} \in [a, b]$ where a, b are two vectors of rational constants.

The main reason for the interest in linear hybrid automata is that there exist automatic model checking techniques for such systems. Note that by definition, a timed automaton is a special case of linear hybrid automata where the dynamics $F(q) = 1$ for all $q \in Q$. One may argue that the second condition in linear hybrid automata is a bit restrictive. Indeed, it requires that the dynamics depend only on the discrete states, not the values of the variables.

3.6.2 Runs of Linear Hybrid Automata

There are two types of transitions for a linear hybrid automaton as follows:

- continuous transitions that only change the continuous part of the state while the discrete location remains unchanged, denoted as $(q, x) \xrightarrow{\delta, \xi(t)} (q, x')$, where $\delta > 0$ and $\xi : [0, \delta] \rightarrow \text{Inv}(q)$ represents a state trajectory satisfying the flow condition $F(q)$ and respects the invariant condition, namely $\xi(t) \models \text{Inv}(q)$ and $\dot{\xi}(t) \models F(q)$ (or equivalently $\xi(t) \in \text{Inv}(q)$ and $\dot{\xi}(t) \in F(q)$) for all $t \in [0, \delta]$;
- discrete transitions $(q, x) \xrightarrow{\alpha} (q', x')$, which happen instantaneously and change both the discrete location and continuous state following the guard conditions and reset maps, namely $x \models G(q, q')$ and $x' \in R(q, q', x)$.

A run of a linear hybrid automaton is an alternating series of continuous and discrete transitions in the following form:

$$(q_0, x_0) \xrightarrow{\delta_0, \xi_0(t)} (q_0, x'_0) \xrightarrow{\alpha_0} (q_1, x_1) \xrightarrow{\delta_1, \xi_1(t)} (q_1, x'_1) \xrightarrow{\alpha_1} \dots,$$

where $\xi_i(t)$ stands for a continuous state trajectory that satisfies the corresponding flow condition and invariant condition at q_i , namely $\dot{\xi}_i(t) \in F(q_i)$ and $\xi_i(t) \in \text{Inv}(q_i)$.

While $\xi_i(t)$ can be arbitrarily curved as long as the flow condition and invariant condition are respected, it can be shown that there always exists a linear trajectory as long as there exists a trajectory between two states in a linear hybrid automaton.

Lemma 3.2 *Let A be a linear hybrid automaton, and let q be a location of A . If there exists a continuous trajectory $\xi(t)$ at the location q such that $\xi(0) = x_0$ and $\xi(\delta) = x_1 \in \text{Inv}(q)$, namely $(q, x_0) \xrightarrow{\delta, \xi(t)} (q, x_1)$, then there exists a valid linear trajectory $\rho(t) = x_0 + t(\frac{x_1 - x_0}{\delta})$ in A at the location q , such that $(q, x_0) \xrightarrow{\delta, \rho(t)} (q, x_1)$.*

Proof Note that $\text{Inv}(q)$ is convex, and $x_0, x_1 \in \text{Inv}(q)$. Hence, the linear connection between x_0 and x_1 lies inside $\text{inv}(q)$. In other words, the linear function $\rho(t) = x_0 + t(\frac{x_1 - x_0}{\delta}) \in \text{Inv}(q)$ for all $t \in [0, \delta]$ and $\rho(0) = x_0, \rho(\delta) = x_1$. To show that the linear trajectory is a valid run of A , we need to check the flow condition, namely whether $\dot{\rho}(t) \in F(q)$ for all $t \in [0, \delta]$. It is clear that

$$\dot{\rho}(t) = \frac{x_1 - x_0}{\delta}.$$

Also note that $F(q)$ is a convex affine formula on $\dot{x}$, and therefore can be written in the form of $A_q \dot{x} \leq b_q$ for some constant matrix A_q and constant vector b_q .

Since $\frac{d}{dt}\xi(t) \in F(q)$ for all $t \in [0, \delta]$,

$$A_q \frac{d}{dt}\xi(t) \leq b_q,$$

for all $t \in [0, \delta]$. Take an integral with respect to t from 0 to δ , we have

$$\begin{aligned}
& \int_0^\delta A_q \frac{d}{dt} \xi(t) dt \leq \int_0^\delta b_q dt \\
\Rightarrow & A_q \int_0^\delta \frac{d}{dt} \xi(t) dt \leq b_q \delta \\
\Rightarrow & A_q \xi(t) \Big|_0^\delta \leq b_q \delta \\
\Rightarrow & A_q (\xi(\delta) - \xi(0)) \leq b_q \delta \\
\Rightarrow & A_q \left(\frac{x_1 - x_0}{\delta} \right) \leq b_q,
\end{aligned}$$

which implies that the flow condition for the linear trajectory ρ holds true. $\square$

Therefore, if there exists a run connecting two states (q_0, s_0) and (q_T, s_T) in a linear hybrid automaton A , then (q_T, s_T) can be reached from (q_0, s_0) in A by a sequence of linear trajectories as well. For this reason, we can restrict our reachability analysis of linear hybrid automata by just focusing on linear trajectories. Another reason why we are interested in linear hybrid automata is the fact that we can use linear hybrid automata to approximate general nonlinear hybrid systems. Two approaches are developed to transform or approximate a nonlinear hybrid automaton into a linear hybrid automaton, namely clock translation and linear phase-portrait approximation [11], which are briefly discussed below.

Clock Translation

The variable x_i of the hybrid automaton H is *clock-translatable* if the following holds:

- for each $q \in Q$ and each x_i , the dynamic of x_i is in the form of $\dot{x}_i = f_i^q(x_i)$ where $f_i^q : \mathbb{R} \rightarrow \mathbb{R}$ is an integrable function with constant sign over the invariant condition. For example, for any two admissible states (q, a) and (q, b) , $f_i^q(a_i)$ and $f_i^q(b_i)$ are both positive or negative. In each invariant, initial, guard condition and reset map, each x_i and x'_i are in the form of $x'_i = x_i$ or $x_i \sim c$ or $x'_i \sim c$ for $\sim = \{<, \leq, >, \geq\}$ and rational constants c ;
- for each initial states, $x_i = c$ for some constant c . For every discrete transition q, q' , either $f_i^q = f_i^{q'}$ and $x'_i = x_i$ or $x'_i = c$ for some constant c .

Then, it is possible to translate x_i into a clock t_i^x with equivalent invariant, initial, guard conditions, and reset maps. To illustrate the main idea, let's revisit the thermostat example.

Example 3.40 Consider a hybrid automaton model for a thermostat as given in Fig. 3.17. Suppose the unit of t here is in minutes. It is not hard to prove that the variable x of the thermostat automaton in Fig. 3.17 is clock-translatable. The initial value of x is 70 and $x(t) = 70e^{-t}$. It takes $\ln(\frac{70}{69})$ minutes to reach the lower bound

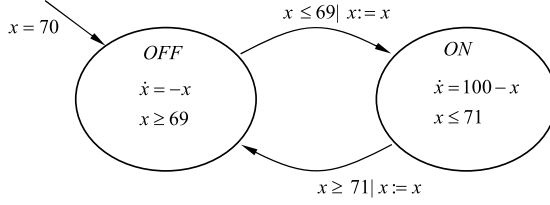


Fig. 3.17 Thermostat automaton

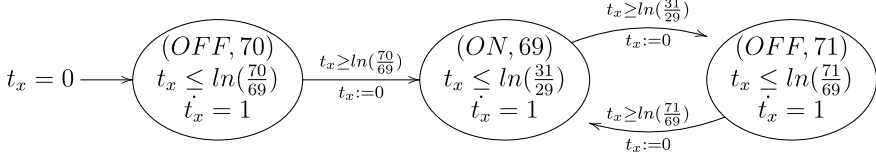


Fig. 3.18 Clock translation of the thermostat automaton

of the invariant condition $x \geq 69$. Therefore, the invariant set $x \geq 69$ is translated to $t_x \leq \ln(\frac{70}{69})$. In a similar fashion, the guard set is translated to $t_x \geq \ln(\frac{70}{69})$ and the reset map is $t'_x = 0$ since $f(ON, x) \neq f(OFF, x)$. In the ON mode, the initial value is 69 and the dynamic is $x(t) = -31e^{-t} + 100$. It takes $\ln(\frac{31}{29})$ minutes to reach the upper bound of the invariant condition. When the automaton re-enters the OFF mode, $x = 71$ and the dynamic becomes $x(t) = 71e^{-t}$ for $\ln(\frac{31}{29})$ minutes before reaching 69. The resulting automaton is shown in Fig. 3.18. $\square$

Linear Phase-Portrait Approximation

The basic idea behind linear phase-portrait approximation is quite intuitive. It refers to using weaker affine formulas to replace the nonlinear ones. For example in the ON mode of Fig. 3.17, the invariant condition of ON mode and the guard condition for the transition (OFF, ON) imply that $69 \leq x \leq 71$. Then from the dynamics $\dot{x} = 100 - x$, we can obtain the relaxed linear condition $\dot{x} \in [29, 31]$. One thing to note is that the system obtained from a linear phase-portrait is an over-approximation (in the sense that the set of trajectories of the original system is a subset of the trajectories that the linear phase-portrait approximation may exhibit) of the original system simply because of the relaxed restrictions. Therefore, for safety verification, if the approximated system satisfies the property, then it implies that the original system also satisfies it. However, the reverse is not necessarily true.

3.6.3 Symbolic Reachability Analysis of Linear Hybrid Automata

Reachability analysis is an essential step in model checking. For example, CTL model checking is mainly a backward reachability analysis. Linear hybrid automata are popular as there exist computer tools to implement its reachability analysis efficiently.

Recall the reachability analysis of timed automata where we make use of clock zones and operations on zones that are closed, such as a time drift, discrete transition, and reset. A similar approach is applicable to linear hybrid automata but instead of using zones, we use *regions* which is a set of states of a hybrid automaton. The pair

$$(q, P) = \{q, v(x) \mid v(x) \in P \cap \text{Inv}(q)\},$$

for $q \in Q$ and a polyhedron P is a region, where $v(x)$ is the evaluation of variables x . Note that unlike zones, one region could cover more than one discrete state. For example, if each (q, R_q) is a region, $R = \bigcup_{q \in Q} (q, R_q)$ is also a region.

Operations on Affine Formulas

To carry out symbolic reachability analysis, we represent regions as affine formulas. Therefore, operations on regions, such as intersections, unions, set differences, and emptiness testing, can be conducted through affine formulas as conjunctions, disjunctions, and satisfiability checking for corresponding affine formulas. The good news is that affine formulas are closed under these operations, and efficient algorithms exist for these operations.

For instance, the union of two regions represented by affine formulas ψ_1 and ψ_2 can be simply computed as a disjunction of ψ_1 and ψ_2 , namely $\psi_1 \vee \psi_2$, which is also in the form of affine formulas.

On the other hand, the intersection of these two regions can be computed as a conjunction of ψ_1 and ψ_2 , namely $\psi_1 \wedge \psi_2$. However, to show that the conjunction of two affine formulas is still an affine formula is a bit tricky. Without loss of generality, we assume that $\psi_1 = \phi_1^1 \vee \phi_1^2 \vee \dots \vee \phi_1^n$, where each ϕ_1^i is a conjunctive affine formula for all $1 \leq i \leq n$. Similarly, we assume that $\psi_2 = \phi_2^1 \vee \phi_2^2 \vee \dots \vee \phi_2^m$. Then, the conjunction between ψ_1 and ψ_2 is

$$\begin{aligned} \psi_1 \wedge \psi_2 &= (\phi_1^1 \vee \phi_1^2 \vee \dots \vee \phi_1^n) \wedge (\phi_2^1 \vee \phi_2^2 \vee \dots \vee \phi_2^m) \\ &= \bigvee_{1 \leq i \leq n, 1 \leq j \leq m} (\phi_1^i \wedge \phi_2^j), \end{aligned} \tag{3.16}$$

which is in the form of affine formulas as each $\phi_1^i \wedge \phi_2^j$ is a conjunctive affine formula for each i and j .

Similarly, the complement of a region can be computed as the negation of an affine formula $\neg\psi$. The set difference can be computed as $\psi_1 \wedge (\neg\psi_2)$.

Finally, the emptiness check of a region represented by an affine formula is equivalent to the satisfiability of the affine formula, which can be solved by a feasibility checking of linear programs. To see this, assume that the region corresponds to an affine formula in a disjunctive normal form $\phi_1 \vee \phi_2 \vee \dots \vee \phi_n$, where each ϕ_i is a conjunctive affine formula. Then the affine formula is satisfiable if and only if at least one of these conjunctive affine formulas ϕ_i is satisfiable. Hence, it boils down to checking the satisfiability of a conjunctive affine formula ϕ_i . Note that the conjunctive affine formula ϕ_i is given as a conjunction of atomic affine formulas in the form of

$$a_1x_1 + a_2x_2 + \dots + a_nx_n \sim a_0, \quad (3.17)$$

where $\sim$ is a comparison operator from the set $\{<, \leq, =, >, \geq\}$. The satisfiability of the conjunctive affine formula corresponds to whether the feasibility of these affine constraints, which can be effectively checked by linear programming, e.g., simplex methods.

Quantifier Elimination

Given an affine formula ψ and a set of variables x , we also need to compute the result $\exists x \cdot \psi$, for instance, the existence of a feasible flow rate. We will show here that the resulting quantifier elimination is also an affine formula.

We illustrate the basic idea using a conjunctive affine formula φ_i , which is a conjunct of atomic affine formulas $(a_1x_1 + a_2x_2 + \dots + a_nx_n \leq a_0)$. Let's assume that we aim to eliminate the variable x_1 . If $a_1 = 0$, then the corresponding atomic affine formula does not depend on x_1 . For the case when $a_1 > 0$, we transform the atomic affine formula into the following form:

$$x_1 \leq \frac{a_0}{a_1} - \frac{a_2}{a_1}x_2 - \dots - \frac{a_n}{a_1}x_n. \quad (3.18)$$

If $a_1 < 0$, then we obtain

$$x_1 \geq \frac{a_0}{a_1} - \frac{a_2}{a_1}x_2 - \dots - \frac{a_n}{a_1}x_n. \quad (3.19)$$

Let's denote the right-hand side as e_i which is deduced from the i th atomic affine constraint. Similarly, we can obtain $x_1 \sim e_j$ from the j th atomic affine constraint, for which $\sim$ could be either $\leq$, $\geq$, $<$, or $>$. For any pair of such derived inequalities, say $x_1 \leq e_i$ and $x_1 \geq e_j$, we can eliminate the variable x_1 by requesting $e_i \geq e_j$. It is easy to show that there exists x_1 to satisfy both $x_1 \leq e_i$ and $x_1 \geq e_j$, which are equivalent to the original atomic affine formula, as long as the inequality $e_i \geq e_j$ holds. We illustrate the quantifier elimination through the following example.

Example 3.41 Let's consider a conjunctive affine formula as follows:

$$(x + y - z < 6) \wedge (2y + 3z \geq -2) \wedge (-x + 2y + 3z \leq 10).$$

The first conjunct gives us a strict upper bound constraint on x , which is $x < 2 - y + z$. The second conjunct does not depend on x , so we just keep it. The third conjunct will give a lower bound constraint $x \geq -10 + 2y + 3z$. To eliminate x , we request that every lower bound on x is less or equal to every upper bound. This leads to

$$-10 + 2y + 3z < 2 - y + z,$$

which can be rewritten as $3y + 2z < 12$. Together with the conjunct without x , we obtain the result after eliminating x

$$(3y + 2z < 12) \wedge (2y + 3z \geq -2).$$

□

Symbolic Reachability Analysis

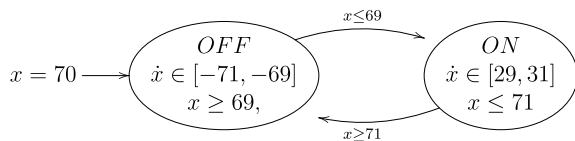
Now, we are ready to introduce the symbolic reachability analysis for linear hybrid automata using affine formulas. Given a region described by an affine formula R , we can compute its one-step reachable states, called the image of R , after a transition in a linear hybrid automaton. The transition could be an instantaneous discrete transition or a continuous transition after some positive time elapse.

Discrete Transition: For discrete transitions, we first compute the intersection of the region R with the discrete transition relations of the linear hybrid automaton. Note that the discrete transition relations are represented by affine formulas Δ . Let's use the thermostat, which is shown in Fig. 3.19, as an example:

$$\begin{aligned} \Delta &= \Delta_{OFF2ON} \vee \Delta_{ON2OFF} \\ &= ((\text{mode} = OFF) \wedge (x \leq 69) \wedge (\text{mode}' = ON) \wedge (x' - x = 0)) \\ &\quad \vee ((\text{mode} = ON) \wedge (x \geq 71) \wedge (\text{mode}' = OFF) \wedge (x' - x = 0)). \end{aligned}$$

Here, the unprimed version of variables, e.g., x , is used to represent the states in R (used to check for guard conditions), while the primed version x' stands for the image

Fig. 3.19 Linear thermostat automaton



after the transition (since there is no state jump, we have $x' - x = 0$). Similarly, the variable `mode` and its primed version `mode'` represent the discrete location before and after a discrete transition, respectively.

In the thermostat example, we have two discrete transitions, namely $OFF \rightarrow ON$ and $ON \rightarrow OFF$, which are captured by two affine formulas Δ_{OFF2ON} and Δ_{ON2OFF} , respectively. Since either the transition $OFF \rightarrow ON$ or the transition $ON \rightarrow OFF$ could happen (depends on the current mode and the value of x), we take a disjunction between Δ_{OFF2ON} and Δ_{ON2OFF} .

To compute the image after the discrete transition, we first take an intersection between the region R with the transition relation Δ , then we eliminate those unprimed variables. The result will be an affine formula on primed variables. As the last step, we rename the primed variable into its unprimed corresponding version, e.g., x' is renamed as x . To illustrate these steps, we take a look at an example.

Example 3.42 Consider the thermostat example and a region $R = OFF \times \{60 \leq x \leq 70\}$. To compute the posterior image after a discrete transition, we first take intersection between the region R with the transition relation Δ and obtain the following:

$$R \wedge \Delta = ((\text{mode} = OFF) \wedge (60 \leq x \leq 69)) \wedge ((\text{mode}' = ON) \wedge (x' - x = 0)),$$

which is non-empty. Eliminating the unprimed variables, we obtain

$$((\text{mode}' = ON) \wedge (60 \leq x' \leq 69)).$$

Renaming the primed variable, we obtain

$$Post_e(R) = ((\text{mode} = ON) \wedge (60 \leq x \leq 69)).$$

□

Continuous Transition: For continuous transitions, we need to compute the region of states s' such that there exists a state s in R and a time duration δ , and a rate vector r such that the state s and the rate r satisfies the rate constraints RC ; for every $0 \leq t \leq \delta$, the state $s + rt$ satisfies the invariant condition Inv ; the final state s' equals $s + r\delta$. With this understanding, to compute the image of a region after a continuous transition, we first require that there exists a rate vector r that satisfies the rate constraints, and a non-negative constant δ representing the time evolution, such that all the states satisfying the invariant condition hold for all such $0 \leq t \leq \delta$.

$$\exists \delta \geq 0, \dot{x} \models RC, x \models R \left((x + t\dot{x} \models Inv) \wedge (0 \leq t \leq \delta) \wedge (x' = x + \delta\dot{x}) \right). \quad (3.20)$$

Note that the term $\delta\dot{x}$ makes (3.20) no longer a linear constraint. We therefore make use of the fact that the increment $(x' - x)$ equals the value of the duration variable δ multiplied by the rate of change of x namely $(x' - x) = \delta\dot{x}$. In view of this, an atomic affine constraint of the form

$$a_1\dot{x}_1 + a_2\dot{x}_2 + \cdots a_n\dot{x}_n \sim a_0,$$

is replaced by the constraint

$$[a_1(x'_1 - x_1) + a_2(x'_2 - x_2) + \cdots a_n(x'_n - x_n) \sim a_0\delta].$$

Include δ into variables x and x' , and rewrite the constraint into an atomic affine formula

$$(a_1x'_1 - a_1x_1 + a_2x'_2 - a_2x_2 + \cdots + a_nx'_n - a_nx_n - a_0\delta \sim 0). \quad (3.21)$$

Take the intersection with the invariant conditions. Then, use quantifier elimination to project out the starting values of the continuously updated variables along with the time duration and rename every primed variable x' back to x .

This is illustrated through the following example.

Example 3.43 Consider a region $R = \{(1 \leq x \leq 3) \wedge (1 \leq y \leq 2)\}$, rate constraint $(\dot{x} = 1) \wedge (-1 \leq \dot{y} \leq -0.5)$, and invariant condition $inv = \{(x \leq 5)\}$. Its posterior set can be computed as

$$\begin{aligned} \exists x, y, \exists \delta \geq 0, \exists \dot{x}, \dot{y}, \quad & (1 \leq x \leq 3) \wedge (1 \leq y \leq 2) \wedge (x \leq 5) \\ & \wedge (\dot{x} = 1) \wedge (-1 \leq \dot{y} \leq -0.5) \\ & \wedge (x' = x + \delta\dot{x}) \wedge (y' = y + \delta\dot{y}) \wedge (x' \leq 5). \end{aligned}$$

First, replacing $\dot{x}$ by $x' - x$, $y - y'$, and δ as we shown in (3.21). Then eliminating x , y , and δ , we have

$$(x' \geq 1) \wedge (y' \leq 2) \wedge (x' \leq 5) \wedge (x' + y' \geq 2) \wedge (x' + 2y' \leq 7).$$

Renaming x' and y' , we obtain the posterior set of A as

$$(x \geq 1) \wedge (y \leq 2) \wedge (x \leq 5) \wedge (x + y \geq 2) \wedge (x + 2y \leq 7).$$

□

Forward Reachability Analysis

Given a region R , its successive region is computed in two steps. First, we compute with respect to discrete transitions. Then we search for successors through time elapse just like what was done in building zone automata. We illustrate the computation through the following example.

Example 3.44 Consider the thermostat example as shown in Fig. 3.20. Suppose that we are interested in the thermostat which is on for less than $\frac{1}{4}$ of the first hour of

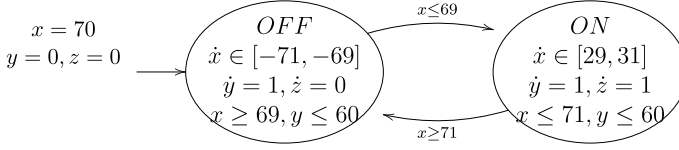


Fig. 3.20 Linear thermostat automaton

operation. We do this by adding two auxiliary variables y and z which denote the total time duration and duration of being on, respectively. Then the problem can be transformed into checking if the predicate $y = 60 \wedge z \geq \frac{1}{4}y$ is reachable.

The reachability analysis starts from the initial region $\varphi_0 = \{(OFF, x = 70, y = 0, z = 0)\}$. The successive region φ_1 is computed in two steps. First, we look for discrete transitions, but for φ_0 , since x does not satisfy guard condition, there is no discrete transition. Then we look for successors through time elapse just like what we did in building zone automata. For this, note that for a region φ , if the following predicate

$$\exists x_1, \dots, x_n, \exists \delta \geq 0, \exists \dot{x}_1, \dots, \dot{x}_n, \varphi(q) \wedge f(q, x) \wedge x'_1 = x_1 + \delta \dot{x}_1 \wedge \dots \wedge x'_n = x_n + \delta \dot{x}_n,$$

holds true for evaluations $a_1, \dots, a_n$ of $x'_1, \dots, x'_n$, it is equivalent to say that the state (q, a) is a successor of a state where φ is true with non-constrained invariant condition.

In this example, for φ_0 , we get the following predicate:

$$\begin{aligned} & \exists x, y, z. \exists \delta \geq 0. \exists \dot{x}, \dot{y}, \dot{z}. x = 70 \wedge y = 0 \wedge z = 0 \\ & \quad \wedge \dot{x} \in [-71, -69] \wedge \dot{y} = 1 \wedge \dot{z} = 0 \\ & \quad \wedge x' = x + \delta \dot{x} \wedge y' = y + \delta \dot{y} \wedge z' = z + \delta \dot{z} \\ & = (\exists \delta \geq 0, 70 - 71\delta \leq x' \leq 70 - 69\delta) \wedge y' = \delta \wedge z' = 0 \\ & = (70 - 71y' \leq x' \leq 70 - 69y' \wedge z' = 0). \end{aligned}$$

Note that the existential quantifiers have been eliminated effectively. After renaming x', y', z' to x, y, z and intersecting with the invariant condition of mode *OFF*, we have the following region $\varphi_1 = \text{post}(\varphi_0)$:

$$\varphi_1 = (OFF, x \geq 69 \wedge 70 - 71y \leq x \leq 70 - 69y \wedge z = 0).$$

This is the successive region which describes the reachable states in mode *OFF* from the initial region.

In a similar fashion, we can get $\varphi_2 = \text{post}(\varphi_1)$. Note that the discrete transition from *OFF* to *ON* is possible only when $x = 69$. Then

$$\varphi_2 = \{(OFF, x \geq 69 \wedge 70 - 71y \leq x \leq 70 - 69y \wedge z = 0), (ON, x = 69 \wedge \frac{1}{71} \leq y \leq \frac{1}{69} \wedge z = 0)\}.$$

Such procedure goes on iteratively until y reaches 60 and then we have all the reachable states. This process has been made fully automatic by existing model checking software such as HYTECH [11]. $\square$

3.6.4 Symbolic Model Checking

In the previous section, we discussed how to do a symbolic (forward) reachability analysis for linear hybrid automata. Now, we are dealing with a more general verification problem. Basically, we ask whether a linear hybrid automaton satisfies a given CTL formula ϕ . Given a CTL formula ϕ , we use $Sat(\phi)$ that denotes the set of states that satisfy ϕ , then the linear hybrid automaton $H \models \phi$ if $(q_0, x_0) \in Sat(\phi)$ for all $(q_0, x_0) \in init$.

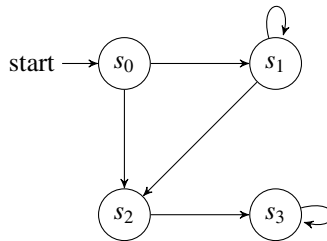
We first give a pseudocode to carry out CTL model checking for any labeled transition system as long as we can do effective backward reachability analysis. Then, we will show that the predecessor operator $pre(\cdot)$ can be effectively computed for linear hybrid automata, and so is the symbolic CTL model checking.

CTL Symbolic Model Checking Algorithm

We first describe a pseudocode to carry out CTL model checking for any labeled transition system T that can be represented symbolically. Denote its initial state as a symbolic object s_0 , which could be a Boolean function for finite transition systems or an affine formula for linear hybrid automata. In addition, we represent the transition relation of T symbolically as Δ .

For example, we can represent a set of states in a finite discrete transition system as a Boolean function, while the transition relation between states becomes operations on these Boolean functions. To be more specific, we take a look at a simple example from [43].

Example 3.45 Consider a finite transition system T given as follows:



The state of the system can be represented by two Boolean variables $\{p, q\}$. The characteristic function is a Boolean function defined by

$$\chi(s_0) = p \wedge \neg q, \chi(s_1) = p \wedge q, \chi(s_2) = \neg p \wedge q, \chi(s_3) = \neg p \wedge \neg q.$$

In this way, the sets of states can be represented as

$$\begin{aligned}\chi(s_0, s_1) &= (p \wedge \neg q) \vee (p \wedge q) = p, \\ \chi(s_1, s_2) &= (p \wedge q) \vee (\neg p \wedge q) = q, \\ \chi(s_1, s_3) &= (p \wedge \neg q) \vee (\neg p \wedge \neg q) = \text{false}.\end{aligned}$$

Furthermore, the transition relation can be represented by introducing the prime version, for example,

$$\begin{aligned}\Delta(s_0, s_1) &= \chi(s_0) \wedge \chi'(s_1) = (p \wedge \neg q) \wedge (p' \wedge q'), \\ \Delta(s_0, s_2) &= \chi(s_0) \wedge \chi'(s_1) = (p \wedge \neg q) \wedge (\neg p' \wedge q').\end{aligned}$$

The transition relation of the whole system is

$$\Delta_T = \bigvee_{s_i \rightarrow s_j} \Delta(s_i, s_j) = (p \wedge q') \vee (\neg p \wedge \neg p' \wedge \neg q'),$$

which is a disjunction of all possible transitions defined in T . □

With the symbolic representations of the sets of states and transition relations of a labeled transition system, we can now carry out model checking symbolically. In particular, a pseudocode for CTL symbolic model checking is given in Algorithm 3.4. The main function of the CTL symbolic model checking algorithm calls the function $\text{CTLStates}(T, \varphi)$ to compute the states in T satisfying the CTL formula φ , which is returned as a symbolic object S .

The main idea of $\text{CTLStates}(T, \varphi)$ is to divide a complicated formula into simpler sub-formulas, and eventually reduce it down to atomic propositions p that we already know true or false for all states in T . This is the same idea that we used in our CTL model checking for finite state transition systems. The only difference here is that the sets of states, transition relations, and (backward) reachability analysis are all done symbolically instead of explicitly state-by-state. With this in mind, it will be easier to understand the pseudocodes for the function $\text{CTLStates}(T, \varphi)$.

- For CTL state formula $\varphi = p$, the label function will return the set of states that satisfy the atomic formula p .
- For negation $\neg\varphi$, CTL solver returns the complement of the states of satisfying the formula φ , namely $\neg\text{CTLStates}(T, \varphi)$.
- For conjunction of two CTL formulas $\varphi_1 \wedge \varphi_2$, the CTL model checking solver returns the intersection between $\text{CTLStates}(T, \varphi_1)$ and $\text{CTLStates}(T, \varphi_2)$, i.e., $\text{CTLStates}(T, \varphi_1) \wedge \text{CTLStates}(T, \varphi_2)$.
- If the formula is of the form $\varphi = \exists \circ \varphi_1$, we compute the predecessor of the states in $\text{CTLState}(T, \varphi_1)$, namely the set of states in T that can be driven into the set $\text{Sat}(\varphi_1)$, denoted symbolically by S_{φ_1} , by one transition step, denoted as

$pre(CTLState(T, \varphi_1))$. The computation of $pre(CTLState(T, \varphi_1))$ is through the function $EXStates(T, S_{\varphi_1})$, which is critical for the whole symbolic CTL model checking process and will be investigated in a separate subsection.

- If the formula is given as $\varphi = \exists \Box \varphi_1$, then we compute as follows: $S^0 = CTLState(T, \varphi_1)$, $S^{i+1} = S^i \wedge pre(S^i)$ iteratively, until $S^{i+1} = S^i$ and return S^i . These iterative computations are carried out by a function named $EGStates(T, S_{\varphi})$.
- For a formula ϕ of the form $\exists[\phi_1 \sqcup \phi_2]$, we first find all states that are labeled by ϕ_2 , denoted by $S_2 = CTLState(\varphi_2)$, and then search backwards along a path till an initial state with all the states along the path are labeled with ϕ_1 , denoted by $S_1 = CTLState(\varphi_1)$. The backward search process can be computed iteratively as follows. $S^0 = \emptyset$, $S^{i+1} = S_2 \vee (S_1 \wedge pre(S^i))$, until $S^{i+1} = S^i$ and return S^i . These iterative computations are carried out by a function named $EUStates(T, S_{\varphi_1}, S_{\varphi_2})$.

Algorithm 3.4 (CTL Symbolic Model Checking Algorithm)

```

function CheckCTL( $T, s_0, \varphi$ )
 $S = CTLStates(T, \varphi)$ 
return ( $s_0 \wedge \neg S$ ) = 0
function CTLStates( $T, \varphi$ )
if  $\varphi = p$  then
    return  $p$ 
else if  $\varphi = \neg \varphi_1$  then
    return  $\neg CTLStates(T, \varphi_1)$ 
else if  $\varphi = \varphi_1 \wedge \varphi_2$  then
    return  $CTLStates(T, \varphi_1) \wedge CTLStates(T, \varphi_2)$ 
else if  $\varphi = \exists \circ \varphi_1$  then
    return  $EXStates(T, CTLStates(T, \varphi_1))$ 
else if  $\varphi = \exists \Box \varphi_1$  then
    return  $EGStates(T, CTLStates(T, \varphi_1))$ 
else if  $\varphi = \exists[\varphi_1 \sqcup \varphi_2]$  then
    return  $EUStates(T, CTLStates(T, \varphi_1), CTLStates(T, \varphi_2))$ 
end if

function EXStates( $T, S_{\varphi}$ )
return  $pre(S_{\varphi}, \Delta)$ 

function EGStates( $T, S_{\varphi}$ )
 $S^0 := S_{\varphi}$ 
 $S^1 := S^0 \wedge pre(S^0, \Delta)$ 
 $i = 0$ 
while  $S^{i+1} \neq S^i$  do
     $i = i + 1$ 
     $S^{i+1} := S^i \wedge pre(S^i, \Delta)$ 
end while
return  $S^{i+1}$ 

```

```

function EUStates( $T, S_{\varphi_1}, S_{\varphi_2}$ )
 $S^0 := 0$ 
 $S^1 := S_{\varphi_2} \vee (S_{\varphi_1} \wedge \text{pre}(S^0, \Delta))$ 
 $i = 0$ 
while  $S^{i+1} \neq S^i$  do
     $i = i + 1$ 
     $S^{i+1} := S_{\varphi_2} \vee (S_{\varphi_1} \wedge \text{pre}(S^i, \Delta))$ 
end while
return  $S^{i+1}$ 

```

Predecessor Computation

It is clear that the ability to compute the predecessor set for a given set of states, $\text{pre}(\cdot)$, is essential to carrying out the symbolic model checking. As a simple illustration of the main idea, we first take a look at a finite transition system example.

Example 3.46 Let's continue our previous example, where we already illustrated how to use Boolean functions to represent the states and transition relations in a finite transition systems. The predecessor set computation over sets of states are performed as symbolic manipulations of corresponding Boolean formulas. For example, the states that can be reached from the initial state s_0 after one step of transition can be computed as

$$\text{post}(\{s_0\}, \Delta_{TS}) = \exists(p, q), (p \wedge \neg q) \wedge ((p \wedge q') \vee (\neg p \wedge \neg p' \wedge \neg q')),$$

which involves the conjunction of two Boolean functions, namely $\chi(s_0)$ and Δ_{TS} , and quantifier elimination. After quantifier elimination, we obtain

$$\text{post}(\{s_0\}, \Delta_{TS}) = q'.$$

Since it is convenient-to-use unprimed version of the Boolean variables, we replace every primed Boolean variable with its unprimed version. Hence, we obtain

$$\text{post}(\{s_0\}, \Delta_{TS}) = q,$$

which represents the set of states that are reachable from s_0 after one step of transition in TS , which actually contains states s_1 and s_2 , since $\chi(\{s_1, s_2\}) = q$. The computation shows that the operation $\text{post}(\{s_0\}, \Delta_{TS})$ is a set consisting of states s_1 and s_2 .

Dual to the forward reachability analysis, we can also compute the predecessor set of a given set of states. For example, the predecessor set for $\{s_0, s_1\}$ can be computed symbolically as

$$\begin{aligned} pre(\{s_0, s_1\}, \Delta_{TS}) &= \exists(p', q')((p \wedge q') \vee (\neg p \wedge \neg p' \wedge \neg q')) \wedge (q') \\ &= p. \end{aligned}$$

The results of operation $pre(\{s_0, s_1\}, \Delta_{TS})$ is set $\{s_0, s_1\}$. $\square$

As this example shows, the reachability analysis, either forward or backward, can be carried out by operating on Boolean functions, and the results returned remain as Boolean functions. This gives us a way to do model checking without explicitly enumerating individual states. This is the essential idea behind symbolic model checking, and the idea can be extended beyond Boolean functions. In the sequel, we will take a look at how to use piecewise linear functions or affine functions to carry on the symbolic computation of the predecessor for linear hybrid automata.

The predecessor of a region R for a linear hybrid automaton needs to consider two types of transitions—discrete transition and continuous-time elapse, just like we did in the forward reachability analysis in the previous subsection. First, we look for discrete transitions to see whether it is possible for states to be driven into R by one discrete transition. This can be illustrated through the following example.

Example 3.47 Consider a discrete transition $e : v' \rightarrow v$ with a guard set $x \leq 3$ and a reset map $(x' \geq 5) \wedge (y' = x)$. Assume that $inv(v) = inv(v') = (x \geq 0) \wedge (y \geq 0)$. The predecessor set for a region $R = (v, \{(x \geq 6) \wedge (y \leq 2)\})$ with respect to this transition can be computed as follows:

$$pre_e(R) = \left(v', \begin{array}{c} \exists x', y', (x \geq 0) \wedge (y \geq 0) \wedge (x \leq 3) \wedge (x' \geq 5) \wedge (y' = x) \\ \wedge (x' \geq 6) \wedge (y' \leq 2) \wedge (x' \geq 0) \wedge (y' \geq 0) \end{array} \right).$$

Eliminating x', y' , we have

$$pre_e(R) = (v', 0 \leq x \leq 2),$$

which represents the states that can be driven to the region R by the discrete transition e . $\square$

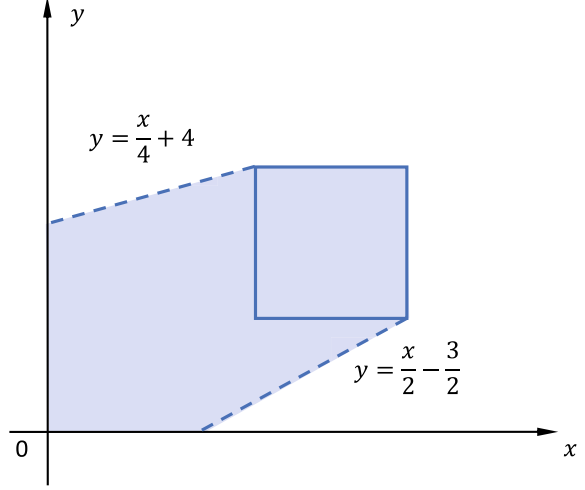
For continuous transitions, we need to compute the region of states s such that there exists a state s' in R , a time duration δ and a rate vector r satisfying the rate constraints RC , and for every $0 \leq t \leq \delta$ the state $s + rt$ satisfies the invariant condition Inv and the final state $s' = s + r\delta$. With this understanding, we have

$$\exists \delta \geq 0, \dot{x} \models RC, x' \models R \left((x + t\dot{x} \models Inv) \wedge (0 \leq t \leq \delta) \wedge (x' = x + \delta\dot{x}) \right), \quad (3.22)$$

and the predecessor set with respect to time elapse can be obtained by a quantifier elimination. The following example illustrates the computation of a predecessor set with respect to time elapse.

Example 3.48 Consider a region $R = \{(4 \leq x \leq 7) \wedge (2 \leq y \leq 5)\}$, rate constraint $(2 \leq \dot{x} \leq 4) \wedge (1 \leq \dot{y} \leq 1)$, and invariant condition $inv = \{(x \geq 0) \wedge (y \geq 0)\}$. Its predecessor set can be computed as

Fig. 3.21 The time predecessors for a simple linear hybrid automaton



$$\begin{aligned} \exists x', y', \exists \delta \geq 0, \exists \dot{x}, \dot{y}, \quad & (4 \leq x' \leq 7) \wedge (2 \leq y' \leq 5) \wedge (x' \geq 0) \wedge (y' \geq 0) \\ & \wedge (2 \leq \dot{x} \leq 4) \wedge (1 \leq \dot{y} \leq 1) \\ & \wedge (x' = x + \delta \dot{x}) \wedge (y' = y + \delta \dot{y}) \wedge (x \geq 0) \wedge (y \geq 0). \end{aligned}$$

Using the trick that we introduced earlier by replacing the nonlinear term $\delta \dot{x}$ by c_x and $\delta \dot{y}$ by c_y , we have the following equivalent set:

$$\begin{aligned} \exists x', y', \exists \delta \geq 0, \exists c_x, c_y, \quad & (4 \leq x' \leq 7) \wedge (2 \leq y' \leq 5) \wedge (x' \geq 0) \wedge (y' \geq 0) \\ & \wedge (2\delta \leq c_x \leq 4\delta) \wedge (\delta \leq c_y \leq \delta) \\ & \wedge (x' = x + c_x) \wedge (y' = y + c_y) \wedge (x \geq 0) \wedge (y \geq 0), \end{aligned}$$

which contains only linear constraints.

Eliminating x', y', δ, c_x , and c_y , we obtain the predecessor set of R as

$$(x \leq 7) \wedge (y \leq 5) \wedge (y \leq \frac{x}{4} + 4) \wedge (y \geq \frac{x}{2} - \frac{3}{2}),$$

which is plotted in Fig. 3.21. □

For a linear hybrid automaton, the predecessor set of a given region R will be the union of its discrete predecessor $pre_e(R)$ and its continuous-time predecessor $pre_t(R)$, i.e., $pre(R) = pre_e(R) \vee pre_t(R)$. We conclude this section with an example of symbolic model checking for linear hybrid automaton [10].

Example 3.49 Consider a simple temperature control system as shown in Fig. 3.22. The temperature must be maintained between 3 and 15. When the temperature reaches 15, one of the two cooling rods must be used to lower the system temperature. A rod

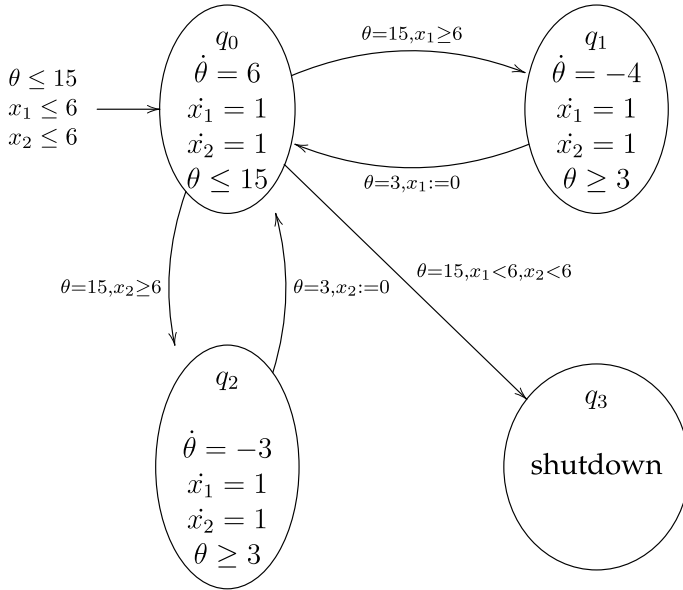


Fig. 3.22 Linear thermostat automaton

can be used again only if 6 time units have passed to allow it to cool down. If no rod is available when the temperature hits the upper bound, the system will shut down. We are interested to verify whether one can avoid shutdown, which can be written into a CTL formula as $\forall \Box \neg (q = q_3)$.

First, we convert the CTL into its ENF, which is $\neg \exists \Diamond (q = q_3)$ or equivalently $\neg \exists \text{true} \sqcup (q = q_3)$. Then, we are ready to apply the CTL model checking algorithm.

To do so, we first compute the set of states (regions) that satisfies the sub-formula $\exists \text{true} \sqcup (q = q_3)$, for which we can call the $\text{EUState}(T, S_{\varphi_1}, S_{\varphi_2})$ function with $\varphi_1 = \text{true}$ and $\varphi_2 = (q = q_3)$. Note that $(q = q_3)$ is an atomic proposition, and stands for the region

$$S_{\varphi_2} = (q_3, \mathbb{R}^3).$$

Following the algorithm of $\text{EUState}(T, S_{\varphi_1}, S_{\varphi_2})$, we have $S^0 = 0$

$$S^1 = S_{\varphi_2} \vee (S_{\varphi_1} \wedge \text{pre}(S^0)) = S_{\varphi_2} = (q_3, \mathbb{R}^3).$$

Increasing i by 1, we have

$$S^2 = S_{\varphi_2} \vee (S_{\varphi_1} \wedge \text{pre}(S^1)) = S_{\varphi_2} \vee \text{pre}(S^1),$$

where $\text{pre}(S^1) = \text{pre}_e(S^1) \vee \text{pre}_t(S^1)$, and

$$\text{pre}_e(S^1) = \text{pre}_e((q_3, \mathbb{R}^3)) = (q_0, \theta = 15 \wedge x_1 < 6 \wedge x_2 < 6),$$

$$pre_t(S^1) = pre_t((q_3, \mathbb{R}^3)) = (q_3, \mathbb{R}^3).$$

Hence, we have

$$S^2 = S_{\varphi_2} \vee pre(S^1) = (q_3, \mathbb{R}^3) \vee (q_0, \theta = 15 \wedge x_1 < 6 \wedge x_2 < 6).$$

Since, $S^2 \neq S^1$, we further increase i by 1, and further compute

$$S^3 = S_{\varphi_2} \vee (S_{\varphi_1} \wedge pre(S^2)) = S_{\varphi_2} \vee pre(S^2).$$

Similarly, we have $pre(S^2) = pre_e(S^2) \vee pre_t(S^2)$, and

$$pre_e(S^2) = pre_e((q_3, \mathbb{R}^3)) \vee pre_e((q_0, \theta = 15 \wedge x_1 < 6 \wedge x_2 < 6)) = (q_3, \mathbb{R}^3),$$

$$pre_t(S^2) = pre_t((q_3, \mathbb{R}^3)) \vee pre_t((q_0, \theta = 15 \wedge x_1 < 6 \wedge x_2 < 6)).$$

The computation of $pre_t((q_0, \theta = 15 \wedge x_1 < 6 \wedge x_2 < 6))$ needs some explanations, which can be computed by

$$\begin{aligned} \exists \theta', x'_1, x'_2, \exists \delta \geq 0, \quad & (\theta' = 15) \wedge (x'_1 < 6) \wedge (x'_2 < 6) \wedge (\theta \leq 15) \\ & \wedge (\theta' = \theta + \delta) \wedge (x'_1 = x_1 + \delta) \wedge (x'_2 = x_2 + \delta), \end{aligned}$$

which is equivalent to $(\theta \leq 15) \wedge (6x_1 < \theta + 21) \wedge (6x_2 < \theta + 21)$ after eliminating θ' , x'_1 , and x'_2 . Then, we have

$$S^3 = (q_3, \mathbb{R}^3) \vee (q_0, (\theta \leq 15) \wedge (6x_1 < \theta + 21) \wedge (6x_2 < \theta + 21)).$$

After several iterations (the details are left for readers), a fix-point is obtained and the function EUState returns the regions that satisfy the sub-formula $\exists \text{true} \sqcup (q = q_3)$ as

$$\begin{aligned} & (q = q_0, \theta \leq 15 \wedge 6x_1 < \theta + 21 \wedge 6x_2 < \theta + 21 \vee 6x_2 + 3 < \theta) \vee \\ & (q = q_1, 3 \leq \theta \leq 15 \wedge 4x_2 + \theta < 19) \vee \\ & (q = q_2, 3 \leq \theta \leq 15 \wedge 3x_1 + \theta < 15) \vee (q = q_3, \mathbb{R}^3). \end{aligned}$$

The region that satisfies $\neg \exists \text{true} \sqcup (q = q_3)$ can then be obtained as

$$\begin{aligned} & (q = q_0, \theta \leq 15 \wedge \theta + 21 \leq 6x_1 \wedge \theta + 21 \leq 6x_2 \vee \theta \leq 6x_2) \vee \\ & (q = q_1, 3 \leq \theta \leq 15 \wedge 19 \leq 4x_2 + \theta) \vee \\ & (q = q_2, 3 \leq \theta \leq 15 \wedge 15 \leq 3x_1 + \theta). \end{aligned}$$

Since the initial region $(q = q_0, \theta \leq 15 \wedge x_1 \geq 6 \wedge x_2 \geq 6)$ is contained in the above region, we can conclude that the system satisfies the CTL specification. $\square$

3.7 Verification of More General Hybrid Systems

In this section, we will consider hybrid automata with more general continuous dynamics. First, we will consider hybrid systems with continuous linear dynamics, for which we will introduce the basic ideas of approximating reachable states. Second, we will briefly discuss deductive approaches to formal verification, for which we use barrier certificates for safety verification as an example.

3.7.1 Linear Dynamics

In linear hybrid automata, the derivative of continuous variables, $\dot{x}$, at each location does not depend on x , which gives a differential inclusion with constant right-hand side, namely $\dot{x} \in F$ for some given convex polyhedron F . This dramatically simplifies the computation of reachable states as we can always rely on searching linear trajectories. What if we drop this assumption and make $\dot{x}$ depend on x ? It turns out to be a much harder problem, and we will study it in this subsection.

Consider the simplest extension, restricting ourselves to linear time-invariant dynamics at each location

$$\dot{x}(t) = Ax(t),$$

where $x \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n \times n}$. Also, for simplicity, we only focus on one discrete location, since the concatenation with discrete transitions is straightforward.

The evolution of the linear dynamics from a given initial state $x(t_0) = x_0$ is given by (see Appendix A)

$$x(t) = e^{A(t-t_0)}x_0,$$

where e^{At} is the matrix exponential of the matrix A , defined by

$$e^{At} = I + At + \frac{(At)^2}{2!} + \frac{(At)^3}{3!} + \dots$$

It is convenient to denote the solution $x(t)$ as

$$\phi(t - t_0, x_0) = e^{A(t-t_0)}x_0,$$

and call $\phi(\cdot, \cdot)$ a state transition operator since the state is transited from x_0 at t_0 to $\phi(t - t_0, x_0)$ at time t .

It is worth pointing out that the state transition operator ϕ is linear and therefore it maps a convex set into a convex set. For example, if the initial condition x_0 belongs to a line segment

$$x_0 \in \left\{ x_0 \in \mathbb{R}^n \mid \exists 0 \leq \alpha \leq 1, x_0 = (1 - \alpha)p + \alpha q \right\},$$

then at any given time instant t , $x(t)$ must belong to

$$x(t) \in \left\{ x \in \mathbb{R}^n \mid \exists 0 \leq \alpha \leq 1, x = (1 - \alpha)(e^{A(t-t_0)} p) + \alpha(e^{A(t-t_0)} q) \right\},$$

which is also a line segment. If we denote the line segment as $\overline{pq}$, then the map of the line segment can be written as

$$\phi(t - t_0, \overline{pq}) = \left\{ x \in \mathbb{R}^n \mid \exists 0 \leq \alpha \leq 1, x = (1 - \alpha)(e^{A(t-t_0)} p) + \alpha(e^{A(t-t_0)} q) \right\}.$$

Next, we show that the operator ϕ preserves convexity. By definition, a set $S \subseteq \mathbb{R}^n$ is convex if the line segment connecting any pair of points in S lies entirely in S . Therefore, given a convex set S , we pick arbitrarily two points p and q from S , and consider the line segment connecting p and q , denoted by $\overline{pq}$. As we showed above, the state transition operator ϕ maps all points on $\overline{pq}$ to a line segment with boundary points as $p_t = \phi(t - t_0, p)$ and $q_t = \phi(t - t_0, q)$. Since the points p and q were selected arbitrarily, we can conclude that the map of S under the state transition operator ϕ ,

$$S_t = \phi(t - t_0, S) = \left\{ x \in \mathbb{R}^n \mid \exists x_0 \in S : x = \phi(t - t_0, x_0) \right\},$$

is also convex.

Although convexity is preserved, the shape of the convex set could vary from time to time. In principle, one can compute the trajectories of any points in S at all time instants $t_0 \leq \tau \leq t$. However, this is not practical, since the points that need to be computed could be uncountable, and it may require infinitely many time instants. Therefore, we first need to sample the time instants and only focus on certain time points (usually uniformly distanced). Secondly, we approximate (usually over-approximate) the convex set S by a bounded polyhedron with finite vertices. Between the sampling instants, the states that can be reached are also approximated by a polyhedron.

This idea is illustrated in Fig. 3.23. First, a sampling period $\delta > 0$ is picked, which can be seen as a trade-off between the computational accuracy and complexity—smaller δ gives higher accuracy but it is more computationally expensive, while a larger sampling period gives a fast exploration of the state space but with a higher approximation error. Once a sampling period δ is fixed, we can propagate from the initial region X_0 through the state transition operator to obtain $X_1 = \phi(\delta, X_0)$ as shown in Fig. 3.23a, for which the convexity is preserved. However, the state trajectories connecting X_0 and X_1 could be curved, for which we usually need to approximate. A popular approximation is to over-approximate (if the concern is a safety property, otherwise under-approximate for reachability concerns) all the states in between using a bounded polyhedron, denoted by Ω_0 in Fig. 3.23b. Next, we further propagate the states in X_1 by δ to obtain $X_2 = \phi(\delta, X_1)$, and approximate the states in between the sampling period using another bounded polyhedron Ω_1 as

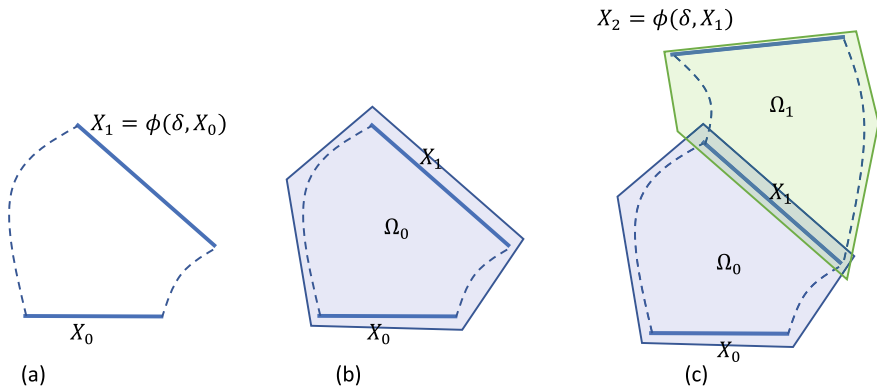


Fig. 3.23 Sampled approximation

shown in Fig. 3.23c. We repeat this step until Ω_i intersects with the unsafe region at step i , which returns failure of safety concern. This is very similar to the situation in our symbolic reachability analysis in the previous section, but we have to approximate the reachable states instead of accurately representing them. There are a lot of approximate schemes proposed in the literature to obtain a good balance between accuracy and complexity. Interested readers may refer to Notes and Further Reading section (Sect. 3.9).

3.7.2 Barrier Certificate

So far, we have relied on searching all reachable states to obtain formal guarantees. This could suffer from severe scalability concerns, known as the state explosion problem in model checking. We have seen abstraction-based and symbolic approaches mitigating the state explosion problem in model checking. This subsection aims to illustrate an alternative approach to formal verification—a deductive approach. For simplicity, we will use the safety of a continuous dynamical system as an example.

Theorem 3.2 *Let $\dot{x} = f(x)$ with $x \in \mathbb{R}^n$ be a continuous system with an invariant set giving as $Inv \subseteq \mathbb{R}^n$ and an initial condition as $Init \subset Inv$. An unsafe region of concern is given as $unSafe \subset Inv$. If there exists a continuously differentiable function $B : \mathbb{R}^n \rightarrow \mathbb{R}$ satisfying*

- $B(x) \leq 0$ for all initial states $x \in Init$;
- $B(x) > 0$ for all unsafe states $x \in unSafe$;
- $\frac{\partial B(x)}{\partial x} f(x) \leq 0$ for all states $x \in Inv$,

then all trajectories of the continuous dynamics starting from $Init$ will not reach the unsafe region, namely the safety specification is respected.

Proof This can be shown by contradiction. Assume that there exists a trajectory starting inside *Init*, but evolves into *unsafe* after time t . Namely, $x_0 \in \text{Init}$ at the initial time t_0 , which means that $B(x_0) \leq 0$. However, $B(x(t)) > 0$ since $x(t) \in \text{unsafe}$. Due to the continuity of $B(x)$, there must exist a time instant $\tau \in [t_0, t]$ such that $B(x(\tau)) = 0$. We consider two cases.

First, if $B(x_0) < 0$, then $\tau > t_0$ and we compute the following integral:

$$\int_0^\tau \frac{\partial B(x(t))}{\partial x} f(x(t)) dt = B(x(t)) \Big|_0^\tau = 0 - B(x_0) > 0.$$

However, this contradicts the third condition, i.e., $\frac{\partial B(x)}{\partial x} f(x) \leq 0$ for all states $x \in \text{Inv}$.

Second, if the initial state is at the boundary of the certificate, i.e., $B(x_0) = 0$, then we compute the following integral:

$$\int_0^t \frac{\partial B(x(s))}{\partial x} f(x(s)) ds = B(x(s)) \Big|_0^t = B(x(t)) - B(x_0) = B(x(t)) > 0,$$

since $x(t) \in \text{unsafe}$. This, again, contradicts the third condition, i.e., $\frac{\partial B(x)}{\partial x} f(x) \leq 0$ for all states $x \in \text{Inv}$. Therefore, the state trajectory cannot go into *unsafe*. $\square$

The following example from [19] is used to illustrate the main idea of safety verification through barrier certificates.

Example 3.50 Consider a continuous dynamical system

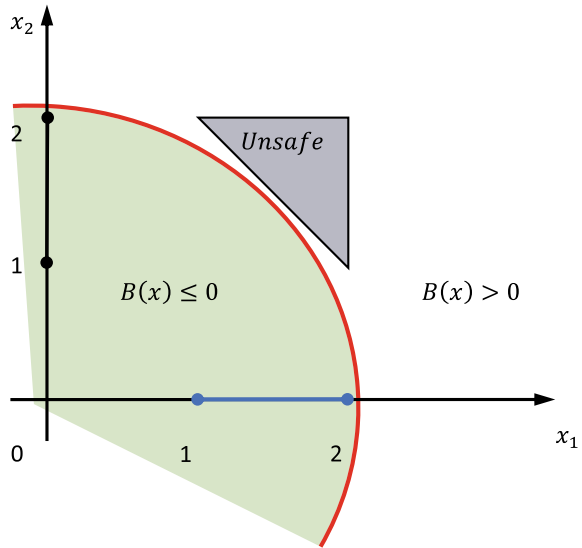
$$\dot{x} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} x,$$

where $x \in \mathbb{R}^2$. It is assumed that the invariant set associated with the location is $\text{Inv} = (x_1 \geq 0) \wedge (x_2 \geq 0)$. The initial domain *Init* is given by $\text{Init} = (1 \leq x_1 \leq 2) \wedge (x_2 = 0)$. The safety concern is to verify whether the unsafe region $(x_1 \leq 2) \wedge (x_2 \leq 2) \wedge (x_1 + x_2 \geq 3)$ is reachable from the initial region. To check the safety concern, we consider the following barrier certificate candidate $B(x) = x_1^2 + x_2^2 - 4.3$ as illustrated in Fig. 3.24.

It can be shown that all three conditions hold true. Namely, (1) $B(x) \leq 0$ for all initial states $x \in \text{Init}$; (2) $B(x) > 0$ for all unsafe states $x \in \text{unsafe}$; (3) $\frac{\partial B(x)}{\partial x} f(x) = [2x_1, 2x_2] \begin{bmatrix} -x_2 \\ x_1 \end{bmatrix} = -2x_1x_2 + 2x_1x_2 = 0$ for all states $x \in \text{Inv}$. Therefore, we can conclude that the unsafe region is not reachable from the initial states based on the Barrier Certificate Theorem. Interested readers may do some numerical simulations in MATLAB to see whether this is true. $\square$

Some remarks are in order. First, as we saw from the proof, the function works like a barrier that separates an unsafe region from safe regions, so the function B is called a barrier certificate for the continuous safety problem, and the theorem above

Fig. 3.24 Barrier certificate, for Example 3.50



is called a Barrier Certificate Theorem. Secondly, the Barrier Certificate Theorem provides a sufficient condition, and one can check the safety concern by finding such a barrier certificate B . If no such a barrier certificate can be found, we cannot draw any conclusions, i.e., the check is sound but not complete. In addition, there are many variations of the Barrier Certificate Theorem to mitigate the conservativeness of the method. Thirdly, although finding such a barrier certificate could be very challenging, it is worth pointing out that efficient numerical methods exist if restricted to certain forms of barrier certificate (e.g., polynomial) and/or when the dynamic differential equation $\dot{x} = f(x)$ has a certain structure, e.g., Hamiltonian dynamics. Finally, the barrier certificate has significant computational advantages over model checking as we do not need to search the reachable state space, which contains uncountable infinite states. It therefore provides alternative methods to the searching-based model checking, and falls into the category of deductive methods for formal verification.

3.8 Summary

In this chapter, we introduced several formal verification topics with a particular focus on model checking for labeled transition systems with respect to temporal logic specifications. Hybrid systems can be seen as labeled transition systems as well, but with an infinite number of states and actions, for which traditional model checking techniques for finite state transition systems cannot be directly applied. Hence, we follow two different approaches to handling the challenges caused by the continuous state variables in hybrid systems. One is by obtaining an equivalent abstract transition

system that has only a finite number of states, namely an abstraction. The other is through symbolic model checking that represents the sets of states and operates on these sets of states in a symbolic manner, rather than enumerating each state explicitly.

- Transition systems are graph models that describe the evolution of the states under the action of transitions.
- LTL is a temporal logic reasoning about infinite sequences of states.
- LTL formula can be translated into NBA. The number of states in the obtained NBA could be exponential with respect to the length of the formula.
- The LTL model checking problem can be converted to the language inclusion problem between two Büchi automata. One Büchi automaton is from the transition system and the other is converted from the negation of the LTL formula ϕ .
- The time complexity of the introduced LTL model checking is linear in the size of the transition system and exponential in the length of the formula.
- CTL is logic to describe the branching structure of the states.
- The CTL model checking problem of arbitrary CTL formula ϕ can be solved by successively applying the state labeling algorithm to the sub-formulas of ϕ , starting from the shortest, most deeply nested sub-formulas and working onwards to include all of ϕ .
- The CTL model checking algorithm is of polynomial time complexity.
- LTL and CTL have different expressive power and are generally incomparable. They are both subclasses of a more general temporal logic, CTL*.
- If two transition systems are bisimilar, they accept or reject the same LTL or CTL formulas. In fact, bisimulation preserves CTL* properties.
- An LTL/CTL model checking problem on a transition system can be carried on a bisimilar transition system.
- Timed automata are formal models for real-time programs. A timed automaton can be transformed into a finite region transition system for model checking.
- TCTL is a real-time variant of CTL. TCTL model checking on a timed automaton can be transformed into checking a CTL formula on a region transition system derived from the timed automaton.
- Symbolic model checking is a practical technique to potentially reduce the computation complexity. Timed automata can be converted to zone automata. Model checking of linear hybrid automata is based on regions and the predecessor operator.
- For general hybrid dynamics, we usually need to sample and approximate the reachable states.
- Deductive methods, such as barrier certificates, have advantages in scalability, but usually require insights into the system, especially the physical dynamics.
- Although numerical methods exist, the construction of a barrier certificate could be highly nontrivial. This is similar to the problem of finding a Lyapunov function, which will be discussed in the next chapter.

3.9 Notes and Further Reading

Temporal logic, transition systems, and model checking have been successfully used in applications such as the verification of software and communication protocols [2, 20–22]. Software model checkers have been developed, such as SPIN [23] for LTL model checking, symbolic model checker NuSMV [24], and PRISM [25] for probabilistic model checking. Due to space limitation, our treatment of these topics is unavoidably very brief and even superficial in some cases. Interested readers may find a more detailed and comprehensive treatment of these topics in books, e.g., [2, 6, 21, 22].

Timed automata were proposed in the early 1990s [13], where a formal timed language theory on timed automata was developed. Tools such as KRONOS [18] and UPPAAL [17] have been developed for the verification of real-time systems modeled as timed automata. Our discussions on multi-rate timed automata and rectangular automata are mainly based on results from [26, 27]. Generally speaking, the reachability (hence verification) problem is undecidable for hybrid systems. Only a small portion of hybrid systems with simple or specific continuous-variable dynamics, such as timed automata and initialized rectangular automata, have finite quotient transition systems, and thus the reachability problem for such specific hybrid systems is known to be decidable [4]. It is known that even a slight generalization of the multi-rate timed automata or rectangular automata could make the reachability problem undecidable [4]. Another class of hybrid automata with finite bisimulation, called o-minimal hybrid automata, was introduced in [28]. In [27], an excellent survey on abstractions of hybrid automata was given.

Motivated by the undecidability of general hybrid systems, many efforts have been devoted to the computation/approximation of reachable sets for a given hybrid system. A good deal of research effort has focused on developing sophisticated techniques drawn from optimal control, game theory, and computational geometry to calculate or approximate the reachable sets for various classes of hybrid systems, and several software tools have been developed for such a purpose; see, e.g., [29, 30]. The tool HyTech [11] uses symbolic reachability analysis for linear hybrid automata and represents the reachable sets as polyhedrons. Linear hybrid automata are a special case of hybrid automata, where guards, reset mapping, and invariant sets only involve linear expressions, and the continuous dynamics are restricted to linear constraints such as $\dot{x} = \dot{y} \wedge 1 \leq \dot{x} \leq 2$. More general continuous dynamics are handled in tools like CheckMate [31] and later refined by the tool d/dt [32], which compute over approximations of reachable sets using polyhedral-based representations; the resulting approximation of the reachable set is called flow-pipe approximation.

Polyhedra-based representations are appealing and are widely used. There are open-source libraries available for manipulating them. However, the complexity of operations on polyhedrons is exponential with respect to the dimensions of the continuous variables, so the size of systems that can be handled is quite modest. To reduce computational complexity, the tool SpaceEx [33] represents continuous sets

using Zonotopes [34] and support functions to compute an over-approximation of the reachable states.

Deductive verification methods for hybrid system verification are also being developed in the literature. A good example is the use of barrier certificates for safety analysis of continuous, stochastic, and hybrid systems [35]. The basic idea is to find a barrier function that is decreasing along system trajectories and has a zero level set (barrier) that no solution trajectory crosses. Should the barrier separate the initial states from unsafe regions, the safety of the dynamical system can be deduced. The barrier certificate is conceptually intuitive, but the finding of barrier certificates is not easy and is similar to the generation of a Lyapunov function. Computational methods based on, e.g., sum of squares (SOS) [36] have been developed to generate appropriate barrier certificates, but the dimension of the systems that can be handled by the computational tool is limited.

To handle higher dimensional dynamical systems and/or systems without precise mathematical models, which is usually the case in practical applications, simulation-based [37, 38] and sampling-based approaches [39, 40] have been developed in the literature. These numerical-based approaches aim to find witness traces (counterexamples) of the model to be checked that violates certain safety properties of concern. However, if the methods cannot find such a witness trace after a long period of testing, one cannot conclude that the safety property holds for sure. Another trend in the software verification community is to automatically synthesize code from required specifications [41]. This has a close relationship with the formal synthesis problems that will be discussed in Chap. 6.

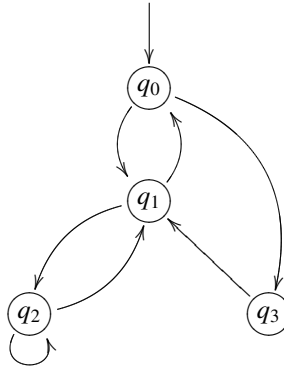
3.10 Exercises

1. **Simulations:** It is strongly recommended to simulate each of the examples in this chapter so as to gain insight into the system's behavior and verify the claimed results. MATLAB and Stateflow provide an excellent starting point for simulation work. Additional specialized commercial software is also available as noted above.
2. Consider the labeled transition system (T, l) as shown below



Show that $(T, l) \models \Diamond a \wedge \Diamond b$ but $(T, l) \not\models \Diamond(a \wedge b)$.

3. Consider the labeled transition system (T, l) as shown below



where $l(q_0)=\emptyset$, $l(q_1)=\{Close\}$, $l(q_2)=\{Close, Start, Cook\}$, $l(q_3)=\{Start\}$.

(a) Check whether (T, l) satisfies $\forall \Box (Start \Rightarrow \forall \Diamond Cook)$.

(b) Check whether (T, l) satisfies $\forall \Box (Close \wedge Start) \Rightarrow \forall \Diamond Cook$.

4. You may recall the puzzle of a ferryman, goat, cabbage, and wolf all on one side of a river, who want to cross to the other side safely. The ferryman can cross the river with at most one passenger in his boat. Of course, the boat does not need to be fully occupied, and the ferryman can cross the river alone. For obvious reasons, neither the goat and the cabbage nor the goat and the wolf should be left alone on the same side of the river. The question is whether there exists a series of boat trips such that all three (goat, cabbage, and wolf) can be carried by the ferryman safely to the other side of the river.

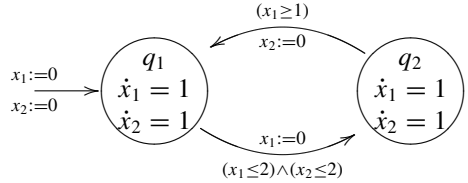
Hint: This is a planning problem, but it can be solved by model checking. The behavior of the goat, wolf, and cabbage is provided by a simple two-state transition system. The state identifiers indicate the position of each of the players: 0 stands for the current (i.e., starting) riverside, and 1 for the other side of the river (i.e., the goal). The ferryman behaves very similarly but has in addition the possibility to cross the river alone. The transition system to be checked should be a composition of these four small transition systems. Then we ask if the goal state is reachable from the initial state such that it has a state at which all the players are on the other side, and during the transitions to that state, the players are never left in an unsafe, conflicting situation. In other words, we seek a path satisfying $LTL \varphi_1 \sqcup \varphi_2$, where φ_2 asserts the final goal state, and φ_1 expresses the safety condition (if the goat is with the cabbage or the wolf, then the ferryman is there, too, to prevent any undesirable behavior). Thus, we assert that all paths satisfy $\neg(\varphi_1 \sqcup \varphi_2)$, i.e., no path satisfies $\varphi_1 \sqcup \varphi_2$. We hope this is not the case, and the model checking algorithm will give us a counterexample path which does satisfy $\varphi_1 \sqcup \varphi_2$. It is suggested to use Spin (<http://spinroot.com>) to solve this LTL model checking question and find such a path.

5. Solve the Wolf–Goat–Cabbage problem through CTL model checking. You may want to use NuSMV (<http://nusmv.fbk.eu/>) software.
6. Show that the self-bisimulation defined in Definition 3.12 is an equivalence relation, namely it is reflexive, symmetric, and transitive.
7. Show that a self-bisimulation relation $R \subseteq S \times S$ of a labeled transition system (T, l) induces a partition of the state set S into a number of equivalent classes

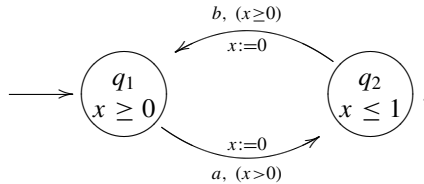
$$S = \bigcup_{s \in S} [s]_R,$$

where $[s]_R = \{s' \in S \mid (s, s') \in R\}$. Namely, the following properties hold for any $s, s' \in S$:

- (a) $s \in [s]_R$;
 - (b) if $s' \in [s]_R$, then $[s]_R = [s']_R$;
 - (c) if $s' \notin [s]_R$, then $[s]_R \cap [s']_R = \emptyset$;
 - (d) $S = \bigcup_{s \in S} [s]_R$.
8. Considering the labeled transition system (T, l) and the binary relation defined in Example 3.22, show that $(T, l) \cong (T/R, l/R)$.
 9. Show that the union of self-bisimulation relations for a labeled transition system (T, l) is also a self-bisimulation relation for (T, l) .
 10. Consider the timed automaton T given below



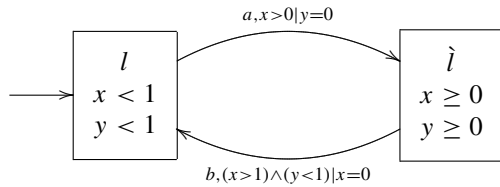
- (a) Convert the above timed automaton into a (time-abstract) transition system.
 - (b) Obtain an auto-bisimulation relation R for the timed automaton T such that its quotient system T/R has finite states.
11. Consider the following timed automaton A as



with labels $l(q_1) = \{Respond\}$ and $l(q_2) = \{Request\}$.

- (a) Can A read the timed language $\{((ab)^\omega, \bar{\tau}) \mid \tau_{2i+1} - \tau_{2i} \leq 1, \forall i \geq 0\}$?

- (b) Illustrate a typical run of the timed automaton A .
 - (c) Write A as a hybrid automaton.
 - (d) Write its (labeled) timed transition system (T_A^t, L) and time-abstract transition system (T_A, L) .
 - (e) Given a TCTL formula $\Phi = \forall \square (Request \rightarrow \forall \diamond^{\leq 1} Respond)$, derive the corresponding (finite) region transition system to check whether the timed automata A satisfies Φ .
 - (f) Try this example in UPPAAL (<http://www.uppaal.org/>).
12. Check whether the obtained finite quotient transition system in Example 3.32 is time-abstract bisimilar to the original timed automaton.
 13. Redo Example 3.35 and verify the bisimulation relation between the obtained region transition system and the timed automaton.
 14. Consider a timed automaton A with two locations as shown below



Check whether it is possible to keep the clock y less than one, i.e., the CTL formula $\exists \square (y < 1)$.

15. Show that clock zones are closed under the operations of intersection, time elapse, and resetting.
16. Obtain the Zone automaton for the timed automaton in Example 3.38.
17. Redo Example 3.49 with UPPAAL.⁵
18. Consider the following continuous dynamical system:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} -7x_1 + 3x_2 \\ -3x_1 \end{bmatrix},$$

where $x \in \mathbb{R}^2$. It is assumed that the invariant set associated with the location is $Inv = \mathbb{R}^2$. The initial domain $Init$ is given by $Init = (5 \leq x_1 \leq 6) \wedge (-1 \leq x_2 \leq 1)$. The safety concern is to verify whether the unsafe region $(x_2 \leq -4) \vee (x_2 \geq 4)$ is reachable from the initial region. To check the safety concern, we consider the following barrier certificate candidate:

$$B(x) = 7x_1^2 - 6x_1x_2 + 28x_2^2 - 320.$$

Show that $B(x)$ satisfies all the conditions in Theorem 3.2 and therefore is a barrier certificate indeed.

⁵ <http://www.uppaal.org/>.

References

1. Stephen Edwards, Luciano Lavagno, Edward A Lee, and Alberto Sangiovanni-Vincentelli. Design of embedded systems: Formal models, validation, and synthesis. *Proceedings of the IEEE*, 85(3):366–390, 1997.
2. E. M. Clarke, O. Grumberg, and D. Peled. *Model Checking*. MIT press, 1999.
3. Matt Kaufmann, J Strother Moore, and Panagiotis Manolios. *Computer-aided reasoning: an approach*. Kluwer Academic Publishers, 2000.
4. T. Henzinger, P. W. Kopke, A. Puri, and P. Varaiya. What’s decidable about hybrid automata? *Journal of Computer and System Sciences*, 57:94–124, 1998.
5. Michael Sipser. *Introduction to the Theory of Computation*, volume 2. Thomson Course Technology Boston, 2006.
6. C. Baier and J. P. Katoen. *Principles of model checking*. MIT press Cambridge, 2008.
7. Amir Pnueli. The temporal logic of programs. In *Foundations of Computer Science, 1977., 18th Annual Symposium on*, pages 46–57, Oct 1977.
8. Moshe Y Vardi. An automata-theoretic approach to linear temporal logic. In *Logics for concurrency*, pages 238–266. Springer, 1996.
9. Edmund M. Clarke, E Allen Emerson, and A Prasad Sistla. Automatic verification of finite-state concurrent systems using temporal logic specifications. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 8(2):244–263, 1986.
10. R. Alur, C. Courcoubetis, N. Halbwachs, T. Henzinger, P. H. Ho, X. Nicollin, A. Olivero, J. Sifakis, and S. Yovine. The algorithmic analysis of hybrid systems. *Theoretical Computer Science*, 138:3–34, 1995.
11. T. Henzinger, P. Ho, and H. Wong-Toi. Hytech: A model checker for hybrid systems. In *Computer aided verification*, pages 460–463. Springer, 1997.
12. R. Milner. *Communication and Concurrency*. Prentice-Hall, Inc., Upper Saddle River, NJ, USA, 1989.
13. R. Alur and D. Dill. The theory of timed automata. *Theoretical Computer Science*, 126:193–235, 1994.
14. Eugene Asarin and Oded Maler. As soon as possible: Time optimal control for timed automata. In *Hybrid Systems: Computation and Control*, pages 19–30. Springer, 1999.
15. Michael Melholt Quottrup, Thomas Bak, and RI Zamanabadi. Multi-robot planning: A timed automata approach. In *Robotics and Automation, 2004. Proceedings. ICRA’04. 2004 IEEE International Conference on*, volume 5, pages 4417–4422. IEEE, 2004.
16. R. Alur, C. Courcoubetis, and D. Dill. Model-checking for real-time systems. In *Logic in Computer Science, 1990. LICS ’90*, pages 414–425, June 1990.
17. Gerd Behrmann, Alexandre David, and Kim G Larsen. A tutorial on uppaal. In *Formal methods for the design of real-time systems*, pages 200–236. Springer, 2004.
18. Sergio Yovine. Kronos: A verification tool for real-time systems. *International Journal on Software Tools for Technology Transfer (STTT)*, 1(1):123–133, 1997.
19. Hervé Guéguen, Marie-Anne Lefebvre, Janan Zaytoon, and Othman Nasri. Safety verification and reachability analysis for hybrid systems. *Annual Reviews in Control*, 33(1):25–36, 2009.
20. Béatrice Bérard, Michel Bidoit, Alain Finkel, François Laroussinie, Antoine Petit, Laure Petrucci, and Philippe Schnoebelen. *Systems and software verification: model-checking techniques and tools*. Springer Publishing Company, Incorporated, 2010.
21. Rajeev Alur. *Principles of cyber-physical systems*. MIT Press, 2015.
22. Edmund M Clarke, Thomas A Henzinger, Helmut Veith, and Roderick Bloem. *Handbook of model checking*, volume 10. Springer, 2018.
23. Gerard J Holzmann. The model checker spin. *IEEE Transactions on software engineering*, 23(5):279–295, 1997.
24. Alessandro Cimatti, Edmund Clarke, Enrico Giunchiglia, Fausto Giunchiglia, Marco Pistore, Marco Roveri, Roberto Sebastiani, and Armando Tacchella. Nusmv 2: An opensource tool for symbolic model checking. In *Computer Aided Verification*, pages 359–364. Springer, 2002.

25. Marta Kwiatkowska, Gethin Norman, and David Parker. Prism: Probabilistic symbolic model checker. In *Computer performance evaluation: modelling techniques and tools*, pages 200–204. Springer, 2002.
26. T. Henzinger. Hybrid automata with finite bisimulations. In Z. Füllöp and G. Gécseg, editors, *ICALP'95: Automata, Languages, and Programming*. Springer-Verlag, 1995.
27. R. Alur, T. Henzinger, G. Lafferriere, and G. J. Pappas. Discrete abstractions of hybrid systems. In P. J. Antsaklis, editor, *Proceedings of the IEEE: Special issue on hybrid systems*, volume 88, pages 971–984. IEEE Press, 2000.
28. G. Lafferriere, G. J. Pappas, and S. Sastry. O-minimal hybrid systems. *Mathematics of control, signals and systems*, 13(1):1–21, 2000.
29. A. Chutinan and B. H. Krogh. Computational techniques for hybrid system verification. *IEEE Trans. Automat. Contr.*, 48(1):64–75, 2003.
30. Claire J Tomlin, Ian Mitchell, Alexandre M Bayen, and Meeko Oishi. Computational techniques for the verification of hybrid systems. *Proceedings of the IEEE*, 91(7):986–1001, 2003.
31. B Izaías Silva, Keith Richeson, Bruce Krogh, and Alongkri Chutinan. Modeling and verifying hybrid dynamic systems using checkmate. In *Proceedings of 4th International Conference on Automation of Mixed Processes*, pages 323–328, 2000.
32. Eugene Asarin, Thao Dang, and Oded Maler. The d/dt tool for verification of hybrid systems. In *Computer Aided Verification*, pages 365–370. Springer, 2002.
33. Goran Frehse, Colas Le Guernic, Alexandre Donzé, Scott Cotton, Rajarshi Ray, Olivier Lebel-tel, Rodolfo Ripado, Antoine Girard, Thao Dang, and Oded Maler. Spaceex: Scalable verification of hybrid systems. In *Computer Aided Verification*, pages 379–395. Springer, 2011.
34. Antoine Girard, Colas Le Guernic, and Oded Maler. Efficient computation of reachable sets of linear time-invariant systems with inputs. In *International Workshop on Hybrid Systems: Computation and Control*, pages 257–271. Springer, 2006.
35. Stephen Prajna and Ali Jadbabaie. Safety verification of hybrid systems using barrier certificates. In *Hybrid Systems: Computation and Control*, pages 477–492. Springer, 2004.
36. Antonis Papachristodoulou and Stephen Prajna. A tutorial on sum of squares techniques for systems analysis. In *Proceedings of the 2005, American Control Conference, 2005.*, pages 2686–2700. IEEE, 2005.
37. Truong Nghiem, Sriram Sankaranarayanan, Georgios Fainekos, Franjo Ivancić, Aarti Gupta, and George J Pappas. Monte-carlo techniques for falsification of temporal properties of non-linear hybrid systems. In *Proceedings of the 13th ACM international conference on Hybrid systems: computation and control*, pages 211–220. ACM, 2010.
38. Thao Dang, Alexandre Donzé, Oded Maler, and Noa Shalev. Sensitive state-space exploration. In *Decision and Control, 2008. CDC 2008. 47th IEEE Conference on*, pages 4049–4054. IEEE, 2008.
39. M.S. Branicky, M.M. Curtiss, J. Levine, and S. Morgan. Sampling-based planning, control and verification of hybrid systems. *IEE Proceedings - Control Theory and Applications*, 153:575–590(15), September 2006.
40. Erion Plaku, Lydia E Kaviraki, and Moshe Y Vardi. Falsification of ltl safety properties in hybrid systems. *International Journal on Software Tools for Technology Transfer*, 15(4):305–320, 2013.
41. Amir Pnueli and Roni Rosner. On the synthesis of a reactive module. In *Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 179–190. ACM, 1989.
42. Aceto, Luca and Ingólfsdóttir, Anna and Larsen, Kim Guldstrand and Srba, Jiri. *Reactive systems: modelling, specification and verification*. Cambridge University Press, 2007.
43. Chaki, Sagar and Gurfinkel, Arie. *Handbook of Model Checking*, pages 219–245. Springer, 2018.

Chapter 4

Stability and Stabilization



Stability is a basic requirement in dynamical systems. Generally speaking, a dynamical system is stable when there exists at least one operating point, typically an equilibrium where the state stays if there is no external stimulus, such that all the state trajectories starting from somewhere nearby will stay close to the operating point at all times, and may eventually converge to that point. This is known as Lyapunov stability, and some control problems, like trajectory tracking, can be transformed into stability problems, see e.g., [1, 2].

This chapter introduces basic results for the stability and stabilization of hybrid systems. Most of the stability results in the literature have been developed for switched systems, piecewise affine systems or their special cases. Here, the main results are also described in the language of hybrid automata so as to be consistent with descriptions elsewhere in this book. In the present chapter on stability, we do not explicitly consider state jumps and we use the term switched systems and hybrid systems interchangeably. We use switching signals to denote the sequence of discrete mode transitions.

The organization of this chapter is as follows: In Sect. 4.1, we briefly review several important results from the Lyapunov stability theory of continuous dynamical systems. Note that several properties of continuous-time and discrete-time continuous dynamical systems are summarized in Appendix A. In Sect. 4.2, we introduce the stability concepts of hybrid systems and discuss two examples to illustrate stability issues of hybrid systems under given discrete switching signals. Results on the stability analysis of hybrid systems under arbitrary switching are introduced in Sect. 4.3. Stability under rate limited switching, such as dwell time and average dwell time, is studied in Sect. 4.4. The general case of hybrid system stability under restricted switching is investigated through multiple Lyapunov functions in Sect. 4.5. Then, in Sect. 4.6, we turn to the synthesis of a stabilizing switching logic controller for a given collection of continuous variable dynamical systems and several stabilization conditions and design methods are discussed. Section 4.7 summarizes the results presented in this chapter. Additional results and references are discussed in the Sect. 4.8.

4.1 Lyapunov Stability Theory

In this section, we present a brief review of results from Lyapunov stability theory that are important to our discussion on the stability of hybrid systems in later sections. This section mainly follows the treatment of Lyapunov stability theory in [1].

4.1.1 Lyapunov Stability

Consider an autonomous dynamical system of first-order ordinary differential equations

$$\dot{x}(t) = f(x(t)), \quad (4.1)$$

where $x(t) \in \mathbb{R}^n$ and $f : D \rightarrow \mathbb{R}^n$ is locally Lipschitz on a domain $D \subseteq \mathbb{R}^n$. A state $x_e \in D$ is an *equilibrium point* of (4.1) if $f(x_e) = 0$. Also, without loss of generality, we assume that the origin is an (isolated) *equilibrium*, that is, $x_e = 0$, $f(0) = 0$ for $t \geq 0$. If $x_e \neq 0$, we can introduce a change of variables $z = x - x_e$ such that

$$\dot{z} = \dot{x} - \dot{x}_e = f(z + x_e) = g(z),$$

since $\dot{x}_e = 0$ as x_e is constant. The differential equation $\dot{z} = g(z)$ is clearly equivalent to the original system (in stability) and it now has an equilibrium at $z_e = 0$.

Definition 4.1 The equilibrium $x_e = 0$ is said to be *stable* in the sense of Lyapunov if for each $\epsilon > 0$, there exists a $\delta = \delta(\epsilon)$ such that

$$\|x_0\| < \delta \Rightarrow \|\phi(t; x_0)\| < \epsilon \text{ for all } t \geq 0,$$

where $\phi(t; x_0)$ denotes the solution of (4.1) corresponding to the initial condition x_0 , evaluated at t .

It is not easy to check the stability of a system directly using the definition, since it requires explicit knowledge of the solutions. Lyapunov stability theory provides us stability criteria for dynamical systems without explicitly providing their solutions. In doing so, we will employ the *Lyapunov's Second Method* (also called the *Lyapunov's Direct Method*). The following is a well-known Lyapunov Stability Theorem.

Theorem 4.1 *The equilibrium $x_e = 0$ of System (4.1) is stable in the sense of Lyapunov if there exists a continuous differentiable function $V : D \rightarrow \mathbb{R}$ such that $V(x(t))$ is positive definite in D (namely $V(x(t)) > 0$ for all non-zero $x \in D$) and its derivative along the state trajectories $x(t)$ of (4.1) is negative semi-definite, namely $\dot{V} = \frac{d}{dt} V(x(t)) = \frac{\partial V}{\partial x} f(x) \leq 0$ for all $x \in D$.*

Proof For any $\epsilon > 0$, consider an open ball, $B_r = \{x \in \mathbb{R}^n : \|x\| < r\} \subset D$, whose radius $r \in (0, \epsilon]$. Let $\alpha = \min_{\|x\|=r} V(x)$ be the minimum value of V attains on the

boundary of B_r . Note that the minimum is well defined since the boundary is a closed and bounded set (i.e., compact) and in addition V is continuous.

Now consider a subset of B_r , $\Omega_\beta = \{x \in B_r : V(x) \leq \beta\}$, where $\beta \in (0, \alpha)$. The set Ω_β , therefore, is a set contained in an ϵ -neighborhood of the equilibrium 0 where the function V takes values less than the minimum, α , attained on the boundary of B_r .

It can be shown that any solution $x(t)$ of the system $\dot{x} = f(x)$ starting from $x(0) \in \Omega_\beta$ will remain inside of Ω_β . By assumption

$$\dot{V}(x) = \frac{\partial V}{\partial x} f(x) \leq 0,$$

which implies that $V(x(t))$ is a monotonically decreasing function of time, and therefore, we have

$$V(x(t)) \leq V(x(0)) \leq \beta,$$

for all $t \geq 0$.

Since V is continuous and $V(0) = 0$, there must exist a $\delta > 0$ such that $\|x\| \leq \delta$ implies $V(x) \leq \beta$ so there exists another open ball $B_\delta = \{x \in \mathbb{R}^n : \|x\| < \delta\}$ such that $B_\delta \subset \Omega_\beta \subset B_r$. This means that if $x(0) \in B_\delta$, then $x(t) \in \Omega_\beta$ and we can conclude that $x(t) \in \Omega_\beta$ for all $t \geq 0$. Furthermore, since Ω_β is a subset of B_r , which is an open ball about the origin of radius less than ϵ . Hence, we conclude that $\|x(t)\| \leq \epsilon$ for all $t \geq 0$. This is precisely the definition of Lyapunov stability and so the proof is complete. $\square$

An illustration of the sets, B_r , Ω_β and B_δ , defined in the proof is given in Fig. 4.1. The function $V(x)$ is called a Lyapunov function and the theorem gives a sufficient condition for the stability of the equilibrium point. We can think of the Lyapunov function as a generalization of the energy inside a physical system, and for a stable system, its energy should not increase along the system's trajectory. The following example illustrates the construction of a Lyapunov function based on the system's energy.

Example 4.1 Consider a pendulum with mass m and length l as illustrated in Fig. 4.2. We can derive the equation of motion of the mass using Newton's law

$$-mgsin\theta = ml\ddot{\theta},$$

where g is the gravitational constant. To write the dynamic equation into a state-space form, we define state variables as $x_1 = \theta$ and $x_2 = \dot{\theta}$. Then, we can convert the above differential equation into the following state-space equation

$$\begin{aligned}\dot{x}_1 &= x_2, \\ \dot{x}_2 &= -\frac{g}{l} \sin x_1.\end{aligned}$$

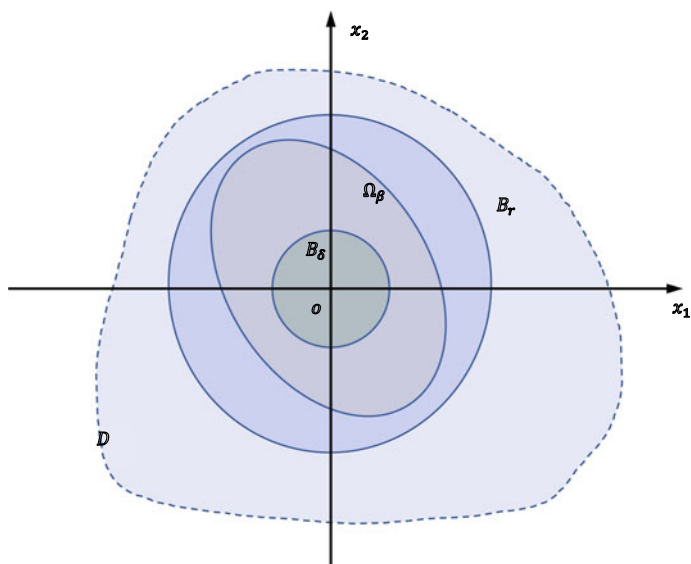
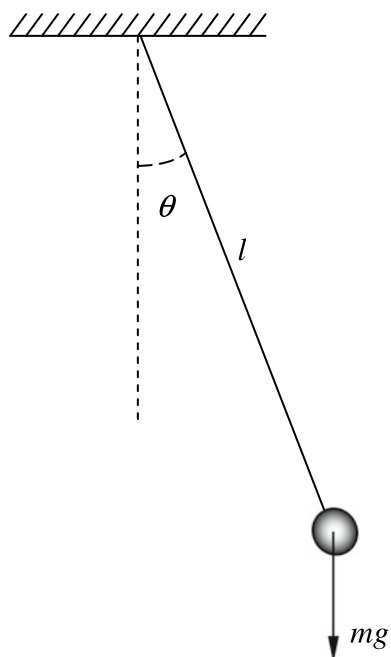


Fig. 4.1 Illustration of the proof for Theorem 4.1

Fig. 4.2 A pendulum example



To show that the downright equilibrium $x^* = 0$ of the pendulum is a stable equilibrium point, consider the function

$$V(x) = gl(1 - \cos x_1) + \frac{l^2 x_2^2}{2},$$

defined over the set $\Omega = \{x \in \mathbb{R}^2 : -\pi < x_1 < \pi\}$. Note that the first part in $V(x)$ corresponds to the potential energy, while the second part is the kinetic energy of the pendulum. Then

$$V(0) = 0; \quad (4.2)$$

$$V(x) > 0, \quad \forall x \in \Omega \setminus \{0\}, \quad (4.3)$$

and

$$\dot{V}(x) = \frac{\partial V}{\partial x} f(x) = [gl \sin x_1, l^2 x_2] \begin{bmatrix} x_2 \\ -\frac{g}{l} \sin x_1 \end{bmatrix} \equiv 0, \quad (4.4)$$

so in view of the above theorem the equilibrium $x^* = 0$ is stable. $\square$

Asymptotic Stability

Stability only tells us that if we slightly perturb the system from its equilibrium, the system's trajectory won't go too far away from the equilibrium. Consider the above pendulum example: if the mass is pulled away from the rest position, it will swing back and forth around the rest position. Eventually, the mass will be at rest again (due to frictions). However, this is not captured by the basic Lyapunov stability concept. Therefore, we also desire that the equilibrium be attractive in the following sense.

Definition 4.2 The equilibrium $x_e = 0$ of System (4.1) is said to be *attractive* if there exists a $\delta > 0$ such that

$$\|x_0\| < \delta \Rightarrow \|\phi(t; x_0)\| \rightarrow 0, \quad \text{as } t \rightarrow \infty,$$

where $\phi(t; x_0)$ stands for the solution of the state equation $\dot{x} = f(x)$ with initial condition $x(0) = x_0$.

However, as the following example shows, we need to further strengthen the attractiveness, as an equilibrium could be attractive while unstable in the sense of Lyapunov.

Example 4.2 Consider the nonlinear system of equations

$$\begin{aligned} \dot{x}_1 &= \frac{x_1^2(x_2 - x_1) + x_2^5}{(x_1^2 + x_2^2)[1 + (x_1^2 + x_2^2)^2]}, \\ \dot{x}_2 &= \frac{x_2^2(x_2 - 2x_1)}{(x_1^2 + x_2^2)[1 + (x_1^2 + x_2^2)^2]}, \end{aligned}$$

with the origin as an isolated equilibrium. Note that the denominators are always positive and $\dot{x}_2$ switches sign on $x_2 = 2x_1$, so any trajectory will diverge from the origin at first, no matter how close it starts from the origin. However, when the trajectories hit the plane $x_2 = 2x_1$, they will begin to decrease and finally converge to the origin. $\square$

Therefore, we strengthen the stability concept as follows:

Definition 4.3 The equilibrium $x_e = 0$ of System (4.1) is said to be *asymptotically stable* if it is both stable and attractive.

Accordingly, we can strengthen the Lyapunov stability theorem to give a sufficient condition for asymptotic stability.

Theorem 4.2 *The equilibrium $x_e = 0$ of System (4.1) is asymptotically stable in the sense of Lyapunov if there exists a continuous differentiable function $V : D \rightarrow \mathbb{R}$ such that $V(x(t))$ is positive definite in D (namely $V(x(t)) > 0$ for all non-zero $x \in D$) and its derivative along the state trajectories $x(t)$ of (4.1) is strictly negative definite, namely $\dot{V} = \frac{d}{dt} V(x(t)) = \frac{\partial V}{\partial x} f(x) < 0$ for all non-zero $x \in D$.*

Proof Since $V(x(t))$ is a monotone decreasing function of time t and bounded below, there must exist a real $c \geq 0$ such that $\lim_{t \rightarrow \infty} V(x(t)) = c$. Let's assume that $c > 0$. By the continuity of V , there exists $d > 0$ such that the open ball $B_d = \{x : \|x\| < d\}$ satisfies $B_d \subset \Omega_c = \{x : V(x) \leq c\}$.

By our assumption, however, $x(t)$ cannot enter the ball B_d since $V(x(t))$ is always greater than c . So let $-\gamma = \max_{x \in \Omega_c - B_d} \dot{V}(x) < 0$. This implies that

$$V(x(t)) = V(x(0)) - \int_0^t \dot{V}(x(\tau)) d\tau \leq V(x(0)) - \gamma t.$$

This, however, implies that V must eventually become negative as time t elapses. This is a contradiction to the fact that $V(x(t))$ is positive definite. So, c cannot be positive and must be zero. So the state asymptotically converges to zero and we can conclude that the equilibrium at zero is asymptotically stable. $\square$

Example 4.3 Consider the system $\dot{x} = -x^3$ and consider the following candidate Lyapunov function

$$V(x) = x^4,$$

which is clearly positive definite. Compute its derivative as

$$\dot{V}(x) = \frac{\partial V}{\partial x} \dot{x} = 4x^3(-x^3) = -4x^6,$$

which is negative definite. Therefore, the origin is globally asymptotically stable. Actually, this idea can be generalized to the following system

$$\dot{x} = -g(x(t)),$$

with $g : \mathbb{R} \rightarrow \mathbb{R}$ being an odd function in the sense that its graph lies in the first and third quadrants (i.e., $xg(x) > 0$ for all $x \neq 0$). For such systems, we can construct a Lyapunov function as

$$V(x) = \int_0^x g(s)ds.$$

We left as an exercise for the reader to show that the constructed V is positive definite and its directional derivative $\dot{V} = \frac{\partial V}{\partial x} \dot{x}$ is negative definite. $\square$

Exponential Stability

If, furthermore, there exist positive real constants α, β, γ , and $c \geq 1$ such that for all x and t , we have

$$\alpha \|x(t)\|^c \leq V(x(t)) \leq \beta \|x(t)\|^c,$$

and

$$\dot{V}(x(t)) \leq -\gamma \|x(t)\|^c,$$

then we can obtain a stronger version of asymptotic stability.

Definition 4.4 The equilibrium $x_e = 0$ of System (4.1) is *exponentially stable* if there exist $\kappa > 0$, $\lambda > 0$, and for every $\epsilon > 0$, there exists a $\delta(\epsilon) > 0$, such that

$$\|\phi(t, x_0)\| \leq \kappa e^{-\lambda t}, \text{ for all } t \geq 0,$$

whenever $\|x_0\| < \delta(\epsilon)$.

To see why, we first exploit the comparison principle: Assume that $V(t)$ is a scalar function that satisfies the following differential inequality

$$\dot{V} \leq \gamma V,$$

for some constant γ . Let $W(t)$ be a solution to the differential equation

$$\dot{W} = \gamma W,$$

with initial condition $W(t_0) = V(t_0)$. Then it can be shown that

$$V(t) \leq e^{\gamma(t-t_0)} W(t_0) = W(t),$$

for all $t \geq t_0$.

By the comparison principle and the following differential inequality

$$\dot{V}(x) \leq -\gamma \|x\|^c \leq -\frac{\gamma}{\beta} V(x),$$

we obtain

$$V(x) \leq V(x_0) e^{-\frac{\gamma}{\beta}(t-t_0)},$$

which implies that

$$\|x(t)\| \leq \left(\frac{V(x)}{\alpha} \right)^{\frac{1}{c}} \leq \left(\frac{V(x_0) e^{-\frac{\gamma}{\beta}(t-t_0)}}{\alpha} \right)^{\frac{1}{c}} \leq \left(\frac{\beta}{\alpha} \right)^{\frac{1}{c}} \|x_0\| e^{-\frac{\gamma}{c\beta}(t-t_0)}.$$

Then, the exponential stability for the equilibrium point $x_e = 0$ follows.

Global Stability

So far, we have restricted our discussion to local stability properties. However, many applications need global stability guarantees, namely starting from any initial point $x(0) \in \mathbb{R}^n$ the state trajectory will eventually converge to the equilibrium point. It is reasonable to guess that the global stability can be guaranteed if we have $\dot{V}(x) < 0$ for all $x \in \mathbb{R}^n$. Unfortunately, this is not true. The condition $\dot{V}(x) < 0$ only implies that the state trajectory $x(t)$ approaches a lower level set of $V(x)$, i.e., the value of $V(x(t))$ is monotonically decreasing with respect to t , but there is no guarantee that the state trajectory $x(t)$ asymptotically converges to the origin as illustrated in Fig. 4.3. To avoid such a situation, we need to further request that the Lyapunov function is radially unbounded.

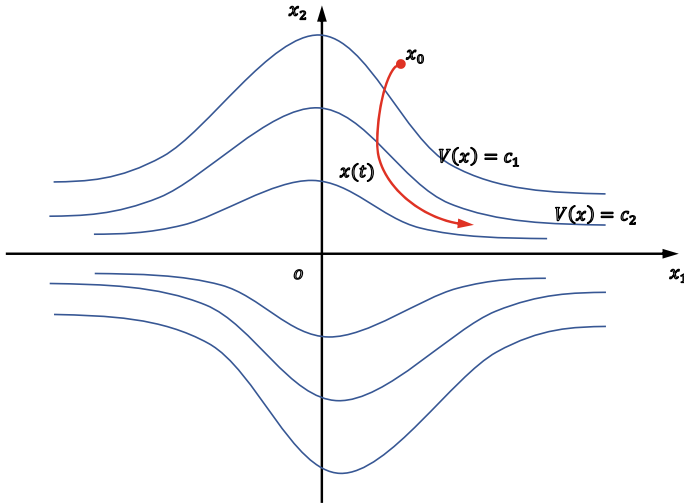


Fig. 4.3 Although the value of $V(x(t))$ is monotonically decreasing ($c_2 < c_1$) along the state trajectory $x(t)$, the state may still fail to converge to the origin

Definition 4.5 The Lyapunov function $V : \mathbb{R}^n \rightarrow \mathbb{R}$ is radially unbounded if $V(x) \rightarrow \infty$ as $\|x\| \rightarrow \infty$.

Now, we can give a sufficient condition for global asymptotic stability.

Theorem 4.3 Let $x = 0$ be an equilibrium point for $\dot{x} = f(x)$, where f is locally Lipschitz. If there exists a radially unbounded and continuously differentiable function $V : \mathbb{R}^n \rightarrow \mathbb{R}$ such that $V(0) = 0$, $V(x) > 0$ for $x \neq 0$, and $\dot{V}(x) < 0$ for all $x \neq 0$, then the equilibrium point $x = 0$ is globally asymptotically stable.

Proof Consider any $x \in \mathbb{R}^n$ and let $c = V(x)$. Since V is radially unbounded we know that for any $c > 0$ there exists $\epsilon > 0$ such that $V(x) > c$ whenever $\|x\| > \epsilon$. So the set $\Omega_c = \{x : V(x) \leq c\}$ must be a subset of the closed ball $B_\epsilon = \{x : \|x\| \leq \epsilon\}$ and it follows that $V(x(t)) \leq V(x_0) = c$ for all $t \geq 0$. This implies that $x(t) \in B_\epsilon$ for all $t \geq 0$. Since c can be made arbitrarily large, this implies that $x(0)$ can be taken arbitrarily large and still lie within a bounded Ω_c . $\square$

Let's illustrate the result using a simple example

Example 4.4 Consider a system

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = -x_1^3 - x_2 \end{cases},$$

and consider the following candidate Lyapunov function

$$V(x) = \frac{1}{2}x^T \begin{bmatrix} k & k \\ k & 1 \end{bmatrix} x + \int_0^{x_1} s^3 ds,$$

where $0 < k < 1$. This is positive definite for all $x \in \mathbb{R}^2$ and is radially unbounded. The derivative is

$$\dot{V}(x) = -(1-k)x_2^2 - kx_1^4,$$

which is negative definite for all $x \in \mathbb{R}^2$ since $0 < k < 1$. Therefore, the origin is globally asymptotically stable. $\square$

4.1.2 Stability of Linear Time-Invariant Systems

The Lyapunov stability theorem does not give a systematic procedure on how to construct Lyapunov functions. In fact, finding Lyapunov functions for general nonlinear systems is not easy. However, Lyapunov functions for linear time-invariant systems have special structures and there exist systematic approaches to find them.

Consider a linear time-invariant system of the form

$$\dot{x}(t) = Ax(t), \quad (4.5)$$

where $x \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n \times n}$. Clearly, $A0 = 0$, so the origin is an equilibrium point of (4.5). Consider a quadratic candidate Lyapunov function

$$V(x) = x^T Px, \quad (4.6)$$

where P is a real ($P \in \mathbb{R}^{n \times n}$) symmetric ($P = P^T$) and positive definite ($P > 0$) matrix. The directional derivative of V is

$$\begin{aligned} \dot{V}(x) &= \dot{x}^T Px + x^T P \dot{x} \\ &= x^T (A^T P + PA)x. \end{aligned}$$

Let Q be a symmetric positive definite matrix that satisfies

$$A^T P + PA + Q = 0,$$

which is known as a Lyapunov equation. If such Q exists, we can conclude that

$$\dot{V} = x^T (A^T P + PA)x = -x^T Qx < 0,$$

for $x \neq 0$. This immediately implies that $V = x^T Px$ is indeed a Lyapunov function for (4.5), and the origin is asymptotically stable. This result can be stated as follows:

Theorem 4.4 *For a linear time-invariant system (4.5), if there exist symmetric positive definite matrices P and Q such that the following Lyapunov equation*

$$A^T P + PA = -Q, \quad (4.7)$$

holds, then the origin of the LTI system is asymptotically stable.

On the other hand, we can solve the LTI system explicitly for a given initial condition $x(t_0) = x_0$. In particular, the state trajectory starting from $x(0) = x_0$ can be expressed as

$$x(t) = \sum_{i=1}^{\sigma} \sum_{k=1}^{n_i} A_{ik} t^k e^{\lambda_i t} x_0,$$

where A_{ik} is a suitable residue matrix, σ represents the number of distinct eigenvalues of A and n_i is the multiplicity of the i th distinct eigenvalue. Clearly, $x(t)$ asymptotically approaches zero as long as all the terms in the sum approach zero as t goes to infinity, which happens if and only if all the real parts of λ_i are negative. In other words, the LTI system $\dot{x} = Ax$ is asymptotically stable if and only if all eigenvalues of A have negative real parts. Matrices with this property are called Hurwitz matrices.

Next we show that the existence of a quadratic Lyapunov function is also necessary for a stable LTI system. Actually, we can pick

$$P = \int_0^{\infty} e^{A^T t} Q e^{A t} dt,$$

for a given real symmetric and positive definite matrix $Q \in \mathbb{R}^{n \times n}$. Clearly $P \in \mathbb{R}^{n \times n}$. Furthermore, P is symmetric since

$$P^T = \int_0^{\infty} \left(e^{A^T t} Q e^{A t} \right)^T dt = \int_0^{\infty} (e^{A t})^T Q^T (e^{A^T t})^T dt = \int_0^{\infty} e^{A^T t} Q e^{A t} dt = P.$$

We now show that P is positive definite. Consider

$$x^T P x = \int_0^{\infty} x^T e^{A^T t} Q e^{A t} x dt = \int_0^{\infty} z^T(t) Q z(t) dt,$$

with $z(t) = e^{A t} x$. Since Q is positive definite, so $z^T(t) Q z(t) > 0$ for all $z(t) \neq 0$. The only x for which $z(t)$ can be zero is $x = 0$ since $e^{A t}$ is non-singular. Therefore, P is positive definite.

Next, we show that P , Q satisfy the Lyapunov equation. With the pick of P , we have

$$A^T P + P A = \int_0^{\infty} \left(A^T e^{A^T t} Q e^{A t} + e^{A^T t} Q e^{A t} A \right) dt.$$

But

$$\frac{d}{dt} \left(e^{A^T t} Q e^{A t} \right) = A^T e^{A^T t} Q e^{A t} + e^{A^T t} Q e^{A t} A,$$

so we have

$$A^T P + P A = \int_0^{\infty} \frac{d}{dt} \left(e^{A^T t} Q e^{A t} \right) dt = e^{A^T t} Q e^{A t} \Big|_0^{\infty} = \lim_{t \rightarrow \infty} \left(e^{A^T t} Q e^{A t} \right) - e^{A^T 0} Q e^{A 0}.$$

Since we assumed that the LTI system is asymptotically stable, $x(t) \rightarrow 0$ as $t \rightarrow \infty$. Hence, $\lim_{t \rightarrow \infty} \left(e^{A^T t} Q e^{A t} \right) = 0$, and

$$A^T P + P A = -Q,$$

namely P and Q satisfy the Lyapunov equation. Therefore, $V(x) = x^T P x$ is indeed a Lyapunov function for the LTI system (4.5).

In addition, we can show that the solution of the Lyapunov equation is unique in the sense that for a given real symmetric positive definite matrix Q , the matrix P can uniquely determined. To see this, assume this is not the case and there exists another matrix $\bar{P}$ such that

$$A^T \bar{P} + \bar{P} A + Q = 0.$$

Subtract the two Lyapunov equations to get

$$A^T (P - \bar{P}) + (P - \bar{P}) A = 0.$$

Multiplying both sides by $e^{A^T t}$ from left and e^{At} from right, we have

$$e^{A^T t} A^T (P - \bar{P}) e^{At} + e^{A^T t} (P - \bar{P}) A e^{At} = 0,$$

for all $t \geq 0$. On the other hand

$$\frac{d}{dt} (e^{A^T t} (P - \bar{P}) e^{At}) = A^T e^{A^T t} (P - \bar{P}) e^{At} + e^{A^T t} (P - \bar{P}) e^{At} A = 0,$$

which implies $e^{A^T t} (P - \bar{P}) e^{At}$ is constant for all $t \geq 0$. Because of the stability conditions, we know that $e^{At} \rightarrow 0$ as $t \rightarrow \infty$. So $e^{A^T t} (P - \bar{P}) e^{At}$ must be a constant and equal to zero for all time. This, together with the fact that e^{At} is non-singular, implies that

$$P - \bar{P} = 0,$$

and so the solution is unique.

Now, we know that the solution of the Lyapunov equation is unique, and the existence of such a quadratic Lyapunov function is both necessary and sufficient for an asymptotically stable LTI system. Furthermore, we can show a stronger version of stability for LTI systems.

For an LTI system, $\dot{x} = Ax$, we know that if this system's origin is asymptotically stable there exist symmetric positive definite matrices P and Q that satisfy the Lyapunov equation $A^T P + P A + Q = 0$. So the directional derivative of $V = x^T P x$ can be bounded as

$$\dot{V} = x^T (A^T P + P A) x \leq -x^T Q x.$$

We also know that

$$\underline{\lambda}(P) \|x\|^2 \leq x^T P x \leq \bar{\lambda}(P) \|x\|^2,$$

where $\underline{\lambda}(P)$ and $\bar{\lambda}(P)$ stand for the minimum and maximum eigenvalue of P , respectively (note that the eigenvalues of a real symmetric matrix are all real numbers).

Rearranging the above inequality, we obtain

$$\|x\|^2 \geq \frac{x^T P x}{\bar{\lambda}(P)} = \frac{V(x(t))}{\bar{\lambda}(P)}.$$

Now going back to the $\dot{V}$ and using the lower bound on $\|x\|^2$, we obtain

$$\dot{V} = -x^T Q x \leq -\underline{\lambda}(Q)\|x\|^2 \leq -\frac{\underline{\lambda}(Q)}{\bar{\lambda}(P)} V(x(t)).$$

This is a linear differential inequality constraint and implies

$$V(t) \leq e^{-\frac{\underline{\lambda}(Q)}{\bar{\lambda}(P)}t} V(0).$$

Since

$$\underline{\lambda}(P)\|x(t)\|^2 \leq V(t) \leq \bar{\lambda}(P)\|x(t)\|^2,$$

we have

$$\|x(t)\|^2 \leq \frac{1}{\underline{\lambda}(P)} V(t) \leq \frac{1}{\underline{\lambda}(P)} e^{-\frac{\underline{\lambda}(Q)}{\bar{\lambda}(P)}t} V(0).$$

Together with the fact that $V(0) \leq \bar{\lambda}(P)\|x(0)\|^2$, we can conclude

$$\|x(t)\|^2 \leq \frac{\bar{\lambda}(P)}{\underline{\lambda}(P)} e^{-\frac{\underline{\lambda}(Q)}{\bar{\lambda}(P)}t} \|x(0)\|^2,$$

which implies the exponential stability—we basically showed that an asymptotically stable LTI system is also exponentially stable. In summary, we have the following theorem.

Theorem 4.5 *For a linear continuous-time time-invariant system $\dot{x}(t) = Ax(t)$, the following statements are equivalent:*

- (1) *the system is asymptotically stable;*
- (2) *the system is exponentially stable;*
- (3) *matrix A is Hurwitz;*
- (4) *the Lyapunov equation*

$$A^T P + P A = -Q, \quad (4.8)$$

has a unique solution $P > 0$ for any $Q > 0$;

- (5) *the above Lyapunov equation has a unique solution $P > 0$ for some $Q > 0$.*

Note that a matrix A is Hurwitz when all of its eigenvalues have negative real parts. Relation (4.8) indicates that the quadratic function $V(x) = x^T P x$ is a Lyapunov function for the system since it is positive definite ($V(x) = x^T P x > 0$, because $P > 0$) and its derivative (along the state trajectories) is negative ($\dot{V}(x) = x^T (A^T P + P A)x = -x^T Q x < 0$).

All linear quadratic Lyapunov results for continuous-time systems have discrete-time counterparts. The discrete-time version of Theorem 4.5 is now stated.

Theorem 4.6 *For linear discrete-time time-invariant system $x(k+1) = Ax(k)$, the following statements are equivalent:*

- (1) *the system is asymptotically stable;*
- (2) *the system is exponentially stable;*
- (3) *matrix A is Schur;*
- (4) *the Lyapunov equation*

$$A^T P A - P = -Q, \quad (4.9)$$

has a unique solution $P > 0$ for any $Q > 0$;

- (5) *the above Lyapunov equation has a unique solution $P > 0$ for some $Q > 0$.*

Note that a matrix A is Schur when all of its eigenvalues have magnitude < 1 .

4.1.3 Lyapunov Stability for Time-Varying Systems

We now extend the study of Lyapunov stability to time-varying systems of the following form

$$\dot{x}(t) = f(t, x(t)), \quad x(t_0) = x_0. \quad (4.10)$$

The origin is an equilibrium point if $f(t, 0) = 0$ for all $t \geq t_0$.

Definition 4.6 This equilibrium is said to be stable in the sense of Lyapunov if for all $\epsilon > 0$ and for all t_0 there exists $\delta(\epsilon, t_0) > 0$ such that $\|x(t)\| \leq \epsilon$ for all $t \geq t_0$ as long as $\|x(t_0)\| \leq \delta$. Furthermore, the equilibrium is said to be asymptotically stable if

- (1) it is stable;
- (2) there exists $c(t_0) > 0$ such that $\|x(t)\| \rightarrow 0$ as $t \rightarrow +\infty$ for all $\|x(t_0)\| < c$.

Let's take a look at a simple example.

Example 4.5 Consider the system

$$\dot{x} = -\frac{x}{1+t}.$$

Note that $f(t, 0) = 0$, so the origin is an equilibrium point. To check whether it is stable or not, we solve its solution by a separation of variables

$$\frac{\dot{x}}{x} = -\frac{1}{1+t} dt,$$

that can be integrated from t_0 to t to obtain

$$x(t) = x(t_0) \exp\left(\int_{t_0}^t \left(-\frac{1}{1+s}\right) ds\right) = x(t_0) \frac{1+t_0}{1+t}.$$

It is clear that $\|x(t)\| \leq \|x(t_0)\|$ for all $t \geq t_0$. So for any $\epsilon > 0$ we can choose $\delta \leq \epsilon$, then it holds that $\|x(t)\| \leq \epsilon$ for all $t \geq t_0$ as long as $\|x(t_0)\| \leq \delta$. Therefore, the equilibrium point is stable in the sense of Lyapunov. Actually, it is asymptotically stable since $x(t) \rightarrow 0$ as $t \rightarrow +\infty$. In other words, for any $\epsilon > 0$, we can find $T > 0$ such that $\|x(t)\| \leq \epsilon$ for all $t \geq t_0 + T$. In particular, given $\epsilon > 0$, for

$$\|x(t)\| \leq \|x(t_0)\| \frac{1+t_0}{1+t_0+T} \leq \epsilon,$$

we can get

$$T \geq t_0 \left(\frac{\|x(t_0)\|}{\epsilon} - 1 \right) - 1,$$

which provides a lower bound on T . This implies that for a very large t_0 , we need a very large T for the system to settle. Especially, $T \rightarrow \infty$ as $t_0 \rightarrow \infty$. In other words, the system's rate of approaching to the equilibrium point becomes arbitrarily slow for later initial times. This is obviously a problem. $\square$

To address the concern illustrated by the above example, we modify the Lyapunov stability concept for time-varying systems by requesting that for a given $\epsilon > 0$ the existence of $\delta > 0$ is independent of t_0 . This is called uniform stability, which is formally defined as follows:

Definition 4.7 This equilibrium is said to be uniformly stable in the sense of Lyapunov if for all $\epsilon > 0$ and for all t_0 there exists $\delta(\epsilon) > 0$ such that $\|x(t)\| \leq \epsilon$ for all $t \geq t_0$ as long as $\|x(t_0)\| \leq \delta$. Furthermore, the equilibrium is said to be uniformly asymptotically stable if

- (1) it is uniformly stable;
- (2) there also exists $c > 0$ (independent of t_0) such that $\|x(t)\| \rightarrow 0$ as $t \rightarrow +\infty$ for all $\|x(t_0)\| < c$.

To state the Lyapunov stability theorems for time-varying systems, it would be convenient to introduce so-called comparison functions.

Definition 4.8 A function $\alpha : \mathbb{R}^+ \mapsto \mathbb{R}^+$ is of class $\mathcal{K}$ if it is continuous, strictly increasing, and $\alpha(0) = 0$. If α is also unbounded, then it is said to be of class $\mathcal{K}_\infty$. A function $\beta : \mathbb{R}^+ \times \mathbb{R}^+ \mapsto \mathbb{R}^+$ is of class $\mathcal{KL}$ if for each t , $\beta(\cdot, t) \in \mathcal{K}$, and for each fixed r , $\beta(r, \cdot)$ is decreasing and $\lim_{t \rightarrow \infty} \beta(r, t) = 0$.

It is not hard to show that comparison functions have the following properties:

- if $\alpha \in \mathcal{K}$ over $[0, a)$, then $\alpha^{-1} \in \mathcal{K}$ over $[0, \alpha(a))$;
- if $\alpha \in \mathcal{K}_\infty$, then $\alpha^{-1} \in \mathcal{K}_\infty$;

- if $\alpha_1, \alpha_2 \in \mathcal{K}$, then $\alpha_1 \circ \alpha_2 \in \mathcal{K}$;
- if $\alpha_1, \alpha_2 \in \mathcal{K}$ and $\beta \in \mathcal{KL}$, then $\alpha_1(\beta(\alpha_2(r), s)) \in \mathcal{KL}$.

Now, we can state the Lyapunov stability theorem for time-varying systems as follows:

Theorem 4.7 *Let $x = 0$ be an equilibrium point for $\dot{x} = f(x, t)$ and let $V : [0, \infty) \times \mathcal{D} \rightarrow \mathbb{R}$ be a C^1 (continuously differentiable) function over $\mathcal{D} \subset \mathbb{R}^n$ such that*

$$\alpha_1(\|x\|) \leq V(t, x) \leq \alpha_2(\|x\|).$$

If

$$\dot{V}(t, x) = \frac{\partial V}{\partial t} + \frac{\partial V}{\partial x} f(t, x) \leq -\gamma(\|x\|),$$

for all $t \geq 0$ and for all $x \in \mathcal{D}$, where α_1, α_2 , and γ are class $\mathcal{K}$ functions on $\mathcal{D}$, then $x = 0$ is uniformly asymptotically stable.

Proof Consider any $\epsilon > 0$ and define $\delta = \alpha_2^{-1}(\alpha_1(\epsilon))$. Then using the bound on V and $\dot{V}$, we can deduce that $\|x(t_0)\| < \delta$ implies

$$\alpha_1(\|x(t)\|) \leq V(t, x(t)) \leq V(t_0, x(t_0)) \leq \alpha_2(\|x(t_0)\|) \leq \alpha_2(\delta) = \alpha_1(\epsilon).$$

This implies that $\|x(t)\| \leq \epsilon$ for all $t \geq t_0$, which implies the Lyapunov stability of the equilibrium point.

Denote $V(t) = V(t, x(t))$ and let $\theta(r) = \gamma(\alpha_2^{-1}(r))$. Since $\dot{V} \leq -\gamma(\|x\|)$, we have

$$\dot{V} \leq -\gamma(\alpha_2^{-1}(V(t))) = -\theta(V(t)).$$

It can easily shown that θ is a class $\mathcal{K}$ function and locally Lipschitz. There exists a unique solution $y(t)$ to the differential equation $\dot{y} = -\theta(y)$ such that $y(t_0) = V(t_0)$ and $y(t) = \phi(V(t_0), t - t_0)$ for some $\mathcal{KL}$ function ϕ . By the comparison principle we, therefore, know that

$$V(t) \leq \phi(V(t_0), t - t_0),$$

which implies that

$$\|x(t)\| \leq \gamma^{-1}\left(\phi\left(\alpha_2(\|x(t_0)\|), t - t_0\right)\right).$$

This implies that $\|x(t)\| \rightarrow 0$ as $t \rightarrow \infty$, which assures that the equilibrium is uniformly asymptotically stable. $\square$

Some remarks are in order. First, in the proof, we see that the existence of class $\mathcal{K}$ functions α_1 and α_2 such that

$$\alpha_1(\|x\|) \leq V(x) \leq \alpha_2(\|x\|),$$

implies that $V(x)$ is positive definite and radially unbounded. Actually, the reverse is also true. Let $V : D \rightarrow \mathbb{R}$ be a continuous positive definite function defined on a domain $D \subset \mathbb{R}^n$ that includes the origin. Let $N_r(0) \subset D$ be an open neighborhood of the origin for some $r > 0$. Then there exist class $\mathcal{K}$ function α_1 and α_2 defined on $[0, r]$ such that

$$\alpha_1(\|x\|) \leq V(x) \leq \alpha_2(\|x\|),$$

for all $x \in B_r$. Moreover, if $D = \mathbb{R}^n$ and $V(x)$ is radially unbounded, then α_1 and α_2 can be chosen to be class $\mathcal{K}_\infty$ with the preceding inequality still holding for all $x \in \mathbb{R}^n$.

Secondly, if the comparison functions α_1 , α_2 and γ can be bounded by $k\|x\|^c$ for some constants k and c , then it is possible to strengthen the result to deduce uniform exponential stability. In particular, if there exist positive constants c , k_1 , k_2 , and k_3 such that

$$\alpha_1(\|x\|) \geq k_1\|x\|^c, \quad \alpha_2(\|x\|) \leq k_2\|x\|^c, \quad \gamma(\|x\|) \geq k_3\|x\|^c,$$

then we have

$$k_1\|x\|^c \leq V(x) \leq k_2\|x\|^c,$$

and

$$\dot{V}(t, x) \leq -\gamma(\|x\|) \leq -k_3\|x\|^c \leq -\frac{k_3}{k_2} V(t, x).$$

Then, we can obtain

$$V(t, x(t)) \leq V(t_0, x(t_0)) e^{-\frac{k_3}{k_2}(t-t_0)},$$

which implies that

$$\|x(t)\| \leq \left(\frac{V(t, x)}{k_1} \right)^{\frac{1}{c}} \leq \left(\frac{k_2}{k_1} \right)^{\frac{1}{c}} \|x(t_0)\| e^{-\frac{k_3}{ck_2}(t-t_0)}.$$

This shows that the equilibrium at the origin is exponentially stable.

4.1.4 Converse Lyapunov Theorem

All the Lyapunov stability theorems we have reviewed so far request the existence of a Lyapunov function $V(x)$ that satisfies certain conditions, and most of them (except for the case of LTI systems) are sufficient only. If one cannot find such a Lyapunov function, we cannot say much about the stability of the system. It is, therefore, a valid question to ask whether a Lyapunov function exists. Converse theorems for

Lyapunov functions answer such questions and have also been extensively studied to determine under what conditions it is guaranteed to find a Lyapunov function.

Theorem 4.8 *Let the equilibrium $x_e = 0$ of $\dot{x} = f(x, t)$ be given, where f is continuously differentiable and its Jacobian matrix $[\partial f / \partial x]$ is bounded on $D \subset \mathbb{R}^n$. Let k and λ be positive constants such that the trajectories of the system satisfy*

$$\|x(t)\| \leq k \|x(t_0)\| e^{-\lambda(t-t_0)},$$

for all $t \geq t_0$. Then there exists a function $V : [0, \infty) \times D \rightarrow \mathbb{R}$ that satisfies

$$\begin{aligned} c_1 \|x\|^2 &\leq V(t, x) \leq c_2 \|x\|^2, \\ \frac{\partial V}{\partial t} + \frac{\partial V}{\partial x} f(x, t) &\leq -c_3 \|x\|^2, \\ \left\| \frac{\partial V}{\partial x} \right\| &\leq c_4 \|x\|^2, \end{aligned}$$

for some positive constants c_1, c_2, c_3 , and c_4 . Moreover, if the origin is globally exponentially stable, then $V(t)$ is defined and satisfies the above inequalities on $\mathbb{R}^n$.

Proof Consider the following function

$$V(x, t) = \int_t^{t+T} \phi^T(\tau; t, x) \phi(\tau; t, x) d\tau,$$

where $\phi(\tau; t, x)$ stands for the state trajectory of the system $\dot{x} = f(x, t)$ with the initial condition $x(t) = x$, and T is a positive constant.

First, since the solution is exponentially decaying, namely $\|x(t)\| \leq k \|x(t_0)\| e^{-\lambda(t-t_0)}$, so we have

$$V(x, t) = \int_t^{t+T} \|\phi(\tau; t, x)\|^2 d\tau \leq \int_t^{t+T} k^2 e^{2\lambda(t-\tau)} d\tau \|x\|^2 = \frac{k^2}{2\lambda} (1 - e^{-2\lambda T}) \|x\|^2.$$

On the other hand, since the Jacobian is bounded there exists $L > 0$ such that $\left\| \frac{\partial f}{\partial x}(x, t) \right\| \leq L$. This means that $\|f(x, t)\| \leq L \|x\|$, which suggests

$$\|\phi(\tau; t, x)\|^2 \geq \|x\|^2 e^{-2L(\tau-t)}.$$

Hence,

$$V(x, t) = \int_t^{t+T} \|\phi(\tau; t, x)\|^2 d\tau \geq \int_t^{t+T} e^{-2L(\tau-t)} d\tau \|x\|^2 = \frac{1}{2L} (1 - e^{-2LT}) \|x\|^2.$$

So, we can conclude that

$$c_1 \|x\|^2 \leq V(x, t) \leq c_2 \|x\|^2,$$

with $c_1 = \frac{1-e^{-2LT}}{2L}$ and $c_2 = \frac{k^2(1-e^{-2LT})}{2\lambda}$.

Now denote $\phi_t(\tau; t, x) = \frac{\partial}{\partial t} \phi(\tau; t, x)$ and $\phi_x(\tau; t, x) = \frac{\partial}{\partial x} \phi(\tau; t, x)$. Note that the identity

$$\phi_t(\tau; t, x) + \phi_x(\tau; t, x) f(x, t) = 0,$$

holds for all $\tau \in [t, t+T]$. So the directional derivative of V is

$$\begin{aligned} \dot{V} &= \phi^T(t+T; t, x) \phi(t+T; t, x) - \phi^T(t; t, x) \phi(t; t, x) \\ &\quad + \int_t^{t+T} 2\phi^T(\tau; t, x) \phi_t(\tau; t, x) d\tau + \int_t^{t+T} 2\phi^T(\tau; t, x) \phi_x(\tau; t, x) d\tau f(x, t) \\ &= \phi^T(t+T; t, x) \phi(t+T; t, x) - \|x\|^2 \\ &\quad + \int_t^{t+T} 2\phi^T(\tau; t, x) [\phi_t(\tau; t, x) + \phi_x(\tau; t, x) f(x, t)] d\tau \\ &\leq -(1 - k^2 e^{2\lambda T}) \|x\|^2 = -c_3 \|x\|^2, \end{aligned}$$

for which $c_3 = 1 - k^2 e^{2\lambda T}$ and $c_3 > 0$ if we choose $T > \ln \frac{2k^2}{2\lambda}$. Finally, $\|\frac{\partial V}{\partial x}\|$ is bounded since $\|\frac{\partial f}{\partial x}\| \leq L$ uniformly. $\square$

There actually exists a stronger version of converse Lyapunov theorem which establishes that the existence of Lyapunov function is always necessary for uniformly asymptotic stability. But the proof is much more involved, so we simply state the result without proof. Interested readers may refer to e.g., [1] for proofs.

Theorem 4.9 *Let the equilibrium $x_e = 0$ of $\dot{x} = f(x, t)$ be given, where f is continuously differentiable on $D = \{x \in \mathbb{R}^n : \|x\| < r\}$ and the Jacobian matrix $[\partial f / \partial x]$ is bounded uniformly in t . Let β be a class $\mathcal{KL}$ function and let r_0 be positive constant such that $\beta(r_0, 0) < r$. Assume that the trajectories of the system satisfy*

$$\|x(t)\| \leq \beta(\|x(t_0)\|, t - t_0),$$

for all $x(t_0) \in D$ and all $t \geq t_0 > 0$. Then there exists a continuously differentiable function $V : \mathbb{R} \times D \rightarrow \mathbb{R}$ and class $\mathcal{K}$ function $\alpha_1, \alpha_2, \gamma$, and ω such that

$$\begin{aligned} \alpha_1(\|x\|) &\leq V(t, x) \leq \alpha_2(\|x\|), \\ \frac{\partial V}{\partial t} + \frac{\partial V}{\partial x} f(x, t) &\leq -\gamma(\|x\|), \\ \left\| \frac{\partial V}{\partial x} \right\| &\leq \omega(\|x\|). \end{aligned}$$

The converse Lyapunov theorems tell us that the existence of Lyapunov functions is necessary as well, and the proofs are constructive. However, the Lyapunov functions constructed in the proof need the solution, which is usually hard to obtain—actually if we can solve for solutions, then we can directly check the stability properties instead of using Lyapunov methods.

Our treatment on Lyapunov stability theory follows classical nonlinear control theory, and readers may refer to books, such as [1, 3, 4], for further details.

4.2 Stability of Hybrid Automata

With the preparation of the basic Lyapunov stability theory, we now turn to the stability of hybrid systems. We will first define the equilibrium and stability concepts for hybrid systems.

Definition 4.9 Consider an autonomous hybrid automaton $H = \{Q, X, f, \text{Init}, \text{Inv}, E, G, R\}$. The continuous state $x_e = 0$ is an equilibrium point of H if

- (1) $f(q, 0) = 0, \forall q \in Q$;
- (2) $R((q, q'), 0) \subset Q \times Q \times \{0\}$.

From the above definition, discrete transitions are enabled from $(q, 0)$ to $(q', 0)$. Therefore, if $(q_0, 0) \in \text{Init}$ and $(\mathcal{T}, q, x)$ is a hybrid execution starting at $(q_0, 0)$, then $x(t) = 0, \forall t \in \mathcal{T}$.

The following statements give precise definitions on the stability in the sense of Lyapunov of an equilibrium of a hybrid automaton H .

Definition 4.10 The equilibrium point $x_e = 0$ of a hybrid automaton H is

- *stable* if $\forall \epsilon > 0, \exists \delta > 0$ such that for all hybrid executions $(\mathcal{T}, q, x)$ starting at (q_0, x_0) ,

$$\|x_0\| < \delta \Rightarrow \|x(t)\| < \epsilon, \forall t \in \mathcal{T};$$

- *unstable* if not stable;
- *asymptotically stable* if it is stable and δ can be chosen so that for all hybrid executions $(\mathcal{T}, q, x)$ starting at (q_0, x_0) ,

$$\|x_0\| < \delta \Rightarrow \lim_{t \rightarrow \mathcal{T}_\infty} \|x(t)\| = 0,$$

where $\mathcal{T}_\infty = \sum_{i=1}^{\infty} (\tau'_i - \tau_i)$ (Note that $\mathcal{T}_\infty < \infty$ for Zeno behaviors and $\mathcal{T}_\infty = \infty$ otherwise);

- *exponentially stable* if $\forall \epsilon > 0, \exists \delta = \delta(\epsilon) > 0, \kappa = \kappa(\epsilon) > 0$ and $\lambda = \lambda(\epsilon) > 0$ such that

$$\|x_0\| < \delta \Rightarrow \|x(t)\| \leq \kappa e^{-\lambda t} \|x_0\|, \forall t \in \mathcal{T}.$$

These properties are called *global* if they hold for arbitrary x_0 .

From the stability results in classical continuous systems, one would naturally expect that a hybrid system would be stable if the system dynamics in each discrete state are stable, at least in the case where there is no state jump. Interestingly, the stability issues of hybrid systems include several remarkable phenomena. Consider the following motivating examples, which are adopted from [5, 6].

Example 4.6 Consider a hybrid system described by a hybrid automaton

- $Q = \{q_1, q_2\}$;
- $X = \mathbb{R}^2$;
- $f(q_1, x) = A_1x$ and $f(q_2, x) = A_2x$, where

$$A_1 = \begin{bmatrix} -1 & -100 \\ 10 & -1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} -1 & 10 \\ -100 & -1 \end{bmatrix}; \quad (4.11)$$

- $Init = Q \times X$;
- $Inv(q_1) = X$, and $Inv(q_2) = X$;
- $E = \{(q_1, q_2), (q_2, q_1)\}$;
- $G(q_1, q_2) = \{x \in \mathbb{R}^2 : x_2 = -0.2x_1\}$, and $G(q_2, q_1) = \{x \in \mathbb{R}^2 : x_2 = 5x_1\}$;
- $R((q_1, q_2), x) = \{x\}$, and $R((q_2, q_1), x) = \{x\}$.

Here the hybrid system is switching between the two linear time-invariant (LTI) subsystems

$$\dot{x} = A_1x, \quad \dot{x} = A_2x.$$

The eigenvalues of A_1 are $-1 \pm 31.62i$ and those of A_2 are the same since $A_2 = A_1^T$. Therefore, they are both exponentially stable since all eigenvalues of A_1 and A_2 have negative real parts. In addition, they have a common equilibrium point, which is the origin. However, this simple hybrid system is unstable, since a divergent trajectory can be generated for an initial state which may even be very close to the equilibrium point. See Fig. 4.4, where the initial condition is $x_0 = \begin{bmatrix} -0.01 \\ 0.02 \end{bmatrix}$. $\square$

This motivating example shows that even when all the subsystems are exponentially stable, the hybrid system may be unstable. On the other hand, switching between unstable subsystems may generate convergent trajectories as the following example shows.

Example 4.7 Consider a system switching between two LTI subsystems that can be described by a hybrid automaton

- $Q = \{q_1, q_2\}$;
- $X = \mathbb{R}^2$;
- $f(q_1, x) = A_1x$ and $f(q_2, x) = A_2x$, where

$$A_1 = \begin{bmatrix} 1 & -100 \\ 10 & 1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 1 & 10 \\ -100 & 1 \end{bmatrix}; \quad (4.12)$$

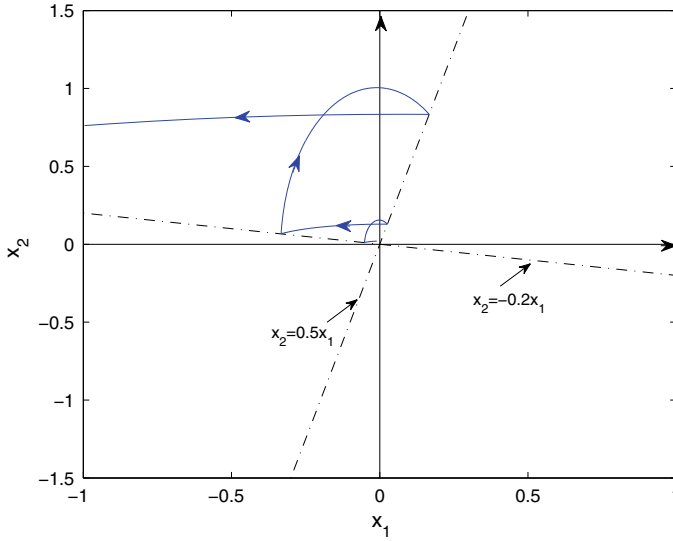


Fig. 4.4 Divergent trajectories in the phase plane can be generated by switching between two stable subsystems. The guard sets (also known as switching surfaces/manifolds) are plotted for illustration

- $Init = Q \times X$;
- $Inv(q_1) = X$, and $Inv(q_2) = X$;
- $E = \{(q_1, q_2), (q_2, q_1)\}$;
- $G(q_1, q_2) = \{x \in \mathbb{R}^2 : x_2 = -x_1\}$, and $G(q_2, q_1) = \{x \in \mathbb{R}^2 : x_2 = 0\}$;
- $R((q_1, q_2), x) = \{x\}$, and $R((q_2, q_1), x) = \{x\}$.

The eigenvalues of both A_1 and A_2 are $1 \pm 31.62i$, therefore both A_1 and A_2 are unstable since both have positive real part eigenvalues. However, the overall hybrid system is globally asymptotically stable. This is illustrated in Fig. 4.5 as the state trajectory, with initial condition $x_0 = \begin{bmatrix} 0.4 \\ -0.2 \end{bmatrix}$, approaches the origin as time goes to infinity. $\square$

As these examples suggest, the stability of hybrid systems depends not only on the dynamics of each subsystem, but also on the properties of the logic part, here are the switching signals. Therefore, the stability study of hybrid systems can be divided into two kinds of problems. One is the stability analysis of hybrid systems under given switching signals (maybe arbitrary, slow switching etc.); the other is the synthesis of switching signals for a given collection of dynamical systems to be stable (stabilization).

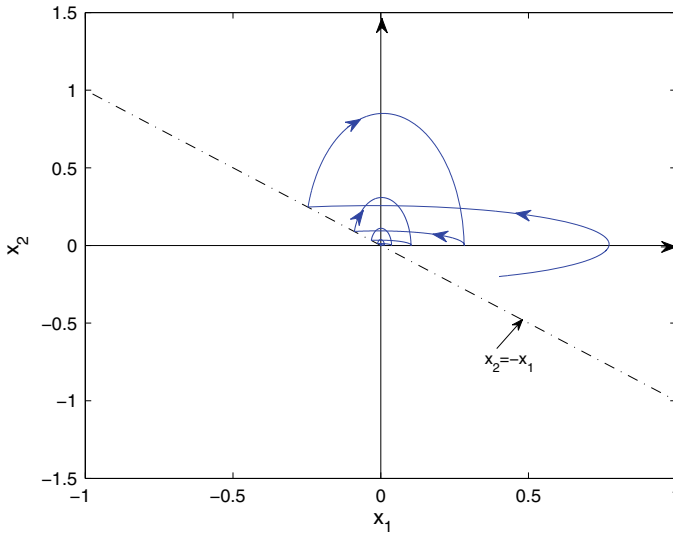


Fig. 4.5 Switching between unstable subsystems may generate stable (convergent) closed-loop behaviors

4.3 Arbitrary Switching

In this section, we consider stability analysis problems in hybrid systems where there are no restrictions on the discrete event dynamics. This may be due to not having *a priori* knowledge of the discrete event logic, such as partitions of the state space, constraints, etc. Under these conditions, one usually assumes that all discrete transitions are possible; this is called arbitrary switching in the literature. It is the easiest switched systems stability problem to address, but it only applies to systems where all the subsystems are stable and all switchings are possible, which make the results quite conservative—as Example 4.7 shows that stable behaviors may be obtained by switching between unstable dynamics.

For this problem formulation, it is necessary to require that all the subsystems are stable, since under arbitrary switching one may remain at an unstable subsystem all the time, which is a valid behavior, and so cause unstable behavior for the hybrid system. In general, as we have seen in Example 4.6, a hybrid system may become unstable even when all of its subsystems are exponentially stable. Therefore, to identify conditions for which hybrid systems are stable under arbitrary switching signals even when all subsystems are stable is of value.

4.3.1 Common Lyapunov Functions

On the other hand, if there exists a common Lyapunov function for all the subsystems, i.e., a continuously differentiable, radially unbounded, positive definite function $V : \mathbb{R}^n \rightarrow \mathbb{R}$, for which the derivative $\dot{V}(x)$ is negative definite along all subsystems' trajectories, then the stability of the hybrid system is guaranteed under arbitrary switching. This provides us with a possible way to solve this problem and this approach is discussed next.

Theorem 4.10 *Consider a hybrid automaton $H = \{Q, X, f, \text{Init}, \text{Inv}, E, G, R\}$. If there exists a continuously differentiable function $V : \mathbb{R}^n \rightarrow \mathbb{R}$ such that*

- (1) *there exist class $\mathcal{K}$ functions α_1 and α_2 such that for all $x \in \bigcup_{q \in Q} \text{Inv}(q)$ we have $\alpha_1(\|x\|) \leq V(x) \leq \alpha_2(\|x\|)$;*
- (2) *there exists a class $\mathcal{K}$ function α_3 such that for each $q \in Q$ and for all $x \in \text{Inv}(q)$, we have $\frac{\partial V}{\partial x} f_q(x) \leq -\alpha_3(\|x\|)$,*

then H is uniformly asymptotically stable and $V(x)$ is called a common Lyapunov function.

The proof of this theorem is similar to what we did for the time-varying case, so it is left as an exercise. Note that the existence of a common Lyapunov function is only a sufficient condition for the stability of H when considering arbitrary switching. The common Lyapunov function approach is conservative and it can be shown that there exist systems that are asymptotically stable for which a common Lyapunov function does not exist.

A natural question then is under what condition there exists a common Lyapunov function. The following converse theorem answers the question of when the hybrid system is globally asymptotically stable.

Theorem 4.11 *There exists a radially unbounded smooth common Lyapunov function $V(x)$ if*

- (1) *the hybrid system H is globally uniformly asymptotically stable;*
- (2) *$\forall q \in Q$, $f(q, x)$ is locally Lipschitz and bounded for x .*

This converse common Lyapunov theorem can be proved by following similar arguments as in the proofs of the classical converse Lyapunov theorem, which is left as an exercise.

Although the converse Lyapunov theorem gives a sufficient condition for the existence of a common Lyapunov function, the task of finding such Lyapunov function for hybrid systems is, in general, challenging. The form of common Lyapunov functions may not necessarily be quadratic, although most of the available results pertain to the existence of common quadratic Lyapunov functions. Additionally, a helpful necessary condition for the existence of a common Lyapunov function is that any convex combination of the individual subsystems is globally asymptotically stable. That is to say, if there exists an unstable convex combination, then the hybrid system H cannot be globally asymptotically stable; see Example 4.8 for a more detailed discussion.

4.3.2 Common Quadratic Lyapunov Functions

As in classical dynamical systems, the problem of constructing a common Lyapunov function for hybrid systems is, in general, hard. However, when considering linear time-invariant dynamics in every discrete state, the problem is greatly simplified and well understood. First, we consider a collection of continuous-time LTI systems

$$\dot{x}(t) = A_i x(t), \quad t \in \mathbb{R}^+, \quad i \in \mathcal{I}, \quad (4.13)$$

where $\mathcal{I} = \{1, 2, \dots, N\}$ stands for a finite index set. For all $i \in \mathcal{I}$, the state matrices $A_i \in \mathbb{R}^{n \times n}$. Note that the origin $x_e = 0$ is a common equilibrium for the systems described in (4.13). The hybrid system of interest is built by allowing arbitrary switching among these LTI systems (4.13). Since no restrictions are imposed on the switching signals, we will not consider particular discrete event dynamics in the hybrid system. This is in the problem of arbitrary switching (linear) systems or of switched linear systems under arbitrary switching.

A common quadratic Lyapunov function (CQLF) for system (4.13) has the form

$$V(x) = x^T P x, \quad (4.14)$$

where $P = P^T$ (symmetric) and $P > 0$ (positive definite). In addition, its time derivative along any trajectory of system (4.13) is negative definite, or alternatively

$$A_i^T P + P A_i = -Q_i, \quad i \in \mathcal{I}, \quad (4.15)$$

where Q_i are symmetric and positive definite for all $i \in \mathcal{I}$, since for all $i \in \mathcal{I}$

$$\begin{aligned} \dot{V}(x) &= \dot{x}^T P x + x^T P \dot{x} \\ &= x^T A_i^T P x + x^T P A_i x \\ &= x^T (A_i^T P + P A_i) x \\ &= -x^T Q_i x, \end{aligned}$$

and $\dot{V}(x) < 0$ if Q_i is positive definite.

The existence of a CQLF for all subsystems (same P) assures the quadratic stability of the hybrid system. Quadratic stability is a special case of exponential stability, which implies asymptotic stability, and has attracted a lot of research interest because of its importance in practice.

A CQLF for system (4.13) can be determined by solving the following linear matrix inequalities (LMIs) [7], where P is a positive definite symmetric matrix, $P \in \mathbb{R}^{n \times n}$, such that

$$P A_i + A_i^T P < 0, \quad \forall i \in \mathcal{I}, \quad (4.16)$$

hold simultaneously. It should be noted that, the standard interior point methods for LMIs may become computationally ineffective as the number of modes increases. This motivates us to identify easily verifiable conditions that guarantee the existence of a CQLF for the system (4.13).

Due to convexity of the condition (4.16), the existence of a CQLF for the system (4.13) requires that any convex combination of the state matrices, namely $\sum_{i \in \mathcal{I}} \alpha_i A_i$ for $\alpha_i \geq 0$ and $\sum_{i \in \mathcal{I}} \alpha_i = 1$, is asymptotically stable (with negative real parts for all eigenvalues of matrices $\sum_{i \in \mathcal{I}} \alpha_i A_i$). We use the following example from [8] to illustrate this condition.

Example 4.8 Consider a switched linear system (4.13) with two state matrices

$$A_1 = \begin{bmatrix} -0.1 & -1 \\ 2 & -0.1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} -0.1 & 2 \\ -1 & -0.1 \end{bmatrix}.$$

Note that A_1 and A_2 are both asymptotically stable, but their convex combination for $\alpha_1 = \alpha_2 = 0.5$, i.e., $0.5A_1 + 0.5A_2 = \begin{bmatrix} -0.1 & 0.5 \\ 0.5 & -0.1 \end{bmatrix}$ is unstable. So, the switching system (4.13) with $\mathcal{I} = \{1, 2\}$ and the above A_1, A_2 as its state matrices cannot be stable under arbitrary switching.

This is very easy to check, but it is necessary only. For example, consider a switching system (4.13) with $\mathcal{I} = \{1, 2\}$ and its state matrices given as

$$A_1 = \begin{bmatrix} -0.1 & -1 \\ 2 & -0.1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} -0.1 & -2 \\ 1 & -0.1 \end{bmatrix}.$$

It can be seen that all convex combinations of A_1 and A_2 are stable, but the arbitrary switching system with these two state matrices is unstable. Actually, the state trajectories of these two subsystems, i.e., $\dot{x} = A_1 x$ and $\dot{x} = A_2 x$, are similar to the case shown in Example 4.6, and as in that example, unbounded state trajectories can be generated as, in this case, by switching between these two stable subsystems. $\square$

4.3.3 Commutative Systems

Let us first look at a special case, where the subsystems' state matrices are pairwise commutative, i.e., $A_i A_j = A_j A_i$ for all $i, j \in \mathcal{I}$. Because of the commutativity, it is easy to derive that

$$A_i^{k_1} A_j^{k_2} = A_j^{k_2} A_i^{k_1},$$

for any non-negative integer k_1 and k_2 , and

$$e^{A_i t_1} e^{A_j t_2} = e^{A_j t_2} e^{A_i t_1},$$

for any non-negative real t_1 and t_2 . By direct computation, it is straightforward to verify that, in this case, the switched system is stable if and only if all its subsystems are stable [9].

Theorem 4.12 *For an arbitrary switching system (4.13), if all subsystem matrices are Hurwitz stable (i.e., all eigenvalues of the state matrix A_i have negative real parts) and pairwise commutative ($A_i A_j = A_j A_i$, $\forall i, j \in \mathcal{I}$), then the system under arbitrary switching is asymptotically stable.*

Proof For any switching signal $\sigma(t)$ and any time $t > 0$, assume $\tau_i(t)$ stands for the total activation time of subsystem i in the period $[0, t]$. By pairwise commutativity, it holds that

$$x(t) = e^{A_1 \tau_1} e^{A_2 \tau_2} \dots e^{A_N \tau_N} x(0),$$

where $\sum_{i=1}^N \tau_i = t$ and $\tau_i \geq 0$.

On the other hand, due to the fact that all A_i are stable, there exist positive scalars α_i and λ_i such that

$$\|e^{A_i \tau_i}\| \leq \alpha_i e^{-\lambda_i \tau_i}.$$

Therefore

$$\begin{aligned} \|x(t)\| &= \|e^{A_1 \tau_1} e^{A_2 \tau_2} \dots e^{A_N \tau_N} x(0)\| \\ &\leq \alpha_1 e^{-\lambda_1 \tau_1} \alpha_2 e^{-\lambda_2 \tau_2} \dots \alpha_N e^{-\lambda_N \tau_N} \|x(0)\|, \end{aligned}$$

which goes to zero as time t goes to infinity.

Since there are no restrictions on switching signals and initial conditions, the hybrid system is asymptotically stable under all possible switching sequences among these subsystems. $\square$

A simple example is the case of Hurwitz diagonal matrices which of course commute with each other. In this case, the above theorem implies that the system under arbitrary switching will be stable.

Note that in the case of commutative systems, a common quadratic Lyapunov function does exist and it can be determined by solving a collection of chained Lyapunov equations as the following theorem shows [9].

Theorem 4.13 *Consider system (4.13) with index set $\mathcal{I} = \{1, \dots, N\}$. Given a symmetric positive definite matrix P_0 , let $P_1, \dots, P_N$ be the unique symmetric positive definite matrices that satisfy the Lyapunov equations*

$$\begin{aligned} A_1^T P_1 + P_1 A_1 &= -P_0, \\ A_i^T P_i + P_i A_i &= -P_{i-1}, \quad i = 2, \dots, N. \end{aligned} \tag{4.17}$$

Then the function $V(x) = x^T P_N x$ is a CQLF for systems $\dot{x}(t) = A_i x(t)$, $i \in \mathcal{I}$.

Proof Let $V(x) = x^T P_N x$. Once $\sigma(t) = i$, the derivative of V along the trajectories of this system is given by

$$\dot{V} = x^T (A_i^T P_N + P_N A_i) x.$$

Hence, it suffices to show that the matrix $A_i^T P_N + P_N A_i$ is negative definite, denoted as $A_i^T P_N + P_N A_i < 0$, for all $i = 1, 2, \dots, N$.

For this purpose, let us denote

$$P_{i,j} = A_i^T P_j + P_j A_i,$$

for $i = 1, 2, \dots, N$ and $j = i, i+1, \dots, N$. If we show that $P_{i,j} < 0$ for $i = 1, 2, \dots, N$ and $j = i, i+1, \dots, N$, then the result follows by choosing $j = N$ for each i . Hence, let $i \in \{1, 2, \dots, N\}$. Now assume that $P_{i,j} < 0$ for some $j \in \{i, i+1, \dots, N\}$. This is a reasonable assumption since $P_{i,i} = -P_{i-1} < 0$ always satisfies the assumption.

Then, by commutativity of A_i and A_{j+1} and the stability of A_{j+1} we find that

$$\begin{aligned} A_{j+1}^T P_{i,j+1} + P_{i,j+1} A_{j+1} &= A_{j+1}^T (A_i^T P_{j+1} + P_{j+1} A_i) + (A_i^T P_{j+1} + P_{j+1} A_i) A_{j+1} \\ &= A_i^T (A_{j+1}^T P_{j+1} + P_{j+1} A_{j+1}) + (A_{j+1}^T P_{j+1} + P_{j+1} A_{j+1}) A_i \\ &= -(A_i^T P_j + P_j A_i) = -P_{i,j}, \end{aligned}$$

which shows that $P_{i,j+1} = -P_{i,j} < 0$. Hence, the claim holds by induction. $\square$

In addition, the matrix P_N is given by

$$P_N = \int_0^\infty e^{A_N^T t_N} \dots \left(\int_0^\infty e^{A_1^T t_1} P_0 e^{A_1 t_1} dt_1 \right) \dots e^{A_N t_N} dt_N. \quad (4.18)$$

This can be easily verified as it is known that the unique solution of the Lyapunov equation $A_1^T P_1 + P_1 A_1 = -P_0$ is given by $P_1 = \int_0^\infty e^{A_1^T t_1} P_0 e^{A_1 t_1} dt_1$. Iteratively using this result, we obtain the form of P_N above.

Note that, for a given choice of P_0 , the matrices $A_1, \dots, A_N$ can be chosen in any order in (4.17) to yield the same solution P_N . This is due to the uniqueness of solutions to Lyapunov equations and the pairwise commutativity between state matrices.

Similar results to the above can be derived for discrete-time systems, namely

Theorem 4.14 Consider a collection of discrete-time LTI system

$$x(k+1) = A_i x(k), \quad i = 1, \dots, N, \quad k \in \mathbb{Z}^+, \quad (4.19)$$

where $\mathbb{Z}^+$ stands for the set of non-negative integers. It is assumed that matrices A_i commute pairwise and are all discrete-time asymptotically stable, i.e., all eigenval-

ues of A_i are within the unit circle. Given a symmetric positive definite matrix P_0 , let $P_1, \dots, P_N$ be the unique symmetric positive definite matrices that satisfy the Lyapunov equations

$$A_1^T P_1 A_1 + P_1 = -P_0, \quad (4.20)$$

$$A_i^T P_i A_i + P_i = -P_{i-1}, \quad i = 2, \dots, N. \quad (4.21)$$

Then the function $V(x) = x^T P_N x$ is a CQLF for the systems $x(k+1) = A_i x(k)$, with $i = 1, \dots, N$.

We left the proof for the discrete case as an exercise.

Finally, the above results assert that commutativity assures the existence of a CQLF, but the converse is not true. To show this, consider a counterexample, where in (4.13)

$$A_1 = \begin{bmatrix} -2 & 1 \\ 1 & -2 \end{bmatrix}, \quad A_2 = \begin{bmatrix} -1 & 1 \\ 1 & -2 \end{bmatrix}.$$

Under arbitrary switching, it can be verified that $V(x) = x^T x$ is a CQLF, note, however, that $A_1 A_2 \neq A_2 A_1$.

4.3.4 Triangular Systems

Besides the case of commutative state matrices, it has been shown that CQLF exists for arbitrary switching systems with either all upper or lower triangular state matrices [8]. To illustrate why a CQLF exists for triangular systems, we consider a simple case with $\mathcal{I} = \{1, 2\}$. Let two state matrices be given as upper-triangular matrices

$$A_1 = \begin{bmatrix} -a_1 & b_1 \\ 0 & -c_1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} -a_2 & b_2 \\ 0 & -c_2 \end{bmatrix}.$$

Since A_1 and A_2 are Hurwitz stable (all eigenvalues have negative real parts), we have that a_i and c_i are positive for $i = 1, 2$.

The second component of the state x follows the equation

$$\dot{x}_2 = -c_i x_2,$$

where c_i is either c_1 or c_2 which are both positive due to the stability assumption for A_1 and A_2 . Thus, x_2 decays to zero exponentially fast. Furthermore, the first component of x satisfies

$$\dot{x}_1 = -a_i x_1 + b_i x_2,$$

which can be seen as an exponentially stable system $\dot{x}_1 = -a_i x_1$ perturbed by the decaying disturbance $b_i x_2$. Thus, x_1 also converges to zero exponentially fast. The

argument can be easily extended to more than two subsystems and higher dimensional systems, and even to nonlinear switching systems, see e.g., [8].

Actually, it can be shown that here the CQLF is of a diagonal form, i.e., $P = \begin{bmatrix} p_1 & 0 \\ 0 & p_2 \end{bmatrix}$ with $p_1, p_2 > 0$. To show this, we substitute the diagonal P into the left-hand side of Lyapunov equations to obtain

$$A_i^T P + P A_i = \begin{bmatrix} -2p_1 a_i & p_1 b_i \\ p_1 b_i & -2p_2 c_i \end{bmatrix}, \quad i = 1, 2.$$

To guarantee that $A_i^T P + P A_i$ is negative definite for both $i = 1, 2$, select any $p_1 > 0$ to make $-2p_1 a_i < 0$. In addition, it is always feasible to select p_2 large enough to make the determinant of $A_i^T P + P A_i$

$$\begin{vmatrix} -2p_1 a_i & p_1 b_i \\ p_1 b_i & -2p_2 c_i \end{vmatrix} = 4p_2 p_1 a_i c_i - p_1^2 b_i^2 \quad (4.22)$$

positive for both $i = 1, 2$. Then (4.15) is satisfied and we can conclude that the system is exponentially stable. Again, this construction can be extended to higher dimensional systems.

4.3.5 A Lie Algebraic Condition

A more general condition on the existence of CQLF for System (4.13) can be characterized through the Lie algebra generated by the subsystems' state matrices [10]. The matrix Lie algebra generated by the matrices $A_i, i \in \mathcal{I}$ (with respect to the standard Lie bracket $[A_i, A_j] = A_i A_j - A_j A_i$), denoted as $\mathfrak{g} = \{A_i : i \in \mathcal{I}\}_{LA}$, is a linear space (over $\mathbb{R}$) spanned by the iterated Lie brackets of these matrices. For example, consider the Lie algebra generated by two matrices A_1 and A_2 , then

$$\mathfrak{g} = \{A_1, A_2\}_{LA} = \text{span}\{A_1, A_2, [A_1, A_2], [A_1, [A_1, A_2]], \dots\}. \quad (4.23)$$

Given a Lie algebra $\mathfrak{g}$, the sequence $\mathfrak{g}^{(k)}$ is defined iteratively as

$$\mathfrak{g}^{(1)} = \mathfrak{g}, \quad (4.24)$$

$$\mathfrak{g}^{(k+1)} = [\mathfrak{g}^{(k)}, \mathfrak{g}^{(k)}], \quad (4.25)$$

where $[\mathfrak{g}^{(k)}, \mathfrak{g}^{(k)}]$ is a linear space spanned by all the products $[a, b]$ with $a \in \mathfrak{g}^{(k)}$ and $b \in \mathfrak{g}^{(k)}$. It is easy to show that $\mathfrak{g}^{(k+1)} \subset \mathfrak{g}^{(k)}$. A Lie algebra $\mathfrak{g}$ is called *solvable* if $\mathfrak{g}^{(k)} = 0$ for a finite k . As a special case, if A_i commute pairwise, then its Lie algebra is solvable as $\mathfrak{g}^{(1)} = \mathfrak{g} = \text{span}\{A_i; i \in \mathcal{I}\}$, and $\mathfrak{g}^{(2)} = 0$ since $[A_i, A_j] = A_i A_j - A_j A_i = 0$ for all $i, j \in \mathcal{I}$. It has been shown in [10] that, if the Lie algebra

generated by the state matrices A_i is solvable, then the switched system (4.13) is asymptotically stable under arbitrary switching signals.

Theorem 4.15 *If all the matrices A_i , $i \in \mathcal{I}$ in system (4.13) are Hurwitz and the Lie algebra $\{A_i : i \in \mathcal{I}\}_{LA}$ is solvable then there exists a CQLF.*

This is because the matrices A_i , $i \in \mathcal{I}$ in a solvable Lie algebra can be simultaneously put in upper-triangular form and this implies stability since a family of linear systems with stable upper-triangular matrices possess a common quadratic Lyapunov function (see previous sections). Interested readers may refer to [8, 10] for more detailed and formal discussions on the Lie algebraic conditions. Extensions to nonlinear switched systems can be found in [11].

Note that the commutative and triangular conditions discussed in Sects. 4.3.3 and 4.3.4 are special cases of the above result, and the solvability condition for the Lie algebra implies the existence of a CQLF. However, there exist arbitrary switching systems that are asymptotically stable but do not have a CQLF. We use the following example from [12] to illustrate the claim.

Example 4.9 Consider a switched linear system $\dot{x} = A_i x$ with

$$A_1 = \begin{bmatrix} -1 & -1 \\ 1 & -1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} -1 & -8 \\ \frac{1}{8} & -1 \end{bmatrix}.$$

Based on a straightforward computation, it can be shown that there is no CQLF for this switched system. However, the switched system is asymptotically stable, as it can be verified by the “worst case switching” argument. The worst case switching basically modulates the switching between the vector fields $A_1 x$ and $A_2 x$, for the current state x , following a simple rule: always switch to the subsystem with the vector field pointing away from the origin. It represents the worst scenario with respect to stability concerns, and if the system remains stable (the distance to the origin eventually decreased when the state trajectory revisits the same radius line) under the worst case switching, then the switched system is stable under arbitrary switching signals. $\square$

4.3.6 Switched Quadratic Lyapunov Functions

In view of the above example, the existence of a CQLF is only sufficient for the asymptotic or exponential stability of hybrid systems under arbitrary switching signals. A less conservative class of Lyapunov functions, called switched quadratic Lyapunov functions, was proposed in the literature [13].

Here, we investigate the stability of the following discrete-time arbitrary switching LTI systems

$$x(k+1) = A_i x(k), \quad k \in \mathbb{Z}^+, \quad (4.26)$$

where $x \in \mathbb{R}^n$, and $i \in \mathcal{I}$. Basically, since every subsystem is asymptotically stable (with the origin as the common equilibrium point), there exists a positive definite symmetric matrix P_i that solves the Lyapunov equation for each i th subsystem

$$A_i^T P_i A_i - P_i < 0,$$

for all $i \in \mathcal{I}$. Next, these matrices P_i are patched together based on the switching signals $\sigma(k)$ to construct a global Lyapunov function as

$$V(k, x(k)) = x^T(k) P_{\sigma(k)} x(k), \quad (4.27)$$

where $\sigma(k) : k \rightarrow \mathcal{I}$ stands for the switching signal at step k . Since all P_i are positive definite, it is clear that the function $V(k, x(k)) = x^T(k) P_{\sigma(k)} x(k)$ is positive definite. If it further holds that $\Delta V(k, x(k)) = V(k+1, x(k+1)) - V(k, x(k))$ is negative definite along the solution of (4.26), then the origin of the system (4.26) is globally asymptotically stable as stated in the following theorem from [13].

Theorem 4.16 *If there exist positive definite symmetric matrices $P_i \in \mathbb{R}^{n \times n}$ ($P_i = P_i^T$) for $i \in \mathcal{I}$, satisfying*

$$\begin{bmatrix} P_i & A_i^T P_j \\ P_j A_i & P_j \end{bmatrix} > 0. \quad (4.28)$$

for all $i, j \in \mathcal{I}$, then the discrete-time arbitrary switching system (4.26) is asymptotically stable.

Proof Consider the function

$$V(k, x(k)) = x^T(k) P_{\sigma(k)} x(k), \quad (4.29)$$

as a Lyapunov function candidate, where $\sigma(k) : k \rightarrow \mathcal{I}$ stands for the switching signal at step k .

The function $V(k, x(k))$ is a positive definite function, decrescent, and radially unbounded since $V(k, 0) = 0, \forall k \geq 0$ and

$$\alpha \|x(k)\|^2 \leq V(k, x(k)) \leq \beta \|x(k)\|^2,$$

for all $x(k) \in \mathbb{R}^n$ and $k \geq 0$ with $\alpha = \min_{i \in \mathcal{I}} \lambda_{\min}(P_i)$ and $\beta = \max_{i \in \mathcal{I}} \lambda_{\max}(P_i)$.¹

For any switching signal $\sigma(k)$

$$\begin{aligned} \Delta V(k, x(k)) &= V(k+1, x(k+1)) - V(k, x(k)) \\ &= x^T(k+1) P_{\sigma(k+1)} x(k+1) - x^T(k) P_{\sigma(k)} x(k). \end{aligned}$$

¹ The notation $\lambda_{\min}(P_i)$ stands for the minimum eigenvalue of P_i , while $\lambda_{\max}(P_i)$ stands for the maximum eigenvalue of P_i . Since every matrix P_i is positive definite and symmetric, the constants α, β are both positive scalars.

Without loss of generality, it is assumed that $\sigma(k) = i$ and $\sigma(k+1) = j$ for $i, j \in \mathcal{I}$. Hence

$$\begin{aligned}\Delta V(k, x(k)) &= x^T(k+1)P_{\sigma(k+1)}x(k+1) - x^T(k)P_{\sigma(k)}x(k) \\ &= x(k)^T A_i^T P_j A_i x(k) - x^T(k)P_i x(k).\end{aligned}$$

Since

$$\begin{bmatrix} P_i & A_i^T P_j \\ P_j A_i & P_j \end{bmatrix} > 0, \quad (4.30)$$

for all $i, j \in \mathcal{I}$, it is equivalent by Schur complement formula² to

$$P_i - A_i^T P_j A_i > 0,$$

for all $i, j \in \mathcal{I}$. Therefore,

$$\Delta V(k, x(k)) = x(k)^T (A_i^T P_j A_i - P_i)x(k) < 0,$$

for all $x(k) \in \mathbb{R}^n$, which means that $\Delta V(k, x(k))$ is negative definite along the solutions of (4.26). $\square$

In view of the above, stability checking for arbitrarily switching linear systems can be performed by solving LMIs. It is clear that when $P_i = P_j$ for all $i, j \in \mathcal{I}$, the switched quadratic Lyapunov function becomes the CQLF. Therefore, the stability criterion based on the switched quadratic Lyapunov function generalizes the CQLF approach and usually gives us less conservative results.

Interested readers may also refer to [6, 8, 14–16] for further discussions and references on stability conditions under arbitrary switching.

4.4 Constrained Switching

Switched systems may fail to preserve stability under arbitrary switching. On the other hand, one may have some knowledge about possible discrete event dynamics in the hybrid system, which may be deduced from partitions of the state space, such as invariant sets, guard sets, etc. This knowledge may be transferred to restrictions on possible discrete transitions. For example, there must exist certain lower bounds on the time interval between two successive switchings, which may be due to the fact that the state trajectories spend some period of time in traveling from the initial set to certain guard sets, if these two sets are separated. As a typical example,

² Schur complement formula tells us that the LMI $\begin{bmatrix} Q & S \\ S^T & R \end{bmatrix} > 0$ (with $Q = Q^T, R = R^T$) is equivalent to $R > 0$ and $Q - SR^{-1}S^T > 0$.

consider changing gears in a car where only certain switching sequences are used and the switch happens at certain instants dependent on the engine's rpm and torque requirements. With such kind of *prior* knowledge about the switching signals, we may derive less conservative stability results for a given hybrid system instead of just using the worst case arguments.

In this section, we are interested in the stability of hybrid systems with *constrained switching* and also with the absence of a common Lyapunov function.

4.4.1 Stability with Dwell Time

By studying Example 4.7 in Sect. 4.2, where divergent trajectories are generated through switching between two stable systems, one notices that the unboundedness is caused by the failure to absorb the energy increase caused by the switching. In addition, when there is an unstable subsystem (e.g., controller failure or sensor fault), it may cause instability if one either stays too long there or switches too frequently to the unstable subsystem. Therefore, a natural question is what if we restrict the switching signal to some constrained subclass of signals. Intuitively, if one stays at stable subsystems long enough and switches less frequently, i.e., slow switching, one may dissipate the energy caused by switching or unstable modes, and maybe possible to attain stability. These ideas are proved to be reasonable and are captured by concepts such as switching signals with a bounded *dwell time* [17] or a bounded *average dwell time* [18] that are introduced below.

The most direct way to characterize the concept of slow switching is perhaps to request a lower bound on two consecutive switching times, which is known as the dwell time [17].

Definition 4.11 A positive scalar τ_d is called the *dwell time* if the time interval between any two consecutive switchings is no smaller than τ_d .

Under the assumption that all subsystems in the hybrid system (4.13) are exponentially stable with the origin as the common equilibrium point, it can be shown that there exists a scalar $\tau_d > 0$ such that the hybrid system remains exponentially stable if the dwell time is larger than τ_d . In addition, the rate of convergence can be estimated. Formally, we state the result as follows:

Theorem 4.17 Let all subsystems in (4.13), $\dot{x} = A_i x$ for $i \in \mathcal{I}$, be exponentially stable with stability margin λ_0 (for $\lambda_0 > 0$), i.e., the state transition matrix satisfies $\|e^{A_i(t-t')}\| \leq \mu e^{-\lambda_0(t-t')}$ for any $t \geq t' \geq 0$ and all $i \in \mathcal{I}$, where constant $\mu \geq 1$ and $\|\cdot\|$ denotes the induced Euclidean norm. For every $0 < \lambda < \lambda_0$, the switched system (4.13) is stable with stability margin λ for any admissible switching signal $\sigma : [0, +\infty) \rightarrow \mathcal{I}$ with dwell time $\tau_d \geq \frac{\ln \mu}{\lambda_0 - \lambda}$.

Proof Consider any initial condition $x(0) = x_0$, and denote the switching time in the interval $[0, t)$ as $\{t_1, t_2, \dots, t_k\}$. Then, we have

$$x(t) = e^{A_{\sigma(t_k)}(t-t_k)} e^{A_{\sigma(t_{k-1})}(t_k-t_{k-1})} \dots e^{A_{\sigma(t_1)}(t_2-t_1)} e^{A_{\sigma(0)}t_1} x_0. \quad (4.31)$$

Since the switching signal $\sigma(t)$ has dwell time no less than τ_d , i.e., $t_i - t_{i-1} \geq \tau_d$ for all $i = \{1, 2, \dots, k\}$. Then, for every i

$$\|e^{A_{\sigma(t_i)}(t_i-t_{i-1})}\| \leq \mu e^{-\lambda_0(t_i-t_{i-1})} \leq \mu e^{-\lambda_0\tau_d}. \quad (4.32)$$

In addition, since the dwell time is lower bounded by $\tau_d \geq \frac{\ln \mu}{\lambda_0 - \lambda}$ for some $0 < \lambda < \lambda_0$, we have

$$\begin{aligned} \|e^{A_{\sigma(t_{i-1})}(t_i-t_{i-1})}\| &\leq \mu e^{-\lambda_0(t_i-t_{i-1})} \leq e^{(\lambda_0-\lambda)} e^{-\lambda_0(t_i-t_{i-1})} \\ &= e^{-(\lambda_0-\lambda)(t_i-t_{i-1}-\tau_d)-\lambda(t_i-t_{i-1})} \leq e^{-\lambda(t_i-t_{i-1})}. \end{aligned} \quad (4.33)$$

Hence, we obtain

$$\begin{aligned} \|x(t)\| &\leq \|e^{A_{\sigma(t_k)}(t-t_k)}\| \|e^{A_{\sigma(t_{k-1})}(t_k-t_{k-1})}\| \dots \|e^{A_{\sigma(t_1)}(t_2-t_1)}\| \|e^{A_{\sigma(0)}t_1}\| \|x_0\| \\ &\leq e^{-\lambda t} \|x_0\|. \end{aligned} \quad (4.34)$$

This proves the theorem. $\square$

The result implies that if the switching occurs slowly (i.e., each mode is active for at least τ_d time units), then the hybrid system will be exponentially stable with a decay rate λ no larger than the smallest of the decay rates of the subsystems. This result can be extended to switched nonlinear systems, which is left as an exercise.

4.4.2 Stability with Average Dwell Time

In fact, it really does not matter if one occasionally has a smaller dwell time between switchings, provided this does not occur too frequently. This concept is captured by the concept of “average dwell time” [18].

Definition 4.12 A positive constant τ_a is called the *average dwell time* if $N_\sigma(t) \leq N_0 + \frac{t}{\tau_a}$ holds for all $t > 0$ and some scalar $N_0 \geq 0$, where $N_\sigma(t)$ denotes the number of discontinuities of a given switching signal σ over $[0, t)$.

Here the constant τ_a is called the *average dwell time* and N_0 the *chatter bound*. When a class of switching signals satisfies

$$N_\sigma(t) \leq N_0 + \frac{t}{\tau_a}.$$

we say that we have an average dwell time no less than τ_a because

$$N_\sigma(t) \leq N_0 + \frac{t}{\tau_a} \Leftrightarrow \frac{t}{N_\sigma(t) - N_0} \geq \tau_a,$$

which means that on average the “dwell time” between any two consecutive switchings is no smaller than τ_a . The idea is that there may exist consecutive switching separated by less than τ_a , but the average time interval between consecutive switchings is not less than τ_a [18].

Theorem 4.18 *Assume that all subsystems, $\dot{x} = A_i x$ for $i \in \mathcal{I}$, in the hybrid system are exponentially stable. Then, there exists a scalar $\tau_a > 0$ such that the hybrid system is exponentially stable if the average dwell time is larger than τ_a .*

Proof Assume that the switching time instants are $t_1, t_2, \dots$, and that the i_k th subsystems is activated on $[t_k, t_{k+1})$ ($t_0 = 0$). For any $t_m \leq t < t_{m+1}$

$$x(t) = e^{A_{i_m}(t-t_m)} e^{A_{i_{m-1}}(t_m-t_{m-1})} \dots e^{A_{i_0}(t_1-t_0)} x_0. \quad (4.35)$$

Assume that $\|e^{A_j t}\| \leq \mu e^{-\alpha t}$. Then

$$\|x(t)\| \leq \mu^m e^{-\alpha t} \|x_0\| = \mu^{N_\sigma(t)} e^{-\alpha t} \|x_0\| = e^{-\alpha t + N_\sigma(t) \ln \mu} \|x_0\|. \quad (4.36)$$

If for $0 < \alpha_0 < \alpha$, we choose the average dwell time as

$$N_\sigma(t) \leq N_0 + \frac{t}{\tau_a}, \tau_a = \frac{\ln \mu}{\alpha - \alpha_0} \iff -\alpha t + N_\sigma(t) \ln \mu \leq N_0 \ln \mu - \alpha_0 t, \quad (4.37)$$

then obviously $\|x(t)\| \leq \mu^{N_0} e^{-\alpha_0 t} \|x_0\|$, which implies that the switched system is exponentially stable. $\square$

The above result highlights the fact that switching among stable (linear) systems always leads to a stable switching system when a delay in switching is permitted. Compare this result with the results earlier in this chapter on arbitrary switching where instabilities may occur.

4.4.3 Discrete-Time Case

The stability results for slow switching systems can be extended to discrete-time switched systems, where the dwell time τ_d (or average dwell time τ_a) is counted as the number of sampling periods.

Definition 4.13 A positive constant τ_a is called the *average dwell time* if $N_\sigma(k) \leq N_0 + \frac{k}{\tau_a}$ holds for all $k > 0$ and some scalar $N_0 \geq 0$, where $N_\sigma(k)$ denotes the number of switchings of a given switching signal σ over $[0, k)$.

Similar results can be developed for the discrete-time case.

Theorem 4.19 *Assume that there exists a positive scalar λ_0 such that A_i / λ_0 is stable for all $i \in \mathcal{I}$. Then, for any given $\lambda \in (\lambda_0, 1)$, there is a finite constant τ_a such that the*

hybrid system, with $x(k+1) = A_i x(k)$ as its subsystems in (4.26), is exponentially stable with decay rate λ provided the average dwell time is no less than τ_a .

Proof Let $k_1, k_2, \dots$ be the time points that switching occurs, and write σ_j for the value of $\sigma(k)$ on $[k_{j-1}, k_j)$. Then for any k satisfying $k_0 = 0 < \dots < k_j < k < k_{j+1}$, we have

$$x(k) = A_{\sigma_{j+1}}^{k-k_j} A_{\sigma_j}^{k_j-k_{j-1}} \dots A_{\sigma_1}^{k_1} x_0.$$

It is known that for Schur stable systems $x(k+1) = A_i x(k)$, the eigenvalues of A_i are strictly inside the unit circle and there always exist positive scalars $\lambda_i < 1$ and h_i 's, such that $|A_i^k| \leq h_i \lambda_i^k$ for any $k \geq 1$. Therefore

$$\begin{aligned} \|x(k)\| &\leq \left(\prod_{q=1}^{j+1} h_{\sigma_q} \lambda_{\sigma_q}^{k_q - k_{q-1}} \right) \|x_0\| \\ &\leq c h^{N_\sigma(k)} (\lambda^*)^k \|x_0\|, \end{aligned}$$

where $h = \max_i h_i$, $c = h$, and $\lambda^* = \max_i \lambda_i < 1$.

Since the switching signal σ satisfies an average dwell time scheme, i.e.,

$$N_\sigma(k) \leq N_0 + \frac{k}{\tau_a},$$

for some positive scalars N_0, τ_a . Now set $N_0 = \frac{\ln h_0}{\ln h}$ and $\tau_a = \frac{\ln h}{\ln \lambda - \ln \lambda^*}$ for positive scalars h_0 and λ (with $\lambda^* < \lambda < 1$). Then, the above average dwell time condition is equivalent to

$$h^{N_\sigma(k)} (\lambda^*)^k \leq h_0 \lambda^k. \quad (4.38)$$

Hence

$$\|x(k)\| \leq c h^{N_\sigma(k)} (\lambda^*)^k \|x_0\| \leq c h_0 \lambda^k \|x_0\|,$$

which implies exponential stability for the switched system with decay rate $\lambda_i \leq \lambda < 1$. $\square$

It is possible to extend the average dwell time technique to the case where both stable and unstable subsystems coexist. When one considers unstable dynamics, slow switching (i.e., long enough dwell or average dwell time) is not sufficient for stability, and it is also required to make sure that the switched system does not spend too much time in the unstable subsystems. The reason to consider unstable subsystems in switched systems is because there are cases where switching to unstable subsystems may be unavoidable; such is the case, e.g., when controller/sensor failure occurs or there are packet dropouts in communication. It is interesting to identify conditions under which the stability of the switched systems is still preserved [19–22].

Example 4.10 To illustrate the stability conditions, we consider a switched linear system (4.13) with $\mathcal{I} = \{1, 2\}$ and

$$A_1 = \begin{bmatrix} 1 & -5 \\ 0 & 2 \end{bmatrix}, \quad A_2 = \begin{bmatrix} -5 & -4 \\ -1 & -2 \end{bmatrix}.$$

The eigenvalues of A_1 are 1 and 2, therefore, $\dot{x} = A_1x$ is an unstable system. The eigenvalues of A_2 are -6 and -1 , thus $\dot{x} = A_2x$ is a stable system. Consider any initial condition $x(0) = x_0$. Let $\bar{\lambda}_1 = \lambda_{\max}(A_1) = 2$, $\bar{\lambda}_2 = \lambda_{\max}(A_2) = -1$, T_1 denote the total activation time of unstable subsystem $\dot{x} = A_1x$ during $[t_0, t)$ and thus the total activation time of stable subsystem $\dot{x} = A_2x$ is $t - T_1$. Without loss of generality, we assume that the system $\dot{x} = A_1x$ is activated during $[t_{2j}, t_{2j+1})$ and the system $\dot{x} = A_2x$ is activated during $[t_{2j+1}, t_{2j+2})$, $j = 0, 1, 2, \dots$, where $t_0 = 0$. For any $t \in [t_{2j+1}, t_{2j+2})$, we have

$$x(t) = e^{A_2(t-t_{2j+1})} e^{A_1(t_{2j+1}-t_{2j})} \dots e^{A_2(t_2-t_1)} e^{A_1(t_1-t_0)} x_0.$$

Note that there exists constants $\mu \geq 1$ such that

$$\|e^{A_1(t_{2j+1}-t_{2j})}\| \leq \mu e^{\lambda_{\max}(A_1)(t_{2j+1}-t_{2j})} = \mu e^{\bar{\lambda}_1(t_{2j+1}-t_{2j})},$$

$$\|e^{A_2(t_{2j}-t_{2j-1})}\| \leq \mu e^{\lambda_{\max}(A_2)(t_{2j}-t_{2j-1})} = \mu e^{\bar{\lambda}_2(t_{2j}-t_{2j-1})}.$$

Therefore, we have

$$\begin{aligned} \|x(t)\| &\leq \|e^{A_2(t-t_{2j+1})}\| \|e^{A_1(t_{2j+1}-t_{2j})}\| \dots \|e^{A_2(t_2-t_1)}\| \|e^{A_1(t_1-t_0)}\| \|x_0\| \\ &\leq \mu^j e^{\bar{\lambda}_1 T_1 + \bar{\lambda}_2(t-T_1)} \|x_0\| = \mu^{N_\sigma(t)} e^{\bar{\lambda}_1 T_1 + \bar{\lambda}_2(t-T_1)} \|x_0\| \\ &= e^{N_\sigma(t) \ln \mu + \bar{\lambda}_1 T_1 + \bar{\lambda}_2(t-T_1)} \|x_0\|, \end{aligned}$$

where $N_\sigma(t)$ is defined in Definition 3.5. Similarly, we have $\|x(t)\| \leq \mu^{N_\sigma(t)} e^{\bar{\lambda}_1 T_1 + \bar{\lambda}_2(t-T_1)} \|x_0\|$ for any $t \in [t_{2j}, t_{2j+1})$. Let $N_\sigma(t) \leq N_0 + \frac{t}{\tau_a}$, τ_a be the average dwell time. It is clear that, if the following condition

$$\left(N_0 + \frac{t}{\tau_a}\right) \ln \mu + \bar{\lambda}_1 T_1 + \bar{\lambda}_2(t - T_1) \leq N_0 \ln \mu - \lambda_0 t, \quad (4.39)$$

holds, then we have $\|x(t)\| \leq \mu^{N_0} e^{-\lambda_0 t} \|x_0\|$ with constant $0 < \lambda_0 < -\bar{\lambda}_2$, which means that the switched system is exponentially stable. A sufficient condition for inequality (4.39) is

$$\begin{aligned} \bar{\lambda}_1 T_1 + \bar{\lambda}_2(t - T_1) &\leq -\lambda^* t \\ \tau_a &\geq \tau_a^* = \frac{\ln \mu}{\lambda^* - \lambda_0}, \end{aligned}$$

where $0 < \lambda_0 < \lambda^*$. Suppose the desired exponential decay rate is $\lambda_0 = 0.5$. Let $\lambda^* = 0.7$, $\mu = e$. Then, we have

$$\frac{T_1}{t} \leq \frac{-\lambda^* - \bar{\lambda}_2}{\bar{\lambda}_1 - \bar{\lambda}_2} = \frac{-0.7 + 1}{2 + 1} = \frac{1}{10}$$

and the average dwell time $\tau_a^* = \frac{\ln \mu}{\lambda^* - \lambda_0} = \frac{1}{0.7 - 0.5} = 5$. Therefore, we could construct the following time-driven periodic switching law: Activate the unstable subsystem $\dot{x} = A_1 x$ during $[0, 1)$ s, then immediately activate the stable subsystem $\dot{x} = A_2 x$ for a time interval of length 9 s, then again activate the unstable subsystem $\dot{x} = A_1 x$ for the time interval of length of 1 s. Thus, the time period of the switching law is 10 s and the average dwell time $\tau_a = \frac{10}{2} \geq \tau_a^* = 5$. Under the periodic switching law, the switched system is exponentially stable in the sense that $\|x(t)\| \leq \mu_0 e^{-0.5t} \|x_0\|$ with constants $\mu_0 = e^{N_0}$ and $N_0 > 0$. $\square$

It should be emphasized that not all restrictions on switching signals can be easily captured by the slow switching formulation discussed above. For example, it is not straightforward to transform the invariant set constraints, guard set constraints and the like into dwell time or average dwell time restrictions as these constraints are state dependent in the form of partitions of the state space. Hence, more general tools to study hybrid system stability are needed. In the next section, we introduce multiple Lyapunov functions to study the stability of hybrid systems.

4.5 Multiple Lyapunov Functions

As discussed in Sect. 4.3, the existence of a common Lyapunov function implies the stability of a hybrid system under arbitrary switching. This is a conservative result because there are stable switched systems, where a common Lyapunov function does not exist. In addition, in practice, the switching of a hybrid system may not be arbitrary. The stability analysis under constrained switching has been usually pursued in the framework of multiple Lyapunov functions (MLF). The basic idea is using multiple Lyapunov or Lyapunov-like functions, each corresponding to a single subsystem or certain region in the state space, concatenated together to produce a non-traditional Lyapunov function. This is non-traditional in the sense that the MLF may not be monotonically decreasing along the state trajectories, or may have discontinuities and be only piecewise differentiable. The reason for considering non-traditional Lyapunov functions is that, traditional Lyapunov function may not exist for certain hybrid systems with restricted switching signals. For such cases, one may still construct a collection of appropriate Lyapunov-like functions, which only requires non-positive Lie-derivatives for certain subsystems in a particular region of the state space, instead of being negative globally.

4.5.1 Multiple Lyapunov Functions Theorem

Lyapunov-like functions are defined as a family of real-valued functions $\{V_i, i \in \mathcal{I}\}$, each associated with the vector field $\dot{x} = f_i(x)$.

Definition 4.14 A subsystem has an associated Lyapunov-like function V_i in region Ω_i , when

- (1) there exist constant scalars $\beta_i \geq \alpha_i > 0$ such that

$$\alpha_i \|x\|^2 \leq V_i(x) \leq \beta_i \|x\|^2,$$

hold for any $x \in \Omega_i$;

- (2) for all $x \in \Omega_i$ and $x \neq 0$, $\dot{V}_i(x) < 0$.

Here $\dot{V}_i(x) = \frac{\partial V_i(x)}{\partial x} f_i(x)$, where the partial derivative of V with respect to the column vector x is defined by

$$\frac{\partial V_i(x)}{\partial x} := \left[\frac{\partial V_i(x)}{\partial x_1}, \dots, \frac{\partial V_i(x)}{\partial x_n} \right], \quad (4.40)$$

which is a row vector. The first condition implies positiveness and radial unboundedness for $V_i(x)$ when $x \in \Omega_i$, while the second condition guarantees decreasing energy (value of function $V_i(x)$) along trajectories of subsystem i inside Ω_i . If these regions Ω_i cover the whole space, then a cluster of Lyapunov-like functions is obtained. Concatenating these Lyapunov-like functions together, a non-traditional Lyapunov function is obtained, called *multiple Lyapunov function* (MLF), which can be used to study the global stability of switched systems. The MLF approach is proved to be a powerful tool for studying the stability of switched systems.

Theorem 4.20 Consider a switched system modeled as a hybrid automaton H with $x_e = 0$ as its equilibrium point (i.e., $f(q, 0) = 0$ for all $q \in Q$) and $R((q_i, q_j), x) = x$ for any $q_i, q_j \in Q$. Assume that there exist an open set $D \subseteq X$ and a family of continuously differentiable functions $V_q : D \rightarrow \mathbb{R}$, $q \in Q$ such that for all $q \in Q$

- (1) $V_q(0) = 0$;
- (2) $V_q(x) > 0$ for all $x \in D \setminus \{0\}$;
- (3) $\frac{\partial V_q(x)}{\partial x} f(q, x) \leq 0$ for all $x \in D$.

If for all $(\tau, q, x) \in \mathcal{E}_H(q_0, x_0)$ with $(q_0, x_0) \in \text{Init}$ and for all $\hat{q} \in Q$, the sequence

$$\{V_{q(\tau_i)}(x(\tau_i)) | q(\tau_i) = \hat{q}\},$$

is non-increasing (or empty), then $x_e = 0$ is a stable equilibrium of H .

Proof We outline the proof for the case of $Q = \{1, 2\}$. First, the ϵ -ball is defined as $B_\epsilon = \{x | \|x\| \leq \epsilon, x \in \mathbb{R}^n\}$, with $\epsilon \geq 0$. Given an ϵ -ball $B_\epsilon \subseteq D$, we pick the constants c_i so that the level set $\{x | V_i(x) \leq c_i, x \in \mathbb{R}^n\}$ for $i = 1, 2$ is contained in

B_ϵ , i.e., $\{x|V_i(x) \leq c_i, x \in \mathbb{R}^n\} \subseteq B_\epsilon$. Furthermore, we pick constants c'_i so that the level set $\{x|V_i(x) \leq c'_i, x \in \mathbb{R}^n\}$ for $i = 1, 2$ is contained in the intersection of two level sets $\{x|V_1(x) \leq c_1\}$ and $\{x|V_2(x) \leq c_2\}$ respectively, i.e.,

$$\{x|V_i(x) \leq c'_i\} \subseteq \{x|V_1(x) \leq c_1\} \cap \{x|V_2(x) \leq c_2\},$$

for both $i = 1$ and 2 . Finally, we pick a constant δ such that the δ -ball, $B_\delta = \{x|\|x\| \leq \delta, x \in \mathbb{R}^n\}$, is contained in the intersection of two level sets $\{x|V_1(x) \leq c'_1\}$ and $\{x|V_2(x) \leq c'_2\}$. i.e.,

$$B_\delta \subseteq \{x|V_1(x) \leq c'_1\} \cap \{x|V_2(x) \leq c'_2\}.$$

These sets are illustrated in Fig. 4.7. For the rest we will argue that all trajectories starting with initial conditions inside the δ -ball will remain in the ϵ -ball for ever. To show why this is the case, we pick an arbitrary initial state x_0 from B_δ and assume that the initial mode is $q = 1$ and the state trajectory follows $\dot{x} = f(1, x)$. Since $x_0 \in B_\delta \subseteq \{x|V_1(x) \leq c'_1\}$ and $\frac{\partial V_1(x)}{\partial x} f(1, x) \leq 0$, $x(t)$ will not leave the level set $\{x|V_1(x) \leq c'_1\}$ before the mode switches to $q = 2$. Assume that the switching occurs at t_1 , and we have $x(t_1) \in \{x|V_1(x) \leq c'_1\} \subseteq \{x|V_2(x) \leq c_2\}$. Similarly, we argue that $x(t)$ will not leave the level set $\{x|V_2(x) \leq c_2\}$ before it switches back to $q = 1$. Since the level set $\{x|V_2(x) \leq c_2\} \subseteq B_\epsilon$, the state trajectory stays in the ϵ -ball. Assume that the second switching occurs at t_2 , and the hybrid system switches back

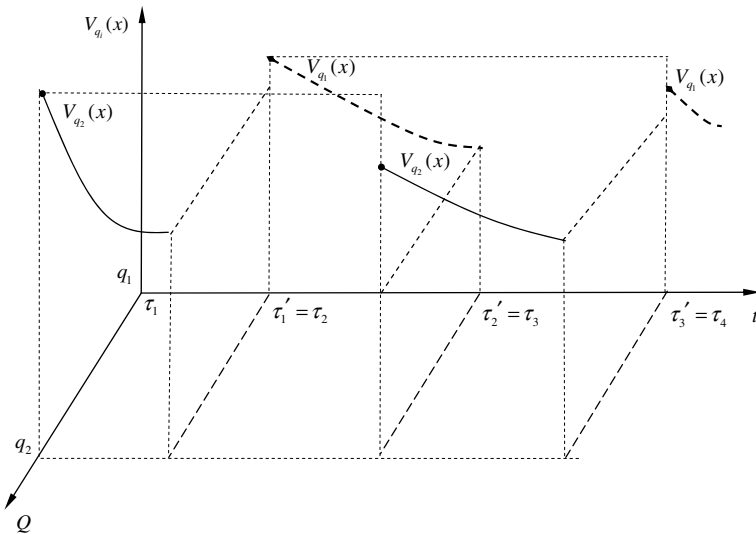
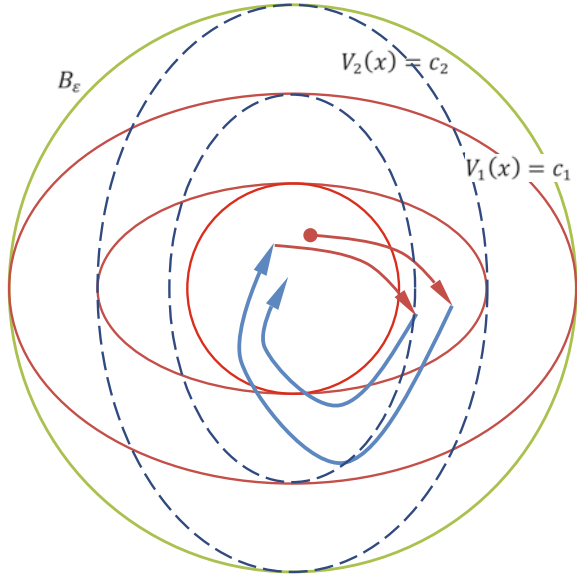


Fig. 4.6 Illustration of Theorem 4.20. For every subsystem, if its Lyapunov function's value V_i at the start point of each interval does not exceed the value at the entering point of the previous activating interval, then the switched system is asymptotically stable

Fig. 4.7 Illustration of the proof for Theorem 4.20



to the mode $q = 1$. Due to the assumption that the sequence $\{V_{q(\tau_i)}(x(\tau_i)) | q(\tau_i) = \hat{q}\}$ is non-increasing, we have $V_1(x(t_2)) \leq V_1(x(t_0))$, hence $x(t_2) \in \{x | V_1(x) \leq c'_1\}$ since $x(t_0) \in \{x | V_1(x) \leq c'_1\}$. We can continue this argument to show that the state trajectory will remain in B_ϵ for all $t \geq 0$, which implies the stability (in the sense of Lyapunov). $\square$

The above MLF theorem is adopted from the [5] and written in the language of hybrid automata. Some comments are in order. Note that if the value of the Lyapunov-like function V_q for every q at the entering point of each activating interval is less than the value at the entering point of the previous activating interval, then the hybrid system is stable. This is illustrated in Fig. 4.6. However, checking this condition could be difficult, since it may require the calculation of all possible trajectories and keeping track of all the time instants when each mode is being activated. Hence, it is difficult to construct the sequence $\{V_{q(\tau_i)}(x(\tau_i)) | q(\tau_i) = \hat{q}\}$. A simpler but more conservative condition is to request that $\{V_{q(\tau_i)}(x(\tau_i))\}$ forms a non-increasing sequence, as it implies that $\{V_{q(\tau_i)}(x(\tau_i)) | q(\tau_i) = \hat{q}\}$ is non-increasing. Therefore, a direct corollary of the above theorem is as follows:

Corollary 4.1 *Consider a switched system modeled as a hybrid automaton H with $x_e = 0$ as its equilibrium point, and $R((q_i, q_j), x) = x$ for any $q_i, q_j \in Q$. Assume that there exists an open set $D \subseteq X$, and a family of continuously differentiable functions $V_q : D \rightarrow \mathbb{R}$, $q \in Q$ such that for all $q \in Q$*

- (1) $V_q(0) = 0$;
- (2) $V_q(x) > 0$ for all $x \in D \setminus \{0\}$;
- (3) $\frac{\partial V_q(x)}{\partial x} f(q, x) \leq 0$ for all $x \in D$.

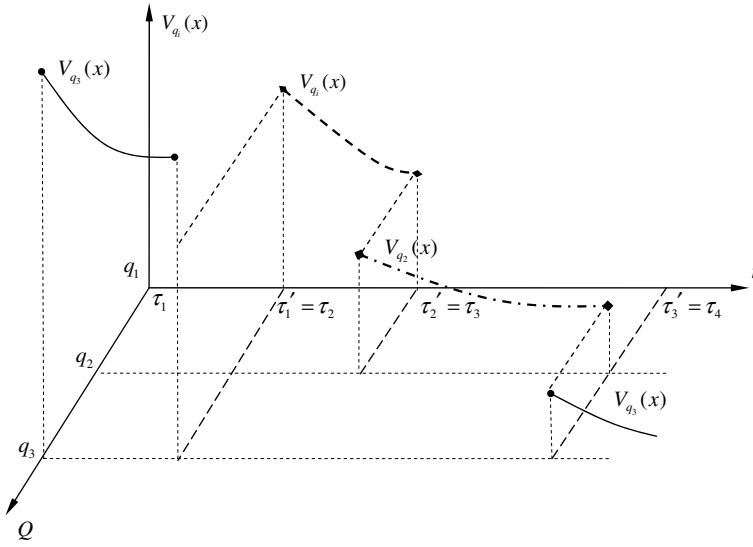


Fig. 4.8 Illustration of Corollary 4.1. The values V_i of the Lyapunov functions at the switching times τ_i form a non-increasing sequence

If for all $(\tau, q, x) \in \mathcal{E}_H(q_0, x_0)$ with $(q_0, x_0) \in \text{Init}$, the sequence $\{V_{q(\tau_i)}(x(\tau_i))\}$ is non-increasing, then $x_e = 0$ is a stable equilibrium of H .

Figure 4.8 illustrates the condition in Corollary 4.1, i.e., the values V_i of the Lyapunov functions at the switching times τ_i form a non-increasing sequence. For completeness, we also state the MLF theorem for the case when the model is given by switched systems.

Theorem 4.21 Consider a switched system consisting of a collection of subsystems $\dot{x} = f_i(x)$ with $x_e = 0$ as the common equilibrium point. Suppose that each subsystem has an associated Lyapunov-like function V_i in its active region Ω_i , each with the equilibrium point $x_e = 0$. Also, suppose that $\bigcup_i \Omega_i = \mathbb{R}^n$. Let $s(t)$ be a class of piecewise constant switching sequences such that $s(t)$ can take value i only if $x(t) \in \Omega_i$, and in addition

$$V_j(x(t_{i,j})) \leq V_i(x(t_{i,j})),$$

where $t_{i,j}$ denotes the time that the state switches from subsystem i to subsystem j , i.e., $x(t_{i,j}^-) \in \Omega_i$ while $x(t_{i,j}) \in \Omega_j$. Then, the switched system is stable under the switching signals $s(t)$.

This can be easily shown by applying Corollary 4.1 to the case of switched systems. Next, we strengthen the condition to obtain a stronger stability property for switched systems [6].

Theorem 4.22 Assume that there is a family of Lyapunov-like functions $\{V_i | i \in \mathcal{I}\}$ for each stable subsystem. If for any two switching instants t_i and t_j such that $i < j$ and $\sigma(t_i) = \sigma(t_j)$, we have

$$V_{\sigma(t_j)}(x(t_{j+1})) - V_{\sigma(t_i)}(x(t_{i+1})) \leq -\rho \|x(t_{i+1})\|^2,$$

for some constant $\rho > 0$, then the switched system is asymptotically stable.

Proof Due to finiteness of $\mathcal{I}$, there exists an index $i \in \mathcal{I}$ associated with an infinite sequence of switching times $t_{i_1}, t_{i_2}, \dots$, such that $\sigma(t) = i$. The sequence $V_{\sigma(t_{i_1})}(x(t_{i_1+1})), V_{\sigma(t_{i_2})}(x(t_{i_2+1})), \dots$ is decreasing and is positive, and therefore, has a limit $L > 0$. We have

$$\begin{aligned} 0 &= L - L = \lim_{k \rightarrow \infty} V_{\sigma(t_{i_{k+1}})}(x(t_{i_{k+1}+1})) - \lim_{k \rightarrow \infty} V_{\sigma(t_{i_k})}(x(t_{i_k+1})) \\ &= \lim_{k \rightarrow \infty} [V_{\sigma(t_{i_{k+1}})}(x(t_{i_{k+1}+1})) - V_{\sigma(t_{i_k})}(x(t_{i_k+1}))] \\ &\leq \lim_{k \rightarrow \infty} -\rho \|x(t_{i_k+1})\|^2 \leq 0, \end{aligned}$$

which implies that $x(t)$ converges to zero. $\square$

The above MLF theorem requires that the Lyapunov-like functions do not increase their values at each switching instant, which is a more conservative condition compared to Theorem 4.20. However, the conditions in Theorem 4.21 are easier to check as one does not have to memorize the previous activation history and store the values of Lyapunov-like functions at the entering or exiting point for each mode, which is needed to check the conditions in Theorem 4.20.

Next, let's use an example from [8] to illustrate the above MLF theorems.

Example 4.11 Consider a switched linear system

$$\dot{x} = A_i x,$$

with $i \in \mathcal{I} = \{1, 2\}$ and

$$A_1 = \begin{bmatrix} -0.1 & -1 \\ 2 & -0.1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} -0.1 & -2 \\ 1 & -0.1 \end{bmatrix},$$

and under the following switching law

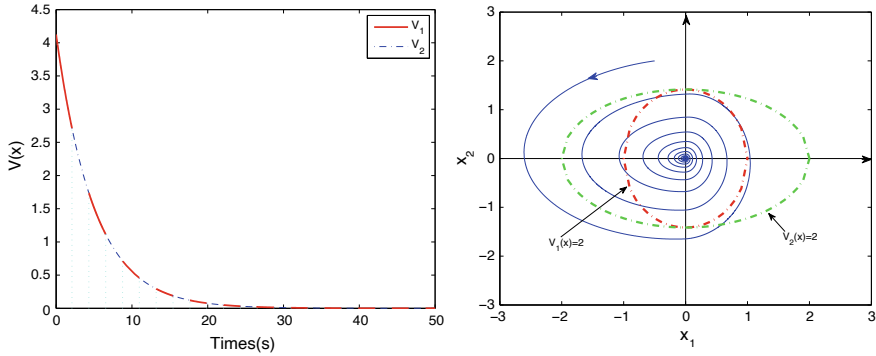


Fig. 4.9 Illustration of trajectory and Lyapunov functions in Example 4.11

$$\dot{x} = \begin{cases} A_1 x & \text{if } x_1 \geq 0, \\ A_2 x & \text{if } x_1 < 0. \end{cases}$$

We can choose Lyapunov-like functions as follows:

$$V_1(x) = 2x_1^2 + x_2^2, \quad V_2(x) = 0.5x_1^2 + x_2^2.$$

It can be verified that the Lyapunov-like function value decreases when its corresponding subsystem is active. Moreover, on the switching surface $\{x | x_1 = 0\}$ their values match, which means that the MLF V_σ is continuous along solutions of the switched system. Hence, at every switching time, denoted as τ_i for $i = 0, 1, 2, \dots$, the Lyapunov-like function values, $V_{\sigma(\tau_i)}(x(\tau_i))$, are decreasing as illustrated in Fig. 4.9. In view of the MLF theorem, the switched system is stable. $\square$

It should be pointed out that different versions of the MLF theorem, which could be less conservative than the version we presented above do exist. For example, the Lyapunov-like function for the q th subsystem may not be monotonically decreasing when the q th subsystem is activated. In other words, the hybrid system is stable even when there are unstable subsystems. More precisely, the Lyapunov-like function for the q th subsystem may not be monotonically decreasing when the q th subsystem is activated. The value may increase, actually, but the hybrid system remains stable provided that the increase of the Lyapunov-like function is bounded by a continuous function, see e.g., [23]; e.g., Fig. 4.10. The following example illustrates this result. It is still worth emphasizing that the MLF method is sufficient only condition for the stability of hybrid automata.

Example 4.12 Consider a switched linear system

$$\dot{x} = A_i x,$$

with $i \in \mathcal{I} = \{1, 2\}$ and

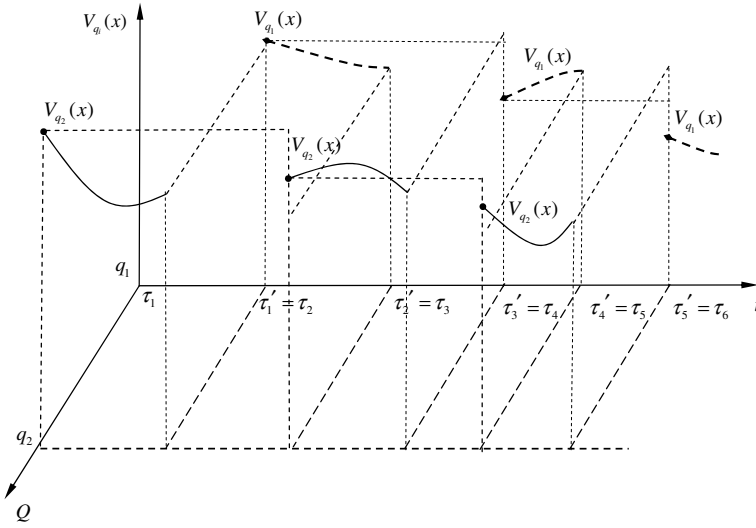


Fig. 4.10 The switched system may remain stable even when subsystems are unstable as long as the increase of the Lyapunov-like function is bounded by a continuous function

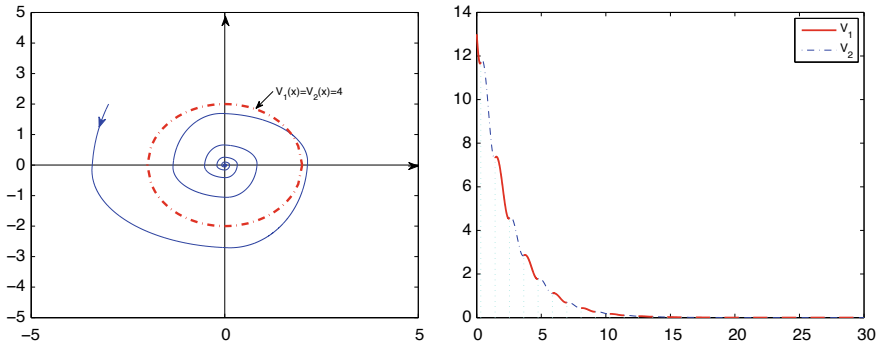


Fig. 4.11 Illustration of trajectory and Lyapunov functions in Example 4.12

$$A_1 = \begin{bmatrix} 0.1 & -1 \\ 2 & 0.1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 0.1 & -2 \\ 1 & 0.1 \end{bmatrix}.$$

Note that neither subsystem is stable, since the eigenvalues of both A_1 and A_2 are $0.1 \pm \sqrt{2}j$. However, if the switching law

$$\dot{x} = \begin{cases} A_1 x & \text{if } x_1 x_2 \leq 0 \\ A_2 x & \text{if } x_1 x_2 > 0, \end{cases}$$

is used, the state trajectories of the switched system converge to the origin. To understand why this is the case, we can choose $V_1(x) = x^T x$ and $V_2(x) = x^T x$

as Lyapunov-like functions. Each V_i actually increases its value within the active period of its corresponding subsystem. However, it can be verified that at every switching time, denoted as τ_{2i} for $i = 0, 1, 2, \dots$, when the system switches into the first subsystem, $\dot{x} = A_1 x$, the Lyapunov-like function value $V_1(x(\tau_{2i}))$ is decreasing. Similarly, the sequence $V_2(x(\tau_{2i+1}))$ is also decreasing, as illustrated in Fig. 4.11. $\square$

4.5.2 Piecewise Quadratic Lyapunov Functions

The critical challenge of applying the MLF theorems to practical hybrid systems is how to construct a proper family of Lyapunov-like functions that

- (1) satisfy positive definiteness, namely $V_i(x_e) = 0$ and $V_i(x) > 0$ for $x \in \Omega_i$ and $x \neq x_e$;
- (2) have negative definite derivative: for $x \in \Omega_i$, $\dot{V}_i(x) = \frac{\partial V_i(x)}{\partial x} f_i(x) \leq 0$.

Usually, it is hard to find such $V_i(x)$. In the special case of linear subsystems, piecewise quadratic functions could be good candidates. The most important advantage of piecewise quadratic Lyapunov-like functions might be that the problem can be formulated into LMIs where efficient software packages are available.

Therefore, in this subsection, we return to switched LTI system

$$\dot{x}(t) = A_i x(t), \quad (4.41)$$

where the origin ($x_e = 0$) is a common equilibrium. Since we do not assume that the subsystems, $\dot{x}(t) = A_i x(t)$, are stable, there may not exist a quadratic Lyapunov function in a classical sense for each subsystem. However, it is still possible to restrict the subsystem in a certain region of the state space, say $\Omega_i \subset \mathbb{R}^n$, where the abstracted energy of the i th subsystem could be decreasing along the trajectories inside this region (there is no requirement outside the region Ω_i). When all these regions Ω_i taken together cover the whole state space, then we obtain a cluster of Lyapunov-like functions. Broadly speaking, the problem entails searching for Lyapunov-like functions whose associated Ω -region cover the state space.

Assume that the state space $\mathbb{R}^n$ has a partition given by $\{\Omega_1, \dots, \Omega_N\}$. Here, we aim to find conditions expressed in LMIs for the existence of quadratic Lyapunov-like function of the form $V_i(x) = x^T P_i x$ assigned to each region Ω_i . By definition, a Lyapunov-like function $V_i(x) = x^T P_i x$ needs to satisfy the following two conditions.

Condition 1: There exist constant scalars $\beta_i \geq \alpha_i > 0$ such that

$$\alpha_i \|x\|^2 \leq V_i(x) \leq \beta_i \|x\|^2,$$

holds for any $x \in \Omega_i$.

Consider the quadratic Lyapunov-like function candidate $V_i(x) = x^T P_i x$, and require that

$$\alpha_i x^T I x \leq x^T P_i x \leq \beta_i x^T I x,$$

holds for any $x \in \Omega_i$. That is

$$\begin{cases} x^T (\alpha_i I - P_i) x \leq 0 \\ x^T (P_i - \beta_i I) x \leq 0 \end{cases},$$

holds for any $x \in \Omega_i$.

Condition 2: For all $x \in \Omega_i$ and $x \neq 0$, $\dot{V}_i(x) < 0$.

This negativity of the Lyapunov-like function's derivative along the trajectories of the subsystems can be represented as: $\exists P_i, (P_i = P_i^T)$ such that

$$x^T [A_i^T P_i + P_i A_i] x < 0, \quad (4.42)$$

for $x \in \Omega_i$.

Note that the above positive definiteness and energy decreasing conditions both need to be satisfied in a local region, here Ω_i . In order to constrain the above two conditions to local regions, two steps are needed. First, the region must be expressed or constrained in regions that can be characterized by quadratic forms, $x^T Q x \geq 0$. Examples of such quadratic forms are cones and ellipsoids. If a region is described by half-planes

$$c_a^T x \geq 0 \text{ and } c_b^T x \geq 0,$$

and

$$c_a^T x \leq 0 \text{ and } c_b^T x \leq 0,$$

then the quadratic form characterizing the region is obtained by multiplying the two half-planes together

$$x^T Q x \geq 0,$$

where $Q = c_a c_b^T + c_b c_a^T$. Refer to [7] where more general quadratic forms are used to express hyperplanes and polyhedra. Therefore, we assume that each region Ω_i has a quadratic representation or approximation given by

$$\Omega_i = \{x \mid x^T Q_i x \geq 0\}.$$

Second, a technique called $\mathcal{S}$ -procedure [7] may be applied to replace constrained stability conditions to conditions without constraints. To illustrate, let us consider the condition 2 (4.42), that is, $\exists P_i, (P_i = P_i^T)$ such that

$$x^T [A_i^T P_i + P_i A_i] x < 0,$$

for $x \in \Omega_i$, i.e., $x^T Q_i x \geq 0$. By introducing a new unknown variable $\vartheta_i \geq 0$, there is potentially more freedom in finding a $P_i > 0$ and $\vartheta_i \geq 0$ satisfying the unconstrained relaxed condition

$$A_i^T P_i + P_i A_i + \vartheta_i Q_i < 0. \quad (4.43)$$

A solution to this relaxed problem is also a solution to the constrained problem, and the two problems are actually equivalent in certain cases, for example, for every x such that $x^T Q_i x > 0$.

Also, applying $\mathcal{S}$ -procedure to Condition 1, the above constrained inequalities are implied by the following LMIs

$$\begin{cases} \alpha_i I - P_i + \mu_i Q_i \leq 0 \\ P_i - \beta_i I + \nu_i Q_i \leq 0 \end{cases},$$

where $\mu_i \geq 0$ and $\nu_i \geq 0$ are unknown scalars. Define two scalars, $\alpha = \min_i \{\alpha_i\}$ and $\beta = \max_i \{\beta_i\}$. Notice that $0 < \alpha \leq \beta$.

Combining the above two conditions, we introduce methods to find quadratic Lyapunov-like functions within each partition, which guarantees that the abstract energy of the subsystem is decreasing in certain regions of the state space.

Linear Matrix Inequalities

The condition that requires that the values of the Lyapunov-like functions are decreasing at switching instants can be expressed as

$$x^T P_j x \leq x^T P_i x,$$

for state x where the trajectory passes from region Ω_i to Ω_j . The states where this condition is satisfied also have to be expressed or contained in regions characterized by quadratic forms. Assume this region, denoted as $\Omega_{i,j}$, can be expressed or approximated by a region,

$$\Omega_{i,j} = \{x \mid x^T Q_{i,j} x \geq 0\}.$$

Then the above inequality can be transformed into an LMI based also on the $\mathcal{S}$ -procedure as

$$P_j + \eta_{i,j} Q_{i,j} \leq P_i,$$

In summary, the above discussion can be summarized by the following sufficient conditions for continuous-time linear switched systems to be exponentially stable [24].

Theorem 4.23 *If there exist matrices P_i ($P_i = P_i^T$) and scalars $\alpha > 0$, $\beta > 0$, $\mu_i \geq 0$, $\nu_i \geq 0$, $\theta_i \geq 0$, $\vartheta_i \geq 0$ and $\eta_{i,j}$, solving the optimization problem:*

$$\begin{aligned} & \min \beta \\ & s.t. \begin{cases} \alpha I + \mu_i Q_i \leq P_i \leq \beta I - \nu_i Q_i \\ A_i^T P_i + P_i A_i + \vartheta_i Q_i \leq -I \\ P_j + \eta_{i,j} Q_{i,j} \leq P_i \end{cases}, \end{aligned} \quad (4.44)$$

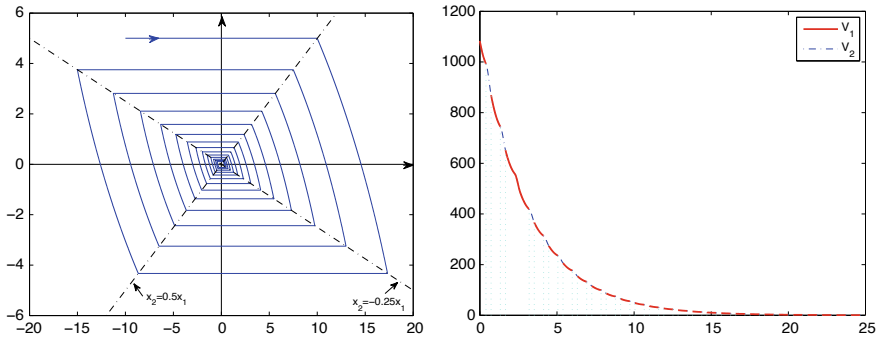


Fig. 4.12 Illustration of trajectory and Lyapunov functions in Example 4.13

then the switched system (4.41) is exponentially stable.

The left-hand side of the first condition requires positive definiteness of the quadratic Lyapunov-like functions. The right-hand side is introduced to find an upper bound of the Lyapunov-like functions to determine an upper bound of the convergence rate. If only stability is of interest, this right-hand side of the inequality can be neglected. The second condition is the requirement that the energy is decreasing in every region Ω_i . The energy decrease has to be less than the negative identity matrix to conclude exponential stability. Stability is guaranteed if the right-hand side is zero instead. Finally, the third condition is the requirement that the energy is non-increasing at switching instants, where (i, j) is the set of tuples characterizing neighboring regions for which $x(t)$ can possibly travel from Ω_i to Ω_j .

Exponential stability is verified if there is a solution to the above LMI problem. In addition, a bound on the convergence rate can be estimated [24] by

$$\|x(t)\| \leq \sqrt{\frac{\beta}{\alpha}} e^{-\frac{1}{2\beta}t} \|x_0\|,$$

where $x(t)$ is the continuous trajectory with initial state x_0 .

Example 4.13 Consider a switched linear system

$$\dot{x} = A_i x,$$

with $i \in \mathcal{I} = \{1, 2\}$ and

$$A_1 = \begin{bmatrix} 0 & 10 \\ 0 & 0 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 1.5 & 2 \\ -2 & -0.5 \end{bmatrix},$$

and under the following switching law

$$\sigma(t) = \begin{cases} 1, & \text{if } \alpha(t^-) = 3 \text{ and } x_2(t) = -0.25x_1(t), \\ 2, & \text{if } \alpha(t^-) = 1 \text{ and } x_2(t) = 0.5x_1(t). \end{cases}$$

Note that both subsystems are unstable as the eigenvalues of A_1 are zero and those of A_2 lie at $0.5 \pm j\sqrt{3}$. Simulations of the state trajectories show convergence to the origin. To verify stability, the state space is partitioned into two regions

$$\Omega_1(x) = \{x : x^T Q_1 x \geq 0\}, \quad \Omega_2(x) = \{x : x^T Q_2 x \geq 0\},$$

with

$$Q_1 = \begin{bmatrix} -0.25 & -0.25 \\ -0.25 & 2 \end{bmatrix}, \quad Q_2 = \begin{bmatrix} 0.25 & 0.25 \\ 0.25 & -2 \end{bmatrix},$$

which correspond to the active regions of the first subsystem and the second subsystem, respectively.

By solving the LMI problem above, we obtain

$$P_1 = \begin{bmatrix} 0.100 & -0.4500 \\ -0.4500 & 41.1167 \end{bmatrix}, \quad P_2 = \begin{bmatrix} 4.3792 & 3.8292 \\ 3.8292 & 6.8833 \end{bmatrix},$$

with a value of $\beta = 41.12$. Hence the switched system is (exponentially) stable as illustrated in Fig. 4.12. $\square$

4.6 Design of Stabilizing Switching Sequences

The above MLF results in Sect. 4.4 provide methodologies to design switching logics between vector fields to achieve stable trajectories. This is so because the MLF results provide conditions on switching signals, under which the hybrid/switched system is stable. In this section, we will explicitly consider the design of stable hybrid/switched systems. In particular, we consider the design of stabilizing switching signals among a collection of LTI systems $\dot{x} = A_i x$. First recall that when all subsystems are stable, introducing some dwell time will guarantee the stability of the switched system. Interestingly, even when all subsystems are unstable, there may still exist stabilizing switching signals as illustrated in our second motivating example, Example 4.7 in Sect. 4.2. In the following, the cases of quadratic switching stabilization and piecewise quadratic switching stabilization are considered.

4.6.1 Quadratic Stabilization

In the switching stabilization literature, earlier efforts focused on quadratic stabilization for certain classes of systems. The switched system (4.13) is *quadratically*

stabilizable, when there exist switching signals $\sigma(t)$, a quadratic function

$$V(x) = x^T P x, \quad (4.45)$$

with P a symmetric and positive definite matrix in $\mathbb{R}^{n \times n}$, and a positive constant $\epsilon > 0$ such that

$$\dot{V}(x) = \left[\frac{\partial}{\partial x} V(x) \right] A_{\sigma(t)} x < -\epsilon x^T x, \quad \forall x \neq 0. \quad (4.46)$$

When $\mathcal{I} = \{1, 2\}$, it can be shown that the existence of a stable convex combination of a pair of two stable LTI subsystem matrices implies the existence of a state-dependent switching rule that stabilizes the switched system along with a quadratic Lyapunov function [25]. Formally, this result can be stated as follows:

Theorem 4.24 *If the matrix pencil³ $\gamma_\alpha(A_1, A_2)$ contains a stable matrix, then there exists a piecewise constant switching signal that makes the switched system quadratically stable.*

Proof Since the matrix pencil $\gamma_\alpha(A_1, A_2)$ contains a stable matrix, there exists a constant $\alpha \in [0, 1]$, so that the convex combination

$$A_\alpha = \alpha A_1 + (1 - \alpha) A_2,$$

is stable. Thus, there exists symmetric positive definite matrix P and Q such that

$$A_\alpha^T P + P A_\alpha = -Q,$$

this can be rewritten as

$$\alpha(A_1^T P + A_1 P) + (1 - \alpha)(A_2^T P + P A_2) = -Q,$$

or

$$\alpha x^T (A_1^T P + A_1 P) x + (1 - \alpha) x^T (A_2^T P + P A_2) x = -x^T Q x < 0.$$

Define the regions

$$\Omega_1 = \{x | x^T (A_1^T P + A_1 P) x < 0, \quad x \in \mathbb{R}^n\},$$

and

$$\Omega_2 = \{x | x^T (A_2^T P + P A_2) x < 0, \quad x \in \mathbb{R}^n\}.$$

Since $0 < \alpha < 1$, it follows that for each non-zero $x \in \mathbb{R}^n$, at least one of $x^T (A_1^T P + A_1 P) x$ and $x^T (A_2^T P + P A_2) x$ is negative. Therefore,

³ Here, the matrix pencil $\gamma_\alpha(A_1, A_2)$ is defined as a collection of matrices A_α that can be written as a convex combination of A_1 and A_2 , i.e., $A_\alpha = \alpha A_1 + (1 - \alpha) A_2$ for some $\alpha \in [0, 1]$.

$$\Omega_1 \cup \Omega_2 = \mathbb{R}^n / \{0\}.$$

Using this property, it is possible to construct a switching signal, for example, activate the first subsystem when the state x is in the region Ω_1 and switch to the second subsystem when x is in Ω_2 , such that $V(x) = x^T P x$ satisfies the condition (4.46) along solutions of the switched system, which implies quadratic stability. $\square$

Furthermore, it can be shown that the stable convex combination condition is also necessary for the quadratic stabilizability of two mode switched LTI system. To prove the necessity, we introduce the following lemma on the $\mathcal{S}$ -procedure.

Lemma 4.1 *Let $T_0, T_1 \in \mathbb{R}^{n \times n}$ be symmetric matrices. Consider the following condition on T_0 and T_1*

$$\zeta^T T_0 \zeta > 0 \text{ for any } \zeta \neq 0 \text{ such that } \zeta^T T_1 \zeta \geq 0, \quad (4.47)$$

and

$$\exists \tau_1 \geq 0 \text{ such that } T_0 - \tau_1 T_1 > 0. \quad (4.48)$$

Condition (4.48) always implies (4.47). Furthermore, if there is some ζ_0 such that $\zeta_0^T T_1 \zeta_0 > 0$, then (4.47) always implies (4.48).

Theorem 4.25 *If the switched system (4.13) with $\mathcal{I} = \{1, 2\}$ is quadratically stabilizable, then the matrix pencil $\gamma_\alpha(A_1, A_2)$ contains a stable matrix.*

Proof Assume that the system is quadratically stabilizable with the quadratic Lyapunov function $V(x) = x^T P x$, $P > 0$. Then

$$\frac{d}{dt} V(x) = x^T (A_{\sigma(t)}^T P + P A_{\sigma(t)}) x < -\epsilon x^T x,$$

for some positive scalar $\epsilon > 0$. Therefore, for any $x \neq 0$,

$$x^T (A_1^T P + P A_1 + \epsilon I) x < 0, \text{ whenever } x^T (A_2^T P + P A_2 + \epsilon I) x \geq 0, \quad (4.49)$$

or

$$x^T (A_2^T P + P A_2 + \epsilon I) x < 0, \text{ whenever } x^T (A_1^T P + P A_1 + \epsilon I) x \geq 0. \quad (4.50)$$

Apply the previous lemma with $T_0 = -(A_1^T P + P A_1 + \epsilon I)$ and $T_1 = (A_2^T P + P A_2 + \epsilon I)$, and assume that there exists x such that $x^T (A_1^T P + P A_1 + \epsilon I) x > 0$. Then, condition (4.49) is equivalent to the existence of a scalar $\delta_1 \geq 0$ such that

$$(A_1^T P + P A_1) + \delta_1 (A_2^T P + P A_2) < -(1 + \delta_1) \epsilon I, \quad (4.51)$$

Similarly, under the assumption that there exists x such that $x^T (A_2^T P + P A_2 + \epsilon I) x > 0$, then (4.50) will hold if and only if $\exists \delta_2 \geq 0$ such that

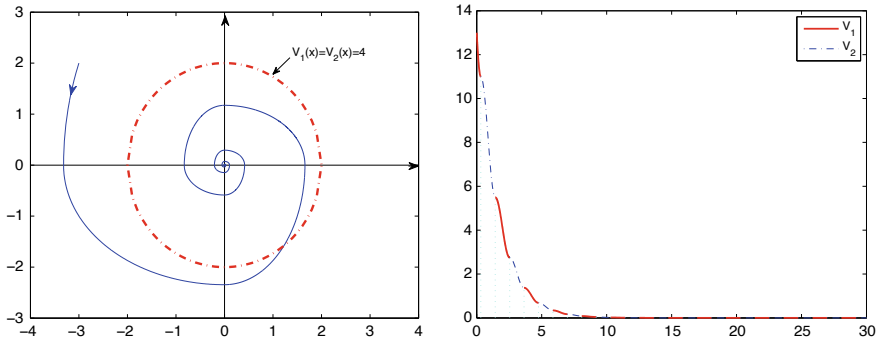


Fig. 4.13 Illustration of trajectory and Lyapunov functions in Example 4.14

$$\delta_2(A_1^T P + P A_1) + (A_2^T P + P A_2) < -(1 + \delta_2)\epsilon I. \quad (4.52)$$

Note that (4.52) is implied by (4.51). Indeed, by continuity, if (4.51) holds for some $\delta_1 \geq 0$ then it must hold for some $\delta_1 > 0$. Then (4.52) is equivalent to (4.51) for $\delta_2 = 1/\delta_1$. Condition (4.51) can also be written as

$$\frac{(A_1 + \delta_1 A_2)^T}{1 + \delta_1} P + P \frac{(A_1 + \delta_1 A_2)}{1 + \delta_1} < -\epsilon I,$$

which implies that the matrix $\frac{(A_1 + \delta_1 A_2)}{1 + \delta_1}$ is asymptotically stable. So there exists a stable convex combination between A_1 and A_2 with $\alpha = \frac{1}{1 + \delta_1}$.

For the case when for all $x \neq 0$, either $x^T (A_1^T P + P A_1 + \epsilon I)x \leq 0$ or $x^T (A_2^T P + P A_2 + \epsilon I)x \leq 0$, then, either A_1 or A_2 is asymptotically stable, which also implies that there exists a stable convex combination between A_1 and A_2 . $\square$

Therefore, the existence of a stable convex combination state matrix is necessary and sufficient for the quadratic stabilizability of a two mode switched LTI system.

Theorem 4.26 *The switched system (4.13) with $\mathcal{I} = \{1, 2\}$ is quadratically stabilizable if and only if the matrix pencil $\gamma_\alpha(A_1, A_2)$ contains a stable matrix.*

To gain more insight, we use an example from [8] to illustrate the condition.

Example 4.14 Consider a switched linear system

$$\dot{x} = A_i x,$$

with $i \in \mathcal{I} = \{1, 2\}$ and

$$A_1 = \begin{bmatrix} 0 & -1 \\ 2 & 0 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 0 & -2 \\ 1 & 0 \end{bmatrix}.$$

Note that both subsystems have their eigenvalues at the origin, and hence are unstable (or called marginally stable). In addition, there is no stable convex combination between A_1 and A_2 . Hence, by the above theorem, the system is not quadratically stabilizable.

But the system can be stabilized by the following switching law

$$\dot{x} = \begin{cases} A_1 x, & \text{if } x_1 x_2 \leq 0, \\ A_2 x, & \text{if } x_1 x_2 > 0, \end{cases}$$

since it can be verified that

$$V_1(x) = V_2(x) = x^T x,$$

decreases their values along the trajectories of the switched system and $V_\sigma(x)$ is continuous. By the MLF theorem of the previous section, we can conclude stability of the switched system under this switching signal. Actually, it can be shown that the switched system is globally asymptotically stable as illustrated in Fig. 4.13.

To understand why the switched system is not quadratically stable, we observe that the derivative of V is zero for $x_1 x_2 = 0$, which contains non-zero x . Hence, we cannot find $\epsilon > 0$ to satisfy the (4.46) for all $x \neq 0$. $\square$

A generalization to more than two LTI subsystems can be achieved by using a “minimum projection strategy”, i.e., $i = \arg \min_{i \in \mathcal{I}} x^T P A_i x$.

Theorem 4.27 *If there exist constants $\alpha_i \in [0, 1]$, and $\sum_{i \in \mathcal{I}} \alpha_i = 1$ such that $A_\alpha = \sum_{i \in \mathcal{I}} \alpha_i A_i$, is stable then the min-projection strategy quadratically stabilizes the switched system.*

However, the existence of a stable convex combination matrix A_α is only sufficient for switched LTI systems with more than two modes. There are examples of systems for which no stable convex combination state matrix exists, yet the system is quadratically stabilizable using certain switching signals [26].

A necessary and sufficient condition for the quadratic stabilizability of switched controller systems was derived in [27].

Theorem 4.28 *The switched system in (4.41) is quadratically stabilizable if and only if there exists a positive definite real symmetric matrix $P = P^T > 0$ such that the set of matrices $\{A_i P + P A_i^T\}$ is strictly complete, i.e., for any $x \in \mathbb{R}^n / \{0\}$, there exists $i \in \mathcal{I}$ such that $x^T (A_i P + P A_i^T) x < 0$. In addition, a stabilizing switching signal can be selected as $\sigma(t) = \arg \min_i \{x^T(t) (A_i P + P A_i^T) x(t)\}$. $\square$*

Similarly, for the discrete-time case, it is necessary and sufficient for quadratic stabilizability to check whether there exists a positive symmetric matrix P such that the set of matrices $\{A_i^T P A_i - P\}$ is strictly complete [27]. Obviously, the existence of a convex combination of state matrices A_α automatically satisfies the above strict completeness conditions due to convexity, while the inverse statement is not true in general. Unfortunately, checking the strict completeness of a set of matrices is NP hard [27].

Other approaches include [25] and extensions of [25] to the output-dependent switching and discrete-time cases [14, 28]. For robust stabilization of polytopic uncertain switched linear systems, a quadratic stabilizing switching law was designed based on LMI techniques in [28].

Quadratic stability means that there exists a positive constant ϵ such that $\dot{V}(x) < -\epsilon x^T x$ for all $x \neq 0$. All of these methods guarantee stability by using a common quadratic Lyapunov function, which is conservative in the sense that there are switched systems that can be asymptotically or even exponentially stabilized without using a common quadratic Lyapunov function. There have been some results in the literature that proposes constructive synthesis methods for switched systems using multiple Lyapunov functions [6]. A stabilizing switching law design based on multiple Lyapunov functions was proposed in [29], where piecewise quadratic Lyapunov functions were employed for the two mode switched LTI systems. Exponential stabilization for switched LTI systems was considered in [30], also based on piecewise quadratic Lyapunov functions, and the synthesis problem was formulated as a bilinear matrix inequality (BMI) problem. In the next subsection, we will briefly describe the BMI conditions derived in [30].

4.6.2 Piecewise Quadratic Stabilization

Recall the sufficient conditions stated in Theorem 4.23 for the continuous-time linear system (4.41) to be exponentially stable: If there exist matrices P_i ($P_i = P_i^T$) and scalars $\alpha > 0$, $\beta > 0$, $\mu_i \geq 0$, $\nu_i \geq 0$, $\theta_i \geq 0$, $\vartheta_i \geq 0$ and $\eta_{i,j}$, that satisfy

$$\begin{cases} \alpha I + \mu_i Q_i \leq P_i \leq \beta I - \nu_i Q_i \\ A_i^T P_i + P_i A_i + \vartheta_i Q_i \leq -I \\ P_j + \eta_{i,j} Q_{i,j} \leq P_i \end{cases},$$

then the switched linear system (4.41) is exponentially stable.

Compared to the stability analysis problem based on this result, there are two more issues that need to be solved for switching controller synthesis: One is the partition of the state space $\mathbb{R}^n$ and the other is the identification of the set that switching occurs, i.e., $\Omega_{i,j}$.

State Space Partition

The partitions, denoted by Ω_i , is to facilitate the identification of a Lyapunov-like function for each one of these subsystems. After successfully obtaining these Lyapunov-like functions associated with each region Ω_i , one may patch them together using the conditions in Theorem 4.23 so as to guarantee global stability.

For this, it is necessary to require that all these regions Ω_i cover the whole state space, i.e., $\Omega_1 \cup \dots \cup \Omega_N = \mathbb{R}^n$. This condition merely says that there are no regions in the state space where none of the subsystems is activated.

Since we will restrict our attention to quadratic Lyapunov-like functions for reason of computational efficiency, we will consider regions given (or approximated) by quadratic forms

$$\Omega_i = \{x \in \mathbb{R}^n \mid x^T Q_i x \geq 0\},$$

where $Q_i = \mathbb{R}^{n \times n}$ are symmetric matrices, and $i \in \mathcal{I} = \{1, \dots, N\}$.

The following lemma gives a sufficient condition for the covering property.

Lemma 4.2 *If for every $x \in \mathbb{R}^n$*

$$\sum_{i=1}^N \theta_i x^T Q_i x \geq 0, \quad (4.53)$$

where $\theta_i \geq 0$, $i \in \mathcal{I}$, then $\bigcup_{i=1}^N \Omega_i = \mathbb{R}^n$.

Consider the *largest region function strategy*, i.e.,

$$i(x) = \arg \left(\max_{i \in \mathcal{I}} x^T Q_i x \right). \quad (4.54)$$

That is the selection of subsystems (at state x) corresponds to the largest value of the region function $x^T Q_i x$. This switching strategy was previously introduced in [30] for continuous-time switched linear systems.

Switching Condition

In order to guarantee exponential stability, we also need to make sure that

1. subsystem i is active only when $x(t) \in \Omega_i$;
2. when switching occurs, it is required to guarantee that the Lyapunov function value is not increasing.

To verify the first condition for the largest region function strategy (4.54), suppose that the covering condition (4.53) holds, i.e.,

$$\sum_{i=1}^N \theta_i x^T Q_i x \geq 0,$$

for some $\theta_i \geq 0$, $i \in \mathcal{I}$. Then, based on the largest region function strategy, $i(x) = \arg(\max_{i \in \mathcal{I}} x^T Q_i x)$, the state x with current active mode i satisfies $x^T Q_i x \geq 0$. This implies $x \in \Omega_i$. Thus, the first condition holds for the largest region function strategy.

The second energy decreasing condition upon switching is not easy to handle. It is because we lack information about the direction of the vector fields on the switching hyperplane. In other words, when $x \in \Omega_i \cap \Omega_j$, i.e., on the switching plane, it is difficult to determine which is the direction of the trajectory where the switching is occurring.

We can make a compromise and require that

$$x^T P_i x = x^T P_j x,$$

for states at the switching plane, i.e., $x \in \Omega_i \cap \Omega_j$.

The set $\Omega_i \cap \Omega_j$ can be represented as the following quadratic form

$$\Omega_i \cap \Omega_j = \{x | x^T (Q_i - Q_j)x = 0\}.$$

Again, applying $\mathcal{S}$ -procedure, we obtain

$$P_i - P_j + \eta_{i,j}(Q_i - Q_j) = 0,$$

for an unknown scalar $\eta_{i,j}$.

Synthesis Condition

In summary, the above discussion can be described by the following sufficient conditions for the continuous-time linear system (4.41) to be exponentially stabilized.

Theorem 4.29 *If there exist matrices P_i ($P_i = P_i^T$) and scalars $\alpha > 0$, $\beta > 0$, $\mu_i \geq 0$, $\nu_i \geq 0$, $\theta_i \geq 0$, $\vartheta_i \geq 0$ and $\eta_{i,j}$, solving the optimization problem:*

$$\begin{aligned} & \min \beta \\ & s.t. \begin{cases} \alpha I + \mu_i Q_i \leq P_i \leq \beta I - \nu_i Q_i \\ A_i^T P_i + P_i A_i + \vartheta_i Q_i \leq -I \\ P_j = P_i + \eta_{i,j}(Q_i - Q_j) \\ \theta_1 Q_1 + \cdots + \theta_N Q_N \geq 0 \end{cases}, \end{aligned}$$

for all $i, j \in \mathcal{I} = \{1, \dots, N\}$, then the linear system (4.41) can be exponentially stabilized by the largest region function strategy defined by Eq. (4.54).

Some remarks are in order. First, the optimization problem above is a Bilinear Matrix Inequality (BMI) problem, due to the product of unknown scalars and matrices. BMI problems are NP hard, and not computationally efficient. However, practical algorithms for optimization problems over BMIs exist and typically involve approximations, heuristics, branch-and-bound methods, or local search. One possible way

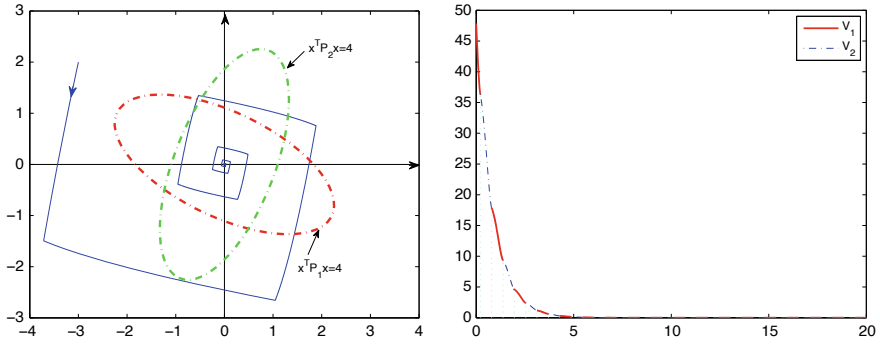


Fig. 4.14 Illustration of trajectory and Lyapunov functions in Example 4.15

to compute the BMI problem is to grid up the unknown scalars, and then solve a set of LMIs for fixed values of these parameters. It is argued that the gridding of the unknown scalars can be made quite sparsely [30].

We use the following example from [30] to illustrate the design process.

Example 4.15 Consider a switched linear system

$$\dot{x} = A_i x,$$

with $i \in \mathcal{I} = \{1, 2\}$ and

$$A_1 = \begin{bmatrix} 1 & -5 \\ 0 & 1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 1 & 0 \\ 5 & 1 \end{bmatrix},$$

Note that the eigenvalues of their convex combination $\alpha A_1 + (1 - \alpha)A_2$ are given as $1 \pm j5\sqrt{\alpha(1 - \alpha)}$ for $0 \leq \alpha \leq 1$. Hence, neither subsystem is stable and they are not quadratically stabilizable.

But the system can be stabilized using the methods developed in this section. In particular, we can form and solve corresponding BMI problem (using some common techniques like grid the unknown scalars) to obtain

$$\beta = 3.7941, \quad \alpha = 0.2101$$

and

$$\begin{aligned} Q_1 = -Q_2 = Q &= \begin{bmatrix} -0.8242 & 0.8648 \\ 0.8648 & 0.8053 \end{bmatrix}, \\ P_1 = P + 2\eta Q &= \begin{bmatrix} 1.1896 & 1.1440 \\ 1.1440 & 3.2447 \end{bmatrix}, \\ P_2 = P &= \begin{bmatrix} 3.3325 & 1.1044 \\ 1.1044 & 1.1509 \end{bmatrix}. \end{aligned}$$

Following the largest region function strategy defined by Equation (4.54), the switched linear system (4.41) is exponentially stable as illustrated in Fig. 4.14. $\square$

4.7 Summary

In this chapter, we discussed fundamental issues in the stability and stabilization of hybrid systems, with a particular focus on hybrid systems that can be modeled as switched systems.

- Stability is a basic concept of continuous dynamical systems, and Lyapunov stability theory provides us with a powerful tool. We started this chapter with a brief review of the Lyapunov stability theory fundamentals.
- Two motivating examples were given to illustrate the fact that the stability of hybrid systems cannot be decided solely based on the continuous dynamics of each mode. It was demonstrated that switching between two stable linear systems may generate divergent trajectories, while stable closed-loop behaviors can be achieved by switching between two unstable systems. It is, therefore, necessary to generalize the Lyapunov stability theory to address switched and hybrid systems' stability related issues.
- If we have no *prior* knowledge on the switching signals, we can ask whether the switched system remains stable under arbitrary switching. It turns out that the existence of a common Lyapunov function (not necessarily quadratic) is not only sufficient but also necessary for the stability of switched systems under arbitrary switching.
- Finding such a common Lyapunov function is not easy, and many efforts have focused on finding or identifying conditions for the existence of common Lyapunov functions of certain forms, such as quadratic or piecewise quadratic. For several hybrid systems including commutative systems, triangular systems, systems with Lie algebraic solvability conditions, a common Lyapunov function exists and can be determined. For more general switched systems, common Lyapunov functions in quadratic or polyhedral forms are typically considered. For discrete-time switched linear systems, the existence of switched quadratic Lyapunov functions including common quadratic Lyapunov functions as a special case could guarantee asymptotic stability under arbitrary switching. Additionally, the asymptotic stability problem for switched linear systems with arbitrary switching is equivalent to the robust asymptotic stability problem for polytopic uncertain linear time-variant systems.
- The conditions that guarantee stability under arbitrary switching are usually conservative. This motivates the study of the stability of switched systems under restricted switching signals. In this pursuit, Multiple Lyapunov Functions (MLF) provide us with an effective tool.
- If we restrict the switching frequency and introduce delays in switching, it is easier to achieve stability. Important concepts such as dwell time and its extension,

average dwell time, give us an elegant way to characterize switching signals. When all individual subsystems are asymptotically stable, a switched system is stable if the switching is slow enough (i.e., each mode is active for at least some dwell time). Actually, it really does not matter if one mode occasionally has a smaller dwell time between switchings, provided this does not occur too frequently. That is, if all the subsystems are exponentially stable then the switched system remains exponentially stable provided that the average dwell time is sufficiently large.

- Several versions of Multiple Lyapunov Functions (MLF) theorems were presented. The basic idea behind them is to associate each individual continuous dynamical system with an energy-like function, and then characterize the conditions on the class of switching signals that would not cause unmanageable energy increases due to switching.
- MLF are extensions of traditional Lyapunov functions, and they share the same difficulties in solving real-world problems, as it is usually hard to find appropriate Lyapunov functions for a given dynamical system. Numerical approaches were considered, focusing on piecewise affine systems and piecewise quadratic Lyapunov functions, for which solvable conditions can be obtained using linear matrix inequalities.
- In addition to stability analysis problems, we also considered stabilization issues in switched systems, namely the design of switching signals to stabilize a switched system consisting of a collection of unstable subsystems. The MLF approach provided us with a powerful tool for designing switching logics between vector fields so to achieve a stable trajectory.

4.8 Notes and Further Reading

In this chapter, we presented a brief introduction to stability and stabilization of switched systems. For further information on the stability and stabilization of switched and hybrid systems, interested readers may refer to survey papers [6, 14–16, 31, 32], to monographs [8, 33, 34] and the references cited therein.

The multiple Lyapunov function (MLF) theorem (Theorem 4.20) is based on the results in [5], where a proof for the theorem can be found. It is worth pointing out that there exist different versions of the MLF theorem, which could be less conservative than the version we presented above. For example, the Lyapunov-like function for the q th subsystem may be not monotonically decreasing when the q th subsystem is activated. The value may actually increase, but the hybrid system remains stable provided that the increase of the Lyapunov-like function is bounded by a continuous function, see e.g., [23]. Surveys on the switched systems stability and stabilization can be found in [6, 8, 14–16, 35].

The descriptions of switched quadratic Lyapunov functions and LMI conditions are based on [13], and the synthesis approach using a piecewise quadratic Lyapunov function is based on the work reported in [30]. Notice that the stability analysis using piecewise quadratic Lyapunov functions is sufficient only and could be conservative.

To reduce the possible conservativeness, new kinds of polynomial Lyapunov functions were introduced for the stability analysis of switched systems. The computation of such polynomial Lyapunov functions can be efficiently performed using convex optimization, based on the sum of squares (SOS) decomposition of multivariate polynomials [36]. It is also possible to use SOS techniques together with the $\mathcal{S}$ -procedure to construct piecewise polynomial Lyapunov functions [37], with each polynomial as a SOS while incorporating the state constraints, so to generalize piecewise quadratic Lyapunov functions.

We only reviewed sufficient conditions for the existence of stabilizing switching signals for a given collection of linear systems. A more elusive problem has been the necessary part of the switching stabilizability problem, and a particularly challenging part has been the problem of necessary and sufficient conditions for switching stabilizability. A necessary and sufficient condition for stabilizing second-order switched linear systems was derived in [38]. In [39], a necessary and sufficient condition was proposed for the existence of a switching control law (in static state feedback form) for asymptotic stabilization of continuous-time switched linear systems.

Closely related to stability, passivity and dissipativity are energy-like concepts that have been shown to be useful in the study of systems with continuous dynamics, both continuous and discrete-time. These concepts have been generalized and used in the analysis and design of hybrid systems; see for example [1, 40–43] and references therein.

There are several additional system concepts and properties related to stability, which were not addressed here due to space limitations. They include reachability, controllability and observability for switched and piecewise affine systems and they have been extensively studied in the literature, see e.g., [44–46] and the references therein. They are extensions of classical system concepts to the more general class of hybrid systems. The interested readers are encouraged to explore the literature on these topics [19, 28, 38, 39, 47–53].

4.9 Exercises

1. In this chapter, a number of examples were used to illustrate different aspects of the theory. In particular,
 - (a) Pendulum (Example 4.1);
 - (b) Destabilizing Switching (Example 4.6);
 - (c) Stabilizing Switching (Example 4.7);
 - (d) Arbitrary Switching (Example 4.8);
 - (e) Common Quadratic Lyapunov Function (Example 4.9);
 - (f) Multiple Lyapunov Function (Examples 4.10 and 4.11);
 - (g) Piecewise Quadratic Lyapunov Function (Examples 4.12);
 - (h) Stable, Unstable Subsystems (Example 4.13);
 - (i) Multiple Lyapunov Function Stabilization (Example 4.14);

(j) Piecewise Quadratic Lyapunov Function Stabilization (Example 4.15).

It is strongly recommended to simulate the examples in this chapter so to gain insight of the system's behavior and verify the claimed results. Commercial software, such as MATLAB and Stateflow, provides an excellent starting point for simulation work.

2. Consider the following system

$$\dot{x} = \begin{bmatrix} -2 & 1 \\ 1 & -2 \end{bmatrix} x.$$

Show, using a Lyapunov function, that the system is asymptotically stable.

3. Consider the following system

$$\dot{x} = -g(x(t)),$$

with $g : \mathbb{R} \rightarrow \mathbb{R}$ being odd in the sense that its graph lies in the first and third quadrants (i.e., $xg(x) > 0$ for all $x \neq 0$). For such systems, we can construct a Lyapunov function as

$$V(x) = \int_0^x g(s) ds.$$

Show that the constructed $V(x)$ is positive definite and its directional derivative $\dot{V} = \frac{\partial V}{\partial x} \dot{x}$ is negative definite.

4. Prove Theorem 4.6.

5. Show that comparison functions have the following properties:

- (a) if $\alpha \in \mathcal{K}$ over $[0, a)$, then $\alpha^{-1} \in \mathcal{K}$ over $[0, \alpha(a))$;
- (b) if $\alpha \in \mathcal{K}_\infty$, then $\alpha^{-1} \in \mathcal{K}_\infty$;
- (c) if $\alpha_1, \alpha_2 \in \mathcal{K}$, then $\alpha_1 \circ \alpha_2 \in \mathcal{K}$;
- (d) if $\alpha_1, \alpha_2 \in \mathcal{K}$ and $\beta \in \mathcal{KL}$, then $\alpha_1(\beta(\alpha_2(r), s)) \in \mathcal{KL}$.

6. Consider the switched system given by $x(k+1) = A_{\sigma(k)}x(k)$, where the switching signal $\sigma(k) \in \{1, 2, \dots, N\}$. It is assumed that A_i commute, i.e., $A_i A_j = A_j A_i$ for any pair of $i, j \in \{1, 2, \dots, N\}$. Show that such a switched system is stable under arbitrary switching as long as all A_i are stable;7. Consider an arbitrary switching system $\dot{x} = A_i x$ with

$$A_1 = \begin{bmatrix} -2 & 1 \\ 1 & -2 \end{bmatrix}, \quad A_2 = \begin{bmatrix} -1 & 1 \\ 1 & -2 \end{bmatrix}.$$

Show that $V(x) = x^T x$ is a CQLF.

8. Consider an arbitrary switching system $\dot{x} = A_i x$ with

$$A_1 = \begin{bmatrix} -0.1 & -1 \\ 2 & -0.1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} -0.1 & -2 \\ 1 & -0.1 \end{bmatrix}.$$

Show that all convex combinations of A_1 and A_2 are stable, but there does not exist a CQLF.

9. Consider the switched system given by $\dot{x} = A_i x$, for $i = 1, 2$ with $A_1 = \begin{bmatrix} -2 & 1 \\ -1 & -2 \end{bmatrix}$ and $A_2 = \begin{bmatrix} -3 & -1 \\ 0 & -2 \end{bmatrix}$.

- (a) Find positive definite matrices P_1 and P_2 such that

$$A_i^T P_i + P_i A_i = -I,$$

where $I \in \mathbb{R}^{2 \times 2}$ denotes the identity matrix;

- (b) Use the obtained P_1 and P_2 to compute conditions on the average dwell time τ_d such that the switched system is globally asymptotically stable;

Hint: You may find the following relationships helpful:

$$\lambda_{\min}(P) \|x\|^2 \leq x^T P x \leq \lambda_{\max}(P) \|x\|^2,$$

which holds for positive definite P . Note that $\lambda_{\min}(P)$ and $\lambda_{\max}(P)$ denote the smallest and largest eigenvalue of P respectively, and $\|x\|^2 = x^T x$.

- (c) Assume that we are switching between A_1 and A_2 periodically in the sense that we stay at each subsystem for a period of exact τ_d before switching into the other subsystem. Is it possible to generate unstable trajectories? Can you derive a lower bound (could be a numerical solution) on τ_d above which the stability of the periodically switching system is guaranteed?

10. Consider the following piecewise affine system

$$\dot{x} = \begin{cases} A_1 x, & \text{if } x_1 \leq 0, \\ A_2 x, & \text{if } x_1 > 0, \end{cases}$$

$$\text{where } A_1 = \begin{bmatrix} -5 & -4 \\ -1 & -2 \end{bmatrix} \text{ and } A_2 = \begin{bmatrix} -2 & -4 \\ 20 & -2 \end{bmatrix}.$$

- (a) Show that there does not exist a common quadratic Lyapunov function between the two subsystems;
 (b) Prove that the system is asymptotically stable using multiple Lyapunov approach.

Hint: You may think of using a piecewise quadratic Lyapunov function with

$$P_1 = \begin{pmatrix} 1 & 0 \\ 0 & 3 \end{pmatrix} \text{ and } P_2 = \begin{pmatrix} 10 & 0 \\ 0 & 3 \end{pmatrix}.$$

11. Extend Theorem 4.17 to the case of switched nonlinear systems, namely $\dot{x} = f_i(x)$ for $i \in \mathcal{I}$.
 12. Consider a switched linear system (4.13) with $\mathcal{I} = \{1, 2\}$ and

$$A_1 = \begin{bmatrix} -0.1 & -1 \\ 2 & -0.1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} -0.1 & -2 \\ 1 & -0.1 \end{bmatrix},$$

and under the following switching law

$$\dot{x} = \begin{cases} A_1 x, & \text{if } x_1 \geq 0, \\ A_2 x, & \text{if } x_1 < 0. \end{cases}$$

- (a) Verify the stability of the switched system.
 - (b) Do MATLAB simulation for the trajectories with different initial conditions.
13. The problem of finding a CQLF can also be translated into a sum of squares (SOS) program which can be solved efficiently using existing tools. Namely a multivariate polynomial $p(x)$, $x \in \mathbb{R}^n$ is an SOS, if there exist polynomials $f_1(x), \dots, f_m(x)$ such that

$$p(x) = \sum_{i=1}^m f_i^2(x).$$

Naturally, we have $p(x) \geq 0$ if it is an SOS for all $x \in \mathbb{R}^n$. Recall that a CQLF has the form $V(x) = x^T P x$, where P is positive definite. Then the following inequalities hold:

$$\begin{aligned} x^T P x &\geq 0, \\ P A_i + A_i^T P &< 0, \quad \forall i \in \mathcal{I}. \end{aligned} \tag{4.55}$$

Then we have the following two SOS conditions that guarantee (4.55) to be true.

$$\begin{aligned} x^T P x - \epsilon x^T x &\text{ is SOS,} \\ -P A_i + A_i^T P - \delta_i x^T x &\text{ is SOS } \forall i \in \mathcal{I}, \end{aligned} \tag{4.56}$$

where $\epsilon, \delta_i, i \in \mathcal{I}$ are small positive constants. Solving (4.56) as an SOS program for P and then we can find the CQLF. Consider a switching system (4.13) with $\mathcal{I} = \{1, 2\}$ and its state matrices given as

$$A_1 = \begin{bmatrix} -1 & 0.25 \\ -1 & -1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} -1 & -1 \\ 0.25 & -1 \end{bmatrix}.$$

Use software, e.g., SOSTOOLS (<http://www.cds.caltech.edu/sostools>), to solve (4.56) for this system to get a CQLF and prove the stability of this system under arbitrary switching.

14. Consider the switched system and Lyapunov-like functions in Example 4.11, verify that Lyapunov-like function value decreases when its corresponding subsystem is active.

15. Design a stabilizing switching signal that switches between the following two systems

$$\dot{x} = A_i x,$$

with $A_1 = \begin{bmatrix} 0.1 & 0.3 \\ 0.6 & -0.2 \end{bmatrix}$ and $A_2 = \begin{bmatrix} -0.13 & -0.16 \\ -0.33 & 0.03 \end{bmatrix}$. Do MATLAB simulations.

16. Let's revisit the min-projection strategy mentioned after Example 4.14. Consider a switching system (4.13)

$$\dot{x} = A_i x, \text{ where } i \in \mathcal{I} = \{1, 2, \dots, N\}.$$

The min-projection strategy is to choose the system mode at any time t according to

$$i = \arg \min_{j \in \mathcal{I}} x^T P A_j x. \quad (4.57)$$

- (a) Consider a switching system with $\mathcal{I} = \{1, 2\}$ and its state matrices given as

$$A_1 = \begin{bmatrix} -1 & 5 \\ 0 & -1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} -1 & 0 \\ -5 & -1 \end{bmatrix}.$$

Select P to be the identity matrix, apply the min-projection rule, derive the switching condition and do MATLAB simulation. Plot the state trajectories. Is the system stabilized?

- (b) Actually, the stabilization of min-projection strategy in the above example is not a coincidence. Prove that if $P > 0$ and there exists constants $\gamma > 0$ and $\delta_i \geq 0, i \in \mathcal{I}$ such that

$$\sum_{i=1}^N \delta_i > 0 \text{ and } \sum_{i=1}^N \delta_i (2x^T P A_i x + \gamma x^T x) \leq 0,$$

then the min-projection strategy exponentially stabilizes the closed-loop switching system.

- (c) Consider another switching system with $\mathcal{I} = \{1, 2, 3\}$ and its state matrices given as

$$\begin{aligned} A_1 &= \begin{bmatrix} 1.8631 & -0.0053 & 0.9129 \\ 0.2681 & -6.4962 & 0.0370 \\ 2.2497 & -6.7180 & 1.6428 \end{bmatrix}, \quad A_2 = \begin{bmatrix} -2.4311 & -5.1032 & 0.4565 \\ -0.8690 & 0.0869 & 0.0185 \\ 0.0369 & -5.9869 & 0.8214 \end{bmatrix}, \\ A_3 &= \begin{bmatrix} 0.0372 & -0.0821 & -2.7388 \\ 0.1941 & 0.2904 & -0.1110 \\ -1.0360 & 3.0486 & -4.9284 \end{bmatrix}. \end{aligned} \quad (4.58)$$

Let P be

$$P = \begin{bmatrix} 0.7001 & -0.2550 & -0.3024 \\ -0.2550 & 0.7219 & -0.1128 \\ -0.3024 & -0.1128 & 0.3549 \end{bmatrix}.$$

Apply the min-projection rule and simulate the system in MATLAB. Is the system stabilized? Do you observe Zeno behavior (infinite switching in finite time)?

- (d) As you should have observed in the above example, Zeno behavior could happen with min-projection strategy. In this case, the system enters a sliding mode and never leaves which could be undesirable in practical systems. Therefore, the min-projection strategy with hysteresis is suggested.

- Select a $\tilde{\gamma} \in (0, \gamma)$.
- Initially, the first system mode denoted as i is selected according to (4.57).
- The system mode remains the same until $x^T P A_i x + \tilde{\gamma} x^T x = 0$.
- Then select the system mode according to (4.57).

Apply this adjusted min-projection strategy with hysteresis to (4.58) and run simulations in MATLAB. Is the system stabilized? Do you observe Zeno behavior?

17. Consider the following switching controller for the wheeled mobile robot example (Example 2.19). First, it is convenient to consider the following state and control coordinate transformation:

$$\begin{aligned} y_1 &= \theta, \\ y_2 &= x_1 \cos \theta + x_2 \sin \theta, \\ y_3 &= x_1 \sin \theta - x_2 \cos \theta, \end{aligned}$$

and

$$\begin{aligned} D_1 &= \left\{ x \in \mathbb{R}^3 : |x_3| \geq \frac{\|x\|}{2} \right\}, \\ D_2 &= \left\{ x \in \mathbb{R}^3 : x \notin D_1 \right\}. \end{aligned}$$

Now, let's define a set of candidate controllers

$$\begin{aligned} u_1 &= \begin{bmatrix} u_1^1 \\ u_2^1 \end{bmatrix} = \begin{bmatrix} -4y_2 - 6\frac{y_3}{y_1} - y_3y_1 \\ -y_1 \end{bmatrix}, \\ u_2 &= \begin{bmatrix} u_1^2 \\ u_2^2 \end{bmatrix} = \begin{bmatrix} -y_2 - \text{sgn}(y_2y_3)y_3 \\ -\text{sgn}(y_2y_3) \end{bmatrix}, \end{aligned}$$

where $\text{sgn}(\cdot)$ is the signum function

$$\operatorname{sgn}(x) = \begin{cases} 1, & x \geq 0, \\ -1, & x < 0. \end{cases}$$

With these controllers, the closed-loop system can be represented as

$$\dot{x}(t) = f_{\sigma}(x(t)),$$

where $x = [x_1, x_2, \theta]^T$, $\sigma \in \{1, 2\}$, and

$$f_i(x) = \begin{bmatrix} u_1^i \cos \theta \\ u_1^i \sin \theta \\ u_2^i \end{bmatrix}, \quad i = 1, 2.$$

Finally, define the switching law by

$$\sigma(t) = \begin{cases} 1, & x(t) \in D_1, \\ 2, & x(t) \in D_2. \end{cases}$$

Show that the closed-loop system is asymptotically stable and run simulations of the closed-loop system with different initial conditions.

References

1. Hassan K Khalil and Jessy W Grizzle. *Nonlinear systems*. Prentice hall Upper Saddle River, NJ, 2002.
2. Eduardo D Sontag. *Mathematical control theory: deterministic finite dimensional systems*, volume 6. Springer, 1998.
3. Shankar Sastry. *Nonlinear systems: analysis, stability, and control*, volume 10. Springer Science & Business Media, 2013.
4. Jean-Jacques E Slotine, Weiping Li, et al. *Applied nonlinear control*, volume 199. Prentice hall Englewood Cliffs, NJ, 1991.
5. M. S. Branicky. Multiple lyapunov functions and other analysis tools for switched and hybrid systems. *IEEE Trans. Automat. Contr.*, 43(4):475–482, 1998.
6. R. A. DeCarlo, M. S. Branicky, S. Pettersson, and B. Lennartson. Perspectives and results on the stability and stabilizability of hybrid systems. In P. J. Antsaklis, editor, *Proceedings of the IEEE: Special issue on hybrid systems*, volume 88, pages 1069–1082. IEEE Press, 2000.
7. S. Boyd, L. El Ghaoui, E. Feron, and V. Balakrishnan. *Linear Matrix Inequalities in System and Control Theory*. SIAM, 1994.
8. D. Liberzon. *Switching in Systems and Control*. Birkhauser, Boston, 2003.
9. K. S. Narendra and J. Balakrishnan. A common lyapunov function for stable LTI systems with commuting a-matrices. *IEEE Trans. Automat. Contr.*, 39(12):2469–2471, 1994.
10. D. Liberzon, J. P. Hespanha, and A. S. Morse. Stability of switched linear systems: A lie-algebraic condition. *Syst. Contr. Lett.*, 37(3):117–122, 1999.
11. Michael Margaliot and Daniel Liberzon. Lie-algebraic stability conditions for nonlinear switched systems and differential inclusions. *Systems & control letters*, 55(1):8–16, 2006.
12. W. Dayawansa and C. F. Martin. A converse lyapunov theorem for a class of dynamical systems which undergo switching. *IEEE Trans. Automat. Contr.*, 44(4):751–760, 1999.

13. J. Daafouz, R. Riedinger, and C. Iung. Stability analysis and control synthesis for switched systems: a switched lyapunov function approach. *IEEE Trans. Automat. Contr.*, 47(11):1883–1887, 2002.
14. D. Liberzon and A. S. Morse. Basic problems in stability and design of switched systems. *IEEE Contr. Syst. Magazine*, 19(5):59–70, 1999.
15. Hai Lin and Panos J Antsaklis. Stability and stabilizability of switched linear systems: a survey of recent results. *IEEE Transactions on Automatic control*, 54(2):308–322, 2009.
16. R. Goebel, R. Sanfelice, and A. R. Teel. Hybrid dynamical systems. *IEEE Control Systems Magazine*, 29(2):28–93, 2009.
17. A. S. Morse. Supervisory control of families of linear set-point controllers—part 1: Exact matching. *IEEE Trans. Automat. Contr.*, 41(10):1413–1431, 1996.
18. J. P. Hespanha and A. S. Morse. Stability of switched systems with average dwell-time. In *Proc. 38th IEEE Conf. Decision Control*, pages 2655–2660, 1999.
19. Hai Lin, Guisheng Zhai, and Panos J Antsaklis. Robust stability and disturbance attenuation analysis of a class of networked control systems. In *Decision and Control, 2003. Proceedings. 42nd IEEE Conference on*, volume 2, pages 1182–1187. IEEE, 2003.
20. G. Zhai, B. Hu, K. Yasuda, and A. N. Michel. Disturbance attenuation properties of time-controlled switched systems. *Journal of the Franklin Institute*, 338(7):765–779, 2001.
21. G. Zhai, B. Hu, K. Yasuda, and A. N. Michel. Qualitative analysis of discrete-time switched systems. In *Proceedings of the 2002 American Control Conference*, volume 3, pages 1880–1885, 2002.
22. S.-L. Dai, H. Lin, and S. S. Ge. Scheduling-and-control codesign for a collection of networked control systems with uncertain delays. *IEEE Transactions on Control Systems Technology*, 18(1):66–78, 2010.
23. H. Ye, A. N. Michel, and L. Hou. Stability theory for hybrid dynamical systems. *IEEE Trans. Automat. Contr.*, 43(4):461–474, 1998.
24. S. Pettersson and B. Lennartson. Hybrid system stability and robustness verification using linear matrix inequalities. *Inter. J. Contr.*, 75(16-17):1335–1355, 2002.
25. M. A. Wicks, P. Peleties, and R. A. DeCarlo. Switched controller design for the quadratic stabilization of a pair of unstable linear systems. *European J. Control*, 4:140–147, 1998.
26. S. Pettersson and B. Lennartson. Stabilization of hybrid systems using a min-projection strategy. In *Proc. 2001 American Contr. Conf.*, pages 223–228, 2001.
27. E. Skafidas, R. J. Evans, A. V. Savkin, and I. R. Petersen. Stability results for switched controller systems. *Automatica*, 35(4):553–564, 1999.
28. Guisheng Zhai, Hai Lin, and Panos J Antsaklis. Quadratic stabilizability of switched linear systems with polytopic uncertainties. *International Journal of Control*, 76(7):747–753, 2003.
29. M. A. Wicks and R. A. DeCarlo. Solution of coupled lyapunov equations for the stabilization of multi-modal linear systems. In *Proc. 1997 American Contr. Conf.*, pages 1709–1713, 1997.
30. S. Pettersson. Synthesis of switched linear systems. In *Proc. 42nd IEEE Conf. Decision Control*, pages 5283–5288, 2003.
31. A. N. Michel. Recent trends in the stability analysis of hybrid dynamical systems. *IEEE Trans. Circuits Syst. I*, 46(1):120–134, 1999.
32. João P Hespanha. Stabilization through hybrid control. *encyclopedia of life support systems (EOLSS)*, 2004.
33. M. Johansson. *Piecewise Linear Control Systems: A Computational Approach*, volume 284 of *Lecture Notes in Control and Information Sciences*. Springer-Verlag, 2003.
34. R. Goebel, R. G. Sanfelice, and A. R. Teel. *Hybrid Dynamical Systems: modeling, stability, and robustness*. Princeton University Press, 2012.
35. Robert Shorten, Fabian Wirth, Oliver Mason, Kai Wulff, and Christopher King. Stability criteria for switched and hybrid systems. *SIAM review*, 49(4):545–592, 2007.
36. S. Prajna and A. Papachristodoulou. Analysis of switched and hybrid systems—beyond piecewise quadratic methods. In *Proc. 42nd IEEE Conf. Decision Control*, pages 2779–2784, 2003.
37. Antonis Papchristodoulou and Stephen Prajna. Robust stability analysis of nonlinear hybrid systems. *IEEE Transactions on Automatic Control*, 54(5):1034–1041, 2009.

38. Xuping Xu and Panos J Antsaklis. Stabilization of second-order LTI switched systems. *International Journal of Control*, 73(14):1261–1279, 2000.
39. Hai Lin and Panos J Antsaklis. Switching stabilizability for continuous-time uncertain switched linear systems. *IEEE Transactions on Automatic Control*, 52(4):633–646, 2007.
40. Wassim M Haddad, VijaySekhar Chellaboina, and Sergey G Nersesov. *Impulsive and hybrid dynamical systems: stability, dissipativity, and control*. Princeton University Press, 2006.
41. Panos J Antsaklis, Bill Goodwine, Vijay Gupta, Michael J McCourt, Yue Wang, Po Wu, Meng Xia, Han Yu, and Feng Zhu. Control of cyberphysical systems using passivity and dissipativity based methods. *European Journal of Control*, 19(5):379–388, 2013.
42. Etika Agarwal, Michael J McCourt, and Panos J Antsaklis. Dissipativity of hybrid systems: Feedback interconnections and networks. In *2016 American Control Conference (ACC)*, pages 6060–6065. IEEE, 2016.
43. Etika Agarwal, Michael J McCourt, and Panos J Antsaklis. Dissipativity of finite and hybrid automata: An overview. In *2017 25th Mediterranean Conference on Control and Automation (MED)*, pages 1176–1182. IEEE, 2017.
44. Zhendong Sun, Shuzhi Sam Ge, and Tong Heng Lee. Controllability and reachability criteria for switched linear systems. *Automatica*, 38(5):775–786, 2002.
45. Guangming Xie and Long Wang. Controllability and stabilizability of switched linear-systems. *Systems & Control Letters*, 48(2):135–155, 2003.
46. Alberto Bemporad, Giancarlo Ferrari-Trecate, and Manfred Morari. Observability and controllability of piecewise affine and hybrid systems. *IEEE transactions on automatic control*, 45(10):1864–1876, 2000.
47. Xuping Xu and Panos J Antsaklis. Design of stabilizing control laws for second-order switched systems. *IFAC Proceedings Volumes*, 32(2):1196–1201, 1999.
48. Bo Hu, Xuping Xu, Panos J Antsaklis, and Anthony N Michel. Robust stabilizing control laws for a class of second-order switched systems. *Systems & Control Letters*, 38(3):197–207, 1999.
49. Bo Hu, Xuping Xu, Anthony N Michel, and Panos J Antsaklis. Stability analysis for a class of nonlinear switched systems. In *Decision and Control, 1999. Proceedings of the 38th IEEE Conference on*, volume 5, pages 4374–4379. IEEE, 1999.
50. Lei Fang, Hai Lin, and Panos J Antsaklis. Stabilization and performance analysis for a class of switched systems. In *Decision and Control, 2004. CDC. 43rd IEEE Conference on*, volume 3, pages 3265–3270. IEEE, 2004.
51. Hai Lin and Panos J Antsaklis. Stability and persistent disturbance attenuation properties for a class of networked control systems: switched system approach. *International Journal of Control*, 78(18):1447–1458, 2005.
52. Hai Lin and Panos J Antsaklis. Hybrid state feedback stabilization with l_2 performance for discrete-time switched linear systems. *International Journal of Control*, 81(7):1114–1124, 2008.
53. Xenofon D Koutsoukos and Panos J Antsaklis. Design of stabilizing switching control laws for discrete-and continuous-time linear systems using piecewise-linear lyapunov functions. *International Journal of Control*, 75(12):932–945, 2002.

Chapter 5

Optimal Control



In the optimal control of hybrid systems, a stabilizing control law is designed to optimize, i.e., minimize or maximize, an appropriate performance criterion. Optimal control has been an active research area in Systems and Control, at least, since the 1950s, when seminal contributions such as Pontryagin's minimum principle and Bellman's dynamic programming were introduced. Optimal control problems for hybrid systems have attracted considerable attention since the mid 1990s, and extensive studies have been reported in the literature. Early studies may be found in the control systems literature involving relays or dynamical systems with hysteresis [1]. Significant efforts have been devoted to the extensions of dynamic programming and Pontryagin's minimum principle to hybrid systems using different formalisms, see e.g., [2–7] and the references therein.

In the present chapter, we focus on several fundamental results in the optimal control synthesis for hybrid systems. We start with a review of the calculus of variations and Pontryagin's minimum principle in Sect. 5.2. Then, we discuss extensions of the Pontryagin's minimum principle to hybrid optimal control problems in Sect. 5.3. The derived versions of the hybrid minimum principle cover general cases, including state-dependent switching, state resets, and associated costs. Unfortunately, the conditions derived for optimality in hybrid systems based on the powerful methods of minimum principles do not necessarily lead to computationally efficient methodologies to determine optimal solutions. The resulting multi-point boundary value problems that need to be solved are usually quite hard to solve even numerically. For this reason, in the second half of the chapter, we turn to important special cases of general hybrid systems—namely switched and piecewise linear systems—and focus on numerical methods. In particular, the two-stage optimization and embedding optimization approaches are discussed in Sect. 5.4 for the optimal control of switched systems. In Sect. 5.5, by representing piecewise linear control systems models as mixed-logic dynamic system models, it is shown that the optimal control problem may be solved as a mixed-integer optimization problem.

5.1 Optimal Control Problem

Hybrid optimal control problems are extensions of classical optimal control problems, we, therefore, begin with a brief introduction to the classical continuous-time optimal control theory.

The classical continuous-time optimal control problem can be stated as follows. Consider a continuous-time control system

$$\dot{x}(t) = f(x(t), u(t), t), \quad x(t_0) = x_0, \quad (5.1)$$

where $x \in \mathbb{R}^n$ is the system state vector, $u \in U \subseteq \mathbb{R}^m$ is the control input vector, t is time, t_0 is the initial time, and x_0 is the initial state. The system dynamics are represented by the vector field $f : \mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R} \rightarrow \mathbb{R}^n$, and it is assumed that $f \in C^h$ for some $h \geq 1$ with $h = 2$ unless otherwise specified.¹

An optimal control problem refers to the design of a control signal u for System (5.1) to minimize a cost functional that maps a control signal $u(t)$ and its corresponding state trajectory $x(t)$ of System (5.1) into a real number. The cost functional is usually given in the following form

$$J(u(\cdot)) = \int_{t_0}^{t_f} L(x(\tau), u(\tau), \tau) d\tau + \phi(x(t_f), t_f), \quad (5.2)$$

where $L : \mathbb{R}^n \times U \times \mathbb{R} \rightarrow \mathbb{R}$ and $\phi : \mathbb{R}^n \times \mathbb{R} \rightarrow \mathbb{R}$ are given functions called *running cost* and *final or terminal cost*, respectively. Here, the running cost function L and the final cost function ϕ have the same differentiability as the state function f .

Note that once u is given, the state trajectory x of System (5.1) is determined. We start with a simple form of the optimal control problem, where the terminal time t_f and the terminal state x_f are fixed. The set of control signals defined over the range $[t_0, t_f]$ that can drive the state from x_0 at t_0 to x_f at t_f is called the admissible control signals set, denoted by $\mathcal{U}_{ad}$. It is usually assumed that U is a compact set (closed and bounded) in $\mathbb{R}^m$, and hence $\mathcal{U}_{ad}$ consists of bounded and piecewise continuous signals. Formally, the optimal control problem can be stated as follows:

Optimal Control Problem: Minimize J given in (5.2) with respect to all admissible control signals $u \in \mathcal{U}_{ad}$ that drive the initial state x_0 to the final state x_f at time t_f for System (5.1).

It is also possible to make either the terminal time t_f or terminal state x_f free instead of fixed. Accordingly, we will have optimal control problems with free terminal time or free endpoint. It is also possible to free both t_f and x_f , but require a terminal constraint in the sense that $\Psi(t_f, x(t_f)) = 0$ must hold true for the termi-

¹ Here, $f \in C^h$ means that the function f is a h -times continuously differentiable function with $h \geq 0$ being an integer, for $h = 0$ f is just continuous, and C^∞ means the function f is smooth, or infinitely many times differentiable.

nal state and time—this corresponds to request that the final state must hit a given surface. We will see these variations in the following sections.

5.2 The Minimum Principle

This section reviews the classical optimal control theory, with a particular focus on the calculus of variations and the Pontryagin's minimum principle for classical optimal control problems. There are several excellent textbooks on the classical optimal control theory, for example, [8–10]. Our treatment mainly follows [8]. This section aims to provide necessary background for our study of hybrid optimal control problems. If the readers are already familiar with classical optimal control theory, this section can be skipped.

5.2.1 Calculus of Variations

Calculus of variations was introduced to find curves (functions) to optimize (usually minimize) the value of a functional, e.g., minimize the time required to travel from point a to point b . A *functional* J is a rule of correspondence that assigns to each function $x(t)$ in a certain class Ω a unique real number. For example,

$$J(x) = \int_0^1 x^2(t) dt,$$

is a functional defined over Ω that contains all bounded scalar functions $x(t) : [0, 1] \rightarrow \mathbb{R}$. Here Ω is called the domain of the functional, and the set of real numbers associated with the functions in Ω is called the range of the functional.

In the calculus of variations and optimal control, we need to define closeness between two curves. Hence, we assume that a norm $\|\cdot\|$ is defined on Ω , which assigns to each function $x(t) \in \Omega$ a non-negative real number, denoted as $\|x(t)\|$. For example, we can define the norm for Ω that contains all bounded scalar functions $x(t)$ for $0 \leq t \leq 1$ as

$$\|x(t)\| = \max_{0 \leq t \leq 1} |x(t)|. \quad (5.3)$$

The optimal curve should be such that neighboring curves (in the sense of closeness measured using the norm defined e.g., in (5.3)) do not lead to smaller costs. Thus, the derivative of the cost function about the optimal curve should be zero. In order to find an optimal curve, we take small variations about a candidate optimal solution and attempt to evaluate the change in the cost. If $x(t)$ and $x(t) + \delta x(t)$ are functions for which the functional J is defined, then increment of J , denoted by $\Delta J(x, \delta x)$ or ΔJ in short, is $\Delta J(x, \delta x) = J(x + \delta x) - J(x)$. Here δx is called the *variant* of the function x .

For example, the increment of the functional $J(x) = \int_0^1 x^2(t)dt$ is

$$\begin{aligned}\Delta J(x, \delta x) &= J(x + \delta x) - J(x) \\ &= \int_0^1 [x(t) + \delta x(t)]^2 dt - \int_0^1 x^2(t) dt \\ &= \int_0^1 \{2x(t)\delta x(t) + [\delta x(t)]^2\} dt.\end{aligned}$$

The *variation of J* , denoted as $\delta J(x, \delta x)$ or δJ in short, is the linear approximation to the increment of J , i.e., $\Delta J(x, \delta x)$. If δx is small, then δJ should be a good approximation to ΔJ . In the previous example, we have

$$\delta J(x, \delta x) = \int_0^1 2x(t)\delta x(t)dt.$$

A functional J with domain Ω has a *relative extremum* at x^* if there is an $\epsilon > 0$ such that for any function x in Ω that satisfies $\|x - x^*\| < \epsilon$, the increment of J has the same sign. If $\Delta J = J(x) - J(x^*) \geq 0$, then $J(x^*)$ is a *relative minimum*. Accordingly, x^* is called an *extremal*, and $J(x^*)$ is referred to as an extremum.

Fixed End-Point Problem

Our first problem is to find a curve $x(t)$ that minimizes the cost functional

$$J(x) = \int_{t_0}^{t_f} g(x(t), \dot{x}(t), t)dt, \quad (5.4)$$

where $x(t)$ is a curve with $x(t_0) = x_0$ and $x(t_f) = x_f$ fixed.

First, we compute the increment of J due to a variation of x

$$\begin{aligned}\Delta J(x, \delta x) &= J(x + \delta x) - J(x) \\ &= \int_{t_0}^{t_f} g(x + \delta x, \dot{x} + \delta \dot{x}, t)dt - \int_{t_0}^{t_f} g(x, \dot{x}, t)dt \\ &= \int_{t_0}^{t_f} [g(x + \delta x, \dot{x} + \delta \dot{x}, t) - g(x, \dot{x}, t)]dt,\end{aligned} \quad (5.5)$$

where $\dot{x}(t) = \frac{d}{dt}[x(t)]$, $\delta \dot{x}(t) = \frac{d}{dt}[\delta x(t)]$. Apply Taylor series expansion around $x(t), \dot{x}(t)$, to obtain

$$\begin{aligned}\Delta J(x, \delta x) &= \int_{t_0}^{t_f} \left\{ g(x, \dot{x}, t) + \frac{\partial g}{\partial x}(x, \dot{x}, t)\delta x(t) + \frac{\partial g}{\partial \dot{x}}(x, \dot{x}, t)\delta \dot{x}(t) \right. \\ &\quad \left. - g(x, \dot{x}, t) + o(\delta x(t), \delta \dot{x}(t)) \right\} dt.\end{aligned} \quad (5.6)$$

For simplicity in notation, we denote $\frac{\partial g}{\partial x} = [\frac{\partial g}{\partial x_1}, \dots, \frac{\partial g}{\partial x_n}]$ as a row vector, while δx and $\delta \dot{x}$ are denoted as column vectors. Ignoring the higher-order parts in $\Delta J(x, \delta x)$, we obtain

$$\delta J(x, \delta x) = \int_{t_0}^{t_f} \left\{ \frac{\partial g}{\partial x}(x, \dot{x}, t) \delta x(t) + \frac{\partial g}{\partial \dot{x}}(x, \dot{x}, t) \delta \dot{x}(t) \right\} dt. \quad (5.7)$$

Since $\delta \dot{x}(t) = \frac{d}{dt}[\delta x(t)]$, we can integrate-by-parts for the second item in the integral

$$\begin{aligned} \int_{t_0}^{t_f} \frac{\partial g}{\partial \dot{x}}(x, \dot{x}, t) \delta \dot{x}(t) dt &= \int_{t_0}^{t_f} \frac{\partial g}{\partial \dot{x}}(x, \dot{x}, t) d(\delta x(t)) \\ &= \left[\frac{\partial g}{\partial \dot{x}}(x, \dot{x}, t) \right] \delta x(t) \Big|_{t_0}^{t_f} - \int_{t_0}^{t_f} \frac{d}{dt} \left(\frac{\partial g}{\partial \dot{x}}(x, \dot{x}, t) \right) \delta x(t) dt. \end{aligned}$$

Therefore, we have

$$\delta J(x, \delta x) = \left[\frac{\partial g}{\partial \dot{x}}(x, \dot{x}, t) \right] \delta x(t) \Big|_{t_0}^{t_f} + \int_{t_0}^{t_f} \left[\frac{\partial g}{\partial x}(x, \dot{x}, t) - \frac{d}{dt} \left(\frac{\partial g}{\partial \dot{x}}(x, \dot{x}, t) \right) \right] \delta x(t) dt. \quad (5.8)$$

Also, note that $\delta x(t_0) = 0$ and $\delta x(t_f) = 0$ since $x(t_0) = x_0$ and $x(t_f) = x_f$ are both fixed. Hence

$$\delta J(x, \delta x) = \int_{t_0}^{t_f} \left[\frac{\partial g}{\partial x}(x, \dot{x}, t) - \frac{d}{dt} \left(\frac{\partial g}{\partial \dot{x}}(x, \dot{x}, t) \right) \right] \delta x(t) dt. \quad (5.9)$$

Given an extremal curve x^* , small perturbations around x^* should have vanishingly small effects on the cost functional J , namely a necessary condition indicates that the cost function's linear sensitivity to small variations around the optimal should be zero. Therefore, for an extremal curve x^* , it is necessary that $\delta J(x^*, \delta x) = 0$ for arbitrary δx . It implies that

$$\frac{\partial g}{\partial x}(x^*(t), \dot{x}^*(t), t) - \frac{d}{dt} \left[\frac{\partial g}{\partial \dot{x}}(x^*(t), \dot{x}^*(t), t) \right] = 0, \quad (5.10)$$

for all $t \in [t_0, t_f]$ together with the boundary condition $x^*(t_0) = x_0, x^*(t_f) = x_f$.

Note that $g(\cdot)$ is given, and assumed to be continuously differentiable, so the above equation is actually a nonlinear differential equation, which is known as the *Euler equation*. The difficulty in solving the Euler equation is mainly due to the boundary conditions, which are actually split, resulting in a problem known as a two-point boundary value problem. Should the boundary condition be given either at the initial point t_0 or at t_f , we can solve the differential equation by either forward or backward numerical integration. The split boundary conditions make the problem difficult to solve.

Free End-Point Problems

If the final time t_f is free, then we need to consider the variations in t_f as well:

$$\begin{aligned}\Delta J(x, \delta x, \delta t_f) &= \int_{t_0}^{t_f + \delta t_f} g(x + \delta x, \dot{x} + \delta \dot{x}, t) dt - \int_{t_0}^{t_f} g(x, \dot{x}, t) dt \\ &= \int_{t_0}^{t_f} [g(x + \delta x, \dot{x} + \delta \dot{x}, t) - g(x, \dot{x}, t)] dt + \int_{t_f}^{t_f + \delta t_f} g(x + \delta x, \dot{x} + \delta \dot{x}, t) dt.\end{aligned}$$

The first integrand can be treated as before, so we pay attention to the second integrand, which can be written as

$$\int_{t_f}^{t_f + \delta t_f} g(x + \delta x, \dot{x} + \delta \dot{x}, t) dt = \left[g(x(t_f), \dot{x}(t_f), t_f) \right] \delta t_f + o(\delta t_f). \quad (5.11)$$

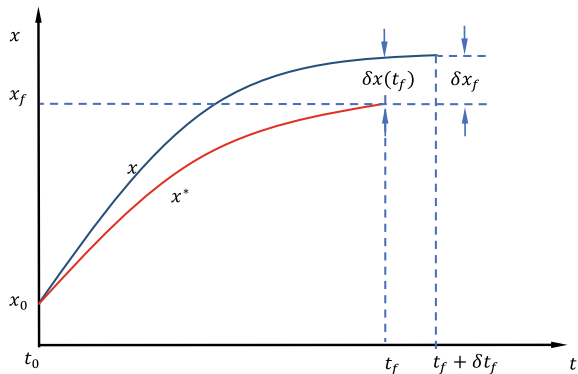
Hence, we obtain (by only retaining first-order terms)

$$\begin{aligned}\delta J(x, \delta x, \delta t_f) &= \left[\frac{\partial g}{\partial \dot{x}}(x(t_f), \dot{x}(t_f), t_f) \right] \delta x(t_f) + \left[g(x(t_f), \dot{x}(t_f), t_f) \right] \delta t_f \\ &\quad + \int_{t_0}^{t_f} \left[\frac{\partial g}{\partial x}(x, \dot{x}, t) - \frac{d}{dt} \left(\frac{\partial g}{\partial \dot{x}}(x, \dot{x}, t) \right) \right] \delta x(t) dt.\end{aligned} \quad (5.12)$$

Note that $\delta x(t_f)$ is not the same as δx_f . Actually, $\delta x(t_f)$ is the difference in ordinates at t_f , while δx_f is the difference of the terminal points between two curves, namely $\delta x(t_f) = x(t_f) - x^*(t_f)$, while $\delta x_f = x(t + \delta t_f) - x^*(t_f)$ as illustrated in Fig. 5.1. In addition, $\delta x(t_f)$ and δt_f are not independent. By inspection of Fig. 5.1, $\delta x(t_f)$ can be linearly approximated by

$$\delta x(t_f) \approx \delta x_f - \dot{x}^*(t_f) \delta t_f. \quad (5.13)$$

Fig. 5.1 An extremal x^* and a neighboring comparison curve x



Plug (5.13) into (5.12), and we obtain

$$\begin{aligned}\delta J(x, \delta x, \delta t_f) &= \left[\frac{\partial g}{\partial \dot{x}}(x(t_f), \dot{x}(t_f), t_f) \right] \delta x_f \\ &\quad + \left\{ \left[g(x(t_f), \dot{x}(t_f), t_f) \right] - \left[\frac{\partial g}{\partial \dot{x}}(x(t_f), \dot{x}(t_f), t_f) \right] \dot{x}(t_f) \right\} \delta t_f \\ &\quad + \int_{t_0}^{t_f} \left[\frac{\partial g}{\partial x}(x, \dot{x}, t) - \frac{d}{dt} \left(\frac{\partial g}{\partial \dot{x}}(x, \dot{x}, t) \right) \right] \delta x(t) dt.\end{aligned}\quad (5.14)$$

If t_f and $x(t_f)$ are unrelated, then δx_f and δt_f are independent, so their coefficients must be zero for an extremal curve x^* . Hence, we have the following boundary conditions for Euler equation

$$\frac{\partial g}{\partial \dot{x}}(x^*(t_f), \dot{x}^*(t_f), t_f) = 0 \quad (5.15)$$

and

$$g(x^*(t_f), \dot{x}^*(t_f), t_f) - \left[\frac{\partial g}{\partial \dot{x}}(x^*(t_f), \dot{x}^*(t_f), t_f) \right] \dot{x}^*(t_f) = 0, \quad (5.16)$$

which together imply that

$$g(x^*(t_f), \dot{x}^*(t_f), t_f) = 0. \quad (5.17)$$

This provides a boundary condition, together with $x(t_0) = x_0$, for the Euler equation.

With Terminal Cost

Now we add a terminal cost into the cost functional

$$J(x) = \phi(x(t_f), t_f) + \int_{t_0}^{t_f} g(x(t), \dot{x}(t), t) dt, \quad (5.18)$$

where $x(t)$ is a curve with $x(t_0) = x_0$ fixed.

The contribution from the terminal cost ϕ to the variation of J , can be estimated as

$$\begin{aligned}\Delta \phi &= \phi(x(t_f + \delta t_f), t_f + \delta t_f) - \phi(x^*(t_f), t_f^*) \\ &= \frac{\partial \phi}{\partial x}(x^*(t_f), t_f^*) \delta x_f + \frac{\partial \phi}{\partial t}(x^*(t_f), t_f^*) \delta t_f + o(\delta t_f, \delta x_f).\end{aligned}$$

Hence, we obtain (by only retaining first-order terms)

$$\begin{aligned}
\delta J(x, \delta x, \delta t_f) &= \frac{\partial \phi}{\partial x}(x^*(t_f), t_f^*) \delta x_f + \frac{\partial \phi}{\partial t}(x^*(t_f), t_f^*) \delta t_f + \frac{\partial g}{\partial \dot{x}}(x^*(t_f), \dot{x}^*(t_f), t_f^*) \delta x_f \\
&\quad + \left\{ g(x^*(t_f), \dot{x}^*(t_f), t_f^*) - \left[\frac{\partial g}{\partial \dot{x}}(x^*(t_f), \dot{x}^*(t_f), t_f^*) \right] \dot{x}^*(t_f) \right\} \delta t_f \\
&\quad + \int_{t_0}^{t_f} \left[\frac{\partial g}{\partial x}(x^*, \dot{x}^*, t) - \frac{d}{dt} \left(\frac{\partial g}{\partial \dot{x}}(x^*, \dot{x}^*, t) \right) \right] \delta x(t) dt.
\end{aligned} \tag{5.19}$$

Besides the Euler equation

$$\frac{\partial g}{\partial x}(x^*, \dot{x}^*, t) - \frac{d}{dt} \left(\frac{\partial g}{\partial \dot{x}}(x^*, \dot{x}^*, t) \right) = 0 \tag{5.20}$$

holds for $t \in [t_0, t_f]$, the following conditions need to hold for an extremal curve x^* .

If we can ensure that δt_f and δx_f are independent, then it is necessary that

$$\frac{\partial \phi}{\partial x}(x^*(t_f), t_f^*) + \frac{\partial g}{\partial \dot{x}}(x^*(t_f), \dot{x}^*(t_f), t_f^*) = 0 \tag{5.21}$$

as the coefficient of δx_f and

$$\frac{\partial \phi}{\partial t}(x^*(t_f), t_f^*) + \left\{ g(x^*(t_f), \dot{x}^*(t_f), t_f^*) - \left[\frac{\partial g}{\partial \dot{x}}(x^*(t_f), \dot{x}^*(t_f), t_f^*) \right] \dot{x}^*(t_f) \right\} = 0$$

as the coefficient of δt_f .

However, situations could be more complicated if δt_f and δx_f are related, which will be dealt with in the next subsections in the context of optimal control.

5.2.2 Necessary Conditions for Optimal Control Solutions

Recall the continuous-time control system

$$\dot{x}(t) = f(x(t), u(t), t), \quad x(t_0) = x_0, \tag{5.22}$$

and the cost functional

$$J = \phi(x(t_f), t_f) + \int_{t_0}^{t_f} L(x(t), u(t), t) dt, \tag{5.23}$$

with terminal constraints

$$\psi(x(t_f), t_f) = 0. \tag{5.24}$$

To take into consideration the state differential equation constraint and the terminal constraint, we use Lagrange multipliers $p(t) \in \mathbb{R}^n$ and $v \in \mathbb{R}$ to obtain the following augmented performance index

$$J_a = \phi(x(t_f), t_f) + v\psi(x(t_f), t_f) + \int_{t_0}^{t_f} \left\{ L(x, u, t) + p^T(t)[f(x, u, t) - \dot{x}(t)] \right\} dt. \quad (5.25)$$

If we define $g_a(x, \dot{x}, u, p, t) = L(x, u, t) + p^T(t)[f(x, u, t) - \dot{x}(t)]$ and $\Phi = \phi + v\psi$, then J_a can be rewritten as

$$J_a = \Phi(x(t_f), t_f) + \int_{t_0}^{t_f} g_a(x, \dot{x}, u, p, t) dt, \quad (5.26)$$

which is in the same form as the cost functional considered in the previous subsection. Hence, we obtain

$$\begin{aligned} \delta J_a = & \left[\frac{\partial \Phi}{\partial x}(x, t) + \frac{\partial g_a}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right]_{t_f} \delta x_f \\ & + \left\{ \frac{\partial \Phi}{\partial t}(x, t) \Big|_{t_f} + g_a(x, \dot{x}, u, p, t) \Big|_{t_f} - \left[\frac{\partial g_a}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right]_{t_f} \dot{x}(t_f) \right\} \delta t_f \\ & + \int_{t_0}^{t_f} \left\{ \left[\frac{\partial g_a}{\partial x}(x, \dot{x}, u, p, t) - \frac{d}{dt} \left(\frac{\partial g_a}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right) \right] \delta x(t) \right. \\ & \left. + \left[\frac{\partial g_a}{\partial u}(x, \dot{x}, u, p, t) \right] \delta u(t) + \left[\frac{\partial g_a}{\partial p}(x, \dot{x}, u, p, t) \right] \delta p(t) \right\} dt. \end{aligned} \quad (5.27)$$

First, we consider the items inside the integral. Note that $g_a(x, \dot{x}, u, p, t) = L(x, u, t) + p^T(t)[f(x, u, t) - \dot{x}(t)]$, so we have

$$\begin{aligned} \frac{\partial g_a}{\partial x} &= \frac{\partial L}{\partial x} + p^T \frac{\partial f}{\partial x}, \\ \frac{\partial g_a}{\partial \dot{x}} &= -p^T, \\ \frac{\partial g_a}{\partial u} &= \frac{\partial L}{\partial u} + p^T \frac{\partial f}{\partial u}, \\ \frac{\partial g_a}{\partial p} &= f^T(x, u, t) - \dot{x}(t)^T. \end{aligned}$$

For extremal curves (optimal solutions) x^*, u^* and p^* , all coefficients must vanish, so we have (considering the coefficient of $\delta x(t)$)

$$\begin{aligned} & \frac{\partial g_a}{\partial x}(x, \dot{x}, u, p, t) - \frac{d}{dt} \left(\frac{\partial g_a}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right) = 0 \\ \implies & \frac{\partial L}{\partial x}(x^*(t), u^*(t), t) + [p^*(t)]^T \frac{\partial f}{\partial x}(x^*(t), u^*(t), t) + \left[\frac{d}{dt} p^*(t) \right]^T = 0 \\ \implies & \dot{p}^*(t) = - \left[\frac{\partial L}{\partial x}(x^*(t), u^*(t), t) + p^{*T}(t) \frac{\partial f}{\partial x}(x^*(t), u^*(t), t) \right]^T, \end{aligned} \quad (5.28)$$

which is called a *co-state equation*.

In addition, considering the coefficient of $\delta u(t)$, we have

$$\frac{\partial L}{\partial u}(x^*(t), u^*(t), p^*(t), t) + p^{*T}(t) \left(\frac{\partial f}{\partial u}(x^*(t), u^*(t), p^*(t), t) \right) = 0. \quad (5.29)$$

Furthermore, if we define a Hamiltonian function as

$$\mathcal{H}(x, u, p, t) = L(x, u, t) + p^T(t) f(x, u, t), \quad (5.30)$$

then the above condition can be written as

$$\frac{\partial \mathcal{H}}{\partial u}(x^*(t), u^*(t), p^*(t), t) = 0, \quad (5.31)$$

which is called an *optimization condition*.

The last item inside the integral implies that the optimal solution should satisfy the *state equation*

$$\dot{x}^*(t) = f(x^*(t), u^*(t), t). \quad (5.32)$$

With the definition of Hamiltonian function $\mathcal{H}$, the state and co-state equations can be written as

$$\dot{x}^*(t) = \frac{\partial \mathcal{H}^T}{\partial p}(x^*(t), u^*(t), p^*(t), t), \quad (5.33)$$

$$\dot{p}^*(t) = -\frac{\partial \mathcal{H}^T}{\partial x}(x^*(t), u^*(t), p^*(t), t). \quad (5.34)$$

Next, we consider the items outside the integral, that is

$$\begin{aligned} & \left[\frac{\partial \Phi}{\partial x}(x, t) + \frac{\partial g_a}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right]_{t_f} \delta x_f \\ & + \left\{ \frac{\partial \Phi}{\partial t}(x, t) \Big|_{t_f} + g_a(x, \dot{x}, u, p, t) \Big|_{t_f} - \left[\frac{\partial g_a}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right]_{t_f} \dot{x}(t_f) \right\} \delta t_f. \end{aligned}$$

Note that $\frac{\partial g_a}{\partial \dot{x}} = -p^T$ and $g_a - \frac{\partial g_a}{\partial \dot{x}} \dot{x} = g_a + p^T \dot{x} = \mathcal{H}$. Therefore, the items outside the integral can be written as

$$\left[\frac{\partial \Phi}{\partial x}(x, t) - p^T(t) \right]_{t_f} \delta x_f + \left[\frac{\partial \Phi}{\partial t}(x, t) + \mathcal{H}(x, \dot{x}, u, p, t) \right]_{t_f} \delta t_f.$$

There are several cases that need to be considered.

Fixed t_f and Fixed x_f

Then $\delta x_f = 0$ and $\delta t_f = 0$, and the boundary conditions are simply $x(t_f) = x_f$.

Fixed t_f But Free x_f

Then $\delta t_f = 0$ and the coefficient for δx_f should be zero, which implies the following boundary condition

$$\frac{\partial \phi}{\partial x}(x^*(t_f), t_f) - p^{*T}(t_f) = 0, \quad (5.35)$$

or equivalently

$$p^*(t_f) = \frac{\partial \phi^T}{\partial x}(x^*(t_f), t_f), \quad (5.36)$$

which is called a *transversality condition*.

Fixed x_f But Free t_f

If x_f is fixed but t_f is free, then we have $\delta x_f = 0$. For this case, it is necessary for the coefficient of δt_f to be zero. Hence, we obtain the following condition

$$\frac{\partial \phi}{\partial t}(x(t_f), t_f) + \mathcal{H}(x(t_f), u(t_f), p(t_f), t_f) = 0. \quad (5.37)$$

Before we consider the most general case when both x_f and t_f are not fixed but are not independent either, we take a look at an example to illustrate the necessary conditions we have obtained for an optimal control solution so far.

Example 5.1 We use $q \in \mathbb{R}^n$ to denote the generalized coordinate (state) of a mechanical system, and $u = \dot{q}$ to represent the generalized velocity. The kinetic energy of the system is denoted as $T(q, u)$, while the potential energy of the system is denoted as $U(q)$. Hamilton's principle of least action says that a conservative system moves so as to minimize the time integral of its “action”—the difference between the kinetic and potential energy. Therefore, Hamilton's principle suggests to solve an optimal control problem for the system

$$\dot{q}(t) = u(t) \quad (5.38)$$

with respect to the cost functional

$$I(u) = \int_{t_0}^{t_f} [T(q, u) - U(q)] dt. \quad (5.39)$$

Define the Lagrangian

$$L(q, u) = T(q, u) - U(q), \quad (5.40)$$

and the Hamiltonian

$$\mathcal{H}(q, u, p) = L(q, u) + p^T u. \quad (5.41)$$

Consequently, the following equations should be satisfied by an optimal solution

$$\dot{q} = \frac{\partial \mathcal{H}^T}{\partial p} = u, \quad (5.42)$$

$$\dot{p} = -\frac{\partial \mathcal{H}^T}{\partial q} = -\frac{\partial L^T}{\partial q}, \quad (5.43)$$

$$0 = \frac{\partial \mathcal{H}}{\partial u} = \frac{\partial L}{\partial u} + p^T. \quad (5.44)$$

Combining these equations yields the well-known Euler-Lagrange equations

$$\frac{\partial L}{\partial q} - \frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}} \right) = 0. \quad (5.45)$$

Also, notice that $p^T = -\frac{\partial L}{\partial u}$, so $\mathcal{H}$ can be represented as

$$\mathcal{H}(q, u, p) = L(q, u) - \frac{\partial L}{\partial u} U(q). \quad (5.46)$$

Furthermore, if $T(q, u)$ is a homogeneous quadratic form in $u = \dot{q}$, we have

$$\frac{\partial T}{\partial u} u = 2T. \quad (5.47)$$

Note that $L(q, u) = T(q, u) - U(q)$, so $\frac{\partial L}{\partial u} = \frac{\partial T}{\partial u}$. Hence, we have

$$\mathcal{H}(q, u, p) = T(q, u) - U(q) - 2T(q, u) = -(T(q, u) + U(q)), \quad (5.48)$$

which implies that $-\mathcal{H} = T + U$ is the total energy.

If L and U are not explicit functions of time t , then we have

$$\begin{aligned} \frac{d}{dt} \mathcal{H}(q, u, p) &= \frac{\partial \mathcal{H}}{\partial q} \dot{q} + \frac{\partial \mathcal{H}}{\partial u} \dot{u} + \dot{p}^T u \\ &= \frac{\partial \mathcal{H}}{\partial u} \dot{u} + \left(\frac{\partial \mathcal{H}}{\partial q} + \dot{p}^T \right) u \\ &= 0, \end{aligned}$$

since $\frac{\partial \mathcal{H}}{\partial u} = 0$ and $\dot{p}^T = -\frac{\partial \mathcal{H}}{\partial q}$. Hence, $\mathcal{H}$ is a constant, which implies the well-known energy conservation principle. $\square$

Free t_f and Free x_f , But Dependent

Now we consider a more general case, where t_f is free but we request that the final state satisfies the terminal constraint

$$\Psi(x(t_f), t_f) = 0. \quad (5.49)$$

In this case, δt_f and δx_f are no longer independent since to remain on the surface of $\Psi(x(t), t) = 0$ the value of δx_f will depend on δt_f as illustrated in Fig. 5.2.

Inspection of Fig. 5.2 suggests that the vector with components δx_1 , δx_2 and δt_f must be contained in a plane tangent to the surface at the point $(x^*(t_f), t_f)$. This means that admissible variations must be orthogonal to the normal vector of the tangent plane at the point $(x^*(t_f), t_f)$, namely

$$\left[\frac{\partial \phi}{\partial x}(x^*(t_f), t_f) - p^{*T}(t_f) \right] \delta x_f + \left[\frac{\partial \phi}{\partial t}(x^*(t_f), t_f) + \mathcal{H}(x^*(t_f), u^*(t_f), p^*(t_f), t_f) \right] \delta t_f = 0,$$

which means that the vector

$$\begin{bmatrix} \frac{\partial \phi}{\partial x}(x^*(t_f), t_f) - p^{*T}(t_f) \\ \frac{\partial \phi}{\partial t}(x^*(t_f), t_f) + \mathcal{H}(x^*(t_f), u^*(t_f), p^*(t_f), t_f) \end{bmatrix} \quad (5.50)$$

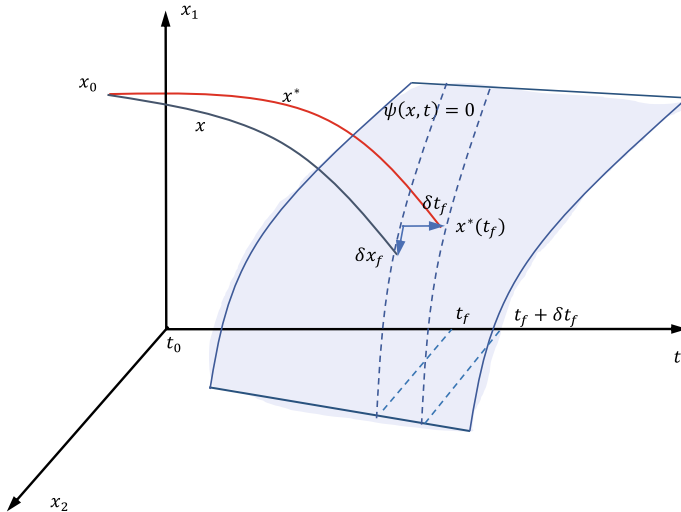


Fig. 5.2 An extremal x^* and a neighboring comparison curve x

is orthogonal to the vector

$$\begin{bmatrix} \delta x_f \\ \delta t_f \end{bmatrix}. \quad (5.51)$$

Hence

$$\begin{bmatrix} \frac{\partial \phi}{\partial x}(x^*(t_f), t_f) - p^{*T}(t_f) \\ \frac{\partial \phi}{\partial t}(x^*(t_f), t_f) + \mathcal{H}(x^*(t_f), u^*(t_f), p^*(t_f), t_f) \end{bmatrix} \quad (5.52)$$

can be written as a linear combination of the gradient vector

$$\begin{bmatrix} \frac{\partial \Phi}{\partial x}(x^*(t_f), t_f) \\ \frac{\partial \Phi}{\partial t}(x^*(t_f), t_f) \end{bmatrix}. \quad (5.53)$$

Namely, there exist constants λ and μ such that

$$\frac{\partial \phi}{\partial x}(x^*(t_f), t_f) - p^{*T}(t_f) = \lambda \frac{\partial \Phi}{\partial x}(x^*(t_f), t_f), \quad (5.54)$$

$$\frac{\partial \phi}{\partial t}(x^*(t_f), t_f) + \mathcal{H}(x^*(t_f), u^*(t_f), p^*(t_f), t_f) = \mu \frac{\partial \Phi}{\partial t}(x^*(t_f), t_f). \quad (5.55)$$

This, together with the initial condition $x(t_0) = x_0$, provides boundary conditions for the Euler equation, state and co-state equations. To solve for extremal solutions, we start with the optimization condition to derive the form of an optimal control u , usually as a function of the state x and co-state p . Then, we plug in the obtained control u into the state and co-state equations, which can be combined into autonomous (without external control u) differential equations of x and p (together with boundary conditions). Should we be able to solve the differential equations for extremal x and p , we can plug into the derived control u in the first step and obtain a candidate of an optimal control signal. The following example from [8] illustrates these steps.

Example 5.2 Consider a control system

$$\begin{cases} \dot{x}_1(t) = x_2(t), \\ \dot{x}_2(t) = -x_2(t) + u(t), \end{cases} \quad (5.56)$$

with the following cost functional

$$J(u) = \int_{t_0}^{t_f} \frac{1}{2} u^2(t) dt, \quad (5.57)$$

to be minimized. The admissible control u is assumed to be not bounded. First, we obtain the Hamiltonian as

$$\mathcal{H}(x, u, p) = \frac{1}{2} u^2(t) + p_1(t)x_2(t) - p_2(t) + p_2(t)u(t). \quad (5.58)$$

The necessary conditions for an optimal solution $x^*(t)$, $u^*(t)$, $p^*(t)$ are

- Co-state equation:

$$\begin{cases} \dot{p}_1^*(t) = -\frac{\partial \mathcal{H}}{\partial x_1} = 0, \\ \dot{p}_2^*(t) = -\frac{\partial \mathcal{H}}{\partial x_2} = -p_1^*(t) + p_2^*(t). \end{cases} \quad (5.59)$$

- Optimization condition:

$$0 = \frac{\partial \mathcal{H}}{\partial u} = u^*(t) + p_2^*(t). \quad (5.60)$$

- State equation:

$$\begin{cases} \dot{x}_1^*(t) = x_2^*(t), \\ \dot{x}_2^*(t) = -x_2^*(t) + u^*(t). \end{cases} \quad (5.61)$$

From the optimization condition, we can solve for $u^*(t) = -p_2(t)$. Plug in this u^* into the state equation, to obtain

$$\begin{cases} \dot{x}_1^*(t) = x_2^*(t), \\ \dot{x}_2^*(t) = -x_2^*(t) - p_2^*(t). \end{cases} \quad (5.62)$$

Together with the co-state equation, we obtain a set of linear, first-order, homogeneous, ordinary different equations, which can be solved (see Appendix A or refer to any textbook on linear systems). Solving these equations gives

$$\begin{aligned} x_1^*(t) &= c_1 + c_2(1 - e^{-t}) + c_3(-t - \frac{1}{2}e^{-t} + \frac{1}{2}e^t) + c_4(1 - \frac{1}{2}e^{-t} - \frac{1}{2}e^t), \\ x_2^*(t) &= c_2e^{-t} + c_3(-1 + \frac{1}{2}e^{-t} + \frac{1}{2}e^t) + c_4(\frac{1}{2}e^{-t} - \frac{1}{2}e^t), \\ p_1^*(t) &= c_3, \\ p_2^*(t) &= c_3(1 - e^t) + c_4e^t, \end{aligned} \quad (5.63)$$

where c_1 , c_2 , c_3 , and c_4 are constants to be decided from boundary conditions.

Case 1: Let's first assume that the initial state x_0 and starting time t_0 are fixed and given as $x(0) = [0, 0]^T$, while the final time t_f and state x_f are given as well with $x(2) = [5, 2]^T$. With these boundary conditions, we can get four linear equations (by plugging $x(0)$ and $x(2)$ into $x_1^*(t)$ and $x_2^*(t)$, respectively) with four unknowns (c_1 , c_2 , c_3 , and c_4), so we can solve these linear equation and obtain the following unique solution $c_1 = 0$, $c_2 = 0$, $c_3 = -7.289$, and $c_4 = -6.103$. Then, we obtain the following optimal control signal

$$u^*(t) = -p_2^*(t) = -c_3(1 - e^t) - c_4e^t = 7.289(1 - e^t) + 6.103e^t. \quad (5.64)$$

Case 2: Let's consider a different scenario where, instead of fixing the terminal state x_f , we request that the terminal constraint in the form of $\Phi(x(t_f), t_f) = 0$ needs to hold at $t_f = 2$. In particular, we select

$$\Phi(x, t) = x_1(t) + 5x_2(t) - 15. \quad (5.65)$$

Besides the boundary condition $x(0) = [0, 0]^T$ as before, which implies $c_1 = c_2 = 0$, we need additional boundary conditions from the terminal constraint. Recall that there need to exist constants λ and μ such that

$$\begin{aligned} \frac{\partial \phi}{\partial x}(x^*(t_f), t_f) - p^*(t_f) &= \lambda \frac{\partial \Phi}{\partial x}(x^*(t_f), t_f), \\ \frac{\partial \phi}{\partial t}(x^*(t_f), t_f) + \mathcal{H}(x^*(t_f), u^*(t_f), p^*(t_f), t_f) &= \mu \frac{\partial \Phi}{\partial t}(x^*(t_f), t_f). \end{aligned}$$

For our example, here $\phi(x, t) = 0$ and $\Phi(x, t) = x_1(t) + 5x_2(t) - 15$ does not depend on time t explicitly. So the boundary condition can be simplified into

$$\begin{aligned} -p^*(t_f) &= \lambda \frac{\partial \Phi}{\partial x}(x^*(t_f), t_f), \\ \mathcal{H}(x^*(t_f), u^*(t_f), p^*(t_f), t_f) &= 0. \end{aligned}$$

From the first equation, we obtain

$$\begin{aligned} -p_1^*(2) &= \lambda \frac{\partial \Phi}{\partial x_1}(x^*(2), 2) = \lambda, \\ -p_2^*(2) &= \lambda \frac{\partial \Phi}{\partial x_2}(x^*(2), 2) = 5\lambda. \end{aligned} \quad (5.66)$$

Together with the final constraint

$$x_1^*(2) + 5x_2^*(2) - 15 = 0, \quad (5.67)$$

we can now solve for c_3, c_4 , which are $c_3 = -0.894, c_4 = -1.379$. For this case, the optimal control signal is given as

$$u^*(t) = -p_2^*(t) = -c_3(1 - e^t) - c_4e^t = 0.894(1 - e^t) + 1.379e^t. \quad (5.68)$$

Also, similar to the arguments in Example 5.1, the Hamiltonian is a constant along optimal solutions, together with the boundary condition $\mathcal{H}(x^*(t_f), u^*(t_f), p^*(t_f), t_f) = 0$, we now can conclude that the Hamiltonian $\mathcal{H}$ is constant zero along the optimal solutions in this example. $\square$

As we can see, the obtained optimal control signal is in an open-loop form. For some special cases, we may be able to get feedback forms as shown in the following example.

Example 5.3 Consider the following linear control system

$$\dot{x}(t) = A(t)x(t) + B(t)u(t), \quad x(t_0) = x_0, \quad (5.69)$$

where $x(t) \in \mathbb{R}^n$, $u(t) \in \mathbb{R}^m$, $A(t) \in \mathbb{R}^{n \times n}$ and $B(t) \in \mathbb{R}^{n \times m}$. The goal is to design $u(t)$ to minimize the cost functional in a quadratic form as

$$J(u) = \frac{1}{2}x^T(t_f)Mx(t_f) + \frac{1}{2} \int_{t_0}^{t_f} \left[x^T(t)Qx(t) + u^T(t)Ru(t) \right] dt, \quad (5.70)$$

where $M, Q \in \mathbb{R}^{n \times n}$ are symmetric and semi-positive definite, $R \in \mathbb{R}^{m \times m}$ is symmetric and positive definite. Also, it is assumed that t_f is fixed while $x(t_f) = x_f$ is free.

The Hamiltonian $\mathcal{H}$ is

$$\mathcal{H}(x, u, p, t) = \frac{1}{2}x^T(t)Qx(t) + \frac{1}{2}u^T(t)Ru(t) + p^T(t)A(t)x(t) + p^T(t)B(t)u(t), \quad (5.71)$$

and necessary conditions for optimality are

$$\begin{aligned} \dot{x}^*(t) &= A(t)x^*(t) + B(t)u^*(t), \\ \dot{p}^*(t) &= -Q(t)x^*(t) - A^T(t)p^*(t), \\ 0 &= \frac{\partial \mathcal{H}}{\partial u} = R(t)u^*(t) + B^T(t)p^*(t). \end{aligned} \quad (5.72)$$

Use the last equation to obtain

$$u^*(t) = -R^{-1}(t)B^T(t)p^*(t). \quad (5.73)$$

Plug this $u^*(t)$ into the state equation to obtain

$$\dot{x}^*(t) = A(t)x^*(t) - B(t)R^{-1}(t)B^T(t)p^*(t). \quad (5.74)$$

Together with the co-state equation, we have the following:

$$\begin{bmatrix} \dot{x}^*(t) \\ \dot{p}^*(t) \end{bmatrix} = \begin{bmatrix} A(t) & -B(t)R^{-1}(t)B^T(t) \\ -Q(t) & -A^T(t) \end{bmatrix} \begin{bmatrix} x^*(t) \\ p^*(t) \end{bmatrix}, \quad (5.75)$$

which is a linear autonomous system (without control) and its solution can be obtained as

$$\begin{bmatrix} x^*(t) \\ p^*(t) \end{bmatrix} = \Phi(t, t_0) \begin{bmatrix} x^*(t_0) \\ p^*(t_0) \end{bmatrix}, \quad (5.76)$$

where $\Phi(t, t_0)$ stands for the state transition matrix that transits the state from the initial state x_0 to the current state $x(t)$. Partitioning $\Phi(t, t_0)$ and setting $t = t_f$ and $t_0 = t$, we obtain

$$\begin{bmatrix} x^*(t_f) \\ p^*(t_f) \end{bmatrix} = \begin{bmatrix} \Phi_{11}(t_f, t) & \Phi_{12}(t_f, t) \\ \Phi_{21}(t_f, t) & \Phi_{22}(t_f, t) \end{bmatrix} \begin{bmatrix} x^*(t) \\ p^*(t) \end{bmatrix}. \quad (5.77)$$

Hence, we have

$$x^*(t_f) = \Phi_{11}(t_f, t)x^*(t) + \Phi_{12}(t_f, t)p^*(t), \quad (5.78)$$

and

$$p^*(t_f) = \Phi_{21}(t_f, t)x^*(t) + \Phi_{22}(t_f, t)p^*(t). \quad (5.79)$$

In addition, according to the boundary condition (fixed t_f and free x_f), we have

$$p^*(t_f) = \frac{\partial}{\partial x} \left[\frac{1}{2} x^T M x \right]_{t_f} = M x^*(t_f). \quad (5.80)$$

Therefore, we have

$$\Phi_{21}(t_f, t)x^*(t) + \Phi_{22}(t_f, t)p^*(t) = M\Phi_{11}(t_f, t)x^*(t) + M\Phi_{12}(t_f, t)p^*(t).$$

Solving for $p^*(t)$, we have

$$p^*(t) = \left[\Phi_{22}(t_f, t) - M\Phi_{12}(t_f, t) \right]^{-1} \left[M\Phi_{11}(t_f, t) - \Phi_{21}(t_f, t) \right] x^*(t),$$

which is rewritten as

$$p^*(t) = K(t)x^*(t),$$

$$\text{where } K(t) = \left[\Phi_{22}(t_f, t) - M\Phi_{12}(t_f, t) \right]^{-1} \left[M\Phi_{11}(t_f, t) - \Phi_{21}(t_f, t) \right].$$

Therefore, the optimal control

$$u^*(t) = -R^{-1}(t)B^T(t)p^*(t) = -R^{-1}(t)B^T(t)K(t)x^*(t) \quad (5.81)$$

is in a form of linear state feedback.

To determine $K(t)$, we note that

$$p^*(t) = K(t)x^*(t) \implies \dot{p}^*(t) = \dot{K}(t)x^*(t) + K(t)\dot{x}^*(t).$$

Plug in $\dot{p}^*(t) = -Q(t)x^*(t) - A^T(t)K(t)x^*(t)$ and $\dot{x}^*(t) = A(t)x^*(t) - B(t)R^{-1}(t)B^T(t)K(t)x^*(t)$ into the above equation, we have

$$-Q(t)x^*(t) - A^T(t)K(t)x^*(t) = \dot{K}(t)x^*(t) + K(t)A(t)x^*(t) - K(t)B(t)R^{-1}(t)B^T(t)K(t)x^*(t),$$

for all $t \in [t_0, t_f]$. Since x_0 can be arbitrary, so is $x^*(t)$. It follows that $K(t)$ must be a solution of the following matrix differential equation

$$\dot{K}(t) = -A^T(t)K(t) - K(t)A(t) - Q(t) + K(t)B(t)R^{-1}(t)B^T(t)K(t), \quad (5.82)$$

which is known as the *Riccati equation* with the boundary condition $K(t_f) = M$ (since $p^*(t_f) = K(t_f)x^*(t_f) = Mx^*(t_f)$). Solving the Riccati equation backwards with the boundary condition, we obtain $K(t)$, and therefore, an optimal control law

$$u^*(t) = -R^{-1}(t)B^T(t)K(t)x^*(t). \quad (5.83)$$

□

5.2.3 Pontryagin's Minimum Principle

So far, we have solved the optimal control problem by assuming that the admissible control signals are not bounded. This is critical in the calculus of variations since it enables $\delta u(t)$ to be arbitrary and it becomes necessary for the coefficient of $\delta u(t)$ in the expression of δJ to vanish. However, any physically realizable control generally has limitations. We, therefore, need to impose bounds on control $u(t)$ in our design. If the optimal control occurs on the boundary for a non-zero measure period of time (this is actually common as we will show by examples later in this section), we cannot assume that $\delta u(t)$ is arbitrary since $u(t) + \delta u(t)$ may become infeasible for $u(t)$ at the boundary of feasible controls. This poses an obstacle that prevents us from directly applying the necessary conditions obtained in the previous section from the calculus of variations to solve optimal control problems with bounded control constraints.

If we assume that all feasible control signals form a set $\mathcal{U}$ that is bounded, then $u^* \in \mathcal{U}$ is a relative minimum for a cost functional $J(u)$ if

$$\Delta J = J(u) - J(u^*) \geq 0 \quad (5.84)$$

holds for any admissible control $u \in \mathcal{U}$ that is sufficiently close to u^* . Like before, we denote $u = u^* + \delta u$, then

$$\Delta J(u^*, \delta u) = \delta J(u^*, \delta u) + o(\delta u), \quad (5.85)$$

where δJ is linear in δu . Since δu may not be arbitrary when u^* is on the boundary of $\mathcal{U}$, for which we only can choose the δu such that $u + \delta u \in \mathcal{U}$. So a necessary condition for u^* to minimize $J(u)$ is that

$$\delta J(u^*, \delta u) \geq 0, \quad (5.86)$$

if u^* lies on the boundary of $\mathcal{U}$ during any portion of the time interval $[t_0, t_f]$.

Note that this only changes the argument with respect to the coefficient of δu in the expansion of ΔJ . So we only consider the term related to δu in the expression ΔJ as the rest items are necessarily to be zero along optimal solutions

$$\Delta J(u^*, \delta u) = \int_{t_0}^{t_f} \left\{ \left[\frac{\partial \mathcal{H}}{\partial u}(x^*(t), u^*(t), p^*(t), t) \right] \delta u(t) \right\} dt + \text{higher-order terms}.$$

The integrand is the first-order approximation to the change in $\mathcal{H}$ caused by a change in u alone, that is

$$\left[\frac{\partial \mathcal{H}}{\partial u}(x^*(t), u^*(t), p^*(t), t) \right] \delta u(t) \approx \mathcal{H}(x^*(t), u^*(t) + \delta u(t), p^*(t), t) - \mathcal{H}(x^*(t), u^*(t), p^*(t), t).$$

Therefore

$$\Delta J(u^*, \delta u) = \int_{t_0}^{t_f} \left\{ \mathcal{H}(x^*(t), u^*(t) + \delta u(t), p^*(t), t) - \mathcal{H}(x^*(t), u^*(t), p^*(t), t) \right\} dt + \text{higher-order terms}.$$

For sufficiently small δu , the sign of $\Delta J(u^*, \delta u)$ is determined by the integral. It means that for $\Delta J(u^*, \delta u) \geq 0$ to hold true for all admissible δu , it is necessary that

$$\int_{t_0}^{t_f} \left\{ \mathcal{H}(x^*(t), u^*(t) + \delta u(t), p^*(t), t) - \mathcal{H}(x^*(t), u^*(t), p^*(t), t) \right\} dt \geq 0,$$

for which it is necessary that

$$\mathcal{H}(x^*(t), u^*(t) + \delta u(t), p^*(t), t) \geq \mathcal{H}(x^*(t), u^*(t), p^*(t), t),$$

for all admissible $\delta u(t)$ and for all $t \in [t_0, t_f]$.

In other words, $u^*(t)$ minimizes the Hamiltonian $\mathcal{H}(x^*(t), u(t), p^*(t), t)$ among all admissible $u(t)$, which can be written as the following minimization condition

$$u^*(t) = \arg \min_{u(t) \in \mathcal{U}} \mathcal{H}(x^*(t), u(t), p^*(t), t). \quad (5.87)$$

We will use this minimization to replace the condition $\frac{\partial \mathcal{H}}{\partial u}(x^*(t), u(t), p^*(t), t) = 0$ in the unbounded control case. All the other necessary conditions remain the same for the bounded control case.

Pontryagin's Minimum Principle

In summary, we list the necessary conditions for an optimal control problem

$$\dot{x}(t) = f(x(t), u(t), t), \quad x(t_0) = x_0, \quad (5.88)$$

with respect to the cost functional

$$J(u(\cdot)) = \phi(x(t_f), t_f) + \int_{t_0}^{t_f} L(x(\tau), u(\tau), \tau) d\tau. \quad (5.89)$$

It is assumed that the initial time and state is fixed with $x(t_0) = x_0$, while the final time t_f is free and we request that the final state satisfies the terminal constraint given by

$$\psi(x(t_f), t_f) = 0. \quad (5.90)$$

Define the Hamiltonian $\mathcal{H}(x, u, p, t) = L(x, u, t) + p^T(t)f(x, u, t)$, and it is necessary for optimal solutions of the above optimal control problem to satisfy

- State equation:

$$\dot{x}^*(t) = \frac{\partial \mathcal{H}^T}{\partial p}(x^*(t), u^*(t), p^*(t), t) = f(x^*(t), u^*(t), t).$$

- Co-state equation:

$$\dot{p}^*(t) = -\frac{\partial \mathcal{H}^T}{\partial x}(x^*(t), u^*(t), p^*(t), t).$$

- Minimization condition:

$$\mathcal{H}(x^*(t), u^*(t), p^*(t), t) = \min_{u(t) \in \mathcal{U}} \mathcal{H}(x^*(t), u(t), p^*(t), t),$$

or equivalently

$$u^*(t) = \arg \min_{u(t) \in \mathcal{U}} \mathcal{H}(x^*(t), u(t), p^*(t), t). \quad (5.91)$$

- Boundary conditions:

$$\begin{aligned} x^*(t_0) &= x_0, \\ \Psi(x^*(t_f), t_f) &= 0, \\ p^{*T}(t_f) &= \frac{\partial \phi}{\partial x}(x^*(t_f), t_f) - \lambda \frac{\partial \Psi}{\partial x}(x^*(t_f), t_f), \\ \mathcal{H}(x^*(t_f), u^*(t_f), p^*(t_f), t_f) &= \mu \frac{\partial \Psi}{\partial t}(x^*(t_f), t_f) - \frac{\partial \phi}{\partial t}(x^*(t_f), t_f), \end{aligned}$$

for some constants λ and μ .

This is known as Pontryagin's minimum principle (PMP), which is a centerpiece of the optimal control theory. PMP provides necessary conditions for the control input and the corresponding state trajectory to be optimal starting from a given initial state x_0 .

Some remarks are in order. First, if both the initial and final conditions are fixed, we simply replace the boundary condition with $x^*(t_0) = x_0$, $x^*(t_f) = x_f$. Secondly,

if t_f is fixed, but the final state x_f is free, we have the following boundary condition

$$p^*(t_f) = \frac{\partial \phi^T}{\partial x}(x^*(t_f), t_f), \quad (5.92)$$

which is called the transversality condition. Thirdly, if x_f is fixed, but t_f is free, the boundary conditions are replaced with $x(t_f) = x_f$ and

$$\mathcal{H}(x(t_f), u(t_f), p(t_f), t_f) = -\frac{\partial \phi}{\partial t}(x(t_f), t_f).$$

To gain further insight, we take a look at a simple example to show how to apply PMP to identify a candidate for optimal control.

Example 5.4 Consider a control system

$$\begin{cases} \dot{x}_1(t) = x_2(t), \\ \dot{x}_2(t) = -x_2(t) + u(t), \end{cases} \quad (5.93)$$

with initial conditions $x(t_0) = x_0$, and the cost functional to be minimized is

$$J(u) = \int_{t_0}^{t_f} \frac{1}{2} [x_1^2(t) + u^2(t)] dt, \quad (5.94)$$

where t_f is specified but $x(t_f)$ is free.

The Hamiltonian is

$$\mathcal{H}(x(t), u(t), p(t)) = \frac{1}{2}x_1^2(t) + \frac{1}{2}u^2(t) + p_1(t)x_2(t) - p_2(t)x_2(t) + p_2(t)u(t). \quad (5.95)$$

Note that the Hamiltonian does not explicitly depend on time t , so the argument t in $\mathcal{H}$ is ignored. Since we aim to use this example to illustrate the effects of bounds on the design of control, we will first consider the case when there are no bounds on control u . For this case, we have the following necessary conditions for optimal solutions:

- Co-state equation:

$$\begin{cases} \dot{p}_1^*(t) = -\frac{\partial \mathcal{H}}{\partial x_1} = 0, \\ \dot{p}_2^*(t) = -\frac{\partial \mathcal{H}}{\partial x_2} = -p_1^*(t) + p_2^*(t). \end{cases} \quad (5.96)$$

- Optimization condition:

$$0 = \frac{\partial \mathcal{H}}{\partial u} = u^*(t) + p_2^*(t). \quad (5.97)$$

- State equation:

$$\begin{cases} \dot{x}_1^*(t) = x_2^*(t), \\ \dot{x}_2^*(t) = -x_2^*(t) + u^*(t). \end{cases} \quad (5.98)$$

- Boundary condition: $x^*(t_0) = x_0$ and

$$p^*(t_f) = \frac{\partial \phi}{\partial x}(x^*(t_f), t_f) = 0. \quad (5.99)$$

From the optimization condition, we have $u^*(t) = -p_2^*(t)$. Plug this u^* into the state equation, we obtain

$$\begin{cases} \dot{x}_1^*(t) = x_2^*(t), \\ \dot{x}_2^*(t) = -x_2^*(t) - p_2^*(t). \end{cases} \quad (5.100)$$

Together with the co-state equation, we obtain a set of linear, first-order, homogeneous, ordinary differential equations, which can be solved like we did in Example 5.2.

Now, if we assume that the control is bounded with

$$-1 \leq u(t) \leq 1, \quad \forall t \in [t_0, t_f], \quad (5.101)$$

the minimization condition needs to be replaced with

$$u^*(t) = \arg \min_{u(t) \in \mathcal{U}} \mathcal{H}(x^*(t), u(t), p^*(t), t). \quad (5.102)$$

Note that

$$\mathcal{H}(x^*(t), u(t), p^*(t)) = \frac{1}{2}x_1^{*2}(t) + \frac{1}{2}u^2(t) + p_1^*(t)x_2(t) - p_2^*(t)x_2^*(t) + p_2^*(t)u(t). \quad (5.103)$$

The control $u(t)$ to minimize $\mathcal{H}(x^*(t), u(t), p^*(t))$ needs to be selected to satisfy the bounds. For this, we only need to consider the terms that involve $u(t)$, namely

$$G(u) = \frac{1}{2}u^2(t) + p_2^*(t)u(t). \quad (5.104)$$

For times when the optimal control is not saturated, we have

$$u^*(t) = -p_2^*(t). \quad (5.105)$$

This occurs when $|p_2^*(t)| \leq 1$. If, however, if $|p_2^*(t)| > 1$, then the control that minimizes $\mathcal{H}$ is given as

$$u^*(t) = \begin{cases} -1, & \text{if } p_2^*(t) > 1, \\ +1, & \text{if } p_2^*(t) < -1. \end{cases} \quad (5.106)$$

Thus, $u^*(t)$ is a saturation function of $p_2^*(t)$. This is clearly different from the unbounded control case. To determine $u^*(t)$ explicitly, one needs to solve the state and co-state equation with the form of $u^*(t)$ determined from the minimization condition (it will be a piecewise linear system for the bounded control case) and boundary conditions. Note that the state, co-state, and boundary conditions are the same for both bounded and unbounded cases. $\square$

5.3 Hybrid Optimal Control

Now we are ready to extend the Pontryagin's minimum principle for classical optimal control problems to the case of hybrid control systems. We will first formulate the hybrid optimal control problems to be considered here and then give necessary conditions for optimality as in the classical case. We also show how to solve hybrid optimal control problems through examples.

5.3.1 Hybrid Optimal Control Problem

Let's consider a hybrid control system $\mathbb{H}$ defined as

$$\dot{x}(t) = f_q(x(t), u(t), t), \quad x(t_0) = x_0, \quad q \in Q, \quad (5.107)$$

where $x(t) \in \mathbb{R}^n$ is the system state vector, $u \in \mathcal{U} \subseteq \mathbb{R}^m$ is the (piecewise) continuous control input vector, t is time, t_0 is the initial time, and x_0 is the initial state.

An admissible trajectory $x : [t_0, t_f] \rightarrow \mathbb{R}^n$ under a control $u : [t_0, t_f] \rightarrow \mathcal{U}$ if there exist time instants

$$t_0 < t_1 < \cdots < t_k < t_{k+1} = t_f, \quad (5.108)$$

and indices $q_0, q_1, \dots, q_k \in Q$, such that

$$\dot{x}(t) = f_{q_i}(x(t), u(t), t), \quad (5.109)$$

holds for $t \in (t_i, t_{i+1})$ (where $i = 0, 1, \dots, k$).

We first consider a basic hybrid optimal control problem without switching costs, state jumps or terminal cost

$$J(u(\cdot)) = \sum_{i=0}^k \int_{t_i}^{t_{i+1}} L_{q_i}(x(s), u(s), s) ds. \quad (5.110)$$

Here, we use t_i to denote the switching instant, and there are two types of switching:

- **Autonomous Switching:** For autonomous switching, a model transition from mode q_i to q_{i+1} occurs once the continuous state x intersects the switching surface $S_{q_i, q_{i+1}}$, namely $x(t_i^-)$ satisfies the switching condition

$$m_{q_i, q_{i+1}}(x(t_i^-), t_i^-) = 0, \quad (5.111)$$

under the dynamics $f_{q_i}(x, u)$ and then passes through it.

- **Controlled Switching:** For controlled switching, there are no constraints on the continuous states, while there is a discrete command triggering the mode transition q_i to q_{i+1} .

Corner Conditions

The main difference between the hybrid minimum principle and the classical Pontryagin's minimum principle lies on the switching conditions. This subsection aims to derive necessary conditions on controlled and autonomous switching instants. For simplicity, we do not consider any switching costs or state jumps during switching at this moment. Also, for simplicity, we consider a single switching from q_1 to q_2 at switching time t_1 . We can obtain the following augmented cost functional J_a

$$J_a(u) = \int_{t_0}^{t_1} g_{a_1}(x, \dot{x}, u, p, t) dt + \int_{t_1}^{t_f} g_{a_2}(x, \dot{x}, u, p, t) dt, \quad (5.112)$$

where

$$g_{a_1}(x, \dot{x}, u, p, t) = L_{q_1}(x, u, t) + p^T(t)[f_{q_1}(x, u, t) - \dot{x}], \quad (5.113)$$

$$g_{a_2}(x, \dot{x}, u, p, t) = L_{q_2}(x, u, t) + p^T(t)[f_{q_2}(x, u, t) - \dot{x}]. \quad (5.114)$$

If x^* is a minimizing extremal for J_a , then $x^*(t)$, $t_0 \leq t \leq t_1$, is an extremal for $J_1 = \int_{t_0}^{t_1} g_{a_1}(x, \dot{x}, u, p, t) dt$ and $x^*(t)$, $t_1 \leq t \leq t_f$, is an extremal for $J_2 = \int_{t_1}^{t_f} g_{a_2}(x, \dot{x}, u, p, t) dt$. To determine the variation of J_a , we introduce the variations δx , $\delta \dot{x}$, δu , δp , and δt_1 .

$$\begin{aligned} \Delta J_a &= \int_{t_0}^{t_1 + \delta t_1} g_{a_1}(x + \delta x, \dot{x} + \delta \dot{x}, u + \delta u, p + \delta p, t) dt - \int_{t_0}^{t_1} g_{a_1}(x, \dot{x}, u, p, t) dt \\ &\quad + \int_{t_1 + \delta t_1}^{t_f} g_{a_2}(x + \delta x, \dot{x} + \delta \dot{x}, u + \delta u, p + \delta p, t) dt - \int_{t_1}^{t_f} g_{a_2}(x, \dot{x}, u, p, t) dt \\ &= \int_{t_0}^{t_1} [g_{a_1}(x + \delta x, \dot{x} + \delta \dot{x}, u + \delta u, p + \delta p, t) - g_{a_1}(x, \dot{x}, u, p, t)] dt \\ &\quad + \int_{t_1}^{t_1 + \delta t_1} g_{a_1}(x + \delta x, \dot{x} + \delta \dot{x}, u + \delta u, p + \delta p, t) dt \end{aligned}$$

$$\begin{aligned}
& + \int_{t_1}^{t_f} \left[g_{a_2}(x + \delta x, \dot{x} + \delta \dot{x}, u + \delta u, p + \delta p, t) - g_{a_2}(x, \dot{x}, u, p, t) \right] dt \\
& - \int_{t_1}^{t_1 + \delta t_1} g_{a_2}(x, \dot{x}, u, p, t) dt.
\end{aligned}$$

By Taylor series expansion and ignoring higher-order terms, we obtain

$$\begin{aligned}
\delta J_a = & \int_{t_0}^{t_1} \left\{ \frac{\partial g_{a_1}}{\partial x}(x, \dot{x}, u, p, t) \delta x + \frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \delta \dot{x} \right. \\
& \left. + \frac{\partial g_{a_1}}{\partial u}(x, \dot{x}, u, p, t) \delta u + \frac{\partial g_{a_1}}{\partial p}(x, \dot{x}, u, p, t) \delta p \right\} dt \\
& + g_{a_1}(x, \dot{x}, u, p, t) \Big|_{t_1^-} \delta t_1 \\
& + \int_{t_1}^{t_f} \left\{ \frac{\partial g_{a_2}}{\partial x}(x, \dot{x}, u, p, t) \delta x + \frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \delta \dot{x} \right. \\
& \left. + \frac{\partial g_{a_2}}{\partial u}(x, \dot{x}, u, p, t) \delta u + \frac{\partial g_{a_2}}{\partial p}(x, \dot{x}, u, p, t) \delta p \right\} dt \\
& - g_{a_2}(x, \dot{x}, t) \Big|_{t_1^+} \delta t_1.
\end{aligned}$$

Here t_1^- and t_1^+ denote the time instants just before and after the switching. Integrate by parts with respect to $\delta \dot{x}$ and use $\delta \dot{x} = \frac{d}{dt} \delta x$ to obtain

$$\begin{aligned}
\delta J_a = & \left[\frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \delta x(t) \Big|_{t_1^-} + g_{a_1}(x, \dot{x}, u, p, t) \Big|_{t_1^-} \delta t_1 \\
& + \int_{t_0}^{t_1} \left\{ \left[\frac{\partial g_{a_1}}{\partial x}(x, \dot{x}, u, p, t) + \frac{d}{dt} \left(\frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right) \right] \delta x \right. \\
& \left. + \frac{\partial g_{a_1}}{\partial u}(x, \dot{x}, u, p, t) \delta u + \frac{\partial g_{a_1}}{\partial p}(x, \dot{x}, u, p, t) \delta p \right\} dt \\
& - \left[\frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \delta x(t) \Big|_{t_1^+} - g_{a_2}(x, \dot{x}, u, p, t) \Big|_{t_1^+} \delta t_1 \\
& + \int_{t_1}^{t_f} \left\{ \left[\frac{\partial g_{a_2}}{\partial x}(x, \dot{x}, u, p, t) + \frac{d}{dt} \left(\frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right) \right] \delta x \right. \\
& \left. + \frac{\partial g_{a_2}}{\partial u}(x, \dot{x}, u, p, t) \delta u + \frac{\partial g_{a_2}}{\partial p}(x, \dot{x}, u, p, t) \delta p \right\} dt.
\end{aligned}$$

Let's denote δx_1 as the difference $x(t_1 + \delta t_1) - x^*(t_1)$, which is different from $\delta x(t_1)$. We use the following linear approximations

$$\delta x(t_1^-) = \delta x_1 - \dot{x}^*(t_1^-)\delta t_1, \quad (5.115)$$

$$\delta x(t_1^+) = \delta x_1 - \dot{x}^*(t_1^+)\delta t_1. \quad (5.116)$$

Hence

$$\begin{aligned} \delta J_a = & \left[\frac{\partial g_{a1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \Big|_{t_1^-} \delta x_1 + \left[g_{a1}(x, \dot{x}, u, p, t) - \frac{\partial g_{a1}}{\partial \dot{x}}(x, \dot{x}, u, p, t)\dot{x} \right] \Big|_{t_1^-} \delta t_1 \\ & + \int_{t_0}^{t_1} \left\{ \left[\frac{\partial g_{a1}}{\partial x}(x, \dot{x}, u, p, t) + \frac{d}{dt} \left(\frac{\partial g_{a1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right) \right] \delta x \right. \\ & \quad \left. + \frac{\partial g_{a1}}{\partial u}(x, \dot{x}, u, p, t)\delta u + \frac{\partial g_{a1}}{\partial p}(x, \dot{x}, u, p, t)\delta p \right\} dt \\ & - \left[\frac{\partial g_{a2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \Big|_{t_1^+} \delta x_1 - \left[g_{a2}(x, \dot{x}, u, p, t) - \frac{\partial g_{a2}}{\partial \dot{x}}(x, \dot{x}, u, p, t)\dot{x} \right] \Big|_{t_1^+} \delta t_1 \\ & + \int_{t_1}^{t_f} \left\{ \left[\frac{\partial g_{a2}}{\partial x}(x, \dot{x}, u, p, t) + \frac{d}{dt} \left(\frac{\partial g_{a2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right) \right] \delta x \right. \\ & \quad \left. + \frac{\partial g_{a2}}{\partial u}(x, \dot{x}, u, p, t)\delta u + \frac{\partial g_{a2}}{\partial p}(x, \dot{x}, u, p, t)\delta p \right\} dt. \end{aligned}$$

For optimal solutions, it is necessary to have the coefficients for the variations δx , $\delta \dot{x}$, δu , δp , δt_1 , and δx_1 to be zero.

Co-State Equations: First of all, consider the coefficient of δx , we need the Euler equations

$$0 = \frac{\partial g_{a1}}{\partial x}(x, \dot{x}, u, p, t) - \frac{d}{dt} \left(\frac{\partial g_{a1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right), \quad t \in (t_0, t_1), \quad (5.117)$$

and

$$0 = \frac{\partial g_{a2}}{\partial x}(x, \dot{x}, u, p, t) + \frac{d}{dt} \left(\frac{\partial g_{a2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right), \quad t \in (t_1, t_f), \quad (5.118)$$

holds for almost all $t \in [t_0, t_f]$ to make the integral zero.

Recall that

$$\begin{aligned} g_{a1}(x, \dot{x}, u, p, t) &= L_{q_1}(x, u, t) + p^T(t)[f_{q_1}(x, u, t) - \dot{x}], \\ g_{a2}(x, \dot{x}, u, p, t) &= L_{q_2}(x, u, t) + p^T(t)[f_{q_2}(x, u, t) - \dot{x}]. \end{aligned}$$

Then

$$\frac{\partial g_{a1}}{\partial x}(x, \dot{x}, u, p, t) = \frac{\partial L_{q_1}}{\partial x}(x, u, t) + p^T \frac{\partial f_{q_1}}{\partial x}(x, u, t), \quad (5.119)$$

and

$$\frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) = -p^T(t). \quad (5.120)$$

Therefore, we can rewrite

$$0 = \frac{\partial g_{a_1}}{\partial x}(x, \dot{x}, u, p, t) + \frac{d}{dt} \left(\frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right), \quad t \in (t_0, t_1), \quad (5.121)$$

as

$$0 = \frac{\partial L_{q_1}}{\partial x}(x, u, t) + p^T \frac{\partial f_{q_1}}{\partial x}(x, u, t) + \dot{p}^T, \quad t \in (t_0, t_1). \quad (5.122)$$

If we define a Hamiltonian as

$$\mathcal{H}_{q_i}(x, u, p, t) = L_{q_i}(x, u, t) + p^T(t) f_{q_i}(x, u, t), \quad (5.123)$$

then the above equation can be written as

$$\dot{p}(t) = -\frac{\partial}{\partial x} \mathcal{H}_{q_1}^T(x, u, p, t), \quad (5.124)$$

for $t \in (t_0, t_1)$, which is the co-state equation. Similarly, we obtain that the co-state equation

$$\dot{p}(t) = -\frac{\partial}{\partial x} \mathcal{H}_{q_2}^T(x, u, p, t), \quad (5.125)$$

needs to hold true for $t \in (t_1, t_f)$.

State Equations: In addition, we need

$$\frac{\partial g_{a_1}}{\partial p}(x, \dot{x}, u, p, t) = 0$$

for $t \in (t_0, t_1)$, and

$$\frac{\partial g_{a_2}}{\partial p}(x, \dot{x}, u, p, t) = 0$$

for $t \in (t_1, t_f)$. Note that

$$\frac{\partial g_{a_1}}{\partial p}(x, \dot{x}, u, p, t) = (f_{q_1}(x, u, t) - \dot{x})^T, \quad (5.126)$$

and

$$\frac{\partial g_{a_2}}{\partial p}(x, \dot{x}, u, p, t) = (f_{q_2}(x, u, t) - \dot{x})^T. \quad (5.127)$$

Hence, we obtain that optimal solutions should satisfy the state equations.

Minimization Conditions: The last two items inside the integral are

$$\int_{t_0}^{t_1} \frac{\partial g_{a_1}}{\partial u}(x, \dot{x}, u, p, t) \delta u dt, \quad (5.128)$$

and

$$\int_{t_1}^{t_f} \frac{\partial g_{a_2}}{\partial u}(x, \dot{x}, u, p, t) \delta u dt. \quad (5.129)$$

If control is unbounded, namely δu can be arbitrary, then we need

$$\frac{\partial g_{a_1}}{\partial u}(x, \dot{x}, u, p, t) = 0, \quad (5.130)$$

for $t \in (t_0, t_1)$ and

$$\frac{\partial g_{a_2}}{\partial u}(x, \dot{x}, u, p, t) = 0, \quad (5.131)$$

for $t \in (t_1, t_f)$.

Note that

$$\frac{\partial g_{a_1}}{\partial u}(x, \dot{x}, u, p, t) = \frac{\partial L_{q_1}}{\partial u}(x, u, t) + p^T \frac{\partial f_{q_1}}{\partial u}(x, u, t) = \frac{\partial}{\partial u} \mathcal{H}_{q_1}(x, u, p, t). \quad (5.132)$$

Hence, we need

$$\frac{\partial}{\partial u} \mathcal{H}_{q_1}(x, u, p, t) = 0, \quad (5.133)$$

for $t \in (t_0, t_1)$ and similarly

$$\frac{\partial}{\partial u} \mathcal{H}_{q_2}(x, u, p, t) = 0, \quad (5.134)$$

for $t \in (t_1, t_f)$, which are known as minimization conditions or optimization conditions.

For the case when the control is bounded, following a similar argument in the previous section, we obtain the following minimization conditions for optimal solutions

$$\mathcal{H}_{q_1}(x^*, u^*, p^*, t) \leq \mathcal{H}_{q_1}(x^*, u, p^*, t) \quad (5.135)$$

for $t \in (t_0, t_1)$ and

$$\mathcal{H}_{q_2}(x^*, u^*, p^*, t) \leq \mathcal{H}_{q_2}(x^*, u, p^*, t) \quad (5.136)$$

for $t \in (t_1, t_f)$.

Boundary Conditions and Transversality Conditions

Now, we turn to consider the items outside of the integral, which are

$$\left\{ \left[\frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \Big|_{t_1^-} - \left[\frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \Big|_{t_1^+} \right\} \delta x_1 + \left\{ \left[g_{a_1}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \dot{x} \right] \Big|_{t_1^-} - \left[g_{a_2}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \dot{x} \right] \Big|_{t_1^+} \right\} \delta t_1.$$

Two cases will be considered.

Controlled Switching: If t_1 and x_1 are unrelated, which is the case for controlled switching as the switching time does not dependent on the continuous state value. For this case, we have

$$\frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \Big|_{t_1^-} = \frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \Big|_{t_1^+},$$

and

$$\left[g_{a_1}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \dot{x}(t) \right] \Big|_{t_1^-} = \left[g_{a_2}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \dot{x}(t) \right] \Big|_{t_1^+}.$$

Note that

$$g_{a_i}(x, \dot{x}, u, p, t) = L_{q_i}(x, u, t) + p^T(t)[f_{q_i}(x, u, t) - \dot{x}], \quad (5.137)$$

so we have

$$\frac{\partial g_{a_i}}{\partial \dot{x}} = -p^T(t).$$

for $i = 1$ or 2 . Hence, the condition for the coefficient of δx_1 to be zero is equivalent to request

$$p^*(t_1^-) = p^*(t_1^+). \quad (5.138)$$

Now, we consider the coefficient of δt_1 . Note that

$$\begin{aligned} & g_{a_i}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_i}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \dot{x}(t) \\ &= L_{q_i}(x, u, t) + p^T(f_{q_i}(x, u, t) - \dot{x}) + p^T \dot{x} \\ &= L_{q_i}(x, u, t) + p^T(f_{q_i}(x, u, t)) \\ &= \mathcal{H}_{q_i}(x, u, p, t). \end{aligned}$$

Hence, the requirement of

$$\left[g_{a_1}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \dot{x} \right] \Big|_{t_1^-} = \left[g_{a_2}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \dot{x} \right] \Big|_{t_1^+}$$

implies that

$$\mathcal{H}_{q_1}(x(t), u(t), p(t), t) \Big|_{t_1^-} = \mathcal{H}_{q_2}(x(t), u(t), p(t), t) \Big|_{t_1^+}. \quad (5.139)$$

Autonomous Switching: For autonomous switching, the switching time t_1 and the state $x(t_1)$ are related, and a typical case is when the state intersects a switching surface $m_{q_1, q_2}(x, t) = 0$, which relates t_1 with $x(t_1)$, in the sense that $m_{q_1, q_2}(x(t_1), t_1) = 0$ must hold at the switching instant t_1 . For this case, δx_1 and δt_1 must be contained in the tangent plane of $m_{q_1, q_2}(x, t) = 0$ at $(x^*(t_1^-), t_1)$. This implies that the vector $[\delta x_1, \delta t_1]^T$ needs to be orthogonal to the gradient

$$\frac{\partial m_{q_1, q_2}}{\partial x}(x^*(t_1^-), t_1) \delta x_1 + \frac{\partial m_{q_1, q_2}}{\partial t}(x^*(t_1), t_1) \delta t_1 = 0. \quad (5.140)$$

Hence, the coefficients of δx_1 and δt_1 in the expression of δJ_a should be able to be written as some linear combination of the gradient vector $[\frac{\partial m_{q_1, q_2}}{\partial x}(x^*(t_1^-), t_1), \frac{\partial m_{q_1, q_2}}{\partial t}(x^*(t_1^-), t_1)]$, that is

$$\frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \Big|_{t_1^-} - \frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \Big|_{t_1^+} = \gamma \frac{\partial m_{q_1, q_2}}{\partial x}(x^*(t_1^-), t_1) \quad (5.141)$$

for some constant γ , and

$$\begin{aligned} & \left[g_{a_1}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \dot{x} \right] \Big|_{t_1^-} - \left[g_{a_2}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \dot{x} \right] \Big|_{t_1^+} \\ &= \mu \frac{\partial m_{q_1, q_2}}{\partial t}(x^*(t_1^-), t_1) \end{aligned}$$

for some constant μ .

The first equation implies that

$$p^T(t_1^-) - p^T(t_1^+) = \gamma \frac{\partial m_{q_1, q_2}}{\partial x}(x(t_1^-), t_1) \quad (5.142)$$

holds true at the switching instant t_1 for some constant γ .

The second equation implies that

$$\mathcal{H}_{q_1}(x(t_1^-), u(t_1^-), p(t_1^-), t_1) - \mathcal{H}_{q_2}(x(t_1^+), u(t_1^+), p(t_1^+), t_1) = \mu \frac{\partial m_{q_1, q_2}}{\partial t}(x^*(t_1^-), t_1) \quad (5.143)$$

holds true at the switching instant t_1 for some constant μ .

Note that if the switching surface does not explicitly depend on time, then $\frac{\partial m_{q_1, q_2}}{\partial t}(x, t) = 0$, and we have

$$\mathcal{H}_{q_1}(x(t_1^-), u(t_1^-), p(t_1^-), t_1) = \mathcal{H}_{q_2}(x(t_1^+), u(t_1^+), p(t_1^+), t_1) \quad (5.144)$$

at the switching instant.

5.3.2 Basic Hybrid Minimum Principle

In summary, we have the following extension of the classical minimum principle to the case of hybrid systems: Consider the hybrid control system $\mathbb{H}$ defined in Eq. (5.107) with the cost functional (5.110). Define the family of system Hamiltonians associated with each discrete state q_i as

$$\mathcal{H}_{q_i}(x, p, u, t) = L_{q_i}(x, u, t) + p^T f_{q_i}(x, u, t), \quad (5.145)$$

for $x, p \in \mathbb{R}^n$, $u \in U$ and $q_i \in Q$. Then there exists an adjoint process p^* for the optimal hybrid control u^* and the corresponding optimal hybrid state trajectory $(q^*(t), x^*(t))$ such that

- (1). the continuous state x^* and the adjoint variable p^* satisfy the *Hamilton's canonical* equations

$$\dot{x}^*(t) = \frac{\partial \mathcal{H}_{q^*}^T}{\partial p}(x^*(t), p^*(t), u^*(t), t), \quad (5.146)$$

$$\dot{p}^*(t) = -\frac{\partial \mathcal{H}_{q^*}^T}{\partial x}(x^*(t), p^*(t), u^*(t), t), \quad (5.147)$$

almost everywhere for $\forall t \in [t_0, t_f]$, with boundary conditions

$$x^*(t_0) = x_0,$$

$$x^*(t_f) = x_f;$$

- (2). for an autonomous switching at t_i , the adjoint p^* satisfies the following *transversality* condition

$$p^T(t_i^-) = p^T(t_i^+) + \gamma_i \frac{\partial m_{q_i, q_{i+1}}(x^*(t_i^-), t_i)}{\partial x}, \quad (5.148)$$

where $\gamma_i \in \mathbb{R}$ and $m_{q_i, q_{i+1}}(x^*(t_i), t_i) = 0$ stands for the switching surface for the discrete transition from mode q_i to q_{i+1} . Meanwhile, the Hamiltonian satisfies

$$\mathcal{H}_{q_i^*}(t_i) = \mathcal{H}_{q_{i+1}^*}(t_i) + \mu_i \frac{\partial m_{q_i, q_{i+1}}}{\partial t}(x^*(t_i^-), t_i), \quad (5.149)$$

- where $\mu_i \in \mathbb{R}$, $\mathcal{H}_{q_i^*}(t_i) = \mathcal{H}_{q_i^*}(x^*, p^*, u^*, t)|_{t_i^-}$ and $\mathcal{H}_{q_{i+1}^*}(t_i) = \mathcal{H}_{q_{i+1}^*}(x^*, p^*, u^*, t)|_{t_i^+}$;
- (3). if t_i is a controlled switching time instant, then the adjoint p^* satisfies the following continuity condition, i.e., $p(t_i^-) = p(t_i^+)$. So is the Hamiltonian, namely $\mathcal{H}_{q_i^*}(t_i) = \mathcal{H}_{q_{i+1}^*}(t_i)$;
 - (4). since there is no state jumps upon switching, hence the continuity condition holds for state trajectory, i.e., $x(t_i^-) = x(t_i^+)$ for all t_i ;
 - (5). the Hamiltonian is minimized with respect to the optimal control, namely

$$\mathcal{H}_{q^*}(x^*, p^*, u^*, t) \leq \mathcal{H}_{q^*}(x^*, p^*, u, t) \quad (5.150)$$

holds almost everywhere for $t \in [t_i, t_{i+1})$ and for all $u \in \mathcal{U}$.

Just as an illustration of the hybrid minimum principle, we apply a traditional calculus of variations problem and recover the classical Weierstrass–Erdmann corner conditions.

Example 5.5 In the calculus of variations, corner conditions characterize the necessary conditions that an optimal curve should satisfy if we allow discontinuities in the derivative of the curve. The problem is to find a continuous curve $x(t)$ to minimize the cost functional

$$J(x) = \int_{t_0}^{t_f} g(x, \dot{x}, t) dt. \quad (5.151)$$

It is assumed that the initial and ending point of the curve are fixed, i.e., $x(t_0) = x_0$ and $x(t_f) = x_f$. Note that the derivative $\dot{x}$ may be discontinuous, and the time instant that $\dot{x}(t)$ is discontinuous is called a corner point. The curve optimization problem can be translated into an optimal control problem

$$\dot{x} = u \quad (5.152)$$

and the cost functional can be written as

$$J(x, u) = \int_{t_0}^{t_f} g(x, u, t) dt. \quad (5.153)$$

The Hamiltonian is

$$\mathcal{H}(x, u, p, t) = g(x, u, t) + p^T u. \quad (5.154)$$

Minimization of $\mathcal{H}$ with respect to u implies

$$\frac{\partial \mathcal{H}}{\partial u} = \frac{\partial g}{\partial u}(x, u, t) + p^T = 0. \quad (5.155)$$

Hence, we have

$$p = -\frac{\partial g^T}{\partial u}(x, u, t) = -\frac{\partial g^T}{\partial \dot{x}}(x, u, t). \quad (5.156)$$

The co-state equation

$$\dot{p} = -\frac{\partial \mathcal{H}^T}{\partial x} = -\frac{\partial g^T}{\partial x}(x, u, t). \quad (5.157)$$

Plug $p = -\frac{\partial g^T}{\partial \dot{x}}(x, u, t)$ into the co-state equation, we obtain

$$\frac{d}{dt} \left\{ -\frac{\partial g^T}{\partial \dot{x}}(x, \dot{x}, t) \right\} = -\frac{\partial g^T}{\partial x}(x, \dot{x}, t), \quad (5.158)$$

which implies that the Euler equation

$$\frac{d}{dt} \left\{ \frac{\partial g}{\partial \dot{x}}(x, \dot{x}, t) \right\} - \frac{\partial g}{\partial x}(x, \dot{x}, t) = 0 \quad (5.159)$$

holds for all $t \in [t_0, t_f]$.

Furthermore, at a corner point $t_i \in (t_0, t_f)$, we need to have $p(t_i^-) = p(t_i^+)$ and $\mathcal{H}|_{t_i^-} = \mathcal{H}|_{t_i^+}$ if there is no constraint at the corner point. These imply

$$\frac{\partial g}{\partial \dot{x}}(x(t_i^-), \dot{x}(t_i^-), t_i) = \frac{\partial g}{\partial \dot{x}}(x(t_i^+), \dot{x}(t_i^+), t_i), \quad (5.160)$$

and

$$\begin{aligned} & g(x(t_i^-), \dot{x}(t_i^-), t_i) - \left[\frac{\partial g}{\partial \dot{x}}(x(t_i^-), \dot{x}(t_i^-), t_i) \right] \dot{x}(t_i^-) \\ &= g(x(t_i^+), \dot{x}(t_i^+), t_i) - \left[\frac{\partial g}{\partial \dot{x}}(x(t_i^+), \dot{x}(t_i^+), t_i) \right] \dot{x}(t_i^+). \end{aligned} \quad (5.161)$$

These two equations are known as *Weierstrass–Erdmann corner conditions*. $\square$

5.3.3 Extensions of the Hybrid Minimum Principle

Next, we consider several extensions of the basic hybrid minimum principle by considering a free terminal time with a terminal constraint and/or terminal cost.

Free t_f But With Terminal Constraint

If the final time t_f and x_f are not fixed, we also need to consider the variation δt_f and δx_f . Basically, we compute

$$\begin{aligned}
 \Delta J_a &= \int_{t_0}^{t_1+\delta t_1} g_{a_1}(x + \delta x, \dot{x} + \delta \dot{x}, u + \delta u, p + \delta p, t) dt - \int_{t_0}^{t_1} g_{a_1}(x, \dot{x}, u, p, t) dt \\
 &\quad + \int_{t_1+\delta t_1}^{t_f+\delta t_f} g_{a_2}(x + \delta x, \dot{x} + \delta \dot{x}, u + \delta u, p + \delta p, t) dt - \int_{t_1}^{t_f} g_{a_2}(x, \dot{x}, u, p, t) dt \\
 &= \int_{t_0}^{t_1} [g_{a_1}(x + \delta x, \dot{x} + \delta \dot{x}, u + \delta u, p + \delta p, t) - g_{a_1}(x, \dot{x}, u, p, t)] dt \\
 &\quad + \int_{t_1}^{t_1+\delta t_1} g_{a_1}(x + \delta x, \dot{x} + \delta \dot{x}, u + \delta u, p + \delta p, t) dt \\
 &\quad + \int_{t_1}^{t_f} [g_{a_2}(x + \delta x, \dot{x} + \delta \dot{x}, u + \delta u, p + \delta p, t) - g_{a_2}(x, \dot{x}, u, p, t)] dt \\
 &\quad - \int_{t_1}^{t_1+\delta t_1} g_{a_2}(x, \dot{x}, u, p, t) dt + \int_{t_f}^{t_f+\delta t_f} g_{a_2}(x + \delta x, \dot{x} + \delta \dot{x}, u + \delta u, p + \delta p, t) dt.
 \end{aligned}$$

By taking Taylor series expansion and ignoring higher-order items, we obtain

$$\begin{aligned}
 \delta J_a &= \left[\frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \delta x(t) \Big|_{t_1^-} + g_{a_1}(x, \dot{x}, u, p, t) \Big|_{t_1^-} \delta t_1 \\
 &\quad + \int_{t_0}^{t_1} \left\{ \left[\frac{\partial g_{a_1}}{\partial x}(x, \dot{x}, u, p, t) + \frac{d}{dt} \left(\frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right) \right] \delta x \right. \\
 &\quad \quad \left. + \frac{\partial g_{a_1}}{\partial u}(x, \dot{x}, u, p, t) \delta u + \frac{\partial g_{a_1}}{\partial p}(x, \dot{x}, u, p, t) \delta p \right\} dt \\
 &\quad + \left[\frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \delta x(t) \Big|_{t_1^+} - g_{a_2}(x, \dot{x}, u, p, t) \Big|_{t_1^+} \delta t_1 \\
 &\quad + \int_{t_1}^{t_f} \left\{ \left[\frac{\partial g_{a_2}}{\partial x}(x, \dot{x}, u, p, t) + \frac{d}{dt} \left(\frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right) \right] \delta x \right. \\
 &\quad \quad \left. + \frac{\partial g_{a_2}}{\partial u}(x, \dot{x}, u, p, t) \delta u + \frac{\partial g_{a_2}}{\partial p}(x, \dot{x}, u, p, t) \delta p \right\} dt \\
 &\quad + g_{a_2}(x, \dot{x}, u, p, t) \Big|_{t_f} \delta t_f.
 \end{aligned}$$

To avoid too much repetition, we only consider terms in δJ_a involving δt_f and δx_f , which are

$$\left[\frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \Big|_{t_f} \delta x(t_f) + g_{a_2}(x, \dot{x}, u, p, t) \Big|_{t_f} \delta t_f. \quad (5.162)$$

Note that $\delta x(t_f) \approx \delta x_f - \dot{x}^*(t_f)\delta t_f$, so it becomes

$$\left[\frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \Big|_{t_f} \delta x_f + \left\{ g_{a_2}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \dot{x}(t) \right\} \Big|_{t_f} \delta t_f. \quad (5.163)$$

If we have terminal constraints in the sense that $x(t_f)$ should be on the surface defined by $\Psi(x, t) = 0$ at t_f , i.e., $\Psi(x(t_f), t_f) = 0$, then the combination of δx_f and δt_f must lie on the tangent plane of the surface $\Psi(x, t) = 0$ at the point $(x^*(t_f), t_f)$. It means that there exist constants λ_f and μ_f such that

$$\left[\frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \Big|_{t_f} = \lambda_f \frac{\partial \Psi}{\partial x}(x^*(t_f), t_f), \quad (5.164)$$

$$\left\{ g_{a_2}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \dot{x}(t) \right\} \Big|_{t_f} = \mu_f \frac{\partial \Psi}{\partial t}(x^*(t_f), t_f). \quad (5.165)$$

Recall that $\frac{\partial g_{a_i}}{\partial \dot{x}} = -p^T(t)$ and $g_{a_i} - \frac{\partial g_{a_i}}{\partial \dot{x}} \dot{x} = \mathcal{H}_{q_i}(x, u, p, t)$. Hence, we have the following boundary conditions

$$p(t_f) = -\lambda_f \frac{\partial \Psi}{\partial x}(x^*(t_f), t_f), \quad (5.166)$$

$$\mathcal{H}_{q_2}(x, u, p, t_f) = \mu_f \frac{\partial \Psi}{\partial t}(x^*(t_f), t_f), \quad (5.167)$$

which holds if the terminal constraint $\Psi(x(t_f), t_f) = 0$ needs to be satisfied with free t_f .

With Terminal Cost

If, in addition, we also consider a final cost $\phi(x(t_f), t_f)$ into the cost functional, that is

$$J(u(\cdot)) = \phi(x(t_f), t_f) + \sum_{i=0}^k \int_{t_i}^{t_{i+1}} L_{q_i}(x(s), u(s), s) ds. \quad (5.168)$$

Then, the contributions from ϕ to δJ_a include

$$\delta \phi = \frac{\partial \phi}{\partial x}(x(t_f), t_f) \delta x_f + \frac{\partial \phi}{\partial t}(x(t_f), t_f) \delta t_f, \quad (5.169)$$

which, together with the other items due to δx_f and δt_f , gives

$$\left[\frac{\partial \phi}{\partial x}(x, t) + \frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \Big|_{t_f} \delta x_f + \left\{ \frac{\partial \phi}{\partial t}(x, t) + g_{a_2}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \dot{x}(t) \right\} \Big|_{t_f} \delta t_f.$$

If we have terminal constraint in the sense that $x(t_f)$ should be on the surface defined by $\Psi(x, t) = 0$ at t_f , i.e., $\Psi(x(t_f), t_f) = 0$, then the combination of δx_f and δt_f must lie on the tangent plane of the surface $\Psi(x, t) = 0$ at the point $(x^*(t_f), t_f)$. It means that there exist constants λ_f and μ_f such that

$$\begin{aligned} \left[\frac{\partial \phi}{\partial x}(x, t) + \frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \Big|_{t_f} &= \lambda_f \frac{\partial \Psi}{\partial x}(x^*(t_f), t_f), \\ \left\{ \frac{\partial \phi}{\partial t}(x, t) + g_{a_2}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \dot{x}(t) \right\} \Big|_{t_f} &= \mu_f \frac{\partial \Psi}{\partial t}(x^*(t_f), t_f). \end{aligned}$$

Recall that $\frac{\partial g_{a_i}}{\partial \dot{x}} = -p^T(t)$ and $g_{a_i} - \frac{\partial g_{a_i}}{\partial \dot{x}} \dot{x} = \mathcal{H}_{q_i}(x, u, p, t)$. Hence, we have the following boundary conditions

$$p(t_f) = \frac{\partial \phi}{\partial x}(x(t_f), t_f) - \lambda_f \frac{\partial \Psi}{\partial x}(x^*(t_f), t_f), \quad (5.170)$$

$$\mathcal{H}_{q_2}(x, u, p, t_f) = \mu_f \frac{\partial \Psi}{\partial t}(x^*(t_f), t_f) - \frac{\partial \phi}{\partial t}(x(t_f), t_f), \quad (5.171)$$

hold if the terminal constraint $\Psi(x(t_f), t_f) = 0$ needs to be satisfied with free t_f .

Hybrid Minimum Principle

We now present an extended version of the *hybrid minimum principle* (HMP) for the hybrid control system $\mathbb{H}$ defined in Eq. (5.107) with the cost functional

$$J(u(\cdot)) = \phi(x(t_f), t_f) + \sum_{i=0}^k \int_{t_i}^{t_{i+1}} L_{q_i}(x(s), u(s), s) ds \quad (5.172)$$

and a terminal state constraint $\Psi(x, t) = 0$.

Define the family of system Hamiltonians associated with each discrete state q_i as

$$\mathcal{H}_{q_i}(x, p, u, t) = L_{q_i}(x, u, t) + p^T f_{q_i}(x, u, t) \quad (5.173)$$

for $x, p \in \mathbb{R}^n$, $u \in U$ and $q_i \in Q$. Then there exists an adjoint process p^* for the optimal hybrid control u^* and the corresponding optimal hybrid state trajectory $(q^*(t), x^*(t))$ such that

- (1). the continuous state x^* and the adjoint variable p^* satisfy the *Hamilton's canonical equations*

$$\dot{x}^*(t) = \frac{\partial \mathcal{H}_{q^*}^T}{\partial p}(x^*(t), p^*(t), u^*(t), t), \quad (5.174)$$

$$\dot{p}^*(t) = -\frac{\partial \mathcal{H}_{q^*}^T}{\partial x}(x^*(t), p^*(t), u^*(t), t), \quad (5.175)$$

hold almost everywhere for $\forall t \in [t_0, t_f]$, with boundary conditions

$$x^*(t_0) = x_0,$$

and

$$p(t_f) = \frac{\partial \phi(x^*(t_f), t_f)}{\partial x} - \lambda_f \frac{\partial \Psi(x^*(t_f), t_f)}{\partial x}, \quad (5.176)$$

for some real constant λ_f ;

- (2). for an autonomous switching at t_i , the adjoint p^* satisfies the following *transversality* condition

$$p^T(t_i^-) = p^T(t_i^+) + \gamma_i \frac{\partial m_{q_i, q_{i+1}}(x^*(t_i^-), t_i)}{\partial x}, \quad (5.177)$$

where $\gamma_i \in \mathbb{R}$ and $m_{q_i, q_{i+1}}(x^*(t_i), t_i) = 0$ stands for the switching surface for the discrete transition from mode q_i to q_{i+1} . Meanwhile, the Hamiltonian satisfies

$$\mathcal{H}_{q_i^*}(t_i) = \mathcal{H}_{q_{i+1}^*}(t_i) + \mu_i \frac{\partial m_{q_i, q_{i+1}}(x^*(t_i^-), t_i)}{\partial t}, \quad (5.178)$$

where $\mu_i \in \mathbb{R}$, $\mathcal{H}_{q_i^*}(t_i) = \mathcal{H}_{q_i^*}(x^*, p^*, u^*, t)|_{t_i^-}$ and $\mathcal{H}_{q_{i+1}^*}(t_i) = \mathcal{H}_{q_{i+1}^*}(x^*, p^*, u^*, t)|_{t_i^+}$;

- (3). if t_i is a controlled switching time instant, then the adjoint p^* satisfies the following continuity condition, i.e., $p(t_i^-) = p(t_i^+)$. So is the Hamiltonian, namely $\mathcal{H}_{q_i^*}(t_i) = \mathcal{H}_{q_{i+1}^*}(t_i)$;
- (4). since there is no state jumps upon switching, hence the continuity condition holds for state trajectory, i.e., $x(t_i^-) = x(t_i^+)$ for all t_i ;
- (5). the Hamiltonian is minimized with respect to the optimal control, namely

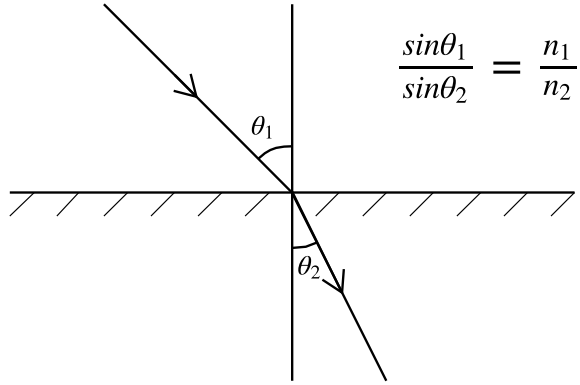
$$\mathcal{H}_{q^*}(x^*, p^*, u^*, t) \leq \mathcal{H}_{q^*}(x^*, p^*, u, t) \quad (5.179)$$

holds almost everywhere for all $t \in [t_i, t_{i+1})$ and for all $u \in \mathcal{U}$.

The following example from [11] illustrates the above hybrid minimum principle.

Example 5.6 We consider Snell's law that could be derived from the HMP and Fermat's Principle. It is known from the statement of Fermat's Principle that "The actual path between two points taken by a beam of light is the one which is traversed in the least time." Under the assumption that the light ray travels in a straight line in each medium, we could derive the Snell's law according to trigonometric relationships and Fermat's Principle as follows:

Suppose that a ray of light goes through a medium with its velocity v_1 to another medium with velocity v_2 . Here, the objective is to minimize the total time

Fig. 5.3 Snell's law

of travel (Fermat's Principle), as shown in Fig. 5.3. Let the initial time t_0 and the final time t_f . The time that takes to meet the boundary (switching manifold) $M(x, y) \in \mathbb{R}^2 : m(t, x, y) = y = 0$ is t_s , $0 < t_s < t_f$. The subscripts 1 and 2 refer to media 1 and 2, respectively. Along its trajectory, at each time let the angles $\theta_1(t)$ and $\theta_2(t)$ with the normal to M . Let $-\frac{\pi}{2} < \theta_1(t) < \frac{\pi}{2}$, $0 < t < t_s$, and $-\frac{\pi}{2} < \theta_2(t) < \frac{\pi}{2}$, $t_s < t < t_f$. Thus, it can be written into the following hybrid optimal control problem

$$\begin{cases} \dot{x} = v_1 \sin \theta_1(t), \\ \dot{y} = v_1 \cos \theta_1(t), \end{cases} \quad (5.180)$$

for $0 < t < t_s$ and

$$\begin{cases} \dot{x} = v_2 \sin \theta_2(t), \\ \dot{y} = v_2 \cos \theta_2(t), \end{cases} \quad (5.181)$$

for $t_s < t < t_f$.

The objective is to minimize the total travel time

$$J(\theta_1, \theta_2) = \int_0^{t_s} d\tau + \int_{t_s}^{t_f} d\tau.$$

The Hamiltonian for each medium is given by

$$\mathcal{H}_i = 1 + \lambda_i v_i \sin \theta_i(t) + \mu_i v_i \cos \theta_i(t), \quad i = 1, 2.$$

Since θ_1 and θ_2 take values in an open set $(-\frac{\pi}{2}, \frac{\pi}{2})$, if θ_i^* is optimal, then we have

$$\left. \frac{\partial \mathcal{H}_i}{\partial \theta_i} \right|_{\theta_i = \theta_i^*} = \lambda_i v_i \cos \theta_i^* - \mu_i v_i \sin \theta_i^* = 0,$$

which yields $\tan \theta_i^* = \frac{\lambda_i}{\mu_i}$.

In the following, we omit the superscript $*$ on θ_i , since all θ_i will be optimal. Along the optimal trajectory, we have

$$\dot{\lambda}_i = -\frac{\partial \mathcal{H}_i}{\partial x_i} = 0 \quad \text{and} \quad \dot{\mu}_i = -\frac{\partial \mathcal{H}_i}{\partial y_i} = 0, \quad i = 1, 2,$$

which means that λ_i and μ_i are constant and so are θ_i , $i = 1, 2$. Therefore, the light ray in each medium goes a straight line.

Next, applying the HMP and adjoint transversality conditions at the switching time t_s , we obtain

$$\begin{aligned} \mathcal{H}_1(t_s^-) &= 1 + \lambda_1 v_1 \sin \theta_1 + \mu_1 v_1 \cos \theta_1 \\ &= 1 + \lambda_2 v_2 \sin \theta_2 + \mu_2 v_2 \cos \theta_2 \\ &= \mathcal{H}_2(t_s). \end{aligned}$$

This gives

$$\lambda_1 v_1 \sin \theta_1 + \mu_1 v_1 \cos \theta_1 = \lambda_2 v_2 \sin \theta_2 + \mu_2 v_2 \cos \theta_2. \quad (5.182)$$

In addition,

$$\lambda_1 = \lambda_2 + \gamma \frac{\partial m}{\partial x} = \lambda_2, \quad (5.183)$$

since $\frac{\partial m}{\partial x} = 0$, and

$$\mu_1 = \mu_2 + \gamma \frac{\partial m}{\partial y} = \mu_2 + \gamma, \quad (5.184)$$

since $\frac{\partial m}{\partial y} = 1$.

Substituting Eqs. (5.183) and (5.184) into Eq. (5.182) yields

$$\lambda_1 v_1 \sin \theta_1 + (\mu_2 + \gamma) v_1 \cos \theta_1 = \lambda_2 v_2 \sin \theta_2 + \mu_2 v_2 \cos \theta_2. \quad (5.185)$$

Since $\lambda_i = \mu_i \tan \theta_i$ from the optimization condition, then we have

$$\mu_i = \lambda_i \cot \theta_i, \quad (5.186)$$

and

$$\mu_2 + \gamma = \mu_1 = \lambda_1 \cot \theta_1, \quad (5.187)$$

where $\cot \theta_i = \frac{\cos \theta_i}{\sin \theta_i}$ for $i = 1, 2$.

Plug (5.186) and (5.187) into (5.185) and note that $\lambda_1 = \lambda_2$, we have

$$\lambda_1 v_1 \sin \theta_1 + (\lambda_1 \cot \theta_1) v_1 \cos \theta_1 = \lambda_1 v_2 \sin \theta_2 + \lambda_1 \cot \theta_2 v_2 \cos \theta_2.$$

Therefore, if $\lambda_1 \neq 0$, then we have

$$v_1 (\sin \theta_1 + \cot \theta_1 \cos \theta_1) = v_2 (\sin \theta_2 + \cot \theta_2 \cos \theta_2). \quad (5.188)$$

This implies

$$\frac{v_1}{v_2} = \frac{\sin \theta_1}{\sin \theta_2},$$

which is known as Snell's Law. $\square$

Note that in the derivation of Snell's law, we do not assume that the light travels in a straight line (it actually shows by the minimum principle that it is necessary to be a straight line). Next, we take a look at a special case of hybrid optimal control problem—switched linear quadratic regulators.

Example 5.7 We focus on linear quadratic optimization for switched systems with linear continuous-variable dynamics. Consider the following switched linear system

$$\begin{aligned} \dot{x}(t) &= A_q x(t) + B_q u_q(t), \quad x(0) = x_0, \\ q(t) &= \sigma(x(t), q(t^-), t), \quad q(0) = q_0, \end{aligned}$$

where $A_k \in \mathbb{R}^{n \times n}$, $B_k \in \mathbb{R}^{n \times m}$, state $x \in \mathbb{R}^n$, control $u \in \mathbb{R}^m$, and discrete mode $q(t) \in \{1, 2, \dots, N\}$ respectively. Our goal is to design a control u_q such that the following linear quadratic criterion is minimized

$$J = \frac{1}{2} \int_0^T (x^T(\tau) Q_q(\tau) x(\tau) + u_{q(\tau)}^T R_q(\tau) u_{q(\tau)}(\tau)) d\tau,$$

over the time interval $[0, T]$, where R_q are symmetric positive definite matrices, and Q_q are symmetric positive semi-definite matrices.

To solve the above linear quadratic optimization, we employ the hybrid minimum principle described above and define the Hamiltonian function associated with each mode as

$$\mathcal{H}_q(p, x, u) = p^T (A_q x + B_q u_q) - \frac{1}{2} (x^T Q_q x + u_q^T R_q u_q),$$

and the Hamiltonian system

$$\begin{aligned} \dot{x} &= \frac{\partial \mathcal{H}_q^T}{\partial p} = A_q x + B_q u_q, \\ \dot{p} &= -\frac{\partial \mathcal{H}_q^T}{\partial x} = -A_q^T p + Q_q x. \end{aligned}$$

The optimal condition for each Hamiltonian $\mathcal{H}_q$ with respect to the continuous control u_q leads to

$$u_q = R_q^{-1} B_q^T p.$$

Assume that there is a time interval $[t_i, t_{i+1}]$ where the active optimal mode is the q_i -mode, then we have

$$\mathcal{H}_{q_i}(p, x, u_{q_i}) > \mathcal{H}_q(p, x, u_q), \quad \forall q \neq q_i.$$

Let $J_i(x(t), t)$ denote the value function J (the optimal cost starting from $(x(t), q_i)$ at time t) on $[t_i, t_{i+1})$ then the Hamilton–Jacobi–Bellman (HJB) equation gives rise to

$$\frac{\partial J_i}{\partial t} = \mathcal{H}_{q_i}\left(-\frac{\partial J_i^T}{\partial x}, x, u_{q_i}\right),$$

on $[t_i, t_{i+1})$, where $p = -\frac{\partial J_i^T}{\partial x}$. Hence, if we choose J_i in the following form

$$J_i(x(t), t) = \frac{1}{2} \left(x^T(t) P_i(t) x(t) + c_i \right),$$

with $P_i(t)$ a symmetric matrix and c_i a constant, then Eq. (5.7) leads to the following *differential Riccati equation* (DRE)

$$\dot{P}_i = -P_i A_{q_i} - A_{q_i}^T P_i + P_i B_{q_i} R_{q_i}^{-1} B_{q_i}^T P_i - Q_{q_i} \quad (5.189)$$

with $p = -P_i x$. Now the limit condition depends on the type of switching produced at time t_{i+1} . If a controlled switching leads to the q_{i+1} mode, then the following *transversality* conditions

$$\begin{aligned} \mathcal{H}_{q_{i+1}}(-P_{i+1}x, x, u_{q_{i+1}}) &= \mathcal{H}_{q_i}(-P_i x, x, u_{q_i}) \\ P_{i+1}x &= P_i x, \end{aligned}$$

hold.

Else if an autonomous switching occurs on a manifold $C_{q_i}(X(\tau_{i+1}), \tau_{i+1}) = 0$ and we have

$$\begin{aligned} P_{i+1}x &= P_i x + \frac{\partial C_{q_i}^T}{\partial x} \Big|_{t=\tau} \pi \\ H_{q_{i+1}}(-P_{i+1}x, x, u_{q_{i+1}}) &= H_{q_i}(-P_i x, x, u_{q_i}) + \frac{\partial C_{q_i}^T}{\partial t} \Big|_{t=\tau} \pi. \end{aligned}$$

Finally, the value function must be continuous along optimal trajectories, and the following additional condition must be held for both cases:

$$x^T P_{i+1} x + c_{i+1} = x^T P_i x + c_i.$$

□

5.3.4 Further Extensions of Hybrid Minimum Principles

We can further extend the basic problem by adding switching cost

$$\sum_{i=1}^k c_{q_i, q_{i+1}}(x(t_i^-), t_i^-), \quad (5.190)$$

to the cost functional and obtain

$$J = \phi(x(t_f), t_f) + \sum_{i=0}^k \int_{t_i}^{t_{i+1}} L_{q_i}(x(s), u(s), s) ds + \sum_{i=1}^k c_{q_i, q_{i+1}}(x(t_i^-), t_i^-). \quad (5.191)$$

For simplicity, we consider a single switching at time t_1 and obtain the following augmented cost functional J_a

$$J_a = \phi(x(t_f), t_f) + c_{q_1, q_2}(x(t_1^-), t_1^-) + \int_{t_0}^{t_1} g_{a_1}(x, \dot{x}, u, p, t) dt + \int_{t_1}^{t_f} g_{a_2}(x, \dot{x}, u, p, t) dt,$$

with $g_{a_1}(x, \dot{x}, u, p, t) = L_{q_1}(x, u, t) + p^T(t)[f_{q_1}(x, u, t) - \dot{x}]$ and $g_{a_2}(x, \dot{x}, u, p, t) = L_{q_2}(x, u, t) + p^T(t)[f_{q_2}(x, u, t) - \dot{x}]$. The variation of J_a can be obtained as

$$\begin{aligned} \delta J_a &= \frac{\partial \phi}{\partial x}(x(t_f), t_f) \delta x_f + \frac{\partial \phi}{\partial t}(x(t_f), t_f) \delta t_f \\ &\quad + \frac{\partial c_{q_1, q_2}}{\partial x}(x(t_1^-), t_1^-) \delta x_1^- + \frac{\partial c_{q_1, q_2}}{\partial t}(x(t_1^-), t_1^-) \delta t_1 \\ &\quad + \left[\frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \delta x(t) \Big|_{t_1^-} + g_{a_1}(x, \dot{x}, u, p, t) \Big|_{t_1^-} \delta t_1 \\ &\quad + \int_{t_0}^{t_1} \left\{ \left[\frac{\partial g_{a_1}}{\partial x}(x, \dot{x}, u, p, t) + \frac{d}{dt} \left(\frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right) \right] \delta x \right. \\ &\quad \left. + \frac{\partial g_{a_1}}{\partial u}(x, \dot{x}, u, p, t) \delta u + \frac{\partial g_{a_1}}{\partial p}(x, \dot{x}, u, p, t) \delta p \right\} dt \\ &\quad + \left[\frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \delta x(t) \Big|_{t_1^+} - g_{a_2}(x, \dot{x}, u, p, t) \Big|_{t_1^+} \delta t_1 \\ &\quad + \int_{t_1}^{t_f} \left\{ \left[\frac{\partial g_{a_2}}{\partial x}(x, \dot{x}, u, p, t) + \frac{d}{dt} \left(\frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right) \right] \delta x \right. \\ &\quad \left. + \frac{\partial g_{a_2}}{\partial u}(x, \dot{x}, u, p, t) \delta u + \frac{\partial g_{a_2}}{\partial p}(x, \dot{x}, u, p, t) \delta p \right\} dt \\ &\quad + g_{a_2}(x, \dot{x}, u, p, t) \Big|_{t_f} \delta t_f. \end{aligned}$$

Note that

$$\begin{aligned}\delta x(t_1^-) &\approx \delta x_1^- - \dot{x}^*(t_1^-)\delta t_1, \\ \delta x(t_1^+) &\approx \delta x_1^+ - \dot{x}^*(t_1^+)\delta t_1, \\ \delta x(t_f) &\approx \delta x_f - \dot{x}^*(t_f)\delta t_f.\end{aligned}$$

We therefore obtain

$$\begin{aligned}\delta J_a &= \left[\frac{\partial \phi}{\partial x}(x, t) + \frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \Big|_{t_f} \delta x_f \\ &\quad + \left\{ \frac{\partial \phi}{\partial t}(x, t) + g_{a_2}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t)\dot{x}(t) \right\} \Big|_{t_f} \delta t_f \\ &\quad \left[\frac{\partial c_{q_1, q_2}}{\partial x}(x, t) + \frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \Big|_{t_1^-} \delta x_1^- + \left[-\frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \Big|_{t_1^+} \delta x_1^+ \\ &\quad \left\{ \frac{\partial c_{q_1, q_2}}{\partial t}(x, t) + g_{a_1}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t)\dot{x}(t) \right\} \Big|_{t_1^-} \delta t_1 \\ &\quad - \left\{ g_{a_2}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t)\dot{x}(t) \right\} \Big|_{t_1^+} \delta t_1 \\ &\quad + \int_{t_0}^{t_1} \left\{ \left[\frac{\partial g_{a_1}}{\partial x}(x, \dot{x}, u, p, t) + \frac{d}{dt} \left(\frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right) \right] \delta x \right. \\ &\quad \quad \left. + \frac{\partial g_{a_1}}{\partial u}(x, \dot{x}, u, p, t) \delta u + \frac{\partial g_{a_1}}{\partial p}(x, \dot{x}, u, p, t) \delta p \right\} dt \\ &\quad + \int_{t_1}^{t_f} \left\{ \left[\frac{\partial g_{a_2}}{\partial x}(x, \dot{x}, u, p, t) + \frac{d}{dt} \left(\frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right) \right] \delta x \right. \\ &\quad \quad \left. + \frac{\partial g_{a_2}}{\partial u}(x, \dot{x}, u, p, t) \delta u + \frac{\partial g_{a_2}}{\partial p}(x, \dot{x}, u, p, t) \delta p \right\} dt.\end{aligned}$$

Following the same steps as before, it is easy to show that

$$\frac{\partial g_{a_i}}{\partial \dot{x}}(x, \dot{x}, u, p, t) = -p^T(t),$$

and

$$g_{a_i}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_i}}{\partial \dot{x}}(x, \dot{x}, u, p, t)\dot{x}(t) = \mathcal{H}_{q_i}(x, u, p, t).$$

First, consider the case that there is no state jumps, then $\delta x_1^- = \delta x_1^+$ since $x(t)$ is continuous in this case. With these new expression of $\frac{\partial g_{a_i}}{\partial \dot{x}} = -p^T$ and $g_{a_i} - \frac{\partial g_{a_i}}{\partial \dot{x}}\dot{x} = \mathcal{H}_{q_i}$, we recompute the necessary conditions upon switching.

Controlled Switching

For controlled switching, δt_1 and δx_1^- are independent, so we need their coefficients both vanish. We first consider the coefficient of δx_1^-

$$\left[\frac{\partial c_{q_1, q_2}}{\partial x}(x, t) + \frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \Big|_{t_1^-} - \left[\frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \Big|_{t_1^+} = 0. \quad (5.192)$$

Since $\frac{\partial g_{a_1}}{\partial \dot{x}} = -p^T$, so we have

$$p^*(t_1^-) = p^*(t_1^+) + \frac{\partial c_{q_1, q_2}^T}{\partial x}(x, t) \Big|_{t_1^-}, \quad (5.193)$$

at the switching instant.

Now, we consider the coefficient of δt_1

$$\begin{aligned} & \left\{ \frac{\partial c_{q_1, q_2}}{\partial t}(x, t) + g_{a_1}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \dot{x}(t) \right\} \Big|_{t_1^-} \\ & - \left\{ g_{a_2}(x, \dot{x}, u, p, t) - \frac{\partial g_{a_2}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \dot{x}(t) \right\} \Big|_{t_1^+} = 0, \end{aligned}$$

which implies that

$$\left[\mathcal{H}_{q_1}(x(t), u(t), p(t), t) + \frac{\partial c_{q_1, q_2}}{\partial t}(x, t) \right] \Big|_{t_1^-} = \mathcal{H}_{q_2}(x(t), u(t), p(t), t) \Big|_{t_1^+}, \quad (5.194)$$

since $g_{a_1} - \frac{\partial g_{a_1}}{\partial \dot{x}} \dot{x} = \mathcal{H}_{q_1}$.

Autonomous Switching

In this case, δt_1 and δx_1^- are related. For this case, δx_1^- and δt_1 must be contained in the tangent plane of $m_{q_1, q_2}(x, t) = 0$ at $(x^*(t_1^-), t_1)$. This implies that the vector $[\delta x_1, \delta t_1]^T$ needs to be orthogonal to the gradient, i.e.,

$$\frac{\partial m_{q_1, q_2}}{\partial x}(x^*(t_1^-), t_1) \delta x_1^- + \frac{\partial m_{q_1, q_2}}{\partial t}(x^*(t_1), t_1) \delta t_1 = 0. \quad (5.195)$$

Hence, the coefficients of δx_1^- and δt_1 in the expression of δJ_a should be able to be written as some linear combination of the gradient vector, namely

$$p^T(t_1^-) - p^T(t_1^+) = \frac{\partial c_{q_1, q_2}}{\partial x}(x(t_1^-), t_1) + \gamma \frac{\partial m_{q_1, q_2}}{\partial x}(x(t_1^-), t_1), \quad (5.196)$$

holds for some constant vector γ , and

$$\begin{aligned} & \mathcal{H}_{q_1}(x(t_1^-), u(t_1^-), p(t_1^-), t_1) - \mathcal{H}_{q_2}(x(t_1^+), u(t_1^+), p(t_1^+), t_1) \\ &= -\frac{\partial c_{q_1, q_2}}{\partial t}(x(t_1^-), t_1) + \mu \frac{\partial m_{q_1, q_2}}{\partial t}(x(t_1^-), t_1) \end{aligned}$$

for some constant μ .

Note that if neither the switching surface nor the switching cost explicitly depends on time, then $\frac{\partial c_{q_1, q_2}}{\partial t}(x, t) = 0$ and $\frac{\partial m_{q_1, q_2}}{\partial t}(x, t) = 0$, and we have

$$\mathcal{H}_{q_1}(x(t_1^-), u(t_1^-), p(t_1^-), t_1) = \mathcal{H}_{q_2}(x(t_1^+), u(t_1^+), p(t_1^+), t_1), \quad (5.197)$$

at the switching instant.

State Jumps

We can also consider state jumps: Upon switching, the state may exhibit state jumps under the following mapping

$$x(t_i^+) = R_{q_i, q_{i+1}}(x(t_i^-), t_i^-). \quad (5.198)$$

For this case, the state trajectory is no longer continuous, and δx_i^- is no longer equivalent to δx_i^+ . Instead, δx_1^- and δx_1^+ satisfy

$$\delta x_1^+ = \frac{\partial R_{q_1, q_2}}{\partial x}(x(t_1^-), t_1^-) \delta x_1^-. \quad (5.199)$$

Then, the coefficient for δx_1^- in δJ_a becomes

$$\left[\frac{\partial c_{q_1, q_2}}{\partial x}(x, t) + \frac{\partial g_{a1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \Big|_{t_1^-} - \left[\frac{\partial g_{a1}}{\partial \dot{x}}(x, \dot{x}, u, p, t) \right] \Big|_{t_1^+} \frac{\partial R_{q_1, q_2}}{\partial x}(x(t_1^-), t_1^-),$$

while the coefficient for δt_1 remains the same.

For controlled switching, both coefficients need to vanish, So we obtain

$$p^{*T}(t_1^-) = p^{*T}(t_1^+) \frac{\partial R_{q_1, q_2}}{\partial x}(x(t_1^-), t_1^-) + \frac{\partial c_{q_1, q_2}}{\partial x}(x, t) \Big|_{t_1^-}, \quad (5.200)$$

at the switching instant.

The conditions need to be changed into the following for autonomous switching

$$p^T(t_1^-) = p^T(t_1^+) \frac{\partial R_{q_1, q_2}}{\partial x}(x(t_1^-), t_1^-) + \frac{\partial c_{q_1, q_2}}{\partial x}(x(t_1^-), t_1) + \gamma \frac{\partial m_{q_1, q_2}}{\partial x}(x(t_1^-), t_1), \quad (5.201)$$

holds for some constant vector γ .

Extended Hybrid Minimum Principle

In summary, we now present a very general version of *hybrid minimum principle* to conclude this section. Consider the hybrid control system $\mathbb{H}$ defined in Eq. (5.107) with the cost functional

$$J = \phi(x(t_f), t_f) + \sum_{i=0}^k \int_{t_i}^{t_{i+1}} L_{q_i}(x(s), u(s), s) ds + \sum_{i=1}^k c_{q_i, q_{i+1}}(x(t_i^-), t_i^-) \quad (5.202)$$

and a terminal state constraint $\Psi(x, t) = 0$. State jumps at switching according to $x(t_i^+) = R_{q_i, q_{i+1}}(x(t_i^-), t_i^-)$. Define the family of system Hamiltonians associated with each discrete state q_i as

$$\mathcal{H}_{q_i}(x, p, u, t) = L_{q_i}(x, u, t) + p^T f_{q_i}(x, u, t), \quad (5.203)$$

for $x, p \in \mathbb{R}^n$, $u \in U$ and $q_i \in Q$. Then there exists an adjoint process p^* for the optimal hybrid control u^* and the corresponding optimal hybrid state trajectory $(q^*(t), x^*(t))$ such that

- (1). the continuous state x^* and the adjoint variable p^* satisfy the *Hamilton's canonical equations*

$$\dot{x}^*(t) = \frac{\partial \mathcal{H}_{q^*}^T}{\partial p}(x^*(t), p^*(t), u^*(t), t), \quad (5.204)$$

$$\dot{p}^*(t) = -\frac{\partial \mathcal{H}_{q^*}^T}{\partial x}(x^*(t), p^*(t), u^*(t), t), \quad (5.205)$$

hold almost everywhere for $\forall t \in [t_0, t_f]$, with boundary conditions

$$x^*(t_0) = x_0,$$

and

$$p^T(t_f) = \frac{\partial \phi(x^*(t_f), t_f)}{\partial x} - \lambda_f \frac{\partial \Psi(x^*(t_f), t_f)}{\partial x}, \quad (5.206)$$

for some real constant λ_f ;

- (2). for an autonomous switching at t_i , the adjoint p^* satisfies the following *transversality condition*

$$p^{*T}(t_i^-) = p^{*T}(t_i^+) \frac{\partial R_{q_i, q_{i+1}}}{\partial x}(x(t_i^-), t_i^-) + \frac{\partial c_{q_i, q_{i+1}}}{\partial x}(x, t) \Big|_{t_i^-} + \gamma_i \frac{\partial m_{q_i, q_{i+1}}(x^*(t_i^-))}{\partial x}, \quad (5.207)$$

where $\gamma_i \in \mathbb{R}$ and $m_{q_i, q_{i+1}}(x^*(t_i), t_i) = 0$ stands for the switching surface for the discrete transition from mode q_i to q_{i+1} . Meanwhile, the Hamiltonian satisfies

$$\mathcal{H}_{q_i^*}(t_i) = \mathcal{H}_{q_{i+1}^*}(t_i) - \frac{\partial c_{q_i, q_{i+1}}(x, t)}{\partial t} \Big|_{t_i^-} + \mu_i \frac{\partial m_{q_i, q_{i+1}}(x^*(t_i^-), t_i)}{\partial t}, \quad (5.208)$$

where $\mu_i \in \mathbb{R}$, $\mathcal{H}_{q_i^*}(t_i) = \mathcal{H}_{q_i^*}(x^*, p^*, u^*, t)|_{t_i^-}$ and $\mathcal{H}_{q_{i+1}^*}(t_i) = \mathcal{H}_{q_{i+1}^*}(x^*, p^*, u^*, t)|_{t_i^+}$;

- (3). for a controlled switching at time instant t_i , adjoint p^* satisfies the following condition

$$p^{*T}(t_i^-) = p^{*T}(t_i^+) \frac{\partial R_{q_i, q_{i+1}}(x(t_i^-), t_i^-)}{\partial x} + \frac{\partial c_{q_i, q_{i+1}}(x, t)}{\partial x} \Big|_{t_i^-}, \quad (5.209)$$

while the Hamiltonian satisfies

$$\left[\mathcal{H}_{q_i}(x(t), u(t), p(t), t) + \frac{\partial c_{q_i, q_{i+1}}(x, t)}{\partial t} \right] \Big|_{t_i^-} = \mathcal{H}_{q_{i+1}}(x(t), u(t), p(t), t) \Big|_{t_i^+}; \quad (5.210)$$

- (4). the state jump relation holds upon switching, i.e., $x(t_i^+) = R_{q_i, q_{i+1}}(x(t_i^-), t_i)$ for all t_i ;
 (5). the Hamiltonian is minimized with respect to the optimal control, namely

$$\mathcal{H}_{q^*}(x^*, p^*, u^*, t) \leq \mathcal{H}_{q^*}(x^*, p^*, u, t) \quad (5.211)$$

holds almost everywhere for all $t \in [t_i, t_{i+1})$ and for all $u \in \mathcal{U}$.

To gain more insight, we present an example from [7] to illustrate the above derived maximum principle for hybrid systems.

Example 5.8 Consider the hybrid control system

$$\dot{x} = f_{q_1}(x, u) = x + xu, \quad (5.212)$$

$$\dot{x} = f_{q_2}(x, u) = -x + xu, \quad (5.213)$$

with the following cost functional

$$J = \int_{t_0}^{t_s} \frac{1}{2} u^2 dt + \frac{1}{1 + x^2(t_s^-)} + \int_{t_s}^{t_f} \frac{1}{2} u^2 dt + \frac{1}{2} x^2(t_f). \quad (5.214)$$

The initial condition is fixed (q_1, x_0) for $x_0 \neq 0$ and $t_0 = 0$. It is assumed a fixed initial and terminal time optimal control. The terminal state x_f is free but penalized with the terminal cost $\phi(x(t_f)) = \frac{1}{2} x^2(t_f)$. For simplicity, we only consider the case when there is only one controlled switching between t_0 and t_f . The associated switching cost $c_{q_1, q_2}(x(t_s), t_s)$ is given as $\frac{1}{1 + x^2(t_s^-)}$. The switching will also cause a state jump, i.e., $x(t_s^+) = -x(t_s^-)$. The optimal control problem is to find the continuous control u and switching time t_s to minimize the cost functional J .

Define the Hamiltonian as

$$\mathcal{H}_{q_1} = \frac{1}{2}u^2 + px(u+1), \quad (5.215)$$

$$\mathcal{H}_{q_2} = \frac{1}{2}u^2 + px(u-1). \quad (5.216)$$

First, from the minimization condition, we have

$$u = -px. \quad (5.217)$$

The co-state equations are

$$\dot{p} = -\frac{\partial \mathcal{H}_{q_1}^T}{\partial x} = -p(u+1) = p(px-1), \quad t \in [t_0, t_s), \quad (5.218)$$

$$\dot{p} = -\frac{\partial \mathcal{H}_{q_2}^T}{\partial x} = -p(u-1) = p(px+1), \quad t \in (t_s, t_f]. \quad (5.219)$$

Terminal and boundary conditions for p can be derived as follows:

$$p(t_f) = -\frac{\partial \phi^T}{\partial x} \Big|_{t_f} = x(t_f), \quad (5.220)$$

$$p(t_s^-) = \frac{\partial R^T}{\partial x} \Big|_{t_s^-} p(t_s^+) + \frac{\partial c_{q_1, q_2}^T}{\partial x} \Big|_{t_s^-} = -p(t_s^+) - \frac{2x(t_s^-)}{(1+x^2(t_s^-))^2}. \quad (5.221)$$

The state equations are

$$\dot{x} = \frac{\partial \mathcal{H}_{q_1}^T}{\partial p} = x(u+1) = -x(px-1), \quad t \in [t_0, t_s), \quad (5.222)$$

$$\dot{x} = \frac{\partial \mathcal{H}_{q_2}^T}{\partial p} = x(u-1) = -x(px+1), \quad t \in (t_s, t_f], \quad (5.223)$$

with initial and boundary conditions

$$x(t_0) = x_0, \quad x(t_s^+) = R(x(t_s^-)) = -x(t_s^-). \quad (5.224)$$

The continuity of Hamiltonian $\mathcal{H}_{q_1}|_{t_s^-} = \mathcal{H}_{q_2}|_{t_s^-}$ gives

$$\left[\frac{1}{2}u^2 + px(u+1)\right] \Big|_{t_s^-} = \left[\frac{1}{2}u^2 + px(u-1)\right] \Big|_{t_s^+}, \quad (5.225)$$

Plug in $u = -px$ and $x(t_s^+) = -x(t_s^-)$, we have

$$x(t_s^-)[p(t_s^-) - p(t_s^+)] = \frac{1}{2}x^2(t_s^-)[p^2(t_s^-) - p^2(t_s^+)]. \quad (5.226)$$

Therefore

$$x(t_s^-)[p(t_s^-) - p(t_s^+)]\left(1 - \frac{1}{2}x^2(t_s^-)[p(t_s^-) + p(t_s^+)]\right) = 0. \quad (5.227)$$

So, at least one of the following needs to be true

$$x(t_s^-) = 0, \quad (5.228)$$

$$p(t_s^-) = p(t_s^+), \quad (5.229)$$

$$1 - \frac{1}{2}x^2(t_s^-)[p(t_s^-) + p(t_s^+)] = 0. \quad (5.230)$$

The first equality cannot be true since $x_0 \neq 0$ and the f_{q_1} won't steer the state to zero in finite time. The third equality is in conflict with (5.221), so it cannot be true either. The only possibility is for second equality to be true, namely

$$p(t_s^-) = p(t_s^+). \quad (5.231)$$

Together with (5.221), we obtain the boundary condition on the co-state as

$$p(t_s^-) = p(t_s^+) = -\frac{x(t_s^-)}{(1 + x^2(t_s^-))^2}. \quad (5.232)$$

Now, we need to solve the state, co-state ODEs with respect to the derived initial, terminal and boundary conditions. This is known as a multi-point boundary value problem, and it is hard to solve even numerically. Fortunately, we may take advantage of the structural properties of the dynamics and further simplify the ODE solving task into a set of algebraic equations that are much easier to solve.

First, we notice that

$$\frac{dp}{dx} = \frac{\dot{p}}{\dot{x}} = \frac{p(px - 1)}{-x(px - 1)} = -\frac{p}{x}$$

for $t \in [t_0, t_s)$. This implies that

$$p = \frac{\alpha}{x},$$

for some constant α and $t \in [t_0, t_s)$. Similarly, we obtain

$$p = \frac{\beta}{x},$$

for some constant β and $t \in (t_s, t_f]$.

Then, the optimal control

$$u = -px = \begin{cases} -\alpha, & t \in [t_0, t_s), \\ -\beta, & t \in (t_s, t_f]. \end{cases} \quad (5.233)$$

Therefore, the state equation becomes

$$\dot{x} = \begin{cases} x(1 - \alpha), & t \in [t_0, t_s), \\ -x(1 + \beta), & t \in (t_s, t_f], \end{cases}$$

and the optimal state trajectory is

$$x(t) = \begin{cases} x_0 e^{(1-\alpha)t}, & t \in [t_0, t_s), \\ -x(t_s^-) e^{-(1+\beta)(t-t_s)} = -x_0 e^{(1-\alpha)t_s - (1+\beta)(t-t_s)}, & t \in (t_s, t_f]. \end{cases}$$

This is because that $x(t_s^-) = x_0 e^{(1-\alpha)t_s}$. Hence

$$p(t) = \begin{cases} \frac{\alpha}{x_0 e^{(1-\alpha)t}}, & t \in [t_0, t_s), \\ \frac{-\beta}{x_0 e^{(1-\alpha)t_s - (1+\beta)(t-t_s)}}, & t \in (t_s, t_f]. \end{cases}$$

From $p(t_s^-) = p(t_s^+)$, we obtain

$$\frac{\alpha}{x_0 e^{(1-\alpha)t_s}} = \frac{-\beta}{x_0 e^{(1-\alpha)t_s - (1+\beta)(t_s - t_s)}}, \quad (5.234)$$

which implies that

$$\alpha = -\beta. \quad (5.235)$$

Also, noting that $p(t_s^-) = p(t_s^+) = -\frac{x(t_s^-)}{(1+x^2(t_s^-))^2}$, we get

$$\frac{\alpha}{x_0 e^{(1-\alpha)t_s}} = -\frac{x_0 e^{(1-\alpha)t_s}}{(1 + x_0^2 e^{2(1-\alpha)t_s})^2}, \quad (5.236)$$

which implies the first algebraic equation

$$\alpha = -\frac{x_0^2 e^{2(1-\alpha)t_s}}{(1 + x_0^2 e^{2(1-\alpha)t_s})^2}. \quad (5.237)$$

Furthermore, from $p(t_f) = x(t_f)$, we have

$$\frac{-\beta}{x_0 e^{(1-\alpha)t_s - (1+\beta)(t_f - t_s)}} = -x_0 e^{(1-\alpha)t_s - (1+\beta)(t_f - t_s)}, \quad (5.238)$$

which implies the second algebraic equation (since $\alpha = -\beta$)

$$\frac{\alpha}{x_0 e^{(1-\alpha)t_s - (1+\beta)(t_f - t_s)}} = -x_0 e^{(1-\alpha)t_s - (1-\alpha)(t_f - t_s)}. \quad (5.239)$$

Given x_0 , t_0 , and t_f , we can solve for α and t_s from these two algebraic equations. Plug the obtained α and t_s into the solution expression of $x(t)$ and $p(t)$, and we can get the optimal control $u(t) = -p(t)x(t)$. $\square$

5.4 Optimal Control of Switched Systems

The hybrid minimum principle that we presented in the previous section is quite general. However, applying these conditions to obtain hybrid optimal control signals and trajectories is far from straightforward, even for switched linear control systems with quadratic cost functionals. In this section, we restrict the hybrid systems we consider to be switched systems, where all discrete transitions are due to controlled switching, and address the optimal control problems where both the continuous control input and the switching signal are design variables. The rest of this section is organized as follows: we first formally define the optimal control problem for switched systems and introduce two different approaches for switched optimal control, namely two-stage optimization and embedding optimization.

5.4.1 Optimal Control Problem for Switched Systems

Consider the following switched continuous-time control system

$$\dot{x}(t) = f_{\sigma(t)}(x(t), u(t)), \quad x(0) = x_0, \quad (5.240)$$

where $x(t) \in X \subseteq \mathbb{R}^n$ is the system state vector, $u(t) \in U \subseteq \mathbb{R}^m$ is the (piecewise) continuous control input vector, $\sigma(t) : \mathbb{R}^+ \rightarrow \mathcal{I} = \{1, 2, \dots, N\}$ is the switching signal, and $f_i : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^n$ is the vector field for each $i \in \mathcal{I}$, and $f_i \in \mathcal{C}^h$ with $h \geq 1$.

In order to control a switched system, we need to choose not only a continuous input vector, but also a switching sequence. A switching sequence over $[t_0, t_f]$ regulates the sequence of active subsystems and is defined as

$$\sigma = ((t_0, \sigma(t_0)), (t_1, \sigma(t_1)), \dots, (t_K, \sigma(t_K))) = ((t_0, i_0), (t_1, i_1), \dots, (t_K, i_K))$$

where $0 < K < \infty$, $t_0 \leq t_1 \leq \dots \leq t_K \leq t_f$, $\sigma(t_k) = i_k \in \mathcal{I}$ for $k = 1, 2, \dots, K$, and K is the finite switching number. By this definition, a pair (t_k, i_k) in σ indicates that at switching instant t_k system (5.240) switches from subsystem i_{k-1} to i_k ; and during the time interval $[t_k, t_{k+1})$ ($[t_K, t_f]$ if $k = K$), subsystem i_k is activated. For a well-posed switched system (5.240), we only consider non-zero sequences,

which allow at most a finite number of switching instants in $[t_0, t_f]$, though different sequences may result in a different number of switchings. If we view σ as an additional discrete input, then the overall control input to the system (5.240) is a pair (σ, u) . Finally, we note that the feature distinguishing a switched system from a general hybrid system is that, its continuous state does not exhibit jumps at the switching instants. Such a feature makes the computation of continuous inputs amenable to the use of conventional optimal control methods.

Here, the optimal control problem is to find a continuous control input $u(t)$ and switching signal $\sigma(t)$ such that the corresponding state trajectory $x(t)$, from a given initial state $x(t_0) = x_0$, minimizes the following cost functional

$$J(\sigma, u) = \int_{t_0}^{t_f} L_\sigma(x(\tau), u(\tau)) d\tau + \phi(x(t_f)) \quad (5.241)$$

subject to (5.240) and satisfies the terminal constraints $\psi(x(t_f)) = 0$.

Denote the optimal control solution as (σ^*, u^*) and its corresponding optimal state trajectory as $x^*(t)$, namely

$$J(\sigma^*, u^*) = \min_{\sigma \in \Sigma_{[t_0, t_f]}, u \in \mathcal{U}_{[t_0, t_f]}} J(\sigma, u). \quad (5.242)$$

Here, we use $\Sigma_{[t_0, t_f]}$ to denote the collection of all feasible switching signals, while $\mathcal{U}_{[t_0, t_f]}$ stands for all feasible continuous control inputs.

5.4.2 Two-Stage Optimization

In the optimal control problem for switched systems, both the control input and the switching signal are design variables, which makes the problem challenging. One key observation is that if the switching signal is fixed and known, the switched system is in fact a time-varying control system. Hence, the problem is reduced to a conventional optimal control problem that can be addressed using the classic optimal control theory from the previous section. This leads to the question of whether we can separate the design of continuous control from the design of switching signals. The following lemma from [12] provides the answer.

Lemma 5.1 *Consider the optimal control problem for system (5.240) with the cost functional (5.241). For a given $x_0 \in \mathbb{R}^n$, suppose that*

- (1). *an optimal solution (σ^*, u^*) exists;*
- (2). *for any given switching sequence σ , there exists a corresponding $u^* = u_\sigma^*$, such that $J_\sigma(u) = J(\sigma, u)$ is minimized.*

Then, we have

$$\min_{\sigma \in \Sigma_{[t_0, t_f]}, u \in \mathcal{U}_{[t_0, t_f]}} J(\sigma, u) = \min_{\sigma \in \Sigma_{[t_0, t_f]}} \left[\min_{u \in \mathcal{U}_{[t_0, t_f]}} J(\sigma, u) \right]. \quad (5.243)$$

Proof First, it is clear that

$$\min_{\sigma \in \Sigma_{[t_0, t_f]}, u \in \mathcal{U}_{[t_0, t_f]}} J(\sigma, u) = J(\sigma^*, u^*) \leq \min_{\sigma \in \Sigma_{[t_0, t_f]}} \left[\min_{u \in \mathcal{U}_{[t_0, t_f]}} J(\sigma, u) \right],$$

since (σ^*, u^*) is assumed to be an optimal solution. We just need to show that the reverse is also true. For this, we notice that

$$\min_{\sigma \in \Sigma_{[t_0, t_f]}} \left[\min_{u \in \mathcal{U}_{[t_0, t_f]}} J(\sigma, u) \right] \leq \min_{u \in \mathcal{U}_{[t_0, t_f]}} J(\sigma^*, u) = J(\sigma^*, u_{\sigma^*}^*),$$

where the first inequality is due to the fact that σ^* is a specific switching signal in $\Sigma_{[t_0, t_f]}$ and the second equality is due to the second assumption.

Actually, we can choose $u_{\sigma^*}^* = u^*$, since for any other u , we must have $J(\sigma^*, u^*) \leq J(\sigma^*, u)$ due to the optimality of (σ^*, u^*) . Hence, we have

$$\min_{\sigma \in \Sigma_{[t_0, t_f]}} \left[\min_{u \in \mathcal{U}_{[t_0, t_f]}} J(\sigma, u) \right] \leq \min_{u \in \mathcal{U}_{[t_0, t_f]}} J(\sigma^*, u) = J(\sigma^*, u_{\sigma^*}^*) = J(\sigma^*, u^*),$$

which completes the proof.

This lemma allows us to decompose the optimal control design for switched systems into two stages as follows:

- *Stage 1.* Fixing σ , solve the inner minimization problem

$$\min_{u \in \mathcal{U}_{[t_0, t_f]}} J(\sigma, u). \quad (5.244)$$

- *Stage 2.* Regarding the optimal cost for each σ as a function

$$J_1 = J_1(\sigma) = \min_{u \in \mathcal{U}_{[t_0, t_f]}} J(\sigma, u), \quad (5.245)$$

minimize J_1 with respect to $\sigma \in \Sigma_{[t_0, t_f]}$.

In more detail, we can further refine the two-stage optimization procedure.

- *Stage 1:*

- Fix the total number of switchings to be K and the sequence of active subsystems, and then solve the optimal control problem to minimize the functional (5.241) with respect to u for the system (5.240) governed by the switching sequence $\sigma = ((t_0, i_0), (t_1, i_1), \dots, (t_K, i_K))$. In the sequel, we denote the corresponding optimal cost as a function $J_1(\tau)$, where $\tau = (t_1, t_2, \dots, t_K)^T$.

(b) Solve the constrained nonlinear optimization problem

$$\begin{aligned} & \min_{\tau} J_1(\tau) \\ & \text{subject to } \tau \in \{(t_1, t_2, \dots, t_K)^T \mid t_0 \leq t_1 \leq \dots \leq t_K \leq t_f\}. \end{aligned}$$

• *Stage 2:*

- (a) Vary the order of active subsystems to find an optimal solution under K switchings.
- (b) Vary the number of switchings K to find an optimal solution.

The key stage is Stage 1, so we will focus on Stage 1 for the rest of this subsection. In Stage 1(a), we need to find an optimal continuous control input u and the corresponding minimum J given a fixed switching signal σ . Although different subsystems are active over different time intervals, Stage 1(a) which seeks $J_1(\tau)$ for the corresponding $\tau = (t_1, t_2, \dots, t_K)^T$ is conventional since these intervals are fixed. In fact, the necessary conditions for optimal control of Stage 1(a) can be derived using the minimum principle for switched systems and hybrid systems.

In Stage 1(b), we need to solve the constrained nonlinear optimization problem with simple constraints. Computational methods for finding local optimal solutions to such problems are abundant in the nonlinear optimization literature. Examples include, but are not limited to, feasible direction methods and penalty function methods.

Equivalent Problem Formulation

As mentioned above, a very important procedure in Stage 1 is to transcribe an optimal control problem into an equivalent conventional optimal control problem parameterized by the switching instants.

For simplicity, we illustrate the main idea of the transformation using a switched system consisting of two subsystems and only one switching between t_0 to t_f

$$\begin{aligned} \dot{x} &= f_1(x, u), \quad t_0 \leq t < t_1, \\ \dot{x} &= f_2(x, u), \quad t_1 \leq t < t_f. \end{aligned}$$

The problem for Stage 1 is to find a switching instant t_1 and a continuous input $u(t)$, $t \in [t_0, t_f]$ such that the cost functional

$$J = \phi(x(t_f)) + \int_{t_0}^{t_f} L(x, u)dt$$

is minimized. Here t_0 , t_f , and $x(t_0) = x_0$ are given.

To transcribe this problem into an equivalent problem parameterized by the switching instants, we introduce a state variable x_{n+1} corresponding to the switching instant t_1 . Let x_{n+1} satisfy

$$\frac{dx_{n+1}}{dt} = 0, \quad (5.246)$$

$$x_{n+1}(0) = t_1. \quad (5.247)$$

Next, a new independent time variable τ is introduced. A piecewise linear relationship between t and τ is established as

$$t = \begin{cases} t_0 + (x_{n+1} - t_0)\tau, & 0 \leq \tau < 1, \\ x_{n+1} + (t_f - x_{n+1})(\tau - 1), & 1 \leq \tau \leq 2. \end{cases} \quad (5.248)$$

Note that $\tau = 0 \implies t = t_0$, $\tau = 2 \implies t = t_f$, and $\tau = 1 \implies t = x_{n+1} = t_1$. The advantage gained by the introduction of τ is that the switching instant and each subsystem's activating period are now fixed.

By introducing x_{n+1} and τ , the original problem is transcribed into the following equivalent optimal control problem.

For a system with dynamics

$$\begin{aligned} \frac{dx(\tau)}{d\tau} &= (x_{n+1} - t_0)f_1(x, u) \\ \frac{dx_{n+1}}{d\tau} &= 0 \end{aligned}$$

in the interval $\tau \in [0, 1)$ and

$$\begin{aligned} \frac{dx(\tau)}{d\tau} &= (t_f - x_{n+1})f_2(x, u) \\ \frac{dx_{n+1}}{d\tau} &= 0 \end{aligned}$$

in the interval $\tau \in [1, 2]$, find an x_{n+1} and a $u(\tau)$, $\tau \in [0, 2]$ such that the cost functional

$$J = \phi(x(2)) + \int_0^1 (x_{n+1} - t_0)L(x, u)d\tau + \int_1^2 (t_f - x_{n+1})L(x, u)d\tau$$

is minimized. Here, t_0 , t_f , and $x(t_0) = x_0$ are given.

This becomes a traditional optimal control problem, and we can obtain the following necessary conditions for optimal control solutions

Define the Hamiltonian

$$\begin{aligned}
& \mathcal{H}(x, p, u, \tau) \\
&= \begin{cases} (x_{n+1} - t_0) \left[L(x, u) + p^T(t) f_1(x, u, t_0 + (x_{n+1} - t_0)\tau) \right], & \tau \in [0, 1), \\ (t_f - x_{n+1}) \left[L(x, u) + p^T(t) f_2(x, u, x_{n+1} + (t_f - x_{n+1})(\tau - 1)) \right], & \tau \in [1, 2]. \end{cases}
\end{aligned} \tag{5.249}$$

We obtain that the canonical equations

$$\dot{x} = \frac{\partial \mathcal{H}^T}{\partial p}(x, p, u, \tau), \tag{5.250}$$

$$\dot{p} = -\frac{\partial \mathcal{H}^T}{\partial x}(x, p, u, \tau), \tag{5.251}$$

and the optimization condition

$$0 = \frac{\partial \mathcal{H}^T}{\partial u}(x, p, u, \tau), \tag{5.252}$$

hold for $\tau \in [0, 2]$.

Note that the solution p and u corresponding to the optimal solution are also functions of τ and x_{n+1} . Therefore, we denote them as $p(\tau, x_{n+1})$ and $u(\tau, x_{n+1})$.

We also obtain the following boundary conditions

$$x(0, x_{n+1}) = x_0, \quad p(2, x_{n+1}) = \frac{\partial \phi}{\partial x}(x(2, x_{n+1})) \tag{5.253}$$

and the continuity condition

$$p(1^-, x_{n+1}) = p(1^+, x_{n+1}). \tag{5.254}$$

These form a two-point boundary value differential algebraic equation (DAE) which is parameterized by x_{n+1} . For each given x_{n+1} , the DAE can be solved using numerical methods. Now assume that we have solved the DAE and obtained the optimal $x(\tau, x_{n+1})$, $p(\tau, x_{n+1})$, and $u(\tau, x_{n+1})$. Then, the optimal value of J is a function of the parameter x_{n+1}

$$J_1(x_{n+1}) = \phi(x(2, x_{n+1})) + \int_0^1 (x_{n+1} - t_0) L(x, u) d\tau + \int_1^2 (t_f - x_{n+1}) L(x, u) d\tau.$$

Note that x_{n+1} represents the switching time, and our task is to seek the optimal switching time between t_0 and t_f . If the optimal switching time happens at the boundary t_0 or t_f , then there is no switching at all. If the switching occurs in an interior point, then it is necessary for the derivative

$$\frac{dJ_1}{dx_{n+1}} = 0. \tag{5.255}$$

Differentiating J_1 with respect to x_{n+1} gives

$$\begin{aligned} \frac{dJ_1}{dx_{n+1}} &= \frac{\partial \phi(x(2, x_{n+1}))}{\partial x} \frac{\partial x(2, x_{n+1})}{\partial x_{n+1}} \\ &+ \int_0^1 \left[L(x, u) + (x_{n+1} - t_0) \left(\frac{\partial L}{\partial x} \frac{\partial x}{\partial x_{n+1}} + \frac{\partial L}{\partial u} \frac{\partial u}{\partial x_{n+1}} + \tau \frac{\partial L}{\partial \tau} \right) \right] d\tau \\ &+ \int_1^2 \left[-L(x, u) + (t_f - x_{n+1}) \left(\frac{\partial L}{\partial x} \frac{\partial x}{\partial x_{n+1}} + \frac{\partial L}{\partial u} \frac{\partial u}{\partial x_{n+1}} + (2 - \tau) \frac{\partial L}{\partial \tau} \right) \right] d\tau. \end{aligned} \quad (5.256)$$

In order to compute $\frac{dJ_1}{dx_{n+1}}$, we need to obtain $\frac{\partial x(\tau, x_{n+1})}{\partial x_{n+1}}$ and $\frac{\partial u(\tau, x_{n+1})}{\partial x_{n+1}}$, which can be obtained from the above state, co-state, and stationarity equations. Instead of pursuing the solutions for general nonlinear dynamics, we are going to use switched linear quadratic regulators to illustrate the key ideas. For nonlinear general cases, interested readers may refer to [12] for details.

Switched Linear Quadratic Regulation Problem

The two-stage optimization method actually provides a basic framework for approaching the optimization problems of switched and hybrid systems. To better illustrate the two-stage strategy, we consider the following simple example with two linear subsystems and only one switching with a fixed index sequence.

Consider a continuous-time switched linear control system

$$\begin{aligned} \dot{x}(t) &= A_1 x(t) + B_1 u(t), \quad t_0 < t < t_1, \\ \dot{x}(t) &= A_2 x(t) + B_2 u(t), \quad t_1 \leq t \leq t_f, \end{aligned}$$

where t_0 and t_f are given. We are to find an optimal switching instant t_1 and an optimal input u to minimize the quadratic cost functional

$$J(t_1, u) = x^T(t_f) Q_f x(t_f) + \int_{t_0}^{t_f} (x^T Q x + u^T R u) dt,$$

with $Q_f \geq 0$, $Q > 0$, and $R > 0$. By introducing a state variable x_{n+1} corresponding to the switching instant t_1 , and a new scaled time variable τ with

$$\begin{aligned} t &= t_0 + (x_{n+1} - t_0)\tau, & 0 \leq \tau \leq 1, \\ t &= x_{n+1} + (t_f - x_{n+1})(\tau - 1), & 1 \leq \tau \leq 2, \end{aligned}$$

the problem is converted into finding an optimal x_{n+1} and an optimal control u for the following system

$$\begin{aligned}\frac{dx(\tau)}{d\tau} &= (x_{n+1} - t_0)(A_1x + B_1u) \\ \frac{dx_{n+1}}{d\tau} &= 0\end{aligned}\quad (5.257)$$

for $\forall \tau \in [0, 1)$, and

$$\begin{aligned}\frac{dx(\tau)}{d\tau} &= (t_f - x_{n+1})(A_2x + B_2u) \\ \frac{dx_{n+1}}{d\tau} &= 0\end{aligned}\quad (5.258)$$

for $\forall \tau \in [1, 2]$ such that the quadratic cost functional

$$\begin{aligned}J(t_1, u) &= x^T(2)Q_fx(2) + \int_0^1 (x_{n+1} - t_0)(x^T Qx + u^T Ru)d\tau \\ &\quad + \int_1^2 (t_f - x_{n+1})(x^T Qx + u^T Ru)d\tau\end{aligned}\quad (5.259)$$

is minimized.

Using a method similar to the method for solving conventional linear quadratic regulator problem (see Example 5.3), it can be shown that the optimal control input is

$$u^* = -\frac{1}{2}R^{-1}B_1^T P(\tau, x_{n+1})x(\tau, x_{n+1}), \quad (5.260)$$

where $P(\tau, x_{n+1})$ satisfy the following parameterized general Riccati equation (parameterized by x_{n+1})

$$-\frac{\partial P}{\partial \tau} = (x_{n+1} - t_0)(Q + PA_1 + A_1^T P - PB_1R^{-1}B_1^T P) \quad (5.261)$$

for $\tau \in [0, 1)$ and

$$-\frac{\partial P}{\partial \tau} = (t_f - x_{n+1})(Q + PA_2 + A_2^T P - PB_2R^{-1}B_2^T P) \quad (5.262)$$

for $\tau \in [1, 2]$, along with the boundary condition $P(2, x_{n+1}) = Q_f$. Then we can solve backwards in τ and obtain the parameterized optimal cost at $\tau = 0$.

Next, to determine the optimal switching time, we need to know $\partial J / \partial x_{n+1}$, that is, the derivative of J with respect to the switching instant. This derivative can be obtained based on the solution of the ordinary differential equations obtained by differentiating the Riccati equations with respect to the switching instants.

The parameterized optimal cost at $\tau = 0$

$$J_1(t_1) = J_1(x_{n+1}) = V^*(x_0, 0, x_{n+1}) = \frac{1}{2} x_0^T P(0, x_{n+1}) x_0. \quad (5.263)$$

Next, if $J_1(x_{n+1})$ is obtained for each $x_{n+1} \in [t_0, t_f]$, then the optimal cost can be calculated by seeking the minimal cost

$$J^* = \min_{x_{n+1} \in [t_0, t_f]} J_1(x_{n+1}) = \min_{x_{n+1} \in [t_0, t_f]} \frac{1}{2} x_0^T P(0, x_{n+1}) x_0. \quad (5.264)$$

This means that the optimal switching instant x_{n+1} should be either the extreme point t_0 or t_f , which means that no switching occurs, or an inner point with the derivative equals zero, namely $\frac{dJ_1}{dx_{n+1}}(x_{n+1}) = 0$.

To determine $\frac{dJ_1}{dx_{n+1}}(x_{n+1})$, differentiate the cost function with respect to x_{n+1}

$$\frac{dJ_1}{dx_{n+1}}(x_{n+1}) = \frac{1}{2} x_0^T \frac{\partial P}{\partial x_{n+1}}(0, x_{n+1}) x_0. \quad (5.265)$$

The value of $\frac{\partial P}{\partial x_{n+1}}(0, x_{n+1})$ can be computed from

$$\begin{aligned} -\frac{\partial}{\partial \tau} \left(\frac{\partial P}{\partial x_{n+1}} \right) &= Q + P A_1 + A_1^T P - P B_1 R^{-1} B_1^T P \\ &\quad + (x_{n+1} - t_0) \left(\frac{\partial P}{\partial x_{n+1}} A_1 + A_1^T \frac{\partial P}{\partial x_{n+1}} - \frac{\partial P}{\partial x_{n+1}} B_1 R^{-1} B_1^T P - P B_1 R^{-1} B_1^T \frac{\partial P}{\partial x_{n+1}} \right) \end{aligned}$$

for $\tau \in [0, 1)$ and

$$\begin{aligned} -\frac{\partial}{\partial \tau} \left(\frac{\partial P}{\partial x_{n+1}} \right) &= Q + P A_2 + A_2^T P - P B_2 R^{-1} B_2^T P \\ &\quad + (t_f - x_{n+1}) \left(\frac{\partial P}{\partial x_{n+1}} A_2 + A_2^T \frac{\partial P}{\partial x_{n+1}} - \frac{\partial P}{\partial x_{n+1}} B_2 R^{-1} B_2^T P - P B_2 R^{-1} B_2^T \frac{\partial P}{\partial x_{n+1}} \right) \end{aligned}$$

for $\tau \in [1, 2]$. The boundary condition $\frac{\partial P}{\partial x_{n+1}}(2, x_{n+1}) = 0$.

Accordingly, the solution of $\frac{\partial P}{\partial x_{n+1}}(0, x_{n+1})$ can be obtained and the optimal x_{n+1} can be determined. The following example from [13] illustrate the design process.

Example 5.9 Consider a switched linear system consisting of two subsystems

$$\dot{x}(t) = \begin{cases} f_0(x, u) = x(t) + u(t), \\ f_1(x, u) = -x(t) + u(t), \end{cases} \quad (5.266)$$

with the following cost functional

$$J(\sigma, u) = \int_0^2 (x^2(t) + u^2(t)) dt. \quad (5.267)$$

Assume that $t_0 = 0$, $t_f = 2$, $x(0) = x_0 = 1$, and $x(2) = x_2 = 1$.

For illustration, we assume that there are at most one switching between t_0 and t_f , that is $K = 1$. According to the two-stage optimization approach, the following four possible switching index sequences are considered

$$\{1\}; \{2\}; \{1, 2\}; \{2, 1\}. \quad (5.268)$$

The first two cases can be solved by classical LQR since there is no switching. For the last two switching index sequences, we can determine the switching instant t_1 and corresponding control $u(t)$ following the approach described in this section.

For example, consider the switching sequence $\{1, 2\}$. First, we introduce a new state x_2 corresponding to the switching instant t_1 , and a new scaled time variable τ with

$$\begin{aligned} t &= x_2 \tau, & 0 \leq \tau \leq 1, \\ t &= x_2 + (2 - x_2)(\tau - 1), & 1 \leq \tau \leq 2. \end{aligned}$$

The problem is converted into finding optimal x_2 and optimal control u for the following system

$$\begin{aligned} \frac{dx(\tau)}{d\tau} &= x_2(x + u) \\ \frac{dx_2}{d\tau} &= 0 \end{aligned} \quad (5.269)$$

for $\forall \tau \in [0, 1)$, and

$$\begin{aligned} \frac{dx(\tau)}{d\tau} &= (2 - x_2)(-x + u) \\ \frac{dx_2}{d\tau} &= 0 \end{aligned} \quad (5.270)$$

for $\forall \tau \in [1, 2]$ such that the quadratic cost functional

$$J(t_1, u) = \int_0^1 x_2(x^2 + u^2)d\tau + \int_1^2 (2 - x_2)(x^2 + u^2)d\tau \quad (5.271)$$

is minimized.

Following the derivation in this subsection, the optimal value function is in the form of

$$V^*(x, \tau, x_2) = \frac{1}{2}x^T P(\tau, x_2)x, \quad (5.272)$$

and $P(\tau, x_2)$ satisfies the following parameterized general Riccati equation (parameterized by x_2)

$$-\frac{\partial P}{\partial \tau} = (x_2 - t_0)(Q + PA_1 + A_1^T P - PB_1 R^{-1} B_1^T P) \quad (5.273)$$

for $\tau \in [0, 1)$ and

$$-\frac{\partial P}{\partial \tau} = (2 - x_2)(Q + PA_2 + A_2^T P - PB_2 R^{-1} B_2^T P) \quad (5.274)$$

for $\tau \in [1, 2]$, along with the boundary condition $P(2, x_2) = 0$. Then we can solve backwards in τ and obtain the parameterized optimal cost at $\tau = 0$.

We first need to solve the following parameterized general Riccati equation (parameterized by x_2)

$$-\frac{\partial P}{\partial \tau} = x_2(Q + PA_1 + A_1^T P - PB_1 R^{-1} B_1^T P) \quad (5.275)$$

for $\tau \in [0, 1)$ and

$$-\frac{\partial P}{\partial \tau} = (2 - x_2)(Q + PA_2 + A_2^T P - PB_2 R^{-1} B_2^T P) \quad (5.276)$$

for $\tau \in [1, 2]$, along with the boundary condition $P(2, x_2) = Q_f$. In this example, we have $Q = 1$, $R = 1$, $A_1 = 1$, $B_1 = 1$, $A_2 = -1$, $B_2 = 1$, and $Q_f = 0$. So, the parameterized general Riccati equations become

$$-\frac{\partial P}{\partial \tau} = x_2(1 + 2P - P^2) \quad (5.277)$$

for $\tau \in [0, 1)$ and

$$-\frac{\partial P}{\partial \tau} = (2 - x_2)(1 - 2P - P^2) \quad (5.278)$$

for $\tau \in [1, 2]$, along with the boundary condition $P(2, x_2) = 0$. Then we can solve backwards in τ and obtain $P(\tau, x_2)$, and thus the parameterized optimal cost is

$$J^* = \min_{x_2 \in [t_0, t_f]} J_1(x_{n+1}) = \min_{x_2 \in [t_0, t_f]} \frac{1}{2} x_0^T P(0, x_2) x_0. \quad (5.279)$$

This means that the optimal switching instant x_{n+1} should be either the extreme point t_0 or t_f , which means that no switching occurs, or an inner point with the derivative equals zero, namely $\frac{dJ_1}{dx_{n+1}}(x_{n+1}) = 0$. $\square$

5.4.3 Embedding Optimization

This section focuses on the derivation of necessary and sufficient conditions for solutions of an optimal control problem for a special class of hybrid systems: two-mode switched systems. In general, a control for such a system is a triplet that consists of

- a sequence of switching instants;
- a sequence of modes;
- a sequence of control inputs (each control input function being associated with a mode).

No constraints are imposed on the switching and the performance index contains no penalty on the switching. This section adopts an “embedding” optimization technique to solve this problem in three steps as follows:

- (i) the switched system is embedded into a larger family of systems and the optimal control problem is formulated for the larger family;
- (ii) a relaxation of the embedded problem, and relationships between solutions of the switched optimal control problem and the embedded optimal control problem, is set forth;
- (iii) necessary conditions for optimality of a solution of the embedded optimal control problem are developed using the minimum principle.

It is readily shown that, when the minimum principle indicates a “bang-bang” type of solution, one obtains a solution to the original problem; otherwise, the solution to the embedded optimal control problem (assuming that it exists) can be approximated arbitrarily close by the trajectory of the switched system generated by an appropriate switching control.

The intuition behind the embedding optimization method [14] is to “smooth” the switching signal by a continuous-valued signal through a convex combination. Hence, the optimal control problem can be solved using traditional approaches as all signals are now continuous. Hopefully, a “bang-bang” type optimal control policy is derived that can be used to recover the switching signal.

To illustrate the main idea, we consider a two-mode switched system described by

$$\dot{x}(t) = f_{\sigma(t)}(x(t), u(t)), \quad x(t_0) = x_0, \quad \sigma(t) \in \mathcal{I} = \{1, 2\}, \quad (5.280)$$

where $x(t) \in X \subseteq \mathbb{R}^n$ is the system states, $u(t) \in U \subseteq \mathbb{R}^m$ is the continuous control inputs, $\sigma(t) : \mathbb{R}^+ \rightarrow \mathcal{I} = \{1, 2\}$ is the switching signal, and f_1 and f_2 are both real-valued vector fields of class C^h with $h \geq 1$. The optimal control problem is to find both the optimal control input and optimal switching signal to optimize the following cost functional

$$J(\sigma, u) = \int_{t_0}^{t_f} L_{\sigma}(x(\tau), u(\tau)) d\tau, \quad (5.281)$$

subject to the system (5.280) while driving the system from an initial state x_0 at time t_0 to a final state x_f at time t_f , where the end time t_f is free and the control input $u \in U$. For simplicity, we ignore the terminal cost in J .

Embedded Optimal Control Problem

We “embed” System (5.280) into a larger family using the following parameterization: let $u_i(t)$ denote the control for actuation in the vector field f_i , $i = 1, 2$, and define the family of systems parameterized by a time-varying parameter $\sigma_E(t)$ as

$$\dot{x}_E(t) = [1 - \sigma_E(t)]f_1(x_E(t), u_1(t)) + \sigma_E(t)f_2(x_E(t), u_2(t)) \quad (5.282)$$

with $x(t_0) = x_0$, where $\sigma_E(t)$ is a time-varying function that takes value in the closed set $[0, 1]$. The associated cost functional is rewritten as

$$J_E(\sigma_E, u_1, u_2) = \int_{t_0}^{t_f} \{[1 - \sigma_E(\tau)]L_1(x_E(\tau), u_1(\tau)) + \sigma_E(\tau)L_2(x_E(\tau), u_2(\tau))\}d\tau. \quad (5.283)$$

It is clear that Eq. (5.282) will reduce to switched system (5.280) if $\sigma_E(t)$ only takes values at the boundary, i.e., only takes discrete values 0 or 1. It is not surprising that, given the same initial condition x_0 , Eq. (5.282) should be able to generate more trajectories than the switched system (5.280) due to the enlarged domain of “switching control” signals $\sigma_E(t) \in [0, 1]$ and independent continuous-time controls $u_1(t), u_2(t) \in U$. However, it can be shown that any trajectory generated by the embedded system (5.282) can be approximated by a solution of the switched system (5.280).

Theorem 5.1 [14] *For the embedded system (5.282) with a given initial condition x_0 and control triplet $\sigma_E(t) \in [0, 1]$, and $u_1(t), u_2(t) \in U$ over the period $[t_0, t_f]$, assume that a solution exists and is unique. Let us denote the solution as $x_E(t)$ with $x_E(t_0) = x_0$. Then, for any desired trajectory-approximation error $\epsilon > 0$, there are switching signals $\sigma_\epsilon(t) \in \{0, 1\}$, and control inputs $u_\epsilon(t) \in U$ defined on $[t_0, t_f]$ such that the generating switching trajectory from the same initial condition $x(t_0) = x_0$ satisfies: $\|x(t) - x_E(t)\| < \epsilon$ for all $t \in [t_0, t_f]$.*

Theorem 5.1 implies that the set of state trajectories of the switched system (5.280) is dense in the set of state trajectories of the embedded system (5.282). Therefore, we can approximate the solutions of the embedded system (5.282) by forcing the signal $\sigma_E(t)$ take values only at the boundary, i.e., “bang-bang” type implementation. The basic idea is to transform the optimal control for the switched system (5.280) into its embedded form (5.282), and then solve the optimal control problem for the embedded system (5.282) with respect to the embedded cost functional in Eq. (5.283).

By allowing $\sigma_E(t)$ to take values in the interval $[0, 1]$, and by increasing the number of controls to two (u_1 and u_2), to ensure satisfaction of a convexity condition, this becomes a classical optimal control problem, for which many methods exist to solve such a problem. The solution is denoted as

$$(\sigma_E^*, u_1^*, u_2^*) = \arg \min_{\sigma_E \in [0,1], u_1, u_2 \in U} J_E(\sigma_E, u_1, u_2).$$

If the optimal control $\sigma_E^*(t)$ is of “bang-bang” type, i.e., $\sigma_E^*(t) \in \{0, 1\}$, then we have solved the optimal control problem for the original switched system (5.280). If $\sigma_E^*(t) \in (0, 1)$ for almost all time instants $t \in [t_0, t_f]$, then using Theorem 5.1, we can obtain a sub-optimal solution with arbitrary small degradation of performance.

The following example aims to shed some light on the existence of “bang-bang” type optimal control signals.

Example 5.10 Consider a switched linear system consisting of N subsystems in the form of

$$\dot{x} = A_i x, \quad i = 1, \dots, N, \quad (5.284)$$

where $x \in \mathbb{R}^n$, $A_i \in \mathbb{R}^{n \times n}$ and $x(t_0) = 0$. We aim to switch between these N modes to minimize the cost functional

$$J = \frac{1}{2} \int_{t_0}^{t_f} x^T Q x dt \quad (5.285)$$

where t_0 is the fixed initial time, t_f is the fixed final time, and Q is symmetric positive definite matrix.

The embedded dynamics are

$$\dot{x} = \sum_{i=1}^N u_i A_i x, \quad (5.286)$$

where $0 \leq u_i \leq 1$ and $\sum_{i=1}^N u_i = 1$. Denote such a set for u as Ω_1 , where

$$\Omega_1 = \left\{ u \in [0, 1]^N \mid \sum_{i=1}^N u_i = 1 \right\}. \quad (5.287)$$

The embedded optimization problem becomes finding the optimal $u \in \Omega_1$ to minimize the cost functional J , while the initial and final conditions remain the same.

The Hamiltonian can be written as

$$\mathcal{H}(x, u, p) = x^T Q x + \sum_{i=1}^N u_i p^T A_i x. \quad (5.288)$$

Based on the minimum condition, the optimal control u_i^* should take either 0 or 1 values that depend on the value of $p^T A_i x$ since u_i appears linearly in $\mathcal{H}$.

Let's first assume that at any time instant t , there exists one and only one i such that

$$p^T A_i x < p^T A_j x, \quad (5.289)$$

for all $j \neq i$ between 1 to N . Under this assumption, the “bang-bang” type optimal solution for the embedded optimization problem exists and can be computed as

$$\sigma^*(t) = \arg \min_i \{p(t)^T A_i x(t)\}. \quad (5.290)$$

For this reason, we call $p(t)^T A_i x(t)$ the switching function.

However, it is possible that for some non-zero measure interval $t \in [a, b]$, $a < b$, such that

$$p^T A_{i_1} x = p^T A_{i_2} x < p^T A_j x \quad (5.291)$$

for some i_1, i_2 , and all $j \neq i_1$ or i_2 . For this case, we need to switch arbitrarily fast between subsystem i_1 and i_2 to approximate the optimal solution from the embedded optimization problem. $\square$

Necessary Conditions

To solve the optimal control for switched systems through embedded optimization, we need the following lemma [14].

Lemma 5.2 *If $\mathcal{X} \subset \mathbb{R}^n$ is a compact set (bounded and closed) and $f, g : \mathcal{X} \rightarrow \mathbb{R}$ are continuous functions, then*

$$\min_{v \in [0,1], u \in \mathcal{X}} [v f(u) + g(u)] = \min \left\{ \min_{u \in \mathcal{X}} [f(u) + g(u)], \min_{u \in \mathcal{X}} g(u) \right\}.$$

Proof First, it is obvious that

$$\min_{v \in [0,1], u \in \mathcal{X}} [v f(u) + g(u)] \leq \min \left\{ \min_{u \in \mathcal{X}} [f(u) + g(u)], \min_{u \in \mathcal{X}} g(u) \right\}.$$

So, we only need to show that the reverse is also true.

Let $v^* u^*$ be optimal. If $f(u^*) < 0$, then

$$\begin{aligned} \min_{u \in \mathcal{X}} [f(u) + g(u)] &\leq 1 \cdot f(u^*) + g(u^*) \\ &\leq v^* f(u^*) + g(u^*) = \min \left\{ \min_{u \in \mathcal{X}} [f(u) + g(u)], \min_{u \in \mathcal{X}} g(u) \right\}. \end{aligned}$$

If $f(u^*) \geq 0$, then

$$\begin{aligned}
\min_{u \in \mathcal{X}} g(u) &\leq 0 \cdot f(u^*) + g(u^*) \\
&\leq v^* f(u^*) + g(u^*) = \min \left\{ \min_{u \in \mathcal{X}} [f(u) + g(u)], \min_{u \in \mathcal{X}} g(u) \right\}.
\end{aligned}$$

Combine these two cases, we have

$$\min_{v \in [0,1], u \in \mathcal{X}} [v f(u) + g(u)] \geq \min \left\{ \min_{u \in \mathcal{X}} [f(u) + g(u)], \min_{u \in \mathcal{X}} g(u) \right\}.$$

Therefore, the lemma holds.

Recall that we consider a switched control system consisting of two subsystems

$$\dot{x} = \begin{cases} f_0(x, u, t), & v(t) = 0, \\ f_1(x, u, t), & v(t) = 1, \end{cases} \quad (5.292)$$

and a cost functional

$$J = \int_{t_0}^{t_f} L(x, u, t) dt, \quad (5.293)$$

where $x \in \mathbb{R}^n$, $u \in \mathcal{U} \subset \mathbb{R}^m$ and $\mathcal{U}$ is compact.

It is assumed that the initial and terminal conditions are given as $x(t_0) = x_0$ and $x(t_f) = x_f$.

Following the embedded optimization problem procedure, we obtain the following dynamics

$$\dot{x} = (1 - v)f_0(x, u_0, t) + v f_1(x, u_1, t) \quad (5.294)$$

and the cost functional

$$J_E = \int_{t_0}^{t_f} \left\{ (1 - v)L(x, u_0, t) + v L(x, u_1, t) \right\} dt. \quad (5.295)$$

The Hamiltonian is defined as

$$\begin{aligned}
&\mathcal{H}_E(x, u_0, u_1, v, p, t) \\
&= (1 - v)L(x, u_0, t) + v L(x, u_1, t) + p^T \left[(1 - v)f_0(x, u_0, t) + v f_1(x, u_1, t) \right] \\
&= v \left\{ L(x, u_1, t) - L(x, u_0, t) + p^T \left[f_1(x, u_1, t) - f_0(x, u_1, t) \right] \right\} \\
&\quad + L(x, u_0, t) + p^T f_0(x, u_0, t)
\end{aligned} \quad (5.296)$$

If we define

$$\begin{aligned}
E_1(x, u_0, u_1, p, t) &= L(x, u_1, t) - L(x, u_0, t) + p^T [f_1(x, u_1, t) - f_0(x, u_1, t)], \\
E_2(x, u_0, p, t) &= L(x, u_0, t) + p^T f_0(x, u_0, t),
\end{aligned}$$

then $\mathcal{H}_E$ can be written as

$$\mathcal{H}_E(x, u_0, u_1, v, p, t) = v(t)E_1(x, u_0, u_1, p, t) + E_2(x, u_0, p, t). \quad (5.297)$$

According to the minimization condition, $\mathcal{H}_E$ is minimized along optimal solutions. In addition, based on the lemma, we have

$$\begin{aligned}
& \min_{v \in [0,1], u_0, u_1 \in \mathcal{U}} \mathcal{H}_E(x, u_0, u_1, v, p, t) \\
&= \min_{v \in [0,1], u_0, u_1 \in \mathcal{U}} [vE_1(x, u_0, u_1, p, t) + E_2(x, u_0, p, t)] \\
&= \min \left\{ \min_{u_0, u_1 \in \mathcal{U}} [E_1(x, u_0, u_1, p, t) + E_2(x, u_0, p, t)], \min_{u_0 \in \mathcal{U}} E_2(x, u_0, p, t) \right\}.
\end{aligned}$$

Note that

$$\begin{aligned}
& E_1(x, u_0, u_1, p, t) + E_2(x, u_0, p, t) \\
&= L(x, u_1, t) - L(x, u_0, t) + p^T [f_1(x, u_1, t) - f_0(x, u_1, t)] + L(x, u_0, t) + p^T f_0(x, u_0, t) \\
&= L(x, u_1, t) + p^T f_1(x, u_1, t) \equiv \mathcal{H}_1(x, u_1, p, t),
\end{aligned}$$

and

$$E_2(x, u_0, p, t) = L(x, u_0, t) + p^T f_0(x, u_0, t) \equiv \mathcal{H}_0(x, u_0, p, t),$$

where $\mathcal{H}_0(x, u_0, p, t)$ and $\mathcal{H}_1(x, u_1, p, t)$ are the Hamiltonian of the subsystem 0 and 1, respectively.

Hence, we obtain

$$\begin{aligned}
& \min_{v \in [0,1], u_0, u_1 \in \mathcal{U}} \mathcal{H}_E(x, u_0, u_1, v, p, t) \\
&= \min \left\{ \min_{u_0 \in \mathcal{U}} \mathcal{H}_0(x, u_0, p, t), \min_{u_1 \in \mathcal{U}} \mathcal{H}_1(x, u_1, p, t) \right\}.
\end{aligned}$$

If $\min_{u_0 \in \mathcal{U}} \mathcal{H}_0(x, u_0, p, t)$ does not equal to $\min_{u_1 \in \mathcal{U}} \mathcal{H}_1(x, u_1, p, t)$ for an extensive period of time, the optimal value for v would be either 0 or 1 for almost all $t \in [t_0, t_f]$ except for some isolated time instants t_i corresponding to switching time.

Therefore, we define

$$\psi(t) = \min_{u_0 \in \mathcal{U}} \mathcal{H}_0(x, u_0, p, t) - \min_{u_1 \in \mathcal{U}} \mathcal{H}_1(x, u_1, p, t), \quad (5.298)$$

and call it a switching function. According to the the switching function ψ , the switching signal is determined by

$$\sigma(t) = \begin{cases} 0, & \psi(t) < 0, \\ 1, & \psi(t) > 0. \end{cases} \quad (5.299)$$

To evaluate the value of the switching function at time t , we need to solve the state and co-state equations derived from minimum principles. We will illustrate the key ideas through the following simple example.

Example 5.11 Consider a scalar switched linear system consisting of two subsystems

$$\dot{x} = \begin{cases} f_0(x, u, t) = x + u, & v(t) = 0, \\ f_1(x, u, t) = -x + u, & v(t) = 1, \end{cases} \quad (5.300)$$

with bounds on control $-1 \leq u(t) \leq 1$ for all t . The cost functional is assumed to be

$$J = \int_{t_0}^{t_f} [x^2(t) + u^2(t)] dt, \quad (5.301)$$

where $t_0 = 0$ and $t_f = 2$. It is assumed that the initial and terminal states are fixed with $x_0 = 1$ and $x_f = 1$.

The embedded system dynamics are

$$\dot{x}(t) = [1 - v(t)][x(t) + u_0(t)] + v(t)[-x(t) + u_1(t)] \quad (5.302)$$

with the same initial, terminal, and control constraints.

The embedded functional cost is

$$J_E(t_0, x_0, v, u_0, u_1) = \int_{t_0}^{t_f} \left\{ x^2(t) + [1 - v(t)]u_0^2(t) + v(t)u_1^2(t) \right\} dt. \quad (5.303)$$

The associated Hamiltonians are

$$\begin{aligned} \mathcal{H}_0(x, u_0, p, t) &= x^2(t) + u_0^2(t) + p(t)[x(t) + u_0(t)], \\ \mathcal{H}_1(x, u_1, p, t) &= x^2(t) + u_1^2(t) + p(t)[-x(t) + u_1(t)]. \end{aligned}$$

The switching function is

$$\begin{aligned} \psi(t) &= \min_{u_0 \in \mathcal{U}} \mathcal{H}_0(x, u_0, p, t) - \min_{u_1 \in \mathcal{U}} \mathcal{H}_1(x, u_1, p, t) \\ &= \min_{u_0 \in [-1, 1]} \left[u_0^2(t) + p(t)u_0(t) \right] - \min_{u_1 \in [-1, 1]} \left[u_1^2(t) + p(t)u_1(t) \right] + 2p(t)x(t). \end{aligned}$$

Note that

$$\min_{u_0 \in [-1, 1]} \left[u_0^2(t) + p(t)u_0(t) \right] = \min_{u_0 \in [-1, 1]} \left\{ \left[u_0 + \frac{p(t)}{2} \right]^2 - \frac{p^2(t)}{4} \right\}, \quad (5.304)$$

so the minimum reaches at $u_0^* = -\frac{p(t)}{2}$ or at the boundary. Similarly

$$\min_{u_1 \in [-1, 1]} \left[u_1^2(t) + p(t)u_1(t) \right] = \min_{u_1 \in [-1, 1]} \left\{ \left[u_1 + \frac{p(t)}{2} \right]^2 - \frac{p^2(t)}{4} \right\}, \quad (5.305)$$

which has the same optimal value as u_0 . Hence, we have

$$\psi(t) = 2p(t)x(t)$$

as the switching function.

To compute the value of switching function at each time instant t , we need to solve for $x(t)$, $p(t)$ through the state

$$\dot{x} = (1 - v)(x + u_0) + v(-x + u_1) \quad (5.306)$$

and co-state equations

$$\dot{p} = -2x + p(2v - 1). \quad (5.307)$$

Since the optimal control for both subsystems is in the form of $-\frac{p(t)}{2}$, which is plugged into the state equation to replace u_0 and u_1 , and we obtain

$$\dot{x} = (1 - v)\left(x - \frac{p}{2}\right) + v\left(-x - \frac{p}{2}\right) = (1 - 2v)x - \frac{p}{2}. \quad (5.308)$$

Note that $x\dot{p} = -2x^2 + xp(2v - 1)$ and $\dot{x}p = (1 - 2v)xp - \frac{p^2}{2}$, so

$$\frac{d}{dt}(xp) = \dot{x}p + x\dot{p} = (1 - 2v)xp - \frac{p^2}{2} - 2x^2 + xp(2v - 1) = -\frac{p^2}{2} - 2x^2, \quad (5.309)$$

which is non-positive for all time $t \in [0, 2]$. This implies that the derivative of the switching function $\psi(t)$ is non-positive, so $\psi(t)$ changes its sign at most once. This means that there is at most one switching time $t_1 \in [0, 2]$, such that $\psi(t) > 0$ before t_1 and $\psi(t) < 0$ after t_1 . This corresponding to either following a switching sequence $\{1, 0\}$ or staying at the mode 1 without switching.

To obtain the optimal control and switching signal, we need to solve

$$\begin{cases} \dot{x} = -x - \frac{p}{2} \\ \dot{p} = -2x + p, \end{cases} \quad (5.310)$$

for $t \in [0, t_1)$ and

$$\begin{cases} \dot{x} = x - \frac{p}{2} \\ \dot{p} = -2x - p, \end{cases} \quad (5.311)$$

for $t \in [t_1, 2]$ with boundary conditions $x(0) = 1$ and $x(2) = 1$. The state constraints and the symmetry of Eqs. (5.310) and (5.311) imply the switching instant $t_1 = 1$. Furthermore, the solutions $x(t)$ and $p(t)$ are continuous. Equations (5.310) and (5.311) can be solved now, which is left as an exercise. $\square$

5.5 Model Predictive Control

Recent decades have seen increasing popularity of the model predictive control (MPC) approach in the industry. Generally speaking, MPC falls into the sampled-data control framework, and its basic idea is to obtain a sub-optimal but stable closed-loop control system by solving an open-loop finite-time optimal control problem at each sampling time.

To illustrate the basic ideas behind MPC, we use as an example of a time-invariant discrete-time nonlinear deterministic system

$$x_{k+1} = f(x_k, u_k), \quad k = 0, 1, \dots, \quad (5.312)$$

with state and control constraints

$$x_k \in X, \quad u_k \in U(x_k), \quad k = 0, 1, \dots \quad (5.313)$$

The overall control goal is to minimize the summation of costs over all stages, where the cost per stage

$$g(x_k, u_k), \quad k = 0, 1, \dots, \quad (5.314)$$

is assumed to be non-negative for all x_k and u_k .

At each sampling time t_k , assuming that the current state $x_k \in X$, an N -stage finite duration optimal control problem needs to be solved, which can be converted into a nonlinear optimization problem of minimizing

$$\sum_{i=k}^{k+N-1} g(x_i, u_i) \quad (5.315)$$

subject to the state equation constraints

$$x_{i+1} = f(x_i, u_i), \quad i = k, k+1, \dots, k+N-1 \quad (5.316)$$

and the state and control constraints

$$x_i \in X, \quad u_i \in U(x_i), \quad i = k, k+1, \dots, k+N-1. \quad (5.317)$$

Let

$$\{u_k^*, u_{k+1}^*, \dots, u_{k+N-1}^*\} \quad (5.318)$$

be the corresponding optimal control sequence. The MPC applies at stage k the first component of this sequence and discards the remaining components, namely $u_k = u_k^*$ and let time evolve. The above process repeats at the next sampling time t_{k+1} with the new state x_{k+1} . Under mild assumptions, the feasibility of the optimization problem at each sampling time can be guaranteed, and the closed-loop stability under MPC can be proved.

The strength of the MPC lies on its capability to deal with constraints and nonlinearities, but the price to pay is its high online computational complexity. Recent years have seen progress on an explicit model predictive control that obtains state feedback control laws based on off-line multi-parametric programming [15, 16].

The main objective of this section is to introduce the MPC for hybrid systems. A key difficulty of MPC for hybrid systems is due to the presence of both continuous real-valued variables and discrete logic statements. To address such a difficulty, we will transform the hybrid system representation into an equivalent mixed-logic dynamical system (MLDS) description [17], where mixed-integer programming optimization approaches can be used. In particular, when the model of the system is an MLDS model and the objective function is quadratic, the optimization problem can be casted as a mixed-integer quadratic program [17–19].

5.5.1 Mixed-Logic Dynamical Systems

Mixed-logic dynamical systems (MLDS) are computationally motivated representations of switched hybrid systems that consist of a collection of linear difference equations involving both real and logical variables, subject to a set of linear inequality constraints. The essence of this approach is to embed the logic part in the state equations by transforming propositional logic statements into *mixed-integer inequalities*, i.e., linear inequalities involving both *continuous variable* $x \in \mathbb{R}^n$ and binary/logical discrete variables $\delta \in \{0, 1\}$. In the following, logic statements are briefly discussed.

Logic Statements

Following the traditional notation, we adopt capital letters X_i to represent statements like “ $x \geq 0$ ” or “Temperature is too high”. X_i is commonly referred to as a *literal*, and has a truth value of either “true” or “false”. Boolean algebra enables statements to be combined into compound statements by means of connectives or operations: “ $\wedge$ ” (and), “ $\vee$ ” (or), “ $\neg$ ” (not), “ $\rightarrow$ ” (implies), “ $\leftrightarrow$ ” (if and only if), “ $\oplus$ ” (exclusive or). It is known that all logical connectives can be defined in terms of a subset of them, for instance, $\{\vee, \neg\}$, which is said to be a complete set of connectives.

If one associates each literal X_i with a logical variable $\delta_i \in \{0, 1\}$, which has a value of either 1 if X_i is “true”, or 0 otherwise, then such a compound statements

can be equivalently translated into a set of integer linear inequalities. For instance, $X_1 \vee X_2$ is equivalent to $\delta_1 + \delta_2 \geq 1$. The equivalence is in the sense that they have the same “truth table”. To be more specific, $X_1 \vee X_2$ is false only if both X_1 and X_2 are false, for which case we have $\delta_1 = 0$ and $\delta_2 = 0$, and therefore, $\delta_1 + \delta_2 = 0$. When either X_1 or X_2 is true or both X_1 and X_2 are true, it will imply the truth of the compound statement $X_1 \vee X_2$, for which we have $\delta_1 + \delta_2 \geq 1$ hold correspondingly.

In fact, the following propositions and linear constraints can easily be seen to be equivalent:

$$\begin{aligned} X_1 \vee X_2 &\iff \delta_1 + \delta_2 \geq 1; \\ X_1 \wedge X_2 &\iff \delta_1 = 1, \delta_2 = 1; \\ \neg X_1 &\iff \delta_1 = 0; \\ X_1 \rightarrow X_2 &\iff \delta_1 - \delta_2 \leq 0; \\ X_1 \leftrightarrow X_2 &\iff \delta_1 - \delta_2 = 0; \\ X_1 \oplus X_2 &\iff \delta_1 + \delta_2 = 1. \end{aligned}$$

Using the above technique, we can transform compound logical statements into linear inequalities involving integer variables $\delta_i \in \{0, 1\}$. This enables us to perform automated logic deduction by solving a mixed-integer (linear) programming problem, for which many methods exist, such as the popular branch-and-bound algorithm among many other heuristics-based algorithms.

Continuous Functions

Furthermore, the systems we are interested in are hybrid, which implies that both discrete logic and continuous variable dynamics are present and interacting. Therefore, it is necessary to establish a link between the two. In particular, we need to describe how to build statements from operating events regarding physical dynamics.

Consider the statement $X = [f(x) \leq 0]$, where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is continuous and $x \in \mathcal{X}$ with $\mathcal{X}$ given as a bounded set in $\mathbb{R}^n$. We define

$$M = \max_{x \in \mathcal{X}} f(x), \quad m = \min_{x \in \mathcal{X}} f(x).$$

Theoretically, an over(under)-estimate of $M(m)$ suffices for our purpose. However, more realistic estimates provide computational benefits. It is easy to verify that

- $[f(x) \leq 0] \wedge [\delta = 1]$ is true, if and only if $f(x) - \delta \leq -1 + m(1 - \delta)$;
- $[f(x) \leq 0] \vee [\delta = 1]$ is true, if and only if $f(x) \leq M\delta$;
- $\neg[f(x) \leq 0]$ is true, if and only if $f(x) \geq \epsilon$, where $\epsilon > 0$ represents a small tolerance (typically the machine precision), beyond which the constraint is regarded as violated;
- in addition, $[f(x) \leq 0] \rightarrow [\delta = 1]$ is true, if and only if $f(x) \geq \epsilon + (m - \epsilon)\delta$;
- $[f(x) \leq 0] \leftrightarrow [\delta = 1]$ is true, if and only if $\epsilon + (m - \epsilon)\delta \leq f(x) \leq M(1 - \delta)$.

To see why, we illustrate through the first translation: “ $[f(x) \leq 0] \wedge [\delta = 1]$ is true, if and only if $f(x) - \delta \leq -1 + m(1 - \delta)$.” The left hand side is a logical statement and holds true only if $[f(x) \leq 0]$ and $[\delta = 1]$. Plug $\delta = 1$ into the right-hand side, and we obtain

$$f(x) - 1 \leq -1 + m(1 - 1),$$

which can be simplified into $f(x) \leq 0$. What if $[\delta = 1]$ is false, then it does not matter whether $[f(x) \leq 0]$ true or false, since $[f(x) \leq 0] \wedge [\delta = 1]$ is false anyway. For this case, we plug $\delta = 0$ into the right-hand side, and we obtain

$$f(x) - 0 \leq -1 + m(1 - 0),$$

which can be simplified into $f(x) \leq m - 1$. This is clearly not satisfiable. Using similar arguments, it is not hard for readers to convince themselves that the above translations make sense. Should f be linear, all constraints obtained would be mixed-integer linear inequalities.

Product Expressions

Next, we transform products of logical variables, and of continuous and logical variables, and express them in terms of (linear) inequalities, which, however, require the introduction of auxiliary variables.

First, a product term of two logical variables $\delta_1 \delta_2$ can be replaced by introducing an auxiliary logical variable $\delta_3 = \delta_1 \delta_2$. Then, it is clear that

$$[\delta_3 = 1] \leftrightarrow [\delta_1 = 1] \wedge [\delta_2 = 1],$$

and therefore

$$\delta_3 = \delta_1 \delta_2 \text{ is equivalent to } \begin{cases} -\delta_1 + \delta_3 \leq 0, \\ -\delta_2 + \delta_3 \leq 0, \\ \delta_1 + \delta_2 - \delta_3 \leq 1. \end{cases} \quad (5.319)$$

Again, the equivalence can be understood by looking at the “truth table” of δ_3 and $\delta_1 \delta_2$. Note that $\delta_i \in \{0, 1\}$ for all i , and $\delta_3 = 1$ only if both $\delta_1 = 1$ and $\delta_2 = 1$. Then we have from the right-hand side

$$\begin{cases} \delta_3 \leq \delta_1 = 1, \\ \delta_3 \leq \delta_2 = 1, \\ \delta_3 \geq \delta_1 + \delta_2 - 1 = 1. \end{cases}$$

Namely, $\delta_3 = 1$. If either δ_1 or δ_2 is zero, we will have $\delta_3 \leq 0$ and $\delta_3 \geq \delta_1 + \delta_2 - 1 \implies \delta_3 \geq 0$ or $\delta_3 \geq -1$. This implies $\delta_3 = 0$.

Moreover, the term $\delta f(x)$, where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and $\delta \in \{0, 1\}$, can be replaced by an auxiliary real variable $y = \delta f(x)$, which satisfies

$$[\delta = 0] \leftrightarrow [y = 0], \quad [\delta = 1] \leftrightarrow [y = f(x)].$$

Therefore

$$y = \delta f(x) \text{ is equivalent to } \begin{cases} y \leq M\delta, \\ y \geq m\delta, \\ y \leq f(x) - m(1 - \delta), \\ y \geq f(x) - M(1 - \delta). \end{cases} \quad (5.320)$$

The equivalence again can be easily verified by plug in $\delta = 0$ or $\delta = 1$ into the inequalities, which will be left as an exercise.

Mixed-Logic Dynamical Systems

So far, we have provided some tools to transform logical statements involving continuous variables into mixed-integer (linear) inequalities, i.e., inequalities involving both continuous variables $x \in \mathbb{R}^n$ and logical (indicator) variables $\delta \in \{0, 1\}$.

These tools will be used now to express relations describing the evolution of systems where physical laws, logic rules, and operating constraints are interdependent.

The resulting *mixed-logic dynamical systems (MLDS)* are described by the following linear relations

$$\begin{cases} x(t+1) = Ax(t) + B_1u(t) + B_2\delta(t) + B_3z(t), \\ y(t) = Cx(t) + D_1u(t) + D_2\delta(t) + D_3z(t), \\ E_2\delta(t) + E_3z(t) \leq E_1u(t) + E_4x(t) + E_5, \end{cases} \quad (5.321)$$

where $t \in \mathbb{Z}$ with $\mathbb{Z}$ being the set of integers, and

$$x = \begin{bmatrix} x_c \\ x_l \end{bmatrix}, \quad x_c \in \mathbb{R}^{n_c}, \quad x_l \in \{0, 1\}^{n_l}, \quad n = n_c + n_l,$$

represent the system states with x_c being continuous components and x_l being logic components;

$$y = \begin{bmatrix} y_c \\ y_l \end{bmatrix}, \quad y_c \in \mathbb{R}^{p_c}, \quad y_l \in \{0, 1\}^{p_l}, \quad p = p_c + p_l,$$

is the output vector with both continuous and logical components; and the control input vector

$$u = \begin{bmatrix} u_c \\ u_l \end{bmatrix}, \quad u_c \in \mathbb{R}^{m_c}, \quad u_l \in \{0, 1\}^{m_l}, \quad m = m_c + m_l,$$

is partitioned similarly. The auxiliary logical and continuous variables are represented by $\delta \in \{0, 1\}^n$ and $z \in \mathbb{R}^{r_c}$, respectively.

As an illustrative example, we convert a piecewise affine system into its mixed-logic dynamical system representation from [17].

Example 5.12 Consider the following discrete-time piecewise linear system

$$x(t+1) = \begin{cases} 0.8x(t) + u(t), & \text{if } x(t) \geq 0, \\ -0.8x(t) + u(t), & \text{if } x(t) < 0, \end{cases}$$

where $x(t) \in [-10, 10]$, $u(t) \in [-1, 1]$, and $t \in \mathbb{Z}$. The condition $x(t) \geq 0$ can be associated to a binary variable $\delta(t)$ such that

$$[\delta(t) = 1] \leftrightarrow [x(t) \geq 0],$$

which can be expressed by the following inequalities

$$\begin{aligned} -m\delta(t) &\leq x(t) - m, \\ -(M + \epsilon)\delta(t) &\leq -x(t) - \epsilon, \end{aligned}$$

where $M = \max_{x \in [-10, 10]}(x) = 10$, $m = \min_{x \in [-10, 10]}(x) = -10$, and ϵ is a small positive scalar. Then the state equation can be written as

$$x(t+1) = 1.6\delta(t)x(t) - 0.8x(t) + u(t).$$

By defining a new variable $z(t) = \delta(t)x(t)$, the above equation can be expressed as

$$\begin{cases} x(t+1) = 1.6z(t) - 0.8x(t) + u(t) \\ z(t) \leq M\delta(t) \\ z(t) \geq m\delta(t) \\ z(t) \leq x(t) - m(1 - \delta(t)) \\ z(t) \geq x(t) - M(1 - \delta(t)), \end{cases}$$

which is in the form of MLDS. □

In principle, the inequality in (5.321) might be satisfied for many values of $\delta(t)$ and/or $z(t)$. On the other hand, we wish that $x(t+1)$ were uniquely determined by $x(t)$ and $u(t)$. To this end, we assume that system (5.321) is well-posed, which means that for all $x(t), u(t)$ within a given bounded set the variables $\delta(t), z(t)$ are defined by (5.321) in a unique way. This allows us to assume that $x(t+1)$ is uniquely defined once $x(t), u(t)$ are given, and therefore, that state trajectory exists and is uniquely determined by the initial state $x(0)$ and input signal $u(0), u(1), \dots$. It is clear that the well-posedness assumption stated above is usually guaranteed by the procedure used

to generate the linear inequalities (5.321), and therefore, this hypothesis is typically fulfilled by the mixed-logic dynamical relations derived from modeling real-world plants.

5.5.2 Mixed-Integer Programming

This section discusses optimal control problems of hybrid systems that can be expressed as mixed-logical dynamical systems. Recall the general form of mixed-logical dynamical systems that are described in Eq. (5.321):

$$\begin{cases} x(t+1) = Ax(t) + B_1u(t) + B_2\delta(t) + B_3z(t), \\ y(t) = Cx(t) + D_1u(t) + D_2\delta(t) + D_3z(t), \\ E_2\delta(t) + E_3z(t) \leq E_1u(t) + E_4x(t) + E_5, \end{cases} \quad (5.322)$$

where $t \in \mathbb{Z}$,

$$x = \begin{bmatrix} x_c \\ x_l \end{bmatrix}, \quad x_c \in \mathbb{R}^{n_c}, \quad x_l \in \{0, 1\}^{n_l}, \quad n = n_c + n_l,$$

is the system states with the x_c components being continuous and the x_l components being 0 or 1. The control input u is partitioned similarly. The auxiliary logical and continuous variables are represented by $\delta \in \{0, 1\}^{r_l}$ and $z \in \mathbb{R}^{r_c}$, respectively. Furthermore, the optimal control problem for discrete-time hybrid systems can thus be rewritten as the following optimal control problem for MLD systems

$$\begin{aligned} J^*(U_N, x_0) &= \min_{U_N} \|Px_N\|_2 + \sum_{k=0}^{N-1} \{\|Q_1u_k\|_2 + \|Q_2\delta_k\|_2 + \|Q_3z_k\|_2 + \|Q_4x_k\|_2\} \\ \text{subject to } &\begin{cases} x_{k+1} = Ax_k + B_1u_k + B_2\delta_k + B_3z_k, \\ E_2\delta_k + E_3z_k \leq E_1u_k + E_4x_k + E_5, \\ x_N \in \psi_f, \\ x(0) = x_0, \end{cases} \end{aligned}$$

where $k \in \mathbb{Z}$.

Next we show how to transform the optimal control problem for MLDS into a mixed-integer programming problem. In fact, the translation for a quadratic performance index is simply obtained by substituting the state update equation, namely

$$x_k = A^k x_0 + \sum_{j=0}^{k-1} A^j (B_1 u_{k-1-j} + B_2 \delta_{k-1-j} + B_3 z_{k-1-j})$$

and the optimization vector

$$\zeta = \{u_0, \dots, u_{N-1}, \delta_0, \dots, \delta_{N-1}, z_0, \dots, z_{N-1}\}$$

in the above formulation.

Then, the above optimal control problem can be formulated as a *mixed-integer quadratic programming* problem (MIQP), namely

$$\begin{aligned} \min_{\zeta} & \zeta^T H_1 \zeta + \zeta^T H_2 x_0 + x_0^T H_3 x_0 + c_1^T \zeta + c_2^T x_0 + c \\ \text{subject to} & G\zeta \leq W + Sx_0, \end{aligned}$$

where $H_1, H_2, H_3, c_1, c_2, G, W, S$ are matrices of suitable dimensions.

Similarly, if the cost function is measured by 1-norm or ∞ -norm performance indices, it is also possible to transform the finite horizon optimal control problem into a mixed-integer linear programming (MILP) [18, 19]. To illustrate this, consider the finite horizon optimal control problem for MLD systems as follows:

$$\begin{aligned} J^*(U_N, x_0) = \min_{U_N} & \|Px_N\|_p + \sum_{k=0}^{N-1} \{ \|Q_1 u_k\|_p + \|Q_2 \delta_k\|_p + \|Q_3 z_k\|_p + \|Q_4 x_k\|_p \} \\ \text{subject to} & \begin{cases} x_{k+1} = Ax_k + B_1 u_k + B_2 \delta_k + B_3 z_k \\ E_2 \delta_k + E_3 z_k \leq E_1 u_k + E_4 x_k + E_5 \\ x_N \in \psi_f \\ x(0) = x_0 \end{cases} \end{aligned}$$

where $\|\cdot\|_p$ stands for 1-norm or ∞ -norm.

For the case of $p = 1$ or ∞ , the translation requires the introduction of slack variables. In particular, for $p = \infty$ the sum of the components of any vector

$$\{\varepsilon_0^u, \dots, \varepsilon_{N-1}^u, \varepsilon_0^\delta, \dots, \varepsilon_{N-1}^\delta, \varepsilon_0^z, \dots, \varepsilon_{N-1}^z, \varepsilon_0^x, \dots, \varepsilon_N^x, \}$$

that satisfies

$$\begin{aligned} -1_m \varepsilon_k^u &\leq Q_1 u_k, & k = 0, 1, \dots, N-1 \\ -1_m \varepsilon_k^u &\leq -Q_1 u_k, & k = 0, 1, \dots, N-1 \\ -1_{r_l} \varepsilon_k^\delta &\leq Q_2 \delta_k, & k = 0, 1, \dots, N-1 \\ -1_{r_l} \varepsilon_k^\delta &\leq -Q_2 \delta_k, & k = 0, 1, \dots, N-1 \\ -1_{r_c} \varepsilon_k^z &\leq Q_3 z_k, & k = 0, 1, \dots, N-1 \\ -1_{r_c} \varepsilon_k^z &\leq -Q_3 z_k, & k = 0, 1, \dots, N-1 \\ -1_n \varepsilon_k^x &\leq Q_4 x_k, & k = 0, 1, \dots, N-1 \\ -1_n \varepsilon_k^x &\leq -Q_4 x_k, & k = 0, 1, \dots, N-1 \\ -1_n \varepsilon_N^x &\leq Px_N, \\ -1_n \varepsilon_N^x &\leq -Px_N, \end{aligned}$$

represents an upper bound on $J(U_N, x(0))$, where 1_m is the column vector of ones with length m , and

$$x_k = A^k x_0 + \sum_{j=0}^{k-1} A^j (B_1 u_{k-1-j} + B_2 \delta_{k-1-j} + B_3 z_{k-1-j}).$$

It can be shown that the vector

$$\zeta = \{\varepsilon_0^u, \dots, \varepsilon_{N-1}^u, \varepsilon_0^\delta, \dots, \varepsilon_{N-1}^\delta, \varepsilon_0^z, \dots, \varepsilon_{N-1}^z, \varepsilon_0^x, \dots, \varepsilon_N^x, u_0, \dots, u_{N-1}\}$$

that satisfies the above condition and simultaneously minimizes

$$J(\zeta) = \varepsilon_0^u + \dots + \varepsilon_{N-1}^u + \varepsilon_0^\delta + \dots + \varepsilon_{N-1}^\delta + \varepsilon_0^z + \dots + \varepsilon_{N-1}^z + \varepsilon_0^x + \dots + \varepsilon_N^x$$

also solves the original optimal control problem. Therefore, we obtain the following MILP problem

$$\begin{aligned} & \min_{\zeta} J(\zeta) \\ \text{subject to } & \begin{cases} -1_m \varepsilon_k^u \leq \pm Q_1 u_k, \quad k = 0, 1, \dots, N-1 \\ -1_{r_l} \varepsilon_k^\delta \leq \pm Q_2 \delta_k, \quad k = 0, 1, \dots, N-1 \\ -1_{r_c} \varepsilon_k^z \leq \pm Q_3 z_k, \quad k = 0, 1, \dots, N-1 \\ -1_n \varepsilon_k^x \leq \pm Q_4 (A^k x_0 + \sum_{j=0}^{k-1} A^j (B_1 u_{k-1-j} + B_2 \delta_{k-1-j} + B_3 z_{k-1-j})), \\ -1_n \varepsilon_N^x \leq \pm P (A^N x_0 + \sum_{j=0}^{N-1} A^j (B_1 u_{k-1-j} + B_2 \delta_{k-1-j} + B_3 z_{k-1-j})), \\ x_{k+1} = A x_k + B_1 u_k + B_2 \delta_k + B_3 z_k \quad k \geq 0 \\ E_2 \delta_k + E_3 z_k \leq E_1 u_k + E_4 x_k + E_5 \quad k \geq 0 \\ x_N \in \psi_f \\ x(0) = x_0 \end{cases} \end{aligned}$$

where the variable x_0 appears only in the constraints as a parameter vector.

Given a value x_0 of the initial state, the MILP or MIQP can be solved to obtain the optimizer $\zeta^*(x_0)$, and therefore, the optimal input $U_N^*(x_0)$. There exists a method called multi-parametric programming [18, 19] that can be used to efficiently compute optimal control laws, which is in the form of a piecewise affine state feedback.

5.6 Summary

In this chapter, we studied hybrid optimal control problems, where the sequence of discrete modes, switching times, and continuous control signals are to be found to minimize a given cost functional. For this, we began with a brief review of the calculus of variations and Pontryagin's minimum principle from the classical optimal control theory. Several versions of the hybrid minimum principle were introduced as extensions of the Pontryagin minimum principle. We mainly used arguments based on the calculus of variations to justify these necessary conditions upon switching and under possible state jumps. The latter part of this chapter focused on numerical

approaches to solving hybrid optimal control problems. We discussed the two-stage optimization and embedded optimization approaches for switched control systems, and also the model predictive control approach for mixed-logic dynamic systems. Some key points from this chapter are summarized below.

- The calculus of variations and Pontryagin's minimum principle provide necessary conditions an optimal solution needs to satisfy. The main idea is that small perturbations in an optimal control solution should have vanishingly small effects on the cost. Hence, the optimal solution is only local and often given in an open-loop form.
- The necessary conditions for an optimal control solution lead to a two-point boundary value problem, which usually involves nonlinear differential equations with split boundary conditions. The combination of nonlinearity and split boundary conditions make the two-point boundary value problem very challenging to solve even numerically. Iterative numerical approaches to solving the two-point boundary value problem, such as shooting, gradient, and quasi-linearization methods, have been proposed in the literature.
- Linear time-invariant control systems with quadratic cost functionals is one of few special cases that can be solved analytically by the Pontryagin minimum principle; and also by dynamic programming. This is the linear quadratic regulation (LQR) problem, the solution of which can be obtained by solving a matrix differential equation in Riccati form.
- In order to extend the Pontryagin's minimum principle to the hybrid optimal control problems, one needs to consider the variations of the cost function due to perturbations on the switching instant and state trajectories upon switching. This leads to new transversality conditions at the switching instant, but we need to distinguish between controlled switching from autonomous switching.
- The described hybrid minimum principle leads to multi-point boundary value problem, which is even harder to solve than the two-point boundary value problem needed to be solved in classical optimal control. For this reason, we focused on special cases of hybrid optimal control problems in the rest of this chapter.
- As a special case of hybrid optimal control, we first considered the switched optimal control problem, where only controlled switches are involved. Two computational approaches namely the two-stage optimization and the embedding optimization approach were presented to determine both the optimal continuous control inputs and the optimal switching instants for switched systems.
- As a second special case of hybrid optimal control, we considered optimal control for discrete-time piecewise affine control systems. It was shown how a discrete-time piecewise affine system can be expressed as an equivalent mixed-logic dynamic system (MLDS); then the problem of optimal control synthesis for discrete-time piecewise affine control systems can be solved using mixed-integer (linear/quadratic) programming.

5.7 Notes and Further Reading

Our presentation on the minimum principle is inspired by [8, 20]. There are many excellent textbooks on classical optimal control theory, see e.g., [8–10, 20, 21]. In our discussion, we assume the existence of optimal solutions and derive necessary conditions for an optimal solution. However, the existence of optimal control is a nontrivial issue. The existence of solutions to a general time optimal control problem was studied by Filippov [22] and was extended to the more general problem by Cesari [23]. With suitable change of variables, a standard optimal control problem can be transformed into a time optimal control problem.

The literature on the optimal control of hybrid systems is extensive, and our treatment is by necessity introductory and brief. Various versions of hybrid minimum or maximum principles have been proposed, depending on whether or not the sequence of discrete states is assumed to be given, whether autonomous switches are considered besides controlled switches, and whether switching costs occur and/or state jumps are allowed upon switching. We started with a simple version of the hybrid minimum principle, and ended with a general hybrid minimum principle, which follow the versions reported in [2, 3, 5, 6, 24–26].

Earlier efforts were devoted to extensions of the maximum/minimum principle and dynamical programming techniques to hybrid systems, which unfortunately have not led to computationally efficient solutions of general hybrid optimal control problems. For this reason, we considered two special cases and focused on numerical solutions. The first special case considered was the switched optimal control problem, where only controlled switches can occur and the state trajectory is continuous (no state jumps). Two numerical approaches to switched optimal control were discussed, namely the two-stage optimization approach and the embedded optimization approach. The material on the two-stage optimization method is based on [12], where the readers can find more numerical examples; see also [12, 27–33]. The two-stage optimization approach has also been adopted in [34–36]. Our treatment of the embedded optimization approach follows [14], where readers may find proofs and more examples. The idea of embedding or converting the switched system into a family of systems more suitable for treating optimal control using traditional numerical or theoretical methods has been pursued by other researchers as well, see e.g., [37, 38]. The discussion on model predictive control of piecewise affine systems and the mixed-logic dynamical systems follows the results in [16, 17, 19, 39].

It should be pointed out that there are several software tool boxes available to support the computation of the optimal control solutions of switched and hybrid systems. For example, the Multi-Parametric Toolbox (MPT) [19] is a free MATLAB toolbox for design, analysis, and deployment of optimal controllers for constrained linear, nonlinear, and hybrid systems. Convex Dynamic Programming (CDP) Tool [40] is another MATLAB toolbox developed to solve hybrid optimal control problems.

Finally, readers particularly interested in computational methods for optimal control problems of switched systems are strongly encouraged to consult the survey papers [27, 32] and the references therein.

5.8 Exercises

1. **Simulations:** It is strongly recommended to simulate each of the examples in this chapter so as to gain insight of the system's behavior and verify the claimed results. MATLAB and Stateflow provide an excellent starting point for simulation work. Additional specialized commercial software is also available as noted above.
2. Among all curves $x(t)$ with $x(0) = 1$ and $x(1) = 0$, find an extremal curve that has the shortest length. Recall that the length of a curve $x(t)$ dependent on a parameter t between $t = a$ and $t = b$ is given by

$$J = \int_a^b \sqrt{1 + \dot{x}^2(t)} dt.$$

3. Consider a double integrator

$$\begin{cases} \dot{x}_1(t) = x_2(t), \\ \dot{x}_2(t) = u(t), \end{cases}$$

where $-1 \leq u(t) \leq 1$ for all $0 \leq t < t_f$. The initial state is assumed to be fixed $x(0) = [x_1(0), x_2(0)]^T$. Let the cost function be

$$J(u(\cdot)) = \int_0^{t_f} d\tau = t_f,$$

where t_f is the first time at which $x_f = [0, 0]^T$. The optimal control problem is thus to minimize the time to reach the origin from any initial condition. Hence, it is called a time optimal control problem.

- (a) Find an optimal control $u^*(t)$ such that the cost functional $J(u(\cdot))$ is minimized.
 - (b) Plot the corresponding optimal trajectory of state $x^*(t)$.
 - (c) Represent the controlled system as a hybrid automaton.
 - (d) Do MATLAB simulations.
4. Consider the following control system

$$\begin{aligned} \dot{x}_1 &= x_2 \\ \dot{x}_2 &= -ax_1 + u \end{aligned}$$

with $-1 \leq u \leq 1$ and $a > 0$. Our control task is to design the control law that transfers an arbitrary initial state x_0 to the origin in minimum time.

5. Consider the following control system

$$\dot{x}(t) = u(t).$$

The goal is to design control law $u(t)$ to drive from an arbitrary initial state x_0 to the origin, and meanwhile minimize the control cost

$$J = \int_{t_0}^{t_f} |u(t)| dt.$$

The terminal time t_f is free, and the admissible controls satisfy $|u(t)| \leq 1$ for all t .

6. Consider a model of wheeled robots given by

$$\begin{cases} \dot{x} = v \cos \theta \\ \dot{y} = v \sin \theta \\ \dot{\theta} = u, \end{cases}$$

where $(x, y)^T \in \mathbb{R}^2$ are the coordinates for the center of the vehicle and $\theta \in [-\pi, \pi]$ is the heading. The control u is the turning rate bounded by v/R , where $v > 0$ is the constant forward velocity of the vehicle and $R > 0$ is its lower bound on the turning radius. Our control problem is to drive the vehicle from a given initial state (x_0, y_0, θ_0) to a specified point (x_f, y_f, θ_f) in minimum time.

7. Consider the soft-landing problem of a lunar rocket, which satisfies the following equation of motion

$$m(t)\ddot{x}(t) = -gm(t) - k\dot{m}(t).$$

Here $x(t)$ is altitude of the rocket, $m(t)$ stands for the mass of the rocket, and g and k are gravity and thrust constants, respectively. The mass rate is constrained by $-M \leq \dot{m} \leq 0$. The soft-landing problem requires to land to the surface $x(t_f) = 0$ with a minimum duration of time. Formulate the soft-landing problem as an optimal control problem, and derive the form of the optimal control law.

8. The law of reflection states that when a ray of light reflects off a surface, the angle of incidence is equal to the angle of reflection. Derived the law of reflection from the hybrid minimum principle and Fermat's Principle. Fermat's Principle tells us that "The actual path between two pointstaken by a beam of light is the one which is traversed in the least time."
9. Consider the following hybrid control system

$$\dot{x} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = f_{q_1}(x, u) = \begin{bmatrix} x_2 \\ -x_1 + u \end{bmatrix}$$

and

$$\dot{x} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = f_{q_2}(x, u) = \begin{bmatrix} x_2 \\ u \end{bmatrix}.$$

The hybrid optimal control problem is to minimize the following cost functional

$$J(t_0, t_f, h_0, L) = \int_{t_0}^{t_f} \frac{1}{2} u(\tau)^2 d\tau + \frac{1}{2} (x_1(t_s))^2 + \frac{1}{2} (x_2(t_f))^2,$$

where the t_s stands for a switching instant that triggered by crossing the switching surface $m(x) = x_2(t_s) = 0$. The terminal cost $\phi(x(t_f)) = \frac{1}{2}(x_2(t_f))^2$ and the switching cost $c(x(t_s)) = \frac{1}{2}(x_1(t_s))^2$. It is assumed that the terminal time t_f is fixed, and there is no terminal constraint.

10. Switched systems are a special class of hybrid systems. Consider the maximum principle (MP) version for switched systems. Define the Hamiltonian for each mode q as

$$\mathcal{H}_q(x, p, u) = -L_q(x, u) + p^T f_q(x, u)$$

and the Hamiltonian system as

$$\dot{x} = \frac{\partial \mathcal{H}_q^T}{\partial p}(x, p, u), \quad \dot{p} = -\frac{\partial \mathcal{H}_q^T}{\partial x}(x, p, u).$$

Let an execution $\chi^* = (x^*, q^*, u^*, \sigma^*)$ be optimal.

- (a) What are the Hamilton's canonical equations?
- (b) What are the transversality conditions?

11. Consider the switched linear systems consisting of two subsystems

$$\text{subsystem1 : } \begin{cases} \dot{x}_1 = u \\ \dot{x}_2 = 0.5x_2 \end{cases} \quad (5.323)$$

$$\text{subsystem2 : } \begin{cases} \dot{x}_1 = 0.1x_1 + x_2 \\ \dot{x}_2 = -x_1 + 0.1x_2 \end{cases} \quad (5.324)$$

Assume that $t_0 = 0, t_f = 5, x(0) = x_0 = [1, -1]^T$. The cost function is given as

$$J(\sigma, u) = x^T(t_f)x(t_f) + \int_0^{t_f} (x^T(t)x(t) + u^2(t))dt. \quad (5.325)$$

For illustration, we assume that there is at most one switching between t_0 and t_f , that is, $K = 1$. According to the two-stage optimization approach, the following four possible switching index sequences are considered

$$\{1\}; \{2\}; \{1, 2\}; \{2, 1\}. \quad (5.326)$$

The first two cases can be solved by classical LQR since there is no switching. For the last two switching index sequences, determine the switching instant t_1 and corresponding control $u(t)$.

12. Consider a saturated linear system $\dot{x} = Ax + b \cdot \text{sat}(v)$, where the saturation function $\text{sat}(\cdot)$ is defined as

$$\text{sat}(v) = \begin{cases} -1, & v \leq -1 \\ v, & -1 < v \leq 1 \\ 1, & v > 1 \end{cases}.$$

Derive necessary conditions for optimal solutions that minimize the cost $J(x_0, v) = \int_0^\infty (x^T R x + v^2) dt$ with positive definite symmetric matrix R .

13. Consider the switched system given by $\dot{x} = A_i x + b_i u$, for $i = 1, 2$ with $A_1 = \begin{pmatrix} 0 & 1 \\ -1 & -2 \end{pmatrix}$, $b_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$, and $A_2 = \begin{pmatrix} -3 & -1 \\ 0 & -2 \end{pmatrix}$, $b_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$. Let $x_0 = \begin{pmatrix} 2 \\ 3 \end{pmatrix}$ and $t_f = 10$. We are to minimize the cost function

$$J(x_0, u, \sigma) = \int_0^{t_f} (x^T(t)x(t) + u^2(t))dt,$$

where the switching signal is allowed to switch at most once.

- Use two-stage optimization methods to compute and plot its optimal switching time, control signals and state trajectory.
 - Can you solve this optimal control problem using embedding optimization methods as well?
14. Consider the following discrete-time piecewise affine system

$$x(k+1) = \begin{cases} x(k) + u(k), & \text{if } x(k) \geq 0 \\ -x(k) + u(k), & \text{if } x(k) < 0 \end{cases},$$

where $x(k) \in [-10, 10]$, $u(k) \in [-1, 1]$. We are to minimize the cost function

$$J(x_0, U_N) = \sum_{k=0}^3 (x^2(k) + u^2(k)),$$

with $x(0) = 5$.

- Derive an equivalent finite-time optimal control problem for a mixed-logic dynamical system.
 - Derive its corresponding mixed-integer quadratic optimization problem.
 - Numerically solve the derived mixed-integer quadratic optimization problem and plot the state trajectory.
15. Consider the following discrete-time piecewise linear system

$$\begin{aligned}
x(t+1) &= 0.8 \begin{bmatrix} \cos \alpha(t) & -\sin \alpha(t) \\ \sin \alpha(t) & \cos \alpha(t) \end{bmatrix} x(t) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u(t), \quad t \in \mathbb{Z} \\
y(t) &= [1 \ 0] x(t), \\
\alpha(t) &= \begin{cases} \frac{\pi}{3} & \text{if } [1 \ 0] x(t) \geq 0, \\ -\frac{\pi}{3} & \text{if } [1 \ 0] x(t) < 0, \end{cases} \\
x(t) &\in [-10, 10] \times [-10, 10], \\
u(t) &\in [-1, 1].
\end{aligned}$$

- (a) Transform the discrete-time piecewise linear system into an equivalent mixed-logical dynamical (MLD) system.
- (b) Given an initial state x_0 and a final time horizon N , please find a control sequence $U_{N-1} = \{u(0), u(1), \dots, u(N-1)\}$ that drives the system from the initial state $x_0 = [-1 \ 1]^T$ to the final state $x_f = [0 \ 0]^T$ and minimize the performance index

$$\begin{aligned}
J(U_N, x_0) &= \sum_{t=0}^{N-1} \|u(t) - u_f\|_{Q_1}^2 + \|\delta(t, x_0, U_t) - \delta_f\|_{Q_2}^2 \\
&\quad + \|z(t, x_0, U_t) - z_f\|_{Q_3}^2 + \|x(t, x_0, U_{t-1}) - x_f\|_{Q_4}^2 \\
&\quad + \|y(t, x_0, U_{t-1}) - y_f\|_{Q_5}^2
\end{aligned}$$

subject to

$$x(N, x_0, U_N) = x_f$$

and the MLD system dynamics, where $z_f = [0 \ 0 \ 0 \ 0]^T$, $\delta_f = 1$, $u_f = 0$, and $\|x\|_Q^2 \triangleq x^T Q x$ with the weights $Q_1 = 1$, $Q_2 = 0.01$, $Q_3 = 0.01 I_4$, $Q_4 = I_2$, and $Q_5 = 0$.

References

1. H. Witsenhausen. A class of hybrid-state continuous-time dynamic systems. *Automatic Control, IEEE Transactions on*, 11(2):161–167, Apr 1966.
2. M.S. Branicky, V.S. Borkar, and S.K. Mitter. A unified framework for hybrid control: model and optimal control theory. *Automatic Control, IEEE Transactions on*, 43(1):31–45, 1998.
3. H.J. Sussmann. A maximum principle for hybrid optimal control problems. In *Decision and Control, 1999. Proceedings of the 38th IEEE Conference on*, volume 1, pages 425–430 vol. 1, 1999.
4. S. Hedlund and A. Rantzer. Convex dynamic programming for hybrid systems. *Automatic Control, IEEE Transactions on*, 47(9):1536–1540, Sep 2002.
5. C.G. Cassandras, D.L. Pepyne, and Y. Wardi. Optimal control of a class of hybrid systems. *IEEE Trans. Automat. Contr.*, 46(3):398–415, 2001.

6. M. S. Shaikh and P. E. Caines. On the hybrid optimal control problem: Theory and algorithms. *IEEE Trans. Automat. Control*, 52(9):1587–1603, 2007.
7. A. Pakniyat and P. E. Caines. On the minimum principle and dynamic programming for hybrid systems. *IFAC Proc. Volumes*, 47(3):9629–9634, 2014.
8. Donald E Kirk. *Optimal control theory: an introduction*. Prentice-Hall, 1970.
9. Daniel Liberzon. *Calculus of variations and optimal control theory: a concise introduction*. Princeton University Press, 2011.
10. Frank L Lewis, Draguna Vrabie, and Vassilis L Syrmos. *Optimal control*. John Wiley & Sons, 2012.
11. M. S. Shaikh and P. E. Caines. On relationships between Weierstrass-Erdmann corner condition, snell’s law and the hybrid minimum principle. In *2007 International Bhurban Conf. on Applied Sciences & Technology*, pages 117–122. IEEE, 2007.
12. Xuping Xu and Panos J Antsaklis. Optimal control of switched systems based on parameterization of the switching instants. *IEEE transactions on automatic control*, 49(1):2–16, 2004.
13. Zhendong Sun and Shuzhi S. Ge. *Switched linear systems: control and design*. Springer Science & Business Media, 2006.
14. S. C. Bengua and R. A. DeCarlo. Optimal control of switching systems. *Automatica*, 41(1):11–27, January 2005.
15. Alberto Bemporad, Manfred Morari, Vivek Dua, and Efstratios N Pistikopoulos. The explicit linear quadratic regulator for constrained systems. *Automatica*, 38(1):3–20, 2002.
16. Francesco Borrelli, Alberto Bemporad, and Manfred Morari. *Predictive control for linear and hybrid systems*. Cambridge University Press, 2017.
17. A. Bemporad and M. Morari. Control of systems integrating logic, dynamics, and constraints. *Automatica*, 35(3):407–427, 1999.
18. Francesco Borrelli, Mato Baotić, Alberto Bemporad, and Manfred Morari. Dynamic programming for constrained optimal control of discrete-time linear hybrid systems. *Automatica*, 41(10):1709–1721, 2005.
19. F. Borrelli. *Constrained Optimal Control of Linear and Hybrid Systems*, volume 290 of *Lecture Notes in Control and Information Sciences*. Springer-Verlag, 2003.
20. AE Bryson and Ho YC. *Applied Optimal Control: Optimization, Estimation and Control*. CRC Press, 1975.
21. Brian DO Anderson and John B Moore. *Optimal control: linear quadratic methods*. Prentice-Hall, 1990.
22. A. F Filippov. On certain questions in the theory of optimal control. *Journal of the Society for Industrial and Applied Mathematics, Series A: Control*, 1(1):76–84, 1962.
23. Lamberto Cesari. *Optimization theory and applications: problems with ordinary differential equations*. Springer, 1983.
24. Benedetto Piccoli. Hybrid systems and optimal control. In *Proceedings of the 37th IEEE Conference on Decision and Control (Cat. No. 98CH36171)*, volume 1, pages 13–18. IEEE, 1998.
25. P Riedinger, F Kratz, C Iung, and C Zanne. Linear quadratic optimization for hybrid systems. In *Proceedings of the 38th IEEE Conference on Decision and Control (Cat. No. 99CH36304)*, volume 3, pages 3059–3064. IEEE, 1999.
26. Ali Pakniyat and Peter E Caines. On the hybrid minimum principle. arXiv preprint [arXiv:1710.05521](https://arxiv.org/abs/1710.05521), 2017.
27. Feng Zhu and Panos J Antsaklis. Optimal control of hybrid switched systems: A brief survey. *Discrete Event Dynamic Systems*, 25(3):345–364, 2015.
28. Xuping Xu and Panos J Antsaklis. Optimal control of switched systems via non-linear optimization based on direct differentiations of value functions. *International Journal of Control*, 75(16-17):1406–1426, 2002.
29. Xuping Xu and Panos J Antsaklis. Optimal control of switched systems: new results and open problems. In *American Control Conference, 2000. Proceedings of the 2000*, volume 4, pages 2683–2687. IEEE, 2000.
30. Xuping Xu and Panos J Antsaklis. A dynamic programming approach for optimal control of switched systems. In *Decision and Control, 2000. Proceedings of the 39th IEEE Conference on*, volume 2, pages 1822–1827. IEEE, 2000.

31. Xuping Xu and Panos J Antsaklis. An approach for solving general switched linear quadratic optimal control problems. In *Decision and Control, 2001. Proceedings of the 40th IEEE Conference on*, volume 3, pages 2478–2483. IEEE, 2001.
32. Xuping Xu and Panos J Antsaklis. Results and perspectives on computational methods for optimal control of switched systems. In *International Workshop on Hybrid Systems: Computation and Control*, pages 540–555. Springer, 2003.
33. Xuping Xu and Panos J Antsaklis. Optimal control of hybrid autonomous systems with state jumps. In *American Control Conference, 2003. Proceedings of the 2003*, volume 6, pages 5191–5196. IEEE, 2003.
34. Magnus Egerstedt, Yorai Wardi, and Henrik Axelsson. Transition-time optimization for switched-mode dynamical systems. *Automatic Control, IEEE Transactions on*, 51(1):110–115, 2006.
35. Humberto Gonzalez, Ramanarayan Vasudevan, Maryam Kamgarpour, S Shankar Sastry, Ruzena Bajcsy, and Claire Tomlin. A numerical method for the optimal control of switched systems. In *Decision and Control (CDC), 2010 49th IEEE Conference on*, pages 7519–7526. IEEE, 2010.
36. Yorai Wardi and Magnus Egerstedt. Algorithm for optimal mode scheduling in switched systems. In *American Control Conference (ACC), 2012*, pages 4546–4551. IEEE, 2012.
37. Tuhin Das and Ranjan Mukherjee. Optimally switched linear systems. *Automatica*, 44(5):1437–1441, 2008.
38. Eduardo Mojica-Nava, Nicanor Quijano, and Naly Rakoto-Ravalontsalama. A polynomial approach for optimal control of switched nonlinear systems. *International Journal of Robust and Nonlinear Control*, 2013.
39. A. Bemporad, F. Borrelli, and M. Morari. On the optimal control law for linear discrete time hybrid systems. In *Hybrid systems: Computation and control*, volume 2289 of *Lecture Notes in Computer Science*, pages 105–119. Springer, 2002.
40. S. Hedlund and A. Rantzer. Cdp tool: A matlab tool for optimal control of hybrid systems. department of automatic control. In *Lund Institute of Technology*, 1999.

Chapter 6

Formal Synthesis



So far, we have presented and discussed basic results for the modeling, verification, stability, and optimal control of hybrid systems. These results have been introduced by both the computer science and control engineering communities. The computer science community has been primarily interested in verifying the correctness of embedded codes in the face of real-time constraints and interactions with the physical world; while the control engineering community has been mainly focusing on the effects of discontinuities (such as switching of dynamics and jumping of states) caused by the use of digital processors and communication networks that are increasingly becoming ubiquitous.

The past decade has witnessed efforts to merge these two schools of thought, and a noticeable trend in the recent hybrid system literature is the emphasis on the synthesis of hybrid controllers for continuous or hybrid dynamical systems to satisfy complicated temporal logic specifications. This approach is known as symbolic control or hybrid supervisory control. The basic idea is first to obtain equivalent or approximating finite abstractions of the hybrid or continuous systems to be controlled. Then the design is carried out in the discrete domain using model checking [1], game-theoretic approaches [2], or discrete event supervisory control theory [3]. The final step is to convert the designed discrete supervisor back to a hybrid controller (since the controller usually contains both discrete transitions and continuous flows) so that it can be used to control the original hybrid or continuous system. The effectiveness of the abstraction-based method depends on whether or not there exists a finite state discrete abstracted model for the original hybrid or continuous system. Significant research efforts have been devoted to developing abstraction methods based on reachability analysis; see, e.g., [4, 5]. However, the number of abstracted states could grow exponentially with respect to the dimension of the physical dynamics. We, therefore, introduce symbolic approaches to formal synthesis as well, which can be seen as extensions of symbolic model checking methods to the formal synthesis problems. Symbolic approaches have shown great promise in tackling the state explosion problem.

This chapter aims to give an accessible introduction to the literature on formal synthesis methods in hybrid systems, so we begin with the very basics, such as discrete games and differential games, before our discussion on hybrid systems. The rest of the chapter is organized as follows. First, in Sect. 6.1, we review the game-theoretical synthesis approaches from the computer science literature. Particularly, discrete games with respect to safety, reachability, and Büchi winning conditions are introduced. Then, we give a brief review of the differential game theory for continuous state systems in Sect. 6.2. The connection between discrete games and differential games is explored in Sect. 6.3. Next, we turn to hybrid systems and discuss timed games and hybrid games in Sects. 6.4 and 6.5, respectively, for which we mainly illustrate symbolic approaches. Section 6.6 focuses primarily on discussing the abstraction-based formal synthesis procedure for linear and nonlinear continuous systems and hybrid systems with multi-affine dynamics. In Sect. 6.7, we conclude this chapter with a discussion of trajectory optimization with respect to real-time temporal logic.

6.1 Discrete Games

The formal synthesis problem seeks to automatically design a controller to achieve a given formal specification (e.g., safety, reachability, or a set of temporal logic specifications). The key challenge is due to the uncertainty of the environment. One way to capture the environmental uncertainties is to consider the worst case and cast the synthesis into a game between the controller and the environment, where the controller tries to make the moves that satisfy the specification while the environment tries to violate the specification. The design will be conservative, but it is guaranteed to work even in an adversarial environment. This section aims to give a brief review of games for discrete transition systems, while the games under a continuous state differential dynamics will be covered in the next section. After reviewing both discrete games and differential games, we will solve hybrid games.

6.1.1 Game Structure

We begin with the design of discrete controllers for a finite discrete game, which can be formally defined as follows.

Definition 6.1 A (two-player) game structure is a 4-tuple $G = (Q, \Sigma, q_0, \delta)$, where

- Q is a finite set of discrete states;
- $\Sigma = \Sigma_1 \times \Sigma_2$ is a finite set of discrete inputs, where Σ_1 contains the controller's inputs and Σ_2 consists of environment's inputs, which cannot be controlled (without loss of generality, we assume that Σ_1 and Σ_2 are mutually exclusive);
- $q_0 \in Q$ is the initial state of the game;
- $\delta : Q \times \Sigma \rightarrow 2^Q$ is the transition relation.

Intuitively, starting from a state $q \in Q$, Player 1 chooses an action $\sigma_1 \in \Sigma_1$ and, simultaneously and independently, Player 2 chooses an action $\sigma_2 \in \Sigma_2$. Then the game proceeds further to a new state q' according to transition relation δ , namely $q' \in \delta(q, \sigma_1, \sigma_2)$. The states in $\delta(q, \sigma_1, \sigma_2)$ are called the (σ_1, σ_2) -successor of q .

Definition 6.2 A source- q run of the game structure G is a (finite or infinite) sequence of alternating state and action pair

$$q_0(\sigma_1^0, \sigma_2^0)q_1(\sigma_1^1, \sigma_2^1) \cdots ,$$

where $q_0 = q$ and for all $i \geq 0$, the state q_{i+1} is a (σ_1^i, σ_2^i) -successor of q_i .

Definition 6.3 A strategy of Player i , for $i = 1, 2$, is a function mapping a sequence of (nonempty) states to a set of actions, namely $f_i : Q^+ \rightarrow 2^{\Sigma_i}$.

Definition 6.4 Given a pair of strategies for Player 1 and Player 2, denoted by f_1 and f_2 , respectively, the outcome $\mathcal{O}_G(q, f_1, f_2)$ of the game G from a state q for strategies f_1 and f_2 is a subset of the source- q runs of G . A run $q_0(\sigma_1^0, \sigma_2^0)q_1(\sigma_1^1, \sigma_2^1) \cdots$ belongs to $\mathcal{O}_G(q, f_1, f_2)$ if $q_0 = q$ and for all $i \geq 0$, we have $\sigma_1^i \in f_1(q_0q_1 \cdots q_i)$ and $\sigma_2^i \in f_2(q_0q_1 \cdots q_i)$ and the state q_{i+1} is a (σ_1^i, σ_2^i) -successor of q_i .

We turn to a simple example [45] to illustrate these definitions.

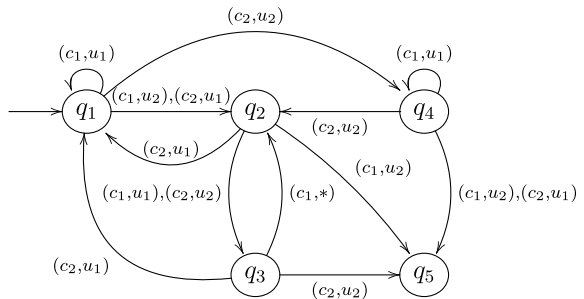
Example 6.1 Consider a game structure $G = (Q, \Sigma, q_0, \delta)$, where $Q = \{q_1, q_2, q_3, q_4, q_5\}$, $q_0 = q_1$, and $\Sigma_1 = \{c_1, c_2\}$, $\Sigma_2 = \{u_1, u_2\}$. The transition relations are as follows:

$$\delta = \left\{ \begin{array}{l} (q_1, c_1, u_1, q_1), (q_1, c_1, u_2, q_2), (q_1, c_2, u_1, q_2), (q_1, c_2, u_2, q_4), \\ (q_2, c_1, u_1, q_3), (q_2, c_1, u_2, q_5), (q_2, c_2, u_1, q_1), (q_2, c_2, u_2, q_3), \\ (q_3, c_1, u_1, q_2), (q_3, c_1, u_2, q_2), (q_3, c_2, u_1, q_1), (q_3, c_2, u_2, q_5), \\ (q_4, c_1, u_1, q_4), (q_4, c_1, u_2, q_5), (q_4, c_2, u_1, q_5), (q_4, c_2, u_2, q_2) \end{array} \right\}.$$

Graphically, the game structure G is plotted in Fig. 6.1. A source- q_1 run is given as below:

$$q_1(c_1, u_1)q_1(c_2, u_1)q_2(c_1, u_1)q_3 \cdots$$

Fig. 6.1 A discrete game structure, where * is used to represent any feasible event



Consider memoryless strategies $f_1(q) = \{c_2\}$ and $f_2(q) = \{u_2\}$ for all $q \in Q$. Then, a source- q_1 run that belongs to the outcome $\mathcal{O}_G(q, f_1, f_2)$ could be

$$q_1(c_2, u_2)q_4(c_2, u_2)q_2(c_2, u_2)q_3 \cdots$$

□

Some remarks are in order. First, the turn-based game (players play alternatively, namely the state space Q is divided into two disjoint sets $Q_1 \cup Q_2$ and the Player i chooses actions when and only when the state is at $q \in Q_i$) is a special case of the simultaneous game defined here. We leave it as an exercise for readers to translate a turn-based game into a game structure defined here. Second, although we only defined a game between two players, the generalization to multiple players is straightforward. Third, we can also define a label function $l : Q \rightarrow 2^P$ that maps each state q to a subset of atomic propositions $P \subseteq \mathcal{P}$, which hold true at state q .

6.1.2 Safety Games

Games are played with respect to certain winning conditions. A common winning condition is to keep states within a subset of states as many practical concerns, such as the collision avoidance or mutual exclusion requirements, can be cast into such kind of winning conditions. This is known as a safety game and can be formally defined as follows.

Definition 6.5 Consider a subset of states $F \subseteq Q$, the first player (i.e., the controller) wins a run of the game if the run always stays in F ; otherwise, it loses the run.

We term this winning condition as $\square$ -condition. Computation of a winning strategy for a safety game is performed via the determination of the set F^* of winning states. This can be done iteratively by initializing $F_0 = F$. At every iteration, we extract F_{i+1} by removing from F_i all the states, denoted by $\pi(F_i)$, from which the first player cannot force the game to stay in F_i . It is not hard to see that a state $q \in F_i$ if, starting from q , the controller can stay in F for at least i steps. This procedure converges to the set F^* of winning states. According to the winning condition of the safety game, the controller has a winning strategy if $q_0 \in F^*$. Formally, the aforementioned mapping π can be defined as follows.

Definition 6.6 For a game structure G , we define a mapping $\pi : 2^Q \rightarrow 2^Q$ for $P \subseteq Q$ as

$$\pi(P) = \{q \in Q : \exists \sigma_1 \in \Sigma_1, \forall \sigma_2 \in \Sigma_2, \delta(q, \sigma_1, \sigma_2) \subseteq P\}. \quad (6.1)$$

Intuitively, the controllable predecessor of P , $\pi(P)$, denotes the states from which the controller, by choosing a control input σ_1 , can force the system into P in the next step no matter what the environment chooses.

Example 6.2 Let's revisit Example 6.1. We can conclude that $q_1 \in \pi(\{q_1, q_2\})$ since starting from q_1 if Player 1 chooses c_1 , then no matter what Player 2 chooses the next state is still within the set $\{q_1, q_2\}$. However, $q_2 \notin \pi(\{q_1, q_2\})$ since starting from q_2 if Player 1 chooses c_1 , then Player 2 can choose either u_1 or u_2 to drive the state outside of $\{q_1, q_2\}$. On the other hand, if Player 1 chooses c_2 , then Player 2 can choose u_2 to drive the state outside of $\{q_1, q_2\}$ as well. $\square$

We can now propose the algorithm for calculating the winning states for a safety game.

Algorithm 6.1 (Synthesis for Safety Games)

Input: A game structure G and a winning condition $\square F$

Output: Winning states $F^* \subseteq Q$

1: Initialization $F_0 = F$;

2: **while** $i \geq 0$ **do**

3: $F_{i+1} := F_i \cap \pi(F_i)$

4: **if** $F_{i+1} == F_i$ **then**

5: **break**;

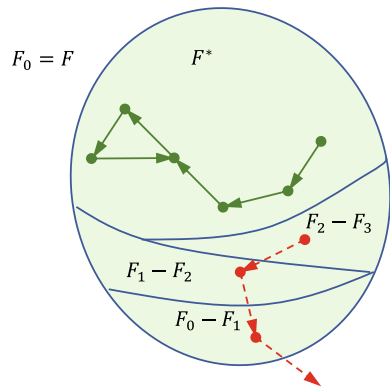
6: $i = i + 1$

7: **Return** $F^* = F_i$.

At every iteration, F_{i+1} is created by removing from F_i those states that are not in $\pi(F_i)$, namely those states that cannot be forced to stay in F_i by controllers. By induction, it is not hard to show that the states in F_i can be forced to remain in F_i by a controller for at least i steps. Once a fixed point is reached, all states in F^* will be able to remain in F^* forever. The basic idea behind the design algorithm is illustrated in Fig. 6.2.

Example 6.3 Consider the game structure in Fig. 6.1. Our goal is to avoid State q_5 , namely to stay within the set of states $F = \{q_1, q_2, q_3, q_4\}$. To solve this safety game, we apply the algorithm with

Fig. 6.2 Illustration of the algorithm solving a safety game



$$F_0 = F = \{q_1, q_2, q_3, q_4\}.$$

The controllable predecessor set of F_0 , $\pi(F_0) = \{q_1, q_2, q_3\}$, since at the state q_4 no matter what the controller does, it is possible for the environment to drive the state to q_5 . For instance, if controller chooses c_1 , the environment may choose u_2 to drive the state from q_4 to q_5 . Similarly, if the controller chooses c_2 , the environment may choose u_1 to drive the state from q_4 to q_5 . Then, we obtain

$$F_1 = F_0 \cap \pi(F_0) = \{q_1, q_2, q_3\}.$$

Repeating the computation, we obtain $\pi(F_1) = \{q_1, q_2, q_3\}$, and therefore

$$F_2 = F_1 \cap \pi(F_1) = \{q_1, q_2, q_3\}.$$

Note that we have $F_2 = F_1$, so the iteration stops and the algorithm returns the winning set $F^* = \{q_1, q_2, q_3\}$, starting from which the controller has a strategy to keep the state evolution inside F^* . $\square$

It is clear that the safety property can be enforced by the controller (Player 1) as long as the initial condition $q_0 \in F^*$. Furthermore, we can obtain a memoryless strategy $f_1 : Q \rightarrow 2^{\Sigma_1}$ for Player 1 as follows:

$$f_1(q) = \left\{ \sigma_1 \in \Sigma_1 \mid \forall \sigma_2 \in \Sigma_2, \delta(q, (\sigma_1, \sigma_2)) \subseteq F^* \right\}. \quad (6.2)$$

Note that the set $f_1(q)$ is a non-empty subset of Σ_1 for states $q \in F^*$. In addition, if a run starts from a state in F^* and Player 1 follows the strategy f_1 defined in (6.2), Player 1 can force the state to remain inside F^* . Therefore, the above strategy is a winning strategy for Player 1 with respect to the safety game $\square F$ as long as $q_0 \in F^*$.

Example 6.4 For instance, the control strategy for the safety game in Example 6.3 can be derived as follows: $f_1(q_1) = c_1$, $f_1(q_2) = c_2$, and $f_1(q_3) = c_1$.

Since the initial state q_1 is within F^* , the safety game is winnable for the controller with the above memoryless control strategy.

It actually does not matter what we assign for states outside of F^* since the controller is going to lose anyway at these states. For this example, we can simply set $f_1(q) = \Sigma_1$ for all states $q \notin F^*$. $\square$

Actually, it is not hard to see that for states $q \notin F^*$ there always exists a sequence of moves by the environment (Player 2) to drive the state q outside of F eventually. In other words, no matter what the controller (Player 1) does, it will lose the safety game if the system starts from some state $q \notin F^*$. In this sense, the safety game for a discrete game structure is deterministic—the outcome from each state can be determined before the game is played without any ambiguity. In our example, the winning region for Player 1 with respect to the safety game ($\square F$) is given by

$F^* = \{q_1, q_2, q_3\}$, while its complement $\bar{F}^* = \{q_4, q_5\}$ is the winning region by the environment (Player 2).

6.1.3 Reachability Games

Dual to safety winning conditions, we can alternatively consider the game from the second player's perspective, in which case this player wins a run of the game if the state of the system is driven outside of F at least once. We term this winning condition as the $\diamond$ -condition.

Definition 6.7 Given a game structure G and a set $F \subseteq Q$, the first player wins a run of the reachability game $\diamond F$ if some states from F is reached by the run; otherwise, it loses the run.

The controllable predecessor operator π also plays an important role in the synthesis of games with other types of winning conditions. For instance, for discrete games with $\diamond$ -conditions, each F_i contains the states from which a visit to F can be enforced after at most i steps. Thus, the algorithm for discrete $\diamond$ -games can be stated as follows:

Algorithm 6.2 (Synthesis for Discrete Reachability Game)

Input: A game structure G and a winning condition $\diamond F$

Output: Winning states $F^* \subseteq Q$

1: Initialization $F_0 = \emptyset$;

2: **while** $i \geq 0$ **do**

3: $F_{i+1} := F \cup \pi(F_i)$

4: **if** $F_{i+1} == F_i$ **then**

5: **break**;

6: $i = i + 1$

7: **Return** $F^* = F_i$.

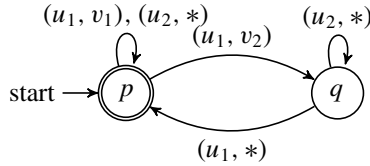
At every iteration, F_{i+1} is created by adding to F_i those states that can be driven into F_i in one step, i.e., states in $\pi(F_i)$. By induction, it is not hard to show that the states in F_i can be driven to F by a controller in at most i steps (left as an exercise). Once a fixed point is reached, all states in F^* can reach F eventually.

Furthermore, as long as F^* is non-empty, we can obtain a memoryless strategy $f_1 : Q \rightarrow 2^{\Sigma_1}$ for states in F^* as follows:

$$f_1(q) = \left\{ \sigma_1 \in \Sigma_1 \mid \forall \sigma_2 \in \Sigma_2, \delta(q, (\sigma_1, \sigma_2)) \subseteq F^* \right\}. \quad (6.3)$$

Note that for states $q \in F^*$, the set $\{\sigma_1 \in \Sigma_1 \mid \forall \sigma_2 \in \Sigma_2, \delta(q, (\sigma_1, \sigma_2)) \subseteq F^*\}$ is a non-empty subset of Σ_1 . Hence, the memoryless strategy, $f_1 : Q \rightarrow 2^{\Sigma_1}$, defined above is a winning strategy for Player 1, and the target set F is said to be reachable from the initial state q_0 if and only if $q_0 \in F^*$.

Example 6.5 Consider the following game structure:



Here, the controllable event set $\Sigma_c = \{u_1, u_2\}$, and the uncontrollable event set $\Sigma_u = \{v_1, v_2\}$. In the game structure, $*$ is used to represent any feasible event. For example, the transition $p \xrightarrow{(u_2, *)} p$ stands for two transitions $p \xrightarrow{(u_2, v_1)} p$ and $p \xrightarrow{(u_2, v_2)} p$, namely no matter what Player 2 chooses, Player 1 can force the self-transition to p if it chooses u_2 .

To solve the reachability game, we compute the states that can reach $F = \{p\}$ through the following iteration $F_{i+1} := F_i \cup \pi(F_i)$, starting with $F_0 = \emptyset$. First,

$$F_1 = F \cup \pi(F_0) = F \cup \pi(\emptyset) = \{p\},$$

since $\pi(\emptyset) = \emptyset$. Next, we increase i by 1 and obtain

$$F_2 = F \cup \pi(F_1) = F \cup \pi(\{p\}) = \{p\} \cup \{p, q\} = \{p, q\},$$

since $\pi(\{p\}) = \{p, q\}$. Next, we compute

$$F_3 = F \cup \pi(F_2) = F \cup \pi(\{p, q\}) = \{p\} \cup \{p, q\} = \{p, q\},$$

which is identical to F_2 , namely $F_3 = F_2$. Hence, we have $F^* = \{p, q\}$. The winning strategy can be deduced as follows:

$$\sigma(p) = u_2, \sigma(q) = u_1.$$

This is a memoryless strategy since it only depends on the current state: If the current state is at p , then trigger u_2 . If the current state is at q , then trigger u_1 instead. $\square$

Similar to the safety game, the reachability game is also deterministic—the set of winning states for Player 1 is given by F^* while the set of winning states for Player 2 is its complement.

6.1.4 Büchi Games

Beyond safety and reachability winning conditions, we can also consider more general winning conditions, such as Büchi winning conditions that can be used to represent ω -regular properties. The winning conditions in a Büchi game are termed as $\square\Diamond$ -conditions.

Definition 6.8 Given a game structure G and a set $F \subseteq Q$, the first player wins a run of the Büchi game, denoted as $\Box\Diamond F$, if some state from F is visited infinitely often by the run; otherwise, it loses the run.

The algorithm for $\Box\Diamond$ -games can be stated as follows:

Algorithm 6.3 (Synthesis for Discrete Büchi Game)

Input: A game structure G and a winning condition $\Box\Diamond F$

Output: Winning states $W^* \subseteq Q$

```

1: Initialization  $W_0 = Q$ ;
2: for  $i = 0, 1, \dots$  do
3:    $H_0 := \emptyset$ 
4:   for  $j = 0, 1, \dots$  do
5:      $H_{j+1} := \pi(H_j) \cup (F \cap \pi(W_i))$ 
6:   until  $H_{j+1} = H_j$ ;
7:    $W_{i+1} := H_j$ 
8: until  $W_{i+1} = W_i$ ;
9: Return  $W^* = W_i$ .
```

The major iteration starts with $W_0 = Q$. For that value of W , we iterate on H to compute the set of states from which a single visit to F can be enforced. For the next major iteration, we take W_1 to be this set. We now recompute H with respect to W_1 which yields the set of states from which one can enforce a visit to F followed by a visit to W_1 (and hence two visits to F). This process constructs a decreasing sequence $W_0, W_1, \dots$, where each W_i represents the set of states from which the plant can be driven to F at least i times.

Example 6.6 Revisit the game structure in Example 6.5 with $F = \{p\}$. To solve the Büchi game, we follow the design algorithm with $W_0 = Q = \{p, q\}$ and $H_0 = \emptyset$ as the initialization.

For the internal iteration, we have

$$H_1 := \pi(H_0) \cup (F \cap \pi(W_0)) = \emptyset \cup (\{p\} \cap \{p, q\}) = \{p\},$$

since $\pi(\emptyset) = \emptyset$ and $\pi(Q) = Q$. Next, we compute H_2 through

$$H_2 := \pi(H_1) \cup (F \cap \pi(W_0)) = \pi(\{p\}) \cup (\{p\} \cap \{p, q\}) = \{p, q\},$$

since $\pi(\{p\}) = \{p, q\}$. It is easy to compute

$$H_3 := \pi(H_2) \cup (F \cap \pi(W_0)) = \pi(\{p, q\}) \cup (\{p\} \cap \{p, q\}) = \{p, q\}.$$

Note that we have $H_2 = H_3$, so we exit the inner loop and set $W_1 = H_2 = \{p, q\}$.

Now we have that $W_1 = W_0$, so we stop the outer-loop iteration as well and return the winning states $W^* = W_1 = \{p, q\}$. The winning strategy can be deduced as follows:

$$\sigma(p) = u_2, \sigma(q) = u_1.$$

Similar to the reachability game, we obtain a memoryless winning strategy. $\square$

6.2 Differential Games

After reviewing discrete games, we will now turn to (continuous) differential games in this section, which will be useful in our study of hybrid games. In particular, we will mainly focus on a (two-player) non-cooperative differential game in the form of the following dynamics:

$$\dot{x}(t) = f(x(t), u_1(t), u_2(t), t), \quad t \in [t_0, t_f], \quad (6.4)$$

with an initial condition $x(t_0) = x_0$, and the control inputs

$$u_1(t) \in \mathcal{U}_1, \quad u_2(t) \in \mathcal{U}_2,$$

for some given sets $\mathcal{U}_1$ and $\mathcal{U}_2$. Certain regularity conditions, such as the Lipschitz conditions, are usually assumed on $f(\cdot, \cdot, \cdot, \cdot)$, to guarantee the existence and uniqueness of a system state trajectory from a given initial condition x_0 and control signals $u_i(\cdot)$ for $i=1,2$.

The setup involves two players, Player 1 and Player 2, who choose control signals u_1 and u_2 , respectively. It is non-cooperative in the sense that each player is just interested in minimizing his/her individual's own cost of the following form:

$$J_i = \int_{t_0}^{t_f} L_i(x(\tau), u_1(\tau), u_2(\tau), \tau) d\tau + \phi_i(x(t_f), t_f), \quad (6.5)$$

where ϕ_i is a terminal payoff, while L_i accounts for a running cost, and $i \in \{1, 2\}$.

To formulate a differential game, it is important to specify what kind of information is available to each player, known as an information structure. First, it is necessary for both players to know the differential dynamics $f(\cdot, \cdot, \cdot, \cdot)$, together with the constraints $\mathcal{U}_1$ and $\mathcal{U}_2$, and the corresponding cost functions J_1 and J_2 . In addition, it is usually assumed that the initial state x_0 and the instantaneous time $t \in [t_0, t_f]$ are available for both players, namely there is a global clock and both players have access to the clock. Furthermore, different information structures may arise depending on whether the current state of the system $x(t)$ is available to each player, whether Player i knows the control signal implemented by the other player, denoted by $u_{-i}(\cdot)$, or whether such information is known but with some time delays. Optimal strategies of each player will depend on the information structure. In this section, we will only focus on two cases as follows:

- (i) only the initial data x_0 and current time t are available;
- (ii) the current state $x(t)$ is also accessible.

Before we try to solve the differential game formulated so far, we note that the above two-player differential game formulation can be easily generalized to a n -player non-cooperative game as follows:

$$\dot{x}(t) = f(x(t), u_1(t), \dots, u_n(t), t), \quad t \in [t_0, t_f], \quad (6.6)$$

with initial condition $x(t_0) = x_0$, and $u_i(t) \in \mathcal{U}_i$, for some given set $\mathcal{U}_i$ and $i \in \{1, 2, \dots, n\}$. Each player P_i wants to minimize a cost of the form

$$J_i = \int_{t_0}^{t_f} L_i(x(\tau), u_1(\tau), \dots, u_n(\tau), \tau) d\tau + \phi_i(x(t_f), t_f). \quad (6.7)$$

If there is only one player ($n = 1$), it becomes an optimal control problem that we studied in Chap. 5. Hence, the optimal control problem is a special case of differential games—a single-player differential game. Actually, solutions for differential games can be obtained by solving coupled optimal control problems—the solution of the differential game would be an optimal control signal $u_i(\cdot)$ for Player i that minimizes the corresponding cost J_i . It is coupled since the state trajectory $x(t)$ is decided jointly by the control input signals from all players, and the running cost of each player may be influenced by other players as well. Nonetheless, the techniques we learned in Chap. 5, for example, the calculus of variations and the minimum principle, can be adopted to solve the differential game as presented in the next subsection.

6.2.1 Open-Loop Strategy

First, we consider the simplest case when the information available to each player only contains the initial state x_0 and the current time instant t . In this setup, the solution of the differential game must be in the form of an open loop, since the optimal strategy $u_i(\cdot)$ for each player should only depend on the initial condition x_0 and the current time instant t . For simplicity, we will use a two-player game to illustrate the main idea. A solution to the differential game is in the following sense.

Definition 6.1 A pair of control functions $(u_1^*(t), u_2^*(t))$ is a Nash equilibrium for the game (6.4)–(6.5) within the class of open-loop strategies if the following holds:

- The control $u_1^*(\cdot)$ provides a solution to the optimal control problem for Player 1:

$$\min_{u_1 \in \mathcal{U}_1} J_1(u_1) = \int_{t_0}^{t_f} L_1(x(\tau), u_1(\tau), u_2^*(\tau), \tau) d\tau + \phi_1(x(t_f), t_f),$$

with respect to

$$\dot{x}(t) = f(x(t), u_1(t), u_2^*(t), t), \quad t \in [t_0, t_f], \quad x(t_0) = x_0.$$

- The control $u_2^*(\cdot)$ provides a solution to the optimal control problem for Player 2:

$$\min_{u_2 \in \mathcal{U}_2} J_2(u_2) = \int_{t_0}^{t_f} L_2(x(\tau), u_1^*(\tau), u_2(\tau), \tau) d\tau + \phi_2(x(t_f), t_f),$$

with respect to

$$\dot{x}(t) = f(x(t), u_1^*(t), u_2(t), t), \quad t \in [t_0, t_f], \quad x(t_0) = x_0.$$

It basically says that, at a Nash equilibrium $(u_1^*(t), u_2^*(t))$, neither player has any intention to change his/her control strategy unilaterally, since nothing can be gained by any divergence from u_i^* (the control u_i^* already minimizes the cost function J_i given the current situation).

To find the Nash equilibrium solutions, we thus need to simultaneously solve two coupled optimal control problems in the sense that the optimal solution $u_1^*(\cdot)$ of the first problem enters as a parameter in the second problem, and vice versa. Applying the Pontryagin minimum principle from Chap. 5, we define the Hamiltonian as follows:

$$\mathcal{H}_1(x^*, u_1, u_2^*, p_1^*, t) = L_1(x^*, u_1, u_2^*, t) + p_1^{*T} f(x^*, u_1, u_2^*, t), \quad (6.8)$$

$$\mathcal{H}_2(x^*, u_1^*, u_2, p_2^*, t) = L_2(x^*, u_1^*, u_2, t) + p_2^{*T} f(x^*, u_1^*, u_2, t). \quad (6.9)$$

The Pontryagin minimum principle tells us that at each time t , the value $u_i^*(t)$ of the optimal control must minimize the corresponding Hamiltonian $\mathcal{H}_i$ with respect to $u_i \in \mathcal{U}_i$, namely

$$u_i^*(t) = \arg \min_{u_i(t) \in \mathcal{U}_i} \mathcal{H}(x^*(t), u_i(t), u_{-i}^*, p_i^*(t), t), \quad (6.10)$$

for $i = 1, 2$, where $x^*(t)$ and $p_i^*(t)$ satisfied the (coupled) canonical equations

$$\dot{x}^*(t) = \frac{\partial \mathcal{H}_i^T}{\partial p_i} (x^*(t), u_i^*(t), u_{-i}^*, p_i^*(t), t), \quad (6.11)$$

$$\dot{p}_i^*(t) = -\frac{\partial \mathcal{H}_i^T}{\partial x} (x^*(t), u_i^*(t), u_{-i}^*, p_i^*(t), t), \quad (6.12)$$

with boundary conditions $x(t_0) = x_0, p_i^T(t_f) = \frac{\partial \phi_i}{\partial x}(x, t) \Big|_{t_f}$. This provides necessary conditions for a Nash equilibrium. Similar to solving optimal control problems, we first derive a form of optimal controls u_i^* from the optimality conditions, and then plug them into the canonical equations. Solving the derived two-point boundary

value problem will give us candidates for the Nash equilibrium. The definition of the Nash equilibrium can be easily extended to n -player games, and similar necessary conditions for n -player games can be obtained as well.

Example 6.7 We consider a two-player differential game following the dynamics

$$\dot{x}(t) = -x(t) + u_1(t) + u_2(t),$$

where Player 1 wants to minimize the quadratic cost function

$$J_1 = \int_0^T (x^2 + u_1^2) dt,$$

while Player 2 tries to minimize the quadratic cost function

$$J_2 = \int_0^T (x^2 + u_2^2) dt.$$

It is assumed that both players only know the initial condition $x(0) = x_0$ and the current time instant, so the solution will be open-loop. Next, we derive necessary conditions for an open-loop Nash equilibrium solution (u_1^*, u_2^*) .

First, the Hamiltonians of Players 1 and 2 are

$$\mathcal{H}_1(x, u_1, p_1, t) = x^2 + u_1^2 + p_1(-x + u_1 + u_2^*), \quad (6.13)$$

$$\mathcal{H}_2(x, u_2, p_2, t) = x^2 + u_2^2 + p_2(-x + u_1^* + u_2), \quad (6.14)$$

where $p_i, i = 1, 2$, denotes the co-state variable of Player i and satisfies the following adjoint equations:

$$\dot{p}_1 = p_1 - 2x, \quad (6.15)$$

$$\dot{p}_2 = p_2 - 2x. \quad (6.16)$$

According to the minimization condition, we have

$$u_1^* = -\frac{1}{2}p_1, \quad u_2^* = -\frac{1}{2}p_2,$$

which are substituted into the state equation and, together with the adjoint equations, we have the following canonical equations:

$$\dot{x} = -x + -\frac{1}{2}p_1 + -\frac{1}{2}p_2, \quad (6.17)$$

$$\dot{p}_1 = p_1 - 2x, \quad (6.18)$$

$$\dot{p}_2 = p_2 - 2x, \quad (6.19)$$

with boundary conditions $x(0) = x_0$, $p_1(T) = 0$, and $p_2(T) = 0$. Although the boundary condition splits, the canonical equations are linear and homogeneous. Therefore, we can solve the canonical equations uniquely, which is left as an exercise. Once we get the solutions for $p_1(t)$ and $p_2(t)$, we can plug in $u_1^*(t) = -\frac{1}{2}p_1(t)$ and $u_2^*(t) = -\frac{1}{2}p_2(t)$ to obtain the open-loop Nash equilibrium solution. $\square$

6.2.2 Closed-Loop Strategy for One-Player Game

Now we turn to the case when the current state $x(t)$ is accessible to both players, for which the optimal strategy will be in the form of state feedback. We will first take a look at a single-player game (optimal control problem) and develop tools for solving two-player differential games under this information structure.

Dynamic Programming

Dynamic programming was developed by R. E. Bellman in the later 1950s and has been successfully used to solve control problems for nonlinear, time-varying systems. Dynamic programming is based on the following key observation: An optimal solution should have the property that from any point on an optimal solution trajectory, the remaining trajectory from that point to the end is necessarily optimal for the corresponding problem initiated at that point. This is known as the *principle of optimality* and can be easily shown by contradiction. Since, otherwise, one could construct a new solution with a smaller total cost, which contradicts the optimality of the original solution.

This argument is illustrated in Fig. 6.3. It is assumed that the optimal path from a to e is through b , that is, the path $a - b - e$ minimizes the cost from a to e , denoted as J_{ae}^* .

The principle of optimality tells us that the segment $b - e$ should be optimal as well (among all feasible paths starting from b and ending at e), namely the segment $b - e$ yields the minimal cost from b to e , denoted as J_{be} . To see this, we assume that there is another path $b - c - e$ that yields a smaller cost than the segment $b - e$, that is $J_{bce} < J_{be}$. If so, we can construct a new path $a - b - c - e$ which gives us a better solution since

$$J_{ab} + J_{bce} < J_{ab} + J_{be} = J_{ae}^*. \quad (6.20)$$

This is in contradiction with the optimality of the path $a - b - e$.

The principle of optimality may seem obvious, but its implication is significant. The principle of optimality helps us to decompose a multi-stage optimization problem into a sequence of single-stage decision making problems, which can dramatically simplify the problem. To see how this works, we illustrate the main idea behind dynamic programming in Fig. 6.4. We assume that our goal is to drive the state trajectory $x(t)$ from the initial state x_0 to the target state x_f and meanwhile minimizing

Fig. 6.3 Illustration of the principle of optimality

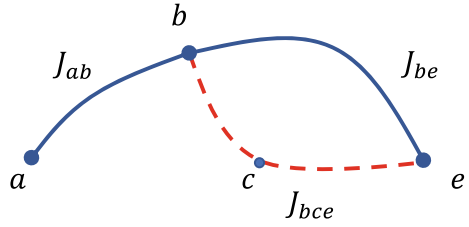
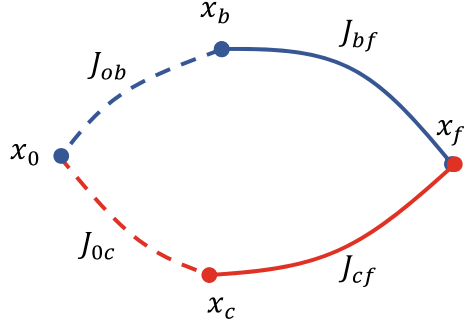


Fig. 6.4 Application of the principle of optimality can decompose a multi-stage optimization problem into a sequence of single-stage decision making problems



a cost function J . For simplicity, both x_0 and x_f are assumed to be fixed. If we know how to drive from states x_b or x_c to x_f with optimal cost $J_{b,f}$ and $J_{c,f}$, respectively, then we only need to worry about how to drive from x_0 to x_b or x_0 to x_c optimally. The overall optimal solution would be the one that gives us a lower cost between $J_{0b} + J_{bf}$ and $J_{0c} + J_{cf}$.

Cost-to-Go Function

Recall that the optimal control aims to minimize the cost functional

$$J(u(\cdot)) = \int_{t_0}^{t_f} L(x(\tau), u(\tau), \tau) d\tau + \phi(x(t_f), t_f), \quad (6.21)$$

with respect to all admissible controls $u \in \mathcal{U}_{ad}$ that drive the initial state x_0 to the final state x_f at time t_f subject to the differential state equation

$$\dot{x}(t) = f(x(t), u(t), t). \quad (6.22)$$

An admissible control $u(\cdot)$ that gives the least value of the cost function J is called an *optimal control* and is denoted as $u^*(\cdot)$. Mathematically, we have

$$J(u^*(\cdot)) = \min_{u \in \mathcal{U}_{ad}} \{J(u(\cdot)) \mid x(t_0) = x_0, x(t_f) = x_f\}. \quad (6.23)$$

Note that the value of $J(u^*(\cdot))$ depends on the initial condition of x_0 and t_0 , and thus we introduce the cost-to-go function. The *cost-to-go function* is defined as the cost with respect to all admissible controls $u \in \mathcal{U}_{ad}$ from any state $x \in \mathbb{R}^n$ at time $t \in [t_0, t_f]$ to the final state $x(t_f) = x_f$. It is described by

$$J(x, t) = \int_t^{t_f} L(x(\tau), u(\tau), \tau) d\tau + \phi(x(t_f), t_f). \quad (6.24)$$

The value function $J^*(x, t)$ is defined as the *optimal cost-to-go function* from time t to final time t_f over the family of admissible control inputs, i.e.,

$$J^*(x, t) := \min_{u \in \mathcal{U}_{ad}} J(x, t) = \min_{u \in \mathcal{U}_{ad}} \int_t^{t_f} L(x(\tau), u(\tau), \tau) d\tau + \phi(x(t_f), t_f). \quad (6.25)$$

An admissible control $v(\cdot)$ is said to solve the optimal cost-to-go function $J^*(x, t)$ if the equation

$$v(\cdot) = \arg \min_{u(\cdot) \in \mathcal{U}_{ad}} \int_t^{t_f} L(x(\tau), u(\tau), \tau) d\tau + \phi(x(t_f), t_f), \quad (6.26)$$

holds, which means that the control $v(\cdot)$ is an optimal control that drives the state trajectory of the system from the state x at time t to the final state x_f at time t_f while minimizing the cost function J . When there is no such admissible control that drives the state x to x_f at time t_f , then we set

$$J^*(x, t) = +\infty.$$

It is clear that the optimal cost-to-go function at $t = t_0$ is actually the value of the cost function for the original optimal control problem, i.e.,

$$J^*(x(t_0), t_0) = J(u^*(\cdot)).$$

Hence, the optimal cost-to-go function is also known as a value function.

Hamilton–Jacobi–Bellman Equation

Now, we give the condition that an optimal cost-to-go function $J^*(x, t)$ needs to satisfy which is known as the *Hamilton–Jacobi–Bellman* (HJB) equation. In the following, we give a heuristic derivation of the HJB equation based on the principle of optimality.

Suppose that the current state is $x(t)$ at time instant t . Then, the cost-to-go from $(x(t), t)$ to $(x(t + \Delta t), t + \Delta t)$ for a very small $\Delta t > 0$ can be written as follows:

$$J(x, t) = \int_t^{t+\Delta t} L(x, u, \tau) d\tau + \int_{t+\Delta t}^{t_f} L(x, u, \tau) d\tau + \phi(x(t_f), t_f). \quad (6.27)$$

Therefore, we have

$$J(x, t) = \int_t^{t+\Delta t} L(x, u, \tau) d\tau + J(x(t + \Delta t), t + \Delta t), \quad (6.28)$$

where $x(t + \Delta t)$ is the state at time $t + \Delta t$ following the state equation $\dot{x} = f(x, u, t)$ from the current state $x(t)$. The term $J(x(t + \Delta t), t + \Delta t)$ stands for the cost-to-go from state $(x(t + \Delta t), t + \Delta t)$.

According to the principle of optimality, the optimal cost-to-go from $(x(t), t)$ should satisfy the following equation:

$$J^*(x, t) = \min_{v(\tau), t < \tau < t + \Delta t} \left[\int_t^{t+\Delta t} L(x, v, \tau) d\tau + J^*(x + \Delta x, t + \Delta t) \right],$$

where $J^*(x + \Delta x, t + \Delta t)$ is the optimal cost-to-go from state $(x + \Delta x, t + \Delta t)$.

Using the Taylor series expansion of $J^*(\cdot, \cdot)$ at (x, t) ,

$$\begin{aligned} & J^*(x, t) \\ = & \min_{v, t < \tau < t + \Delta t} \left[\int_t^{t+\Delta t} L(x, v, \tau) d\tau + J^*(x, t) + \frac{\partial J^*(x, t)}{\partial x} \Delta x + \frac{\partial J^*(x, t)}{\partial t} \Delta t + o(\Delta x, \Delta t) \right], \end{aligned}$$

where $o(\Delta x, \Delta t)$ stands for the high-order terms in Δx and Δt in the Taylor series expansion. Rearranging the equation, we obtain

$$-\frac{\partial J^*(x, t)}{\partial t} \Delta t = \min_{v(\tau), t < \tau < t + \Delta t} \left[L(x, v(t), t) \Delta t + \frac{\partial J^*(x, t)}{\partial x} f(x, u(t), t) \Delta t + o(\Delta x, \Delta t) \right].$$

Here, we approximate the state $x(t + \Delta t)$ by $f(x, u, t) \Delta t$.

Since $\Delta t > 0$, dividing both sides by Δt and taking the limit $\Delta t \rightarrow 0$, we derive the following equation:

$$-\frac{\partial J^*(x, t)}{\partial t} = \min_{v(\cdot)} \left\{ L(x, v(t), t) + \frac{\partial J^*(x, t)}{\partial x} f(x, v(t), t) \right\}. \quad (6.29)$$

Equation (6.29) for the optimal cost-to-go function J^* is called the *Hamilton–Jacobi–Bellman* equation. From Eq. (6.29), the optimal control can be obtained as

$$u^*(t) = \arg \min_{v(\cdot)} \left\{ L(x, v(t), t) + \frac{\partial J^*(x, t)}{\partial x} f(x, v(t), t) \right\}. \quad (6.30)$$

It is worth pointing out that the optimal control is in the form of state feedback and, for the given function J^* , the optimal control law can be derived from the current

state x through the point-wise minimization. Let's take a look at a simple example [46] of the formulation and solving of the HJB equation.

Example 6.8 Consider a control system

$$\dot{x} = x + u, \quad (6.31)$$

and the following cost functional:

$$J = \int_0^1 \frac{1}{4} u^2(t) dt + \frac{1}{4} x^2(1). \quad (6.32)$$

The HJB equation can be formulated as (taking $V^*(x, t)$ as the optimal cost-to-go function)

$$-\frac{\partial V^*(x, t)}{\partial t} = \min_{u(\cdot)} \left\{ \frac{1}{4} u^2(t) + \frac{\partial V^*(x, t)}{\partial x} [x(t) + u(t)] \right\}. \quad (6.33)$$

Denote $\mathcal{H}(x, u, t) = \frac{1}{4} u^2(t) + \frac{\partial V^*(x, t)}{\partial x} [x(t) + u(t)]$, and the minimum of $\mathcal{H}$ reaches at $\frac{\partial \mathcal{H}(x, u, t)}{\partial u} = 0$, which implies

$$\frac{\partial \mathcal{H}(x, u, t)}{\partial u} = \frac{1}{2} u + \frac{\partial V^*(x, t)}{\partial x} = 0. \quad (6.34)$$

Hence the control that minimizes the right-hand side of the HJB equation is

$$\bar{u}(t) = -2 \frac{\partial V^*(x, t)}{\partial x}. \quad (6.35)$$

Plugging $\bar{u}(t)$ into the derived HJB equation, we obtain

$$-\frac{\partial V^*(x, t)}{\partial t} = \frac{1}{4} \left(-2 \frac{\partial V^*(x, t)}{\partial x} \right)^2 + \frac{\partial V^*(x, t)}{\partial x} \left[x(t) - 2 \frac{\partial V^*(x, t)}{\partial x} \right]. \quad (6.36)$$

Hence, we obtain the following partial differential equation:

$$0 = \frac{\partial V^*(x, t)}{\partial t} - \left(\frac{\partial V^*(x, t)}{\partial x} \right)^2 + \frac{\partial V^*(x, t)}{\partial x} x(t), \quad (6.37)$$

with a boundary condition $V^*(x, 1) = \frac{1}{4} x^2(1)$.

We guess that the optimal cost function is of a quadratic form, i.e., $V^*(x, t) = \frac{1}{2} p(t) x^2(t)$, with $p(1) = \frac{1}{2}$. Therefore, $\frac{\partial V^*(x, t)}{\partial x} = p(t) x(t)$, and $\bar{u}(t) = -2 \frac{\partial V^*(x, t)}{\partial x} = -2 p(t) x(t)$. Also, $\frac{\partial V^*(x, t)}{\partial t} = \frac{1}{2} \dot{p}(t) x^2(t)$.

Plugging these terms into the HJB equation, we obtain

$$0 = \frac{1}{2} \dot{p}(t) x^2(t) - p^2(t) x^2(t) + p(t) x^2(t). \quad (6.38)$$

It can be simplified to

$$0 = \frac{1}{2} \dot{p}(t) - p^2(t) + p(t), \quad (6.39)$$

which is an ordinary differential equation with boundary condition $p(1) = \frac{1}{2}$. Solving the differential equation using separation of variables, we get

$$p(t) = \frac{e^{1-t}}{e^{1-t} - e^{t-1}}. \quad (6.40)$$

Therefore, the optimal cost function

$$V^*(x, t) = \frac{e^{1-t}}{2(e^{1-t} - e^{t-1})} x^2(t), \quad (6.41)$$

and the optimal control is given by

$$u^*(t) = \frac{-2e^{1-t}}{e^{1-t} - e^{t-1}} x(t), \quad (6.42)$$

which is a state feedback with time-varying linear gain. $\square$

Actually, we rarely can obtain analytic solutions for the HJB equations. Instead, we usually turn to numerical solutions. The main idea behind numerical solutions of the HJB equations is to “discretize” the continuous optimal control problem into a discrete multi-stage optimization problem. It involves the first discretization in time by picking a proper sampling period, which will convert the continuous-time control system into a discrete-time control system. Next, we also need to “quantize” both the state space x and control input u into finite discrete grids. Then, we can do the value iteration using the grid spanned by x and t backwards starting from the terminal states (x_f, t_f) . During the value iteration, it is possible that a state (x_i, t_i) on the grid may be driven to some points not defined in the grid (somewhere in between some grid points), for which we need to “interpolate” the value of the (x_i, t_i) using the values of neighboring grid points.

Sufficiency

Next, we show that if a continuously differentiable function $\hat{V} : \mathbb{R}^n \times [t_0, t_f] \rightarrow \mathbb{R}$ satisfies the HJB equation

$$-\frac{\partial \hat{V}(x, t)}{\partial t} = \min_{u(\cdot)} \left\{ L(x, u(t), t) + \frac{\partial \hat{V}(x, t)}{\partial x} f(x, u(t), t) \right\}, \quad (6.43)$$

for all $t \in [t_0, t_f]$ and all $x \in \mathbb{R}^n$ and the boundary condition $\hat{V}(x, t_f) = \phi(x, t_f)$, then $\hat{V}(x, t)$ is the optimal cost-to-go from the state x at time t . Hence, $\hat{V}(x_0, t_0)$ is the optimal cost for the optimal control problem, while the control

$$\hat{u}(t) = \arg \min_{u(\cdot)} \left\{ L(\hat{x}, u, t) + \frac{\partial \hat{V}(\hat{x}, t)}{\partial x} f(\hat{x}, u, t) \right\}, \quad (6.44)$$

is the optimal control, where $\hat{x}$ is the corresponding state trajectory with $\hat{x}(t_0) = x_0$ of the control $\hat{u}$.

To see why, we note that $\hat{u}$ and its corresponding $\hat{x}$ minimize the right-hand side of the HJB equation. Therefore, we have

$$-\frac{\partial \hat{V}(\hat{x}, t)}{\partial t} = L(\hat{x}, \hat{u}, t) + \frac{\partial \hat{V}(\hat{x}, t)}{\partial x} f(\hat{x}, \hat{u}, t). \quad (6.45)$$

Note that $\frac{\partial \hat{V}(\hat{x}, t)}{\partial t} + \frac{\partial \hat{V}(\hat{x}, t)}{\partial x} f(\hat{x}, \hat{u}, t)$ forms the total derivative of $\hat{V}(\hat{x}, t)$, so we have

$$0 = L(\hat{x}, \hat{u}, t) + \frac{d}{dt} \hat{V}(\hat{x}, t). \quad (6.46)$$

Integrating the above equality from t_0 to t_f , we have

$$0 = \int_{t_0}^{t_f} L(\hat{x}, \hat{u}, t) dt + \int_{t_0}^{t_f} \frac{d}{dt} \hat{V}(\hat{x}, t) dt, \quad (6.47)$$

$$= \int_{t_0}^{t_f} L(\hat{x}, \hat{u}, t) dt + \hat{V}(\hat{x}(t_f), t_f) - \hat{V}(\hat{x}(t_0), t_0), \quad (6.48)$$

where $\hat{V}(\hat{x}(t_f), t_f) = \phi(\hat{x}(t_f), t_f)$ due to the boundary condition. So, we have

$$\hat{V}(\hat{x}(t_0), t_0) = \int_{t_0}^{t_f} L(\hat{x}, \hat{u}, t) dt + \phi(\hat{x}(t_f), t_f) = J(t_0, x_0, \hat{u}). \quad (6.49)$$

On the other hand, let u be any arbitrary admissible control and x be its corresponding state trajectory with $x(t_0) = x_0$, then

$$-\frac{\partial \hat{V}(x, t)}{\partial t} \leq L(x, u, t) + \frac{\partial \hat{V}(x, t)}{\partial x} f(x, u, t), \quad (6.50)$$

which implies

$$0 \leq L(x, u, t) + \frac{d}{dt} \hat{V}(x, t), \quad (6.51)$$

and therefore,

$$0 \leq \int_{t_0}^{t_f} L(x, u, t) dt + \int_{t_0}^{t_f} \frac{d}{dt} \hat{V}(x, t) dt, \quad (6.52)$$

$$= \int_{t_0}^{t_f} L(x, u, t) dt + \hat{V}(x(t_f), t_f) - \hat{V}(x(t_0), t_0). \quad (6.53)$$

Then, we have

$$\hat{V}(x_0, t_0) \leq \int_{t_0}^{t_f} L(x, u, t) dt + \phi(x(t_f), t_f) = J(t_0, x_0, u). \quad (6.54)$$

Hence, the cost $\hat{V}(x_0, t_0)$ yielded by $\hat{u}$ is lower than any other control u , which means that $\hat{u}$ is an optimal control and $\hat{V}(x, t)$ is an optimal cost functional.

Non-smoothness

Although it was shown that if there exists a continuously differentiable function satisfying the HJB equation then the solution is the optimal cost-to-go function for the optimal control problem, there are optimal control problems where the optimal cost-to-go function is not necessarily continuously differentiable. The following is a well-known example in the optimal control literature that illustrates this point.

Example 6.9 Consider a control system

$$\dot{x} = xu, \quad (6.55)$$

with constraints $-1 \leq u \leq 1$ and the following cost functional:

$$J = x(t_f). \quad (6.56)$$

This is a fixed-time, free end-point problem, and its optimal solution can be easily found by inspection.

- If $x_0 > 0$, then apply $u = -1$ which results in $\dot{x} = -x$, hence the cost is $x(t_f) = e^{-(t_f-t)}x_0$.
- On the other hand, if $x_0 < 0$, then apply $u = 1$ which results in $\dot{x} = x$, hence the cost is $x(t_f) = e^{t_f-t}x_0$.
- If $x_0 = 0$, then $\dot{x} = 0$ for all u and the state remains at the origin resulting in the cost $x(t_f) = 0$.

Therefore, the optimal cost-to-go function is given by

$$V^*(x, t) = \begin{cases} e^{-(t_f-t)}x, & x > 0, \\ e^{t_f-t}x, & x < 0, \\ 0, & x = 0. \end{cases} \quad (6.57)$$

It is clear that $V^*(x, t)$ is not continuously differentiable. Away from the origin, it indeed satisfies the HJB equation

$$-\frac{\partial V^*(x, t)}{\partial t} = \min_{u(\cdot)} \left\{ \frac{\partial V^*(x, t)}{\partial x} x u \right\} = - \left| \frac{\partial V^*(x, t)}{\partial x} x \right|,$$

with the boundary condition $V^*(x, t_f) = x$. □

Therefore, we need a solution in a more general sense, for which a viscosity solution of the HJB equation has been proposed in the literature. It can be shown that the optimal cost-to-go is a unique viscosity solution of the HJB equation, and also of the local Lipschitz equation. Therefore, the HJB equation also provides a necessary condition for optimal solutions. A detailed discussion of viscosity solutions is beyond the scope of this book, and we refer interested readers to books on optimal control theory, e.g., [6, 7], for further details.

6.2.3 State-Feedback Strategy for Two-Player Games

Now we turn to two-player non-cooperative differential games with the information structure that the current state $x(t)$ is available to both players. The solution will be in the form of a state feedback and in the following sense.

Definition 6.2 A pair of control functions $(u_1^*(x, t), u_2^*(x, t))$ is a Nash equilibrium for the game (6.4)–(6.5) within the class of state-feedback strategies if the following holds:

- The control $u_1^*(\cdot, \cdot)$ provides a solution to the optimal control problem for Player 1:

$$\min_{u_1 \in \mathcal{U}_1} J_1(u_1) = \int_{t_0}^{t_f} L_1(x, u_1, u_2^*, \tau) d\tau + \phi_1(x(t_f), t_f),$$

with respect to

$$\dot{x}(t) = f(x, u_1, u_2^*, t), \quad t \in [t_0, t_f], \quad x(t_0) = x_0.$$

- The control $u_2^*(\cdot, \cdot)$ provides a solution to the optimal control problem for Player 2:

$$\min_{u_2 \in \mathcal{U}_2} J_2(u_2) = \int_{t_0}^{t_f} L_2(x, u_1^*, u_2, \tau) d\tau + \phi_2(x(t_f), t_f),$$

with respect to

$$\dot{x}(t) = f(x, u_1^*, u_2, t), \quad t \in [t_0, t_f], \quad x(t_0) = x_0.$$

Similar to the open-loop case, the solution can be obtained by solving two coupled optimal control problems. Based on the dynamic programming techniques developed in the previous subsection, we obtain the following sufficient condition.

Hamilton–Jacobi Equation

State-feedback policies $\{u_1^*, u_2^*\}$ provide a Nash equilibrium, in state-feedback policies, if there exist continuously differentiable functions $\hat{V}_i : \mathbb{R}^n \times [t_0, t_f] \rightarrow \mathbb{R}$ that satisfy the following partial differential equations:

$$-\frac{\partial \hat{V}_i(x, t)}{\partial t} = \min_{u_i \in \mathcal{U}_i} \left\{ L_i(x, u_i, u_{-i}^*, t) + \frac{\partial \hat{V}_i(x, t)}{\partial x} f(x, u_i, u_{-i}^*, t) \right\}, \quad (6.58)$$

for all $t \in [t_0, t_f]$, all $x \in \mathbb{R}^n$, and $i = 1, 2$ with the boundary condition $\hat{V}_i(x, t_f) = \phi_i(x, t_f)$. The corresponding Nash equilibrium cost for Player i of the game is equal to $\hat{V}_i(x_0, t_0)$.

This is known as the Hamilton–Jacobi equation, and the HJ equation in short. We will see a simple example to illustrate the formulation and solving of the HJ equations.

Example 6.10 Let’s revisit Example 6.7 for a two-player differential game with dynamics

$$\dot{x}(t) = -x(t) + u_1(t) + u_2(t),$$

where the Player 1 wants to minimize the quadratic cost function

$$J_1 = \int_0^T (x^2 + u_1^2) dt,$$

while the Player 2 tries to minimize the quadratic cost function

$$J_2 = \int_0^T (x^2 + u_2^2) dt.$$

Let’s assume now that both players not only know the initial condition $x(0) = x_0$ but also the current state $x(t)$. So the solution will be in a state-feedback form, and we will derive conditions for a closed-loop Nash equilibrium solution (u_1^*, u_2^*) this time. The HJ equations are as follows:

$$-\frac{\partial V_1(x, t)}{\partial t} = \min_{u_1} \left\{ x^2 + u_1^2 + \frac{\partial V_1(x, t)}{\partial x} (-x + u_1 + u_2^*) \right\}, \quad (6.59)$$

$$-\frac{\partial V_2(x, t)}{\partial t} = \min_{u_2} \left\{ x^2 + u_2^2 + \frac{\partial V_2(x, t)}{\partial x} (-x + u_1^* + u_2) \right\}, \quad (6.60)$$

with boundary conditions $V_1(x, T) = 0$ and $V_2(x, T) = 0$.

Minimizing the righthand side of the above HJ equations, we have

$$u_1^* = -\frac{1}{2} \frac{\partial V_1(x, t)}{\partial x}, \quad u_2^* = -\frac{1}{2} \frac{\partial V_2(x, t)}{\partial x}.$$

We postulate a quadratic value function of the form

$$V_i(x, t) = q_i(t)x^2(t),$$

where $q_i(t)$, $i = 1, 2$, is a function that needs to be determined. Note that

$$\frac{\partial V_i(x, t)}{\partial x} = 2q_i(t)x(t),$$

then

$$u_i^* = -q_i(t)x(t),$$

for $i = 1, 2$, which is in a state-feedback form. Substituting $u_i^* = -q_i(t)x(t)$ into the HJ equations, we have

$$-\dot{q}_i x^2 - 2q_i x \dot{x} = x^2 + q_i^2 x^2 + 2q_i x (-x - q_1 x - q_2 x), \quad (6.61)$$

for $i = 1, 2$. Plug in $\dot{x} = -x - q_1 x - q_2 x$, then

$$-\dot{q}_i x^2 - 2q_i (-1 - q_1 - q_2) x^2 = x^2 + q_i^2 x^2 + 2q_i (-1 - q_1 - q_2) x^2.$$

Canceling x^2 since $x(t)$ is not constant zero, we have

$$-\dot{q}_i = 1 + q_i^2 + 4q_i (-1 - q_1 - q_2),$$

for $i = 1, 2$ and with boundary conditions $q_1(T) = 0$ and $q_2(T) = 0$. Solving these two coupled ordinary differential equations, we can obtain $q_1(t)$ and $q_2(t)$, and therefore, the feedback control

$$u_i^* = -q_i(t)x(t).$$

The details are left as exercises. □

6.2.4 Two-Player Zero-Sum Games

To gain more insight, we take a look at a special case of the two-player differential games that we defined so far. Consider a two-player differential game with dynamics of the form

$$\dot{x}(t) = f(x(t), u_1(t), u_2(t), t), \quad t \in [t_0, t_f], \quad (6.62)$$

with an initial condition $x(t_0) = x_0$, and $u_1(t) \in \mathcal{U}_1$, $u_2(t) \in \mathcal{U}_2$ for some given set $\mathcal{U}_1$ and $\mathcal{U}_2$.

The cost is of the form

$$J = \int_{t_0}^{t_f} L(x(\tau), u_1(\tau), u_2(\tau), \tau) d\tau + \phi(x(t_f), t_f), \quad (6.63)$$

where ϕ is a terminal payoff, while L accounts for a running cost. It is zero-sum when $J_1 = J$ while $J_2 = -J$, namely the gain of one player is the loss of the other player. Hence, Player 1 wants to minimize J while Player 2 wants to maximize J . A two-player zero-sum game is a special case of a two-player non-cooperative game, and the Nash equilibrium of a zero-sum game has a special form too.

Definition 6.3 A saddle point is defined as a pair of strategies (u_1^*, u_2^*) achieving

$$J(u_1^*, u_2) \leq J(u_1^*, u_2^*) \leq J(u_1, u_2^*),$$

for all feasible u_1 and u_2 . For such strategies, (u_1^*, u_2^*) , $J(u_1^*, u_2^*)$ is called to be the value of the game.

Intuitively, for a saddle-point solution (u_1^*, u_2^*) , neither player can benefit from any unilateral move. A saddle point is, therefore, a special case of a Nash equilibrium. Saddle-point solutions can be obtained by solving a coupled optimal control problem, and we can talk about different information structures as well. For open-loop strategies, we can apply the obtained necessary conditions from minimum principles without much difficulty since a saddle point is also a Nash equilibrium. Therefore we turn our attention to solutions in a state-feedback form. For a two-player zero-sum game, the HJ equation for non-cooperative games from the previous subsection can be further written in the following form.

Hamilton–Jacobi–Isaacs (HJI) Equation

If a continuously differentiable function $\hat{V} : \mathbb{R}^n \times [t_0, t_f] \rightarrow \mathbb{R}$ satisfies the following partial differential equation:

$$-\frac{\partial \hat{V}(x, t)}{\partial t} = \min_{u_1 \in \mathcal{U}_1} \max_{u_2 \in \mathcal{U}_2} \left\{ L(x, u_1, u_2, t) + \frac{\partial \hat{V}(x, t)}{\partial x} f(x, u_1, u_2, t) \right\}, \quad (6.64)$$

$$= \max_{u_2 \in \mathcal{U}_2} \min_{u_1 \in \mathcal{U}_1} \left\{ L(x, u_1, u_2, t) + \frac{\partial \hat{V}(x, t)}{\partial x} f(x, u_1, u_2, t) \right\}, \quad (6.65)$$

for all $t \in [t_0, t_f]$ and all $x \in \mathbb{R}^n$ and the boundary condition $\hat{V}(x, t_f) = \phi(x, t_f)$, then the pair of policies defined as follows is a saddle-point equilibrium in state-feedback policies:

$$u_1^*(x, t) = \arg \min_{u_1 \in \mathcal{U}_1} \max_{u_2 \in \mathcal{U}_2} \left\{ L(x, u_1, u_2, t) + \frac{\partial \hat{V}(x, t)}{\partial x} f(x, u_1, u_2, t) \right\}, \quad (6.66)$$

$$u_2^*(x, t) = \arg \max_{u_2 \in \mathcal{U}_2} \min_{u_1 \in \mathcal{U}_1} \left\{ L(x, u_1, u_2, t) + \frac{\partial \hat{V}(x, t)}{\partial x} f(x, u_1, u_2, t) \right\}. \quad (6.67)$$

Moreover, the value of the game is equal to $\hat{V}(x_0, t_0)$.

6.2.5 Pursuit–Evasion Games

In this subsection, as an example, we illustrate how to solve HJI equations using a pursuit–evasion game, which can be seen as a special case of two-player zero-sum game. In particular, the state vector $x(t)$ in a pursuit–evasion game can be decomposed into two distinct components $x_1(t)$ and $x_2(t)$, associated with each player. Accordingly, the dynamics of the players are typically in a decoupled form

$$\begin{cases} \dot{x}_1(t) = f_1(x_1, u), & x_1(t) \in \mathbb{R}^{n_1}, \quad u \in \mathcal{U}, \\ \dot{x}_2(t) = f_2(x_2, d), & x_2(t) \in \mathbb{R}^{n_2}, \quad d \in \mathcal{D}, \end{cases} \quad (6.68)$$

for $t \geq 0$. The outcome of such games is determined by a (closed) capture set $\mathcal{C} \subset \mathbb{R}^{n_1} \times \mathbb{R}^{n_2}$. We say that the pursuer P_1 captures the evader P_2 at the first time t_f at which $x(t)$ enters the capture set $\mathcal{C}$. In case $x(t)$ never enters $\mathcal{C}$, the capture time t_f is $+\infty$. The pursuer P_1 wants to minimize an integral cost with variable horizon

$$J = \int_0^{t_f} L(x(\tau), u(\tau), d(\tau), \tau) d\tau + \phi(x(t_f), t_f), \quad (6.69)$$

while the evader wants to maximize it.

Example 6.11 ([8]) Consider a pursuer P_1 that moves in the plane with maximum velocity v_1 and an evader P_2 that also moves in the plane, but with maximum velocity $v_2 < v_1$. The dynamics of this game are given by

$$\begin{cases} \dot{x}_1 = u, & x_1 \in \mathbb{R}^2, \quad \|u\| \leq v_1, \\ \dot{x}_2 = d, & x_2 \in \mathbb{R}^2, \quad \|d\| \leq v_2, \end{cases} \quad (6.70)$$

where x_1 stands for the state of the pursuer and x_2 stands for the state of the evader. The pursuit is over when $x_1 = x_2$ and therefore the capture set is given by

$$\mathcal{C} = \{(x_1, x_2) \in \mathbb{R}^4 : x_1 = x_2\}.$$

We assume that the pursuer wants to minimize the capture time

$$J := \int_0^{t_f} dt,$$

whereas the evader P_2 wants to maximize it.

The HJI equation of this game is

$$\begin{aligned} -\frac{\partial V(x_1, x_2, t)}{\partial t} &= \min_{u \in \mathcal{U}} \max_{d \in \mathcal{D}} \left\{ 1 + \frac{\partial V(x_1, x_2, t)}{\partial x_1} u + \frac{\partial V(x_1, x_2, t)}{\partial x_2} d \right\} \\ &= \max_{d \in \mathcal{D}} \min_{u \in \mathcal{U}} \left\{ 1 + \frac{\partial V(x_1, x_2, t)}{\partial x_1} u + \frac{\partial V(x_1, x_2, t)}{\partial x_2} d \right\} \\ &= 1 - v_1 \left\| \frac{\partial V(x_1, x_2, t)}{\partial x_1} \right\| + v_2 \left\| \frac{\partial V(x_1, x_2, t)}{\partial x_2} \right\|, \end{aligned}$$

for all $t \geq 0$ and all $x \in \mathbb{R}^4$ and along with the following boundary condition:

$$V(x_1, x_2, t_f) = 0, \quad \forall t \geq 0, \quad x_1 = x_2 \in \mathbb{R}^2.$$

Intuitively, the time to capture should be a linear function of the initial distance between the pursuer and evader, so the solution to this HJI equation should be of the form

$$V(x_1, x_2, t_f) = \alpha \|x_1 - x_2\| = \alpha \left((x_1 - x_2)^T (x_1 - x_2) \right)^{\frac{1}{2}},$$

for some appropriate constant α (to be determined). This function trivially satisfies the boundary condition and since

$$\frac{\partial V(x_1, x_2, t)}{\partial x_1} = \frac{\alpha(x_1 - x_2)}{\|x_1 - x_2\|}, \quad \frac{\partial V(x_1, x_2, t)}{\partial x_2} = \frac{\alpha(x_2 - x_1)}{\|x_1 - x_2\|},$$

when we substitute V into the HJI equation, we obtain

$$0 = 1 - \alpha v_1 + \alpha v_2,$$

which implies

$$\alpha = \frac{1}{v_1 - v_2} > 0.$$

From this, we conclude that the value of this game is given by

$$V(0, x_1(0), x_2(0)) = \frac{\|x_1(0) - x_2(0)\|}{v_1 - v_2}.$$

To compute the saddle-point state-feedback policies, we need to find the values u^* and d^* that achieve the outer minimum in HJI equation and the outer maximum in HJI equation, respectively, which turn out to be

$$u^*(t) = -v_1 \frac{\frac{\partial V(x_1, x_2, t)}{\partial x_1}}{\left\| \frac{\partial V(x_1, x_2, t)}{\partial x_1} \right\|} = \frac{v_1(x_2 - x_1)}{\|x_1 - x_2\|}, \quad (6.71)$$

$$d^*(t) = v_2 \frac{\frac{\partial V(x_1, x_2, t)}{\partial x_2}}{\left\| \frac{\partial V(x_1, x_2, t)}{\partial x_2} \right\|} = \frac{v_2(x_2 - x_1)}{\|x_1 - x_2\|}. \quad (6.72)$$

This makes sense as it says that the best way to escape is to run at the maximum speed away from the pursuer, i.e., along the direction $x_2 - x_1$. On the other hand, the best option for the pursuer is to approach the evader at the maximum speed along the direction $x_2 - x_1$. $\square$

This section gave a very brief introduction to the differential game theory as a parallel to the discrete games. Interested readers may refer to books on differential games, such as [8–10].

6.3 Unifying Discrete Games and Differential Games

Careful readers may observe the difference between the discrete games and the (continuous) differential games—discrete games are with respect to qualitative specifications (like safety, reachability) while differential games (and the mainstream game theory) are with respect to quantitative costs. This section attempts to unify these two types of games and can be skipped in the first round of reading. The material of this section is mainly based on lecture notes [11].

6.3.1 Solving Discrete Games by Value Iteration

First, we solve discrete games from a quantitative perspective, for which we use a safety game to illustrate the ideas. Let's recall that a safety game is formulated as follows. For a game structure $G = (Q, \Sigma, q_0, \delta)$ and a safe region $F \subset Q$, the safety

game $\Box F$ aims to have the state remain in F by properly selecting controllable transitions $\Sigma_1 \subset \Sigma$. For convenience, we restate the game for the safety game solution below. Note that the only difference in this version of the safety game algorithm is that the iteration stage i is negative and we search backwards.

Algorithm 6.4 (Synthesis for Safety Games $\Box F$)

Input: A game structure G and a winning condition $\Box F$

Output: Winning states $F^* \subseteq Q$

1: Initialization $F_0 = F$;

2: **while** $i \leq 0$ **do**

3: $F_{i-1} := F_i \cap \pi(F_i)$

4: **if** $F_{i-1} == F_i$ **then**

5: **break**;

6: $i = i - 1$

7: **Return** $F^* = F_i$.

Our main idea is to associate a value function, $J(q, i) : Q \times \mathbb{Z}_{\leq 0} \rightarrow \{0, 1\}$, with each state q at each iteration, representing the future reward or cost to be incurred by the system given that its current state is q and at the i th iteration,

$$J(q, i) = \begin{cases} 1, & q \in F_i, \\ 0, & q \notin F_i. \end{cases}$$

Note that $F_0 = F$, and $J(q, 0) = 1$ for all states $q \in F$, and $J(q, 0) = 0$ otherwise.

Next, the following value iteration will be used to compute $J(q, i)$ from $J(q, i - 1)$:

$$J(q, i - 1) - J(q, i) = \min \left\{ 0, \max_{\sigma_1 \in \Sigma_1} \min_{\sigma_2 \in \Sigma_2} \left[\min_{q' \in \delta(q, \sigma_1, \sigma_2)} J(q', i) - J(q, i) \right] \right\}, \quad (6.73)$$

which is a discrete version of the Hamilton–Jacobi equation for a two-player zero-sum discrete dynamic game.

In the following, we will show by induction that the set,

$$F_i = \left\{ q \in Q \mid J(q, i) = 1 \right\},$$

contains exactly those states q that starting from q the controller can make the state trajectory remain inside F for at least $|i|$ number of transitions.

First, this is the case for $i = 0$, since $F_0 = F$, and $J(q, 0) = 1$ for all states $q \in F$, and $J(q, 0) = 0$ otherwise. To prove for the general case, we assume that the claim is true for i , namely $F_i = \left\{ q \in Q \mid J(q, i) = 1 \right\}$. All we need to show is that the set $\left\{ q \in Q \mid J(q, i - 1) = 1 \right\}$ contains exactly those states from which the controller can maintain the state trajectory inside F for at least $|i| + 1$ number of transitions no matter what the disturbance σ_2 does.

For this, we first show that $\{q \in Q \mid J(q, i-1) = 1\} \subseteq F_{i-1}$. For any state q whose value $J(q, i-1) = 1$ can only happen if $J(q, i) = 1$ since the right-hand side of the discrete HJ equation is less or equal to zero, namely

$$J(q, i-1) - J(q, i) \leq 0 \implies J(q, i-1) \leq J(q, i).$$

So, we have (by plugging in $J(q, i) = 1$ and $J(q, i-1) = 1$ into the discrete HJ equation)

$$0 = \min \left\{ 0, \max_{\sigma_1 \in \Sigma_1} \min_{\sigma_2 \in \Sigma_2} \left[\min_{q' \in \delta(q, \sigma_1, \sigma_2)} J(q', i) - 1 \right] \right\},$$

which is possible only if

$$\max_{\sigma_1 \in \Sigma_1} \min_{\sigma_2 \in \Sigma_2} \left[\min_{q' \in \delta(q, \sigma_1, \sigma_2)} J(q', i) \right] = 1.$$

This means that $q' \in F_i$. Therefore, all transitions from q will remain inside F_i after one step. The controller can, therefore, keep the state evolution (starting from q) inside F for at least $|i| + 1$ transitions, namely $q \in F_{i-1}$.

Next, we show that $F_{i-1} \subseteq \{q \in Q \mid J(q, i-1) = 1\}$. Assume this is not true, then there must exist a state $q \in F_{i-1}$ but $J(q, i-1) = 0$. Since $F_{i-1} \subseteq F_i$ (this is because $J(q, i-1) \leq J(q, i)$), we have $q \in F_i$ so $J(q, i) = 1$ by assumption. Then, the discrete HJ equation becomes

$$-1 = \max_{\sigma_1 \in \Sigma_1} \min_{\sigma_2 \in \Sigma_2} \left[\min_{q' \in \delta(q, \sigma_1, \sigma_2)} J(q', i) - 1 \right], \quad (6.74)$$

which implies that $\max_{\sigma_1 \in \Sigma_1} \min_{\sigma_2 \in \Sigma_2} \left[\min_{q' \in \delta(q, \sigma_1, \sigma_2)} J(q', i) \right] = 0$, which means that there exist one transition from q that drive it to a state q' outside of F_i . From q' , it can stay in F for at most $|i|$ transitions, which means that q can be driven outside of F with at most $|i| + 1$ transitions. This is a contradiction to the assumption that $q \in F_{i-1}$. Therefore, the set, $F_i = \{q \in Q \mid J(q, i) = 1\}$, contains exactly those states q that starting from q the controller can keep the state trajectory remain inside F for at least $|i|$ number of transitions.

In summary, we proved the following result.

Theorem 6.1 *If $J(q, i) : Q \times \mathbb{Z}_{\leq 0} \rightarrow \{0, 1\}$ satisfies the discrete HJ equation*

$$J(q, i-1) - J(q, i) = \min \left\{ 0, \max_{\sigma_1 \in \Sigma_1} \min_{\sigma_2 \in \Sigma_2} \left[\min_{q' \in \delta(q, \sigma_1, \sigma_2)} J(q', i) - J(q, i) \right] \right\}, \quad (6.75)$$

for $i \leq 0$ and with a boundary condition $J(q, 0) = 1$ if $q \in F$ and $J(q, 0) = 0$ otherwise, then the set

$$F_i = \{q \in Q \mid J(q, i) = 1\},$$

contains exactly those states q that starting from q the controller can keep the state trajectory remaining inside F for at least $|i|$ number of transitions.

Note that it is clear that

$$F = F_0 \supseteq F_{-1} \supseteq F_{-2} \supseteq \cdots \supseteq F_i \supseteq F_{i-1} \supseteq \cdots. \quad (6.76)$$

Intuitively, for any state q in F_{i-1} , it can remain in F for at least $|i| + 1$ transitions, so $q \in F_i$ as well, and hence $F_{i-1} \subseteq F_i$. Furthermore, the set difference $F_{i-1} \setminus F_i$ contains exactly the states in F_i that can be forced to leave F (by Player 2) after $|i|$ number of transitions.

For finite Q and Σ , a fixed point $J^*(q)$ of the discrete HJ equation is reached in a finite number of steps, whose existence and uniqueness are guaranteed by Tarski's fixed-point theorem. The set of winning states for the controller is

$$F^* = \left\{ q \in Q \mid J^*(q) = 1 \right\},$$

and F^* is the largest controlled invariant subset of F .

Finally, let's see why we need the $\min\{0, \cdot\}$ on the right-hand side of the HJ equation. Consider the case when $q \notin F_i$, equivalently $J(q, i) = 0$, but there exists σ_1 to drive q into F_i in one transition no matter what σ_2 does, i.e., $\exists \sigma_1, \forall \sigma_2, \delta(q, \sigma_1, \sigma_2) \subseteq F_i$. Therefore,

$$\max_{\sigma_1 \in \Sigma_1} \min_{\sigma_2 \in \Sigma_2} \left[\min_{q' \in \delta(q, \sigma_1, \sigma_2)} J(q', i) - J(q, i) \right] = 1 - 0 = 1.$$

If we do not have $\min\{0, \cdot\}$ on the right-hand side, we will mistakenly update $J(q, i - 1) - J(q, i) = 1 \implies J(q, i - 1) = 1$, which implies $q \in F_{i-1}$. Therefore, the addition of $\min\{0, \cdot\}$ will make sure that $J(q, i - 1) \leq J(q, i)$, namely once its value becomes zero for a state q (unsafe), it won't be marked safe in the future (remains as zero forever).

6.3.2 Differential Safety Games

Next, we take a look at how to solve a safety control problem (a qualitative specification) for a continuous control system through differential games. Consider a continuous control system

$$\dot{x}(t) = f(x(t), u(t), d(t), t), \quad (6.77)$$

where $x \in \mathbb{R}^n, u(t) \in \mathcal{U}, d(t) \in \mathcal{D}$ for some given compact sets $\mathcal{U} \subseteq \mathbb{R}^u$ and $\mathcal{D} \subseteq \mathbb{R}^d$.

Given a set of states $F \subseteq \mathbb{R}^n$, our goal is to keep the state $x(t)$ inside F for all allowed disturbances d . Namely, we would like to compute the maximum controlled invariant subset of F , which is denoted by

$$F^* = \left\{ x_0 \in \mathbb{R}^n \mid \exists u \in \mathcal{U}, \forall d \in \mathcal{D}, \forall t \geq 0, \phi(x_0, u, d, t) \in F \right\}.$$

Here we use $\phi(x_0, u, d, t)$ to represent the state trajectory of (6.77) from the initial condition x_0 , under the control signal u and disturbance d . This is actually the continuous counterpart of the discrete safety game and therefore is called a differential safety game.

The algorithm for discrete safety games can be generalized for differential safety games conceptually. In analogy to F_i in the discrete game, we define the set F_t for $t \geq 0$

$$F_t = \left\{ x_0 \in \mathbb{R}^n \mid \exists u \in \mathcal{U}, \forall d \in \mathcal{D}, \forall 0 \leq s \leq t, \phi(x_0, u, d, s) \in F \right\},$$

and $F_0 = F$. The set F_t contains the states starting from which the controller can keep the state trajectory inside F for at least a time interval of length t no matter what the disturbance does. We can see F^* as a limit of F_t as $t \rightarrow \infty$.

We assume that there exists a continuously differentiable function $l : \mathbb{R}^n \rightarrow \mathbb{R}$ such that $l(x) < 0$ for $x \in \text{int}\{F\}$ (interior of F), $l(x) = 0$ for $x \in \partial F$ (on the boundary of F), and $l(x) > 0$ for $x \in F^c$ (outside of F). It is also assumed that the normal vector $\frac{\partial l}{\partial x}(x) \neq 0$ for all $l(x) = 0$.

The idea is to recast the computation of the maximum controlled invariant subset as a two-player zero-sum differential game with the following cost:

$$J := l(x(t_f)).$$

Note that it only contains a terminal cost $l(\cdot)$ without any running cost. Also, the terminal time t_f is free. The controller tries to minimize the terminal cost (keep $x(t_f)$ as close as possible to F), while the disturbance wants to maximize the terminal cost. By introducing a variable change $t - t_f$, we can fix the terminal time at 0, while keeping the initial time free (denoted as $\tau \leq 0$).

Define the value function as

$$V(x, t) = \min_{u \in \mathcal{U}_{[t, 0]}} \max_{d \in \mathcal{D}_{[t, 0]}} l(x(0)).$$

The definition of the value function actually gives advantages to the disturbances, since the controller works against the worst case of all possible disturbances.

Based on the principle of optimality, we can derive the following condition: If a continuously differentiable function $V : \mathbb{R}^n \times [t, 0] \rightarrow \mathbb{R}$ satisfies the following partial differential equation:

$$-\frac{\partial V}{\partial t}(x, t) = \min_{u \in \mathcal{U}} \max_{d \in \mathcal{D}} \left\{ \frac{\partial V}{\partial x}(x, t) f(x, u, d, t) \right\}, \quad (6.78)$$

for $t < 0$ and with a boundary condition $V(x, 0) = l(x)$, then

$$\hat{F}_\tau = \{x_0 \mid V(x_0, \tau) \leq 0\},$$

represents the set of states for which starting from $x(\tau) = x_0$, there exists a controller for u such that for all disturbances d , the terminal cost of the corresponding trajectory $l(x(0)) \leq 0$, which means that $x(0) \in F$.

However, this is not exactly what we need, since it may contain those states that leave F at some point but re-enter F before the final time. Instead we would like to characterize the set of states for which there exists a u such that for all d and for all $t \in [\tau, 0]$, $x(t) \in F$. This requirement can be encoded either after the computation of V or by modifying the Hamilton–Jacobi equation:

$$-\frac{\partial V}{\partial t}(x, t) = \min \left\{ 0, \mathcal{H}\left(x, \frac{\partial V}{\partial x}(x, t)\right) \right\}, \quad (6.79)$$

for $t < 0$ and with the boundary condition $V(x, 0) = l(x)$. Here the Hamiltonian is defined as

$$\mathcal{H}\left(x, \frac{\partial V}{\partial x}(x, t)\right) = \min_{u \in \mathcal{U}} \max_{d \in \mathcal{D}} \left\{ \frac{\partial V}{\partial x}(x, t) f(x, u, d, t) \right\}.$$

With the modified HJ equation, we now can state the main result as follows:

Theorem 6.2 *If a continuously differentiable function $V : \mathbb{R}^n \times [t, 0] \rightarrow \mathbb{R}$ satisfies the modified HJ equation*

$$-\frac{\partial V}{\partial t}(x, t) = \min \left\{ 0, \min_{u \in \mathcal{U}} \max_{d \in \mathcal{D}} \left[\frac{\partial V}{\partial x}(x, t) f(x, u, d, t) \right] \right\}, \quad (6.80)$$

for $t < 0$ and with the boundary condition $V(x, 0) = l(x)$, then

$$F_\tau = \{x_0 \mid V(x_0, \tau) \leq 0\},$$

represents the set of states for which starting from $x(t) = x_0$, there exists a control u such that for all disturbances d the whole state trajectory stays inside F , namely $x(t) \in F$ for all $t \in [\tau, 0]$.

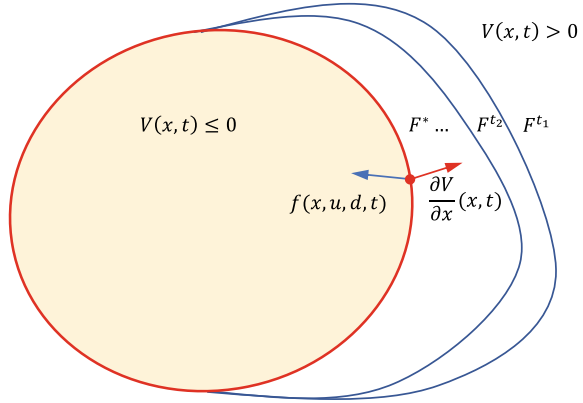
Proof To see why, we will show both $\{x \in \mathbb{R}^n \mid V(x, \tau) \leq 0\} \subseteq F_\tau$ and $F_\tau \subseteq \{x \in \mathbb{R}^n \mid V(x, \tau) \leq 0\}$ hold true. We will prove this by contradiction.

First, for x satisfies $V(x, \tau) \leq 0$, we show that x remains in F for at least $-\tau$. Assume the opposite case, then there must exist $t \in [\tau, 0]$ such that $V(x, t) = 0$, i.e., at the boundary of $\{x \in \mathbb{R}^n \mid V(x, t) \leq 0\}$. Due to the HJ equation

$$-\frac{\partial V}{\partial t}(x, t) = \min \left\{ 0, \min_{u \in \mathcal{U}} \max_{d \in \mathcal{D}} \left[\frac{\partial V}{\partial x}(x, t) f(x, u, d, t) \right] \right\},$$

we have

Fig. 6.5 Pictorial illustration of the level sets of the value function $V(x, t)$ and the ideas behind the proof of Theorem 6.2



$$\min_{u \in \mathcal{U}} \max_{d \in \mathcal{D}} \left[\frac{\partial V}{\partial x}(x, t) f(x, u, d, t) \right] < 0.$$

Otherwise, $\frac{\partial V}{\partial t}(x, t) = 0$, which implies $V(x, t') = 0$ for all $t' \in [t, 0]$, i.e., the value of V will be frozen from this point. The condition

$$\min_{u \in \mathcal{U}} \max_{d \in \mathcal{D}} \left[\frac{\partial V}{\partial x}(x, t) f(x, u, d, t) \right] < 0,$$

means that no matter what the disturbance does, the controller can drag the state back to the set $\{x \mid V(x, t) \leq 0\}$ again (as illustrated in Fig. 6.5). Hence, it is impossible for the trajectory to cross the boundary of the level set of V , which acts very similarly to a Barrier certificate. Also, $\{x \mid V(x, t) \leq 0\} \subseteq \{x \mid V(x, 0) \leq 0\} = F$, since $\frac{\partial V}{\partial t}(x, t) \geq 0$ and $t \leq 0$. This contradicts the assumption that the state trajectory starting from $\{x \in \mathbb{R}^n \mid V(x, \tau) \leq 0\}$ can leave F within a period of length $-\tau$. This actually shows that $\{x \in \mathbb{R}^n \mid V(x, \tau) \leq 0\} \subseteq F_\tau$.

To prove that the reverse also holds, namely $F_t \subseteq \{x \in \mathbb{R}^n \mid V(x, t) \leq 0\}$, we consider a point $x \in F_t$, but $V(x, t) > 0$. Since $F_t \subseteq F$, $V(x, 0) = l(x) \leq 0$. Then, we have $V(x, t) > V(x, 0)$, which contradicts the fact that $V(x, t)$ is non-decreasing with respect to time t (since $\frac{\partial V}{\partial t}(x, t) \geq 0$ and $t \leq 0$).

Some remarks are in order. First, due to the fact that $\frac{\partial V}{\partial t}(x, t) \geq 0$, we have $V(x, 0) \geq V(x, t_1) \geq V(x, t_2)$ for $t_2 \leq t_1 \leq 0$. Also, notice that $V(x, 0) = l(x)$, hence

$$\{x \mid V(x, 0) \leq 0\} = \{x \mid l(x) \leq 0\} = F,$$

and

$$F^* \subseteq \dots \subseteq F_{t_2} \subseteq \dots \subseteq F_{t_1} \subseteq \dots \subseteq F_0 = F,$$

for $t_2 \leq t_1 \leq 0$. This is illustrated in Fig. 6.5.

Second, let's assume that $V^*(x, t)$ satisfies the modified HJ equation (6.80) for all t , and that it converges uniformly in x as $t \rightarrow -\infty$ to a function $V^*(x)$. Then the set

$$F^* = \left\{ x \in \mathbb{R}^n \mid V^*(x) \leq 0 \right\},$$

is the largest controlled invariant set contained in F . Furthermore, the least restrictive controller that renders F^* invariant is given as

$$g(x) = \begin{cases} \left\{ u \in \mathcal{U} \mid \min_d \frac{\partial V}{\partial x}(x, t) f(x, u, d, t) \geq 0 \right\}, & x \in \partial F^*, \\ \mathcal{U}, & x \in (F^*)^c \cup \text{int}(F^*). \end{cases} \quad (6.81)$$

It basically drags the trajectory back into F^* once it is at the boundary (always possible), while gives a maximum permission if it is inside F^* . If the initial state starts inside F^* , the given control law will guarantee the safety since the whole state trajectory will remain in F^* . However, if the initial state is outside of F^* , it is always possible for disturbances to drive it outside of F eventually, so in a sense we give up on this case by allowing the maximum permissiveness on the control signals. Also, it is worth pointing out that the control law is memoryless and in a form of state feedback, which is similar to the case of discrete safety games.

6.4 Timed Games

So far, we have reviewed design algorithms for discrete games with different winning conditions and differential games with different information structures. These ideas and design procedures can be extended to hybrid systems, in which the players have both discrete and continuous moves. The game proceeds in a sequence of rounds. In every round, each player either chooses to take a discrete transition (discrete move) provided that the guard condition is satisfied or chooses to stay at the current discrete mode and let time elapse (continuous move). If one player chooses a discrete move, then the variables are updated and no time elapses. If all players choose continuous moves, then the variables evolve according to the flow condition for some non-zero time duration. In this section, we will mainly use timed games to illustrate the key concepts and elements for hybrid games.

6.4.1 Timed Game Automata

A timed game automaton is a timed automaton whose set of events $\mathcal{E}$ is partitioned into two disjoint sets $\mathcal{E}^c$ (the controllable events) and $\mathcal{E}^u$ (the uncontrollable events). Formally, timed game automata can be defined as follows.

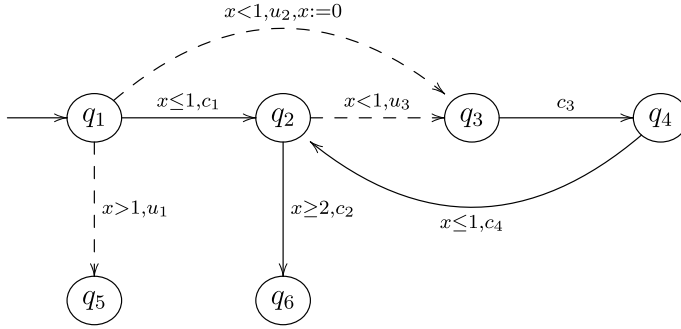


Fig. 6.6 Example timed game

Definition 6.9 A timed game automaton TG is a tuple

$$TG = \{Q, Q^0, \mathcal{E}, \mathbb{C}, I, \rightarrow\},$$

where

- Q is a finite set of discrete states;
- $Q^0 \subseteq Q$ is a finite set of initial discrete states;
- $\mathcal{E}$ stands for a finite set of events that can be decomposed into two disjoint sets $\mathcal{E}_c$ and $\mathcal{E}_u$;
- $\mathbb{C}$ consists of a finite set of clocks;
- $I : Q \rightarrow \Phi(\mathbb{C})$ is a map that labels each $q \in Q$ with a clock constraint;
- $\rightarrow \subseteq Q \times \mathcal{E} \times \Phi(\mathbb{C}) \times 2^{\mathbb{C}} \times Q$ is a transition relation. An element $(q, e, \varphi, \lambda, q')$ represents a transition from q to q' on event e such that the clock constraint $\varphi \in \Phi(\mathbb{C})$ and resetting the clocks $\lambda \subseteq \mathbb{C}$ to zero.

The state of a timed game automaton TG is a tuple $(q, \eta) \in Q \times \mathbb{R}_{\geq 0}^{|\mathbb{C}|}$, where q stands for the discrete mode while η is the valuation of the clocks. Similar to timed automata, there are two types of transitions in a timed game automaton TG as follows:

- Continuous-time evolution (denoted as $\rightarrow^\tau$): For two states (q, η) and (q, η') , if there exists $d \geq 0$ such that $\eta' = \eta + d$ and for all $0 < \delta \leq d$, $(\eta + \delta)(x)$ satisfies $I(q)$, then we have $(q, \eta) \rightarrow^\tau (q, \eta')$.
- Discrete transition (denoted as $\rightarrow^e$): For an $e \in \mathcal{E}$, $(q, \eta) \rightarrow^e (q', \eta')$ if $(q, e, \varphi, \lambda, q') \in \rightarrow$ as defined in TG in the sense that $\eta(x)$ satisfies the guard condition φ and $\eta' = \eta[\lambda := 0]$ after reset.

Discrete transitions triggered by actions in $\mathcal{E}_c$ are said to be controllable, while those with actions in $\mathcal{E}_u$ are uncontrollable.

Example 6.12 A simple timed game automaton [47] is plotted in Fig. 6.6, where the dashed lines represent uncontrollable transitions while solid lines stand for controllable transitions. It can be written into a tuple $\{Q, Q^0, \mathcal{E}, \mathbb{C}, I, \rightarrow\}$, where

$Q = \{q_1, q_2, q_3, q_4, q_5, q_6\}$, $Q_0 = \{q_1\}$, $\mathcal{E}_c = \{c_1, c_2, c_3, c_4\}$, $\mathcal{E}_u = \{u_1, u_2, u_3\}$, and one clock variable $\mathbb{C} = \{x\}$. The invariant condition $I(q) = \{x \geq 0\}$ for all $q \in Q$. The transition relation $\rightarrow$ consists of transitions such as $(q_1, u_1, x > 1, \emptyset, l_5)$, $(q_1, u_2, x < 1, \{x\}, q_3)$, and so on. $\square$

Definition 6.10 A run starting in a timed game automaton TG from a state (q_0, η_0) is a sequence of alternating state and step pairs,

$$\rho = (q_0, \eta_0) \xrightarrow{\delta_0} (q_0, \eta_0 + \delta_0) \xrightarrow{e_0} (q_1, \eta_1) \xrightarrow{\delta_1} (q_1, \eta_1 + \delta_1) \xrightarrow{e_1} \dots$$

We call a run ρ finite if there are only a finite number of transitions in ρ and $\sum_i \delta_i < \infty$. Otherwise, ρ is an infinite run. The last state (q_n, η_n) in a finite run ρ is termed as $last(\rho)$. From the last state of a finite run, we may extend the run by either letting time elapse or triggering a discrete transition. A run is maximal if it cannot be further extended. It is obvious that all infinite runs are maximal. The collection of all runs from the state (q_0, η_0) is denoted as $Runs((q_0, \eta_0), TG)$. We also represent a state in TG concisely as $s_i = (q_i, \eta_i)$.

Definition 6.11 Let $TG = \{Q, Q^0, \mathcal{E}, \mathbb{C}, I, \rightarrow\}$ be a timed game automaton. A strategy f is a partial function from the runs of TG to $\mathcal{E}_c \cup \tau$ such that for every finite run ρ , if $f(\rho)$ in $\mathcal{E}_c$, then $last(\rho) \rightarrow^{f(\rho)} (q', \eta')$ for some state (q', η') .

In plain language, a timed strategy f picks a controllable transition or just lets time elapse based on the history (finite prefix) of a (potentially infinite) run. The choice of the strategy f needs to be valid—for example, if a discrete transition is selected by f -based on a prefix ρ , then the corresponding guard condition should hold true for the last state of ρ . Next, we can define the outcome of a timed game by following a strategy f .

Definition 6.12 Let TG be a timed game automaton and f a strategy over TG . The outcome

$Outcome(s, f)$ of f from a state $s = (q, \eta)$ is the subset of $Run(s, TG)$ defined inductively by the following:

- (1). $s \in Outcome(s, f)$;
- (2). if $\rho \in Outcome(s, f)$, then its extension $\rho' = (\rho \rightarrow^e s')$ is contained in $Outcome(s, f)$ as well if $\rho' \in Runs(s, TG)$ and one of the following conditions holds:
 - (a) $e \in \mathcal{E}_u$;
 - (b) $e \in \mathcal{E}_c$ and $e = f(\rho)$;
 - (c) $e = \tau$ if there exists $\delta \geq 0$ and $\forall 0 \leq \delta' \leq \delta$, there exists a state s'' such that $last(\rho) \rightarrow^{\delta'} s''$ and $f(\rho \rightarrow^{\delta'} s'') = \tau$;
- (3). for an infinite run ρ , $\rho \in Outcome(s, f)$ if all the finite prefixes of ρ are in $Outcome(s, f)$.

Roughly speaking, given a timed game automaton TG and a set of states K , the safety game is to design a strategy f that keeps all runs in $Outcome(s, f)$ remain

inside K starting from a proper initial state $s \in K$. On the other hand, to win a reachability game starting from a state s , we need to design a strategy f such that some states in K can be reached by all runs in $Outcome(s, f)$. If such a strategy f exists for the state s , it is called a winning strategy, and the state s is winning the safety/reachability game. For instance, in the timed game automaton plotted in Fig. 6.6, we would like to design a strategy f such that all runs eventually reach q_6 , namely a reachability game $\Diamond q_6$.

Note that we cannot directly apply the algorithms that we introduced in the previous section for discrete games to solve timed games. This is because the algorithms for discrete games are based on an iterative computation of predecessor sets, for which we need to check individual states to see whether they can be driven to the target set no matter what the other player moves are, but in timed games the state is uncountable.

To mitigate this difficulty, there are two possible solutions. First, by obtaining a finite abstraction, we can translate the timed game into an equivalent discrete game, for which the algorithms introduced in the previous section can be directly applied. Alternatively, we can compute the predecessor set in a symbolic manner just like what we did for symbolic model checking in the verification chapter.

We will illustrate the basic idea behind the abstraction approach in the next subsection and explore the symbolic approach to timed game solving in the subsection that follows.

6.4.2 Abstraction of a Timed Game

In this subsection, we will briefly introduce the basic ideas of an abstraction-based approach to solving timed games.

Simulation Graph

The abstraction of a timed game automaton TG is a discrete game structure, called a simulation graph $G_{TG} = (S, \mathcal{E}, s_0, \delta)$, whose states in S are symbolic states in the form of (q, Z) , where $q \in Q$ and Z is a zone of $\mathbb{R}_{\geq 0}^n$ (as defined in Sect. 3.5.6). It is clear that we only need to consider zones that respect the invariant condition (clock constraints) for the location q , namely we request that $Z \subseteq I(q)$ for all symbolic states in S .

To define the transition relation δ in G_{TG} , we need the following notation.

Definition 6.13 Let $s = (q, Z)$ be a symbolic state and $e \in \mathcal{E}$, we can define the e -successor of s by

$$Post_e(s) = \left\{ (q', \eta') \mid \exists (q, \eta) \in s, (q, \eta) \xrightarrow[g, Y]{e} (q', \eta') \right\},$$

where the transition $(q, e, g, Y, q') \in \rightarrow$ is defined in the timed game automaton TG , and $\eta \models g$ while η' is obtained by reset the clock variables $x_i \in Y$ to zero in η , namely $\eta' = \eta[Y := 0]$.

We can also define the timed-successor of a symbolic state.

Definition 6.14 Let Z be a zone, we can define the timed-successor of Z at a location $q \in Q$ by

$$Z_q^\nearrow = \{\eta + d \mid \exists \eta \in Z \cap \text{Inv}(q), d \geq 0, \eta + d \in \text{Inv}(q)\}.$$

Then, the timed-successor of a symbolic state $s = (q, Z)$ is $s' = (q, Z_q^\nearrow)$.

It is not hard to see that $\eta + t \in \text{Inv}(q)$ for all $0 \leq t \leq d$ since $\text{Inv}(q)$ is convex and $\eta, \eta + d \in \text{Inv}(q)$. Hence, the timed-successor of a symbolic state (q, Z) is also a symbolic state (since zones are closed under time evolution in timed automata).

Now, we are ready to define the translation relation δ between two symbolic states: $(q, Z) \rightarrow^a (q', Z') \in \delta$ for an $a \in \mathcal{E}$ if the following conditions hold:

1. a transition between q and q' triggered by $a \in \mathcal{E}$ is defined in the timed automaton, say $(q, a, g, Y, q') \in \rightarrow$;
2. the zone Z has a non-empty intersection with the guard condition set, denoted by $\llbracket g \rrbracket$;
3. we have $Z' = ((Z \cap \llbracket g \rrbracket)[Y := 0])_{q'}^\nearrow$, which means that the discrete transition can be fired since the clock values in Z satisfy the guard condition g , the clock variables in the set Y will be reset once the transition is fired, and the new zone Z' is obtained by letting time elapse at the new location q' .

The initial state in the simulation graph $G_{TG} = (S, \mathcal{E}, s_0, \delta)$ of a timed game automaton TG is defined as the symbolic state $s_0 = (q_0, \mathbf{0}_{q_0}^\nearrow)$, where $\mathbf{0}$ is a special zone that contains only a zero vector, namely all clocks start from zero.

Example 6.13 Let's revisit the timed game automaton in Fig. 6.6 with a reachability specification $\Diamond q_6$. For illustration, we obtain its simulation graph as shown in Fig. 6.7, where the target state is marked with double circles.

On the simulation graph, the environment can always choose u_1 at the initial state and transit to the state $(q_5, x \geq 1)$, from where there is no way that the controller can reach the goal. However, if we look at the timed game automaton in Fig. 6.6, at the initial location q_1 , the controller can trigger the transition c_1 before the clock reaches 1, i.e., $x < 1$, to avoid the firing of u_1 . Actually, if the controller does this, it can win by trigger c_2 at q_2 once $x \geq 2$. If the uncontrollable transition u_2 or u_3 occur before the controller takes action at q_1 or q_2 , the states will be transited to location q_3 . Once at the location q_3 , the controller can trigger c_3 and c_4 to return to q_2 . In this way, the controller can win eventually. $\square$

From this example, we can see that the simulation graph-based design is sound but not complete—if the simulation graph finds a winning policy, it will win for

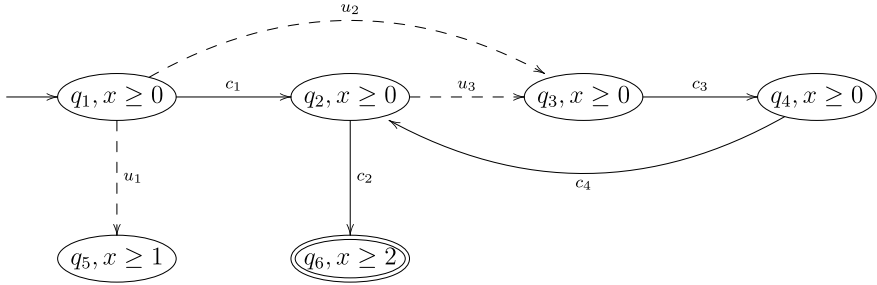


Fig. 6.7 The simulation graph of the example timed game

the original timed game as well; however, the simulation graph may fail to derive a winning policy even if the original timed game is winnable. The reason why the simulation graph obtained in our previous example fails to obtain such a winning strategy is due to the fact that the simulation graph is too coarse. For example, the simulation graph incorrectly classifies the initial state as being uncontrollable. Hence, we need a finer abstraction.

Time-Abstract Bisimulation and Quotient Graph

Recall the region equivalence relation for timed automata that we used for the model checking of timed automata in Chap. 3. Two clock valuations η and $\eta' \in \mathbb{R}^n$ are said to be equivalent if the following hold:

1. if there exists $x_i \in \mathbb{C}$ such that $\eta(x_i) > c_i$, then $\eta'(x_i) > c_i$, and vice versa. Note that c_i and c_j are used to represent the largest integers being compared with i th clock and j th clock, respectively;
2. otherwise, for the clocks $x_i, x_j \in \mathbb{C}$ with $\eta(x_i), \eta'(x_i) \leq c_i$ and $\eta(x_j), \eta'(x_j) \leq c_j$, the following conditions hold:

- (a) the integer parts coincide, namely $\lfloor \eta(x_i) \rfloor = \lfloor \eta'(x_i) \rfloor$;
- (b) if the fractional part $\langle \eta(x_i) \rangle = 0$, then $\langle \eta'(x_i) \rangle = 0$, and vice versa;
- (c) if $\langle \eta(x_i) \rangle \leq \langle \eta(x_j) \rangle$, then $\langle \eta'(x_i) \rangle \leq \langle \eta'(x_j) \rangle$, and vice versa.

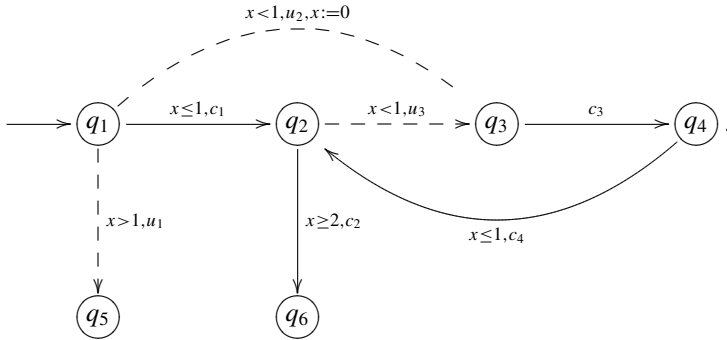
The equivalence relation induced is termed by $\approx$, and it has been shown that the relation $\approx$ can be used to construct a (time-abstract) bisimilar quotient transition system, called region transition system. We can follow the same idea to obtain a (time-abstract) bisimilar quotient game structure of a timed game automaton.

Consider a timed game automaton TG with set of events $\mathcal{E} = \mathcal{E}_c \cup \mathcal{E}_u$. Let $G_{TG}^\approx$ be the quotient graph of TG with respect to the time-abstract bisimulation relation $\approx$ (defined in Sect. 3.5). From $G_{TG}^\approx = (C, C_0, \rightarrow_\approx)$, we build a graph game $G = (C, C_0, \rightarrow^c \cup \rightarrow^u)$, where $\rightarrow^c$ and $\rightarrow^u$ are constructed as follows:

- for each edge $C \xrightarrow{e} C'$, for some $e \in \mathcal{E}_c$, an edge $C \rightarrow^c C'$ is added to $\rightarrow^c$ in G ;
- for each edge $C \xrightarrow{e} C'$, for some $e \in \mathcal{E}_u$, an edge $C \rightarrow^u C'$ is added to $\rightarrow^u$ in G ;
- for each edge $C \xrightarrow{\tau} C'$, an edge $C \rightarrow^c C'$ is added to G .

In other words, discrete transitions keep their controllability status, while timed transitions are considered controllable. The following example illustrates the construction of the graph game from a timed game automaton.

Example 6.14 Let's revisit the timed game automaton defined earlier, that is,



Our control goal is to reach q_6 , namely $\Diamond q_6$. Since there is only one clock variable x , its equivalent clock regions are

$$\{(x = 0), (0 < x < 1), (x = 1), (1 < x < 2), (x = 2), (x > 2)\}.$$

Based on the region equivalence and associated bisimulation relation, we can obtain a graph game as shown in Fig. 6.8 after combining some nodes and double circling the target node.

From this abstraction game, we can see that the controller can win the reachability game. For example, at the initial location q_1 , the controller can trigger the transition c_1 before the clock reaches 1, i.e., $x < 1$, to avoid the firing of u_1 . Actually, if the controller does this, it can win by trigger c_2 at q_2 once $x \geq 2$. If the uncontrollable transition u_2 or u_3 occurs before the controller takes action at q_1 or q_2 , the states will be transited to location q_3 . Once at the location q_3 , the controller can trigger c_3 and c_4 (before $x > 1$) to return to q_2 . In this way, the controller can win eventually. $\square$

6.4.3 Symbolic Approach

The abstraction-based approach described in the previous subsection could suffer from the state explosion problem since the number of quotient states could grow exponentially with respect to the number of clock variables. In addition, as we know,

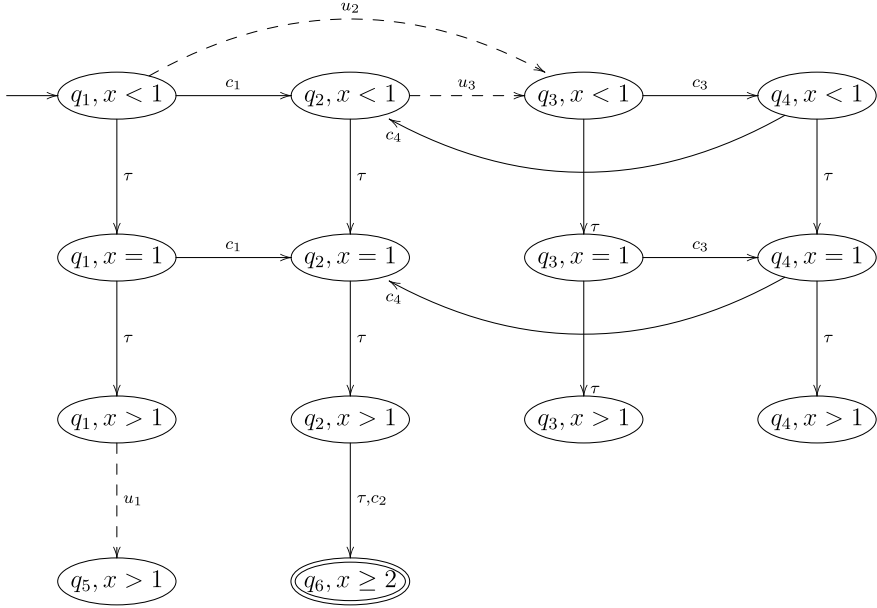


Fig. 6.8 The quotient graph of the example timed game

finite bisimilar abstractions only exist for a very limited classes of hybrid system models. These form the major obstacles to a successful application of abstraction-based approaches in real-world applications. Hence, we turn to an alternative approach—a symbolic approach. The essential part of the symbolic approach is to compute the predecessor operators symbolically. Once we can compute predecessor sets for timed games, then we can apply the game solving algorithms used for discrete games to corresponding timed games.

Since timed game automata consist of both discrete transitions and continuous transitions, we need to deal with two types of predecessors as well. We will take a look at the discrete transitions first.

Definition 6.15 Let S be a subset of states and $e \in \mathcal{E}$, we can define the e -predecessor of S by

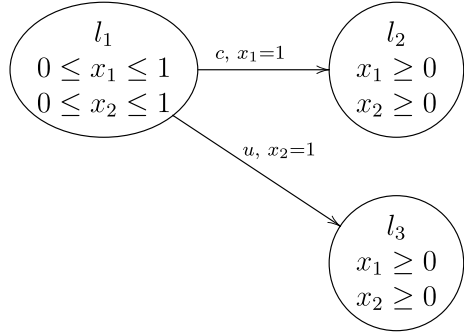
$$Pre_e(S) = \{(q, \eta) \mid \exists (q', \eta') \in S, (q, \eta) \xrightarrow{e} (q', \eta')\}.$$

Definition 6.16 Controllable predecessor of S is defined by

$$cPre(S) = \bigcup_{e \in \mathcal{E}_c} Pre_e(S).$$

Definition 6.17 Uncontrollable predecessor of S is defined by

Fig. 6.9 Timed game automaton for Example 6.15



$$uPre(S) = \bigcup_{e \in \mathcal{E}_u} Pre_e(S).$$

The computation of the predecessor set under discrete transitions depends on the guard set and reset maps, which will be illustrated in the next example.

Example 6.15 Consider a portion of a timed game given in Fig. 6.9, which consists of both a controllable transition and an uncontrollable transition, namely $\mathcal{E}_c = \{c\}$ and $\mathcal{E}_u = \{u\}$.

Let's assume that the target region is given by $S = \{(l_2, x_1 \geq 0 \wedge x_2 \geq 0)\}$. The controllable predecessor of S , $cPre(S)$ can be computed as follows:

$$\begin{aligned}
 cPre(S) &= \bigcup_{e \in \mathcal{E}_c} Pre_e(S) = Pre_c(S) \\
 &= \{(l, v) \mid \exists (l', v') \in S, (l, v) \xrightarrow{c} (l', v')\} \\
 &= \{(l_1, x_1 = 1 \wedge 0 \leq x_2 \leq 1)\}.
 \end{aligned}$$

Similarly, we can compute the uncontrollable predecessor for $S' = \{(l_3, x_1 \geq 0 \wedge x_2 \geq 0)\}$ as follows:

$$\begin{aligned}
 uPre(S') &= \bigcup_{e \in \mathcal{E}_u} Pre_e(S') = Pre_u(S') \\
 &= \{(l, v) \mid \exists (l', v') \in S', (l, v) \xrightarrow{u} (l', v')\} \\
 &= \{(l_1, x_2 = 1 \wedge 0 \leq x_1 \leq 1)\}.
 \end{aligned}$$

□

In addition to discrete transitions, we also need to consider the predecessors under a continuous transition, namely letting time elapse. In particular, we introduce the following timed predecessor operator for timed automata.

Definition 6.18 The safe timed predecessor of a set S with respect to a set U is defined by

$$Pred_t(S, U) = \left\{ s \mid \exists \delta \in \mathbb{R}_{\geq 0}, \text{ such that } s \xrightarrow{\delta} s', s' \in S \text{ and } Post_{[0, \delta]}(s) \cap U = \emptyset \right\},$$

where $Post_{[0, \delta]}(s) = \{s' \mid \exists t \in [0, \delta], s.t. s \xrightarrow{t} s'\}$ stands for the set of states that lies on the state trajectory starting from s to s' under a continuous transition within a time duration $[0, \delta]$.

It basically requires that the state trajectory reaches some state in S while avoiding the set U . The reason why we need this “reach-while-avoid” operator should be clear when we put both discrete and timed predecessors together as the following definition of controllable predecessor hints.

Definition 6.19 The controllable predecessor operator π is defined as

$$\pi(S) = Pred_t(S \cup cPre(S), uPre(\bar{S})), \quad (6.82)$$

where $\bar{S}$ stands for the complement of the set S .

The set $S \cup cPre(S)$ consists of the states that are either already in S or can be transited to S by a controllable transition. The set $uPre(\bar{S})$ are those states that can escape S (reach the outside of S) by following some uncontrollable transition. These should be avoided by the controller since once the state is in the set of $uPre(\bar{S})$, there is no guarantee that the state can reach S as there is a possibility that the environment can fire an uncontrollable transition to drive such a state out of S . Hence, the controllable predecessor computes the states that can be driven to the set $S \cup cPre(S)$ while avoiding $uPre(\bar{S})$, which therefore guarantees that the state in $\pi(S)$ will be driven to S no matter what the uncontrollable environment behavior. Now let's take a look at an example of the computation of the controllable predecessor.

Example 6.16 Let's continue Example 6.15 with $S = \{(l_2, x_1 \geq 0 \wedge x_2 \geq 0)\}$. The controllable predecessor of S , $\pi(S) = Pred_t(S \cup cPre(S), uPre(\bar{S}))$ can be computed as follows.

First, we already computed

$$cPre(S) = \{(l_1, x_1 = 1 \wedge 0 \leq x_2 \leq 1)\},$$

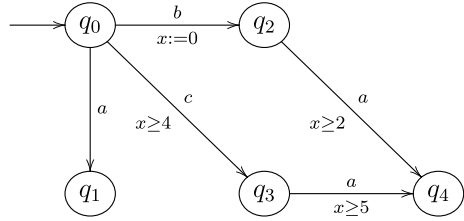
so $S \cup cPre(S) = \{(l_2, x_1 \geq 0 \wedge x_2 \geq 0)\} \cup \{(l_1, x_1 = 1 \wedge 0 \leq x_2 \leq 1)\}$.

Second, note that $\bar{S} = \{(l_1, x_1 \geq 0 \wedge x_2 \geq 0)\} \cup \{(l_3, x_1 \geq 0 \wedge x_2 \geq 0)\}$, we have

$$\begin{aligned} uPre(\bar{S}) &= uPre(\{(l_1, x_1 \geq 0 \wedge x_2 \geq 0)\}) \cup uPre(\{(l_3, x_1 \geq 0 \wedge x_2 \geq 0)\}) \\ &= \emptyset \cup \{(l_1, x_2 = 1 \wedge 0 \leq x_1 \leq 1)\} \\ &= \{(l_1, x_2 = 1 \wedge 0 \leq x_1 \leq 1)\}. \end{aligned}$$

Then, we can compute the controllable predecessor $\pi(S)$ as follows:

Fig. 6.10 An example of the Time optimal control for timed automata



$$\begin{aligned}\pi(S) &= \text{Pred}_t(S \cup c\text{Pre}(S), u\text{Pre}(\bar{S})) \\ &= \{(l_2, x_1 \geq 0 \wedge x_2 \geq 0)\} \cup \{(l_1, 0 \leq x_2 < x_1 \leq 1)\}.\end{aligned}$$

□

With the capability of computing the controllable predecessor $\pi(\cdot)$ for timed games, now we can implement the algorithms for discrete games to solve timed safety games, timed reachability games, and timed Büchi games, respectively. Since the timed game automata have a bisimilar quotient game structure of finite size, the design algorithms are guaranteed to terminate within a finite number of iterations.

6.4.4 Timed Automata Optimal Control

We conclude our discussion on the synthesis using the timed automata model with an example from [12] to illustrate how to use the dynamic programming and principle of optimality to solve a timed automata optimal control problem. Consider a timed automaton as shown in Fig. 6.10.

The goal is to choose transitions such that the target location q_4 can be reached as soon as possible. Hence, we define $F = \{q_4\} \times \{x \geq 0\}$, which can be compactly denoted as $(q_4, x \geq 0)$. Define the value function $V^i : Q \times \mathbb{R}_{\geq 0}^{\mathbb{C}} \rightarrow \mathbb{R}_{\geq 0}$, which gives an upper-bound estimation of the arrival time from a state (q, x) to the target set $F \subseteq Q \times \mathbb{R}_{\geq 0}^{\mathbb{C}}$.

Initially, we set

$$V^0(q, x) = \begin{cases} 0, & (q, x) \in F, \\ \infty, & (q, x) \notin F. \end{cases} \quad (6.83)$$

The value iteration repeats as

$$V^{i+1}(q, x) = \min \{V^i(q, x), \pi_{Act}(V^i(q, x)), \pi_{Idle}(V^i(q, x))\}, \quad (6.84)$$

for $i \geq 0$, where

$$\pi_{Act}(V^i(q, x)) = \min_{a \in \Sigma} \{V^i(\delta(q, x, a))\}, \quad (6.85)$$

represents the discrete transitions triggered by an event a with the new state given by $\delta(q, x, a)$, and the state transition due to time elapse is given by

$$\pi_{Idle}(V^i(q, x)) = \inf_{t \in \mathbb{R}_{\geq 0}} \{t + V^i(q, x + t)\}. \quad (6.86)$$

Once we have $V^{i+1}(q, x) = V^i(q, x)$, the iteration stops and returns $V^*(q, x) = V^{i+1}(q, x)$.

Back to our example, initially the value function is

$$V^0(q, x) = \begin{cases} 0, & (q_4, x \geq 0), \\ \infty, & \text{otherwise.} \end{cases} \quad (6.87)$$

For $i = 0$, we compute $V^1(q, x)$ using the value iteration. For instance,

$$V^1(q_4, x \geq 0) = \min \left\{ V^0(q_4, x \geq 0), \pi_{Act}(V^0(q_4, x \geq 0)), \pi_{Idle}(V^0(q_4, x \geq 0)) \right\},$$

where $\pi_{Act}(V^0(q_4, x \geq 0)) = \min_{a \in \Sigma} \{V^0(\delta(q_4, x \geq 0, a))\} = 0$ and $\pi_{Idle}(V^0(q_4, x \geq 0)) = \inf_{t \in \mathbb{R}_{\geq 0}} \{t + V^0(q_4, x + t)\} = 0$. So, $V^1(q_4, x \geq 0) = V^0(q_4, x \geq 0) = 0$, and hence $V^*(q_4, x \geq 0) = 0$.

As another example for value iteration, consider the set of states $(q_2, x \geq 0)$

$$V^1(q_2, x \geq 0) = \min \left\{ V^0(q_2, x \geq 0), \pi_{Act}(V^0(q_2, x \geq 0)), \pi_{Idle}(V^0(q_2, x \geq 0)) \right\},$$

where $V^0(q_2, x \geq 0) = \infty$, and $\pi_{Idle}(V^0(q_2, x \geq 0)) = \inf_{t \in \mathbb{R}_{\geq 0}} \{t + V^0(q_2, x + t)\} = \infty$. The computation of $\pi_{Act}(V^0(q_2, x \geq 0))$ needs some further look. First, the discrete transition (triggered by a) is only valid if $x \geq 2$, and the new state becomes (q_4, x) . Hence, we split the set $(q_2, x \geq 0)$ into two parts $(q_2, x \geq 2)$ and $(q_2, 0 \leq x < 2)$.

For the set of states $(q_2, x \geq 2)$, we have $\pi_{Act}(V^0(q_2, x \geq 2)) = V^0(q_4, x) = 0$, and therefore $V^1(q_2, x \geq 2) = 0$.

However, the discrete transition is undefined for states in $(q_2, 0 \leq x < 2)$, so $\pi_{Act}(V^0(q_2, 0 \leq x < 2)) = \infty$. So, $V^1(q_2, 0 \leq x < 2) = \infty$.

Set $i = 1$, and we repeat the value iteration for the states $(q_2, 0 \leq x < 2)$,

$$V^2(q_2, 0 \leq x < 2) = \min \left\{ V^1(q_2, 0 \leq x < 2), \pi_{Act}(V^1(q_2, 0 \leq x < 2)), \pi_{Idle}(V^1(q_2, 0 \leq x < 2)) \right\},$$

where $V^1(q_2, 0 \leq x < 2) = \infty$, $V^1(q_2, 0 \leq x < 2) = \infty$, and $\pi_{Idle}(V^1(q_2, 0 \leq x < 2)) = \inf_{t \in \mathbb{R}_{\geq 0}} \{t + V^1(q_2, x + t)\}$. To compute $\pi_{Idle}(V^1(q_2, 0 \leq x < 2))$, note that if $x + t \geq 2$, $V^1(q_2, x + t) = 0$, and the infimum (actually minimum here) of such t is $2 - x$. Therefore, $\pi_{Idle}(V^1(q_2, 0 \leq x < 2)) = 2 - x$, and $V^2(q_2, 0 \leq x < 2) = 2 - x$.

It is not difficult to verify that $V^2(q_2, x \geq 2) = V^1(q_2, x \geq 2) = 0$ and therefore $V^*(q_2, x \geq 2) = 0$. Also, it is not difficult to check that $V^3(q_2, 0 \leq x < 2) = V^2(q_2, 0 \leq x < 2) = 2 - x$, and therefore $V^*(q_2, 0 \leq x < 2) = 2 - x$.

Similarly, we can compute the value function for other locations using the value iteration. For instance, we have $V^*(q_3, x \geq 5) = 0$ and $V^*(q_3, 0 \leq x < 5) = 5 - x$, $V^*(q_0, x \geq 5) = 0$, $V^*(q_0, 3 \leq x < 5) = 5 - x$, $V^*(q_0, 0 \leq x < 3) = 2$, and $V^*(q_1, x \geq 0) = \infty$ since q_1 cannot reach q_4 . Details are left as exercises.

Actually, the idea here can be extended to the optimal control of hybrid automata without continuous control as long as we can explicitly represent the continuous state evolution.

6.5 Hybrid Games

This section focuses on solving hybrid games. First, let's recall the hybrid automata definition (we augment it with external discrete events Σ).

Definition 6.20 A hybrid automaton H is a collection

$$H = \{Q, X, f, \text{Init}, \text{Inv}, \Sigma, E, G, R\},$$

where

- $Q = \{q_1, q_2, \dots\}$ is a finite set of discrete states;
- $X \subseteq \mathbb{R}^n$ represents the state space of the continuous state variables;
- $f : Q \times X \rightarrow \mathbb{R}^n$ assigns to each discrete state $q \in Q$ an analytic vector field $f(q, \cdot)$;
- $\text{Init} \subseteq Q \times X$ is the set of initial states;
- $\text{Inv} : Q \rightarrow 2^X$ assigns to each discrete state $q \in Q$ a set $\text{Inv}(q) \subseteq X$ called the invariant set;
- $E \subseteq Q \times \Sigma \times Q$ is the set of discrete transitions;
- $G : E \rightarrow 2^X$ assigns to each discrete transition $(q, \sigma, q') \in E$ a guard set $G(q, \sigma, q') \subset X$;
- $R : E \times X \rightarrow 2^X$ is a reset map.

The state of a hybrid automaton, denoted by (q, x) , is composed of two parts—the discrete mode $q \in Q$ and the continuous state variable $x \in X$. From a hybrid state (q, x) , there are two types of transitions as follows:

1. Discrete transitions, denoted as $(q, x) \xrightarrow{e} (q', x')$ for an edge $e = (q, \sigma, q') \in E$ with $\sigma \in \Sigma$ and $q' \in Q$, the current state $x \in G(e)$ and $x' \in R(e, x)$.
2. Continuous transitions, denoted as $(q, x) \xrightarrow{\delta} (q, x')$ for a positive constant $\delta > 0$ if there exists a solution $x(t)$ of the differential equation $\dot{x}(t) = f(q, x(t))$ for $0 \leq t \leq \delta$, with $x(0) = x$, $x(\delta) = x'$, and for all $0 \leq t \leq \delta$, $x(t) \in \text{Inv}(q)$.

We assume that the flow function $f(q, x)$ is Lipschitz continuous (w.r.t. x) for all discrete modes q , which will guarantee that such a solution uniquely exists at least locally. We also assume that the hybrid automaton is non-blocking and non-Zeno.

6.5.1 Hybrid Game Automata

By decomposing the discrete transitions into controllable and uncontrollable transitions and further augmenting the continuous transition with both control and disturbance inputs, we define a hybrid game automaton as follows.

Definition 6.21 A hybrid game automaton

$$HG = \{Q, X, \mathcal{U}, \mathcal{D}, f, Init, Inv, \Sigma, E, G, R\},$$

is a hybrid automaton with the following augments:

- $f : Q \times \mathcal{U} \times \mathcal{D} \times X \rightarrow \mathbb{R}^n$ assigns to each discrete state $q \in Q$ an analytic vector field $\dot{x} = f(q, u, d, x)$, where $u \in \mathcal{U}$ represents control signals and $d \in \mathcal{D}$ stands for disturbance inputs;
- the set of events Σ is partitioned into two disjoint sets Σ_c (the controllable events) and Σ_u (the uncontrollable events), and correspondingly E is partitioned into two disjoint sets E_c (the controllable transitions) and E_u (the uncontrollable transitions).

The definitions for runs and strategies for timed game automata can be extended to hybrid game automata in a straightforward manner. The critical step in solving hybrid games is the computation of the controllable predecessor operator $\pi(\cdot)$. Similar to timed games, we need to deal with two types of transitions—discrete transition and continuous evolution.

Definition 6.22 An uncontrollable discrete predecessor $Pre_u : 2^{Q \times X} \rightarrow 2^{Q \times X}$ for a given subset of states $K \subseteq Q \times X$ returns

$$Pre_u(K) = K^c \bigcup \left\{ (q, x) \mid \exists e \in E_u, [x \in G(e)] \wedge [R(e, x) \cap K^c \neq \emptyset] \right\}.$$

It means that either the state is already outside of K or there exists an uncontrollable discrete transition to drive the state outside of K in one step.

Definition 6.23 A controllable discrete predecessor $Pre_c : 2^{Q \times X} \rightarrow 2^{Q \times X}$ for a given subset of states $K \subseteq Q \times X$ returns

$$Pre_c(K) = K \cap \left\{ (q, x) \mid \exists e \in E_c, [x \in G(e)] \wedge [R(e, x) \subseteq K] \right\}.$$

It means that the state in K can remain in K by following a controllable discrete transition.

Definition 6.24 The controllable predecessor operator $\pi : 2^{Q \times X} \rightarrow 2^{Q \times X}$ is defined as

$$\pi(S) = Reach(S \cup Pre_c(S), Pre_u(\bar{S})), \quad (6.88)$$

where $Reach(\cdot, \cdot)$ is the reach-while-avoid operator, and $\bar{S}$ stands for the complement of the set S .

The set $S \cup Pre_c(S)$ consists of the states that are either already in S or can be transited to S by a controllable transition. The set $Pre_u(\bar{S})$, on the other hand, are those states that can escape S (reach the outside of S) by following some uncontrollable transition. Therefore, the controller should avoid the states in $Pre_u(\bar{S})$ because once the state is in the set of $Pre_u(\bar{S})$, there is no guarantee that the state can reach S as there is a possibility that the environment can fire an uncontrollable transition to drive such a state out of S . Hence, the controllable predecessor computes the states that can be driven to the set $S \cup Pre_c(S)$ while avoiding $Pre_u(\bar{S})$, which therefore guarantees that the state in $\pi(S)$ will be driven to S irrespective of the uncontrollable environment behavior.

With the computation of the controllable predecessor operator $\pi : 2^{Q \times X} \rightarrow 2^{Q \times X}$ for a hybrid game automaton, we can, in principle, solve hybrid safety, reachability, and Büchi games. To compute $\pi(\cdot)$, it is essential to compute

- (a) the controllable predecessor $Pre_c(\cdot)$;
- (b) the uncontrollable predecessor $Pre_u(\cdot)$;
- (c) the reach-while-avoid operator $Reach(\cdot, \cdot)$.

Note that the computation of $Pre_c(\cdot)$ or $Pre_u(\cdot)$ is the same as in the timed games, while the computation of $Reach(\cdot, \cdot)$ is highly nontrivial and will be the main focus in the rest of this section.

6.5.2 Solve the Reach-While-Avoid Operator as a Pursuit–Evasion Game

We fix the discrete mode q in our computation of the reach-while-avoid operator, $Reach(G, E)$. Our goal is to compute the states from which there exists control u such that the state trajectory will reach G (goal) while avoiding E (escape set) no matter what the disturbance does. The main idea is to convert the computation of $Reach(G, E)$ into a differential game. This sub-section is mainly based on results from [30, 31].

For this, we make the following assumptions. Let $l_G : X \rightarrow \mathbb{R}$ and $l_E : X \rightarrow \mathbb{R}$ be differentiable functions such that

$$G = \{x \in X \mid l_G(x) \leq 0\}, \quad E = \{x \in X \mid l_E(x) < 0\}. \quad (6.89)$$

Like we did in solving differential safety games (Sect. 6.3.2), we recast the computation of $Reach(G, E)$ into a two-player zero-sum differential game with the terminal cost

$$l_G(x(0)).$$

Define the value function as

$$V(x, t) = \min_{u \in \mathcal{U}_{[t, 0]}} \max_{d \in \mathcal{D}_{[t, 0]}} l_G(x(0)).$$

The Hamiltonian can be defined as

$$\mathcal{H}(x, \frac{\partial V}{\partial x}(x, t)) = \min_u \max_d \left\{ \frac{\partial V}{\partial x}(x, t) f(x, u, d, t) \right\}.$$

It can be shown that the reach-while-avoid operator can be computed using level sets of the value function of the formulated differential game.

Theorem 6.3 *If there exists a continuously differentiable function $J_G : \mathbb{R}^n \times [t, 0] \rightarrow \mathbb{R}$ satisfying the following constrained HJ equation:*

$$-\frac{\partial J_G}{\partial t}(x, t) = \min \left\{ 0, \mathcal{H}(x, \frac{\partial J_G}{\partial x}(x, t)) \right\}, \quad (6.90)$$

subject to the boundary condition

$$J_G(x, t) \geq -l_E(x), \quad (6.91)$$

then the set $G(t) = \{x \in X \mid J_G(x, t) \leq 0\}$ represents the set of states for which starting from $x(t) \in G(t)$, there exists a control u such that for all disturbances d , the whole state trajectory will reach G without entering E during $[t, 0]$.

Proof To show that $\left\{ x \in \mathbb{R}^n \mid J_G(x, t) \leq 0 \right\} \subseteq \text{Reach}(G, E)$. We first show that x with $J_G(x, t) \leq 0$ can reach G . Assume the opposite, then $J_G(x, 0) > 0$. Due to the continuity of J_G , there must exist $\tau \in [t, 0]$ such that $J_G(x, \tau) = 0$, i.e., at the boundary of $\left\{ x \in \mathbb{R}^n \mid J_G(x, \tau) \leq 0 \right\}$.

Note that

$$-\frac{\partial J_G}{\partial t}(x, \tau) = \min \left\{ 0, \mathcal{H}(x, \frac{\partial J_G}{\partial x}(x, \tau)) \right\} \leq 0,$$

and $\mathcal{H}(x, \frac{\partial V}{\partial x}(x, t)) = \min_u \max_d \left\{ \frac{\partial V}{\partial x}(x, t) f(x, u, d, t) \right\}$. Then, it must hold that

$$\min_u \max_d \frac{\partial J_G}{\partial t}(x, \tau) f(x, u, d, \tau) < 0,$$

otherwise $J_G(x, \tau) = 0$ will be frozen (since $\frac{\partial J_G}{\partial t}(x, \tau) = 0$).

However, the condition $\min_u \max_d \frac{\partial J_G}{\partial t}(x, \tau) f(x, u, d, \tau) < 0$ means that no matter what disturbance does, the controller can drag the state back into the set $J_G(x, \tau) \leq 0$. Hence, it is impossible for the trajectory to cross the boundary of the level set of J_G . Therefore, $J_G(x, 0) \leq 0$, which implies $l_G(x, 0) \leq 0$, so G is reached by x .

Actually, we have $J_G(x, \tau) \leq 0$ for all $\tau \in [t, 0]$ from the above arguments, then $-l_E(x) \leq J_G(x, \tau) \leq 0 \implies l_E(x) \geq 0$, meaning that x is outside of E for all $\tau \in [t, 0]$.

The other direction $\text{Reach}(G, E) \subseteq \{x \in \mathbb{R}^n \mid J_G(x) \leq 0\}$ is simple to verify. Since $\frac{\partial J_G}{\partial t}(x, t) \geq 0$, so $J_G(x, t)$ is non-decreasing with respect to t . Hence, $J_G(x, t) \leq J_G(x, 0)$ since $t \leq 0$. For any point $x \in \text{Reach}(G, E)$, it must hold that $J_G(x, 0) = l_G(x(0)) \leq 0$. Therefore, $J_G(x, t) \leq 0$.

From the proof, it should be apparent that the constraint $J_G(x, t) \geq -l_E(x)$ ensures that the evolution of $J_G^*(x, t)$ is frozen in set E . Furthermore, assume that $J_G(x, t)$ satisfies the above constrained HJ equation for all t , and that it converges uniformly in x as $t \rightarrow -\infty$ to a function $J_G(x)$. Then we have

$$\text{Reach}(G, E) = \left(q, \{x \in \mathbb{R}^n \mid J_G(x) \leq 0\} \right).$$

Therefore, the computation of the reach-while-avoid operator for hybrid games with very general nonlinear dynamics can be reduced to the solution of a constrained HJ equation. However, solving the constrained HJ equation is highly nontrivial, and we usually need to turn to numerical methods.

6.6 Control-Oriented Abstraction

At the end of the previous section, we were motivated to obtain an equivalent abstraction for a continuous or a hybrid dynamical system by the fact that the game theory-based approaches developed for discrete systems can be applied to the abstracted systems. The obtained winning strategy needs to be continualized, namely one needs to translate the discrete symbolic commands into continuous (most likely piecewise continuous) control laws that can be applied to the physical dynamics. The hybridization process is possible due to the equivalence between the abstracted and the original dynamical systems. Hence, how to obtain an equivalent abstraction is critical.

In this section, we will focus on the abstraction issue, with a particular focus on obtaining an equivalent abstraction for continuous-variable control systems. The problem of interest here is to design a hybrid controller (involving a collection of continuous feedback control laws and a switching strategy selecting a proper control law from this collection) to achieve a given temporal logic specification. For this, we first introduce an extension of linear temporal logic, called temporal logic over reals, to be able to deal with dense-time real-valued signals. Then, we introduce a family of abstraction schemes to obtain a finite bisimulation and an approximate bisimulation abstraction. Several different types of control systems, such as linear control, multi-affine, and general nonlinear control systems, are considered.

6.6.1 Temporal Logic over Reals

In this section, we consider the following continuous-variable control system with outputs:

$$\Sigma : \begin{cases} \dot{x}(t) = f(x(t), u(t)), \\ y(t) = g(x(t)), \end{cases} \quad (6.92)$$

where $x(t) \in \mathbb{R}^n$ is the state of the system, $u(t) \in \mathbb{R}^m$ is the control input, and $y(t) \in \mathbb{R}^p$ is the observed output of the system.

It is assumed that the initial condition x_0 is restricted to a certain region in the state space $X_0 \subseteq \mathbb{R}^n$. The goal is to design a controller such that the output $y(t)$ generated by the closed-loop system satisfies a given temporal logic specification. For example, one may request that the output $y(t)$ visits certain regions in the output space $\mathbb{R}^p$ with a specific order while avoiding some forbidden regions in $\mathbb{R}^p$.

To capture such requirements in temporal logic, we adopt the temporal logic over reals (RTL) [13] and define a set of atomic propositions as $\Pi = \{\pi_0, \pi_1, \dots, \pi_m\}$, which is a finite set of symbols that label these regions of interest. Furthermore, the denotation $\llbracket \cdot \rrbracket$ of each symbol is defined as the region labeled by π , i.e., $\llbracket \pi_i \rrbracket \subseteq \mathbb{R}^p$ for any π_i in Π . The atom π_i is true if and only if $y(t)$ is in $\llbracket \pi_i \rrbracket$. The symbol π_0 is reserved for initial conditions, $y(0) \in \llbracket \pi_0 \rrbracket$, and correspondingly the state is started from $x(0) \in X_0 \subseteq \mathbb{R}^n$.

The syntax of the propositional temporal logic over the reals can be formally introduced as below.

Definition 6.25 Let Π be a finite set of atomic propositions, i.e., $\Pi = \{\pi_0, \pi_1, \dots, \pi_n\}$. The set of all well-formed Propositional RTL formulas are recursively defined from predicates in Π according to the following rules:

- (1). true, false, and π_i are RTL formulas for all $\pi_i \in \Pi$;
- (2). if φ_1 and φ_2 are RTL formulas, then $\varphi_1 \wedge \varphi_2$, $\varphi_1 \vee \varphi_2$ and $\neg \varphi_1$ are RTL formulas;
- (3). if φ_1 and φ_2 are RTL formulas, then $\varphi_1 \mathcal{U} \varphi_2$ and $\varphi_1 \mathcal{R} \varphi_2$ are RTL formulas.

Formally, the semantics of RTL formulas are defined over continuous-time signals. Given a function $y : \mathbb{R}^+ \rightarrow \mathbb{R}^r$, we define $y|_t$ to be the t time shift of y with definition $y|_t(s) = y(s + t)$ for all $s \in \mathbb{R}^+$.

Definition 6.26 Let $y(t)$ be a function $y : \mathbb{R}^+ \rightarrow \mathbb{R}^r$, and Π a finite set of atomic propositions, i.e., $\Pi = \{\pi_0, \pi_1, \dots, \pi_n\}$ with atom mapping $\llbracket \cdot \rrbracket$. For $t, s \in \mathbb{R}^+$, the semantics of an RTL formula over Π can be defined as follows:

- (1). $(y, \llbracket \cdot \rrbracket) \models \pi_i$ iff $y(0) \in \llbracket \pi_i \rrbracket$;
- (2). $(y, \llbracket \cdot \rrbracket) \models \neg p$ iff $y(0) \notin \llbracket \pi_i \rrbracket$;
- (3). $(y, \llbracket \cdot \rrbracket) \models \varphi_1 \wedge \varphi_2$ if $(y, \llbracket \cdot \rrbracket) \models \varphi_1$ and $(y, \llbracket \cdot \rrbracket) \models \varphi_2$;
- (4). $(y, \llbracket \cdot \rrbracket) \models \varphi_1 \vee \varphi_2$ if $(y, \llbracket \cdot \rrbracket) \models \varphi_1$ or $(y, \llbracket \cdot \rrbracket) \models \varphi_2$;
- (5). $(y, \llbracket \cdot \rrbracket) \models \varphi_1 \mathcal{U} \varphi_2$ if there exists $t \geq 0$ such that $(y|_t, \llbracket \cdot \rrbracket) \models \varphi_2$ and for all s with $0 \leq s \leq t$ we have $(y|_s, \llbracket \cdot \rrbracket) \models \varphi_1$;

- (6). $(y, \llbracket \cdot \rrbracket) \models \varphi_1 \mathcal{R} \varphi_2$ if for all $t \geq 0$ it is $(y|_t, \llbracket \cdot \rrbracket) \models \varphi_2$ or there exists some s such that $0 \leq s \leq t$ and $(y|_s, \llbracket \cdot \rrbracket) \models \varphi_1$.

Intuitively speaking, the formula $\varphi_1 \mathcal{U} \varphi_2$ expresses the property that over the trajectory $y(t)$, φ_1 is true until φ_2 becomes true. On the contrary, the release operator $\varphi_1 \mathcal{R} \varphi_2$ means that φ_2 should hold true and be released when φ_1 becomes true.

Furthermore, we can derive several additional temporal operators such as

- $\Diamond \varphi = \text{true } \mathcal{U} \varphi$ means that the sub-formula φ *eventually* becomes true for a trajectory $y(t)$;
- $\Box \varphi = \neg \Diamond \neg \varphi$ indicates that φ *always* holds true for $y(t)$.

The following examples from [14] illustrate some typical control specifications that can be formulated as temporal logic formulas based on the atomic proposition set Π :

- **Reachability while avoiding regions:** The formula $\neg(\pi_1 \vee \pi_2) \sqcup \pi_3$ represents the requirement that the output finally reaches the region $\llbracket \pi_3 \rrbracket$ while keeping away from the regions $\llbracket \pi_1 \rrbracket$ and $\llbracket \pi_2 \rrbracket$.
- **Sequencing:** The formula $\Diamond(\pi_1 \wedge \Diamond(\pi_2 \wedge \Diamond \pi_3))$ represents the requirement that the output reaches the regions $\llbracket \pi_1 \rrbracket$, $\llbracket \pi_2 \rrbracket$ and $\llbracket \pi_3 \rrbracket$ in order.
- **Coverage:** The formula $\Diamond \pi_1 \wedge \Diamond \pi_2 \wedge \Diamond \pi_3$ represents the requirement that the output eventually reaches all the regions $\llbracket \pi_1 \rrbracket$, $\llbracket \pi_2 \rrbracket$, and $\llbracket \pi_3 \rrbracket$ without any particular order.
- **Recurrence (Liveness):** The formula $\Box(\Diamond \pi_1 \wedge \Diamond \pi_2 \wedge \Diamond \pi_3)$ represents the requirement that the output reaches these regions $\llbracket \pi_1 \rrbracket$, $\llbracket \pi_2 \rrbracket$, and $\llbracket \pi_3 \rrbracket$ infinitely often.

More complicated specifications can be composed from the basic specifications using the logic operators. The goal is to design a hybrid controller such that the trajectories of the closed-loop system satisfy a given specification formula ϕ .

6.6.2 Linear Control Systems

We first consider the case when the dynamical system to be controlled is linear, i.e.,

$$\begin{cases} \dot{x}(t) = Ax(t) + a + Bu(t), \\ y(t) = Cx(t), \end{cases} \quad (6.93)$$

where $x(t) \in \mathbb{R}^n$ is the state of the system, $u(t) \in \mathbb{R}^m$ is the control input, and $y(t) \in \mathbb{R}^p$ is the observed output of the system, while $A \in \mathbb{R}^{n \times n}$, $a \in \mathbb{R}^n$, $B \in \mathbb{R}^{n \times m}$, and $C \in \mathbb{R}^{p \times n}$. The results in this subsection are mainly based on [15, 16], where it is assumed that all regions of interest are polyhedral sets.

Polyhedron

A *polyhedral set* P is a subset of $\mathbb{R}^N$, described by a finite number of linear inequalities. Namely, there exist K non-zero vectors $n_1, \dots, n_K \in \mathbb{R}^n$ and scalars $\alpha_1, \dots, \alpha_K \in \mathbb{R}$, such that

$$P = \{x \in \mathbb{R}^n \mid \forall i = 1, \dots, K : n_i^T x \leq \alpha_i\}. \quad (6.94)$$

Each linear inequality $n_i^T x \leq \alpha_i$ forms a half-space, so this is called the half-space representation of a polyhedral set P . Correspondingly, the hyperplane formed by $n_i^T x = \alpha_i$ is called a *supporting hyperplane* for P . A bounded polyhedral set is called a *polytope*. A polytope can alternatively be characterized as the convex hull of a finite number of points, $v_1, \dots, v_M \in \mathbb{R}^n$,

$$P = \left\{x \in \mathbb{R}^n \mid x = \sum_{i=1}^M \lambda_i v_i, \sum_{i=1}^M \lambda_i = 1, \lambda_i \geq 0\right\}, \quad (6.95)$$

where $v_1, \dots, v_M$ are called vertices of the polytope P . This is called the vertex representation of a polytope.

The half-space representation and the vertex representation for a polytope are equivalent and can be transformed from one to the other [17]. Which definition should be employed depends on what is more suitable for the problem at hand. The intersection of a polytope P with one of its supporting hyperplanes

$$F_i = \{x \in \mathbb{R}^n \mid n_i^T x = \alpha_i\} \cap P, \quad (6.96)$$

is called a *facet* of P if the dimension of the intersection is equal to $n - 1$. The vector n_i is the normal vector of the facet F_i , ($i = 1, \dots, K$), and, by convention, n_i is of unit length and always points out of the polytope P . Also, the corresponding hyperplane $n_i^T x = \alpha_i$ is called a *bounding hyperplane* for P . Later on, we assume that each facet of P corresponds to exactly one n_i , and vice versa. Namely, each n_i corresponds to a bounding hyperplane and is linearly independent of other normal vectors.

The set of vertices of a polytope P is denoted by $\mathcal{V}(P)$. Given a vertex $v \in \mathcal{V}(P)$, we denote by $\mathcal{F}(v)$ the set of all facets containing v . It can be shown that for a polytope, there are at least $n + 1$ vertices, i.e., $M \geq n + 1$. A polytope that has exactly $n + 1$ vertices is called a *simplex* in $\mathbb{R}^n$. Any full-dimensional polytope can be triangularized [18]. In other words, for any full-dimensional polytope P , there exists full-dimensional simplex $S_1, \dots, S_L$ such that

- (1). $P = \cup_{i=1,2,\dots,L} S_i$;
- (2). $S_i \cap S_j$ is either empty or a common facet of S_i and S_j , for all $i, j = 1, \dots, L$ with $i \neq j$;

- (3). the set of vertices of simplex S_i is a subset of $\{v_1, v_2, \dots, v_M\}$ for all $i = 1, \dots, L$.

Affine Functions

The reason why we restrict to polytopes is due to the fact that an affine function's value inside a polytope P will be uniquely determined by its values at the vertices of P . An affine function $f : \mathbb{R}^n \rightarrow \mathbb{R}^r$ is of the form

$$f(x) = Ax + b, \quad (6.97)$$

for constant matrices $A \in \mathbb{R}^{r \times n}$ and vector $b \in \mathbb{R}^r$. Due to the convexity and linearity, it is easy to obtain the following results.

Lemma 6.1 *Let $w \in \mathbb{R}^r$ and $d \in \mathbb{R}$. Then $w^T f(x) > d$ everywhere in a polytope P , if and only if the inequality holds at all the vertices of P , i.e., $w^T f(v_i) > d$ for all $v_i \in \mathcal{V}(P)$.*

It is easy to see that the result remains valid if “ $>$ ” is replaced by “ $\geq, =, <, \text{ or } \leq$.” Also, the result remains valid if f is restricted to a facet F_i . The following result tells us that the function value of an affine function on polytope P is completely determined by the values of f on the vertices of P .

Lemma 6.2 *Let P be a polytope in $\mathbb{R}^n$ and $f : \mathbb{R}^n \rightarrow \mathbb{R}^r$ an affine function. The function value of $x \in P$, $f(x)$ is completely determined by the values of f on the vertices of P , $f(v_i) = g_i$, for all $v_i \in \mathcal{V}(P)$, i.e.,*

$$x \in P \Rightarrow f(x) = \sum_{v \in \mathcal{V}(P)} \lambda_v f(v), \quad \sum_{v \in \mathcal{V}(P)} \lambda_v = 1, \quad \lambda_v \geq 0. \quad (6.98)$$

Moreover, f is unique in the sense that two affine functions coincide in P as long as they have identical values at each vertices v_i . Assume the opposite, there is another affine function $f' : \mathbb{R}^n \rightarrow \mathbb{R}^r$ satisfying $f'(v_i) = g_i$ for all $v_i \in \mathcal{V}(P)$. Then, consider $f - f'$, which is affine and $(f - f')(v_i) = g_i - g_i = 0$ for all $v_i \in \mathcal{V}(P)$. It then follows from the previous lemma that $f - f'$ is the function identically zero and thus $f = f'$.

In particular, we can present an explicit reconstruction for the case of an affine function on a simplex.

Lemma 6.3 *Let S be a simplex in $\mathbb{R}^n$ and $f : \mathbb{R}^n \rightarrow \mathbb{R}^r$ an affine function. The restriction of f to S is a convex combination of its values at the vertices and is given by*

$$f(x) = GW^{-1} \begin{bmatrix} x \\ 1 \end{bmatrix}, \quad x \in P, \quad (6.99)$$

where

$$G = [g_1 \cdots g_{n+1}],$$

$$W = \begin{bmatrix} v_1 & \cdots & v_{n+1} \\ 1 & \cdots & 1 \end{bmatrix},$$

are $r \times (n + 1)$ and $(n + 1) \times (n + 1)$ real matrices, respectively.

Linear Control Problems

For the linear control system (6.93), it is assumed that all the regions are given as polytopes with $\llbracket \pi_i \rrbracket = \{y \in \mathbb{R}^p \mid n_i^T y \leq g_i\}$, where $n_i \in \mathbb{R}^p$ and $g_i \in \mathbb{R}$. Note that $y(t) \in \llbracket \pi_i \rrbracket$ holds true if and only if $n_i^T Cx(t) \leq g_i$, which corresponds to a polytope in the state space $P_i = \{x \in \mathbb{R}^n \mid n_i^T Cx \leq g_i\}$.

We restrict our control $u(t)$ to be in the form of an affine function of state, i.e., $u(t) = kx(t) + \mu$, where k and μ are to be designed. The closed-loop dynamics will be

$$\dot{x}(t) = (A + Bk)x(t) + w, \quad (6.100)$$

which has an affine function on its right-hand side. Our task is to design the controller u , i.e., k and μ , in such a way that all trajectories of the closed-loop system starting from a polytope P_i can either all stay inside P_i or all transit to a neighboring polytope P_j after a finite period of time without intersecting with other regions during this transition process. The closed-loop system becomes a piecewise affine system with polytopes as the partition of the state space, which represents a special class of a hybrid system.

For each polytope, the following two problems are considered:

- *Invariant Control Problem:* The invariant control problem for the linear control system (6.93) with respect to a polytope P seeks an affine feedback control law $u = kx + \mu$ such that all trajectories for the closed-loop system $\xi(t)$ starting from P will remain in P forever, i.e., $\xi(0) \in P \Rightarrow \xi(t) \in P, \forall t \in \mathbb{R}^+$.
- *Control to Facet Problem:* Consider the linear control system (6.93) on a polytope P , and let F be a facet of P . The control to facet problem is to determine whether there exists an affine feedback control law $u = kx + \mu$ such that all trajectories for the closed-loop system starting from P will leave P through F after a finite time τ . Namely, to determine whether there exists a finite escape time $\tau > 0$ such that the following hold:
 - $\xi(t) \in P$ for $0 \leq t < \tau$;
 - $\xi(\tau) \in F$;
 - $\exists \epsilon > 0$ such that $\xi(t) \notin P \cup F$ for $\tau < t < \tau + \epsilon$.

Invariant Control Problem

Since the value of an affine function in a polytope can be determined by its values at the vertices of the polytope, the existence of such an affine feedback can be determined by some linear inequalities evaluated at (a finite number of) vertices of the polytope P . The following proposition characterizes all affine vector fields for which the polytope is an invariant.

Theorem 6.1 *The invariant control problem for the linear control system (6.93) with respect to a polytope P is solvable provided that the following sets are non-empty:*

$$U_P(v_i) = \bigcap_{F \in \mathcal{F}(v_i)} \{u \in \mathbb{R}^m \mid \eta_F^T (Av_i + a + Bu) < 0\}, \quad (6.101)$$

for all $v_i \in \mathcal{V}(P)$, where η_F is the normal vector for the facet F .

It is a very intuitive condition that basically requests the existence of a control signal to make the vector field point inside the polytope P for all vertices. The conditions form a collection of linear inequalities and can be easily checked. Moreover, when the sets are non-empty, a multi-affine control law $u = kx + \mu$ can be constructed, with the control value at vertex v_i being any element in $U_P(v_i)$.

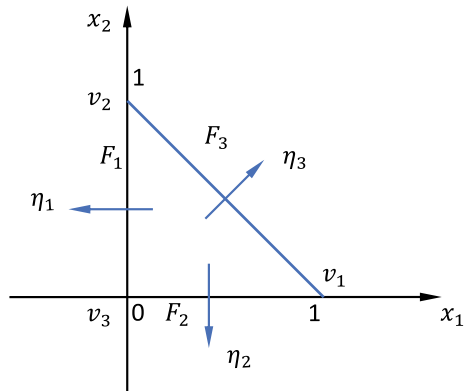
Example 6.17 Consider the following linear control system:

$$\dot{x}(t) = \begin{bmatrix} -1 & 0 \\ 1 & -1 \end{bmatrix} x(t) + \begin{bmatrix} 2 \\ 1 \end{bmatrix} u(t) + \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad (6.102)$$

$$y(t) = x(t). \quad (6.103)$$

Consider a simplex $\mathcal{S}$ defined by the intersection of $x_1 \geq 0$, $x_2 \geq 0$ and $x_1 + x_2 \leq 1$ with three facets $F_1 = \{(x_1, x_2) \in \mathcal{S} \mid x_1 = 0\}$, $F_2 = \{(x_1, x_2) \in \mathcal{S} \mid x_2 = 0\}$, and $F_3 = \{(x_1, x_2) \in \mathcal{S} \mid x_1 + x_2 = 1\}$ and outward normal vectors $\eta_1 = (-1, 0)^T$,

Fig. 6.11 Invariant control example



$\eta_2 = (0, -1)^T$, and $\eta_3 = (1, 1)^T$. It has three vertices $v_1 = (1, 0)^T$, $v_2 = (0, 1)^T$, and $v_3 = (0, 0)^T$ as shown in Fig. 6.11.

The control problem considered here is to keep all states inside $\mathcal{S}$ as long as they start from some point in $\mathcal{S}$, that is, we consider the invariant control problem. Following Theorem 6.1, the invariant control problem is solvable if the following conditions hold for each vertex.

First, for the first vertex $v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, note that $\mathcal{F}(v_1) = \{F_2, F_3\}$, hence $U_P(v_1)$ can be computed by the intersection between

$$\begin{bmatrix} 1 \\ 1 \end{bmatrix}^T \left(\begin{bmatrix} -1 & 0 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ 1 \end{bmatrix} u_1 + \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right) < 0,$$

and

$$\begin{bmatrix} 0 \\ -1 \end{bmatrix}^T \left(\begin{bmatrix} -1 & 0 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ 1 \end{bmatrix} u_1 + \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right) < 0.$$

So, we obtain $U_P(v_1) = \{-1 < u_1 < -\frac{1}{3}\}$, which is non-empty.

Similarly, we obtain $U_P(v_2) = \{-\frac{1}{2} < u_2 < 0\}$ and $U_P(v_3) = \{u_3 > 0\}$.

Since all sets $U_P(v_i)$ are non-empty, we can select $u_1 = -\frac{1}{2}$, $u_2 = -\frac{1}{4}$, and $u_3 = \frac{1}{2}$. Hence, we can obtain the following affine feedback control law:

$$f(x) = GW^{-1} \begin{bmatrix} x \\ 1 \end{bmatrix}, \quad x \in P,$$

where

$$G = \begin{bmatrix} -\frac{1}{2} & -\frac{1}{4} & \frac{1}{2} \end{bmatrix},$$

$$W = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}.$$

The control law

$$u = f(x_1, x_2) = \frac{1}{2} - x_1 - \frac{3}{4}x_2,$$

can keep all states in S . □

Control to Facet Problem

Similarly, the control to facet problem also admits a simple solution.

Theorem 6.2 *Let P be a polytope in $\mathbb{R}^n$ with a facet F and $\dot{x} = Ax + a + Bu$ be a linear control system. The control to facet problem admits a solution if the following*

sets are non-empty:

$$U_P(v_i) = \bigcap_{G \in \mathcal{F}(v_i)} \{u \in \mathbb{R}^m \mid \eta_G^T(Av_i + a + Bu) < 0\}, \quad (6.104)$$

for all $v_i \in \mathcal{V}(P)$ such that $F \notin \mathcal{F}(v_i)$, and

$$U_P(v_i) = \bigcap_{G \in \mathcal{F}(v_i), G \neq F} \left\{ u \in \mathbb{R}^m \mid \begin{array}{l} \eta_G^T(Av_i + a + Bu) < 0 \\ \eta_F^T(Av_i + a + Bu) > 0 \end{array} \right\}, \quad (6.105)$$

for all $v_i \in \mathcal{V}(P)$ such that $F \in \mathcal{F}(v_i)$.

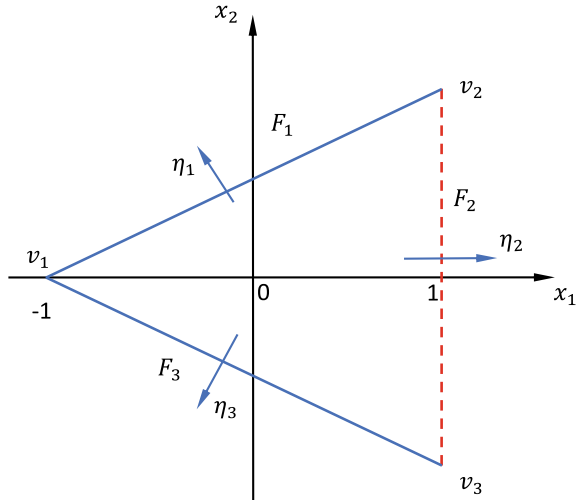
Intuitively speaking, the above sufficient conditions force the vector fields of the closed-loop system to point inside P for all vertices except the vertices of the facet F . It, therefore, guarantees that all trajectories starting from P will exit P through the facet F within a finite time duration. Hence, such a control solves the “Control to Facet Problem”.

Example 6.18 ([19]) Consider the following linear control system:

$$\dot{x}(t) = \begin{bmatrix} -1 & -1 \\ -2 & 1 \end{bmatrix} x(t) + \begin{bmatrix} 2 \\ -2 \end{bmatrix} u(t) + \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \quad (6.106)$$

$$y(t) = x(t). \quad (6.107)$$

Fig. 6.12 Invariant control example



Control is bounded $-1 \leq u \leq 1$. Consider a simplex $\mathcal{S}$ with vertices $v_1 = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$, $v_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, and $v_3 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$ as shown in Fig. 6.12.

The facet between v_1 and v_2 is denoted as F_1 , for which the normal vector $\eta_1 = \frac{1}{\sqrt{5}} \begin{bmatrix} -1 \\ 2 \end{bmatrix}$.

The facet between v_2 and v_3 is denoted as F_2 , for which the normal vector n_2 for the facet F_2 is $\eta_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$.

The facet between v_1 and v_3 is denoted as F_3 , for which the normal vector $\eta_3 = \frac{1}{\sqrt{5}} \begin{bmatrix} -1 \\ -2 \end{bmatrix}$.

The control problem considered here is to drive all states out of $\mathcal{S}$ through the facet F_2 (exit-facet). The control problem is solvable if the following conditions hold for each vertex.

First, for the first vertex $v_1 = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$, note that $\mathcal{F}(v_1) = \{F_1, F_3\}$, neither are exist-facet, hence we have the following if the following set is non-empty:

$$U_P(v_1) = \bigcap_{G \in \mathcal{F}(v_1)} \left\{ u \in \mathbb{R}^m \mid \eta_G^T (A v_1 + a + B u) < 0 \right\},$$

for the vertex v_1 . Plugging into A , B , a , v_1 , η_1 , and η_2 , we obtain

$$\frac{1}{\sqrt{5}} \begin{bmatrix} -1 \\ 2 \end{bmatrix}^T \left(\begin{bmatrix} -1 & -1 \\ -2 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ -2 \end{bmatrix} u_1 + \begin{bmatrix} 3 \\ 1 \end{bmatrix} \right) < 0,$$

and

$$\frac{1}{\sqrt{5}} \begin{bmatrix} -1 \\ -2 \end{bmatrix}^T \left(\begin{bmatrix} -1 & -1 \\ -2 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ -2 \end{bmatrix} u_1 + \begin{bmatrix} 3 \\ 1 \end{bmatrix} \right) < 0,$$

from which, together with the constraint on u_1 , we obtain $U_P(v_1) = \{\frac{1}{2} \leq u_1 \leq 1\}$.

Similarly, for vertex $v_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, we have $\mathcal{F}(v_2) = \{F_1, F_2\}$, so we have

$$\frac{1}{\sqrt{5}} \begin{bmatrix} -1 \\ 2 \end{bmatrix}^T \left(\begin{bmatrix} -1 & -1 \\ -2 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 2 \\ -2 \end{bmatrix} u_2 + \begin{bmatrix} 3 \\ 1 \end{bmatrix} \right) < 0,$$

and

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix}^T \left(\begin{bmatrix} -1 & -1 \\ -2 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 2 \\ -2 \end{bmatrix} u_2 + \begin{bmatrix} 3 \\ 1 \end{bmatrix} \right) > 0,$$

since F_2 is the exist facet. Together with the constraint on u_1 , we obtain $U_P(v_2) = \{-\frac{1}{6} \leq u_1 \leq 1\}$.

Following the same procedure, we obtain $U_P(v_3) = \{-1 \leq u_1 \leq -\frac{1}{2}\}$.

Since all sets $U_P(v_i)$ are non-empty, we can select $u_1 = \frac{1}{2}$, $u_2 = 0$, and $u_3 = -\frac{3}{4}$. Hence, we can obtain the following affine feedback control law:

$$f(x) = GW^{-1} \begin{bmatrix} x \\ 1 \end{bmatrix}, \quad x \in P,$$

where

$$G = \begin{bmatrix} \frac{1}{2} & 0 & -\frac{3}{4} \end{bmatrix},$$

$$W = \begin{bmatrix} -1 & 1 & -1 \\ 0 & 1 & -1 \\ 1 & 1 & 1 \end{bmatrix},$$

$$f(x) = -\frac{7}{16}x_1 + \frac{3}{8}x_2 + \frac{1}{16},$$

which can drive all states in S to exist from facet F_2 . $\square$

Once the above two control problems are solved, we can deduce a finite labeled transition system that is bisimilar to the closed-loop continuous system (seen as an infinite state transition system with label mapping consistent with $\llbracket \cdot \rrbracket$). The construction of the finite abstraction is conceptually simple. Note that the specification regions $\llbracket \pi_i \rrbracket$ for the output $y(t)$ are assumed to be polytopes and mutually exclusive and only share facets if adjacent. Correspondingly, $\llbracket \pi_i \rrbracket$ will imply a collection of polyhedrons in $\mathbb{R}^n$, denoted as P_i . All states in P_i are labeled with π_i for consistency. The collection of all such P_i forms the set of discrete states, while the initial state is the polytope containing x_0 , i.e., $x_0 \in P_0$. There is a transition from P_i to P_j , i.e., $(P_i, P_j) \in \rightarrow_P$ (assume $i \neq j$), if they share a facet F and the control to the facet F is solvable for P_i . There is a self-loop transition for a polytope P_i , i.e., $(P_i, P_i) \in \rightarrow_P$, if the invariant control problem is feasible for the polytope P_i . Hence, we obtain a transition system $T_P = (\{P_i\}, \{P_0\}, \rightarrow_P)$, which has finite states. The symbol π_i is then used to label the discrete state P_i . It is not difficult to see that the deduced labeled transition system is bisimilar to the continuous control system (6.93). Actually, the relation $R \subseteq \{P_i\} \times \mathbb{R}^n$, defined by $(P_i, x) \in R$ if $x \in P_i$, together with its reverse, forms a bisimulation relation between T_P and (6.93).

After obtaining the finite abstraction model T_P , one can use game theory-based methods to design a sequence of region transitions such that the required RTL specifications are satisfied. Due to the bisimulation relation between the linear control system and the abstracted model, the sequence of discrete region transitions can be mimicked by continuous trajectories $x(t)$, i.e., there exist continuous control signals to drive the output $y(t)$ so as to satisfy the RTL specifications. Furthermore, continuous control signals can be designed based on Theorems 6.1 and 6.2. For example, if a self-loop transition for region π_i occurs, then the invariant control law for region P_i can be designed based on results in Theorem 6.1. On the other hand, if there is

a transition from region π_i to π_j , the control law proposed in Theorem 6.2 can be adopted to achieve the region transition.

6.6.3 Multi-affine Control Systems

Next, we consider the symbolic control problem for a class of nonlinear dynamics

$$\dot{x}(t) = f(x(t), u(t)) = g(x) + Bu, \quad (6.108)$$

where $g(x) : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is assumed to be multi-affine, and $B \in \mathbb{R}^{n \times m}$ is constant.

Multi-affine Functions

A map $f : \mathbb{R} \rightarrow \mathbb{R}$ is said to be *affine* when for every $x, y \in \mathbb{R}$ and for every $\alpha, \beta \in \mathbb{R}$ satisfying $\alpha + \beta = 1$, the equality $f(\alpha x + \beta y) = \alpha f(x) + \beta f(y)$ holds. The number $\alpha x + \beta y$ is said to be an affine combination of x and y when $\alpha + \beta = 1$. The constraint $\alpha + \beta = 1$ can be expressed as a single variable $\lambda \in \mathbb{R}$ to define the affine combination as $\lambda x + (1 - \lambda)y$.

Definition 6.27 A map $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is said to be *multi-affine* when its projection to each $x_i, i = 1, \dots, n$, is affine.

Definition 6.28 A map $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is said to be *multi-affine* if for each $i = 1, \dots, m$, the map $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$ is multi-affine.

In other words, when we fix all variables except x_i , the map becomes affine. More specifically, the multi-affine map f consists of the sum of polynomials in the indeterminate $x_1, \dots, x_n$, with the property that the degree of any of the indeterminate $x_1, \dots, x_n$ is less than or equal to 1. Stated differently, f has the form

$$f(x) = \sum_{i_1, \dots, i_n \in \{0, 1\}} c_{i_1, \dots, i_n} x_1^{i_1} \dots x_n^{i_n},$$

where $c_{i_1, \dots, i_n} \in \mathbb{R}$ for all $i_1, \dots, i_n \in \{0, 1\}$ and using the convention that if $i_k = 0$, then $x_k^{i_k} = 1$.

It is also assumed that the regions of interest are all rectangles.

Definition 6.29 A n -rectangle E in $\mathbb{R}^n$ is a set defined by $E = \prod_{i=1}^n (a_i, b_i)$, where $a_i, b_i \in \mathbb{R}$ satisfying $a_i < b_i$ for $i = 1, \dots, n$.

The set of vertices of an n -rectangle is denoted by $\mathcal{V}(E)$ and defined by

$$\mathcal{V}(E) = \{x \in \mathbb{R}^n \mid x_i \in \{a_i, b_i\}\}. \quad (6.109)$$

The facet of an n -rectangle E is the intersection of the closure of E , with the hyperplane defined by $x_i = a_i$ or $x_i = b_i$. Given a vertex $v \in \mathcal{V}(E)$, we denote by $\mathcal{F}(v)$ the set of all facets containing v .

The following result tells us that the function value of a multi-affine function on an n -rectangle E is completely determined by the values of f on the vertices of E .

Lemma 6.4 ([20]) *Let E be an n -rectangle in $\mathbb{R}^n$ and $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ a multi-affine function. The function value of $x \in E$, $f(x)$ is completely determined by the values of f on the vertices of E , i.e.,*

$$x \in E \Rightarrow f(x) = \sum_{v \in \mathcal{V}(E)} \lambda_v f(v), \quad \sum_{v \in \mathcal{V}(E)} \lambda_v = 1. \quad (6.110)$$

Furthermore, the function f constructed is multi-affine and is unique.

Multi-affine Control System

The multi-affine control problem is to design (multi-affine state-feedback) control laws $u(t)$ such that its closed-loop system state trajectories $x(t)$ (with proper initial conditions) satisfy a given RTL formula ϕ . It is assumed that the regions corresponding to the atomic propositions in RTL are all bounded rectangles in $\mathbb{R}^n$. The reason for restricting the control law $u(t)$ to be a multi-affine function of $x(t)$, i.e., $u(t) = k(x(t))$ with $k(x)$ a multi-affine function, is to make the closed-loop system $\dot{x} = g(x) + Bk(x)$ remain multi-affine so as to take advantage of the nice properties of multi-affine functions described in Lemma 6.4.

The basic idea for multi-affine control design is very similar to the case of linear control systems over polytopes. It basically relies on the properties of multi-affine functions (Lemma 6.4) to obtain symbolic abstractions based on partitions of state space induced by n -rectangles. Similarly, we also try to make an n -rectangle E either invariant or all trajectories leaving E through the same facet. Recall that the facet of E is given by either $x_i = a_i$ or $x_i = b_i$. Moreover, to each facet F defined by $x_i = a_i$, we associate a normal vector η_F defined by $\eta_{Fj} = 0$ for $j \neq i$ and $\eta_{Fi} = -1$ for $j = i$. Contrarily, the normal vector η_F for facet defined by $x_i = b_i$ is given as $\eta_{Fj} = 0$ for $j \neq i$ and $\eta_{Fi} = 1$ for $j = i$.

For each n -rectangle, the following two problems are considered:

- **Invariant Control Problem:** The invariant control problem for a multi-affine control system $\dot{x} = g(x) + Bu$ on an n -rectangle E seeks a multi-affine feedback control law $k : \mathbb{R}^n \rightarrow \mathbb{R}^m$ such that all trajectories for the closed-loop system $\xi(t)$ starting from E will remain in E forever, i.e., $\xi(0) \in E \Rightarrow \xi(t) \in E, \forall t \in \mathbb{R}^+$.
- **Control to Facet Problem:** Consider a multi-affine control system $\dot{x} = g(x) + Bu$ on an n -rectangle E , and let F be a facet of E . The control to facet problem is to determine whether there exists a multi-affine feedback control law $k : \mathbb{R}^n \rightarrow \mathbb{R}^m$ such that all trajectories for the closed-loop system starting from E will leave E

through the facet F after a finite time τ . Namely, to determine whether there exists a finite escaping time $\tau > 0$ such that the following hold:

- $\xi(t) \in E$ for $0 \leq t < \tau$;
- $\xi(\tau) \in F$;
- $\exists \epsilon > 0$ such that $\xi(t) \notin E \cup F$ for $\tau < t < \tau + \epsilon$.

Similar to the linear control systems, we have the following results for multi-affine control systems.

Theorem 6.3 *Let E be an n -rectangle in $\mathbb{R}^n$ and $\dot{x} = g(x) + Bu$ be a multi-affine control system. The rectangle invariant problem admits a solution if the following sets are non-empty:*

$$U_E(v) = \bigcap_{F \in \mathcal{F}(v)} \{u \in \mathbb{R}^m \mid \eta_F^T(g(v) + Bu) < 0\}, \quad (6.111)$$

for all $v \in \mathcal{V}(E)$.

If these sets are non-empty, a multi-affine control law k can be constructed using this theorem where the control value at vertex v can be any element in $U_E(v)$.

Theorem 6.4 *Let E be an n -rectangle in $\mathbb{R}^n$ with a facet F and $\dot{x} = g(x) + Bu$ be a multi-affine control system. The control to facet problem admits a solution if the following sets are non-empty:*

$$U_E(v) = \bigcap_{G \in \mathcal{F}(v)} \{u \in \mathbb{R}^m \mid \eta_G^T(g(v) + Bu) < 0\}, \quad (6.112)$$

for all $v \in \mathcal{V}(E)$ such that $F \notin \mathcal{F}(v)$, and

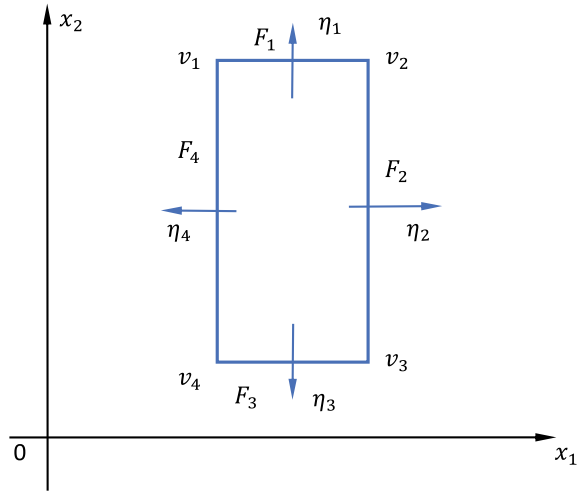
$$U_E(v) = \bigcap_{G \in \mathcal{F}(v), G \neq F} \left\{ u \in \mathbb{R}^m \mid \begin{array}{l} \eta_G^T(g(v) + Bu) < 0 \\ \eta_F^T(g(v) + Bu) > 0 \end{array} \right\}, \quad (6.113)$$

for all $v \in \mathcal{V}(E)$ such that $F \in \mathcal{F}(v)$.

The above results enable us to construct finite transition systems that are bisimilar abstractions for multi-affine control systems. To obtain the abstraction, we use n -rectangle as states and we place a transition from a rectangle E to itself when the invariant control problem has a solution for E , and a transition from E to E' is added when E and E' shares a common facet F and the control to facet problem is solvable for E with respect to the common facet F . The following example is adopted from [5] to illustrate the invariant control problem for multi-affine systems.

Example 6.19 Consider the Lotka–Volterra predator–prey model

$$\begin{cases} \dot{x}_1 = -x_1 + x_1x_2 - u, \\ \dot{x}_2 = x_2 - x_1x_2. \end{cases} \quad (6.114)$$

Fig. 6.13 Invariant control example

Solve the invariant control problem with respect to the rectangle $5 \leq x_1 \leq 6$, $2 \leq x_2 \leq 6$ (Fig. 6.13).

Four vertices $\mathcal{V}(E) = \{v_1 = (5, 6), v_2 = (6, 6), v_3 = (5, 2), v_4 = (6, 2)\}$ are as shown in Figure 6.13. Four facets F_1 , F_2 , F_3 , and F_4 , are with normal vectors $\eta_1 = (0, 1)$, $\eta_2 = (1, 0)$, $\eta_3 = (0, -1)$, and $\eta_4 = (-1, 0)$, respectively.

For the vertex $v_1 = (5, 6)$, its associated facets include $\mathcal{F}(v_1) = \{F_1, F_4\}$, so we have the following constraints:

$$(0, 1)(-5 + 5 \times 6 - u_1, 6 - 5 \times 6)^T < 0,$$

and

$$(-1, 0)(-5 + 5 \times 6 - u_1, 6 - 5 \times 6)^T < 0,$$

which imply that the set $U_E(v_1) = \{u_1 < 25\}$.

Similarly, we can obtain the following sets:

$$U_E(v_2) = \{u_2 > 30\}.$$

$$U_E(v_3) = \{u_3 < 5\}.$$

$$U_E(v_4) = \{u_4 > 6\}.$$

Since all sets are non-empty, the invariant control problem is solvable. In particular, we can choose the following control signal at each vertex:

$$u_1 = 15, u_2 = 40, u_3 = 0, u_4 = 15,$$

and the multi-linear feedback control law can be obtained as

$$k(x_1, x_2) = -\frac{115}{2} + 10x_1 - \frac{35}{4}x_2 + \frac{5}{2}x_1x_2,$$

which can drive all states in S outside of S through the exist facet F_3 . $\square$

6.6.4 Approximate Bisimulation and Nonlinear Control Systems

So far, we have considered only special cases when the continuous dynamics are linear or multi-affine. In this subsection, we consider a general nonlinear control system

$$\Sigma : \begin{cases} \dot{x}(t) = f(x(t), u(t)), \\ y(t) = g(x(t)), \end{cases} \quad (6.115)$$

where $x(t) \in \mathbb{R}^n$ is the state of the system, $u(t) \in U \subseteq \mathbb{R}^r$ is the control input, and $y(t) \in \mathbb{R}^p$ is the observed output of the system.

Once again, the goal is to design a controller such that the output $y(t)$ generated from the closed-loop system satisfies a given temporal logic specification ϕ . It is also assumed that ϕ is build from atomic propositions, $\Pi = \{\pi_0, \pi_1, \dots, \pi_m\}$, where each $\llbracket \pi_i \rrbracket$ stands for a regions in concern. Also, it is assumed that the region $\llbracket \pi_i \rrbracket$ is bounded and convex.

The development here follows the results in [14], and the basic idea is to introduce a simpler linear model in $\mathbb{R}^p$, for which we design a hybrid controller to achieve a modified version of the specification ϕ . Then, a controller is designed for the original nonlinear system to keep its trajectory always within a neighborhood of the trajectory from the designed linear systems provided that their initial conditions are close enough. Note that the modification on the specification ϕ is made in such a way that once the trajectories are close enough to a trajectory satisfying the modified version of ϕ , then the original specification ϕ holds true for all these nearby trajectories. For such a purpose, we under-approximate the region $\llbracket \pi_i \rrbracket$ using a polytope when it needs to be reached by $y(t)$; otherwise, over-approximate it using another polytope. Based on these approximations, we introduce new atomic propositions, $\tilde{\pi}_i$, and construct a new version of the specification ϕ , called $\tilde{\phi}$. Then, based on the previous section's results for controlling a linear control system over polytopes, we can design controllers and successful runs for proper initial conditions. Finally, we design controllers for the nonlinear system such that its output $y(t)$ tracks the trajectory of the closed-loop linear system with specified bounded errors. Then, the resulting output trajectory $y(t)$ is guaranteed to satisfy the initial user specification.

Approximate Simulation

Next, we introduce the tracking problem and modifications on the temporal logic specifications. First, the nonlinear control system (6.115) is abstracted to a linear control system:

$$\Sigma' : \dot{z}(t) = Az(t) + Bv(t), \quad z(t) \in \mathbb{R}^p, \quad z_0 \in \llbracket \pi_0 \rrbracket, \quad v \in V. \quad (6.116)$$

We would like the linear control system to approximate the trajectories of the nonlinear control system (6.115) in the following sense.

Definition 6.30 A relation $\mathcal{W} \subseteq \mathbb{R}^p \times \mathbb{R}^n$ is an approximate simulation relation of precision δ of Σ' by Σ if for all $(z_0, x_0) \in \mathcal{W}$,

1. $\|z_0 - g(x_0)\| \leq \delta$;
2. for all state trajectories $z(t)$ of Σ' such that $z(0) = z_0$, there exists a state trajectory $x(t)$ of Σ such that $x(0) = x_0$ and satisfies $(z(t), x(t)) \in \mathcal{W}$ for all $t \geq 0$.

An interface associated with the approximation simulation relation $\mathcal{W}$ allows us to choose the control inputs for the nonlinear control system (6.115) so that the states in the linear control system (6.116) and the states of the nonlinear control system (6.115) remain in $\mathcal{W}$.

Definition 6.31 A continuous function $u_{\mathcal{W}} : V \times \mathcal{W} \rightarrow U$ is an interface associated with the approximate simulation relation $\mathcal{W}$, if for all $(z_0, x_0) \in \mathcal{W}$, for all trajectories $z(t)$ of Σ' associated with input $v(t)$ such that $z(0) = z_0$, the trajectory of Σ given by

$$\dot{x}(t) = f(x(t), u_{\mathcal{W}}(v(t), z(t), x(t))), \quad x(0) = x_0, \quad (6.117)$$

satisfies for all $t \geq 0$, $(z(t), x(t)) \in \mathcal{W}$.

It is clear from these definitions that interconnecting the linear control system (6.116) and the nonlinear control system (6.115) through the interface $u_{\mathcal{W}}$

$$\begin{cases} \dot{x}(t) = f(x(t), u_{\mathcal{W}}(v(t), z(t), x(t))), & x(0) = x_0, \\ y(t) = g(x(t)), \end{cases}$$

satisfies for all $t \geq 0$, $\|y(t) - z(t)\| \leq \delta$ provided $\|g(x_0) - z_0\| \leq \delta$.

The approximate simulation relation can be constructed by the level sets of a simulation function, which is a positive function bounding the distance between the observations and non-increasing under parallel evolution of the systems.

Definition 6.32 Let $\mathcal{V} : \mathbb{R}^p \times \mathbb{R}^n \rightarrow \mathbb{R}^+$ be a continuous and piecewise differentiable function. Let $u_{\mathcal{V}} : V \times \mathbb{R}^p \times \mathbb{R}^n \rightarrow \mathbb{R}^p$ be continuous function. The function $\mathcal{V}$ is a simulation function of Σ' by Σ , and $u_{\mathcal{V}}$ is an associated interface if for all $(z, x) \in \mathbb{R}^p \times \mathbb{R}^n$,

$$\mathcal{V}(z, x) \geq \|z - g(x)\|^2, \quad (6.118)$$

$$\sup_{v \in \mathcal{V}} \left(\frac{\partial \mathcal{V}(z, x)}{\partial z} (Ax + Bv) + \frac{\partial \mathcal{V}(z, x)}{\partial x} f(x, u_{\mathcal{V}}(v, z, x)) \right) \leq 0. \quad (6.119)$$

Then, the approximate simulation relation can be defined as level sets of the simulation function.

Theorem 6.5 *Let the relation $\mathcal{W} \subseteq \mathbb{R}^p \times \mathbb{R}^n$ be given by*

$$\mathcal{W} = \left\{ (z, x) \mid \mathcal{V}(z, x) \leq \delta^2 \right\}. \quad (6.120)$$

If for all $v \in V$, for all $(z, x) \in \mathcal{W}$, $u_{\mathcal{V}}(v, z, x) \in U$, then $\mathcal{W}$ is an approximate simulation relation of precision δ of Σ' by Σ and $u_{\mathcal{W}} : V \times \mathcal{W} \rightarrow U$ given by $u_{\mathcal{W}}(v, z, x) = u_{\mathcal{V}}(v, z, x)$ is an associated interface.

Usually, it is not easy to find such a simulation function except for some special cases. Also, the arguments used by the simulation function is similar to the Lyapunov functions, so it is usually very conservative.

Modification of the Specification

In our setup, an RTL formula ϕ is provided as a controller specification for the original nonlinear control system, while we need to deduce a new RTL formula for the auxiliary linear system such that once a trajectory satisfies the modified specification, all neighboring trajectories will satisfy the original specification ϕ . Hence, we introduce the notation of δ -contraction so as to capture the robustness of satisfaction for a formula.

Definition 6.33 Given a radius $\delta \in \mathbb{R}^+ \cup \{+\infty\}$ and a point α in a normed space A , the δ -ball centered at α is defined as $B_{\delta}(\alpha) = \{\beta \in A \mid \|\alpha - \beta\| \leq \delta\}$. If $\Gamma \subseteq A$, then

$$C_{\delta}(\Gamma) = \{\alpha \in A \mid B_{\delta}(\alpha) \subseteq \Gamma\}, \quad (6.121)$$

is the δ -contraction and $B_{\delta}(\Gamma) = \{\alpha \in A \mid B_{\delta}(\alpha) \cap \Gamma \neq \emptyset\}$ is the δ -expansion.

Now, we define a new set of atomic propositions

$$\tilde{\Pi} = \left\{ \xi_{\alpha} \mid \alpha = \pi \text{ or } \neg\pi \text{ for } \pi \in \Pi \right\}. \quad (6.122)$$

Next, we describe how to translate an RTL ϕ on Π into a new RTL, denoted as **rob**(ϕ), on $\tilde{\Pi}$. First, we write ϕ into the negation normal form (NNF). Second, replace the occurrence of atomic proposition π and $\neg\pi$ with ξ_{π} and $\xi_{\neg\pi}$, respectively. Third, we define a new atomic map $\llbracket \cdot \rrbracket_{\delta}$ as follows:

$$\forall \xi \in \tilde{\Pi}, \llbracket \xi \rrbracket_\delta = \begin{cases} C_\delta(\llbracket \pi \rrbracket^c) & \text{if } \xi = \xi_{\neg\pi}, \\ C_\delta(\llbracket \pi \rrbracket) & \text{if } \xi = \xi_{\neg\neg\pi}, \end{cases} \quad (6.123)$$

where $\delta \in \mathbb{R}^+$ is a given positive scalar, and $\llbracket \pi \rrbracket^c$ stands for the complement of a the set $\llbracket \pi \rrbracket$.

Intuitively, it means that we δ -expand the region that $y(t)$ must avoid and δ -contract the region that it needs to reach. The following result tells us that if the trajectory satisfies the δ -robust specification, then any other trajectories that remain δ -close to the initial one will satisfy ϕ .

Theorem 6.6 *Consider a formula $\phi \in \Phi_\Pi$, which is built on a set of atoms Π , a map $\llbracket \cdot \rrbracket : \Pi \rightarrow \mathcal{P}(\mathbb{R}^p)$, and a number $\delta \in \mathbb{R}^+$, then for all functions $y(t)$ and $z(t)$ from $\mathbb{R}^+$ to $\mathbb{R}^p$ such that for all $t \geq 0$, $\|z(t) - y(t)\| \leq \delta$, it holds that $(z, \llbracket \cdot \rrbracket_\delta) \models \mathbf{rob}(\phi) \Rightarrow (y, \llbracket \cdot \rrbracket) \models \phi$.*

Then, one can design a hybrid controller for the linear control system to satisfy $\mathbf{rob}(\phi)$ as introduced in Sect. 6.6, i.e., the closed-loop trajectory $z(t)$ satisfies $\mathbf{rob}(\phi)$. Once this is done, the remaining task is to design the interface so that the trajectory $y(t)$ always stays in the δ neighborhood of $z(t)$.

6.7 Optimization-Based Approaches

In the previous section, we introduced an extension of LTL, called RTL. Actually, there are several temporal logic extensions that can handle dense-time continuous-variable signals, including metric temporal logic (MTL) [21] and signal temporal logic (STL) [22]¹. In this section, we focus on the formal synthesis of a control system with respect to STL, which is a special case of MTL. We focus on STL due to the relative simplicity of its syntax and semantics, and the fact that it is popular in the control and robotics literature. Furthermore, as we will see, there are some elegant results that allow us to encode the synthesis problem (i.e., finding a signal that satisfies an STL specification) as an optimization problem.

6.7.1 Signal Temporal Logic

Signal Temporal Logic is defined over a signal $y : \mathbb{T} \mapsto \mathbb{R}^n$, where $\mathbb{T}$ is the time domain. When considering a discrete-time signal, $\mathbb{T}$ is the natural number. For continuous-time signals, $\mathbb{T}$ is the real number. Note that even when we consider a discrete-time signal, the signal value at any time is a continuous variable. First, some notation needs to be defined. We denote the value of signal y at time t as

¹ The material for this section is based on the lecture notes taken by our graduate student Vince Kurtz.

$y(t)$. The values of signal y between times t_1 and t_2 are denoted as $(y, t_1 : t_2)$, and $(y, t : \infty) = (y, t)$ is used to denote the values of signal y from t onward.

Definition 6.4 STL is defined over predicates π . Each predicate is in turn defined by a function $\mu^\pi : \mathbb{R}^n \mapsto \mathbb{R}$ and a constant $c \in \mathbb{R}$.

- $\pi = (\mu^\pi(y(t)) \geq c)$ is an STL formula.
- Given STL formula φ , $\neg\varphi$ is an STL formula.
- Given STL formulas φ_1 and φ_2 , $\varphi_1 \wedge \varphi_2$ is an STL formula.
- Given STL formulas φ_1 and φ_2 and times t_1 and t_2 , $\varphi_1 \sqcup_{[t_1, t_2]} \varphi_2$ is an STL formula.

As with the other logics we have discussed so far, the “and” ($\wedge$) and “not” ($\neg$) operators can be combined to define “or”, “implies”, and so on. Similarly, the temporal operators “always” and “eventually” can be defined as follows:

$$\Diamond_{[t_1, t_2]} \varphi = \text{True} \sqcup_{[t_1, t_2]} \varphi,$$

$$\Box_{[t_1, t_2]} \varphi = \neg \Diamond_{[t_1, t_2]} \neg \varphi.$$

We can now define the semantics of STL as follows, using $\models$ as usual to denote satisfaction. As with LTL and CTL, this will be a recursive definition.

Definition 6.34 The STL semantics is formally defined as follows:

- $y \models \varphi$ if and only if $(y, 0) \models \varphi$;
- $(y, t) \models \pi$ if and only if $\mu^\pi(y(t)) \geq c$;
- $(y, t) \models \neg\varphi$ if and only if $(y, t) \not\models \varphi$;
- $(y, t) \models \varphi_1 \wedge \varphi_2$ if and only if $(y, t) \models \varphi_1$ and $(y, t) \models \varphi_2$;
- $(y, t) \models \varphi_1 \sqcup_{[t_1, t_2]} \varphi_2$ if and only if $\exists t' \in [t + t_1, t + t_2]$ such that $\forall t'' \in [t + t_1, t']$, $(y, t'') \models \varphi_1$, and $(y, t') \models \varphi_2$.

Using the above semantics, we can define the semantics for other Boolean operators as well as temporal operators. For example, the semantics for temporal operators “always” and “eventually”.

- $(y, t) \models \Diamond_{[t_1, t_2]} \varphi$ if and only if $\exists t' \in [t + t_1, t + t_2]$ such that $(y, t') \models \varphi$.
- $(y, t) \models \Box_{[t_1, t_2]} \varphi$ if and only if $\forall t' \in [t + t_1, t + t_2]$, $(y, t') \models \varphi$.

Example 6.20 Consider the signal shown in Fig. 6.14 and the predicates $\pi_1 = (y \geq 0)$ and $\pi_2 = (y < 2)$. Then, we can see that the signal does satisfy the STL $\varphi_1 = \pi_1$, since the initial value of y is positive. However, the STL $\varphi_2 = \Box_{[0, 10]} \pi_1$ does not hold since $y(t)$ is below zero for some short period of time during the range of $[0, 10]$. $\square$

As this example shows, STL allows us to easily specify the properties of one-dimensional signals. But since each predicate is based on the function $\mu^\pi(\cdot)$, we don’t need to limit ourselves to one-dimensional signals as the following example illustrates.

Fig. 6.14 A simple example signal to illustrate STL semantics

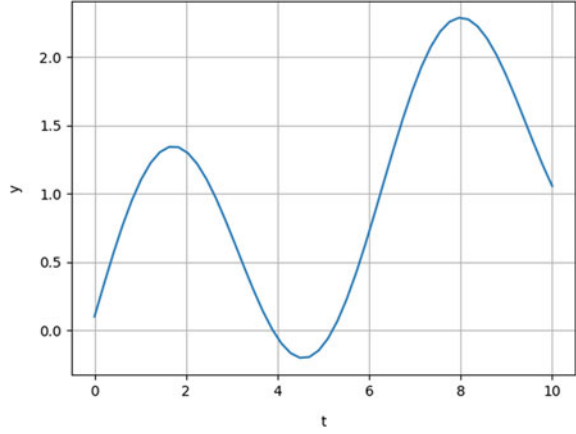
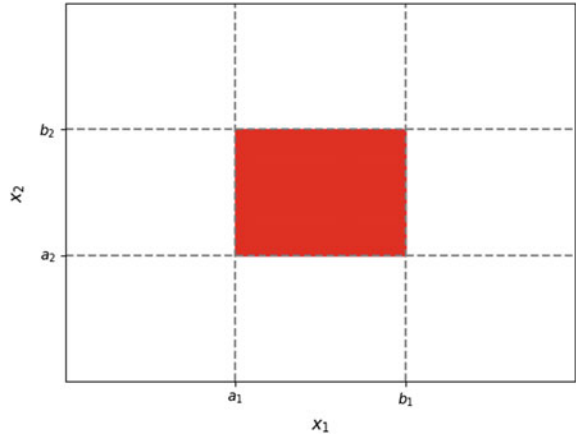


Fig. 6.15 An obstacle avoidance example



Example 6.21 Consider a robot with position $[x_1, x_2]^T$ moving in a 2D workspace. There is an obstacle that the robot must avoid, shown in red in Fig. 6.15. We can encode obstacle avoidance in an STL specification by first defining the following predicates over the signal $y(t) = [x_1(t), x_2(t)]^T$:

- $\pi_1 = ([1, 0]y \geq a_1)$ denotes that the x_1 position must be greater than a_1 ;
- $\pi_2 = ([1, 0]y \leq b_1)$ denotes that the x_1 position must be less than b_1 ;
- $\pi_3 = ([0, 1]y \geq a_2)$;
- $\pi_4 = ([0, 1]y \geq b_2)$

and then combining these predicates into the specification

$$\varphi = \Box_{[0, \infty]} \neg (\pi_1 \wedge \pi_2 \wedge \pi_3 \wedge \pi_4),$$

which requires that the position of the robot never falls into the red region, i.e., avoid the obstacle always.

Our function $\mu^\pi(\cdot)$ is not restricted to be a linear function either (although some STL synthesis methods do require a linear function). For example, if we had a circular obstacle with radius r , we could define a predicate $\pi = (x_1^2 + x_2^2 \geq r^2)$. $\square$

You may have noticed that for some specifications, e.g., φ_1 in the first example, you only need to check the signal at a single point, while for other specifications, such as the obstacle avoidance specification φ in the second example, you need to check the signal over a very long and possibly infinite duration. We can formalize this notion with the concept of the STL time bound.

Definition 6.5 The STL time bound b^φ of formula φ is defined recursively as follows:

- $b^\pi = 0$;
- $b^{\neg\varphi} = b^\varphi$;
- $b^{\varphi_1 \wedge \varphi_2} = \max(b^{\varphi_1}, b^{\varphi_2})$;
- $b^{\varphi_1 \sqcup_{[t_1, t_2]} \varphi_2} = t_2 + \max(b^{\varphi_1}, b^{\varphi_2})$.

The STL time bound tells us the duration over which we need to observe the signal $y(t)$ before we can determine if $y \models \varphi$. For instance, for $\square_{[0,10]} \Diamond_{[0,6]} \varphi$, a trajectory of length $N \geq 10 + 6 = 16$ is sufficient to determine whether the formula φ is satisfiable. Clearly, this bound can be computed in time linear in the length of the formula. If $b^\varphi < \infty$, we say that the formula φ is *bounded*. Otherwise, we say that φ is *unbounded*.

6.7.2 Trajectory Synthesis

For trajectory synthesis, we consider state trajectories x_t of the following deterministic discrete-time control system:

$$x_{t+1} = f(x_t, u_t), \quad (6.124)$$

where $x \in \mathbb{R}^n$, $u \in \mathcal{U} \subset \mathbb{R}^m$, and the initial condition x_0 is given. Note that the control u is constrained to lie in $\mathcal{U}$, which is a compact subset of $\mathbb{R}^m$. Given an initial state x_0 and a sequence of control inputs $\mathbf{u}^N = u_0 u_1 u_2 \dots u_{N-1}$, the resulting horizon- N run of the system (6.124) is a sequence of states, $\mathbf{x}(x_0, \mathbf{u}^N) = x_0 x_1 x_2 \dots x_N$, where $x_t \in \mathcal{X}$ is the state of the system at time index t , and for each $0 \leq t \leq N-1$, there exists a control input $u_t \in \mathcal{U}$ such that $x_{t+1} = f(x_t, u_t)$.

Our goal is to find a control sequence $\mathbf{u}^N$ such that the resulting state trajectory $\mathbf{x}(x_0, \mathbf{u}^N) = x_0, x_1, \dots, x_N$ satisfies a given time-bounded (with bound N) STL specification φ . We can also introduce a generic *cost function* $J(\mathbf{x}, \mathbf{u})$, and then the goal will be to design the control sequence to satisfy the STL specification φ and meanwhile minimize the cost function J .

Our basic idea is to convert the trajectory synthesis problem into an optimization problem, for which the requirement of satisfying the STL formula φ is converted into constraints, together with constraints due to state equation and constraints on control or states.

State Evolution Constraints

The constraints due to physical dynamics are straightforward to obtain. For instance, to request that the state trajectory $\mathbf{x}(x_0, \mathbf{u}^N)$ satisfies the state equation (6.124) for $t = 0, 1, \dots, N$, we can add the following constraints (assume that f is linear of the form $Ax + Bu$):

$$\begin{aligned} x_1 &= Ax_0 + B_u u_0, \\ x_2 &= Ax_1 + B_u u_1, \\ &\vdots \\ x_N &= Ax_{N-1} + B_u u_{N-1}. \end{aligned}$$

In addition, we can add $u_t \in \mathcal{U}$ for all $0 \leq t \leq N-1$ if the control signal is constrained. For the case that $\mathcal{U}$ is a convex polyhedron, we also get linear constraints.

STL Constraints

For each predicate $\pi = (\mu(x) \geq 0)$ at each time step, we define a binary variable $z_t^\pi \in \{0, 1\}$, for $t = 0, 1, \dots, N$, and add the following constraints on z_t^π :

$$\mu(x_t) + M(1 - z_t^\pi) \geq 0, \quad (6.125)$$

$$\mu(x_t) - Mz_t^\pi \leq 0, \quad (6.126)$$

where M is a sufficiently large constant. These constraints enforce that $z_t^\pi = 1$ if and only if $\mu(x_t) \geq 0$, namely the predicate π holds true at time t . Note that if we assume that all predicates μ in φ are linear or affine functions of x , then the obtained constraints will be mixed-integer linear constraints.

Beyond atomic predicates, we will show how to formulate constraints on the variable z_t^φ , which corresponds to an STL formula φ , such that $z_t^\varphi = 1$ if and only if φ holds at position t in the state sequence of length N . This is done iteratively on sub-formulas of φ from inside out. In particular, we recursively generate mixed-integer constraints for every sub-formula of φ as follows (assume the sub-formulas of φ have already been properly encoded):

- for negation $\varphi = \neg\phi$, we add the constraint $z_t^\varphi = 1 - z_t^\phi$;

- for conjunction $\varphi = \bigwedge_{i=1}^m \varphi_i$, we add the following constraints on variable $z_t^\varphi = \bigwedge_{i=1}^m z_t^{\varphi_i}$:

$$z_t^\varphi \leq z_t^{\varphi_i}, \quad (6.127)$$

$$z_t^\varphi \geq 1 - m + \sum_{i=1}^m z_t^{\varphi_i}, \quad (6.128)$$

where $i = 1, 2, \dots, m$;

- for disjunction $\varphi = \bigvee_{i=1}^m \varphi_i$, we add the following constraints on variable $z_t^\varphi = \bigvee_{i=1}^m z_t^{\varphi_i}$:

$$z_t^\varphi \geq z_t^{\varphi_i}, \quad (6.129)$$

$$z_t^\varphi \leq \sum_{i=1}^m z_t^{\varphi_i}, \quad (6.130)$$

where $i = 1, 2, \dots, m$.

For temporal operators, we can first convert them into the Boolean operations on corresponding variables, which can then be written into mixed-integer constraints as follows:

- $\varphi = \square_{[t_1, t_2]} \varphi_1$ becomes $z_t^\varphi = \bigwedge_{t' \in [t+t_1, t+t_2]} z_{t'}^{\varphi_1}$;
- $\varphi = \diamond_{[t_1, t_2]} \varphi_1$ becomes $z_t^\varphi = \bigvee_{t' \in [t+t_1, t+t_2]} z_{t'}^{\varphi_1}$;
- $\varphi = \varphi_1 \sqcup_{[t_1, t_2]} \varphi_2$ becomes $z_t^\varphi = \bigvee_{t' \in [t+t_1, t+t_2]} \left(z_{t'}^{\varphi_2} \wedge \bigwedge_{t'' \in [t+t_1, t']} z_{t''}^{\varphi_1} \right)$.

Once we have built up these constraints to include the definition of the overall specification φ , we add one last constraint:

$$z_0^\varphi = 1. \quad (6.131)$$

This ensures that the overall specification is satisfied at time zero, which is the definition of a signal satisfying a specification.

This mixed-integer encoding process is powerful because it allows us to use specialized mixed-integer solvers to find a signal x that satisfies a specification φ . These solvers are guaranteed to find an optimal solution if one exists: a property known as completeness. Furthermore, if a solution is found, it is guaranteed to satisfy the specification: a property known as soundness.

6.7.3 Robust Semantics

The STL semantics that we introduced above allow us to tell whether or not a signal x satisfies a given specification φ . But we only get a binary answer: $x \models \varphi$ or $x \not\models \varphi$. In many cases, we might also be interested in *how well* a signal satisfies a specification.

Does our robot avoid an obstacle by a wide margin or just barely? Do we fail to reach the target by just a fraction of a second or by several minutes?

One way of quantifying how well a signal satisfies a specification is through an alternative definition of STL semantics, known as *robust semantics*. In the literature, these are sometimes referred to as qualitative semantics. STL robust semantics define a function ρ^φ which maps the signal y to a real value. This value, which is called the “STL score” or “robustness degree”, is positive if the signal satisfies the specification and negative otherwise. The greater the robustness degree, the stronger the satisfaction. As with the original semantics, the STL robust semantics are defined recursively as follows:

- $\rho^\varphi((y, t)) = \mu^\pi(y(t)) - c$;
- $\rho^{\neg\varphi}((y, t)) = -\rho^\varphi((y, t))$;
- $\rho^{\varphi_1 \wedge \varphi_2}((y, t)) = \min(\rho^{\varphi_1}((y, t)), \rho^{\varphi_2}((y, t)))$;
- $\rho^{\Diamond_{[t_1, t_2]} \varphi}((y, t)) = \max_{t' \in [t+t_1, t+t_2]} (\rho^\varphi((y, t')))$;
- $\rho^{\Box_{[t_1, t_2]} \varphi}((y, t)) = \min_{t' \in [t+t_1, t+t_2]} (\rho^\varphi((y, t')))$;
- $\rho^{\varphi_1 \sqcup_{[t_1, t_2]} \varphi_2}((y, t)) = \max_{t' \in [t+t_1, t+t_2]} \left(\min(\rho^{\varphi_1}((y, t')), \min_{t'' \in [t+t_1, t']} (\rho^{\varphi_2}((y, t'')))) \right)$.

The robustness of satisfaction of an STL formula is computed recursively from the above semantics in a straightforward manner by propagating the values of the functions associated with each operand using min and max operators corresponding to various STL operators. For example, the robust satisfaction of $\mu_1 \equiv x - 3 > 0$ at time $t = 0$ is $\rho^{\mu_1} = x_0 - 3$, and the robustness of satisfaction of $\varphi = \Box_{[0,2]} \mu_1$ at time $t = 0$ is

$$\rho^\varphi(\mathbf{x}, t) = \min_{t \in [0,2]} \rho^{\mu_1}(\mathbf{x}, t) = \min_{t \in [0,2]} (x_t - 3).$$

The robustness score $\rho^\varphi(\mathbf{x}, t)$ should be interpreted as how much the signal $\mathbf{x}$ satisfies φ . Its absolute value can be viewed as the “distance” of $\mathbf{x}$ from the set of trajectories satisfying or violating φ , in the space of projections with respect to the functions μ that define the predicates of φ .

The robustness of satisfaction of the STL specification provides a natural objective for the MILP that solves the trajectory optimization problem. Given an STL formula φ , we introduce a variable r_t^φ such that $r_t^\varphi = \rho^\varphi(\mathbf{x}, t)$ and $r_t^\varphi > 0$ if and only if φ holds at position t . The variable r_t^φ for an STL formula φ is defined recursively as follows:

1. for each atomic predicate $\mu \in P$, we set variables $r_t^\mu = \mu(x_t)$ for time indices $t = 0, 1, \dots, N$;
2. for r_t^ψ where ψ is a Boolean formula, we assume that each operand φ has a corresponding variable $r_t^\varphi = \rho^\varphi(\mathbf{x}, t)$. Then the Boolean operations on r_t^ψ are defined recursively as follows:
 - (a) Negation: For $\psi = \neg\varphi$, we add the constraint $r_t^\psi = -r_t^\varphi$.
 - (b) Conjunction: For $\psi = \bigwedge_{i=1}^m \varphi_i$, we introduce new binary variables $p_i^{\varphi_i}$, for $i = 1, 2, \dots, m$, and the following constraints:

$$\sum_{i=1}^m p_t^{\varphi_i} = 1, \quad (6.132)$$

$$r_t^\psi \leq r_t^{\varphi_i}, \quad (6.133)$$

$$r_t^{\varphi_i} - (1 - p_t^{\varphi_i})M \leq r_t^\psi \leq r_t^{\varphi_i} + (1 - p_t^{\varphi_i})M, \quad (6.134)$$

where M is a sufficiently large positive number and $i = 1, 2, \dots, m$.

- (c) Disjunction: For $\psi = \bigvee_{i=1}^m \varphi_i$, we introduce new binary variables $p_t^{\varphi_i}$, for $i = 1, 2, \dots, m$, and the following constraints:

$$\begin{aligned} \sum_{i=1}^m p_t^{\varphi_i} &= 1, \\ r_t^\psi &\geq r_t^{\varphi_i}, \\ r_t^{\varphi_i} - (1 - p_t^{\varphi_i})M &\leq r_t^\psi \leq r_t^{\varphi_i} + (1 - p_t^{\varphi_i})M, \end{aligned}$$

where M is a sufficiently large positive number and $i = 1, 2, \dots, m$.

The encoding for temporal operators follows the same rules as we have shown earlier. By definition, $r_t^\varphi > 0$ if and only if φ is satisfied at time t . Therefore, to define the robustness-based encoding of temporal operators, we can replace the constraints over z_t^φ by these constraints that compute the value of r_t^φ , whereas the additional constraint $z_0^\varphi = 1$ shall be replaced by $r_0^\varphi > 0$.

Example 6.22 Consider a robot moving in a 2D workspace. We assume that the robot's dynamics are given by a simple single integrator,

$$\begin{aligned} x_{t+1} &= x_t + u_t dt, \\ y_t &= x_t, \end{aligned}$$

where $x_t \in \mathbb{R}^2$ is the robot's position and $dt = 0.01s$ is the sampling period. The robot must visit one of two target regions (blue), avoid an obstacle (red), and reach a goal (green). With the obstacle and regions defined as discussed above, this specification can be written as

$$\varphi = \Box_{[0,T]}(\neg \text{obstacle}) \wedge \Diamond_{[0,T]}(\text{target}_1 \vee \text{target}_2) \wedge \Diamond_{[0,T]} \text{goal}. \quad (6.135)$$

For different choices of T , mixed-integer programming gives us the results shown in Fig. 6.16. Note that the optimization time and robustness degrees are given in the legend. $\square$

There are several interesting things to note about this example. First of all, while mixed-integer programming guarantees a globally optimal solution, such a solution is not necessarily unique. More importantly, the time needed to find an optimal trajectory increases dramatically with the number of time steps (in the $T = 35$ case,

it takes over an hour!). This is due to the fact that mixed-integer programming—even in the simplest case of 0–1 integer linear programming—is an NP-complete problem. This means that mixed-integer programming scales exponentially with the number of binary variables, and since we introduced (at least) one new binary variable for each time step, the problem scales (at least) exponentially in the number of time steps.

To mitigate the computation complexity of the trajectory synthesis from the MILP encoding of the STL specification φ , one can turn to a model predictive control (MPC) framework and solve the following finite horizon optimization problem at each time step t :

$$\arg \min_{\mathbf{u}^{H,t}} J(\mathbf{x}(x_t, \mathbf{u}^{H,t})), \quad (6.136)$$

$$\text{s.t. } \mathbf{x}(x_0, \mathbf{u}) \models \varphi, \quad (6.137)$$

where H is a finite horizon and $\mathbf{u}^{H,t}$ is the horizon- H control input computed at each time step t , and $\mathbf{u} = u_0^{H,0} u_0^{H,1} u_0^{H,2} \dots$

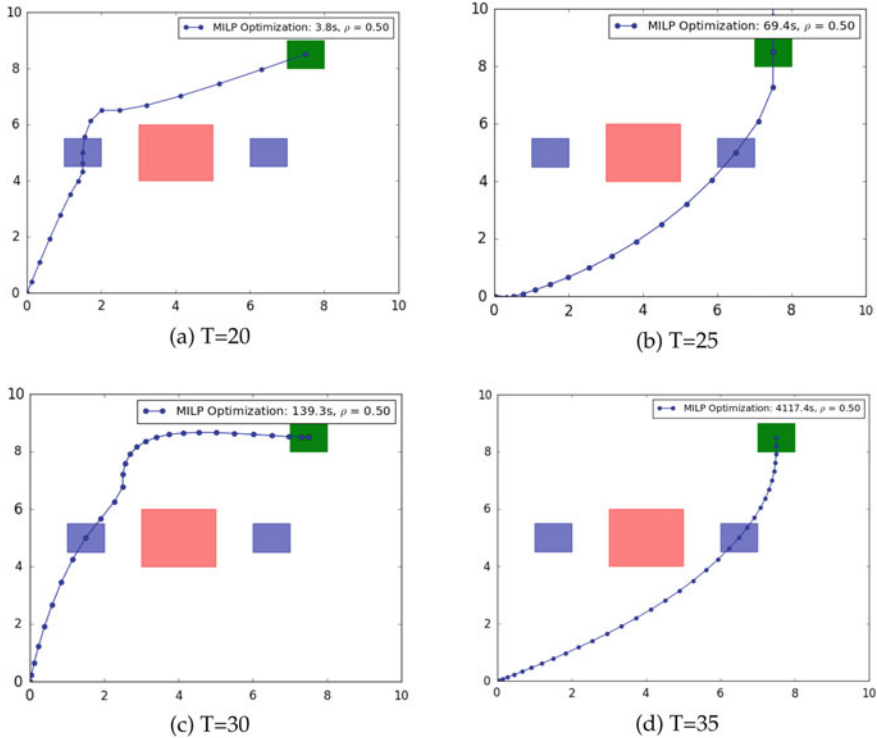


Fig. 6.16 Results of STL synthesis with mixed-integer programming

Intuitively, by solving the above optimization problem at each step t , we will search for a finite trajectory of fixed horizon length H such that the accumulated trajectory satisfies φ . The length of the predictive horizon H is set to be no less than the bound of the specification formula φ . Initially ($t = 0$), we first synthesize control $\mathbf{u}^{H,0}$ using the open-loop MILP encoding, including the STL constraints on the horizon- H trajectory. At the next step of the MPC, we will solve for $\mathbf{u}^{H,1}$, while constraining the previous values of x_0, u_0 in the MILP, the STL constraints on the trajectory up to time H . In this manner, we will keep track of the history of states in order to ensure that the formula is satisfied over the horizon- H prefix of the trajectory, while solving for $\mathbf{u}^{H,t}$ at every time step t .

6.8 Summary

Formal synthesis for hybrid systems is very challenging and is an active research area. By formal synthesis, we mean the design of controllers for a plant (modeled by a hybrid automaton) to satisfy given specifications, such as safety, reachability, or temporal logic specifications. Besides scalability, the main challenges for formal synthesis also include uncertainties since the plant needs to work in an open environment, whose behavior is not fully controllable. In this chapter, we reviewed several existing approaches to the formal synthesis of hybrid control systems that were originally proposed by researchers in computer science and control theory.

- We started this chapter with game-theoretic approaches since formal synthesis problems can be naturally studied as games between the controller and the environment. For games over discrete automata, it was shown that safety, reachability, and Büchi games are all deterministic and one can obtain memoryless winning strategies from winning states.
- Differential games belong to the family of dynamic games. The solution of dynamic games is defined as an equilibrium, e.g., a Nash equilibrium, at which no player has any motivations to diverge unilaterally. Differential games can, therefore, be solved as coupled optimal control problems.
- While discrete games are played with respect to qualitative goals, differential games have a quantitative performance functional. The connection between discrete games and differential games can be achieved by a properly defined value function.
- The extensions from discrete games solving to hybrid games may follow two different approaches: one is through abstractions and the other is through symbolic computations.
- We illustrated abstraction-based approaches through timed games since there exists a finite quotient game automaton for a timed game, over which the algorithms for game solving and supervisor design introduced earlier can be directly applied.
- Existence of finite quotient systems that are equivalent to the original hybrid system is rare, so we turned to a different approach—the design of continuous feedback

controllers to obtain closed-loop regulated continuous dynamics that have finite bisimilar or approximate bisimilar abstractions. We illustrated this idea using several control systems, such as linear, multi-affine, and nonlinear control systems in formal synthesis.

- The main idea of the symbolic approach is to symbolically represent sets of states and operate on these sets of states symbolically instead of explicitly. We illustrate the main idea of the symbolic approach using timed games and sets of states represented by affine formulas.
- Similar to the symbolic reachability analysis in the verification chapter, we also need to handle discrete transitions and continuous evolutions separately for hybrid systems. However, the difference is that the set of states, which may jump outside the target region by following an uncontrollable transition, need to be avoided before reaching the target region. Hence, it becomes a reach-while-avoid operation, which is different from the reachability analysis introduced in the verification chapter.
- The optimization-based approach for formal synthesis follows a different path. Its main idea is to convert the formal specifications, usually given as signal temporal logic, real temporal logic, or other linear temporal logic with real-time extensions, into mixed-integer constraints. The conversion follows similar procedures for the conversion of the Boolean formulas into mixed-integer constraints when we talked about the optimal control of mixed-logic dynamic systems in the optimal control chapter. After the conversion, the formal synthesis problem can be cast in a receding horizon manner by solving a finite mixed-integer optimization program at every time step.

6.9 Notes and Further Reading

Casting a formal synthesis problem into a game between the controller and the environment turns out to be a very fruitful idea. However, we did not discuss how to formulate such a game. The answer to this can be found in the literature on reactive synthesis. The basic problem in reactive synthesis is to synthesize a reactive module that interacts with the uncontrollable environments and satisfies a given specification. The specification is usually given as temporal logic formulas, say LTL, over atomic propositions from environments (uncontrollable) and from the system (controllable). The key issue in reactive synthesis is how to convert the given specification into a game with appropriate winning conditions. If the specification is given as an LTL formula, a usual path is first to translate the LTL specification into a Büchi automaton, and then determinize the obtained Büchi automaton—to obtain an equivalent deterministic ω -automaton. Since deterministic Büchi automata are strictly less expressive than non-deterministic Büchi automata, we need to generalize the acceptance conditions, and therefore need to introduce other types of ω -automata, such as Muller, Rabin, Street, and Parity automata [23, 24]. Once we obtain a deterministic ω -automaton, converting the automaton into a game with corresponding winning

conditions is straightforward. Parity automata are of particular interest in this family since a Parity game converted from a Parity automaton is determined and memoryless winning strategies exist [25]. The games obtained in such a way from the reactive synthesis literature are usually turn-based games, which is different from the games that we introduced earlier in this chapter. In the games introduced in this chapter, both players play simultaneously instead of one after the other (turn-based). However, it is easy to convert a turn-based game into a game that we formulated in this chapter.

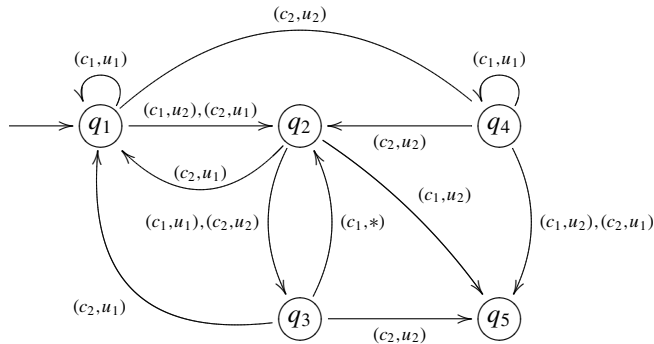
In this chapter, we discussed timed games in detail. There are extensions of timed games to more general hybrid systems, such as linear hybrid games [26, 27], rectangular games [28], o-minimal hybrid games [29], and general nonlinear hybrid games [30, 31]. The basic idea—fix-point computation based on backward reachability analysis—is the same as solving timed games when we are dealing with more general hybrid games. However, the main difficulty is due to the computation of the reach-while-avoid operator. The techniques that we mentioned in the verification chapter for symbolic reachability analysis, such as over- or under-approximation of reachable sets using polyhedrons or ellipsoids, are directly applicable here. A method of particular interest is based on the numerical solutions of the interconnected Hamilton–Jacobi partial differential equations proposed in [31–33] in the late 1990s and early 2000s, and it was shown that the solutions to these Hamilton–Jacobi equations correspond exactly to the boundaries of reachable sets. Interested readers may refer to [34, 35] for recent developments.

Reachability control on simplex for linear and affine dynamics follows the work in [15]. The results on multi-affine systems are mainly based on [20]. The concept of approximate bisimulation was proposed in [36]. Our treatment of the synthesis of hybrid controllers for nonlinear systems mainly follows the results in [37]. Although we only considered the symbolic control for continuous control systems, the extension of the idea to the cases of hybrid systems is not difficult provided that the regions of concern, such as invariant sets and guard sets, are all assumed to be polyhedrons or rectangles (for multi-affine dynamics). For example, the control problems for rectangular multi-affine hybrid systems were investigated in [38] using similar ideas to the ones described in Sect. 6.6.2, while the reachability and control problems for hybrid systems with piecewise affine dynamics defined on simplices were considered in [39] using the techniques discussed in Sect. 6.6.

Symbolic control methods using simulation or approximate simulation-based quotient transition systems have been exploited in the literature and the trend is to design controllers in an automatic way; see, e.g., [16, 40]. As an application of the theory introduced here, symbolic motion planning for robots has been considered in the literature; see, e.g., [14, 41]. Readers who are interested in symbolic control may refer to the book [5, 42] for more comprehensive and detailed discussions on this topic. The design methods introduced in [5] also include reactive synthesis based on similarity games and fixed-point computation, which are rooted in the computer science literature, see, e.g., [43], and form a parallel approach to the supervisory control theory discussed here. Readers who are interested in optimization-based methods may refer to the survey paper [44].

6.10 Exercises

1. **Simulations:** It is strongly recommended to simulate each of the examples in this chapter so as to gain insight into the system's behavior and verify the claimed results. MATLAB and Stateflow provide an excellent starting point for simulation work. Additional specialized commercial software is also available as noted above.
2. Consider a discrete game structure with turn-based players, namely the state space Q is divided into two disjoint sets $Q_1 \cup Q_2$ and the Player i chooses actions when and only when the state is at $q \in Q_i$. Show that a game structure with turn-based players can be translated into a game structure with simultaneous plays.
3. Show that the set F_i computed in the safety game algorithm (Algorithm 6.1) represents the states that can be forced to remain in F_i by a controller for at least i steps.
4. Show that the set F_i computed in the reachability game algorithm (Algorithm 6.2) represents the states that can be driven to F by a controller for at most i steps.
5. Show that the set W_i computed in the Büchi game algorithm (Algorithm 6.3) represents the set of states from which the plant can be driven to F at least i times.
6. Consider a game structure shown below.



- (a) Solve the safety game $\Box\{q_1, q_2, q_3\}$, namely to stay within the set of states $F = \{q_1, q_2, q_3\}$.
 - (b) Solve the reachability game $\Diamond\{q_4\}$.
 - (c) Solve the Büchi game $\Box\Diamond\{q_4\}$.
7. Extend the definition of Nash equilibrium and its necessary conditions to n -player non-cooperative game (6.6)–(6.7) for open-loop strategies.
 8. Redo Example 6.10 by adding the missing details.
 9. Consider a scalar control system

$$\dot{x}(t) = x(t) + u(t).$$

It is desired to find the control law that minimizes the following performance index:

$$J = \frac{1}{4}x^2(T) + \int_0^T \frac{1}{4}u^2(t)dt,$$

where the terminal time T is given.

- (a) Derive the HJB equation and its associated boundary conditions of the problem.
- (b) Find the optimal control law u^* in a state-feedback form.

10. Consider the optimal control problem given by the linear time-variant system

$$\dot{x}(t) = A(t)x(t) + B(t)u(t), \quad (6.138)$$

and cost function

$$J(u(\cdot)) = \int_{t_0}^{t_f} [x(t)^T Q(t)x(t) + u(t)^T R(t)u(t)]dt + x(t_f)^T Q_f x(t_f),$$

where t_0 and t_f are fixed. Assume that $Q(t) : [t_0, t_f] \rightarrow \mathbb{R}^{n \times n}$ and $R(t) : [t_0, t_f] \rightarrow \mathbb{R}^{m \times m}$ are continuous. In addition, for any $t \in [t_0, t_f]$, $Q(t)$ is a symmetric positive semi-definite matrix, and $R(t)$ is a symmetric positive definite matrix for fixed t . Also, assume that $Q_f \in \mathbb{R}^{n \times n}$ is a symmetric positive semi-definite matrix.

- (a) Derive the HJB equation and its associated boundary conditions of the problem.
- (b) Find the optimal control law u^* in a state-feedback form.

11. Consider $N \geq 2$ player game with the following quadratic cost functional:

$$J_i = \frac{1}{2} \int_0^T (x' Q_i x + u_i' R_i u_i) dt + \frac{1}{2} x'(T) Q_f^i x(T),$$

whose state evolution $x(t) \in \mathbb{R}^n$ is governed by the linear system equation

$$\dot{x}(t) = A(t)x(t) + \sum_{i=1}^N B_i(t)u_i(t).$$

Here, $A(\cdot)$, $B_i(\cdot)$, Q_i , R_i , and Q_f^i are matrices of appropriate dimensions. Furthermore, Q_i and Q_f^i are symmetric, and R_i is positive definite.

- (a) Determine the open-loop Nash equilibrium.
- (b) Determine the closed-loop Nash equilibrium.

12. Redo Example 6.13 and fill in the missing details.
13. Show that the timed predecessor operator defined for timed games satisfies the following distribution law:

$$\text{Pred}_t \left(\bigcup_i G_i, \bigcup_j B_j \right) = \bigcup_i \bigcap_j \text{Pred}_t(G_i, B_j).$$

14. Revisit Example 6.14 and solve the timed game using symbolic approaches.
15. Revisit the time optimal control problem for timed automata in Sect. 6.4.4 and fill in the missing details.
16. Consider a classical pursuit–evasion game involving two identical vehicles. If the vehicles get too close to each other, a collision occurs. One of the vehicles (the pursuer) wants to cause a collision, while the other (the evader) tries to avoid the collision. By fixing the evader at the planar origin and aligning with the x -axis, we can obtain the following dynamic model to describe the relative model of pursuer with respect to the evader:

$$\dot{x} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{bmatrix} = \begin{bmatrix} -v + v \cos x_3 + u x_2 \\ v \sin x_3 - u x_1 \\ d - u \end{bmatrix} = f(x, u, d), \quad (6.139)$$

where $[x_1, x_2]^T \in \mathbb{R}^n$ stands for the relative planar location, $x_3 \in [0, 2\pi)$ represents the relative heading angle, and $v \geq 0$ is the linear velocity of each vehicle. The control input is the angular velocity of the evader, $u \in \mathcal{U} = [-1, 1]$, and the disturbance input is the angular velocity of the pursuer, $d \in \mathcal{D} = [-1, 1]$. A collision occurs if $\sqrt{x_1^2 + x_2^2} \leq d_0$ for any value of x_3 . Determine the set of initial states from which the pursuer can cause a collision despite the best efforts of the evader.

17. Prove Theorem 6.1.
18. Prove Theorem 6.2.
19. Consider the following linear control system:

$$\begin{aligned} \dot{x}(t) &= \begin{bmatrix} -1 & 0 \\ 1 & -1 \end{bmatrix} x(t) + \begin{bmatrix} 1 \\ 1 \end{bmatrix} u(t), \\ y(t) &= x(t), \end{aligned}$$

with $x \in \mathbb{R}^2$ and $-1 \leq u \leq 1$. Consider a simplex $\mathcal{S}$ defined by the intersection of $x_1 \geq 0$, $x_2 \geq 0$, and $x_1 + x_2 \leq 1$ with three facets $F_1 = \{(x_1, x_2) \in \mathcal{S} \mid x_1 = 0\}$, $F_2 = \{(x_1, x_2) \in \mathcal{S} \mid x_2 = 0\}$, and $F_3 = \{(x_1, x_2) \in \mathcal{S} \mid x_1 + x_2 = 1\}$ and outward normal vectors $\eta_1 = (-1, 0)^T$, $\eta_2 = (0, -1)^T$, and $\eta_3 = (1, 1)^T$. Three vertices $v_1 = (1, 0)^T$, $v_2 = (0, 1)^T$, and $v_3 = (0, 0)^T$. The control problem considered here is to drive all states out of $\mathcal{S}$ through the facet F_3 (exit-facet).

20. Prove Lemma 6.4.

21. Prove Theorem 6.3.
22. Prove Theorem 6.4.
23. Redo the trajectory synthesis problem of Example 6.22 with respect to the following reach-while-avoid specification:

$$\varphi = \Box_{[0,100]} \neg \psi_{\text{obstacle}} \wedge \Diamond_{[0,100]} \psi_{\text{goal}},$$

where $\psi_{\text{obstacle}} = (3 \leq x_1 \leq 5) \wedge (4 \leq x_2 \leq 6)$ and $\psi_{\text{goal}} = (7 \leq x_1 \leq 8) \wedge (8 \leq x_2 \leq 9)$ are state formulas representing entering the obstacle and goal regions, respectively.

References

1. E. M. Clarke, O. Grumberg, and D. Peled. *Model Checking*. MIT press, 1999.
2. Amir Pnueli and Roni Rosner. On the synthesis of a reactive module. In *Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 179–190. ACM, 1989.
3. P. Ramadge and W. M. Wonham. The control of discrete event systems. *Proceedings of the IEEE*, 77(1):81–98, 1989.
4. R. Alur, T. Henzinger, G. Lafferriere, and G. J. Pappas. Discrete abstractions of hybrid systems. In P. J. Antsaklis, editor, *Proceedings of the IEEE: Special issue on hybrid systems*, volume 88, pages 971–984. IEEE Press, 2000.
5. P. Tabuada. *Verification and Control of Hybrid Systems: A Symbolic Approach*. Springer, Springer New York, 2009.
6. Martino Bardi and Italo Capuzzo-Dolcetta. *Optimal control and viscosity solutions of Hamilton-Jacobi-Bellman equations*. Springer Science & Business Media, 2008.
7. Daniel Liberzon. *Calculus of variations and optimal control theory: a concise introduction*. Princeton University Press, 2011.
8. João P Hespanha. *Noncooperative game theory: An introduction for engineers and computer scientists*. Princeton University Press, 2017.
9. Tamer Başar and Geert Jan Olsder. *Dynamic noncooperative game theory*. SIAM, 1998.
10. Engelbert J Dockner, Steffen Jorgensen, Ngo Van Long, and Gerhard Sorger. *Differential games in economics and management science*. Cambridge University Press, 2000.
11. J. Lygeros, S. Sastry, and C. Tomlin. *EE 291E Lecture notes: The Art of Hybrid Systems*. 2001.
12. Eugene Asarin and Oded Maler. As soon as possible: Time optimal control for timed automata. In *Hybrid Systems: Computation and Control*, pages 19–30. Springer, 1999.
13. Mark Reynolds. Continuous temporal models. In *AI 2001: Advances in Artificial Intelligence*, pages 414–425. Springer, 2001.
14. G. E. Fainekos, A. Girard, H. Kress-Gazit, and G. J. Pappas. Temporal logic motion planning for dynamic robots. *Automatica*, 45(2):343–352, 2009.
15. L. Habets and Jan H. van Schuppen. A control problem for affine dynamical systems on a full-dimensional polytope. *Automatica*, 40(1):21–35, 2004.
16. M. Kloetzer and C. Belta. A fully automated framework for control of linear systems from temporal logic specifications. *Automatic Control, IEEE Transactions on*, 53(1):287–297, 2008.
17. Günter M Ziegler. *Lectures on polytopes*, volume 152. Springer, 1995.
18. Carl W Lee. Subdivisions and triangulations of polytopes. In *Handbook of discrete and computational geometry*, pages 271–290. CRC Press, Inc., 1997.
19. LCGJM Habets and Jan H Van Schuppen. A control problem for affine dynamical systems on a full-dimensional polytope. *Automatica*, 40(1):21–35, 2004.

20. C. Belta and L. C. G. J. M. Habets. Controlling a class of nonlinear systems on rectangles. *Automatic Control, IEEE Transactions on*, 51(11):1749–1759, Nov 2006.
21. Ron Koymans. Specifying real-time properties with metric temporal logic. *Real-time systems*, 2(4):255–299, 1990.
22. Oded Maler and Dejan Nickovic. Monitoring temporal properties of continuous signals. In *Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems*, pages 152–166. Springer, 2004.
23. Shmuel Safra. On the complexity of omega-automata. In *[Proceedings 1988] 29th Annual Symposium on Foundations of Computer Science*, pages 319–327. IEEE, 1988.
24. Nir Piterman. From nondeterministic buchi and streett automata to deterministic parity automata. In *21st Annual IEEE Symposium on Logic in Computer Science (LICS'06)*, pages 255–264. IEEE, 2006.
25. Robert McNaughton. Infinite games played on finite graphs. *Annals of Pure and Applied Logic*, 65(2):149–184, 1993.
26. Massimo Benerecetti, Marco Faella, and Stefano Minopoli. Reachability games for linear hybrid systems. In *Proceedings of the 15th ACM international conference on Hybrid Systems: Computation and Control*, pages 65–74, 2012.
27. Massimo Benerecetti and Marco Faella. Automatic synthesis of switching controllers for linear hybrid systems: Reachability control. *ACM Transactions on Embedded Computing Systems (TECS)*, 16(4):1–27, 2017.
28. Thomas A Henzinger, Benjamin Horowitz, and Rupak Majumdar. Rectangular hybrid games. In *International Conference on Concurrency Theory*, pages 320–335. Springer, 1999.
29. Patricia Bouyer, Thomas Brihaye, and Fabrice Chevalier. O-minimal hybrid reachability games. arXiv preprint [arXiv:0911.4833](https://arxiv.org/abs/0911.4833), 2009.
30. J. Lygeros, C. Tomlin, and S. Sastry. Controllers for reachability specifications for hybrid systems. *Automatica*, 35(3):349–370, 1999.
31. Claire J Tomlin, John Lygeros, and S Shankar Sastry. A game theoretic approach to controller design for hybrid systems. *Proceedings of the IEEE*, 88(7):949–970, 2000.
32. Ian Mitchell and Claire J Tomlin. Level set methods for computation in hybrid systems. In *International Workshop on Hybrid Systems: Computation and Control*, pages 310–323. Springer, 2000.
33. Claire J Tomlin, Ian Mitchell, Alexandre M Bayen, and Meeko Oishi. Computational techniques for the verification of hybrid systems. *Proceedings of the IEEE*, 91(7):986–1001, 2003.
34. Somil Bansal, Mo Chen, Sylvia Herbert, and Claire J Tomlin. Hamilton-Jacobi reachability: A brief overview and recent advances. In *2017 IEEE 56th Annual Conference on Decision and Control (CDC)*, pages 2242–2253. IEEE, 2017.
35. Mo Chen and Claire J Tomlin. Hamilton-Jacobi reachability: Some recent theoretical advances and applications in unmanned airspace management. *Annual Review of Control, Robotics, and Autonomous Systems*, 1:333–358, 2018.
36. A. Girard and G. J. Pappas. Approximation metrics for discrete and continuous systems. *Automatic Control, IEEE Transactions on*, 52(5):782–798, 2007.
37. A. Girard and G. J. Pappas. Hierarchical control system design using approximate simulation. *Automatica*, 45(2):566–571, 2009.
38. LCGJM Habets, M Kloetzer, and Calin Belta. Control of rectangular multi-affine hybrid systems. In *Decision and Control, 2006 45th IEEE Conference on*, pages 2619–2624. IEEE, 2006.
39. LCGJM Habets, Pieter J Collins, and Jan H van Schuppen. Reachability and control synthesis for piecewise-affine hybrid systems on simplices. *Automatic Control, IEEE Transactions on*, 51(6):938–948, 2006.
40. Paulo Tabuada. An approximate simulation approach to symbolic control. *Automatic Control, IEEE Transactions on*, 53(6):1406–1418, 2008.
41. C. Belta, A. Bicchi, M. Egerstedt, E. Frazzoli, E. Klavins, and G.J. Pappas. Symbolic planning and control of robot motion. *Robotics and Automation Magazine, IEEE*, 14(1):61–70, March 2007.

42. Calin Belta, Boyan Yordanov, and Ebru Aydin Gol. *Formal methods for discrete-time dynamical systems*, volume 89. Springer, 2017.
43. Wolfgang Thomas. On the synthesis of strategies in infinite games. In *STACS 95*, pages 1–13. Springer, 1995.
44. Calin Belta and Sadra Sadraddini. Formal methods for control synthesis: An optimization perspective. *Annual Review of Control, Robotics, and Autonomous Systems*, 2:115–140, 2019.
45. Oded Maler. Control from computer science. *Annual Reviews in Control*, 26(2):175–187, 2002.
46. Kirk, Donald E. *Optimal control theory: an introduction*. Courier Corporation, 2004.
47. Bouyer, Patricia and Fahrenberg, Uli and Larsen, Kim Guldstrand and Markey, Nicolas and Ouaknine, Joël and Worrell, James. Model checking real-time systems. In *Handbook of model checking*, pages 1001–1046. Springer, 2018.

Appendix A

Continuous and Sampled-Data Systems

This appendix includes useful results and properties of continuous and discrete dynamical systems. Section A.1 discusses modeling of continuous- and discrete-valued signals that are functions of continuous or discrete time. Modeling of dynamical systems is also discussed [1]. In Sect. A.2, a summary of important results from the theory of linear dynamical systems described by state variable representations is given [2]. Sampled-data systems models are used to study the control of continuous systems using digital devices and are discussed in Sect. A.3. If signal quantization is included, then the sampled-data models become hybrid systems as discussed in Sect. A.3. Notes and Further Reading may be found in Sect. A.4.

A.1 Modeling Signals and Systems

Continuous- and discrete-valued signals are briefly discussed first, followed by a brief introduction to modeling of dynamical systems.

Modeling Signals

Signals are used to represent quantities of interest in a system, such as voltage, pressure, and temperature that may change with time. There are continuous-valued signals and discrete-valued signals often coexisting in a system—the term “mixed signals” is sometimes used in this case. The voltage $v(t)$ in an analog circuit or the temperature $T(t)$ in a room are continuous-valued signals since they take on real values, from the field of reals $\mathcal{R}$; a voltage can be 3.132... volts and a temperature can be 68.47384... degrees. On the other hand, if the signal $q(t)$ denotes the status of a home heating system and it takes on only {on, off} values (or {0, 1}), then it is a discrete-valued signal since the values it takes on can be mapped to the set $\mathbb{N}$ of non-negative integers. The values of the signals of interest typically vary with time, and in $v(t)$, the independent variable t in the argument most often denotes time.

Continuous-Valued Signals: In a continuous-valued signal $v(t)$, the time t can be continuous or discrete and the signal is then continuous time or discrete time. In analog signals, typically both the values of $v(t)$ and the time t are real numbers; that is, $v(t)$ is defined for any real t in some interval and it may take on any real value. Examples include voltages and currents in an RLC circuit. Discrete-time signals are defined only for discrete values of time t and not for any real value t . For instance, a voltage may be known every tenth of a second, but not in between. This is the case, for example, when the value of the signal, which could be representing the temperature or pressure in an engine, is known only from periodic measurements or samples.

The relation between a continuous-time signal $f(t)$ and its sampled version $\{f(t_k)\}$ has been of great interest in several fields, such as signal processing and numerical analysis. For example, the celebrated Sampling theorem prescribes the sampling rate so as to be able to reconstruct the original (frequency band limited) signal $f(t)$ from its samples (see, e.g., [3]). Every electrical engineer has studied continuous- and discrete-time signals in the time domain and in transform domains using the Fourier, Laplace, and Z-transforms.

Discrete-Valued Signals: These are signals that take on values only from a discrete set, e.g., from the binary set $\{0, 1\}$. They are typically discrete-time signals. They are common in switching and hybrid systems. In sampling continuous-valued signals, the continuous values of the discrete-time signal are typically obtained or stored using digital devices that have finite word length. That is, a continuous-valued discrete-time signal becomes a discrete-valued discrete-time signal, also called a digital signal, by quantization, that approximates each continuous value by a discrete approximate value taken from a discrete set; for example, a voltage value equal to 2.12345... is represented as 2.12 if only two decimals are used. Such quantized, discrete-valued signals typically are not studied together with the continuous-valued ones in digital control courses primarily because of the mathematical difficulties. Instead, some probabilistic analysis of the quantization effects is frequently performed separately to validate the design. It should be noted also that today's digital devices tend to use longer word lengths, and so the use of continuous-valued signals instead of discrete-valued signals in the analysis of system behavior is adequate for many practical purposes.

Modeling Dynamical Systems: The dynamical behavior of systems can be understood by studying their mathematical descriptions. The flight path of an airplane subject to certain engine thrust, rudder, and elevator angles and under particular wind conditions, the behavior of an automobile on cruise control when climbing a hill of certain elevation, or the evolution in time of a production system in manufacturing can be predicted using mathematical descriptions of the behavior of interest. Mathematical relations that typically involve differential or difference equations or finite automata and Petri nets are used to describe the behavior of processes and predict their response when certain inputs are applied (see, e.g., [2–5]). Although computer simulation is an excellent tool for validating predicted behavior and thus for enhancing our understanding of processes, it is certainly not an adequate substi-

tute in analysis or design for generating the information captured in a mathematical model, when of course such a model is available.

Throughout the centuries, a great deal of progress has been made in developing mathematical descriptions of physical phenomena. In doing so, laws or principles of physics, chemistry, biology, economics, etc., are invoked to derive mathematical expressions (usually equations), which characterize the evolution in time of the variables that are of interest. The availability of such mathematical descriptions enables us to make use of the vast resources offered by the many areas of applied and pure mathematics to conduct qualitative and quantitative studies of the behavior of processes.

A given model of a physical process may give rise to several different mathematical descriptions. For example, when applying Kirchhoff's voltage and current laws to a low-frequency transistor model, one can derive a set of differential and algebraic equations, or a set consisting only of differential equations, or a set of integro-differential equations, and so forth. The process of mathematical modeling, from a physical phenomenon to a model to a mathematical description, is essential in science and engineering.

To capture phenomena of interest accurately and in tractable mathematical form is a demanding task, as can be imagined, and requires a thorough understanding of the process involved. In most nontrivial cases, this type of modeling process is close to an art form, since a good mathematical description must be detailed enough to accurately describe the phenomena of interest and at the same time simple enough to be amenable to analysis. Depending on the applications on hand, a given mathematical description of a process may be further simplified before it is used in analysis and especially in design procedures.

A point that cannot be overemphasized is that mathematical descriptions characterize processes only approximately. Most often, this is the case because the complexity of physical systems defies exact mathematical formulation. In many other cases, however, it is our own choice that a mathematical description of a given process approximates the actual phenomena only by a certain desired degree of accuracy for simplicity. For example, in the description of RLC circuits, one could use nonlinear differential equations, which take into consideration parasitic effects in the capacitors. Most often, however, it suffices to use linear ordinary differential equations with constant coefficients to describe the voltage-current relations of such circuits, since typically such a description provides an adequate approximation and it is much easier to work with linear rather than nonlinear differential equations.

There are, of course, many examples of systems that cannot be conveniently described by continuous models and differential equations. Such systems include production lines in manufacturing, computer networks, traffic systems, etc., where their evolution in time depends on complex interactions of the timing of various discrete events. Such discrete event dynamical systems are modeled by discrete models, such as finite automata. Since many of these systems are man-made, the

models tend to be easier to construct and more accurate (although they tend to grow very large in the number of states) than in the case of modeling physical systems; however, the same modeling considerations as the ones discussed above still apply.

A.2 Continuous- and Discrete-Time Linear Systems

In this section, several important results from the theory of Linear Dynamical Systems are presented. Detailed treatment of the topic may be found in Linear Systems books, e.g., [2].

Internal Descriptions—State Variables

We start with nonlinear differential equation representations in state variable form.

- *Initial value problem*

$$\dot{x} = f(t, x), \quad x(t_0) = x_0, \quad (\text{A.1})$$

or

$$\phi(t) = x_0 + \int_{t_0}^t f(s, \phi(s))ds, \quad (\text{A.2})$$

where $\phi(t)$ is a solution of (A.1).

- *Successive approximations*

$$\begin{aligned} \phi_0(t) &= x_0, \\ \phi_{m+1}(t) &= x_0 + \int_{t_0}^t f(s, \phi_m(s))ds, \quad m = 0, 1, 2, \dots \end{aligned} \quad (\text{A.3})$$

Under certain conditions, as $m \rightarrow \infty$, ϕ_m converges uniformly (on compact sets) to the unique solution of (A.1).

- *Linearization*

Given is $\dot{x} = f(t, x)$ and a solution ϕ . The Jacobian matrix is

$$\frac{\partial f}{\partial x}(t, x) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}(t, x) & \cdots & \frac{\partial f_1}{\partial x_n}(t, x) \\ \vdots & & \vdots \\ \frac{\partial f_n}{\partial x_1}(t, x) & \cdots & \frac{\partial f_n}{\partial x_n}(t, x) \end{bmatrix}. \quad (\text{A.4})$$

For $A(t) = \frac{\partial f}{\partial x}(t, \phi(t))$,

$$\dot{z} = A(t)z, \quad (\text{A.5})$$

is the linearized equation about the solution ϕ .

- The *solution* of

$$\dot{x} = A(t)x + g(t), \quad (\text{A.6})$$

with $x(t_0) = x_0$, is given by the variation of constants formula

$$\phi(t, t_0, x_0) = \Phi(t, t_0)x_0 + \int_{t_0}^t \Phi(t, s)g(s)ds, \quad (\text{A.7})$$

where the state transition matrix $\Phi(t, t_0)$ is given by

$$\Phi(t, t_0) = I + \int_{t_0}^t A(s_1)ds_1 + \int_{t_0}^t A(s_1) \int_{t_0}^{s_1} A(s_2)ds_2ds_1 + \cdots, \quad (\text{A.8})$$

the Peano–Baker series.

- In the *time-invariant case* $\dot{x} = Ax$,

$$\begin{aligned} \phi(t, t_0, x_0) &= \left(I + \sum_{k=1}^{\infty} \frac{A^k(t-t_0)^k}{k!} \right) x_0, \\ &= \Phi(t, t_0)x_0 \triangleq \Phi(t-t_0)x_0, \\ &= e^{A(t-t_0)}x_0, \end{aligned} \quad (\text{A.9})$$

where

$$e^A = I + \sum_{k=1}^{\infty} \frac{A^k}{k!}. \quad (\text{A.10})$$

External Descriptions—Impulse Response and Transfer Function

- The *unit impulse* (Dirac delta distribution) $\delta(t)$ satisfies

$$\int_a^b f(\tau)\delta(t-\tau)d\tau = f(t),$$

where $a < t < b$.

- The *input–output description* of a linear, continuous-time, causal, time-invariant system that is at rest at $t = t_0$ is given by

$$y(t) = \int_{t_0}^t H_P(t-\tau)u(\tau)d\tau = \int_{t_0}^t H_P(\tau)u(t-\tau)d\tau. \quad (\text{A.11})$$

$H_P(t-\tau)$ ($= H_P(t-\tau, 0)$) is the continuous-time unit impulse response. Specifically, $H_P(t, \tau)$ is the response of the system at time t due to a unit impulse at time τ starting with the system at rest.

- The *input–output description* of a linear, discrete-time, causal, time-invariant system that is at rest at $k = k_0$ is given by

$$y(n) = \sum_{k=k_0}^n H(n-k)u(k) = \sum_{k=k_0}^n H(k)u(n-k). \quad (\text{A.12})$$

$H(n-k)(= H(n-k, 0))$ is the discrete-time unit (pulse) impulse response of the system. Specifically, $H(n, k)$ is the response of the system at time n due to a unit pulse at time k starting with the system at rest.

System Trajectories

Continuous-Time Systems

- Given $\dot{x} = Ax + Bu$,

$$x(t) = e^{At}x(0) + \int_0^t e^{A(t-s)}Bu(s)ds,$$

is its solution, the *variation of constants formula*.

If in addition $y = Cx + Du$, then the *total response of the system* is

$$y(t) = Ce^{A(t-t_0)}x_0 + C \int_{t_0}^t e^{A(t-s)}Bu(s)ds + Du(t). \quad (\text{A.13})$$

The *impulse response* is

$$H(t) = \begin{cases} Ce^{At}B + D\delta(t), & t \geq \tau, \\ 0, & t < 0, \end{cases} \quad (\text{A.14})$$

and the *transfer function* is

$$\hat{H}(s) = C(sI - A)^{-1}B + D. \quad (\text{A.15})$$

Note that $H(s) = \mathcal{L}(H(t, 0))$; that is, the transfer function is the Laplace transform of the impulse response.

Discrete-Time Systems

- Consider the *discrete-time system*

$$x(k+1) = Ax(k) + Bu(k), \quad y(k) = Cx(k) + Du(k). \quad (\text{A.16})$$

Then

$$y(k) = CA^kx(0) + \sum_{j=0}^{k-1} CA^{k-(j+1)}Bu(j) + Du(k), \quad k > 0. \quad (\text{A.17})$$

The *discrete-time unit impulse (pulse) response* is

$$H(k, 0) = \begin{cases} CA^{k-1}B & k \geq 0, \\ D & k = 0, \\ 0 & k < 0, \end{cases} \quad (\text{A.18})$$

and the *transfer function* is

$$\hat{H}(z) = C(zI - A)^{-1}B + D. \quad (\text{A.19})$$

Note that $\hat{H}(z) = \mathcal{Z}\{H(k, 0)\}$; that is, the transfer function is the Z-transform of the impulse (pulse) response.

Equivalent Representations

$$\begin{aligned} \dot{\tilde{x}} &= \tilde{A}\tilde{x} + \tilde{B}u, \\ y &= \tilde{C}\tilde{x} + \tilde{D}u, \end{aligned} \quad (\text{A.20})$$

where

$$\tilde{A} = PAP^{-1}, \quad \tilde{B} = PB, \quad \tilde{C} = CP^{-1}, \quad \tilde{D} = D, \quad (\text{A.21})$$

is equivalent to $\dot{x} = Ax + Bu$, $y = Cx + Du$.

Sampled-Data Systems

- Given $\dot{x} = Ax + Bu$, $y = Cx + Du$, its discrete-time description using an ideal sampler and a zero-order hold (ZOH) is

$$\begin{aligned} \bar{x}(k+1) &= \bar{A}\bar{x}(k) + \bar{B}\bar{u}(k), \\ \bar{y}(k) &= \bar{C}\bar{x}(k) + \bar{D}\bar{u}(k), \end{aligned} \quad (\text{A.22})$$

with

$$\begin{aligned} \bar{A} &= e^{AT}, & \bar{B} &= \left[\int_0^T e^{A\tau} d\tau \right] B, \\ \bar{C} &= C, & \bar{D} &= D, \end{aligned} \quad (\text{A.23})$$

where T is the sampling period.

Internal Stability (Lyapunov Stability) and External (BIBO—Bounded-Input, Bounded-Output) Stability

- $\dot{x} = Ax$ is stable if all eigenvalues of A have real parts ≤ 0 . It is asymptotically stable if all eigenvalues of A have real parts < 0 .
- $x(k+1) = Ax(k)$ is stable if all eigenvalues of A have magnitude ≤ 1 . It is asymptotically stable if all eigenvalues of A have magnitude < 1 .
- A continuous-time system is BIBO stable when all poles of its transfer function have negative real parts. A discrete-time system is BIBO stable when all poles of

its transfer function have magnitude < 1 . Since

$$\{\text{eigenvalues of } A\} \supset \{\text{poles of } H(s)\}, \quad (\text{A.24})$$

asymptotic stability always implies BIBO stability, but not necessarily vice versa. Recall that a continuous-time system is asymptotically stable if and only if all eigenvalues of A have negative real parts; the system is BIBO stable if and only if all poles of $H(s)$ have negative real parts. BIBO stability implies asymptotic stability only when the eigenvalues of A are exactly the poles of $H(s)$, which is the case when the system is both controllable and observable. Similar results hold for the discrete-time case.

An extensive discussion of stability and Lyapunov functions for continuous-time systems may be found in [2, 4].

Controllability and Observability

Controllability and observability are important system concepts. Controllability refers to the ability of the inputs to control the state and observability refers to the ability to estimate the state by observing the inputs and outputs.

Reachability and Controllability

- In continuous-time systems, reachability always implies and is implied by controllability. In discrete-time systems, reachability always implies controllability, but controllability implies reachability only when A is non-singular.
- The controllability matrix for both discrete- and continuous-time time-invariant systems is

$$\mathcal{C} = [B, AB, \dots, A^{n-1}B] \in \mathbb{R}^{n \times mn} \quad (\text{A.25})$$

and it has full (row) rank when the system, denoted by (A, B) , is (completely) reachable (controllable-from-the-origin).

- When a discrete-time system $x(k+1) = Ax(k) + Bu(k)$ [denoted by (A, B)] is completely reachable (controllable-from-the-origin), the input sequence $\{u(i)\}$, $i = 0, \dots, K-1$, that transfers the state from any $x_0 (= x(0))$ to any x_1 in some finite time K ($x_1 = x(K)$, $K > 0$) is determined by solving

$$x_1 = A^K x_0 + \sum_{i=0}^{K-1} A^{K-(i+1)} Bu(i) \quad \text{or}, \quad (\text{A.26})$$

$$x_1 - A^K x_0 = [B, AB, \dots, A^{K-1}B] [u^T(K-1), \dots, u^T(0)]^T. \quad (\text{A.27})$$

A solution for this always exists when $K = n$.

- When a continuous-time system $\dot{x} = Ax + Bu$ [denoted by (A, B)] is controllable, an input that transfers any state $x_0 (= x(0))$ to any other state x_1 in some finite time T ($x_1 = x(T)$) is

$$u(t) = B^T e^{A^T(T-t)} W_r^{-1}(0, T) [x_1 - e^{AT} x_0] \quad t \in [0, T], \quad (\text{A.28})$$

where

$$W_r(0, T) = \int_0^T e^{(T-\tau)A} B B^T e^{(T-\tau)A^T} d\tau, \quad (\text{A.29})$$

is the reachability Gramian of the system.

- (A, B) is reachable if and only if

$$\text{rank}[s_i I - A, B] = n, \quad (\text{A.30})$$

for $s_i, i = 1, \dots, n$, all the eigenvalues of A .

Observability and Constructibility

- In continuous-time systems, observability always implies and is implied by constructibility. In discrete-time systems, observability always implies constructibility, but constructibility implies observability only when A is non-singular.
- The observability matrix for both discrete- and continuous-time, time-invariant systems is

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix} \in \mathbb{R}^{pn \times n} \quad (\text{A.31})$$

and it has full (column) rank when the system is completely observable.

- When a discrete-time system $x(k+1) = Ax(k) + Bu(k)$, $y(k) = Cx(k) + Du(k)$ [denoted by (A, C)] is completely observable, any initial state $x(0) = x_0$ can be uniquely determined by observing the input and output over some finite period of time and using the relation

$$\tilde{y}(k) = CA^k x_0 \quad k = 0, 1, \dots, n-1, \quad (\text{A.32})$$

where $\tilde{y}(k) = y(k) - \left[\sum_{i=0}^{k-1} CA^{k-(i+1)} Bu(i) + D(k)u(k) \right]$. To determine x_0 , solve

$$\begin{bmatrix} \tilde{y}(0) \\ \tilde{y}(1) \\ \vdots \\ \tilde{y}(n-1) \end{bmatrix} = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix} x_0. \quad (\text{A.33})$$

- Consider the continuous-time system $\dot{x} = Ax + Bu$, $y = Cx + Du$. When this system is completely observable, any initial state $x_0 = x(0)$ can be uniquely determined by observing the input and output over some finite period of time T and using the relation

$$\tilde{y}(t) = Ce^{At} x_0, \quad (\text{A.34})$$

where $\tilde{y}(t) = y(t) - \left[\int_0^t C e^{A(t-\tau)} B u(\tau) d\tau + D u(t) \right]$. The initial state x_0 may be determined from

$$x_0 = W_o^{-1}(0, T) \left[\int_0^T e^{A^T \tau} C^T \tilde{y}(\tau) d\tau \right], \quad (\text{A.35})$$

where

$$W_o(0, T) = \int_0^T e^{A^T \tau} C^T C e^{A \tau} d\tau, \quad (\text{A.36})$$

is the observability Gramian of the system.

- (A, C) is observable if and only if

$$\text{rank} \begin{bmatrix} s_i I - A \\ C \end{bmatrix} = n, \quad (\text{A.37})$$

for $s_i, i = 1, \dots, n$, all the eigenvalues of A .

Dual Systems

- $(A_D = A^T, B_D = C^T, C_D = B^T, D_D = D^T)$ is the dual of (A, B, C, D) . Reachability is dual to observability. If a system is reachable (observable), its dual is observable (reachable).

Standard Forms for Uncontrollable and Unobservable Systems

- The standard form for uncontrollable systems is

$$\hat{A} = Q^{-1} A Q = \begin{bmatrix} A_1 & A_{12} \\ 0 & A_2 \end{bmatrix}, \quad \hat{B} = Q^{-1} B = \begin{bmatrix} B_1 \\ 0 \end{bmatrix}, \quad (\text{A.38})$$

where $A_1 \in R^{n_r \times n_r}$, $B_1 \in R^{n_r \times m}$, and (A_1, B_1) is controllable. Q is an appropriate similarity transformation matrix. $n_r < n$ is the rank of the controllability matrix $C = [B, AB, \dots, A^{n-1}B]$; i.e.,

$$\text{rank} C = n_r.$$

- The standard form for unobservable systems is

$$\hat{A} = Q^{-1} A Q = \begin{bmatrix} A_1 & 0 \\ A_{21} & A_2 \end{bmatrix}, \quad \hat{C} = C Q = \begin{bmatrix} C_1 \\ 0 \end{bmatrix}, \quad (\text{A.39})$$

where $A_1 \in R^{n_o \times n_o}$, $C_1 \in R^{p \times n_o}$, and (A_1, C_1) is observable. Q is an appropriate similarity transformation matrix. $n_o < n$ is the rank of the observability matrix

$$\mathcal{O} = \begin{bmatrix} \widehat{C} \\ \widehat{C}\widehat{A} \\ \vdots \\ \widehat{C}\widehat{A}^{n-1} \end{bmatrix};$$

i.e.,

$$\text{rank } \mathcal{O} = n_o.$$

- Kalman's Decomposition Theorem.

$$\widehat{A} = Q^{-1}AQ = \begin{bmatrix} A_{11} & 0 & A_{13} & 0 \\ A_{21} & A_{22} & A_{23} & A_{24} \\ 0 & 0 & A_{33} & 0 \\ 0 & 0 & A_{43} & A_{44} \end{bmatrix}, \quad \widehat{B} = Q^{-1}B = \begin{bmatrix} B_1 \\ B_2 \\ 0 \\ 0 \end{bmatrix}, \quad (\text{A.40})$$

$$\widehat{C} = CQ = [C_1, 0, C_3, 0],$$

where (A_{11}, B_1, C_1) is controllable and observable.

- λ_i is an uncontrollable eigenvalue if and only if

$$\widehat{v}_i[\lambda_i I - A, B] = 0, \quad (\text{A.41})$$

where $\widehat{v}_i$ is the corresponding (left) eigenvector.

- λ_i is an unobservable eigenvalue if and only if

$$\begin{bmatrix} \lambda_i I - A \\ C \end{bmatrix} v_i = 0, \quad (\text{A.42})$$

where v_i is the corresponding (right) eigenvector.

Realizations

- $\dot{x} = Ax + Bu, y = Cx + Du$ is a realization of $H(s)$ ($\hat{y} = H(s)\hat{u}$) if $\hat{y} = [C(sI - A)^{-1}B + D]\hat{u}$.
- Realizations of $H(s)$ exist if and only if $H(s)$ is a proper rational matrix. $\lim_{s \rightarrow \infty} H(s) = D < \infty$.
- The Markov parameters H_i of the system in

$$H(s) = H_0 + H_1 s^{-1} + H_2 s^{-2} + \dots, \quad (\text{A.43})$$

satisfy

$$H_0 = D \text{ and } H_i = CA^{i-1}B, \quad i = 1, 2, \dots \quad (\text{A.44})$$

- A realization $\{A, B, C, D\}$ of $H(s)$ is minimal if and only if it is both controllable and observable.
- Two realizations of $H(s)$ that are minimal must be equivalent representations.

- The order of a minimal realization of $H(s)$ equals its McMillan degree, the order of its characteristic or pole polynomial $p_H(s)$. The order of a minimal realization of $H(s)$ is also given by the rank of the Hankel matrix $M_H(r, r)$.
- Duality can be very useful in obtaining realizations.

Linear State Feedback and Optimal Control

Linear State Feedback

- Given $\dot{x} = Ax + Bu$, $y = Cx + Du$, and the linear state-feedback control law $u = Fx + r$, the closed-loop system is

$$\dot{x} = (A + BF)x + Br, \quad y = (C + DF)x + Dr. \quad (\text{A.45})$$

- If u was implemented via open-loop control, it would be given by

$$\hat{u} = F[sI - (A + BF)]^{-1}x(0) + [I - F(sI - A)^{-1}B]^{-1}\hat{r}. \quad (\text{A.46})$$

- The eigenvalues of $A + BF$ can be assigned to arbitrary real and/or complex conjugate locations by selecting F if and only if the system is controllable. The uncontrollable eigenvalues of (A, B) cannot be shifted using feedback control.

Optimal Control: Linear Quadratic Regulator

- Given $\dot{x} = Ax + Bu$, $z = Mx$, find $u(t)$ that minimizes the quadratic cost

$$J(u) = \int_0^\infty [z^T(t)Qz(t) + u^T(t)Ru(t)]dt. \quad (\text{A.47})$$

Under controllability and observability conditions, the solution is unique and it is given as a linear state-feedback control law

$$u^*(t) = F^*x(t) = -R^{-1}B^T P_c^*x(t), \quad (\text{A.48})$$

where P_c^* is the symmetric, positive definite solution of the algebraic Riccati equation

$$A^T P_c + P_c A - P_c B R^{-1} B^T P_c + M^T Q M = 0. \quad (\text{A.49})$$

Analogous results exist for the discrete-time case.

Linear State Observers and Optimal State Estimation

Linear State Observers

- Given $\dot{x} = Ax + Bu$, $y = Cx + Du$, the Luenberger observer is

$$\dot{\hat{x}} = A\hat{x} + Bu + K(y - \hat{y}), \quad (\text{A.50})$$

where $\hat{y} = C\hat{x} + Du$ or

$$\dot{\hat{x}} = (A - KC)\hat{x} + [B - KD, K] \begin{bmatrix} u \\ y \end{bmatrix}, \quad (\text{A.51})$$

where K is chosen so that all the eigenvalues of $A - KC$ have negative real parts. Then the estimation error $e(t) = x(t) - \hat{x}(t)$ will go to zero asymptotically.

- The eigenvalues of $A - KC$ can be assigned to arbitrary real and/or complex conjugate locations by selecting K if and only if the system [or (A, C)] is observable. The unobservable eigenvalues cannot be shifted using K . This is the dual problem to the control problem of assigning eigenvalues in $A + BF$, and the same methods can be used.

Optimal State Estimation

- Consider $\dot{x} = Ax + Bu + \Gamma w$, $y = Cx + v$, where w and v are process and measurement noise. Let the state estimator be

$$\dot{\hat{x}} = A\hat{x} + Bu + K(y - C\hat{x}), \quad (\text{A.52})$$

$$= (A - KC)\hat{x} + Bu + Ky, \quad (\text{A.53})$$

and consider minimizing the error covariance $E[(x - \hat{x})(x - \hat{x})^T]$. Under certain controllability and observability conditions, the solution is unique and it is given by

$$K^* = P_e^* C^T V^{-1}, \quad (\text{A.54})$$

where P_e^* is the symmetric, positive definite solution of the quadratic (dual) algebraic Riccati equation

$$P_e A^T + A P_e - P_e C^T V^{-1} C P_e + \Gamma W \Gamma^T = 0. \quad (\text{A.55})$$

This problem is dual to the Linear Quadratic Regulator problem.

- The discrete-time case is analogous to the continuous-time case.

Observer-Based Dynamic Controllers

- Given $\dot{x} = Ax + Bu$, $y = Cx + Du$ with the state feedback $u = Fx + r$, if the state is estimated via a Luenberger observer, then the closed-loop system is

$$\begin{bmatrix} \dot{\hat{x}} \\ \dot{e} \end{bmatrix} = \begin{bmatrix} A + BF & -BF \\ 0 & A - KC \end{bmatrix} \begin{bmatrix} x \\ e \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} r, \quad (\text{A.56})$$

$$y = [C + DF, -DF] \begin{bmatrix} x \\ e \end{bmatrix} + Dr. \quad (\text{A.57})$$

The error $e = x - \hat{x}$.

- The design of the control law (F) can be carried out independently of the design of the estimator (K) (Separation property).
- The compensated system behaves to the outside world as if there were no estimator present—after sufficient time has elapsed so the transients have become negligible.

A.3 Modeling Sampled-Data Systems

Sampled-data systems are very common in control. This is because continuous physical systems are typically controlled by digital (cyber) devices, which implement the control algorithms. That is, sampled-data systems are used to model and design digital control systems. Sampled-data systems contain both continuous-time and discrete-time signals, but they are not typically studied as hybrid dynamical systems because all signals are taken to be continuous valued for simplicity, thus omitting the inevitable quantization introduced by the digital devices due to their finite word length—see Modeling of Signals and Systems in Sect. A.1. The effect of quantization is typically introduced later and is modeled as noise. Such an approach is often justified by the long finite word length typically used in modern digital devices. That is, digital control systems which are hybrid dynamical systems are typically modeled as sampled-data systems to simplify their design.

Sampled-Data Systems—From Continuous-Time to Discrete-Time Dynamical Systems

In this section, sampled-data systems are discussed, primarily using state variable descriptions (with differential/difference equations in state variable form). They are also studied using the Laplace transforms (continuous-time) and Z-Transforms (discrete-time); see [6, 7]. A continuous system S is given (see Fig. A.1) and we are interested in controlling it using a digital device.

For this, we will need to design an appropriate interface since the instant values of the digital device output at the tick of its clock will not be recognized by the continuous system. The interface consists of a digital-to-analog (D/A) converter at the input of the plant and an analog-to-digital (A/D) converter at its output. These interface devices are discussed in more detail below. From the input–output point of view, the system can now be seen as accepting an input sequence $\{\bar{u}(k)\}$ and generating an output sequence $\{\bar{y}(k)\}$ so it can be seen as a discrete-time system.

State Variable Representation of Sampled-Data Systems

Suppose that the trajectory of the continuous-time system

$$\dot{x}(t) = Ax(t) + Bu(t), \quad y(t) = Cx(t) + Du(t); \quad x(t_0) = x_0, \quad (\text{A.58})$$

is to be approximated by the trajectory of the discrete-time system

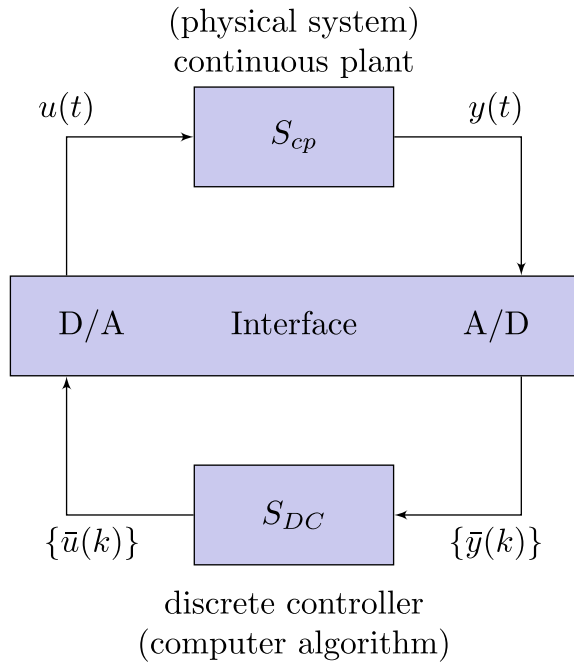
$$\bar{x}(k+1) = \bar{A}\bar{x}(k) + \bar{B}\bar{u}(k), \quad (\text{A.59})$$

$$\bar{y}(k) = \bar{C}\bar{x}(k) + \bar{D}\bar{u}(k), \quad (\text{A.60})$$

matching exactly the values at times t_k , that is, $x(t_k) = \bar{x}(k)$, $y(t_k) = \bar{y}(k)$, when $u(t_k) = \bar{u}(k)$.

We are particularly interested in the sampled-data system in Fig. A.1 where the continuous-time system S is described by the state variable equations above, the A/D

Fig. A.1 A sampled-data control system



converter is an ideal sampler, and the D/A converter is a Zero-Order-Hold ZOH. We would like to describe the discrete-time system $\bar{S}$ by difference equations. Next, we first discuss the A/D and D/A converters, and then we discuss discrete-time models of continuous-time systems.

Interface—(A/D) and (D/A) Converters

An (ideal) *A/D (Analog-to-Digital) converter*, also called the *ideal sampler*, is a device that has as input a continuous-time signal, $y(t)$, and as output a sequence of real numbers, $\{\bar{y}(k)\}$, $k = k_0, k_1, \dots$, determined by the relation

$$\bar{y}(k) = y(t_k). \quad (\text{A.61})$$

In other words, the (ideal) *A/D converter* is a device that samples an input signal $y(t)$ at times $t_0, t_1, \dots$ producing the corresponding sequence $\{y(t_0), y(t_1), \dots\}$ (Fig. A.2).

Such *sampling procedure* of obtaining a sequence of discrete values as an approximate representation of a continuous signal is an example of a *discretization procedure*.

A sampling device cannot extract the value $y(t_k)$, which is a real number, with absolute accuracy, but it generates an approximation of the value. The accuracy of the approximation depends on the number of bits used to represent the value, that is, the number of *quantization levels* used. The resulting signal is a *digital signal*. That is, a digital signal is a discrete-time signal which is also quantized.

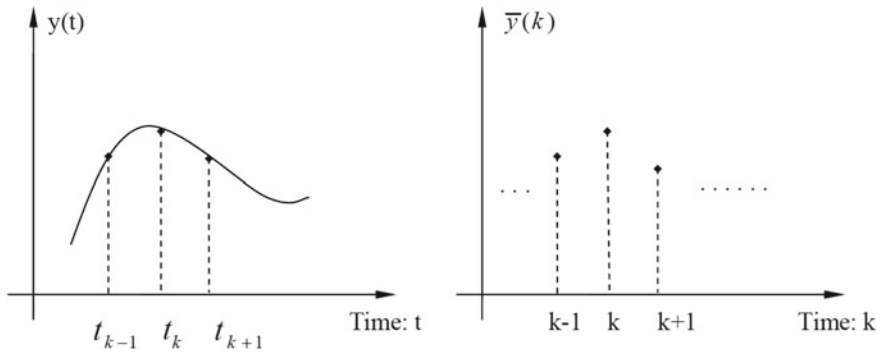


Fig. A.2 Sampling

For example, if $\{y(t_k)\} = \{1.344, 2.123483, 3.418, \dots\}$, the quantized signal could be $\{1.34, 2.12, 3.42, \dots\}$, when a two decimal accuracy is imposed by the finite number of bits representation in the digital device. In sampled-data system theory, quantization is typically treated as an external disturbance or noise in a stochastic framework. This is justified by the fact that today's computers typically have long enough word length so that the quantization error is rather small and may be treated as noise.

The *sampling interval* $t_{k+1} - t_k$ is typically constant and is denoted here by $T (= t_{k+1} - t_k)$ the *sampling period*. There are cases, however, where the sampling interval may not be constant. It could be, for example, that when the signal $y(t)$ changes rapidly, one may desire more samples to be taken as opposed to the case when $y(t)$ remains unchanged for a long period of time in which case few samples are adequate to represent $y(t)$. So the sampling times may depend on the value of y ; this is, for example, the case in some numerical algorithms where the number of points taken depends on the rate of change of the function, that is, how fast the value of the function is changing. In general, deciding how fast to sample a signal is a nontrivial question. It certainly depends on the type of signal (fast or slow changing) and also what one intends to do with the samples. For example, in control systems, if stabilization is the goal, then one may not need as many samples compared to a tracking goal.

A *D/A (Digital-to-Analog) converter* is a device that has as input a discrete-time signal, $\{\bar{u}(k)\}$, $k = k_0, k_0 + 1, \dots$, and as output a continuous-time signal, $u(t)$. Here, the output is determined by the relation

$$u(t) = \bar{u}(k), \quad t_k \leq t < t_{k+1}, \quad k = k_0, k_0 + 1, \dots, \quad (\text{A.62})$$

that is, the output is kept constant during the time interval, at a value equal to the last value of the sequence. Such device is called a *ZOH* (Fig. A.3).

The ZOH device described above implements a particular procedure of *data reconstruction or extrapolation*. The general problem is as follows:

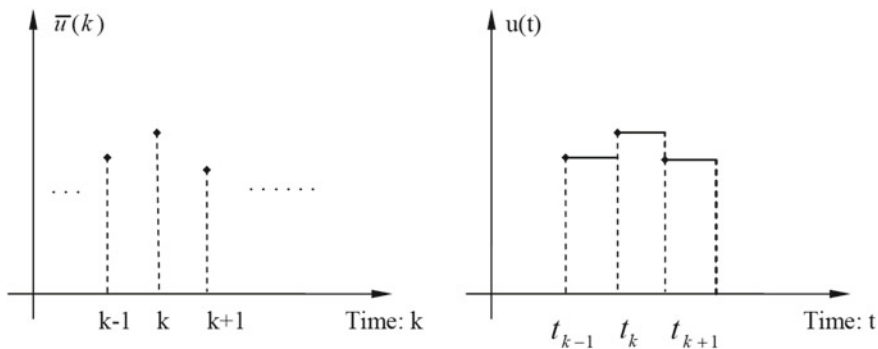


Fig. A.3 Zero-Order Hold

Given a sequence of real numbers $\{\bar{f}(k)\}$, $k = k_0, k_0 + 1, \dots$, derive $f(t)$, $t \geq t_0$, so that for $t = t_k$

$$f(t_k) = \bar{f}(k), \quad k = k_0, k_0 + 1, \dots \quad (\text{A.63})$$

Clearly, there is a lot of flexibility in assigning values to $f(t)$ in between the samples $\bar{f}(k)$; in other words, there is a lot of flexibility in assigning the *inter-sample behavior* in $f(t)$.

A way to approach the problem is to start by writing a power series expansion of $f(t)$ for t , $t_k \leq t < t_{k+1}$, namely

$$f(t) = f(t_k) + f^{(1)}(t_k)(t - t_k) + \frac{f^{(2)}(t_k)}{2!}(t - t_k)^2 + \dots, \quad (\text{A.64})$$

where $f^{(n)}(t_k) = \frac{d^{(n)}f(t)}{dt^n}|_{t=t_k}$, that is, the n th derivative of $f(t)$ evaluated at $t = t_k$ (assuming that the derivatives exist).

Now if the function $f(t)$ is approximated in the interval $t_k \leq t < t_{k+1}$ by the constant value $f(t_k)$ taken to be equal to $\bar{f}(k)$, then

$$f(t) = f(t_k) \quad (= \bar{f}(k)), \quad t_k \leq t < t_{k+1}, \quad (\text{A.65})$$

which is exactly the relation implemented by a ZOH. Note that here the zero-order derivative of the power series is used which leads to an approximation by a constant which is a zero-degree polynomial.

It is clear that more than the first term in the power series can be taken to approximate $f(t)$. If, for example, the first two terms are taken, then

$$f(t) = f(t_k) + f^{(1)}(t_k)(t - t_k) \quad (\text{A.66})$$

$$= f(t_k) + \frac{f(t_k) - f(t_{k-1})}{t_k - t_{k-1}}(t - t_k) \quad (\text{A.67})$$

$$= \bar{f}(k) + \frac{\bar{f}(k) - \bar{f}(k-1)}{t_k - t_{k-1}}(t - t_k) \quad (\text{A.68})$$

for $t_k \leq t < t_{k+1}$, where an approximation for the derivative $f^{(1)}(t)$ was used. The approximation between t_k and t_{k+1} is a ramp with slope determined by $f(t_k) = \bar{f}(k)$ and the previous value $f(t_{k-1}) = \bar{f}(k-1)$. Here, the first-order derivative of the power series is used which leads to an approximation by a first-degree polynomial. A device that implements such approximation is called a *first-order hold (FOH)*. Similarly, one can talk about *second-order hold*, etc. Note that the formula of the above FOH is derived if we decide to use a first-degree polynomial $at + b$ to approximate $f(t)$ on $t_k \leq t < t_{k+1}$ and then enforce $f(t_k) = \bar{f}(k)$ and $f(t_{k-1}) = \bar{f}(k-1)$.

Obtaining a continuous (or piecewise continuous) function from given discrete values may be seen as a *continualization procedure*. Contrast this with the *discretization procedure* introduced by sampling earlier in this section.

Discrete-Time Models of Continuous-Time Systems

Consider the solution of the state equations given by the variation of constants formula

$$x(t) = e^{A(t-t_0)}x(t_0) + \int_{t_0}^t e^{A(t-\tau)}Bu(\tau)d\tau, \quad (\text{A.69})$$

and let $t = t_{k+1}$, $t_0 = t_k$. Then

$$x(t_{k+1}) = e^{A(t_{k+1}-t_k)}x(t_k) + \int_{t_k}^{t_{k+1}} e^{A(t_{k+1}-\tau)}Bu(\tau)d\tau.$$

Since $u(t) = u(t_k)$ is a constant in the interval $t_k \leq t < t_{k+1}$ because of the ZOH device,

$$x(t_{k+1}) = e^{A(t_{k+1}-t_k)}x(t_k) + \left[\int_{t_k}^{t_{k+1}} e^{A(t_{k+1}-\tau)}d\tau \right] Bu(t_k),$$

or

$$\bar{x}(k+1) = \bar{A}(k)\bar{x}(k) + \bar{B}(k)\bar{u}(k), \quad (\text{A.70})$$

where $\bar{x}(k) = x(t_k)$, $\bar{u}(k) = u(t_k)$, and

$$\bar{A}(k) = e^{A(t_{k+1}-t_k)}, \quad (\text{A.71})$$

$$\bar{B}(k) = \left[\int_{t_k}^{t_{k+1}} e^{A(t_{k+1}-\tau)}d\tau \right] B. \quad (\text{A.72})$$

Consider also the output equation $y(t) = Cx(t) + Du(t)$, which in view of the variation of constants formula may be written as

$$y(t) = Ce^{A(t-t_0)}x(t_0) + \int_{t_0}^t Ce^{A(t-\tau)}Bu(\tau)d\tau + Du(t). \quad (\text{A.73})$$

Assume now that the output of the plant is sampled at instants t'_k that do not necessarily coincide with instants t_k at which the input to the plant is adjusted. Let $t_k \leq t'_k < t_{k+1}$. Applying the above for $t = t'_k$, $t_0 = t_k$ and using the fact that $u(t)$ is adjusted via ZOH as before, we obtain

$$y(t'_k) = Ce^{A(t'_k-t_k)}x(t_k) + [\int_{t_k}^{t'_k} Ce^{A(t'_k-\tau)}d\tau]Bu(t_k) + Du(t'_k),$$

or

$$\bar{y}(k) = \bar{C}(k)\bar{x}(k) + \bar{D}(k)\bar{u}(k), \quad (\text{A.74})$$

where $\bar{x}(k) = x(t_k)$, $\bar{y}(k) = y(t'_k)$, $\bar{u}(k) = u(t_k) = u(t'_k)$, and

$$\bar{C}(k) = Ce^{A(t'_k-t_k)}. \quad (\text{A.75})$$

$$\bar{D}(k) = C[\int_{t_k}^{t'_k} e^{A(t_{k+1}-\tau)}d\tau]B + D. \quad (\text{A.76})$$

In general, the above discrete-time system representation is time varying. If $t_{k+1} - t_k = T$, a constant, then

$$\bar{A}(k) = \bar{A} = e^{AT}. \quad (\text{A.77})$$

$$\bar{B}(k) = \bar{B} = [\int_{t_k}^{t_{k+1}} e^{A(t_{k+1}-\tau)}d\tau]B \quad (\text{A.78})$$

$$= [-\int_T^0 e^{A\tau'}d\tau']B = [\int_0^T e^{A\tau}d\tau]B \quad (\text{A.79})$$

where the change of variables $\tau' = t_{k+1} - \tau$ was used in the integral.

If in addition, $t'_k - t_k = a$ a constant, that is, the input and output are sampled at instants that differ by a constant, then

$$\bar{C}(k) = \bar{C} = Ce^{Aa}, \quad (\text{A.80})$$

$$\bar{D}(k) = \bar{D} = C[\int_0^a e^{A\tau}d\tau]B + D. \quad (\text{A.81})$$

Note that under these conditions the representation of the discrete-time system is time invariant.

Typically $t'_k - t_k = a$ is taken to be zero, that is, the output is sampled exactly at the same time instants the input is adjusted. For $a = 0$, then the commonly used expressions for the discrete-time system are derived, namely

$$\bar{x}(k+1) = \bar{A}\bar{x}(k) + \bar{B}\bar{u}(k), \quad (\text{A.82})$$

$$\bar{y}(k) = \bar{C}\bar{x}(k) + \bar{D}\bar{u}(k), \quad (\text{A.83})$$

where $x(k_0) = x_0$, $\bar{x}(k) = x(t_k)$, $\bar{u}(k) = u(t_k)$, $\bar{y}(k) = y(t_k)$, and

$$\bar{A} = e^{AT}, \quad \bar{B} = \left[\int_0^T e^{A\tau} d\tau \right] B, \quad (\text{A.84})$$

$$\bar{C} = C, \quad \bar{D} = D. \quad (\text{A.85})$$

Relations Between the Eigenvalues of the Continuous-Time and Discrete-Time Systems

If $\lambda_i, i = 1, \dots, n$, are the eigenvalues of A , then the eigenvalues of $\bar{A} = e^{AT}$ are $e^{\lambda_i T}$. This result can be derived from the following property.

If a $n \times n$ matrix A has eigenvalues $\lambda_i, i = 1, \dots, n$, then $f(A)$ —a polynomial or a power series in A —has eigenvalues $f(\lambda_i), i = 1, \dots, n$.

So the mapping

$$z = e^{sT}, \quad (\text{A.86})$$

for $s_i = \lambda_i, i = 1, \dots, n$, maps the n λ_i eigenvalues of A to the n z_i eigenvalues of e^{AT} , namely $z_i = e^{\lambda_i T}$. The mapping $z = e^{sT}$ also describes the relation between the poles of the Laplace transform of a continuous signal and the poles of the Z-transform of its sampled version. In particular, given a signal $f(t)$ with the Laplace transform $F(s) (= \frac{n(s)}{d(s)}$, a rational function), let $\{f(kT)\}$ be a sequence of samples (of $f(t)$) and $\bar{F}(z)$ its Z-transform. Then if s_i are the poles of $F(s)$, the poles of $\bar{F}(z)$ are given by

$$z_i = e^{s_i T}. \quad (\text{A.87})$$

This can be shown by using partial fraction expansion in $F(s)$ to determine the corresponding sum of time functions and noticing (from the Z-transform table) that the poles of the Z-transforms of the sampled-data functions are given by $e^{s_i T}$ where s_i are the poles of the terms in the partial fraction expansion.

It is important to note that $z = e^{sT}$ is a many-to-one mapping since many values of s generate the same value of z .

In fact if

$$s_2 = s_1 + j \frac{2\pi}{T} N, \quad N = 0, \pm 1, \pm 2, \dots, \quad (\text{A.88})$$

then

$$\begin{aligned}
e^{s_2 T} &= e^{(s_1 + j \frac{2\pi}{T} N) T} = e^{s_1 T} e^{j 2\pi N} \\
&= e^{s_1 T} (\cos 2\pi N + j \sin 2\pi N) \\
&= e^{s_1 T} (1 + 0) = e^{s_1 T}.
\end{aligned} \tag{A.89}$$

That is all s_2 map to the same point z as s_1 .

Note that the *inter-sample behavior* may be studied using the continuous model

$$x(t) = e^{A(t-t_k)} x(t_k) + \int_{t_k}^t e^{A(t_k-\tau)} d\tau B u(t_k). \tag{A.90}$$

A.4 Notes and Further Reading

Modeling of continuous- and discrete-valued signals and systems was discussed. Some key results in the theory of linear dynamical systems were summarized. References for further reading include [1–4, 6–8]. Note that existence of solutions of differential equations was discussed at the end of Chap. 2 on modeling; the Lyapunov stability and optimal control were also discussed in Chaps. 4 and 5.

Sampled-data systems and digital control are topics that have been studied for the past half-century and results have been reported in many books and papers. Here, a brief introduction to the topic was given emphasizing state variable descriptions. The A/D and D/A converters, taken to be ideal samplers and ZOH devices, respectively, constitute the interface, that makes it possible for the continuous systems to interact with the digital devices. A similar setup where the digital device is represented by an automaton or Petri net, instead of difference equations as in sampled-data systems, may represent hybrid dynamical systems; the interface in this case includes more general continuous-to-discrete and discrete-to-continuous converters. This point of view for the control of hybrid systems was taken in several publications; see, for example, [1] and also Chap. 6 in [9].

Appendix B

Languages and Automata

In this appendix, we provide some background material for regular languages, finite automata, Büchi automata, and generalized Büchi automata and a procedure to convert LTL formulas into generalized Büchi automata.

B.1 Regular Languages and Finite Automata

In this section, we provide the proof of the claim that languages accepted by finite automata are regular languages.

First, let's recall that the class of regular languages, denoted by $L_R \subseteq 2^{\Sigma^*}$, is defined by

- $\emptyset, \{\epsilon\} \in L_R$;
- $\forall \sigma \in \Sigma, \{\sigma\} \in L_R$;
- If $K_1, K_2 \in L_R$, then $K_1 + K_2, K_1 \oplus K_2, K_1^* \in L_R$.

For convenience, we usually denote a regular language by regular expressions, which are defined by

- $\emptyset, \epsilon, \sigma$ are regular expressions for any $\sigma \in \Sigma$;
- if r_1 and r_2 are regular expressions, so are $r_1 + r_2, r_1 r_2$, and r_1^* .

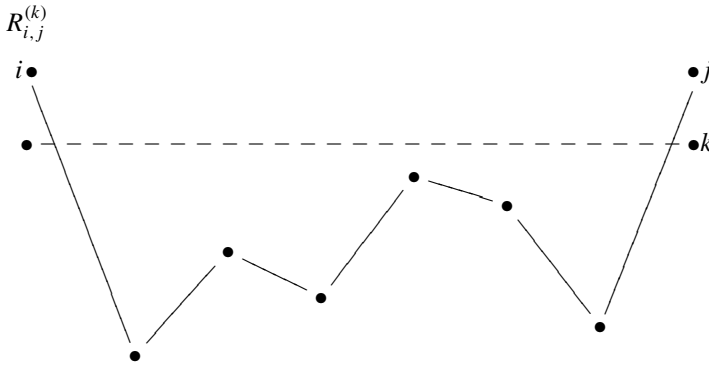
The regular language denoted by a regular expression is defined by

1. $\mathcal{L}(\emptyset) = \emptyset, \mathcal{L}(\epsilon) = \{\epsilon\}$, and $\mathcal{L}(\sigma) = \{\sigma\}$ for any $\sigma \in \Sigma$;
2. $\mathcal{L}(r_1 + r_2) = \mathcal{L}(r_1) \cup \mathcal{L}(r_2)$;
3. $\mathcal{L}(r_1 r_2) = \mathcal{L}(r_1) \mathcal{L}(r_2)$;
4. $\mathcal{L}(r^*) = \mathcal{L}^*(r)$.

The following two theorems tell us that the regular language exactly characterizes and can be captured by the marked behavior of a deterministic finite state machine.

Theorem B.1 Given a deterministic finite automaton (DFA), A , there exists a regular expression r such that $\mathcal{L}(r) = \mathcal{L}(A)$.

Proof Without loss of generality, assume we have a DFA with $Q = \{1, 2, \dots, n\}$. Let's define $R_{i,j}^{(k)}$ as the regular expression whose language is the set of strings that each one of them drives the states from i to j via a path with no intermediate state (not including i, j) whose number is greater than k as illustrates below.

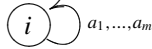


To construct the expression $R_{i,j}^{(k)}$, we use an inductive way from $k = 0$.

$i = j :$

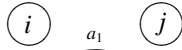


$$R_{i,i}^{(0)} = \epsilon$$

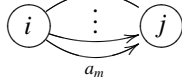


$$R_{i,i}^{(0)} = a_1 + a_2 + \dots + a_m$$

$i \neq j :$

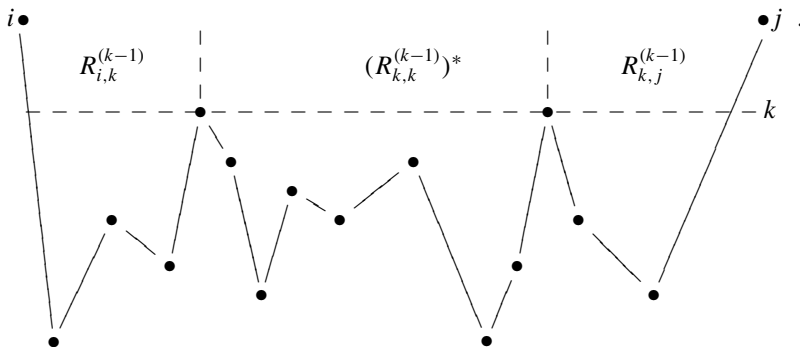


$$R_{i,j}^{(0)} = \emptyset$$



$$R_{i,j}^{(0)} = a_1 + a_2 + \dots + a_m$$

It is clear that $R_{i,j}^{(0)}$ is a regular expression. Suppose $R_{i,j}^{(k-1)}$ is a regular expression. For $R_{i,j}^{(k)}$, there are two possibilities to consider. One is that the intermediate states are all smaller than k . In this case, the expression is already described as $R_{i,j}^{(k-1)}$. Another case is that the path can be divided into several pieces. The first piece denotes the state starts from i to k without passing through k . The last piece denotes the state starts from k to j without passing through k . For the pieces in the middle, each one of them denotes the state starts from k and returns to k without passing through k .



Combining the two cases that are discussed above, we can write

$$R_{i,j}^{(k)} = R_{i,j}^{(k-1)} + R_{i,k}^{(k-1)}(R_{k,k}^{(k-1)})^* R_{k,j}^{(k-1)}.$$

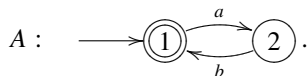
Again, from the construction of $R_{i,j}^{(k)}$, we see that it is also a regular expression. Since $R_{i,j}^{(k)}$ only depends on the expression with smaller superscript, we can always construct $R_{i,j}^{(k)}$ all the way to $R_{i,j}^{(n)}$. If we assume that state 1 is the initial state and any state j could be the accepting state, then we can write

$$\mathcal{L}(A) = \mathcal{L}\left(\bigcup_j R_{1j}^{(n)}\right), j \in F.$$

And $\gamma = \bigcup_i R_i^{(n)}$ is still regular which completes the proof.

The following simple example aims to illustrate the notation, especially $R_{i,j}^{(k)}$, used in the proof.

Example B.1 Consider a finite automaton



$A = (Q, Q_0, \Sigma, \delta, F)$ where $Q = \{1, 2\}$, $Q_0 = F = \{1\}$, $\Sigma = \{a, b\}$, and $n = 2$. We would like to find the corresponding γ . According to Theorem 2, it goes as follows:

- for $k = 0$, by inspection, we find that $R_{1,1}^{(0)} = \epsilon$, $R_{1,2}^{(0)} = a$, $R_{2,1}^{(0)} = b$, and $R_{2,2}^{(0)} = \epsilon$;
- for $k = 1$, according to the equation given in the proof of Theorem 2, we get $R_{1,1}^{(1)} = R_{1,1}^{(0)} + R_{1,1}^{(0)}(R_{1,1}^{(0)})^*R_{1,1}^{(0)} = \epsilon$,

$$R_{1,2}^{(1)} = R_{1,2}^{(0)} + R_{1,1}^{(0)}(R_{1,1}^{(0)})^* R_{1,2}^{(0)} = a + \epsilon(\epsilon)^* a = a,$$

$$R_{2,1}^{(1)} = R_{2,1}^{(0)} + R_{2,1}^{(0)}(R_{1,1}^{(0)})^*R_{1,1}^{(0)} = b + b(\epsilon)^*\epsilon = b,$$

$$R_{2,2}^{(1)} = R_{2,2}^{(0)} + R_{2,1}^{(0)}(R_{1,1}^{(0)})^*R_{1,2}^{(0)} = \epsilon + b(\epsilon)^*a = \epsilon + ab;$$

- for $k = 2$, since what we are interested in is only $R_{1,1}^{(2)}$, we find that

$$\gamma = R_{1,1}^{(2)} = R_{1,1}^{(1)} + R_{1,2}^{(1)}(R_{2,2}^{(1)})^*R_{2,1}^{(1)} = \epsilon + a(\epsilon + ab)^*b = \epsilon + a(ab)^*b = (ab)^*.$$

□

The reverse of Theorem B.1 is also true.

Theorem B.2 *Given a regular expression γ , there exists a DFA, A , such that $\mathcal{L}(A) = \mathcal{L}(\gamma)$.*

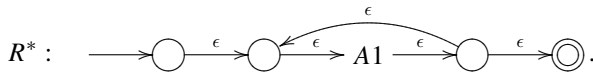
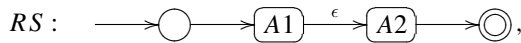
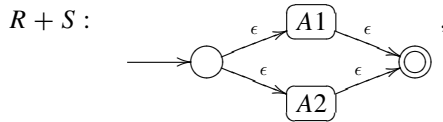
Proof We prove this theorem by structural induction on the expression γ . We start with constructing DFA based on basic expressions as follows.

$$\gamma = \epsilon : \quad \longrightarrow \bigcirc \xrightarrow{\epsilon} \bigcirc \quad .$$

$$\gamma = \emptyset : \quad \longrightarrow \bigcirc \quad \bigcirc \quad .$$

$$\gamma = a \in \Sigma : \quad \longrightarrow \bigcirc \xrightarrow{a} \bigcirc \quad .$$

It is not hard to observe that any expression γ consist of union, concatenation, or closure of smaller expressions. Thus, we show the resulting automaton for these three cases. Suppose we have two automaton A_1, A_2 with $\mathcal{L}(A_1) = \mathcal{L}(R)$, $\mathcal{L}(A_2) = \mathcal{L}(S)$, then



- The first automaton serves $R + S$. Because starting from a new initial state, we can reach either accepting state of automaton for R or for S with strings from $\mathcal{L}(R)$ or $\mathcal{L}(S)$. And then we can follow one of ϵ arc to reach the new accepting states. Therefore, the language of the first automaton equals $\mathcal{L}(R) \cup \mathcal{L}(S)$.
- The second automaton serves RS . The initial state of the new automaton is that of the first automaton and the accepting state of the new automaton is that of the second automaton. The first part of any word from the new automation must belong to the first automaton for R so it can go through it. And the second part must belong to the automaton for S . As a result, we know that the language for this new automaton is $\mathcal{L}(RS)$.

- The third automaton serves R^* . It allows us to go to the accepting state either directly via ϵ or recursively go through the automaton for R and then finally to the accepting state. The word in both cases is accepted by $\mathcal{L}(R^*)$.

In this way, we can construct the automaton inductively and finally get the automaton whose language is $\mathcal{L}(\gamma)$. $\square$

Interested readers may find more detailed treatments of finite automata and regular languages in some classical textbooks, such as [10, 11].

B.2 Büchi Automata

Automata on infinite words are useful for specifying the behavior of nonterminating systems, such as hardware or operating systems. A Büchi automaton is the extension of a finite automaton to accept an infinite input sequence.

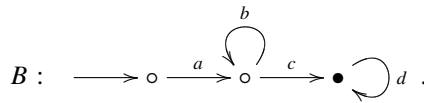
Definition B.1 A Büchi automaton B is an automaton $(Q, Q^0, \Sigma, \delta, F)$, where $F \subseteq Q$ is a set of final states. A string $\alpha \in Q^\omega$ is a run of B if there exists $\beta \in \Sigma^\omega$ such that

1. $\alpha(1) \in Q^0$;
2. $\alpha(i+1) \in \delta(\alpha(i), \beta(i))$, for all $i \in \mathbb{N}$;
3. there exists infinitely many $j \in \mathbb{N}$ such that $\alpha(j) \in F$.

The language recognized or accepted by B is the collection of all such β , called ω -language and denoted by $\mathcal{L}_\omega(B)$.

Intuitively, a run of a Büchi automaton (in case of a deterministic automaton, there is exactly one possible run) consists of infinitely many states in the set of final states.

For example,



The ω -language accepted by the above automaton B is $\mathcal{L}_\omega(B) = ab^*cd^\omega$.

Definition B.2 An ω -regular expression R , over the set Σ , has the following form:

$$R = r_1 S_1^\omega + \dots + r_n S_n^\omega,$$

where $n \geq 1$ and r_i and s_i are regular expression over Σ . Then, $L_\omega(R) = L(r_1)L(s_1)^\omega \cup \dots \cup L(r_n)L(s_n)^\omega$, where the term $(.)^\omega$ is defined as $K^\omega = \{t_1 t_2 t_3 \dots | t_i \in K, i \geq 1\}$, for $K \subseteq \Sigma^+ = \Sigma^* - \{\epsilon\}$. Here, K consists of infinite repetition of non-zero length strings.

Definition B.3 A language $L \subseteq \Sigma^\omega$ is called ω -regular if $L = L_\omega(R)$ for some ω -regular expressions over Σ .

Claim B.1 *The class of languages accepted by a Büchi automaton agrees with the class of ω -regular languages.*

Properties of a Büchi Automaton

Büchi automata are often used in model checking as an automata-theoretic version of formula in linear temporal logic. It can be shown that Büchi automata are closed under the Boolean operations.

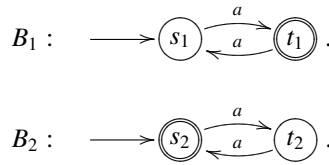
Proposition B.1 *Let B_1, B_2 be two Büchi automata. Then there exists a Büchi automaton B such that $\mathcal{L}_\omega(B) = \mathcal{L}_\omega(B_1) \cup \mathcal{L}_\omega(B_2)$.*

Similar to finite automata, B can be given as the union automata of B_1 and B_2 , i.e., $B_1 \cup B_2$.

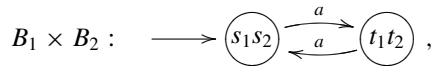
Proposition B.2 *Let B_1, B_2 be two Büchi automata. Then there exists a Büchi automaton B such that $\mathcal{L}_\omega(B) = \mathcal{L}_\omega(B_1) \cap \mathcal{L}_\omega(B_2)$.*

Note that accepting state from both automata maps appear together only finitely many times even if they appear individually infinitely often. Hence, the product operator $\times$ and simply setting $F = F_1 \times F_2$ as we did for finite language automata do not work for the intersection of Büchi automata.

Example B.2 Consider automata B_1 and B_2 as follows:



If we consider these two automata as finite automaton, then $\mathcal{L}(B_1) = \{a^{2i+1}, i \geq 0\}$ and $\mathcal{L}(B_2) = \{a^{2i}, i \geq 0\}$. Therefore, $\mathcal{L}(B_1) \cap \mathcal{L}(B_2) = \emptyset$. Also, for



we have $\mathcal{L}(B_1 \times B_2) = \emptyset$ as there are no states marked in the product automaton. Hence, $\mathcal{L}(B_1) \cap \mathcal{L}(B_2) = \mathcal{L}(B_1 \times B_2) = \emptyset$.

Now, consider B_1 and B_2 as two Büchi automata. It can be verified that $\mathcal{L}_\omega(B_1) = \mathcal{L}_\omega(B_2) = a^\omega$. Therefore, $\mathcal{L}_\omega(B_1) \cap \mathcal{L}_\omega(B_2) = a^\omega$. But, $\mathcal{L}_\omega(B_1 \times B_2) = \emptyset$. $\square$

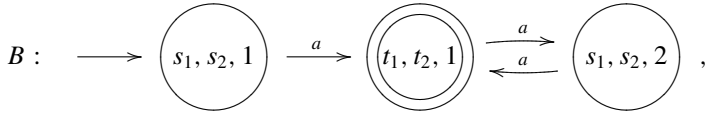
As illustrated by the above example, the intersection of two Büchi automata is not simply by taking the product as we did for finite automata. To understand why the product operation does not work for Büchi automata, we take a close look at an accepted ω -word of the product automaton $B_1 \times B_2$. To be accepted by the product automaton $B_1 \times B_2$, the ω -word needs to visit the states in $B_1 \times B_2$ visiting the

marked states $F = F_1 \times F_2$ infinitely often. In other words, the corresponding run needs to visit F_1 and F_2 simultaneously infinitely often. However, this is too strong as a run that is accepted by B_1 and B_2 only needs to visit both F_1 and F_2 infinitely often, but not necessarily simultaneously. Recall the example above, the run corresponding to the word a^ω , $(s_1 t_1)^\omega$ in B_1 and $(s_2 t_2)^\omega$ in B_2 , visits the marked state s_1 and t_1 infinitely often, respectively. However, they never visit the marked states t_1 and s_2 simultaneously.

To find the intersection of Büchi automata $B_1 = (Q_1, Q_1^0, \Sigma, \delta_1, F_1)$ and $B_2 = (Q_2, Q_2^0, \Sigma, \delta_2, F_2)$, we construct $B = (Q_1 \times Q_2 \times \{1, 2\}, Q_1^0 \times Q_2^0 \times \{1\}, \Sigma, \delta, F_1 \times Q_2 \times \{1\})$. We have $[q'_1, q'_2, j] \in \delta([q_1, q_2, i], \sigma)$ if $q'_1 \in \delta_1(q_1, \sigma)$ and $q'_2 \in \delta_2(q_2, \sigma)$, where

$$j = \begin{cases} 1 & \text{if } i = 2 \text{ and } q_2 \in F_2, \\ 2 & \text{if } i = 1 \text{ and } q_1 \in F_1, \\ i & \text{otherwise.} \end{cases}$$

For the above example, we obtain the intersection as



for which it can be verified that

$$\mathcal{L}_\omega(B) = \mathcal{L}_\omega(B_1) \cap \mathcal{L}_\omega(B_2) = a^\omega.$$

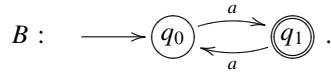
Therefore, the third component is responsible for guaranteeing that accepting states from both B_1 and B_2 appear infinitely often. The third component is initially 0. It changes from 0 to 1 when an accepting state of B_1 is seen. It then changes to 2 from 1 when the accepting state of B_2 is visited, and, in the next state, it returns back to 0. It can be seen that B_3 accepts exactly infinitely many states from F_1 and infinitely many states from F_2 occur.

A simpler intersection is obtained when all of the states of one of the automata are accepting. Assume that all the state of B_1 is accepting while the accepting state set of B_2 is F_2 . The intersection of B_1 and B_2 can be obtained as $B_3 = (Q_1 \times Q_2, Q_1^0 \times Q_2^0, \Sigma, \delta, Q_1 \times F_2)$. Moreover $[q'_1, q'_2] \in \delta([q_1, q_2], \sigma)$ if and only if $q'_1 \in \delta_1(q_1, \sigma)$ and $q'_2 \in \delta_2(q_2, \sigma)$.

The above two propositions show that Büchi automata are closed under both union and intersection, although the construction for the intersection is more complicated than the finite automata case. In addition, it can be shown that Büchi automata are also closed under complementation.

Proposition B.3 *Let B be a Büchi automaton over an alphabet Σ . Then there exists a (possibly non-deterministic) Büchi automaton $\bar{B}$ such that $\mathcal{L}_\omega(\bar{B}) = \Sigma^\omega - \mathcal{L}_\omega(B)$.*

However, the construction of $\bar{B}$ is much more involved than simply taking the complement of F . A word that visits $\Sigma - F$ infinitely often may also visit F infinitely as well. For example, consider a Büchi automaton

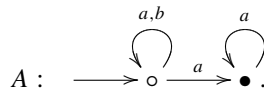


We have that $\mathcal{L}_\omega(B) = \mathcal{L}_\omega(\bar{B}) = \{a^\omega\}$.

Note that the complementation of a deterministic Büchi automaton could become non-deterministic, and non-deterministic Büchi automata are strictly more expressive than deterministic Büchi automata. For example, there is no deterministic Büchi automaton that accepts the language $(a + b)^*a^\omega$ (Any word that has an infinite suffix consisting of only a 's).

To see why, let's show by contradiction. Assume that there exists a deterministic Büchi automaton B that accepts $L = (a + b)^*a^\omega$. First, consider a string $w_0 = a^\omega$. Since $w_0 = a^\omega \in L$, so w_0 is accepted by B , and w_0 drives the states visiting F infinitely often. Hence, there exists a finite prefix of w_0 , denoted as u_0 , that drives the initial state into F . Now, consider another string $w_1 = u_0ba^\omega$. Again, $w_1 \in L$ and is accepted by B , so we can find a finite sub-string after u_0b that drives the state in B back into F again. Denote such a sub-string by u_1 . Similarly, we can find finite sub-strings u_i such that $u_0bu_1bu_2 \cdots bu_i$ drives the initial state of B into F . This process can continue for any large integer i . However, on the other hand, B only has finite states, so is F . Hence, there must exist j with $j \neq i$ and $j > i$ such that the strings $u_0bu_1bu_2b \cdots u_i$ and $u_0bu_1bu_2b \cdots u_ib \cdots u_j$ both drive the initial state to the same state in F , i.e., there exists a loop. Hence, B also accepts the ω -word $s = u_0bu_1bu_2b \cdots u_i(b \cdots u_j)^\omega$. However, $s \notin L$ as b appears in s infinitely often. This is a contradiction.

On the other hand, this language is recognized by a two-state non-deterministic Büchi automaton, e.g.,



Therefore, it is essential to use non-deterministic Büchi automata in the above proposition.

B.3 Generalized Büchi Automaton

Here, we introduce a variant of non-deterministic Büchi automata, called generalized non-deterministic Büchi automata or GNBA in short. GNBA are equally expressive as NBA. The difference between an NBA and a GNBA is that the acceptance condition for a GNBA requires to visit several sets $F_1, \dots, F_k$ infinitely often. Formally, the

syntax of a GNBA is as for an NBA, except that the acceptance condition is a set F consisting of finitely many acceptance sets $F_1, \dots, F_k$ with $F_i \subseteq Q$. That is, if Q is the state space of the automaton, then the acceptance condition of a GNBA is an element F of 2^{2^Q} . Recall that for an NBA, it is an element $F \in 2^Q$. The accepted language of a GNBA $\mathcal{G}$ consists of all infinite words which have an infinite run in $\mathcal{G}$ that visits all sets $F_i \in F$ infinitely often. Thus, the acceptance criterion in a generalized Büchi automaton can be understood as the conjunction of a number of Büchi acceptance conditions.

Definition B.4 A generalized non-deterministic Büchi automaton (GNBA) is a tuple

$$\mathcal{G} = (Q, \Sigma, \delta, Q_0, \mathcal{F}),$$

where Q, Σ, δ, Q_0 are defined as for NBA and F is a (possibly empty) subset of 2^Q . The elements $F \in \mathcal{F}$ are called acceptance sets.

Definition B.5 Runs in a GNBA are defined as for a NBA. That is, a run in $\mathcal{G}$ for the infinite word $a_0a_1 \dots \in \Sigma^\omega$ is an infinite state sequence $q_0q_1q_2 \dots \in Q^\omega$ such that $q_0 \in Q_0$ and $q_{i+1} \in \delta(q_i, a_i)$ for all $i \geq 0$.

Definition B.6 An infinite run $q_0q_1q_2 \dots$ is called accepting if for any $F \in \mathcal{F}$, there exist infinitely many j such that $q_j \in F$.

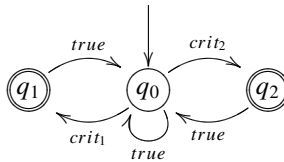
Definition B.7 The accepted language of $\mathcal{G}$ is

$$L_\omega(\mathcal{G}) = \{\sigma \in \Sigma^\omega \mid \text{there exists an accepting run for } \sigma \in \mathcal{G}\}.$$

Let's take a look at an example of GNBA.

Example B.3 The following shows a GNBA $\mathcal{G}$ over the alphabet 2^{AP} where $AP = \{crit_1, crit_2\}$ with the acceptance sets $F_1 = \{q_1\}$ and $F_2 = \{q_2\}$. That is, $\mathcal{F} = \{\{q_1\}, \{q_2\}\}$. The LTL formula we are interested in is

$$\varphi = (\Box \Diamond crit_1) \wedge (\Box \Diamond crit_2).$$



Thus, φ formalizes the property “both processes are infinitely often in their critical section”. Let us justify that indeed $L_\omega(\mathcal{G}) = \mathcal{W}(\varphi)$. This goes as follows.

“ $\supseteq$ ”: Each accepting run has to pass infinitely often through the edges (labeled with $crit_1$ or $crit_2$) leading to the states q_1 and q_2 . Thus, in every accepted word

$\sigma = A_0A_1A_2 \cdots \in \mathcal{L}_\omega(\mathcal{G})$, the atomic propositions $crit_1$ and $crit_2$ occur infinitely often as elements of the sets $A_i \in 2^{AP}$. Thus, $\sigma \in \mathcal{W}(\varphi)$.

“ $\subseteq$ ”: Let $\sigma = A_0A_1A_2 \cdots \in \mathcal{W}(\varphi)$. Since both propositions $crit_1$ and $crit_2$ occur infinitely often in the symbols A_i , the GNBA $\mathcal{G}$ can behave for the input word σ as follows. $\mathcal{G}$ remains in state q_0 until the first input symbol A_i with $crit_1 \in A_i$ appears. The automaton then moves to state q_1 . From there, $\mathcal{G}$ consumes the next input symbol A_{i+1} and returns to q_0 . It then waits in q_0 until a symbol A_j with $crit_2 \in A_j$ occurs, in which case the automaton moves to state q_2 for the symbol A_j and returns to q_0 on the next symbol A_{j+1} . Now the whole procedure restarts, i.e., $\mathcal{G}$ stays in q_0 while reading the symbols $A_{j+1}, \dots, A_{l-1}$ and moves to q_1 as soon as the current input symbol A_l contains $crit_1$, and so on. In this way, $\mathcal{G}$ generates an accepting run of the form

$$q_0^{k_1} q_1 q_0^{k_2} q_2 q_0^{k_3} q_3 q_0^{k_4} q_4 q_0^{k_5} q_5 \dots,$$

for the input word σ . These considerations show that $\mathcal{W}(\varphi) \subseteq \mathcal{L}_\omega(\mathcal{G})$. $\square$

Obviously, every NBA can be seen as a GNBA with exactly one acceptance set. Conversely, every GNBA can be transformed into an equivalent NBA.

Theorem B.3 *For each GNBA $\mathcal{G}$, there exists an NBA $\mathcal{A}$ with $\mathcal{L}_\omega(\mathcal{G}) = \mathcal{L}_\omega(\mathcal{A})$ and $|\mathcal{A}| = O(|\mathcal{G}| \cdot |\mathcal{F}|)$ where $\mathcal{F}$ denotes the set of acceptance sets in $\mathcal{G}$.*

Proof Let $\mathcal{G} = (Q, \Sigma, \delta, Q_0, \mathcal{F})$ be a GNBA. We may assume without loss of generality that $\mathcal{F} \neq \emptyset$. Let $\mathcal{F} = F_1, \dots, F_k$ where $k \geq 1$. The basic idea of the construction of $\mathcal{A}$ is to create k copies of $\mathcal{G}$ such that the acceptance set F_i of the i th copy is connected to the corresponding states of the $(i + 1)$ th copy. The accepting condition for $\mathcal{A}$ consists of the requirement that an accepting state of the first copy is visited infinitely often. This ensures that all other accepting sets F_i of the k copies are visited infinitely often too. Formally, let $\mathcal{A} = (Q', \Sigma', \delta', Q'_0, F')$ where

$$\begin{aligned} Q' &= Q \times \{1, \dots, k\}, \\ Q'_0 &= Q_0 \times \{1\} = \{\langle q_0, 1 \rangle \mid q_0 \in Q_0\}, \text{ and} \\ F' &= F_1 \times \{1\} = \{\langle q_F, 1 \rangle \mid q_F \in F_1\}. \end{aligned}$$

The transition function δ' is given by

$$\delta'(\langle q, i \rangle, A) = \begin{cases} \{\langle q', i \rangle \mid q' \in \delta(q, A)\} & \text{if } q \notin F_i, \\ \{\langle q', i + 1 \rangle \mid q' \in \delta(q, A)\} & \text{otherwise.} \end{cases}$$

We thereby identify $\langle q, k + 1 \rangle$ and $\langle q, 1 \rangle$. It is not difficult to check that $\mathcal{A}$ can be constructed in time and space $O(|\mathcal{G}| \cdot |\mathcal{F}|)$ where $|\mathcal{F}| = k$ is the number of acceptance sets in $\mathcal{G}$. The fact $\mathcal{L}_\omega(\mathcal{G}) = \mathcal{L}_\omega(\mathcal{A})$ can be seen as follows.

$\subseteq$: For a run of $\mathcal{A}$ to be accepting, it has to visit some state $\langle q, 1 \rangle$ infinitely often, where $q \in F_1$. As soon as a run reaches $\langle q, 1 \rangle$, the NBA $\mathcal{A}$ moves to the second copy. From the second copy, the next copy can be reached by visiting $\langle q', 2 \rangle$ with $q' \in F_2$.

NBA $\mathcal{A}$ can only return to $\langle q, 1 \rangle$ if it goes through all k copies. This is only possible if it reaches an accept state in each copy since that is the only opportunity to move to the next copy. So, for a run to visit $\langle q, 1 \rangle$ infinitely often, it has to visit some accept state in each copy infinitely often.

$\subseteq$: By similar reasoning, it can be deduced that every word in $\mathcal{L}_\omega(\mathcal{G})$ is also accepting in $\mathcal{A}$. $\square$

Hence, we know that GNBA has the same express power as NBA, and a given GNBA can be transformed into an equivalent NBA. However, it is easier to translate an LTL formula into a GNBA as shown in the next section.

B.4 Converting LTL to Büchi Automata

Given an LTL formula φ , a Büchi automaton $B_\varphi = (Q, Q^0, \Sigma, \delta, F)$ is to be built, such that $\mathcal{L}_\omega(B_\varphi)$ is exactly the set of paths satisfying the formula φ , that is, $\alpha \models \varphi$ if and only if $\alpha \in \mathcal{L}_\omega(B_\varphi)$. The good news is that it is always possible. Namely, for any LTL formula, there exists such an equivalent Büchi automaton (but the reverse is not true). The basic idea of the translation is to use the collections of all sub-formulas of φ as the state of the Büchi automaton, and the state should contain exactly those sub-formulas that hold true for all runs starting from this state. The obtained Büchi automaton could be very large in the sense that the size of its states could be exponentially large compared with the length of φ .

Next, we introduce the process of converting a given LTL formula to a Büchi automaton.

Definition B.8 Consider an LTL formula φ , the closure of φ , denoted as $\mathcal{C}(\varphi)$, is the collection of all sub-formula of φ and their negation.

For example, if $\varphi = a \sqcup (\neg a \wedge b)$, we have $\mathcal{C}(\varphi) = \{a, b, \neg a, \neg b, \neg a \wedge b, \neg(\neg a \wedge b), \varphi, \neg\varphi\}$. Then, for a formula φ , the Büchi automaton $B_\varphi = (Q, Q_0, \Sigma, \delta, F)$ can be constructed, where

- the states of B_φ denoted as q, q' are the maximal subset of $\mathcal{C}(\varphi)$ such that
 - for all $\psi \in \mathcal{C}(\varphi)$, either $\psi \in \mathcal{C}(\varphi)$ or $\neg\psi \in \mathcal{C}(\varphi)$ but not both;
 - $\psi_1 \cup \psi_2 \in q$ iff ψ_1 and $\psi_2 \in q$ whenever $\psi_1 \cup \psi_2 \in \mathcal{C}(\varphi)$;
 - $\psi_1 \cap \psi_2 \in q$ iff ψ_1 or $\psi_2 \in q$ whenever $\psi_1 \cap \psi_2 \in \mathcal{C}(\varphi)$;
 - if $\psi_1 \sqcup \psi_2 \in q$, then ψ_1 or $\psi_2 \in q$;
 - if $\neg(\psi_1 \sqcup \psi_2) \in q$, then $\neg\psi_2 \in q$;

the reason to choose Q in this way is that the sub-formulas in state q of B_φ hold true for all paths in B_φ starting from q , while all other paths not outside of q do not hold true;

- the initial states of B_φ are those containing φ ;
- $\Sigma = 2^P$;

- the transition $q' \in \delta(q, \sigma)$ is defined if $\sigma = q \cap P$ and
 - if $\circ\psi \in q$, then $\psi \in q'$;
 - if $\neg \circ\psi \in q$, then $\neg\psi \in q'$;
 - if $\psi_1 \sqcup \psi_2 \in q$ and $\psi_2 \notin q$, then $\psi_1 \sqcup \psi_2 \in q'$. This is due to the fact that $\psi_1 \sqcup \psi_2 = \psi_2 \vee (\psi_1 \wedge \circ(\psi_1 \sqcup \psi_2))$;
 - if $\neg(\psi_1 \sqcup \psi_2) \in q$ and $\psi_1 \in q$, then $\neg(\psi_1 \sqcup \psi_2) \in q'$. This is due to the fact that $\neg(\psi_1 \sqcup \psi_2) = \neg\psi_2 \wedge (\neg\psi_1 \vee \circ\neg(\psi_1 \sqcup \psi_2))$;
- $F = \{F_{\psi_1 \sqcup \psi_2} \mid \psi_1 \sqcup \psi_2 \in \mathcal{C}(\varphi)\}$, where $F_{\psi_1 \sqcup \psi_2} = \{q \in Q \mid \psi_1 \sqcup \psi_2 \notin q \text{ or } \psi_2 \in q\}$. This is due to the fact that $\neg(\neg(\psi_1 \sqcup \psi_2) \vee \psi_2) = \psi_1 \sqcup \psi_2 \wedge (\neg\psi_2)$.

B.5 Notes and Further Reading

Here, we gave a brief review of finite automata theory and proved the claims that are made with respect to finite automata and regular languages in Chap. 2. We further reviewed Büchi automata and introduced generalized Büchi automata. The translation from LTL to generalized Büchi automata, and then to Büchi automata was discussed, which is used in our LTL model checking in Chap. 3. This appendix can be used as supplementary materials to Chaps. 3 and 2. Our treatment of finite automata follows classical computer science textbooks on automata and can be found in, e.g., [10, 11]. Readers who would like to learn more on Büchi automata and the LTL translation may refer to [12] for further details.

References

1. Panos J Antsaklis. A brief introduction to the theory and applications of hybrid systems. In *Proc IEEE, Special Issue on Hybrid Systems: Theory and Applications*, 2000.
2. Panos J Antsaklis and Anthony N Michel. *Linear systems*. Springer, 2006.
3. A. V. Oppenheim, A. S. Willsky, and S. H. Nawab. *Signals and Systems*. Pearson, 1997.
4. Hassan K Khalil and Jessie W Grizzle. *Nonlinear systems*. Prentice hall Upper Saddle River, NJ, 2002.
5. Christos G Cassandras and Stephane Lafortune. *Introduction to discrete event systems*. Springer Science & Business Media, 2009.
6. Gene F Franklin, J David Powell, Michael L Workman, et al. *Digital control of dynamic systems*. Addison-wesley Reading, MA, 1998.
7. Graham C Goodwin, Stefan F Graebe, Mario E Salgado, et al. *Control system design*. Upper Saddle River, NJ: Prentice Hall, 2001.
8. C. G. Cassandras and S. Lafortune. *Introduction to Discrete Event Systems (2nd Edition)*. Springer-Verlag, 2008.
9. Jan Lunze and Françoise Lamnabhi-Lagarigue. *Handbook of hybrid systems control: theory, tools, applications*. Cambridge University Press, 2009.
10. J. E. Hopcroft, R. Motwani, and J. D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Prentice Hall, third edition, 2006.

11. Michael Sipser. *Introduction to the Theory of Computation*, volume 2. Thomson Course Technology Boston, 2006.
12. C. Baier and J. P. Katoen. *Principles of model checking*. MIT press Cambridge, 2008.

Index

A

- Abstraction
 - bisimulation quotient, 91
 - quotient transition system, 90
- Affine formula, 129
 - atomic linear predicate, 125
 - convex linear predicate, 125
 - convex polyhedron, 125
 - linear predicate, 125
 - polyhedron, 125
 - quantifier elimination, 130
- Affine function, 369
- Algorithm
 - bisimulation algorithm, 92
 - canonicalization algorithm, 121
 - reset algorithm, 122
- Approximate simulation, 381
 - interface, 381
 - simulation function, 381
- Automata
 - inclusion problem, 74, 76

B

- Bilinear matrix inequality, 214
- Bisimilar, 89
- Bisimulation relation, 88, 111
- Büchi automata, 75, 427
 - deterministic, 430
 - non-deterministic, 430
- Büchi automaton, 75
- Büchi timed automaton, 100

C

- Calculus of variations, 229
 - Euler equation, 231

- functional, 229
 - domain, 229
 - extremal, 230
 - increment, 229
 - range, 229
 - relative extremum, 230
 - relative minimum, 230
 - variant, 229
 - variation, 230
 - Weierstrass–Erdmann corner condition, 259
- Canonical Structure Theorem, 411
- Closed-loop control, 412
- Comparison principle, 163
- Constructibility, 409
- Controllability (-to-the-origin), 408
 - continuous-time system, 408
 - matrix, 408
- Controllable (-to-the-origin), 408

D

- Differential equation
 - Caratheodory solution, 50
 - Filippov solution, 51
- Differential game, 324
 - closed-loop strategy, 336
 - differential safety game, 345
 - largest controlled invariant subset, 349
 - maximum controlled invariant subset, 346
 - modified HJ equation, 347
 - value function, 346
- Hamilton–Jacobi equation, 337
- Hamilton–Jacobi–Isaac (HJI) equation, 339

- Nash equilibrium, 325
- open-loop strategy, 325
- pursuit–evasion game, 340
- two-player zero-sum game, 339
 - game value, 339
 - saddle point, 339
- Differential inequality, 163
- Dirac delta distribution, 405
- Discrete Event System (DES), 12
- Discrete games
 - Büchi game, 323
 - controllable predecessor, 318
 - controlled invariant subset, 345
 - game structure, 316
 - outcome, 317
 - run, 317
 - strategy, 317
 - Hamilton–Jacobi (HJ) equation, 343
 - Nash equilibrium, 336
 - reachability game, 321
 - safety game, 318
 - value function, 343
 - value iteration, 342
- Dual system, 410
- Dynamic programming
 - cost-to-go function, 330
 - Hamilton–Jacobi–Bellman equation, 330, 331
 - optimal cost-to-go, 330
 - principle of optimality, 328
 - value function, 330
- E**
- Example
 - bouncing ball, 25
 - bounding ball, 37
 - hysteresis, 42
 - integrator with upper saturation, 43
 - light switch, 22
 - saturation, 46
 - temperature control, 24, 36–38
 - train-gate-controller, 28
 - two-tank system, 32
 - vending machine, 14, 20
 - wheeled mobile robot, 39, 40, 42
- F**
- Feedback
 - state, 412
- Finite automaton, 12, 14
 - accepting run, 15
 - alphabet, 15
 - blocking, 14
 - closure, 16
 - complementation, 17
 - deterministic, 14, 19
 - determinization, 18
 - language acceptance, 15
 - language equivalence, 15, 19
 - non-blocking, 14
 - non-deterministic, 14
 - run, 15
 - subset construction, 18
 - total, 17
 - word, 15
- G**
- Generalized Büchi automata, 430
 - acceptance condition, 431
 - non-deterministic, 430
 - run, 431
- Gramian
 - observability, 410
 - reachability, 409
- H**
- Hamilton’s principle of least action, 237
- Hybrid automata, 22
 - composition, 27
 - deterministic, 35
 - equilibrium, 176
 - asymptotically stable, 176
 - exponentially stable, 176
 - stable, 176
 - unstable, 176
 - guard set, 23
 - hybrid execution, 31
 - finite, 34
 - infinite, 34
 - maximal, 34
 - prefix, 34
 - Zeno, 34, 51
 - hybrid time trajectory, 29
 - index set, 30
 - length, 30
 - prefix, 30
 - invariant set, 23
 - non-blocking, 34
 - reset map, 23
 - stability, 176
 - global, 177
 - synchronizing events, 27
- Hybrid game
 - hybrid game automata, 361

- controllable discrete predecessor, 362
 - controllable predecessor, 362
 - uncontrollable discrete predecessor, 362
- reach-while-avoid operator
 - value function, 364
- Hybrid optimal control, 250
 - autonomous switching, 251
 - basic hybrid minimum principle, 258
 - boundary conditions, 256, 258, 264, 273
 - co-ctate equations, 253
 - controlled switching, 251
 - corner conditions, 251
 - Hamilton's canonical equations, 258, 263, 273
 - Hamiltonian, 254
 - hybrid minimum principle, 263, 273
 - minimization condition, 255
 - model predictive control, 297
 - state equations, 254
 - state jumps, 272
 - switched systems, 278
 - cost functional, 279
 - embedding optimization, 289
 - optimal state trajectory, 279
 - switched linear quadratic regulation, 284
 - switching sequence, 278
 - two-stage optimization, 279
 - switching cost, 269
 - transversality conditions, 256, 258, 264, 273
- Hybrid system stability
 - arbitrary switching
 - discrete-time, 187
 - commutative systems, 182
 - constrained switching, 190
 - average dwell time, 190, 191
 - dwell time, 190
 - converse Lyapunov theorem, 180
 - Lie algebraic condition, 186
 - triangular systems, 185

I

- Information structure, 324

K

- Kalman's Decomposition Theorem, 411

L

- Linear hybrid automata, 125
 - clock translation, 127
 - linear phase-portrait approximation, 128
 - symbolic reachability analysis, 131
- Linear matrix inequalities, 205
- Linear Quadratic Gaussian (LQG) problem, 413
- Linear time-invariant system, 165
- LTL formula
 - closure, 433
 - subformula, 433
- Luenberger observer, 412
- Lyapunov function
 - class $\mathcal{K}$ function, 171
 - common Lyapunov function, 180
 - common quadratic Lyapunov function, 181
 - switched quadratic Lyapunov function, 187
 - multiple Lyapunov function, 196
 - piecewise quadratic Lyapunov function, 203
- Lyapunov stability
 - comparison functions, 171
 - converse Lyapunov theorem, 173
 - equilibrium, 158
 - asymptotically stable, 162
 - attractive, 161
 - exponentially stable, 163
 - global stability, 165
 - stable, 158
 - linear time-invariant systems
 - asymptotical stability, 168
 - asymptotic stability, 166
 - exponential stability, 169
 - Hurwitz matrix, 166, 169
 - Lyapunov equation, 166, 169, 170
 - Schur matrix, 170
 - Lyapunov function, 159
 - quadratic Lyapunov function, 166
 - radially unbounded, 165
- Lyapunov Stability Theorem, 158
- time-varying systems
 - asymptotic stability, 170
 - exponential stability, 173
 - stability, 170
 - uniform stability, 171
 - uniformly asymptotic stability, 171
- Lyapunov's Direct Method, 158
- Lyapunov's Second Method, 158

M

Matrix

- controllability, 408
- observability, 409

Mixed-logic dynamical systems, 298

- mixed-integer programming, 303

Model checking, 66

- CTL model checking, 82
- LTL model checking, 74

Multi-affine function, 376

Multi-rate timed automata, 114

N

Non-regular language, 21

O

Observability, 409

- continuous-time system, 409
- Gramian, 410
- matrix, 409

Observer, Luenberger, 412

 ω -regular expression, 75, 427 ω -regular language, 427

Open-loop control, 412

Optimal

- control problem, LQR, 412
- estimation problem, LQG, 413

Optimal control, 228, 329, 412

- co-state equation, 236
- cost functional, 234
- final cost, 228
- Hamiltonian function, 236
 - optimization condition, 236
- Lagrange multiplier, 234
- linear quadratic regulator, 242
 - Riccati equation, 245

Pontryagin's minimum principle, 246, 247

- boundary conditions, 247
- co-state equation, 247
- Hamiltonian, 247
- minimization condition, 247
- state equation, 247
- transversality condition, 248

running cost, 228

state equation, 236

terminal constraint, 228, 239

transversality condition, 237

P

Piecewise affine system, 45

- solution existence, 48
- solution uniqueness, 48
- trajectory, 48
- well-posedness, 48, 52

Polytope, 368

- bounding hyperplane, 368

R

Reachability

- continuous-time system, 408
- discrete-time system, 408
- Gramian, 409

Reachable, 408

Rectangle, 376

Rectangular automata, 115

Regular expression, 423

Regular language, 20, 21, 423

- choice, 20
- concatenation, 19
- Kleene closure, 20
- regular expression, 20

Response

- impulse response, 406
- total, 406

Riccati equation

- continuous-time case, 412
- discrete-time case, 413

S

Sampled-data system, 407

Sampling period, rate, 407

Self-bisimulation relation, 89

Separation principle, property, 413

Signal, 401

- continuous-time signals, 402
 - continuous-valued signals, 401
 - digital signals, 402
 - quantization, 402
 - discrete-time signals, 402
 - discrete-valued signals, 401, 402
- Signal temporal logic (STL), 383, 384
- robust semantics, 388
 - time bound, 386

Simplex, 371

Simulation relation, 87

Stability

- causal, 405

Standard form

- Kalman's canonical, 411
- uncontrollable system, 410

State feedback, 412

- eigenvalue assignment, 412
- optimal, 412
- State observer, 412
 - optimal, 413
- Switched system, 41
 - switching signal, 41
 - autonomous switching, 41
 - controlled switching, 41
 - state-dependent, 41
 - time-dependent, 41
- Switching stabilization, 207
 - piecewise quadratic stabilization, 212
 - quadratic stabilization, 207
- Symbolic control
 - linear control systems, 367
 - control to facet problem, 370
 - invariant control problem, 370
 - multi-affine control systems, 376
 - control to facet problem, 377
 - invariant control problems, 377
 - nonlinear control systems, 380
 - trajectory synthesis, 386
- Symbolic model checking, 135
- Symbolic reachability analysis, 129
- System, 402
 - causal, 405
 - continuous-time system, 406
 - discrete-time, 405
 - discrete-time system, 406
 - dual, 410
 - linear dynamical system, 404
 - sampled-data, 407
 - sampled-data system, 414
 - A/D converter, 415
 - D/A converter, 416
 - digital signal, 415
 - discretization, 415
 - inter-sample behavior, 421
 - quantization, 415
 - sampling, 415
 - sampling interval, 416
 - sampling period, 416
 - Zero-Order Hold (ZOH), 416
 - solution, 405
 - transfer function, 406, 407
- System representations, descriptions
 - equivalence, of, 407
 - external, 405
 - input–output, 405
 - standard form, uncontrollable, unobservable, 410
- T**
- Tarski's fixed-point theorem, 345
- Temporal logic
 - atomic proposition, 69
 - computation tree logic, 80
 - existential normal form, 83
 - semantics, 81
 - syntax, 80
 - CTL*, 86
 - linear temporal logic, 71
 - semantics, 74
 - syntax, 72
 - path quantifiers, 80
 - temporal operators, 71
 - timed computation tree logic, 101
- Temporal logic over reals, 366
- Timed automata, 96, 97
 - atomic clock constraints, 96
 - clock constraints, 96
 - atomic clock constraints, 101
 - clock region, 109
 - clock variables, 96
 - clock zone, 118
 - difference-bound matrices, 119
 - canonical form, 120
 - region equivalence, 106
 - region transition system, 105, 111
 - time-abstract transition system, 104, 105
 - zone automata, 118
- Timed game
 - controllable predecessor, 356, 358
 - outcome, 351
 - predecessor, 356
 - safe timed predecessor, 357
 - simulation graph, 352
 - timed game automata, 349
 - graph game, 354
 - quotient graph, 354
 - run, 351
 - symbolic state, 353
 - time-abstract bisimulation, 354
 - zone, 353
 - timed reachability game, 352
 - timed safety game, 351
 - timed strategy, 351
 - value function, 359
 - value iteration, 359
- Timed language, 99
- Timed ω -regular language, 100
- Timed word, 99
- Time sequence, 99
- Transition system, 67
 - labeled transition system, 69

- ω -trace, [69](#)
 - trace, [69](#)
- run, [67](#)
 - complete execution, [67](#)
 - deadlock, [67](#)

U

- Uncontrollable
 - eigenvalues, modes, [411](#)
- Unit impulse, [405](#)